

Tina4 for PHP Developers

One Framework, Native Conventions, Shared Contracts

v3.13.138 • tina4.com

The Intelligent Native Application 4ramework

Table of Contents

Getting Started with Tina4 PHP 71

1. What Is Tina4 PHP . . . 71

2. Prerequisites and Installation . . . 71

    What You Need . . . 71

    Creating a New Project . . . 72

    Starting the Dev Server . . . 73

3. Project Structure Walkthrough . . . 74

4. Your First Route . . . 74

    Test It . . . 75

    Understanding What Happened . . . 75

    Adding More HTTP Methods . . . 75

5. Your First Template . . . 77

    Create a Base Layout . . . 77

    Create a Product Listing Page . . . 77

    Create the Route That Renders the Template . . . 78

    See It in the Browser . . . 78

    How Template Rendering Works . . . 79

    About tina4css . . . 79

6. Understanding .env . . . 79

    Disable the browser auto-open . . . 80

7. The Dev Dashboard . . . 81

8. Manual Setup (No CLI) . . . 81

    Step 1: Install the Package . . . 81

    Step 2: Create index.php . . . 81

Step 3: Create the Folder Structure . . . 82
Step 4: Create .env . . . 82
Step 5: Run It . . . 82
9. Request & Response Fundamentals . . . 82
Reading Query Parameters . . . 83
Reading URL Path Parameters . . . 83
Reading the Request Body . . . 83
Reading Headers . . . 83
Sending JSON Responses . . . 83
Sending HTML / Template Responses . . . 84
Status Codes . . . 84
Worked Example: A Complete Route File . . . 84
10. Exercise: Greeting API + Product List Template . . . 86
Exercise Part A: Greeting API . . . 86
Exercise Part B: Product List Page . . . 87
11. Solutions . . . 87
Solution A: Greeting API . . . 87
Solution B: Product List Page . . . 88
12. Gotchas . . . 89
1. File not auto-discovered . . . 89
2. "Class not found" errors . . . 90
3. JSON response shows HTML . . . 90
4. Template not found . . . 90
5. Port already in use . . . 90
6. Changes not reflected . . . 90
7. .env not loaded . . . 90
8. Debug mode in production . . . 91

Routing 92

1. How Routing Works in Tina4 . . . 92
2. HTTP Methods . . . 92
    HEAD and OPTIONS - automatic, no boilerplate . . . 93
    Explicit Router::head() and Router::options() registration . . . 94
3. Path Parameters . . . 94
    Typed Parameters . . . 95
    Typed Parameters in Action . . . 96
4. Query Parameters . . . 97
5. Route Groups . . . 97
6. Middleware . . . 98
    Middleware on a Single Route . . . 98
    Blocking Middleware . . . 99
    Middleware on a Group . . . 99
    Multiple Middleware . . . 100
7. Route Decorators: @noauth and @secured . . . 100
    @noauth -- Public Routes . . . 100
    @secured -- Protected GET Routes . . . 100
8. Route Chaining: secure() and cache() . . . 101
    secure() . . . 101
    cache() . . . 101
    Chaining Both . . . 101
9. Wildcard and Catch-All Routes . . . 102
    Wildcard Routes . . . 102
    Catch-All Route (Custom 404) . . . 102
10. Route Listing via CLI . . . 102
11. Organizing Route Files . . . 103

Pattern 1: One File Per Resource . . . 103
Pattern 2: Subdirectories by Feature . . . 104
12. Exercise: Build a Full CRUD API for Products . . . 104
Requirements . . . 104
13. Solution . . . 105
14. Gotchas . . . 107
1. Trailing Slashes Are Normalised . . . 107
2. Parameter Names Must Be Unique in a Path . . . 107
3. Method Conflicts . . . 108
4. Route Handler Must Return a Response . . . 108
5. Route Files Must Start with <?php . . . 108
6. Middleware Uses Class-Based Pattern . . . 108
7. Group Prefix Normalisation . . . 108
Request & Response **109**
1. The Two Objects You Always Get . . . 109
2. The Request Object . . . 109
method . . . 109
path . . . 109
params . . . 109
query . . . 109
body . . . 110
headers . . . 110
ip . . . 110
cookies . . . 110
files . . . 110
Inspecting the Full Request . . . 110
3. The Response Object . . . 111

json() -- JSON Response . . . 111
html() -- Raw HTML Response . . . 111
text() -- Plain Text Response . . . 112
render() -- Template Response . . . 112
redirect() -- Redirect Response . . . 112
file() -- File Download Response . . . 112
4. Status Codes . . . 112
HTTP Status Constants . . . 113
Content-Type Constants . . . 114
5. Custom Headers . . . 115
CORS Headers . . . 115
6. Cookies . . . 115
7. File Uploads . . . 116
Handling a Single File Upload . . . 117
Saving the Uploaded File . . . 117
Handling Multiple Files . . . 118
8. File Downloads . . . 119
9. Content Negotiation . . . 119
10. Input Validation . . . 120
The Validator Class . . . 120
The Error Response Envelope . . . 120
Upload Size Limits . . . 121
11. Exercise: Build an Image Upload API . . . 121
Requirements . . . 121
Test with: . . . 122
12. Solution . . . 122
13. Gotchas . . . 124

1. Forgetting return . . . 124
2. Body Is Null for JSON Requests . . . 124
3. Content-Type Mismatch . . . 124
4. File Uploads Return Empty . . . 124
5. Redirect Loops . . . 124
6. Cookie Not Set . . . 125
7. Large Request Body Rejected . . . 125

Templates **126**

1. Why Templates . . . 126
2. Variables and Expressions . . . 126
Accessing Nested Data . . . 126
Method Calls on Values . . . 127
Expressions . . . 127
3. Template Inheritance . . . 127
Base Layout . . . 127
Child Template . . . 128
Using {{ parent() }} . . . 128
4. Includes . . . 129
Passing Variables to Includes . . . 129
5. For Loops . . . 130
The loop Variable . . . 130
Empty Lists . . . 130
Looping Over Key-Value Pairs . . . 131
6. Conditionals . . . 131
if / elseif / else . . . 131
Ternary Operator . . . 131
Testing for Existence . . . 131

Truthiness . . . 131
{% set %} -- Local Variables . . . 131
7. Filters . . . 132
Complete Filter Reference . . . 132
String Filters . . . 132
Array Filters . . . 133
Encoding Filters . . . 134
Numeric Filters . . . 134
JSON Filters . . . 134
Dict Filters . . . 135
Other Filters . . . 135
Chaining Filters . . . 135
Dumping Values for Debugging . . . 135
8. Macros . . . 136
Defining a Macro . . . 136
Using Macros . . . 137
9. Special Tags . . . 137
{% raw %} -- Literal Output . . . 137
{% spaceless %} -- Remove Whitespace . . . 138
{% autoescape %} -- Control Escaping . . . 138
Comments . . . 138
10. tina4css Integration . . . 138
Layout Classes . . . 138
Common Components . . . 139
Utility Classes . . . 139
11. Exercise: Build a Product Catalog Page . . . 139
Requirements . . . 139

Data . . . 140
Test with: . . . 140
12. Solution . . . 140
13. Gotchas . . . 143
1. {% extends %} Must Be the First Tag . . . 143
2. Undefined Variables Show Nothing . . . 143
3. Auto-Escaping Prevents HTML Output . . . 143
4. Variable Scope in Includes . . . 143
5. Macro Arguments Are Positional . . . 144
6. Template File Extension Does Not Matter . . . 144
7. Filters Are Not PHP Functions . . . 144
Database **145**
1. From Arrays to Real Data . . . 145
2. Connecting to a Database . . . 145
The Default: SQLite . . . 145
Connection Strings for Other Databases . . . 145
Firebird URL Forms . . . 145
Firebird: Dual-Driver Support . . . 146
Separate Credentials . . . 146
Connection Pooling . . . 146
Verifying the Connection . . . 147
3. Getting the Database Object . . . 147
4. Raw Queries . . . 147
fetch() -- Get Multiple Rows . . . 147
DatabaseResult . . . 148
Properties . . . 148
Iteration . . . 148

Index Access . . . 148
Countable . . . 148
Conversion Methods . . . 148
Schema Metadata with columnInfo() . . . 148
fetchOne() -- Get a Single Row . . . 149
execute() -- Run a Statement . . . 149
Full Example: A Simple Query Route . . . 149
5. Parameterised Queries . . . 150
A Safe Search Endpoint . . . 150
6. Transactions . . . 151
7. Schema Inspection . . . 152
getTables() . . . 152
getColumns() . . . 152
tableExists() . . . 152
getDatabaseType() . . . 152
A Schema Info Endpoint . . . 152
8. Batch Operations with executeMany() . . . 153
9. Helper Methods: insert(), update(), delete() . . . 153
insert() . . . 154
update() . . . 154
delete() . . . 154
10. Migrations . . . 154
File Naming . . . 154
Generating a Migration . . . 154
Writing the Migration . . . 155
Running Migrations . . . 155
Checking Migration Status . . . 155

Rolling Back . . . 155
Tracking Table . . . 156
Advanced SQL: Stored Procedures and Block Comments . . . 156
A Real Migration Sequence . . . 157
11. Query Caching . . . 157
12. Exercise: Build a Notes App . . . 157
Requirements . . . 157
Test with: . . . 158
13. Solution . . . 159
Migration . . . 159
Routes . . . 159
14. Seeder -- Generating Test Data . . . 162
FakeData . . . 163
Deterministic Output . . . 163
Seeding a Table . . . 163
Overrides . . . 163
Seeding ORM Models . . . 164
When to Use It . . . 164
15. Gotchas . . . 164
1. Forgetting commit() . . . 164
2. Connection String Formats . . . 164
3. SQLite File Paths . . . 165
4. Parameterised Queries with LIKE . . . 165
5. Boolean Values in SQLite . . . 165
6. Migration Already Applied . . . 165
7. Down Migration Not Found During Rollback . . . 165
8. fetch() Returns Empty Array, Not Null . . . 166

ORM **167**

1. From SQL to Objects . . . 167

ORM at a Glance: Four Languages, One Shape . . . 167

Defining a Model . . . 167

Common Query Operations . . . 168

2. Defining a Model . . . 169

Property Types . . . 170

Field Mapping . . . 170

autoMap and Case Conversion . . . 170

3. createTable -- Schema from Models . . . 171

4. CRUD Operations . . . 171

save -- Create or Update . . . 171

create -- Build and Save in One Step . . . 172

findById -- Fetch One Record by Primary Key . . . 172

find -- Query by Filter Array . . . 172

find() vs where() -- naming convention . . . 172

where -- Query with SQL Conditions . . . 173

delete -- Remove a Record . . . 173

Listing Records . . . 173

ModelCollection -- The Page and the Total . . . 174

selectOne -- Fetch a Single Record by SQL . . . 174

load -- Populate an Existing Instance . . . 175

count -- Count Records . . . 175

5. toDict, toJson, and Other Serialisation . . . 175

toDict . . . 175

toJson . . . 175

toArray vs toDict . . . 175

toObject() -- Returns a stdClass . . . 176
All Serialisation Methods . . . 176
6. Relationships . . . 176
$foreignKeys - Auto-Wired Relationships . . . 176
hasMany . . . 177
hasOne . . . 178
belongsTo . . . 178
Declarative Relationships . . . 179
7. Eager Loading . . . 180
Nested Eager Loading . . . 180
toDict with Nested Includes . . . 180
8. Soft Delete . . . 181
Deleting and Restoring . . . 181
Including Soft-Deleted Records . . . 181
Counting with Soft Delete . . . 182
When to Use Soft Delete . . . 182
9. Auto-CRUD . . . 182
The autoCrud Flag . . . 182
Manual Registration . . . 182
Auto-Discovering Models . . . 183
What the Generated Routes Do . . . 183
Sorting and Filtering . . . 184
Custom Routes Alongside Auto-CRUD . . . 184
Introspection . . . 184
10. Database Connection . . . 184
Binding the Database . . . 184
Named (secondary) connections . . . 185

11. Raw SQL and DatabaseResult . . . 185
12. Scopes . . . 186
13. Input Validation . . . 186
14. QueryBuilder Integration . . . 188
15. Exercise: Build a Blog with Relationships . . . 188
Requirements . . . 188
16. Solution . . . 189
17. Gotchas . . . 193
1. Forgetting to call save() . . . 193
2. findById() returns null . . . 193
3. find() vs findById() . . . 193
4. Static vs instance methods . . . 193
5. toDict() includes everything . . . 193
6. save() returns false on invalid data . . . 194
7. Soft delete column is isdeleted, not deletedat . . . 194
8. N+1 query problem . . . 194
9. Auto-CRUD endpoint conflicts . . . 194
10. Soft-deleted records appearing in queries . . . 194
11. $db->fetch() returns DatabaseResult, not an array . . . 195
QueryBuilder **196**
1. Why a Query Builder? . . . 196
2. The Factory: QueryBuilder::fromTable() . . . 196
3. Selecting Columns: select() . . . 196
4. Filtering: where() . . . 197
Multiple where() Calls . . . 197
Conditions Without Parameters . . . 197
5. OR Conditions: orWhere() . . . 197

6. Joins: join() and leftJoin() . . . 198
Inner Join . . . 198
Left Join . . . 198
Multiple Joins . . . 198
7. Grouping: groupBy() . . . 198
8. Filtering Groups: having() . . . 199
9. Sorting: orderBy() . . . 199
10. Pagination: limit() . . . 199
11. Inspecting the SQL: toSql() . . . 200
12. Execution Methods . . . 200
get() -- Fetch All Matching Rows . . . 200
first() -- Fetch a Single Row . . . 201
count() -- Count Matching Rows . . . 201
exists() -- Check for Any Match . . . 201
13. Chaining It All Together . . . 201
Using QueryBuilder in Routes . . . 202
Dynamic Filters . . . 202
14. ORM Integration . . . 203
15. NoSQL: MongoDB Queries . . . 204
Operator Mapping . . . 204
Example . . . 204
16. Gotchas . . . 205
No Database Adapter . . . 205
Parameter Order Matters . . . 205
Default Limit . . . 205
toSql() Does Not Include LIMIT . . . 205
count() Temporarily Replaces Columns . . . 205

OR Precedence . . . 206
16. Exercise: Product Search API . . . 206
Setup . . . 206
The Route . . . 206
Test It . . . 208
Verify the SQL . . . 208
Summary . . . 209
Authentication **210**
1. Locking the Door . . . 210
2. JWT Tokens . . . 210
Generating a Token . . . 210
Token Expiry . . . 210
Validating a Token . . . 211
Reading the Payload . . . 211
The Secret Key and Algorithm . . . 211
3. Password Hashing . . . 211
Hashing a Password . . . 212
Checking a Password . . . 212
Registration Example . . . 212
4. The Login Flow . . . 213
5. Using Tokens in Requests . . . 214
6. Protecting Routes . . . 214
Auth Middleware . . . 214
Applying Middleware to Routes . . . 215
Applying Middleware to a Group . . . 215
7. @noauth and @secured Decorators . . . 215

@noauth -- Skip Authentication . . . 215
@secured -- Require Authentication for GET Routes . . . 216
Role-Based Authorization . . . 216
8. CSRF Protection . . . 217
Generating a Token . . . 217
Validating the Token . . . 217
When to Use CSRF Tokens . . . 217
9. Sessions . . . 218
Session Configuration . . . 218
Using Sessions . . . 218
Session Options . . . 219
10. Exercise: Build Login, Register, and Profile . . . 219
Requirements . . . 219
Test with: . . . 220
11. Solution . . . 220
Migration . . . 220
Middleware . . . 220
Routes . . . 221
12. Gotchas . . . 224
1. Token Expiry Confusion . . . 224
2. Secret Key Management . . . 224
3. CORS with Authentication . . . 224
4. Storing Tokens in localStorage . . . 224
5. Forgetting @noauth on Login . . . 224
6. Password Hash Column Too Short . . . 225
7. Token in URL Query Parameters . . . 225

1. State in a Stateless World . . . 226
2. How Sessions Work . . . 226
3. The Session API . . . 226
Full API Reference . . . 227
4. File Sessions (Default) . . . 227
5. Redis Sessions . . . 228
Why Redis . . . 228
Redis with a Prefix . . . 228
6. MongoDB Sessions . . . 228
7. Valkey Sessions . . . 229
8. Database Sessions . . . 229
9. Reading and Writing Session Data . . . 229
Storing Complex Data . . . 230
10. Flash Messages . . . 230
Setting a Flash Message . . . 231
Reading and Clearing . . . 231
In Templates . . . 231
11. Setting and Reading Cookies . . . 231
Setting a Cookie . . . 232
Reading a Cookie . . . 232
Deleting a Cookie . . . 232
When to Use Cookies vs Sessions . . . 232
12. Remember Me Functionality . . . 233
13. Session Security . . . 234
Configuration Options . . . 234
httpOnly . . . 234
secure . . . 235

sameSite . . . 235
Session Regeneration . . . 235
Destroy a Session . . . 235
14. Exercise: Build a Shopping Cart with Session Storage . . . 235
Requirements . . . 236
Business Rules . . . 236
Test with: . . . 237
15. Solution . . . 237
16. Gotchas . . . 240
1. Sessions Do Not Work with curl Without Cookie Flags . . . 240
2. Session Data Disappears After Server Restart . . . 240
3. Session Cookie Not Sent in Production . . . 240
4. Flash Messages Show Twice . . . 240
5. Large Session Data Causes Slow Requests . . . 240
6. Remember Me Token Not Invalidated on Password Change . . . 240
7. Session Fixation . . . 241
Middleware & Security **242**
1. The Gatekeepers . . . 242
2. What Middleware Is . . . 242
3. Built-in CorsMiddleware . . . 243
Wildcard Origins . . . 244
CORS Without Middleware . . . 244
4. Built-in RateLimiter . . . 244
Built-in RequestLogger . . . 244
Built-in SecurityHeadersMiddleware . . . 245
Combining All Four Built-In Middleware . . . 246
5. Writing Custom Middleware . . . 246

Request Logging Middleware . . . 246
IP Whitelist Middleware . . . 247
Request Validation Middleware . . . 247
Continuation Middleware (the $next callable) . . . 247
Russian-doll order . . . 248
Short-circuiting without calling $next . . . 248
When to pick which . . . 249
Writing Class-Based Middleware . . . 249
JWT Authentication Middleware (Class-Based) . . . 250
6. Applying Middleware to Individual Routes . . . 250
->middleware() is purely additive . . . 251
7. Route Groups with Shared Middleware . . . 251
8. Middleware Execution Order . . . 252
Group + Route Middleware Order . . . 253
9. Short-Circuiting . . . 253
Authentication Short-Circuit . . . 254
Blocking with false . . . 254
Maintenance Mode . . . 254
Read-Only Mode . . . 255
10. Modifying Requests in Middleware . . . 255
Adding Request Metadata . . . 255
11. Real-World Middleware Stack . . . 256
12. Exercise: Build an API Key Middleware . . . 257
Setup . . . 257
Test with: . . . 257
13. Solution . . . 258
14. Gotchas . . . 259

1. Route Middleware Must Be Callable . . . 259
2. Returning the Wrong Thing from Middleware . . . 259
3. Middleware Order Matters . . . 259
4. CORS Preflight Returns 404 . . . 259
5. Rate Limiter Counts Preflight Requests . . . 259
6. Middleware File Not Auto-Loaded . . . 260
7. Group Middleware Expects an Array . . . 260
8. Mixing Up the Two Middleware Styles . . . 260
15. Security . . . 260
1. Secure-by-Default Routing . . . 260
Making a Write Route Public . . . 261
Protecting a GET Route . . . 261
The Rule . . . 262
2. CSRF Protection . . . 262
How It Works . . . 262
The Template . . . 262
The Middleware . . . 262
AJAX Requests . . . 262
Tokens in Query Strings - Blocked . . . 263
Skipping CSRF Validation . . . 263
Disabling CSRF Globally . . . 263
3. Session-Bound Tokens . . . 263
How Stolen Tokens Fail . . . 263
4. Security Headers . . . 264
HSTS - Enforcing HTTPS . . . 264
Customising Headers . . . 264
5. SameSite Cookies . . . 265

6. Login Flow - Complete Example . . . 265
The Login Route . . . 266
The Login Form . . . 267
Protected Pages - Checking the Session . . . 267
Logout - Destroying the Session . . . 268
7. Handling Expired Sessions . . . 268
The Pattern: Redirect to Login, Then Back . . . 268
Token Refresh . . . 269
8. Rate Limiting . . . 269
9. CORS and Credentials . . . 270
10. Security Checklist . . . 270
Gotchas . . . 271
1. "My POST route returns 401 but I didn't add auth" . . . 271
2. "CSRF validation fails on AJAX requests" . . . 271
3. "I disabled CSRF but forms still fail" . . . 271
4. "My Content-Security-Policy blocks inline scripts" . . . 271
5. "User stays logged in after session expires" . . . 271
Exercise: Secure Contact Form . . . 271
Solution . . . 272

Caching **274**

1. From 800ms to 3ms . . . 274
2. Layer 1: Request-Scoped Query Cache (On by Default) . . . 274
3. Layer 2: Persistent Database Cache (Opt-In) . . . 275
When to Use the Persistent Cache . . . 275
When Not to Use It . . . 275
Bypassing the Cache for One Query . . . 276
Inspecting the Query Cache . . . 276

4. Layer 3: Response Caching with ResponseCache Middleware . . . 276
Two Ways to Attach It . . . 277
Cache Headers . . . 277
Caching with Query Parameters . . . 278
What Not to Cache . . . 278
Inspecting the Response Cache . . . 278
5. Cache Backends . . . 279
Memory (Default) . . . 279
File . . . 279
Redis (and Valkey, Memcached, MongoDB) . . . 280
6. Direct Key/Value Cache API . . . 280
cache_set . . . 280
cache_get . . . 281
cache_delete . . . 281
Real-World Pattern: Cache-Aside . . . 281
7. Cache Invalidation Strategies . . . 282
Strategy 1: Time-Based Expiry (TTL) . . . 282
Strategy 2: Event-Based Invalidation . . . 282
Strategy 3: Write-Through Cache . . . 283
8. TTL Management . . . 283
Dynamic TTL . . . 284
9. Cache Statistics . . . 284
10. Combining Cache Layers . . . 285
11. Exercise: Cache an Expensive Product Listing Endpoint . . . 286
Requirements . . . 286
Test with: . . . 287
12. Solution . . . 287

13. Gotchas . . . 290
1. Caching Authenticated Responses . . . 290
2. Cache Stampede . . . 290
3. Memory Cache Lost on Restart . . . 291
4. Stale Data After Database Update . . . 291
5. Cache Key Collisions . . . 291
6. Serialization Overhead . . . 291
7. Importing the Helpers from the Wrong Namespace . . . 291
Queue System **292**
1. Do Not Make the User Wait . . . 292
2. Why Queues Matter . . . 292
3. File Queue (Default) . . . 293
Creating a Queue and Pushing a Job . . . 293
Convenience Method: produce . . . 294
Push with Priority . . . 294
Push with Delay . . . 294
Queue Size . . . 294
4. Pushing from Route Handlers . . . 294
5. Consuming Jobs . . . 295
consume Is a Long-Running Poll . . . 295
Retry with a Manual Delay . . . 296
Manual Pop . . . 296
6. The Job Object . . . 296
Job Methods . . . 296
Job Properties . . . 297
7. Automatic Retry and Dead Letters . . . 297
How the Lifecycle Works . . . 297

maxRetries . . . 297
retryBackoff . . . 298
Inspecting Retrying and Dead Jobs . . . 298
Manually Reviving Jobs . . . 298
Purging Jobs . . . 298
8. Producing Multiple Jobs . . . 298
9. Switching Backends . . . 299
Default: File . . . 299
RabbitMQ . . . 299
Kafka . . . 300
MongoDB . . . 300
10. Exercise: Build an Email Queue . . . 300
Requirements . . . 300
Test with: . . . 300
11. Solution . . . 301
12. Gotchas . . . 302
1. consume runs forever by default . . . 302
2. Calling complete or fail on a pop() result . . . 303
3. Letting jobs fail silently . . . 303
4. Worker not picking up messages . . . 303
5. Payload must be JSON-serializable . . . 303
6. Dead letters pile up . . . 303
7. File backend for production . . . 303
Events **305**
1. Decouple Your Code . . . 305
2. Listening for Events . . . 305
3. Emitting Events . . . 305

4. One-Shot Listeners with once() . . . 306
5. Removing Listeners with off() . . . 306
6. Priority . . . 307
7. Events in Route Handlers . . . 307
8. Checking Registered Listeners . . . 309
9. Exercise: Order Lifecycle Events . . . 309
Requirements . . . 309
Test with: . . . 309
10. Solution . . . 310
11. Gotchas . . . 311
1. Listeners fire synchronously . . . 311
2. Uncaught exceptions in listeners crash the emit . . . 311
3. Forgetting to reset in tests . . . 311
4. Using closures with off() . . . 311
Localization **313**
1. One App, Many Languages . . . 313
2. Locale Files . . . 313
3. Loading and Using I18n . . . 314
4. Fallback Locale . . . 314
5. Locale Switching in API Routes . . . 315
6. Interpolation . . . 316
7. Checking Available Locales . . . 317
8. Localized Validation Errors . . . 317
9. Adding Locales at Runtime . . . 318
10. Exercise: Multilingual API . . . 318
Requirements . . . 318

Test with: . . . 319

11. Gotchas . . . 319

1. Missing keys return the key name . . . 319

2. Locale files not loaded . . . 319

3. Placeholder not replaced . . . 319

4. Locale not resetting between requests . . . 319

Structured Logging **321**

1. Stop Using error_log . . . 321

2. Log Levels . . . 321

3. Basic Logging . . . 321

4. Log Level Filtering . . . 322

5. Logging with Context . . . 322

6. Logging in Route Handlers . . . 323

7. Logging Exceptions . . . 324

8. Request Correlation . . . 324

9. Performance Logging . . . 325

10. Environment Configuration . . . 326

11. Gotchas . . . 326

1. Logging sensitive data . . . 326

2. Logging too much in production . . . 326

3. Forgetting to log errors before rethrowing . . . 326

Email with Messenger **327**

1. Every App Sends Email . . . 327

2. Messenger Configuration via .env . . . 327

Staging delivery safety . . . 328

Common Provider Configurations . . . 328

3. Constructor Override Pattern . . . 329
4. Sending Plain Text Email . . . 329
The send() Method . . . 330
5. Sending HTML Email with Text Fallback . . . 330
6. Adding Attachments . . . 331
Inline Attachments with Custom Names . . . 331
7. CC and BCC . . . 332
8. Custom Headers . . . 332
9. Reading Inbox via IMAP . . . 333
Reading a Specific Email . . . 334
Searching Emails . . . 334
Marking Messages . . . 335
Deleting Messages . . . 335
Listing Folders . . . 335
Reading from a Specific Folder . . . 335
10. Dev Mailbox Capture . . . 335
Sending from a development environment . . . 336
11. Using Templates for Email Content . . . 336
Rendering and Sending . . . 337
12. Sending Email via Queues . . . 338
13. Exercise: Build a Contact Form with Email Notification . . . 339
Requirements . . . 339
Test with: . . . 339
14. Solution . . . 339
15. Gotchas . . . 343
1. Gmail Blocks "Less Secure" Apps . . . 343
2. Emails Go to Spam . . . 343

3. HTML Email Looks Broken . . . 343
4. Attachment File Not Found . . . 343
5. Dev Mode Intercepts Emails Silently . . . 343
6. Email Template Variables Not Substituted . . . 344
7. Connection Timeout on Send . . . 344

Frontend with tina4css **345**

1. The Problem with Frontend Toolchains . . . 345
2. What Ships with Tina4 . . . 345
3. The Grid System . . . 345
4. Components . . . 346
Navbar . . . 346
Buttons . . . 346
Cards . . . 347
Forms . . . 347
Tables . . . 348
Badges . . . 348
Alerts . . . 349
Modals . . . 349
Progress Bars . . . 349
5. SCSS Customization . . . 350
Variables . . . 350
Compiling SCSS . . . 350
Custom Theme Example . . . 351
6. frond.js -- The JavaScript Helper . . . 351
AJAX Requests . . . 352
Form Submission . . . 352
Token Management . . . 353

Loading Indicators . . . 353
7. Building a Dashboard Layout . . . 353
The Base Layout . . . 353
The Dashboard Page . . . 356
The Dashboard Route . . . 357
8. Dark Mode Support . . . 358
Toggle with JavaScript . . . 358
Respecting System Preference . . . 359
Custom Dark Mode Colors . . . 359
9. Responsive Design . . . 359
Responsive Utilities . . . 359
Responsive Tables . . . 359
Spacing Utilities . . . 360
10. Building a Users Page with AJAX . . . 360
11. Exercise: Build a Complete Admin Dashboard . . . 362
Requirements . . . 363
Seed Data . . . 363
Expected Result . . . 363
12. Solution . . . 364
13. Gotchas . . . 365
1. CSS File Not Loading . . . 365
2. SCSS Changes Not Reflected . . . 365
3. Modal Does Not Open . . . 365
4. frond.js AJAX Returns HTML Instead of JSON . . . 365
5. Dark Mode Flickers on Load . . . 365
6. Form Data Not Reaching the Server . . . 366
7. Responsive Grid Not Working . . . 366

14. HtmlElement - Programmatic HTML Builder . . . 366

Testing **367**

1. Why Tests Matter More Than You Think . . . 367

2. Your First Test . . . 367

How It Works . . . 368

3. Assertion Methods . . . 368

assertEqual($actual, $expected, $message) . . . 368

assertTrue($value, $message) . . . 368

assertFalse($value, $message) . . . 369

assertRaises($callable, $exceptionClass, $message) . . . 369

assertNotEqual($actual, $expected, $message) . . . 369

assertNull($value, $message) . . . 369

assertNotNull($value, $message) . . . 369

4. Testing ORM Models . . . 369

Test Database . . . 372

5. Testing Routes . . . 372

Test Client Methods . . . 373

Response Object . . . 374

6. Testing Authentication . . . 374

7. Setup and Teardown . . . 375

8. Running Tests . . . 376

Run All Tests . . . 376

Run a Specific Test File . . . 376

Run a Specific Test Method . . . 377

Verbose Output . . . 377

Failed Test Output . . . 377

9. PHPUnit Integration . . . 377
Code Coverage . . . 378
10. Testing Best Practices . . . 379
Test One Thing Per Test . . . 379
Use Descriptive Assertion Messages . . . 379
Isolate Tests . . . 379
11. Exercise: Test a User Model and Authentication Flow . . . 380
Requirements . . . 380
Expected output: . . . 380
12. Solution . . . 382
tests/UserModelTest.php . . . 382
tests/AuthFlowTest.php . . . 384
13. Gotchas . . . 386
1. Tests Run in Order . . . 386
2. Test Database vs Development Database . . . 386
3. Unique Constraint Failures . . . 386
4. Test Method Not Discovered . . . 386
5. Assertion Arguments Reversed . . . 386
6. Cannot Test Routes That Require Authentication Setup . . . 387
7. Tests Pass Locally but Fail in CI . . . 387
Scaffolding **388**
The CRUD Generator . . . 388
What Each File Contains . . . 388
Run It . . . 391
Individual Generators . . . 391
Model . . . 391
Route . . . 391

Migration . . . 391
Middleware . . . 391
Test . . . 392
Form . . . 392
View . . . 392
CRUD . . . 392
Auth . . . 392
AutoCRUD . . . 392
Usage . . . 392
The Auth Generator . . . 393
Field Types . . . 393
Table Naming Convention . . . 394
Combining Generators . . . 394
Exercise: Scaffold a Blog . . . 394
Gotchas . . . 395
Generators Do Not Overwrite . . . 395
Run Migrate After Generate . . . 395
File Naming Matters . . . 395
Field Changes Need New Migrations . . . 395
Singular Table Names . . . 395
API Documentation with Swagger 396
1. The 47-Endpoint Problem . . . 396
2. What Swagger/OpenAPI Is . . . 396
3. Enabling Swagger . . . 396
The Swagger JSON Endpoint . . . 396
4. Adding Descriptions to Routes . . . 397
Documenting Path Parameters . . . 398

Documenting Query Parameters . . . 398
5. Documenting Request and Response Schemas . . . 399
Request Body . . . 399
Multiple Response Codes . . . 400
6. Tags for Grouping Endpoints . . . 400
Multiple Tags . . . 401
7. Example Values . . . 401
8. Try-It-Out from the Swagger UI . . . 402
Authentication in Try-It-Out . . . 402
9. Customizing the Swagger Info Block . . . 402
10. Generating Client SDKs from the Spec . . . 403
Using OpenAPI Generator . . . 403
11. A Complete Documented API . . . 403
12. Exercise: Document a Complete User API . . . 405
Requirements . . . 405
Verify at: . . . 406
13. Solution . . . 406
14. Gotchas . . . 408
1. Annotations Must Be Directly Above the Route . . . 408
2. Missing @tags Makes Endpoints Hard to Find . . . 408
3. @body Must Be Valid JSON . . . 408
4. Swagger Shows Unannotated Routes . . . 408
5. Response Examples Do Not Match Reality . . . 408
6. Swagger UI Not Available in Production . . . 409
7. SDK Generation Produces Incorrect Types . . . 409
API Client 410

1. Calling External APIs Without Dependencies . . . 410
2. Creating an API Client . . . 410
3. sendRequest() . . . 410
4. GET Requests . . . 410
5. POST, PUT, PATCH, DELETE . . . 411
POST - Create a resource . . . 411
PUT - Full update . . . 411
PATCH - Partial update . . . 411
DELETE - Remove a resource . . . 411
6. Query Parameters . . . 412
7. Custom Headers . . . 412
8. Basic Authentication . . . 412
9. Bearer Token Auth . . . 413
10. Error Handling . . . 413
11. Calling External APIs from Routes . . . 414
12. Exercise: GitHub API Proxy . . . 415
Requirements . . . 415
Test with: . . . 415
13. Gotchas . . . 415
1. No timeout set . . . 415
2. Not checking status before accessing body . . . 416
3. Credentials in source code . . . 416
4. Not handling rate limits . . . 416
GraphQL **417**
1. The Problem GraphQL Solves . . . 417
2. GraphQL vs REST -- A Quick Comparison . . . 417

3. Enabling GraphQL . . . 418
4. Defining a Schema . . . 418
5. Writing Resolvers . . . 419
Testing the Queries . . . 419
6. Mutations . . . 420
Mutation Resolvers . . . 421
Testing Mutations . . . 422
7. Nested Types and Relationships . . . 423
8. Auto-Generating Schema from ORM Models . . . 425
9. The GraphiQL Playground . . . 426
10. Query Variables . . . 426
11. Exercise: Build a GraphQL API for a Blog . . . 427
Requirements . . . 427
Test with these queries: . . . 428
12. Solution . . . 428
Schema . . . 428
Resolvers . . . 429
13. Input Validation . . . 431
Example: Missing Required Argument . . . 431
14. Field-Level Auth Directives . . . 432
Passing Auth Context . . . 432
Schema Example . . . 433
Behavior on Failed Auth . . . 433
15. Gotchas . . . 433
1. Schema File Not Found . . . 433
2. Resolver Not Called . . . 433
3. Nested Resolver Returns Wrong Data . . . 434

4. Mutation Input Not Parsed . . . 434
5. N+1 Query Problem . . . 434
6. GraphQL Playground Returns 404 . . . 434
7. Type Mismatch Between Schema and Resolver . . . 435

Real-time with WebSocket **436**

1. The Refresh Button Problem . . . 436
2. What WebSocket Is . . . 436
3. Router::websocket() -- WebSocket as a Route . . . 436
Starting the Server . . . 437
4. Connection Events . . . 437
Open . . . 437
Message . . . 437
Close . . . 438
A Complete Handler . . . 438
5. Sending to a Single Client . . . 439
Closing a Connection . . . 439
6. Broadcasting to All Clients . . . 440
Broadcast Excluding Sender . . . 440
7. Path-Scoped Isolation . . . 441
8. Building a Live Chat . . . 442
WebSocket Handler . . . 442
9. Live Notifications . . . 443
10. Connecting from JavaScript . . . 444
Basic Connection . . . 445
Auto-Reconnect . . . 445
Using Native WebSocket . . . 446

11. A Complete Chat Page . . . 446
12. Exercise: Build a Real-Time Chat Room . . . 448
Requirements . . . 448
Test by: . . . 448
13. Solution . . . 449
14. Scaling with a Backplane . . . 451
Configuration . . . 451
Requirements . . . 451
When to Choose a Backplane . . . 452
15. Gotchas . . . 452
1. WebSocket Needs a Persistent Server . . . 452
2. Messages Are Strings, Not Objects . . . 453
3. Connection Count Is Per-Path . . . 453
4. Broadcasting Does Not Scale Across Servers . . . 453
5. Large Messages Cause Disconnects . . . 453
6. Memory Leak from Tracking Connected Users . . . 453
7. No Authentication on WebSocket Connect . . . 453
Server-Sent Events (SSE) 455
1. The Polling Problem . . . 455
2. What SSE Is . . . 455
3. Your First Stream . . . 456
4. The SSE Protocol . . . 457
The data: Field . . . 457
The event: Field . . . 457
The id: Field . . . 457
The retry: Field . . . 458
The Delimiter . . . 458

5. Frontend: EventSource . . . 458
Basic Usage . . . 458
Named Events . . . 459
Closing the Connection . . . 459
With Authentication . . . 459
6. Real Examples . . . 460
Live Dashboard . . . 460
Build Progress . . . 460
LLM Chat Streaming . . . 461
File Processing Progress . . . 462
7. Custom Content Types . . . 463
NDJSON Streaming . . . 463
Consuming NDJSON on the Client . . . 463
8. SSE vs WebSocket . . . 464
9. Production Considerations . . . 464
Nginx Proxy Buffering . . . 464
Connection Limits . . . 465
Heartbeat Messages . . . 465
10. Exercise: Build a Live Metrics Dashboard . . . 466
Requirements . . . 466
Test by: . . . 466
11. Solution . . . 466
12. What You Learned . . . 468
WSDL / SOAP **469**
1. When You Need SOAP . . . 469
2. Creating a SOAP Service . . . 469

3. Registering the Service . . . 470
4. Accessing the Auto-Generated WSDL . . . 470
5. Calling the Service with PHP SoapClient . . . 471
6. A Real-World Service: Currency Conversion . . . 472
7. Type Annotations . . . 473
8. SOAP Faults . . . 473
9. Testing with curl . . . 474
10. Gotchas . . . 474
1. Method not appearing in WSDL . . . 474
2. Wrong XSD types . . . 474
3. SOAP Fault not reaching the client . . . 475
4. WSDL URL is wrong in generated document . . . 475
DI Container **476**
1. The Problem with Global State . . . 476
2. Creating a Container . . . 476
3. register() - Transient Services . . . 476
4. singleton() - Shared Instances . . . 477
5. get() and has() . . . 477
6. Services Depending on Other Services . . . 478
7. Binding a Container to Route Handlers . . . 478
8. reset() - Testing . . . 479
9. A Full App Bootstrap . . . 480
10. Gotchas . . . 481
1. Circular dependencies . . . 481
2. Forgetting to use singleton for stateful services . . . 481
3. Not resetting between tests . . . 481

4. Storing request-scoped data in a singleton . . . 481

Service Runner **482**

1. Work That Runs Outside HTTP Requests . . . 482
2. The Service Interface . . . 482
3. Registering and Starting Services . . . 483
4. A Report Generator Service . . . 484
5. A Cache Warmer Service . . . 484
6. Graceful Shutdown . . . 485
7. Running Services as a Standalone Script . . . 486
8. Running with Docker . . . 487
9. Gotchas . . . 488
    1. Services blocking each other . . . 488
    2. Memory leak in long-running workers . . . 488
    3. Database connection dropped . . . 488
    4. Not logging worker errors . . . 488

MCP Dev Tools **489**

1. What is MCP? . . . 489
2. How It Works . . . 489
    Localhost-Only Security . . . 489
3. Connecting AI Tools . . . 489
    Claude Code . . . 489
    Cursor . . . 490
    Manual Connection . . . 490
4. Built-in Tools . . . 490
    Database . . . 490
    Routes . . . 491

Templates . . . 491
Files . . . 491
Migrations . . . 492
Data and ORM . . . 492
Infrastructure . . . 492
Debugging . . . 492
Project introspection . . . 493
Persistent code index . . . 493
Framework documentation (prose) . . . 493
Live API RAG (reflection) . . . 493
Plan management . . . 494
5. Protocol Details . . . 494
Streamable HTTP (current) . . . 495
Legacy HTTP+SSE (still supported) . . . 495
Messages . . . 495
Lifecycle . . . 495
6. Troubleshooting . . . 496
Building Custom MCP Servers **497**
1. Beyond Dev Tools . . . 497
2. Creating an MCP Server . . . 497
3. Registering Tools with #[McpTool] . . . 497
4. Registering Resources with #[McpResource] . . . 498
5. Class-Based MCP Services . . . 499
6. Securing MCP Endpoints . . . 500
7. Testing MCP Tools . . . 500
8. Complete Example: CRM MCP Server . . . 501
9. Best Practices . . . 502

Dev Tools 503

1. Debugging at 2am . . . 503
2. Enabling the Dev Dashboard . . . 503
3. Dashboard Overview . . . 503
    - System Overview . . . 503
4. The Dev Toolbar . . . 504
    - Disabling the Toolbar . . . 504
5. Error Overlay . . . 504
    - Example Error . . . 505
    - Error Overlay in Production . . . 505
6. Request Inspector . . . 505
    - Request Details Panel . . . 506
    - Filtering Requests . . . 506
    - Request Replay . . . 506
7. SQL Query Runner . . . 506
    - Example . . . 507
    - Safety . . . 507
8. Hot Reload . . . 507
    - How It Works . . . 507
    - Watched Directories and File Types . . . 508
    - What Happens on Reload . . . 508
    - Activation . . . 508
    - Disabling Hot Reload . . . 508
9. Gallery -- Interactive Examples . . . 508
    - Creating a Gallery Entry . . . 509
10. Queue Monitor . . . 509

11. System Info . . . 510
12. Exercise: Debug a Failing Route . . . 510
Setup . . . 510
Task . . . 511
Expected Steps . . . 511
13. Solution . . . 512
What You Practiced . . . 512
14. Gotchas . . . 512
1. Dev Dashboard Returns 404 . . . 512
3. Hot Reload Not Working . . . 513
4. Error Overlay Shows in Production . . . 513
5. Request Inspector Shows Too Many Requests . . . 513
6. SQL Runner Modifies Data Accidentally . . . 513
7. Debug Toolbar Breaks Layout . . . 513
Tina4 CLI **515**
1. Getting a New Developer Up to Speed . . . 515
2. tina4 init -- Project Scaffolding . . . 515
Language Detection . . . 515
Init into an Existing Directory . . . 516
3. tina4 serve -- Dev Server . . . 516
Options . . . 517
Serving a Project by Name . . . 517
Production Mode . . . 517
4. tina4 generate model -- ORM Scaffolding . . . 517
Adding Fields . . . 518
Field Types . . . 519
Options . . . 519

5. tina4 generate route -- CRUD Route Scaffolding . . . 519
Options . . . 521
6. tina4 generate migration -- Migration Scaffolding . . . 521
7. tina4 generate middleware -- Middleware Scaffolding . . . 522
8. tina4 doctor -- Environment Health Check . . . 522
9. tina4 test -- Running Tests . . . 523
Options . . . 523
10. tina4 routes -- List All Routes . . . 524
Filtering . . . 524
11. tina4 migrate -- Run Database Migrations . . . 524
Options . . . 524
Migration Status . . . 524
12. Exercise: Scaffold a Complete Feature in 5 Commands . . . 525
Requirements . . . 525
Expected Commands . . . 525
Test with: . . . 525
13. Solution . . . 525
14. Gotchas . . . 527
1. Model Name Must Be PascalCase . . . 527
2. Migration Already Applied . . . 527
3. Generated Routes Conflict with Auto-CRUD . . . 527
4. Port Already in Use . . . 527
5. CLI Not Found After Installation . . . 527
15. Documentation . . . 528
16. Test Port (Dual-Port Development) . . . 528
Vibe Coding with AI **529**
Why Tina4 is Built for AI-Assisted Development . . . 529

The Zero-Dependency Advantage . . . 529
CLAUDE.md -- The AI's Instruction Manual . . . 529
Skills -- Deep Framework Knowledge . . . 530
tina4-maintainer . . . 530
tina4-developer . . . 530
tina4-js . . . 530
The Convention Advantage . . . 530
Real-World Vibe Coding Session . . . 530
Supported AI Tools . . . 531
The AI Chat in Dev Dashboard . . . 532
Tips for Effective Vibe Coding with Tina4 . . . 532
Exercise . . . 533
Gotchas . . . 533
The Philosophy . . . 533
Environment Variables 534
Core Server . . . 535
Routing . . . 537
Secrets and Authentication . . . 538
Database . . . 539
CORS . . . 540
Security Headers . . . 541
Rate Limiting . . . 541
Sessions . . . 542
Redis/Valkey session backend . . . 543
MongoDB session backend . . . 543
Templates . . . 544

Cache . . . 545
Queues . . . 546
Kafka queue backend . . . 546
RabbitMQ queue backend . . . 546
MongoDB queue backend . . . 547
WebSocket Backplane . . . 547
Email . . . 548
Logging . . . 549
Log pipeline (sink, format, rotation) . . . 550
Uploads . . . 551
Localisation . . . 551
Services (background tasks) . . . 551
Application AI Client and Developer AI Tools . . . 551
Swagger / OpenAPI . . . 553
GraphQL . . . 554
HTTP Status Constants . . . 554
Log-Level Constants . . . 554
Configuration Recipes . . . 555
PostgreSQL in production . . . 555
A shared cache on Redis . . . 555
Sessions that survive more than one box . . . 555
A queue on RabbitMQ or Kafka . . . 555
WebSocket broadcasts across instances . . . 556
Locked-down production headers . . . 556
The dev dashboard AI, kept local . . . 556
Minimal .env for Development . . . 556
Minimal .env for Production . . . 557

Feature Module Settings . . . 557

Deployment **559**

1. From Development to Production . . . 559

2. Production .env Configuration . . . 559

Key Differences from Development . . . 560

Sensitive Values . . . 560

3. FrankenPHP Auto-Detection . . . 560

How Auto-Detection Works . . . 560

Installing FrankenPHP . . . 561

4. Docker Deployment . . . 561

Choosing a Runtime . . . 561

Swoole: What Changes About Your Code . . . 562

Official Base Image . . . 562

Dockerfile . . . 563

.dockerignore . . . 563

Build and Run . . . 563

Verify . . . 563

Adding Database Drivers . . . 564

Docker Compose . . . 565

5. PHP-FPM + nginx Configuration . . . 566

PHP-FPM Pool Configuration . . . 566

nginx Configuration . . . 566

6. Health Check Endpoint . . . 567

Custom Health Checks . . . 568

7. Graceful Shutdown . . . 568

Shutdown Timeout . . . 569

Docker Stop Behavior . . . 569

8. Log Rotation . . . 569
Using logrotate (Linux) . . . 569
Docker Logging . . . 569
9. Environment-Specific Configuration . . . 570
Loading a Specific .env File . . . 570
10. SSL/TLS with Let's Encrypt . . . 570
With FrankenPHP (Automatic) . . . 570
With nginx (Manual) . . . 570
With Docker (Using a Reverse Proxy) . . . 571
11. Scaling . . . 571
Multiple Workers . . . 571
Load Balancing with nginx . . . 571
Docker Scaling . . . 572
12. Monitoring . . . 572
Log Aggregation . . . 572
Uptime Monitoring . . . 573
Application Performance Monitoring (APM) . . . 573
13. Exercise: Deploy a Tina4 PHP App with Docker . . . 573
Requirements . . . 573
Test with: . . . 574
14. Solution . . . 574
Dockerfile . . . 574
docker-compose.yml . . . 575
.env.production . . . 575
HTTP Caching, Compression, and ETags . . . 576
15. Gotchas . . . 576
1. SQLite Concurrency in Production . . . 576

2. Data Volume Not Persisted . . . 576
3. .env File Not Loaded in Docker . . . 576
4. Permission Denied on data/ Directory . . . 577
5. Health Check Fails During Startup . . . 577
6. CORS Errors in Production . . . 577
7. SSL Certificate Not Renewing . . . 577

Building a Complete App **578**

1. Putting It All Together . . . 578
2. Planning the App . . . 578
Models . . . 578
Relationships . . . 578
Routes . . . 579
Templates . . . 579
3. Step 1: Init Project and Set Up Database . . . 579
Create Migrations . . . 579
4. Step 2: User Model with Registration and Login . . . 580
Authentication Routes . . . 581
Test Registration and Login . . . 583
5. Step 3: Task Model with CRUD . . . 584
Task Routes . . . 585
Test the Task API . . . 588
6. Step 4: Dashboard Template with tina4css . . . 588
Dashboard Stats Route . . . 593
7. Step 5: Real-Time Updates via WebSocket . . . 594
8. Step 6: Email Notifications on Task Assignment . . . 595
9. Step 7: Add Caching for the Dashboard . . . 596

10. Step 8: Write Tests . . . 597
11. Step 9: Deploy with Docker . . . 601
12. The Complete Project Structure . . . 603
13. What to Build Next . . . 603
14. Closing Thoughts -- The Tina4 Philosophy . . . 604

Release Notes **605**

v3.13.138 (2026-09-24) - Safer requests, templates, and database operations . . . 605
v3.13.105 (2026-08-19) - Cross-API invariants, honest logs, portable tests . . . 606
Queue and ORM audit bugs — five fixed . . . 606
Route inspection scans, never boots . . . 606
Startup log tells the truth about @noauth / @secured . . . 607
Hot reload reaches every module that imports the changed one . . . 607
Firebird migration ledger is case-agnostic . . . 607
Test-harness fixes . . . 607
Skills grow a spine . . . 607
Upgrade notes . . . 608
v3.13.104 (2026-08-17) - OIDC SSO and PostGIS land . . . 608
OIDC / OpenID Connect — provider-neutral, session handoff . . . 608
PostGIS spatial support . . . 608
Swagger is SSO-aware . . . 608
v3.13.102 (2026-08-15) - Unified client and CI hardening . . . 608
v3.13.103 (2026-08-16) - Metrics you can trust, releases that cannot lie . . . 609
What changed . . . 609
PHP connection timeout diagnosis . . . 609
What ships . . . 609
Failure is data . . . 610
Breaking name change . . . 610

v3.13.100 (2026-08-14) - Frond keeps the whole page . . . 610
Template inheritance . . . 610
Bounded caches . . . 610
Frond extension scope . . . 610
AI skill installer . . . 610
Release consistency . . . 610
Upgrade notes . . . 611
v3.13.99 (2026-08-13) - Stability, parity, and secure by default . . . 611
Security . . . 611
Data integrity . . . 612
ORM and validation . . . 612
Request model - the big one . . . 613
Migrations . . . 613
HTTP . . . 613
Dev tooling . . . 613
Proven in all four . . . 614
Bug fixes . . . 614
Possible breaking . . . 614
Coming in 3.13.100 . . . 615
v3.13.98 (2026-08-11) - Skills discipline . . . 615
v3.13.97 (2026-08-07) - Behaviour corrections . . . 616
ORM . . . 616
Sessions . . . 616
DocStore . . . 616
Under the hood . . . 616
Possible breaking . . . 616
v3.13.96 (2026-08-07) - One paginate envelope, one message shape . . . 617

Pagination . . . 617
Messenger . . . 617
Swagger . . . 617
Migrations . . . 618
Server . . . 618
Build and skills . . . 618
Proven in all four . . . 618
Possible breaking . . . 618
v3.13.95 (2026-08-06) - Preparing for the 3.14 stable release . . . 619
Queues . . . 619
Logging . . . 619
Databases . . . 619
Sessions . . . 619
PHP production runtimes . . . 620
Local by default, production by environment variable . . . 620
Bulk insert . . . 620
Tooling . . . 620
Possible breaking . . . 620
v3.13.94 (2026-07-29) - A write with no filter is an error . . . 621
The write path (BREAKING) . . . 621
The Frond sandbox revokes capability, it does not skip a step . . . 622
Messenger: one name, one signature . . . 622
Base images that boot . . . 622
Frond, measured against the engines it replaces . . . 622
Also in this release . . . 623
PHP specifics . . . 623
v3.13.92 (2026-07-27) - HS512 no longer rejects every token you mint . . . 623

HS384 and HS512 rejected every token (Fixed) . . . 623
The header names the algorithm that signed (php#187) . . . 624
nbf is enforced, with 60 seconds of leeway (php#187) . . . 624
The header cannot choose the verifier . . . 624
authenticateRequest forwards its algorithm . . . 624
A misconfigured algorithm throws (Breaking) . . . 625
The tests that hold it . . . 625
v3.13.91 (2026-07-27) - The offenders list finally points at code worth fixing . . . 625
A function is no longer charged for the functions inside it . . . 625
Ruby complexity was double what it should have been . . . 626
LOC means one thing again . . . 626
The dev dashboard reads the coupling it is given . . . 627
v3.13.90 (2026-07-27) - A scaffolded model and its migration finally agree . . . 627
One default, defined once (php#186, ruby#33, nodejs#38) . . . 627
A failed write no longer reports success . . . 627
v3.13.89 (2026-07-27) - A typo'd tag no longer renders what it was hiding . . . 628
An unknown tag raises instead of leaking its body (Breaking, Security) . . . 628
set works as a block (Fixed) . . . 628
The expression corpus grew to 82 . . . 629
v3.13.88 (2026-07-27) - json_encode gives you JSON again . . . 629
json_encode emits JSON, not entities (Breaking, reverts 3.13.87) . . . 629
A value JSON cannot represent becomes null (Fixed) . . . 630
to_json and tojson are the same filter . . . 630
The expression corpus grew to 79 . . . 630
What to change in your templates . . . 630
v3.13.87 (2026-07-27) - Frond expressions agree in all four frameworks . . . 631
Booleans print as true or false (Breaking) . . . 631

{% import "file" as alias %} now works (New) . . . 631
The not operator in output . . . 631
json_encode is HTML escaped (Breaking) . . . 632
The gate that keeps this true . . . 632
v3.13.86 (2026-07-25) - Writes return a DatabaseResult . . . 632
v3.13.85 (2026-07-24) - The dev-admin bundle ships once . . . 633
A gate, so the duplicate cannot come back . . . 633
v3.13.84 (2026-07-24) - Every generated Dockerfile actually starts . . . 633
serve no longer kills PID 1 (all four frameworks) . . . 634
A gate, so this cannot rot again . . . 634
Also in the CLI (tina4 v3.8.58) . . . 634
v3.13.83 (2026-07-24) - Swagger UI stops serving itself in production . . . 635
The banner advertises only what answers (python#99) . . . 635
$app->run() under php-fpm serves the request (php#180) . . . 635
The Composer package sheds 3.4MB (php#181) . . . 636
The CLI runs no watcher in production (tina4 v3.8.57) . . . 636
A stale CA no longer turns a missing certificate into six failures (python#98) . . . 636
v3.13.82 (2026-07-23) - MQTT 3.1.1: talk to any broker, no dependency . . . 636
v3.13.81 (2026-07-21) - tina4php test fails the build when tests fail . . . 637
v3.13.80 (2026-07-19) - The xUnit base class Tina4\Test finally ships . . . 638
v3.13.79 (2026-07-19) - The test suite runs every test file, a renamed session cookie is read back, and a stuck migration clears . . . 638
v3.13.77 (2026-07-16) - The native session cookie is no longer readable by JavaScript . . . 639
v3.13.76 (2026-07-16) - Migrations apply again on a database created before 3.13.55 . . . 639
v3.13.75 (2026-07-14) - Static assets revalidate, so a deploy reaches users without a hard refresh . . . 640
v3.13.74 (2026-07-13) - The dev dashboard connection tester works again . . . 640
v3.13.73 (2026-07-13) - A failed migration re-applies cleanly . . . 640

v3.13.72 (2026-07-12) - Frond number_format, filter precedence, and a sandbox fix 641

v3.13.71 (2026-07-11) - AI skills: sharper tina4_code guidance 642

v3.13.70 (2026-07-11) - Column defaults on INSERT, a Firebird charset override, and stacked Swagger metadata 642

ORM honours column defaults on INSERT (#165) 642

Firebird connection charset override (#160) 642

Stacked Swagger metadata all survives (#59) 642

v3.13.69 (2026-07-10) - The Api client grows up, and a cross-origin redirect leak is closed . . 642

Security 643

Also shipping (previously held) 643

v3.13.68 (2026-07-10) - Firebird counts again 643

v3.13.67 (2026-07-10) - The MCP table browser lists your tables again 644

v3.13.66 (2026-07-10) - A self-describing CLI, and generators that ship their tests 644

The self-describing CLI 644

Generators now ship a test with the code 644

Databases 645

Fixes 645

Breaking 645

v3.13.56 (2026-07-08) - Skills that own up when they drift 645

v3.13.55 (2026-07-07) - One migration tracking schema on every engine 646

v3.13.54 (2026-07-07) - Migrations honour the SET TERM directive 646

v3.13.53 (2026-07-06) - JSONField: JSON document columns 647

v3.13.52 (2026-07-04) - Frond live blocks, pgsql:// URL scheme, SCSS colour functions . . . 647

v3.13.51 (2026-07-03) - MCP Streamable HTTP transport, Firebird fixes 648

v3.13.50 (2026-07-02) - Path route params match INTEGER primary keys on SQLite (Ruby fix) 649

v3.13.47 (2026-06-25) - Open-issue batch: migration comment splitting, global middleware, SCSS interpolation 649

v3.13.46 (2026-06-24) - MySQL/MSSQL batch atomicity tests 650

v3.13.45 (2026-06-24) - SQLite commit resilience + real-service test hardening 650

v3.13.44 (2026-06-24) - Real-service bug-fix sweep (no mocks) 651

v3.13.43 (2026-06-22) - Queue: MongoDB fail/retry/dead-letter route to the active backend . . 652

v3.13.42 (2026-06-22) - Swagger configurability for external and public APIs 652

v3.13.41 (2026-06-22) - Queue reservation/visibility timeout (at-least-once delivery) 653

v3.13.40 (2026-06-22) - MCP security hardening + Swagger/OpenAPI overhaul 654

v3.13.39 (2026-06-21) - Auto-migration, unified critical log level, fail-loud ORM, per-route WebSocket auth 655

v3.13.38 (2026-06-19) - Coordinated security & robustness release 656

v3.13.37 (2026-06-18) - Dev-admin editor: Ruby + more syntax highlighting 656

v3.13.36 (2026-06-18) - Instant WebSocket dev-reload (parity) 656

v3.13.35 (2026-06-17) - Live MCP endpoint for AI agents 657

v3.13.34 (2026-06-17) - Store images + dual-port test 657

v3.13.33 (2026-06-17) - Queues: priority pop + automatic dead-lettering (Warning: behavioural change) 657

v3.13.32 (2026-06-17) - Caching: per-query bypass + X-Cache headers + string-middleware (chapter rewritten) 657

v3.13.31 (2026-06-17) - Version alignment (no functional change in PHP) 657

v3.13.30 (2026-06-16) - Typed route params coerce + /__dev auth-bypass fixed (Warning: behavioural change) 658

v3.13.29 (2026-06-16) - Live API search finds magic methods + ranks qualified queries 658

v3.13.26 (2026-06-16) - pooling fixes: standalone writes auto-commit + independent pooled PostgreSQL connections 658

v3.13.24 (2026-06-15) - unified cache backends across response, KV, and persistent DB cache 659

v3.13.23 (2026-06-15) - request-scoped DB query cache, on by default 659

v3.13.21 (2026-06-15) - docs: render() corrections + version re-sync 660

v3.13.19 (2026-06-15) - return domain objects, construct from JSON, and one database binder 660

$response(...) serializes domain objects 660

Construct a model from JSON, or data-first 660

Warning: Breaking - one database binder: bindDatabase 660

v3.13.18 (2026-06-15) - ORM relationship + QueryBuilder fixes + boolean param binding . . . 661

v3.13.17 (2026-06-15) - PostgreSQL: native-type reads + execute() reports real failure 661

Reads return native PHP types (not strings) 662

execute() propagates failure 662

v3.13.16 (2026-06-15) - createTable() works on PostgreSQL + DatabaseResult index access . 662

createTable() is now engine-aware 662

v3.13.14 (2026-06-13) - Logs reach stdout in containers + per-request logging + schema-qualified tables (#48) 663

PHP also: #119 - legacy-env guard crash under the built-in server 663

Per-request logging - on by default in dev 663

What changed (stdout) 663

Why it spanned all four 663

Schema-qualified tables (#48) + a PostgreSQL fetch() regression 664

Tests 665

v3.13.13 (2026-06-11) - PHP only: large-response truncation fix 665

Responses larger than the OS send buffer are no longer truncated 665

The fix 665

Why PHP only - cross-framework verification 666

Tests 666

v3.13.12 (2026-06-11) - SQL safety + implicit ORM binding + fetchAll correctness 666

fetchAll actually fetches ALL rows now (no silent 100-row truncation) 667

Trailing ; is now stripped from user SQL in fetch() / fetchOne() 667

Implicit ORM binding from TINA4_DATABASE_URL 667

Cross-framework parity 668

Tests 668

v3.13.11 (2026-06-11) - ORM correctness pass (parity bump) . . . 668
Per-issue audit . . . 668
Tests . . . 669
v3.13.9 (2026-06-10) . . . 669
The bug . . . 669
The fix . . . 669
Same algorithm in Python / Ruby / Node . . . 669
Tests . . . 670
What you'll see when you re-install . . . 670
v3.13.7 (2026-06-10) . . . 670
NEW: tina4.request.error event . . . 670
FIX: Stack trace removed from production 500 body (CWE-209) . . . 671
Tests . . . 671
Background . . . 671
v3.13.6 (2026-06-09) . . . 671
#46 - PostgreSQL transaction cascade (no fix needed) . . . 671
#47 - Driver install hints (no change needed) . . . 672
Tests . . . 672
v3.13.5 (2026-06-05) . . . 672
What changes . . . 672
Instance form still works . . . 672
clearRegistry() for test fixtures . . . 673
Implementation notes per framework . . . 673
Test count . . . 673
Upgrade . . . 673
v3.13.4 (2026-06-04) . . . 674
PY-10-02 - @middleware() no longer silently disables auth (SECURITY) . . . 674

PY-10-03 - request.headers is now case-insensitive . . . 674
PY-10-01 - Function-based middleware now runs . . . 675
Python chapter rewrites - book + docs . . . 675
Test count . . . 675
Upgrade . . . 676
v3.13.3 (2026-06-03) . . . 676
Env typed env-var helpers (tina4-python#43) . . . 676
Function-name in log lines (tina4-python#41) . . . 676
Test count . . . 677
Upgrade . . . 677
v3.13.2 (2026-06-03) . . . 677
SCSS calc() with mixed units (tina4-python#42, tina4-php#116, tina4-nodejs#1) . . . 677
Router.group docs taught a crashing pattern (tina4-python#44) . . . 678
DATABASEURL -> TINA4DATABASE_URL drift (tina4-python#45) . . . 678
Test count . . . 678
Upgrade . . . 678
v3.13.1 (2026-06-02) . . . 678
Convenience parity additions (Groups A / B) . . . 678
Decorator-style GraphQL resolvers across the family . . . 679
Class-based service pattern across the family . . . 679
PHP chapter rewrites (docs/php/ and book-2-php/) . . . 680
Test count . . . 680
Upgrade . . . 680
v3.13.0 (2026-06-01) . . . 681
The headline fire: Test class with HTTP helpers . . . 681
Auth.valid_token now returns the payload, not a bare bool - BREAKING . . . 681
Python-specific groups (mirrored to PHP/Ruby/Node in follow-up patches) . . . 682

PHP-specific: Tina4\Debug shim . . . 683
Documentation sweep . . . 683
Aspirational chapters rewritten . . . 683
tina4-js doc drift caught . . . 684
Test counts . . . 684
Upgrade . . . 684
v3.12.14 (2026-06-01) . . . 685
Python - tina4_python.test xUnit testing surface . . . 685
Cross-framework parity check (testing) . . . 686
PHP - :named placeholder translation across non-PDO adapters . . . 686
Cross-framework parity check (:named placeholders) . . . 686
Upgrade . . . 686
v3.12.13 (2026-05-29) . . . 687
Cross-framework dev-admin parity sweep (Tier 1-5) . . . 687
Folded-in from PHP 3.12.11 - file upload regression (tina4-book#139) . . . 689
Folded-in from PHP 3.12.12 + Python 3.12.13 - v2 tina4_migration auto-upgrade (#115) . . . 689
Folded-in from PHP 3.12.11 - request URL parity . . . 690
Upgrade . . . 690
v3.12.10 (2026-05-14) . . . 690
PHP - ORM->save() no longer swallows write failures (#114) . . . 690
Also in the PHP 3.12.7-3.12.9 patch line . . . 691
Python / Ruby / Node . . . 691
Upgrade . . . 691
v3.12.6 (2026-05-06) . . . 691
Python - psycopg2 % substitution no longer trips PL/pgSQL function bodies (#40) . . . 691
Long-standing tina4-js #37 confirmed fixed . . . 692
Upgrade . . . 692

v3.12.5 (2026-05-06) . . . 692
PHP - multipart bodies with file uploads now parse correctly (#135) . . . 692
Upgrade . . . 693
v3.12.4 (2026-05-06) . . . 693
25 new env vars across all 4 frameworks . . . 693
Logging - env-driven file output + rotation . . . 693
Documentation-truth CI gate now strict on both axes . . . 694
Tests added . . . 694
Upgrade path . . . 694
v3.12.3 (2026-05-05) . . . 694
Breaking changes (Ruby + PHP only) . . . 694
Doc parity - CLAUDE.md and book chapter 33 . . . 695
Other fixes . . . 695
Genuine gaps surfaced by the parity audit (follow-up, not blocking 3.12.3) . . . 695
Upgrade path . . . 696
v3.12.2 (2026-05-05) . . . 696
Firebird URL auto-detect . . . 696
Bug fix specific to PHP - mysqli localhost+port quirk . . . 697
Other fixes . . . 697
v3.12.1 (2026-05-04) . . . 697
v3.12.0 (2026-05-04) . . . 697
Why this release . . . 698
What changed . . . 698
Bug fixes shipped alongside the rename . . . 698
Migration - every renamed var . . . 699
Names that stay un-prefixed (not framework config) . . . 700
How to upgrade . . . 700

Parity 700
v3.11.32 (2026-04-25) 700
v3.11.13 (2026-04-16) 701
v3.11.12 (2026-04-16) 702
v3.11.11 (2026-04-16) 703
v3.11.10 (2026-04-15) 703
v3.11.9 (2026-04-15) 704
v3.10.99 (2026-04-12) 704
v3.10.97 (2026-04-11) 705
v3.10.93 (2026-04-11) 705
v3.10.92 (2026-04-10) 706
v3.10.91 (2026-04-10) 706
v3.10.90 (2026-04-09) 706
v3.10.89 (2026-04-09) 706
v3.10.87 (2026-04-09) 707
v3.10.86 (2026-04-09) 707
v3.10.85 (2026-04-09) 707
v3.10.84 (2026-04-09) 707
v3.10.83 (2026-04-08) 708
v3.10.70 (2026-04-06) 708
Version History 708
v3.10.68 (2026-04-03) - Full Parity Release 708
v3.10.67 (2026-04-03) 709
v3.10.66 (2026-04-03) 709
v3.10.65 (2026-04-03) 709
v3.10.60 (2026-04-03) 709

v3.10.57 (2026-04-02) . . . 710
v3.10.54 (2026-04-02) . . . 710
v3.10.48 - April 2, 2026 . . . 711
Bug Fixes . . . 711
v3.10.46 - April 1, 2026 . . . 711
Test Coverage . . . 711
Bug Fixes . . . 711
v3.10.45 - April 1, 2026 . . . 711
Bug Fixes . . . 711
v3.10.44 - April 1, 2026 . . . 711
New Features . . . 711
Bug Fixes . . . 712
Test Coverage . . . 712
v3.10.40 - April 1, 2026 . . . 712
Bug Fixes . . . 712
v3.10.39 - April 1, 2026 . . . 712
Breaking Changes . . . 712
New Features . . . 713
v3.10.38 -- April 1, 2026 . . . 714
Code Metrics & Bubble Chart . . . 714
AI Context Installer . . . 714
Dashboard Improvements . . . 714
Cleanup . . . 714
v3.10.x -- Previous Releases . . . 714
v3.10.29 -- tina4 ai Command (30 March 2026) . . . 714
v3.10.28 -- DevReload Performance Fix (30 March 2026) . . . 715
v3.10.27 -- Frond Macro HTML Escaping Fix (30 March 2026) . . . 715

v3.10.26 -- Rebrand and ORM Transaction Safety (30 March 2026) . . . 716
v3.10.22-24 -- Form Tokens and Template Fixes (29-30 March 2026) . . . 716
v3.10.18-20 -- Frond Parser Fixes (28-29 March 2026) . . . 716
v3.10.14-16 -- Filters, Auto-Commit, and ID Generation (28 March 2026) . . . 717
v3.10.10-12 -- Firebird, Sessions, and Dictionary Access (28 March 2026) . . . 717
v3.10.5 -- Frond Quote-Aware Operator Matching (28 March 2026) . . . 717
v3.10.1 -- autoMap for ORM Field Mapping (28 March 2026) . . . 717
v3.10.0 -- Singleton Frond Engine (28 March 2026) . . . 718
v3.9.x -- QueryBuilder, Sessions, Security . . . 718
v3.9.0 -- QueryBuilder, Sessions, Path Injection (26 March 2026) . . . 718
v3.9.1 -- Secure by Default (27 March 2026) . . . 719
v3.9.2-3 -- NoSQL QueryBuilder and Cookie Fixes (27 March 2026) . . . 719
v3.9.4 -- setcookie Fatal Error Fix (27 March 2026) . . . 719
v3.8.x -- Hot Reload, Security Headers, Connection Pooling . . . 720
v3.8.0 -- Hot Reload and Frond Filters (25 March 2026) . . . 720
v3.8.1-3 -- Security and Validation (25 March 2026) . . . 720
v3.8.7 -- Connection Pooling (26 March 2026) . . . 720
v3.7.x -- Template Auto-Serve, Typed Route Params . . . 720
v3.7.0 -- Template Auto-Serve (25 March 2026) . . . 720
v3.7.1-2 -- Typed Route Parameters (25 March 2026) . . . 720
v3.6.x -- Reference Cleanup . . . 721
v3.6.0 (24 March 2026) . . . 721
v3.5.x -- Bundled Frontend, Swagger, Middleware . . . 721
v3.5.0 (24 March 2026) . . . 721
v3.4.x -- DatabaseResult, File Uploads, WebSocket Broadcast . . . 721
v3.4.0 (24 March 2026) . . . 722
v3.3.x -- Queue API, Route Chaining, Dev Toolbar . . . 722

v3.3.0 (24 March 2026) . . . 722
v3.2.x -- Flexible Route Handlers, Strict Mode, Live Reload . . . 723
v3.2.0 (23 March 2026) . . . 723
v3.2.1-2 -- Strict Mode and Benchmarks (23 March 2026) . . . 723
v3.1.x -- Drop-In Server Support, Migration Guide . . . 724
v3.1.0 (22 March 2026) . . . 724
v3.1.1-2 -- README and CI Fixes (22 March 2026) . . . 724
v3.0.0 -- The Rewrite . . . 724
What Changed . . . 724
Breaking Changes from v2 . . . 725
Release Candidates (21 March 2026) . . . 726
Upgrade Path Summary . . . 726
Upgrading from v2 to v3 **727**
1. Overview . . . 727
Step 0: Run the Automated Upgrade Command . . . 727
2. Package and Installation . . . 727
v2 . . . 727
v3 . . . 728
3. Project Structure Changes . . . 728
v2 Structure . . . 728
v3 Structure . . . 728
4. Routing Changes . . . 729
Method-Based Registration . . . 729
Class-Based Routes with Attributes . . . 729
Auth Defaults . . . 729
5. Database Changes . . . 730
Connection Strings . . . 730

Firebird Notes . . . 731
Transactions . . . 731
6. ORM Changes . . . 731
Auto-Mapping . . . 731
Explicit Mappings Take Precedence . . . 732
Utility Methods . . . 733
Typed Properties . . . 733
7. Template Engine Changes . . . 733
Frond Singleton . . . 733
Custom Filters and Globals . . . 733
Method Calls in Templates . . . 733
8. Migration Tracking Table . . . 734
9. New Features in v3 . . . 734
Common Pitfalls . . . 735
1. POST/PUT/DELETE routes now require authentication . . . 735
2. Database connection strings changed . . . 735
3. Template engine renamed . . . 735
10. Step-by-Step Migration Checklist . . . 735
Feature Catalog **737**
Read the status correctly . . . 737
Where the features live . . . 738
Current parity boundary . . . 738
Dependency boundary . . . 739
Numbered catalog . . . 739
Foundation . . . 739
Database and providers . . . 740

ORM and data layer . . . 741
HTTP core . . . 741
HTTP policies . . . 741
HTTP runtime . . . 742
Frond template engine . . . 742
Frontend assets . . . 743
Authentication . . . 743
Sessions and providers . . . 743
Cache and providers . . . 744
Integrations and storage . . . 745
Enterprise authentication . . . 746
Application runtime . . . 746
CLI . . . 747
Developer runtime . . . 747
Testing and verification tools . . . 748
What parity means . . . 748

Real-time Collaboration (WebRTC) 749

1. The Media Server You Do Not Have to Run . . . 749
2. What You Get, and How It Differs from Raw WebSocket . . . 749
3. Mounting the Module: Realtime::mount() . . . 750
    What each feature wires . . . 751
4. The Config Bootstrap: GET /api/rtc/config . . . 752
5. ICE and TURN Servers . . . 752
    Environment variables . . . 753
6. Signalling: The Mesh Relay . . . 753
    The connection surface . . . 754

7. Chat: Secured Channels, Presence, History . . . 755
Chat history: GET {p}/api/channels/{id}/messages . . . 757
8. Files: Upload and Download . . . 757
Storage backends . . . 758
9. Auth, Identity, and the Data Model . . . 758
Data model . . . 759
10. A Complete Example . . . 760
Backend - index.php . . . 760
Browser - bootstrap from the config endpoint . . . 761
11. Footguns and Hard Rules . . . 762
Where to Go Next . . . 763
AI Client **764**
Configure a provider . . . 764
Complete one prompt . . . 764
Hold a chat . . . 765
Stream text . . . 765
Create embeddings . . . 766
Handle failures . . . 766
Timeouts and retries . . . 767
OpenID Connect SSO **768**
Configure SSO . . . 768
Use the identity . . . 768
Map claims and providers . . . 769
Swagger . . . 769
GIS and PostGIS **770**
Define a point field . . . 770

Query spatial data 770

Return GeoJSON 771

IoT and MQTT **772**

Configure and publish 772

Consume safely 772

--- outline: deep ---

<div v-pre>

Getting Started with Tina4 PHP

1. What Is Tina4 PHP

Tina4 PHP is a batteries-included web framework for PHP 8.2+. The core declares no required third-party packages. PHP extensions are part of the language runtime and do not count as dependencies; selected providers can still require an extra package, such as the MongoDB library. Tina4 PHP contributes to the framework family's 140-entry feature catalog; the catalog is an inventory, not a claim that every entry has reached parity.

It belongs to the Tina4 family: four backend implementations in Python, PHP, Ruby, and Node.js. They share contracts, project structure, template syntax, CLI commands, and `.env` variables. The parity audit records where an implementation still differs.

Tina4 PHP follows PHP conventions. Method names are `camelCase` -- `fetchOne()`, `softDelete()`, `hasMany()`. Class names are `PascalCase`. Constants are `UPPER_SNAKE_CASE`.

By the end of this chapter you will have a working project with an API endpoint and a rendered HTML page.

---

2. Prerequisites and Installation

What You Need

Four things. Nothing exotic.

- **PHP 8.2 or later** -- check with:

```
php -v
```

You should see output like:

```
PHP 8.3.4 (cli) (built: Feb 13 2026 09:27:45) (NTS)
Copyright (c) The PHP Group
Zend Engine v4.3.4, Copyright (c) Zend Technologies
    with Zend OPcache v8.3.4, Copyright (c), by Zend Technologies
```

If you see a version lower than 8.2, upgrade PHP first.

- **Composer** -- PHP's package manager. Check with:

```
composer --version
```

You should see:

```
Composer version 2.7.2 2024-03-11 17:12:18
```

If Composer is not installed, get it from https://getcomposer.org.

- **The Tina4 CLI** -- a Rust-based binary that manages all four Tina4 frameworks:

macOS (Homebrew):

```
brew install tina4stack/tap/tina4
```

Linux / macOS (install script):

```
curl -fsSL https://raw.githubusercontent.com/tina4stack/tina4/main/install.sh | bash
```

Windows (PowerShell):

```
irm https://raw.githubusercontent.com/tina4stack/tina4/main/install.ps1 | iex
```

Verify:

```
tina4 --version
```

```
tina4 0.1.0
```

- **Required PHP extensions** -- these ship with most PHP installations:
- `ext-json` (bundled since PHP 8.0)
- `ext-mbstring`
- `ext-openssl`
- `ext-sqlite3` (for the default SQLite database)
- `ext-fileinfo`

Check:

```
php -m | grep -E "json|mbstring|openssl|sqlite3|fileinfo"
```

```
fileinfo
json
mbstring
openssl
sqlite3
```

If any are missing, install them via your OS package manager (e.g., `apt install php8.3-mbstring` on Ubuntu).

Creating a New Project

One command. The CLI scaffolds everything.

```
tina4 init php my-store
```

```
■ Initialising php project at ./my-store
■ Checking php runtime...
  ✔ php found
■ Checking package manager...
  ✔ composer found
  ✔ Created directory ./my-store
■ Scaffolding php project...
  ✔ Created directory structure
  ✔ Created .env
  ✔ Created index.php
  ✔ Created .gitignore
  ✔ Created composer.json
```

```
■ Installing dependencies...
  ✔ Dependencies installed

✔ Project created at ./my-store

Next steps:
  cd ./my-store
  tina4 serve
```

Install the PHP dependencies:

```
cd my-store
composer install
```

```
Installing dependencies from lock file
  - Installing tina4/tina4-php (v3.2.2): Extracting archive
Generating autoload files
1 package installed
```

One package. No dependency tree. No version conflicts. Just `tina4/tina4-php`.

Starting the Dev Server

```
tina4 serve
```

```
 _____ _              _  _
|_   _(_)_ __   __ _| || |
  | | | | '_ \ / _` | || |_
  | | | | | | | (_| |__   _|
  |_| |_|_| |_|\__,_|  |_|

  Tina4 PHP v3.2.2
  Server running at http://0.0.0.0:7145
  Debug mode: ON
  Press Ctrl+C to stop
```

Open your browser to `http://localhost:7145`. The Tina4 welcome page appears.

Hit the health check:

```
curl http://localhost:7145/health
```

```
{
  "status": "ok",
  "uptime_seconds": 12,
  "version": "3.2.2",
  "framework": "tina4-php"
}
```

The server is running. Time to write code.

> ***Note:*** *No database exists yet. The SQLite file (`data/app.db`) is created automatically the first time your code opens a database connection -- for example, when you configure `TINA4_DATABASE_URL` in `.env` and run a query or migration. Until then, the `data/` directory remains empty.*

---

3. Project Structure Walkthrough

Here is what `tina4 init` created:

```
my-store/
■■■ .env                    # Your configuration (gitignored)
■■■ .gitignore              # Pre-configured
■■■ index.php               # Application entry point
■■■ composer.json           # Composer package definition
■■■ vendor/                 # Composer packages (after composer install)
■■■ src/
■   ■■■ routes/             # Your route handlers go here
■   ■■■ orm/                # Your ORM model classes go here
■   ■■■ templates/          # Frond/Twig templates
■   ■■■ public/             # Static files (CSS, JS, images)
■   ■   ■■■ js/
■   ■   ■■■ css/
■   ■   ■■■ images/
■   ■■■ scss/               # SCSS source files (compiled to CSS)
■■■ migrations/             # SQL migration files
■■■ data/                   # SQLite databases (gitignored)
■■■ logs/                   # Log files (gitignored)
```

> ***Note:*** *The scaffold creates empty directories. Files like* `tina4.css`, `frond.js`, *and error templates become available at runtime through the* `tina4/tina4-php` *Composer package -- they are not copied into your project.*

Five directories matter:

- `src/routes/` -- Every `.php` file here is auto-loaded at startup. Route definitions go here. Subdirectories are fine.
- `src/orm/` -- Every `.php` file here is auto-loaded. ORM model classes go here.
- `src/templates/` -- Frond (Tina4's built-in template engine -- see Chapter 4: Templates) looks here when you call `$response->render("my-page.html", $data)`.
- `src/public/` -- Files served directly. `src/public/images/logo.png` becomes `/images/logo.png`.
- `data/` -- Where the SQLite database lives once created. Gitignored. The `data/` directory starts empty; the database file (e.g., `app.db`) is created automatically on the first database connection.

---

4. Your First Route

Create `src/routes/greeting.php`:

```
<?php
use Tina4\Router;

Router::get("/api/greeting/{name}", function ($request, $response) {
    $name = $request->params["name"];
    return $response->json([
        "message" => "Hello, " . $name . "!",
        "timestamp" => date("c")
```

```
    ]);
});
```

Save the file. The dev server picks it up. No restart needed if live reload is active. Otherwise, restart with `tina4 serve`.

> ***Cross-language note.*** *PHP, Ruby, and Node read path parameters from* `$request->params` / `request.params` / `req.params`*. Python passes them as function arguments instead. Same concept, two shapes: pick the chapter that matches your runtime.*

Test It

```
http://localhost:7145/api/greeting/Alice
```

```
{
  "message": "Hello, Alice!",
  "timestamp": "2026-03-22T14:30:00+00:00"
}
```

Or with curl:

```
curl http://localhost:7145/api/greeting/Alice
```

```
{"message":"Hello, Alice!","timestamp":"2026-03-22T14:30:00+00:00"}
```

The browser pretty-prints. Curl shows compact JSON. Force pretty output with `?pretty=true`:

```
curl "http://localhost:7145/api/greeting/Alice?pretty=true"
```

```
{
  "message": "Hello, Alice!",
  "timestamp": "2026-03-22T14:30:00+00:00"
}
```

Understanding What Happened

Five things, in order:

- You created a file in `src/routes/`. Tina4 discovered it at startup.
- `Router::get("/api/greeting/{name}", ...)` registered a GET route with a path parameter `{name}`.
- A request arrived at `/api/greeting/Alice`. The router matched the pattern. Your handler ran.
- `$request->params["name"]` extracted `"Alice"` from the URL.
- `$response->json(...)` serialized the array to JSON, set `Content-Type: application/json`, and returned `200 OK`.

No base controller. No service provider. No bootstrapping ritual.

Adding More HTTP Methods

Update `src/routes/greeting.php`:

```
<?php
use Tina4\Router;

Router::get("/api/greeting/{name}", function ($request, $response) {
```

```
    $name = $request->params["name"];
    return $response->json([
        "message" => "Hello, " . $name . "!",
        "timestamp" => date("c")
    ]);
});

Router::post("/api/greeting", function ($request, $response) {
    $name = $request->body["name"] ?? "World";
    $language = $request->body["language"] ?? "en";

    $greetings = [
        "en" => "Hello",
        "es" => "Hola",
        "fr" => "Bonjour",
        "de" => "Hallo",
        "ja" => "Konnichiwa"
    ];

    $greeting = $greetings[$language] ?? $greetings["en"];

    return $response->json([
        "message" => $greeting . ", " . $name . "!",
        "language" => $language
    ], 201);
})->noAuth();
```

::: tip Why `->noAuth()`? Tina4 secures `POST`, `PUT`, `PATCH`, and `DELETE` routes by default; they require a valid bearer token. For public, untyped examples like this one, chain `->noAuth()` to disable that check. In production, remove `->noAuth()` and send an `Authorization: Bearer <token>` header instead. See Authentication for token issuance. :::

Test the POST endpoint:

```
curl -X POST http://localhost:7145/api/greeting \
  -H "Content-Type: application/json" \
  -d '{"name": "Carlos", "language": "es"}'
```

```
{"message":"Hola, Carlos!","language":"es"}
```

Status code: `201 Created`. The second argument to `$response->json()` sets it. For a typed, self-documenting alternative, pass an HTTP constant:

```
return $response->json($data, \Tina4\HTTP_CREATED);
```

Tina4 ships a full set of HTTP constants (`HTTP_OK`, `HTTP_BAD_REQUEST`, `HTTP_NOT_FOUND`, `HTTP_UNAUTHORIZED`, `HTTP_UNPROCESSABLE`, etc.) and content-type constants (`APPLICATION_JSON`, `APPLICATION_XML`, `TEXT_HTML`). The three-argument response form lets you set both at once:

```
return $response("<error/>", \Tina4\HTTP_BAD_REQUEST, \Tina4\APPLICATION_XML);
```

---

5. Your First Template

Tina4 uses **Frond** -- a zero-dependency, Twig-compatible template engine built from scratch. If you know Twig, Jinja2, or Nunjucks, this will feel familiar.

Create a Base Layout

Create `src/templates/base.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}My Store{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    <style>
        body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif; m
        .container { max-width: 960px; margin: 0 auto; padding: 20px; }
        .product-card { border: 1px solid #ddd; border-radius: 8px; padding: 16px; margin: 8px 0
        .product-card h3 { margin-top: 0; }
        .price { color: #2d8f2d; font-weight: bold; font-size: 1.2em; }
        .badge { display: inline-block; padding: 2px 8px; border-radius: 4px; font-size: 0.8em;
        .badge-success { background: #d4edda; color: #155724; }
        .badge-danger { background: #f8d7da; color: #721c24; }
        nav { background: #333; color: white; padding: 12px 20px; }
        nav a { color: white; text-decoration: none; margin-right: 16px; }
    </style>
</head>
<body>
    <nav>
        <a href="/">Home</a>
        <a href="/products">Products</a>
    </nav>
    <div class="container">
        {% block content %}{% endblock %}
    </div>
    <script src="/js/frond.js"></script>
</body>
</html>
```

Two blocks: `title` and `content`. Child templates override what they need. The rest stays.

Create a Product Listing Page

Create `src/templates/products.html`:

```
{% extends "base.html" %}

{% block title %}Products - My Store{% endblock %}

{% block content %}
    <h1>Our Products</h1>
    <p>Showing {{ products | length }} product{{ products | length != 1 ? "s" : "" }}</p>

    {% if products | length > 0 %}
        {% for product in products %}
            <div class="product-card">
                <h3>{{ product.name }}</h3>
```

```
                <p>{{ product.description }}</p>
                <p class="price">${{ product.price | number_format(2) }}</p>
                {% if product.in_stock %}
                    <span class="badge badge-success">In Stock</span>
                {% else %}
                    <span class="badge badge-danger">Out of Stock</span>
                {% endif %}
                {% if not loop.last %}
                    {# Don't add separator after the last item #}
                {% endif %}
            </div>
        {% endfor %}
    {% else %}
        <p>No products available at the moment.</p>
    {% endif %}
{% endblock %}
```

Create the Route That Renders the Template

Create `src/routes/pages.php`:

```
<?php
use Tina4\Router;

Router::get("/products", function ($request, $response) {
    $products = [
        [
            "name" => "Wireless Keyboard",
            "description" => "Ergonomic wireless keyboard with backlit keys.",
            "price" => 79.99,
            "in_stock" => true
        ],
        [
            "name" => "USB-C Hub",
            "description" => "7-port USB-C hub with HDMI, SD card reader, and Ethernet.",
            "price" => 49.99,
            "in_stock" => true
        ],
        [
            "name" => "Monitor Stand",
            "description" => "Adjustable aluminum monitor stand with cable management.",
            "price" => 129.99,
            "in_stock" => false
        ],
        [
            "name" => "Mechanical Mouse",
            "description" => "High-precision wireless mouse with 16,000 DPI sensor.",
            "price" => 59.99,
            "in_stock" => true
        ]
    ];

    return $response->render("products.html", ["products" => $products]);
});
```

See It in the Browser

Open `http://localhost:7145/products`. You see:

- A dark navigation bar with "Home" and "Products" links

- The heading "Our Products"
- A subheading: "Showing 4 products"
- Four product cards. Name, description, price, stock badge.
- The Monitor Stand shows a red "Out of Stock" badge
- The other three show green "In Stock" badges

How Template Rendering Works

The chain is short:

- `$response->render("products.html", ["products" => $products])` tells Frond to render `src/templates/products.html`.
- Frond sees `{% extends "base.html" %}` and loads the base template.
- The `{% block content %}` in `products.html` replaces the same block in `base.html`.
- `{{ product.name }}` outputs the value, auto-escaped for HTML safety.
- `{{ product.price | number_format(2) }}` formats the number with 2 decimal places.
- `{% for product in products %}` loops through the array.
- `{% if product.in_stock %}` renders the correct badge.
- `{{ products | length }}` returns the count.

About tina4css

The `tina4.css` file is Tina4's built-in CSS utility framework. Layout utilities, typography, common UI patterns -- no Bootstrap or Tailwind required. It ships with every scaffolded project. Nothing to download.

---

6. Understanding .env

Open `.env` at the project root:

```
TINA4_DEBUG=true
```

That is likely everything. The scaffold creates a minimal `.env` with debug mode enabled. Everything else uses sensible defaults.

The defaults that matter for development:

Variable	Default Value	What It Does
`TINA4_PORT`	`7145`	Server port
`TINA4_DATABASE_URL`	`sqlite:///data/app.db`	SQLite database path (created on first connection)
`TINA4_LOG_LEVEL`	`ALL`	All log messages output

`TINA4_CORS_ORIGINS`	*(empty)*	No cross-origin browser requests; list your front end's origin to allow it
`TINA4_RATE_LIMIT`	`60`	60 requests per minute per IP

Log levels control how much output Tina4 produces:

Level	Behaviour
`ALL` / `DEBUG`	Full verbose output. DevReload active (live-reload, error overlay).
`INFO`	Standard logging. Startup messages, request summaries.
`WARNING`	Warnings and errors only.
`ERROR`	Errors only. Minimal output.

Set `TINA4_LOG_LEVEL=DEBUG` during development for maximum visibility. Use `WARNING` or `ERROR` in production.

To change the port, use the CLI flag or `.env`:

```
tina4 serve --port 8080
```

Or add it to your `.env` file:

```
TINA4_DEBUG=true
TINA4_PORT=8080
```

Restart the server. It runs on port 8080.

How port resolution works: The Rust CLI (`tina4 serve`) determines the port using this priority order:

- **CLI flag** (highest priority): `tina4 serve --port 8080`
- `.env` **file**: `TINA4_PORT=8080`
- **Environment variable**: `PORT=8080`
- **Framework default** (PHP: 7145, Python: 7146, Ruby: 7147, Node.js: 7148)

The CLI reads your `.env` file and checks for `TINA4_PORT` (and falls back to `PORT`). The resolved port is passed to the PHP server. All three methods work -- use whichever fits your workflow.

Disable the browser auto-open

By default, `tina4 serve` opens your browser each time the server restarts. If you save files often, this gets annoying. Add this to `.env` to keep the browser out of your way:

```
TINA4_NO_BROWSER=true
```

For the complete `.env` reference, see Environment Variables.

---

7. The Dev Dashboard

With `TINA4_DEBUG=true` in your `.env`, Tina4 provides a built-in development dashboard. No additional environment variables are needed.

Restart and navigate to:

```
http://localhost:7145/__dev
```

The dashboard opens. Six panels. Each one saves you from adding print statements:

- **System Overview** -- framework version, PHP version, uptime, memory usage, database status
- **Request Inspector** -- recent HTTP requests with method, path, status, duration, request ID. Click any request for full headers, body, database queries, and template renders.
- **Error Log** -- unhandled exceptions with stack traces and occurrence counts
- **Queue Manager** -- pending, reserved, failed, dead-letter messages
- **WebSocket Monitor** -- active WebSocket connections with metadata
- **Routes** -- all registered routes with methods, paths, and middleware

When you visit any HTML page (like `/products`), a **debug overlay** appears at the bottom:

- Request details (method, URL, duration)
- Database queries executed (with timing)
- Template renders (with timing)
- Session data
- Recent log entries

This overlay exists only when `TINA4_DEBUG=true`. Production never sees it.

---

8. Manual Setup (No CLI)

The `tina4` CLI creates the project for you. But if you start from an empty folder (just Composer and a text editor), here is the minimum you need.

Step 1: Install the Package

```
composer require tina4stack/tina4php
```

Step 2: Create `index.php`

This is the entry point. Create a file called `index.php` in your project root:

```
<?php
require_once "./vendor/autoload.php";

$app = new \Tina4\App(basePath: __DIR__, development: true);
```

```
$app->start();

// Dispatch when running under PHP built-in server
if (php_sapi_name() === "cli-server") {
    $response = \Tina4\Router::dispatch(new \Tina4\Request(), new \Tina4\Response());
    http_response_code($response->getStatusCode());
    foreach ($response->getHeaders() as $name => $value) {
        header("$name: $value");
    }
    echo $response->getBody();
}
```

The `App` class boots the framework. The `cli-server` block handles routing when you run PHP's built-in web server.

Step 3: Create the Folder Structure

Tina4 expects this layout:

```
my-project/
■■■ index.php
■■■ .env
■■■ vendor/
■■■ src/
    ■■■ routes/       # Route files go here
    ■■■ templates/    # Twig templates go here
    ■■■ public/       # Static files (CSS, JS, images)
```

Create the directories:

```
mkdir -p src/routes src/templates src/public
```

Step 4: Create `.env`

```
TINA4_DEBUG=true
```

Step 5: Run It

```
tina4 serve
```

The server starts on `http://localhost:7145`. You should see the Tina4 welcome page. From here, add route files in `src/routes/` and templates in `src/templates/`, the same way as a CLI-scaffolded project.

> ***Note:*** *Tina4 PHP refuses to start without the Rust CLI. To bypass it (for example inside a Docker image that already wraps the framework) set* `TINA4_OVERRIDE_CLIENT=true` *in* `.env` *and run the framework's bundled server entry directly. The legacy* `php -S localhost:7145 index.php` *does* ***not*** *wire up the dev watcher, SCSS compilation, or WebSocket dev-reload bridge.*

---

9. Request & Response Fundamentals

Before jumping into the exercises, let's consolidate how route handlers work in Tina4 PHP. Every handler receives two arguments: `$request` (what the client sent) and `$response` (what you send back). Here is the complete picture.

Reading Query Parameters

Query parameters are the key-value pairs after the `?` in a URL. Access them through `$request->query`:

```
// URL: /api/search?q=laptop&page=2
$request->query["q"]      // "laptop"
$request->query["page"]   // "2" (always a string)
$request->query["sort"] ?? "name"  // "name" (default -- param was not sent)
```

`$request->params` is reserved for **route path parameters** (see next section). Don't mix them up.

Reading URL Path Parameters

Route patterns like `/users/{id}` capture segments of the URL. Access them through `$request->params`:

```
<?php
use Tina4\Router;

Router::get("/users/{id:int}/posts/{slug}", function ($request, $response) {
    $id = $request->params["id"];     // 5 (int, because of :int)
    $slug = $request->params["slug"]; // "hello-world" (string)
    return $response->json(["user_id" => $id, "slug" => $slug]);
});
```

The `{id:int}` syntax tells Tina4 to convert the value to an integer. Without `:int`, it stays a string.

Reading the Request Body

POST, PUT, and PATCH requests carry a body. Tina4 parses JSON bodies into an associative array automatically (as long as the client sends `Content-Type: application/json`):

```
Router::post("/api/items", function ($request, $response) {
    $name = $request->body["name"] ?? "";
    $price = $request->body["price"] ?? 0;
    return $response->json(["received_name" => $name, "received_price" => $price]);
})->noAuth();
```

Reading Headers

Headers are available as an associative array with their original casing:

```
$contentType = $request->headers["Content-Type"] ?? "not set";
$authToken = $request->headers["Authorization"] ?? "";
$custom = $request->headers["X-Custom-Header"] ?? "";
```

Sending JSON Responses

`$response->json()` converts an array to JSON and sets the correct `Content-Type`. Pass a status code as the second argument:

```
return $response->json(["id" => 1, "name" => "Widget"]);        // 200 OK (default)
return $response->json(["id" => 1, "name" => "Widget"], 201);   // 201 Created
return $response->json(["error" => "Not found"], 404);           // 404 Not Found
```

Sending HTML / Template Responses

`$response->render()` renders a Frond template from `src/templates/` and passes data to it:

```
return $response->render("products.html", ["products" => $productList, "title" => "Our Products"
```

For raw HTML without a template file:

```
return $response->html("<h1>Hello</h1><p>This works too.</p>");
```

Status Codes

The most common status codes you will use:

Code	Meaning	When to Use
`200`	OK	Successful GET (default)
`201`	Created	Successful POST that created something
`400`	Bad Request	Client sent invalid input
`404`	Not Found	Resource does not exist
`500`	Internal Server Error	Something broke on the server

Worked Example: A Complete Route File

Here is a full route file that ties everything together. It builds a small book lookup API with query parameters, path parameters, JSON responses, and proper status codes. Read through it before attempting the exercises -- it is your reference.

Create `src/routes/books.php`:

```
<?php
use Tina4\Router;

// In-memory data store
$books = [
    ["id" => 1, "title" => "Dune", "author" => "Frank Herbert", "year" => 1965],
    ["id" => 2, "title" => "Neuromancer", "author" => "William Gibson", "year" => 1984],
    ["id" => 3, "title" => "Snow Crash", "author" => "Neal Stephenson", "year" => 1992]
];

Router::get("/api/books", function ($request, $response) use (&$books) {
    // List all books. Supports ?author= filter and ?sort=year.
    $author = $request->query["author"] ?? "";
    $sortBy = $request->query["sort"] ?? "";

    $result = $books;

    // Filter by author if the query param is present
    if ($author !== "") {
        $result = array_values(array_filter($result, function ($b) use ($author) {
            return stripos($b["author"], $author) !== false;
```

```
        }));
    }

    // Sort by year if requested
    if ($sortBy === "year") {
        usort($result, fn($a, $b) => $a["year"] <=> $b["year"]);
    }

    return $response->json(["books" => $result, "count" => count($result)]);
});

Router::get("/api/books/{id:int}", function ($request, $response) use (&$books) {
    // Get a single book by ID. Returns 404 if not found.
    $id = $request->params["id"];
    $book = null;

    foreach ($books as $b) {
        if ($b["id"] === $id) {
            $book = $b;
            break;
        }
    }

    if ($book === null) {
        return $response->json(["error" => "Book with id {$id} not found"], 404);
    }

    return $response->json($book);
});

Router::post("/api/books", function ($request, $response) use (&$books) {
    // Create a new book from the JSON body. Returns 201 on success.
    $title = $request->body["title"] ?? "";
    $author = $request->body["author"] ?? "";
    $year = $request->body["year"] ?? 0;

    if ($title === "" || $author === "") {
        return $response->json(["error" => "title and author are required"], 400);
    }

    $maxId = max(array_column($books, "id"));
    $newBook = [
        "id" => $maxId + 1,
        "title" => $title,
        "author" => $author,
        "year" => $year
    ];
    $books[] = $newBook;

    return $response->json($newBook, \Tina4\HTTP_CREATED);
})->noAuth();
```

::: tip Public endpoints vs secured endpoints `POST`, `PUT`, `PATCH`, and `DELETE` are secured by default. Every example in this chapter uses `->noAuth()` so the `curl` commands work without a token. For real applications, remove `->noAuth()` and send an `Authorization: Bearer <token>` header -- see Authentication. :::

Test it:

```
# List all books
curl http://localhost:7145/api/books

# Filter by author
curl "http://localhost:7145/api/books?author=gibson"

# Sort by year
curl "http://localhost:7145/api/books?sort=year"

# Get a single book
curl http://localhost:7145/api/books/2

# Get a book that does not exist (returns 404)
curl http://localhost:7145/api/books/99

# Create a new book
curl -X POST http://localhost:7145/api/books \
  -H "Content-Type: application/json" \
  -d '{"title": "Foundation", "author": "Isaac Asimov", "year": 1951}'
```

This example covers every building block the exercises use: reading query parameters, reading path parameters, reading the request body, returning JSON with different status codes, and handling missing data. Refer back to it as you work through the exercises below.

---

10. Exercise: Greeting API + Product List Template

Build both features from scratch. No peeking at the examples above.

Exercise Part A: Greeting API

Create an API endpoint at `GET /api/greet` that:

- Accepts a query parameter `name` (e.g., `/api/greet?name=Sarah`)
- Defaults to `"Stranger"` if `name` is missing
- Returns JSON:

```
{
  "greeting": "Welcome, Sarah!",
  "time_of_day": "afternoon"
}
```

- Calculates `time_of_day` from the server's current hour:
- 5:00 - 11:59 = "morning"
- 12:00 - 16:59 = "afternoon"
- 17:00 - 20:59 = "evening"
- 21:00 - 4:59 = "night"

Test your endpoint with:

```
curl "http://localhost:7145/api/greet?name=Sarah"
curl "http://localhost:7145/api/greet"
```

Exercise Part B: Product List Page

Create a page at `GET /store` that:

- Displays at least 5 products (hardcoded)
- Each product has: name, category, price, and a boolean `featured` flag
- Featured products are visually distinct (different background, border, or badge)
- The page shows total product count and featured count
- Uses template inheritance -- a layout template and a page template that extends it
- Includes `tina4.css` and `frond.js`

Your products data:

```
$products = [
    ["name" => "Espresso Machine", "category" => "Kitchen", "price" => 299.99, "featured" => tru
    ["name" => "Yoga Mat", "category" => "Fitness", "price" => 29.99, "featured" => false],
    ["name" => "Standing Desk", "category" => "Office", "price" => 549.99, "featured" => true],
    ["name" => "Noise-Canceling Headphones", "category" => "Electronics", "price" => 199.99, "fe
    ["name" => "Water Bottle", "category" => "Fitness", "price" => 24.99, "featured" => false]
];
```

Expected browser output:

- Page titled "Our Store"
- Text: "5 products, 3 featured"
- Product cards with name, category, price, and a "Featured" badge on highlighted items
- Featured products have a distinct visual style

---

11. Solutions

Solution A: Greeting API

Create `src/routes/greet.php`:

```
<?php
use Tina4\Router;

Router::get("/api/greet", function ($request, $response) {
    $name = $request->query["name"] ?? "Stranger";
    $hour = (int) date("G");

    if ($hour >= 5 && $hour < 12) {
        $timeOfDay = "morning";
    } elseif ($hour >= 12 && $hour < 17) {
        $timeOfDay = "afternoon";
    } elseif ($hour >= 17 && $hour < 21) {
        $timeOfDay = "evening";
    } else {
        $timeOfDay = "night";
    }

    return $response->json([
```

```
        "greeting" => "Welcome, " . $name . "!",
        "time_of_day" => $timeOfDay
    ]);
});
```

Test:

```
curl "http://localhost:7145/api/greet?name=Sarah"

{"greeting":"Welcome, Sarah!","time_of_day":"afternoon"}

curl "http://localhost:7145/api/greet"

{"greeting":"Welcome, Stranger!","time_of_day":"afternoon"}
```

Solution B: Product List Page

Create `src/templates/store-layout.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}Store{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    <style>
        body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif; m
        .container { max-width: 960px; margin: 0 auto; padding: 20px; }
        header { background: #1a1a2e; color: white; padding: 16px 20px; }
        header h1 { margin: 0; }
        .stats { color: #888; margin: 8px 0 20px; }
        .product-grid { display: grid; gap: 16px; }
        .product-card { background: white; border: 2px solid #e0e0e0; border-radius: 8px; paddin
        .product-card.featured { border-color: #ffc107; background: #fffdf0; }
        .product-name { font-size: 1.2em; font-weight: bold; margin: 0 0 4px; }
        .product-category { color: #666; font-size: 0.9em; }
        .product-price { color: #2d8f2d; font-weight: bold; font-size: 1.1em; margin-top: 8px; }
        .featured-badge { background: #ffc107; color: #333; padding: 2px 8px; border-radius: 4px
    </style>
</head>
<body>
    <header>
        <h1>{% block header %}Store{% endblock %}</h1>
    </header>
    <div class="container">
        {% block content %}{% endblock %}
    </div>
    <script src="/js/frond.js"></script>
</body>
</html>
```

Create `src/templates/store.html`:

```
{% extends "store-layout.html" %}

{% block title %}Our Store{% endblock %}
{% block header %}Our Store{% endblock %}

{% block content %}
    <p class="stats">{{ products | length }} products, {{ featured_count }} featured</p>
```

```
    <div class="product-grid">
        {% for product in products %}
            <div class="product-card{{ product.featured ? ' featured' : '' }}">
                <p class="product-name">
                    {{ product.name }}
                    {% if product.featured %}
                        <span class="featured-badge">Featured</span>
                    {% endif %}
                </p>
                <p class="product-category">{{ product.category }}</p>
                <p class="product-price">${{ product.price | number_format(2) }}</p>
            </div>
        {% endfor %}
    </div>
{% endblock %}
```

Create `src/routes/store.php`:

```
<?php
use Tina4\Router;

Router::get("/store", function ($request, $response) {
    $products = [
        ["name" => "Espresso Machine", "category" => "Kitchen", "price" => 299.99, "featured" =>
        ["name" => "Yoga Mat", "category" => "Fitness", "price" => 29.99, "featured" => false],
        ["name" => "Standing Desk", "category" => "Office", "price" => 549.99, "featured" => tru
        ["name" => "Noise-Canceling Headphones", "category" => "Electronics", "price" => 199.99,
        ["name" => "Water Bottle", "category" => "Fitness", "price" => 24.99, "featured" => fals
    ];

    $featuredCount = count(array_filter($products, fn($p) => $p["featured"]));

    return $response->render("store.html", [
        "products" => $products,
        "featured_count" => $featuredCount
    ]);
});
```

Open `http://localhost:7145/store`**.** You see:

- A dark header reading "Our Store"
- Text: "5 products, 3 featured"
- Five product cards in a grid
- Three cards (Espresso Machine, Standing Desk, Noise-Canceling Headphones) have a yellow border, light yellow background, and a "Featured" badge
- Two cards (Yoga Mat, Water Bottle) have a white background with gray border
- Each card shows name, category, and price formatted to two decimal places

---

12. Gotchas

1. File not auto-discovered

Problem: You created a route file but the URL returns 404.

Cause: The file is not in `src/routes/`. It must be inside `src/routes/` (or a subdirectory), and the filename must end with `.php`.

Fix: Move the file to `src/routes/your-file.php`. Restart the server.

2. "Class not found" errors

Problem: `Class 'Tina4\Route' not found` or similar.

Cause: Missing `use` statement or stale autoload.

Fix: Start the file with `<?php` and include `use Tina4\Router;`. Run `composer dump-autoload` if the error persists.

3. JSON response shows HTML

Problem: Your JSON endpoint returns HTML.

Cause: You returned a string instead of using `$response->json()`. Plain strings are treated as HTML.

Fix: Use `$response->json($data)` for JSON endpoints. Never `echo json_encode($data)`.

4. Template not found

Problem: `Template "my-page.html" not found`.

Cause: The file is not in `src/templates/`, or the filename has a typo.

Fix: Check that the file exists at `src/templates/my-page.html`. The name in `$response->render()` is relative to `src/templates/`.

5. Port already in use

Problem: `Error: Address already in use (port 7145)`.

Cause: Another process occupies port 7145.

Fix: Stop the other process, or change the port:

```
TINA4_PORT=8080
```

Or use the CLI flag: `tina4 serve --port 8080`.

6. Changes not reflected

Problem: You edited a file but the browser shows the old version.

Cause: Live reload may not be active. Browser caching can serve stale content.

Fix: Hard-refresh (`Ctrl+Shift+R` or `Cmd+Shift+R`). If that fails, restart the dev server.

7. .env not loaded

Problem: Environment variables have no effect.

Cause: The `.env` file must be at the project root (same directory as `composer.json`). A subdirectory will not work.

Fix: Move `.env` to the project root.

8. Debug mode in production

Problem: Your production site shows stack traces and query details.

Cause: `TINA4_DEBUG=true` in production.

Fix: Set `TINA4_DEBUG=false` in your production `.env`. This hides debug information, enables HTML minification, and activates `.broken` file health checks.

Routing

1. How Routing Works in Tina4

A URL arrives. The framework finds the function that handles it. The function runs. The result goes back. That mapping -- URL to code -- is routing.

In Tina4, routes live in PHP files inside `src/routes/`. Every `.php` file in that directory (and its subdirectories) is auto-loaded at startup. No registration file. No central config. Drop a file in. It works.

The simplest possible route:

```
<?php
use Tina4\Router;

Router::get("/hello", function ($request, $response) {
    return $response->json(["message" => "Hello, World!"]);
});
```

Save that as `src/routes/hello.php`. Start the server. Visit `http://localhost:7145/hello`:

```
{"message":"Hello, World!"}
```

One line registers the route. One line handles the request.

---

2. HTTP Methods

Five methods. Five static calls on the `Route` class.

```
<?php
use Tina4\Router;

Router::get("/products", function ($request, $response) {
    return $response->json(["action" => "list all products"]);
});

Router::post("/products", function ($request, $response) {
    return $response->json(["action" => "create a product"], 201);
})->noAuth();

Router::put("/products/{id}", function ($request, $response) {
    $id = $request->params["id"];
    return $response->json(["action" => "replace product " . $id]);
})->noAuth();

Router::patch("/products/{id}", function ($request, $response) {
    $id = $request->params["id"];
    return $response->json(["action" => "update product " . $id]);
})->noAuth();

Router::delete("/products/{id}", function ($request, $response) {
    $id = $request->params["id"];
```

```
    return $response->json(["action" => "delete product " . $id]);
})->noAuth();
```

> ***Default auth policy.*** *`POST`, `PUT`, `PATCH`, and `DELETE` are secured by default in Tina4; without `->noAuth()` the examples below return `401 Unauthorized`. In production leave the defaults alone and send an `Authorization: Bearer <token>` header on write requests. For these Getting Started examples we chain `->noAuth()` so `curl` works without setting up auth first.*

Test each one:

```
curl http://localhost:7145/products
```

```
{"action":"list all products"}
```

```
curl -X POST http://localhost:7145/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Widget"}'
```

```
{"action":"create a product"}
```

```
curl -X PUT http://localhost:7145/products/42
```

```
{"action":"replace product 42"}
```

```
curl -X PATCH http://localhost:7145/products/42
```

```
{"action":"update product 42"}
```

```
curl -X DELETE http://localhost:7145/products/42
```

```
{"action":"delete product 42"}
```

`GET` reads. `POST` creates. `PUT` replaces. `PATCH` patches. `DELETE` removes. REST convention. Predictable API.

HEAD and OPTIONS - automatic, no boilerplate

Tina4 handles `HEAD` and `OPTIONS` for you. **You don't register them.** They follow from your `Router::get(...)` / `Router::post(...)` / etc. routes:

- `HEAD` is identical to `GET` except the response carries no body (RFC 9110 §9.3.2). Every `GET` route automatically responds to `HEAD`: the framework strips the response body on the way out and preserves `Content-Length` pointing at the byte count the equivalent `GET` would have sent. Cache validators, link checkers, and uptime probes work without you writing anything.
- `OPTIONS` on any registered path returns `204 No Content` with an `Allow:` header listing every method the path supports (RFC 9110 §9.3.7).
- **Wrong method on an existing path** returns `405 Method Not Allowed` with the same `Allow:` header (RFC 9110 §15.5.6 + §10.2.1). Sending `PUT` to a `GET`-only route used to return `404`, which was semantically wrong and confusing for link checkers. Now you get a real `405`.
- `TRACE` **and** `CONNECT` are rejected with `405` (security default for origin servers).

```
# HEAD on a GET route - same headers, empty body
curl -sI http://localhost:7145/products
# HTTP/1.1 200 OK
# Content-Type: application/json
# Content-Length: 33
```

```
# OPTIONS - discover what the path supports
curl -sI -X OPTIONS http://localhost:7145/products
# HTTP/1.1 204 No Content
# Allow: GET, POST, HEAD, OPTIONS

# Wrong method - clear 405 with Allow header
curl -sI -X PUT http://localhost:7145/products
# HTTP/1.1 405 Method Not Allowed
# Allow: GET, POST, HEAD, OPTIONS
```

Explicit `Router::head()` and `Router::options()` registration

The automatic behaviour is enough for 95% of apps. When you need custom HEAD or OPTIONS handlers, register them explicitly:

```
use Tina4\Router;
use Tina4\Request;
use Tina4\Response;

// HEAD handler that doesn't run the full GET body - useful for
// expensive endpoints where the client only needs to check existence
// or read validators (ETag, Last-Modified).
Router::head("/expensive/{id}", function (Request $request, Response $response) {
    $response->header("ETag", computeEtag($request->params["id"]));
    return $response;
});

// OPTIONS handler that returns more than just Allow - for example
// a discovery endpoint.
Router::options("/api/discovery", function (Request $request, Response $response) {
    return $response->json(["version" => "v1", "endpoints" => [...]]);
});
```

The framework still strips the response body for `HEAD` handlers (RFC 9110 §9.3.2 is a MUST), so you can't accidentally leak content even if your handler returns a body.

---

3. Path Parameters

Curly braces capture values from the URL.

```
<?php
use Tina4\Router;

Router::get("/users/{id}/posts/{postId}", function ($request, $response) {
    $userId = $request->params["id"];
    $postId = $request->params["postId"];

    return $response->json([
        "user_id" => $userId,
        "post_id" => $postId
    ]);
});
```

```
curl http://localhost:7145/users/5/posts/99
```

```
{"user_id":"5","post_id":"99"}
```

Notice: `user_id` came back as the string `"5"`, not the integer `5`. Path parameters are strings by default.

> ***Auto-casting:*** *Tina4 automatically casts path parameter values that are purely numeric to integers. For example, requesting* `/users/42/posts/99` *will give you* `$request->params["id"]` *as the integer* `42` *and* `$request->params["postId"]` *as the integer* `99` *-- no explicit* `:int` *type hint required. The* `:int` *type hint adds validation (rejecting non-numeric values with a 404), but the auto-casting happens regardless.*

Typed Parameters

Add a colon and a type to enforce constraints:

```
<?php
use Tina4\Router;

Router::get("/orders/{id:int}", function ($request, $response) {
    $id = $request->params["id"]; // This is now an integer
    return $response->json([
        "order_id" => $id,
        "type" => gettype($id)
    ]);
});
```

```
curl http://localhost:7145/orders/42
```

```
{"order_id":42,"type":"integer"}
```

Pass a non-integer and the route does not match. A 404 comes back instead:

```
curl http://localhost:7145/orders/abc
```

```
{"error":"Not found","path":"/orders/abc","status":404}
```

Supported types:

Type	Matches	Auto-cast	Example
`int`	Digits only	Integer	`{id:int}` matches `42` but not `abc`
`float`	Decimal numbers	Float	`{price:float}` matches `19.99`
`path`	All remaining path segments (catch-all)	String	`{slug:path}` matches `docs/api/auth`
`alpha`	Letters only	String	`{slug:alpha}` matches `hello` but not `hello123`
`alphanumeric`	Letters and digits	String	`{code:alphanumeric}` matches `abc123`

The `{name}` form (no type) matches any single path segment and returns it as a string.

Typed Parameters in Action

Here is a complete example showing the most commonly used typed parameters together:

```
<?php
use Tina4\Router;

// Integer parameter -- only digits match, auto-cast to integer
Router::get("/products/{id:int}", function ($request, $response) {
    $id = $request->params["id"]; // integer, e.g. 42
    return $response->json([
        "product_id" => $id,
        "type" => gettype($id)
    ]);
});

// Float parameter -- decimal numbers, auto-cast to float
Router::get("/products/{id:int}/price/{price:float}", function ($request, $response) {
    $id = $request->params["id"];
    $price = $request->params["price"];
    return $response->json([
        "product_id" => $id,
        "price" => $price,
        "type" => gettype($price)
    ]);
});

// Path parameter -- catch-all, captures remaining segments as a string
Router::get("/files/{filepath:path}", function ($request, $response) {
    $filepath = $request->params["filepath"];
    // filepath could be "images/photos/cat.jpg"
    return $response->json([
        "filepath" => $filepath,
        "type" => gettype($filepath)
    ]);
});
```

```
# Integer route -- matches digits, returns an integer
curl http://localhost:7145/products/42
```

```
{"product_id":42,"type":"integer"}
```

```
# Integer route -- non-integer gives a 404
curl http://localhost:7145/products/abc
```

```
{"error":"Not found","path":"/products/abc","status":404}
```

```
# Path catch-all -- captures everything after /files/
curl http://localhost:7145/files/images/photos/cat.jpg
```

```
{"filepath":"images/photos/cat.jpg","type":"string"}
```

The `:int` and `:float` types act as both a constraint and a converter. If the URL segment does not match the expected pattern, the route is skipped entirely and Tina4 moves on to the next registered route (or returns 404 if nothing matches). The `:path` type is greedy -- it consumes all remaining segments, making it ideal for file paths and documentation URLs.

---

4. Query Parameters

Key-value pairs after the `?` in a URL. Access them through `$request->params`:

```
<?php
use Tina4\Router;

Router::get("/search", function ($request, $response) {
    $q = $request->params["q"] ?? "";
    $page = (int) ($request->params["page"] ?? 1);
    $limit = (int) ($request->params["limit"] ?? 10);

    return $response->json([
        "query" => $q,
        "page" => $page,
        "limit" => $limit,
        "offset" => ($page - 1) * $limit
    ]);
});
```

```
curl "http://localhost:7145/search?q=keyboard&page=2&limit=20"
```

```
{"query":"keyboard","page":2,"limit":20,"offset":20}
```

Missing query parameters do not exist in the array. Always use the null coalescing operator (`??`) for defaults.

---

5. Route Groups

Shared prefix. No repetition.

```
<?php
use Tina4\Router;

Router::group("/api/v1", function () {

    Router::get("/users", function ($request, $response) {
        return $response->json(["users" => []]);
    });

    Router::get("/users/{id:int}", function ($request, $response) {
        $id = $request->params["id"];
        return $response->json(["user" => ["id" => $id, "name" => "Alice"]]);
    });

    Router::post("/users", function ($request, $response) {
        return $response->json(["created" => true], 201);
    });

    Router::get("/products", function ($request, $response) {
        return $response->json(["products" => []]);
    });
});
```

These register as `/api/v1/users`, `/api/v1/users/{id}`, and `/api/v1/products`. Short paths inside the group. Tina4 prepends the prefix.

```
curl http://localhost:7145/api/v1/users

{"users":[]}

curl http://localhost:7145/api/v1/products

{"products":[]}
```

Groups nest:

```
<?php
use Tina4\Router;

Router::group("/api", function () {
    Router::group("/v1", function () {
        Router::get("/status", function ($request, $response) {
            return $response->json(["version" => "1.0"]);
        });
    });

    Router::group("/v2", function () {
        Router::get("/status", function ($request, $response) {
            return $response->json(["version" => "2.0"]);
        });
    });
});

curl http://localhost:7145/api/v1/status

{"version":"1.0"}

curl http://localhost:7145/api/v2/status

{"version":"2.0"}
```

---

6. Middleware

Code that runs before or after your handler. Authentication, logging, rate limiting, input validation -- anything that should apply to multiple routes without polluting each handler.

Middleware on a Single Route

Pass the middleware name as the third argument:

```
<?php
use Tina4\Router;

function logRequest($request, $response, $next) {
    $start = microtime(true);
    error_log("[" . date("Y-m-d H:i:s") . "] " . $request->method . " " . $request->path);

    $result = $next($request, $response);

    $duration = round((microtime(true) - $start) * 1000, 2);
    error_log("  Completed in " . $duration . "ms");

    return $result;
}
```

```
Router::get("/api/data", function ($request, $response) {
    return $response->json(["data" => [1, 2, 3]]);
}, "logRequest");
```

The middleware receives `$request`, `$response`, and `$next`. Call `$next($request, $response)` to continue to the handler. Skip the call and the handler never runs -- the chain stops cold.

Blocking Middleware

A gate that checks for an API key:

```
<?php
use Tina4\Router;

function requireApiKey($request, $response, $next) {
    $apiKey = $request->headers["X-API-Key"] ?? "";

    if ($apiKey !== "my-secret-key") {
        return $response->json(["error" => "Invalid API key"], 401);
    }

    return $next($request, $response);
}

Router::get("/api/secret", function ($request, $response) {
    return $response->json(["secret" => "The answer is 42"]);
}, "requireApiKey");
```

```
curl http://localhost:7145/api/secret
```

```
{"error":"Invalid API key"}
```

Status: `401 Unauthorized`.

```
curl http://localhost:7145/api/secret -H "X-API-Key: my-secret-key"
```

```
{"secret":"The answer is 42"}
```

Middleware on a Group

Third argument to `Router::group()`. Every route inside inherits it.

```
<?php
use Tina4\Router;

function requireAuth($request, $response, $next) {
    $token = $request->headers["Authorization"] ?? "";

    if (empty($token)) {
        return $response->json(["error" => "Authentication required"], 401);
    }

    return $next($request, $response);
}

Router::group("/api/admin", function () {

    Router::get("/dashboard", function ($request, $response) {
        return $response->json(["page" => "admin dashboard"]);
    });
```

```
    Router::get("/users", function ($request, $response) {
        return $response->json(["page" => "user management"]);
    });

}, "requireAuth");
```

No per-route repetition. The group handles it.

Multiple Middleware

Pass an array. They run in order.

```
Router::get("/api/important", function ($request, $response) {
    return $response->json(["data" => "important stuff"]);
}, ["logRequest", "requireApiKey", "requireAuth"]);
```

`logRequest` first, then `requireApiKey`, then `requireAuth`, then the handler. If any middleware skips `$next`, the chain stops there.

---

7. Route Decorators: @noauth and @secured

Two annotations for controlling authentication at the route level.

@noauth -- Public Routes

When your application has global authentication, `@noauth` exempts specific routes:

```
<?php
use Tina4\Router;

/**
 * @noauth
 */
Router::get("/api/public/info", function ($request, $response) {
    return $response->json([
        "app" => "My Store",
        "version" => "1.0.0"
    ]);
});
```

The `@noauth` decorator tells Tina4 to skip authentication for this route, even if global auth middleware is configured.

@secured -- Protected GET Routes

`@secured` marks a GET route as requiring authentication:

```
<?php
use Tina4\Router;

/**
 * @secured
 */
Router::get("/api/profile", function ($request, $response) {
    // $request->user is populated by the auth middleware
    return $response->json([
        "user" => $request->user
```

```
    ]);
});
```

The convention: `POST`, `PUT`, `PATCH`, and `DELETE` routes are secured by default. `GET` routes are public unless you add `@secured`. Reading is open. Writing requires proof.

---

8. Route Chaining: secure() and cache()

Routes return a chainable object. Two methods you can call on any route: `secure()` and `cache()`.

secure()

`secure()` requires a valid bearer token in the `Authorization` header. If the token is missing or invalid, the route returns `401 Unauthorized` without ever reaching your handler:

```
Router::get("/api/account", function ($request, $response) {
    return $response->json(["account" => $request->user]);
})->secure();
```

```
curl http://localhost:7145/api/account
# 401 Unauthorized

curl http://localhost:7145/api/account -H "Authorization: Bearer eyJhbGci..."
# 200 OK
```

This is a lighter alternative to `@secured` -- it works inline without a docblock annotation.

cache()

`cache()` enables response caching for the route. Once the handler runs and produces a response, subsequent requests to the same URL return the cached result without re-executing the handler:

```
Router::get("/api/catalog", function ($request, $response) {
    // Expensive database query
    return $response->json(["products" => $products]);
})->cache();
```

Chaining Both

Chain `secure()` and `cache()` together on the same route:

```
Router::get("/api/data", function ($request, $response) {
    return $response->json(["data" => $data]);
})->secure()->cache();
```

This route requires a bearer token and caches the response. Order does not matter -- `->cache()->secure()` produces the same result.

---

9. Wildcard and Catch-All Routes

Wildcard Routes

Use `{name:path}` at the end of a path to capture everything after it:

```
<?php
use Tina4\Router;

Router::get("/docs/{path:path}", function ($request, $response) {
    return $response->json([
        "section" => "docs",
        "path" => $request->params["path"]
    ]);
});
```

```
curl http://localhost:7145/docs/getting-started
```

```
{"section":"docs","path":"getting-started"}
```

```
curl http://localhost:7145/docs/api/authentication/jwt
```

```
{"section":"docs","path":"api/authentication/jwt"}
```

Catch-All Route (Custom 404)

Handle any unmatched URL:

```
<?php
use Tina4\Router;

Router::get("/{path:path}", function ($request, $response) {
    return $response->json([
        "error" => "Page not found",
        "path" => $request->params["path"]
    ], 404);
});
```

Define this last. Tina4 matches routes in registration order -- first match wins. Place this in a file that sorts alphabetically after your other route files, or it will shadow everything.

You can also create a custom 404 template at `src/templates/errors/404.twig`:

```
{% extends "base.html" %}

{% block title %}Not Found{% endblock %}

{% block content %}
    <h1>404 - Page Not Found</h1>
    <p>The page you are looking for does not exist.</p>
    <a href="/">Go back home</a>
{% endblock %}
```

Tina4 uses this template for any unmatched route when the file exists.

---

10. Route Listing via CLI

Your application grows. You need a map. The CLI provides one.

```
tina4 routes
```

```
Method   Path                          Middleware          Auth
------   ----                          ----------          ----
GET      /hello                        -                   public
GET      /products                     -                   public
POST     /products                     -                   secured
PUT      /products/{id}                -                   secured
PATCH    /products/{id}                -                   secured
DELETE   /products/{id}                -                   secured
GET      /api/v1/users                 -                   public
GET      /api/v1/users/{id:int}        -                   public
POST     /api/v1/users                 -                   secured
GET      /api/admin/dashboard          requireAuth         public
GET      /api/admin/users              requireAuth         public
GET      /api/public/info              -                   @noauth
GET      /api/profile                  -                   @secured
GET      /search                       -                   public
GET      /docs/*                       -                   public
```

Filter by method:

```
tina4 routes --method POST
```

```
Method   Path                          Middleware          Auth
------   ----                          ----------          ----
POST     /products                     -                   secured
POST     /api/v1/users                 -                   secured
```

Search for a path pattern:

```
tina4 routes --filter users
```

```
Method   Path                          Middleware          Auth
------   ----                          ----------          ----
GET      /api/v1/users                 -                   public
GET      /api/v1/users/{id:int}        -                   public
POST     /api/v1/users                 -                   secured
GET      /api/admin/users              requireAuth         public
```

---

11. Organizing Route Files

Tina4 loads every `.php` file in `src/routes/` recursively. The directory structure is yours to organize. Two common patterns:

Pattern 1: One File Per Resource

```
src/routes/
■■■ products.php     # All product routes
■■■ users.php        # All user routes
■■■ orders.php       # All order routes
■■■ pages.php        # HTML page routes
```

Pattern 2: Subdirectories by Feature

```
src/routes/
■■■ api/
■   ■■■ products.php
■   ■■■ users.php
■   ■■■ orders.php
■■■ admin/
■   ■■■ dashboard.php
■   ■■■ settings.php
■■■ pages/
    ■■■ home.php
    ■■■ about.php
```

Both work identically. The directory structure has no effect on URL paths -- only the route definitions inside the files matter. Pick whichever pattern keeps your project navigable.

---

12. Exercise: Build a Full CRUD API for Products

Build a REST API for managing products. All data stored in a PHP array. No database yet -- Chapter 5 handles that.

Requirements

Create `src/routes/product-api.php` with these routes:

Method	Path	Description
`GET`	`/api/products`	List all products. Support `?category=` filter.
`GET`	`/api/products/{id:int}`	Get a single product by ID. Return 404 if not found.
`POST`	`/api/products`	Create a new product. Return 201.
`PUT`	`/api/products/{id:int}`	Replace a product. Return 404 if not found.
`DELETE`	`/api/products/{id:int}`	Delete a product. Return 204 with no body.

Each product: `id` (int), `name` (string), `category` (string), `price` (float), `in_stock` (bool).

Seed data:

```
$products = [
    ["id" => 1, "name" => "Wireless Keyboard", "category" => "Electronics", "price" => 79.99, "i
    ["id" => 2, "name" => "Yoga Mat", "category" => "Fitness", "price" => 29.99, "in_stock" => t
    ["id" => 3, "name" => "Coffee Grinder", "category" => "Kitchen", "price" => 49.99, "in_stock
    ["id" => 4, "name" => "Standing Desk", "category" => "Office", "price" => 549.99, "in_stock"
    ["id" => 5, "name" => "Running Shoes", "category" => "Fitness", "price" => 119.99, "in_stock
];
```

Test with:

```
# List all
curl http://localhost:7145/api/products

# Filter by category
curl "http://localhost:7145/api/products?category=Fitness"

# Get one
curl http://localhost:7145/api/products/3

# Create
curl -X POST http://localhost:7145/api/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Desk Lamp", "category": "Office", "price": 39.99, "in_stock": true}'

# Update
curl -X PUT http://localhost:7145/api/products/3 \
  -H "Content-Type: application/json" \
  -d '{"name": "Burr Coffee Grinder", "category": "Kitchen", "price": 59.99, "in_stock": true}'

# Delete
curl -X DELETE http://localhost:7145/api/products/3

# Not found
curl http://localhost:7145/api/products/999
```

---

13. Solution

Create `src/routes/product-api.php`:

```
<?php
use Tina4\Router;

// In-memory product store (resets on server restart)
$products = [
    ["id" => 1, "name" => "Wireless Keyboard", "category" => "Electronics", "price" => 79.99, "i
    ["id" => 2, "name" => "Yoga Mat", "category" => "Fitness", "price" => 29.99, "in_stock" => t
    ["id" => 3, "name" => "Coffee Grinder", "category" => "Kitchen", "price" => 49.99, "in_stock
    ["id" => 4, "name" => "Standing Desk", "category" => "Office", "price" => 549.99, "in_stock"
    ["id" => 5, "name" => "Running Shoes", "category" => "Fitness", "price" => 119.99, "in_stock
];

$nextId = 6;

// List all products, optionally filter by category
Router::get("/api/products", function ($request, $response) use (&$products) {
    $category = $request->params["category"] ?? null;

    if ($category !== null) {
        $filtered = array_values(array_filter(
            $products,
            fn($p) => strtolower($p["category"]) === strtolower($category)
        ));
        return $response->json(["products" => $filtered, "count" => count($filtered)]);
    }
```

```
    return $response->json(["products" => $products, "count" => count($products)]);
});

// Get a single product by ID
Router::get("/api/products/{id:int}", function ($request, $response) use (&$products) {
    $id = $request->params["id"];

    foreach ($products as $product) {
        if ($product["id"] === $id) {
            return $response->json($product);
        }
    }

    return $response->json(["error" => "Product not found", "id" => $id], 404);
});

// Create a new product
Router::post("/api/products", function ($request, $response) use (&$products, &$nextId) {
    $body = $request->body;

    if (empty($body["name"])) {
        return $response->json(["error" => "Name is required"], 400);
    }

    $product = [
        "id" => $nextId++,
        "name" => $body["name"],
        "category" => $body["category"] ?? "Uncategorized",
        "price" => (float) ($body["price"] ?? 0),
        "in_stock" => (bool) ($body["in_stock"] ?? true)
    ];

    $products[] = $product;

    return $response->json($product, 201);
});

// Replace a product
Router::put("/api/products/{id:int}", function ($request, $response) use (&$products) {
    $id = $request->params["id"];
    $body = $request->body;

    foreach ($products as $index => $product) {
        if ($product["id"] === $id) {
            $products[$index] = [
                "id" => $id,
                "name" => $body["name"] ?? $product["name"],
                "category" => $body["category"] ?? $product["category"],
                "price" => (float) ($body["price"] ?? $product["price"]),
                "in_stock" => (bool) ($body["in_stock"] ?? $product["in_stock"])
            ];
            return $response->json($products[$index]);
        }
    }

    return $response->json(["error" => "Product not found", "id" => $id], 404);
});

// Delete a product
```

```
Router::delete("/api/products/{id:int}", function ($request, $response) use (&$products) {
    $id = $request->params["id"];

    foreach ($products as $index => $product) {
        if ($product["id"] === $id) {
            array_splice($products, $index, 1);
            return $response->json(null, 204);
        }
    }

    return $response->json(["error" => "Product not found", "id" => $id], 404);
});
```

Expected output for the test commands:

List all:

```
{"products":[{"id":1,"name":"Wireless Keyboard","category":"Electronics","price":79.99,"in_stock
```

Filter by category:

```
{"products":[{"id":2,"name":"Yoga Mat","category":"Fitness","price":29.99,"in_stock":true},{"id"
```

Get one:

```
{"id":3,"name":"Coffee Grinder","category":"Kitchen","price":49.99,"in_stock":false}
```

Create:

```
{"id":6,"name":"Desk Lamp","category":"Office","price":39.99,"in_stock":true}
```

(Status: `201 Created`)

Update:

```
{"id":3,"name":"Burr Coffee Grinder","category":"Kitchen","price":59.99,"in_stock":true}
```

Delete: empty response with status `204 No Content`.

Not found:

```
{"error":"Product not found","id":999}
```

(Status: `404 Not Found`)

---

14. Gotchas

1. Trailing Slashes Are Normalised

Tina4 strips trailing slashes automatically. Both `/products` and `/products/` match the same route. You do not need to register both or configure anything; it just works.

2. Parameter Names Must Be Unique in a Path

Problem: `/users/{id}/posts/{id}` produces wrong results -- the second `{id}` overwrites the first.

Fix: Use distinct names: `/users/{userId}/posts/{postId}`.

3. Method Conflicts

Problem: `Router::get("/items/{id}", ...)` and `Router::get("/items/{action}", ...)` collide. The wrong handler runs.

Cause: Both patterns match `/items/42`. First registration wins.

Fix: Use typed parameters to disambiguate: `Router::get("/items/{id:int}", ...)` matches only integers, leaving `/items/export` free. Or restructure: `/items/{id:int}` and `/items/actions/{action}`.

4. Route Handler Must Return a Response

Problem: The handler runs but the browser shows empty or 500.

Cause: No `return` statement. Without `return`, the handler returns `null`. Tina4 has nothing to send.

Fix: Every handler must return something. `return $response->json(...)`, `return $response->html(...)`, `return $response->render(...)`.

5. Route Files Must Start with `<?php`

Problem: Route file is ignored. No errors. No routes registered.

Cause: Missing PHP opening tag. Without `<?php`, the file is not parsed.

Fix: First line of every route file: `<?php`.

6. Middleware Uses Class-Based Pattern

Problem: Passing a function name string as middleware doesn't work.

Fix: Use class-based middleware with `before*`/`after*` static methods. See Chapter 10 for the full middleware pattern:

```
class AuthMiddleware {
    public static function beforeAuth($request, $response) {
        if (!$request->headers['authorization']) {
            return [$request, $response->json(["error" => "Unauthorized"], 401)];
        }
        return [$request, $response];
    }
}
Router::use(AuthMiddleware::class);
```

7. Group Prefix Normalisation

Tina4 auto-normalises group prefixes: it prepends `/` if missing and strips trailing slashes. `Router::group("api/v1", ...)` and `Router::group("/api/v1", ...)` both work. However, for clarity, always start with `/`.

Request & Response

1. The Two Objects You Always Get

Every route handler receives two arguments. `$request` carries what the client sent. `$response` builds what goes back. These two objects are the entire HTTP conversation.

```
<?php
use Tina4\Router;

Router::get("/echo", function ($request, $response) {
    return $response->json([
        "method" => $request->method,
        "path" => $request->path,
        "your_ip" => $request->ip
    ]);
});
```

```
curl http://localhost:7145/echo
```

```
{"method":"GET","path":"/echo","your_ip":"127.0.0.1"}
```

The pattern never changes. Read the request. Build the response. Return it.

---

2. The Request Object

The `$request` object holds everything the client sent. Here is the full inventory.

method

The HTTP method as an uppercase string: `"GET"`, `"POST"`, `"PUT"`, `"PATCH"`, or `"DELETE"`.

```
$request->method // "GET"
```

path

The URL path, stripped of query parameters:

```
// Request to /api/users?page=2
$request->path // "/api/users"
```

params

Path parameters captured from the URL pattern (see Chapter 2):

```
// Route: /users/{id}/posts/{postId}
// Request: /users/5/posts/42
$request->params["id"]     // "5" (or 5 if typed as {id:int})
$request->params["postId"] // "42"
```

query

Query string parameters. An associative array:

```
// Request: /search?q=keyboard&page=2&sort=price
$request->query["q"]    // "keyboard"
```

```
$request->query["page"] // "2"
$request->query["sort"] // "price"
```

The `queryParam()` helper provides a default value when a key is missing:

```
$request->queryParam("q")              // "keyboard"
$request->queryParam("missing", "N/A") // "N/A"
```

body

The parsed request body. JSON requests become associative arrays. Form submissions contain form fields:

```
// POST with {"name": "Widget", "price": 9.99}
$request->body["name"]  // "Widget"
$request->body["price"] // 9.99
```

headers

Request headers as an associative array. Keys are normalised to **lowercase**:

```
$request->headers["content-type"]  // "application/json"
$request->headers["authorization"] // "Bearer eyJhbGci..."
$request->headers["x-custom"]      // "my-value"
```

The `header()` helper method is the preferred way to read a single header. It accepts any casing and handles the lowercase lookup for you:

```
$request->header("Content-Type")  // "application/json"
$request->header("Authorization") // "Bearer eyJhbGci..."
$request->header("X-Custom")      // "my-value"
$request->header("Missing")       // null
```

ip

The client's IP address:

```
$request->ip // "127.0.0.1"
```

Tina4 respects `X-Forwarded-For` and `X-Real-IP` headers behind a reverse proxy.

cookies

Cookies the client sent:

```
$request->cookies["session_id"]  // "abc123"
$request->cookies["preferences"] // "dark-mode"
```

files

Uploaded files (section 7 covers this in detail):

```
$request->files["avatar"] // ["filename", "type", "size", "content"] - content is raw bytes
```

Inspecting the Full Request

A diagnostic route that dumps everything:

```
<?php
use Tina4\Router;

Router::post("/debug/request", function ($request, $response) {
```

```
    return $response->json([
        "method" => $request->method,
        "path" => $request->path,
        "params" => $request->params,
        "query" => $request->query,
        "body" => $request->body,
        "headers" => $request->headers,
        "ip" => $request->ip,
        "cookies" => $request->cookies
    ]);
});
```

```
curl -X POST "http://localhost:7145/debug/request?page=1" \
  -H "Content-Type: application/json" \
  -H "X-Custom: hello" \
  -d '{"name": "test"}'
```

```
{
  "method": "POST",
  "path": "/debug/request",
  "params": {},
  "query": {"page": "1"},
  "body": {"name": "test"},
  "headers": {
    "content-type": "application/json",
    "x-custom": "hello",
    "host": "localhost:7145",
    "user-agent": "curl/8.4.0",
    "accept": "*/*",
    "content-length": "16"
  },
  "ip": "127.0.0.1",
  "cookies": {}
}
```

---

3. The Response Object

The `$response` object shapes what travels back to the client. Every method returns the response object, so calls chain.

json() -- JSON Response

The API workhorse. Pass an array. It becomes JSON:

```
return $response->json(["name" => "Alice", "age" => 30]);
```

```
{"name":"Alice","age":30}
```

Second argument sets the status code:

```
return $response->json(["id" => 7, "name" => "Widget"], 201);
```

`201 Created` with the JSON body.

html() -- Raw HTML Response

Return raw HTML:

```
return $response->html("<h1>Hello</h1><p>This is HTML.</p>");
```

Sets `Content-Type: text/html; charset=utf-8`.

text() -- Plain Text Response

```
return $response->text("Just a plain string.");
```

Sets `Content-Type: text/plain; charset=utf-8`.

render() -- Template Response

Render a Frond template with data (Chapter 4 goes deep):

```
return $response->render("products.html", [
    "products" => $products,
    "title" => "Our Products"
]);
```

Tina4 finds the template in `src/templates/`, renders it, returns the HTML.

redirect() -- Redirect Response

Send the client somewhere else:

```
return $response->redirect("/login");
```

`302 Found` by default. For permanent redirects:

```
return $response->redirect("/new-location", 301);
```

file() -- File Download Response

Push a file to the client:

```
return $response->file("/path/to/report.pdf");
```

Tina4 sets the correct `Content-Type` from the file extension and adds `Content-Disposition` so the browser downloads instead of displaying.

Custom filename:

```
return $response->file("/path/to/report.pdf", "monthly-report-march-2026.pdf");
```

---

4. Status Codes

Every response method accepts a status code. Here are the ones you reach for most:

Code	Meaning	When to Use
`200`	OK	Default. Successful GET, PUT, PATCH.
`201`	Created	Successful POST that created a resource.

204	No Content	Successful DELETE. No body needed.
301	Moved Permanently	URL has permanently changed.
302	Found	Temporary redirect.
400	Bad Request	Invalid input from the client.
401	Unauthorized	Missing or invalid authentication.
403	Forbidden	Authenticated but not allowed.
404	Not Found	Resource does not exist.
409	Conflict	Duplicate or conflicting data.
413	Payload Too Large	Request body or file exceeds the size limit.
422	Unprocessable Entity	Valid JSON but fails business rules.
500	Internal Server Error	Something broke on the server.

You can also chain `status()` explicitly:

```
return $response->status(201)->json(["id" => 7, "created" => true]);
```

Same result as `$response->json(["id" => 7, "created" => true], 201)`. Pick whichever reads cleaner in context.

HTTP Status Constants

Tina4 exports named constants for every status code. Use them instead of raw integers -- the intent reads at a glance and you avoid typos like `419` when you meant `401`:

```
use Tina4\Router;

Router::get("/api/widgets/{id}", function ($request, $response) {
    $widget = Widget::load(["id" => $request->params["id"]]);

    if (!$widget->exists()) {
        return $response->json(["error" => "Widget not found"], \Tina4\HTTP_NOT_FOUND);
    }

    return $response->json($widget->asArray(), \Tina4\HTTP_OK);
});
```

Available constants (all in the `\Tina4` namespace):

Status	Constant	Status	Constant

200	HTTP_OK	404	HTTP_NOT_FOUND
201	HTTP_CREATED	405	HTTP_METHOD_NOT_ALLOWED
202	HTTP_ACCEPTED	409	HTTP_CONFLICT
204	HTTP_NO_CONTENT	410	HTTP_GONE
301	HTTP_MOVED_PERMANENTLY	422	HTTP_UNPROCESSABLE
302	HTTP_FOUND	429	HTTP_TOO_MANY
304	HTTP_NOT_MODIFIED	500	HTTP_INTERNAL_SERVER_ERROR
400	HTTP_BAD_REQUEST	502	HTTP_BAD_GATEWAY
401	HTTP_UNAUTHORIZED	503	HTTP_SERVICE_UNAVAILABLE
403	HTTP_FORBIDDEN	504	HTTP_GATEWAY_TIMEOUT

Content-Type Constants

For non-JSON responses, pair a status constant with a content-type constant. Tina4 also supports a three-argument response form that sets body, status, and content type in one call:

```
use Tina4\Router;

Router::get("/api/legacy-feed", function ($request, $response) {
    $xml = "<feed><item>...</item></feed>";
    return $response($xml, \Tina4\HTTP_OK, \Tina4\APPLICATION_XML);
})->noAuth();
```

Content-type constants:

Constant	Value
APPLICATION_JSON	application/json
APPLICATION_XML	application/xml
APPLICATION_FORM	application/x-www-form-urlencoded
APPLICATION_OCTET_STREAM	application/octet-stream
TEXT_HTML	text/html
TEXT_PLAIN	text/plain
TEXT_CSV	text/csv
TEXT_XML	text/xml

Using constants also makes your code grep-friendly -- searching for `HTTP_BAD_REQUEST` across the codebase finds every validation branch, where searching for `400` finds false positives (file sizes, timeouts, anything with the number 400 in it).

---

5. Custom Headers

Set response headers with `header()`:

```
Router::get("/api/data", function ($request, $response) {
    return $response
        ->header("X-Request-Id", uniqid())
        ->header("X-Rate-Limit-Remaining", "57")
        ->header("Cache-Control", "no-cache")
        ->json(["data" => [1, 2, 3]]);
});
```

```
curl -v http://localhost:7145/api/data 2>&1 | grep "< X-"
```

```
< X-Request-Id: 65f3a7b8c1234
< X-Rate-Limit-Remaining: 57
```

Convention: `Title-Case` for custom headers. Prefix with `X-`.

CORS Headers

Tina4 handles CORS from `TINA4_CORS_ORIGINS` in `.env`, once `CorsMiddleware` is registered (Chapter 10). By default it is empty, which denies every cross-origin request. List the origins your front end is served from:

```
TINA4_CORS_ORIGINS=https://myapp.com,https://admin.myapp.com
```

Manual CORS headers are rarely needed. They are available when you need them:

```
return $response
    ->header("Access-Control-Allow-Origin", "https://myapp.com")
    ->header("Access-Control-Allow-Methods", "GET, POST, PUT, DELETE")
    ->header("Access-Control-Allow-Headers", "Content-Type, Authorization")
    ->json(["data" => "value"]);
```

---

6. Cookies

Set cookies on the response:

```
Router::post("/login", function ($request, $response) {
    // After validating credentials...
    return $response
        ->cookie("session_id", "abc123xyz", [
            "httpOnly" => true,
            "secure" => true,
            "sameSite" => "Strict",
            "maxAge" => 3600,       // 1 hour in seconds
            "path" => "/"
        ])
        ->json(["message" => "Logged in"]);
```

```
});
```

Cookie options:

Option	Type	Default	Description
`httpOnly`	bool	`false`	Invisible to JavaScript
`secure`	bool	`false`	HTTPS only
`sameSite`	string	`"Lax"`	`"Strict"`, `"Lax"`, or `"None"`
`maxAge`	int	session	Lifetime in seconds
`path`	string	`"/"`	URL path scope
`domain`	string	current	Domain scope

Read cookies from the request:

```
Router::get("/profile", function ($request, $response) {
    $sessionId = $request->cookies["session_id"] ?? null;

    if ($sessionId === null) {
        return $response->json(["error" => "Not logged in"], 401);
    }

    return $response->json(["session" => $sessionId]);
});
```

Delete a cookie by setting `maxAge` to `0`:

```
return $response
    ->cookie("session_id", "", ["maxAge" => 0, "path" => "/"])
    ->json(["message" => "Logged out"]);
```

---

7. File Uploads

Uploaded files land in `$request->files` as an associative array keyed by field name. Each entry has the file metadata plus the raw bytes; Tina4 does not write a temporary file to disk, so there is no `tmp_path`.

```
$file = $request->files["avatar"];
// [
//   "fieldName" => "avatar",
//   "filename"  => "photo.png",
//   "type"      => "image/png",
//   "content"   => "<raw bytes>",   // binary, NOT base64
//   "size"      => 102400,
// ]
```

Handling a Single File Upload

```
<?php
use Tina4\Router;

Router::post("/api/upload", function ($request, $response) {
    if (empty($request->files["image"])) {
        return $response->json(["error" => "No file uploaded"], 400);
    }

    $file = $request->files["image"];

    return $response->json([
        "name" => $file["filename"],   // "photo.jpg"
        "type" => $file["type"],       // "image/jpeg"
        "size" => $file["size"],       // 245760 (bytes)
    ]);
});
```

```
curl -X POST http://localhost:7145/api/upload \
  -F "image=@/path/to/photo.jpg"
```

```
{
  "name": "photo.jpg",
  "type": "image/jpeg",
  "size": 245760
}
```

Saving the Uploaded File

The file sits in a temporary location. Move it to permanent storage:

```
<?php
use Tina4\Router;

Router::post("/api/upload", function ($request, $response) {
    if (empty($request->files["image"])) {
        return $response->json(["error" => "No file uploaded"], 400);
    }

    $file = $request->files["image"];

    // Validate file type
    $allowedTypes = ["image/jpeg", "image/png", "image/gif", "image/webp"];
    if (!in_array($file["type"], $allowedTypes)) {
        return $response->json(["error" => "Invalid file type. Allowed: JPEG, PNG, GIF, WebP"],
    }

    // Validate file size (max 5MB)
    $maxSize = 5 * 1024 * 1024;
    if ($file["size"] > $maxSize) {
        return $response->json(["error" => "File too large. Maximum size: 5MB"], 400);
    }

    // Generate a unique filename
    $extension = pathinfo($file["filename"], PATHINFO_EXTENSION);
    $filename = uniqid("img_") . "." . $extension;
    $destination = __DIR__ . "/../../public/uploads/" . $filename;

    // Ensure the uploads directory exists
```

```
    if (!is_dir(dirname($destination))) {
        mkdir(dirname($destination), 0755, true);
    }

    // Save the uploaded bytes - Tina4 stores them in $file["content"], not a temp file
    file_put_contents($destination, $file["content"]);

    return $response->json([
        "message" => "File uploaded successfully",
        "filename" => $filename,
        "url" => "/uploads/" . $filename,
        "size" => $file["size"]
    ], 201);
});
```

```
curl -X POST http://localhost:7145/api/upload \
  -F "image=@/path/to/photo.jpg"
```

```
{
  "message": "File uploaded successfully",
  "filename": "img_65f3a7b8c1234.jpg",
  "url": "/uploads/img_65f3a7b8c1234.jpg",
  "size": 245760
}
```

The file is now available at `http://localhost:7145/uploads/img_65f3a7b8c1234.jpg`.

Handling Multiple Files

When the form uses `multiple` or has multiple file inputs:

```
Router::post("/api/upload-many", function ($request, $response) {
    $results = [];

    foreach ($request->files as $key => $file) {
        $extension = pathinfo($file["filename"], PATHINFO_EXTENSION);
        $filename = uniqid("file_") . "." . $extension;
        $destination = __DIR__ . "/../../public/uploads/" . $filename;

        if (!is_dir(dirname($destination))) {
            mkdir(dirname($destination), 0755, true);
        }

        file_put_contents($destination, $file["content"]);

        $results[] = [
            "original_name" => $file["filename"],
            "saved_as" => $filename,
            "url" => "/uploads/" . $filename
        ];
    }

    return $response->json(["uploaded" => $results, "count" => count($results)], 201);
});
```

---

8. File Downloads

Send files to the client with `$response->file()`:

```
<?php
use Tina4\Router;

Router::get("/api/reports/{filename}", function ($request, $response) {
    $filename = $request->params["filename"];
    $filepath = __DIR__ . "/../../data/reports/" . $filename;

    if (!file_exists($filepath)) {
        return $response->json(["error" => "Report not found"], 404);
    }

    return $response->file($filepath);
});
```

The browser downloads the file. Tina4 detects the MIME type from the extension and sets headers accordingly.

Force a specific download filename:

```
return $response->file($filepath, "Q1-2026-Sales-Report.pdf");
```

---

9. Content Negotiation

One endpoint serves multiple formats. The `Accept` header decides which one:

```
<?php
use Tina4\Router;

Router::get("/api/products/{id:int}", function ($request, $response) {
    $id = $request->params["id"];
    $product = [
        "id" => $id,
        "name" => "Wireless Keyboard",
        "price" => 79.99
    ];

    $accept = $request->header("Accept") ?? "application/json";

    if (strpos($accept, "text/html") !== false) {
        return $response->render("product-detail.html", ["product" => $product]);
    }

    if (strpos($accept, "text/plain") !== false) {
        $text = "Product #" . $id . ": " . $product["name"] . " - $" . $product["price"];
        return $response->text($text);
    }

    // Default: JSON
    return $response->json($product);
});
```

```
# JSON (default)
curl http://localhost:7145/api/products/1
```

```
{"id":1,"name":"Wireless Keyboard","price":79.99}

# Plain text
curl http://localhost:7145/api/products/1 -H "Accept: text/plain"

Product #1: Wireless Keyboard - $79.99

# HTML (renders the template)
curl http://localhost:7145/api/products/1 -H "Accept: text/html"

<!DOCTYPE html>
<html>...rendered template...</html>
```

---

10. Input Validation

Tina4 ships a `Validator` class for declarative input validation. Chain rules. Check the result. When validation fails, `$response->error()` returns a structured error envelope.

The Validator Class

```
use Tina4\Validator;

Router::post("/api/users", function ($request, $response) {
    $v = new Validator($request->body);
    $v->required("name")->required("email")->email("email")->minLength("name", 2);

    if (!$v->isValid()) {
        return $response->error("VALIDATION_FAILED", $v->errors()[0]["message"], 400);
    }

    // proceed with valid data
});
```

The `Validator` takes the request body (an associative array) and exposes chainable methods:

Method	Description
`required(field)`	Field must be present and non-empty
`email(field)`	Field must be a valid email address
`minLength(field, n)`	Field must have at least `n` characters
`maxLength(field, n)`	Field must have at most `n` characters
`numeric(field)`	Field must be a number
`inList(field, values)`	Field must be one of the allowed values

Call `$v->isValid()` to check all rules. Call `$v->errors()` to get the list of failures, each with a `field` and `message` key.

The Error Response Envelope

`$response->error()` returns a consistent JSON error envelope:

```
return $response->error("VALIDATION_FAILED", "Name is required", 400);
```

This produces:

```
{"error": true, "code": "VALIDATION_FAILED", "message": "Name is required", "status": 400}
```

Three arguments: an error code string, a human-readable message, and the HTTP status code. Use this pattern across your API. Consistent error shapes save frontend developers hours of guesswork.

Upload Size Limits

Tina4 enforces a maximum upload size through the `TINA4_MAX_UPLOAD_SIZE` environment variable. The value is in bytes. Default: `10485760` (10 MB).

```
TINA4_MAX_UPLOAD_SIZE=10485760
```

A file exceeding this limit triggers a `413 Payload Too Large` response before your handler runs. To allow larger uploads, increase the value in `.env`:

```
TINA4_MAX_UPLOAD_SIZE=52428800
```

This sets the limit to 50 MB.

---

11. Exercise: Build an Image Upload API

Two endpoints. One accepts images. The other serves them back.

Requirements

Method	Path	Description
`POST`	`/api/images`	Upload an image. Validate type and size. Return the image URL.
`GET`	`/api/images/{filename}`	Return the uploaded image file. 404 if not found.

Rules:

- Accept JPEG, PNG, and WebP only
- Maximum file size: 2MB
- Save to `src/public/uploads/` with a unique filename
- Return original filename, saved filename, file size in KB, and URL
- The GET endpoint serves the raw file, not JSON

Test with:

```
# Upload
curl -X POST http://localhost:7145/api/images \
  -F "image=@/path/to/photo.jpg"

# Download
curl http://localhost:7145/api/images/img_65f3a7b8c1234.jpg --output downloaded.jpg
```

---

12. Solution

Create `src/routes/images.php`:

```
<?php
use Tina4\Router;

Router::post("/api/images", function ($request, $response) {
    // Check if a file was uploaded
    if (empty($request->files["image"])) {
        return $response->json(["error" => "No image file provided. Use field name 'image'."], 4
    }

    $file = $request->files["image"];

    // Validate file type
    $allowedTypes = ["image/jpeg", "image/png", "image/webp"];
    if (!in_array($file["type"], $allowedTypes)) {
        return $response->json([
            "error" => "Invalid file type",
            "received" => $file["type"],
            "allowed" => $allowedTypes
        ], 400);
    }

    // Validate file size (max 2MB)
    $maxSize = 2 * 1024 * 1024;
    if ($file["size"] > $maxSize) {
        return $response->json([
            "error" => "File too large",
            "size_bytes" => $file["size"],
            "max_bytes" => $maxSize
        ], 400);
    }

    // Generate unique filename preserving extension
    $extension = pathinfo($file["filename"], PATHINFO_EXTENSION);
    $savedName = uniqid("img_") . "." . strtolower($extension);
    $uploadDir = __DIR__ . "/../../public/uploads";
    $destination = $uploadDir . "/" . $savedName;

    // Create uploads directory if it does not exist
    if (!is_dir($uploadDir)) {
        mkdir($uploadDir, 0755, true);
    }

    // Save the uploaded bytes - Tina4 stores them in $file["content"], not a temp file
    file_put_contents($destination, $file["content"]);
```

```
    return $response->json([
        "message" => "Image uploaded successfully",
        "original_name" => $file["filename"],
        "saved_name" => $savedName,
        "size_kb" => round($file["size"] / 1024, 1),
        "type" => $file["type"],
        "url" => "/uploads/" . $savedName
    ], 201);
});

Router::get("/api/images/{filename}", function ($request, $response) {
    $filename = $request->params["filename"];

    // Prevent directory traversal
    if (strpos($filename, "..") !== false || strpos($filename, "/") !== false) {
        return $response->json(["error" => "Invalid filename"], 400);
    }

    $filepath = __DIR__ . "/../../public/uploads/" . $filename;

    if (!file_exists($filepath)) {
        return $response->json(["error" => "Image not found", "filename" => $filename], 404);
    }

    return $response->file($filepath);
});
```

Expected output for upload:

```
{
  "message": "Image uploaded successfully",
  "original_name": "photo.jpg",
  "saved_name": "img_65f3a7b8c1234.jpg",
  "size_kb": 240.0,
  "type": "image/jpeg",
  "url": "/uploads/img_65f3a7b8c1234.jpg"
}
```

(Status: `201 Created`)

Expected output for invalid type:

```
{
  "error": "Invalid file type",
  "received": "application/pdf",
  "allowed": ["image/jpeg", "image/png", "image/webp"]
}
```

(Status: `400 Bad Request`)

Expected output for file too large:

```
{
  "error": "File too large",
  "size_bytes": 5242880,
  "max_bytes": 2097152
}
```

(Status: `400 Bad Request`)

The GET endpoint returns the raw image file with the correct `Content-Type` header. The browser renders it. Curl with `--output` saves it to disk.

---

13. Gotchas

1. Forgetting `return`

Problem: The handler runs (log output confirms it) but the browser gets an empty response or 500.

Cause: `$response->json([...])` without `return`. The response object builds the reply but nobody sends it.

Fix: `return $response->json([...])`. Always.

2. Body Is Null for JSON Requests

Problem: `$request->body` is `null` or empty despite sending JSON.

Cause: Missing `Content-Type: application/json` header. Without it, Tina4 does not parse the body as JSON.

Fix: Include `-H "Content-Type: application/json"` with curl. In JavaScript `fetch()`, set `headers: {"Content-Type": "application/json"}`.

3. Content-Type Mismatch

Problem: `$response->json()` returns HTML, or `$response->html()` returns plain text.

Cause: A middleware or error handler overwrites the response. Or you returned a string instead of using a response method.

Fix: Use `$response->json(...)`, `$response->html(...)`, or another response method. Never `echo` -- it bypasses the response object.

4. File Uploads Return Empty

Problem: `$request->files` is empty despite uploading a file.

Cause: The form lacks `enctype="multipart/form-data"`, or curl uses `-d` instead of `-F`.

Fix: HTML forms: `<form enctype="multipart/form-data">`. Curl: `-F "field=@file.jpg"` (with `@`), not `-d`.

5. Redirect Loops

Problem: The browser shows "too many redirects."

Cause: Route A redirects to Route B. Route B redirects back to Route A. Common with login guards: `/login` redirects to `/dashboard`, `/dashboard` redirects to `/login` because the user is not authenticated.

Fix: Trace the redirect chain in your browser's network inspector. Make sure the auth check does not redirect authenticated users away from pages they should access.

6. Cookie Not Set

Problem: `$response->cookie(...)` runs but the browser shows no cookie.

Cause: `secure` is `true`. The cookie only travels over HTTPS. Local development uses `http://localhost`. The cookie is silently dropped.

Fix: Set `"secure" => false` during development. Or use `"secure" => ($_ENV["TINA4_DEBUG"] ?? "false") !== "true"` to auto-switch.

7. Large Request Body Rejected

Problem: POST requests with large bodies return 413.

Cause: Request body exceeds the configured maximum.

Fix: Increase `TINA4_MAX_UPLOAD_SIZE` in `.env`. Default is `10mb`. For file upload endpoints, you may need `50mb` or more:

```
TINA4_MAX_UPLOAD_SIZE=50mb
```

Templates

1. Why Templates

In Chapter 1, `$response->render("products.html", $data)` produced a full HTML page. **Frond** did the rendering -- Tina4's built-in template engine. Zero dependencies. Twig-compatible syntax. If you know Twig, Jinja2, or Nunjucks, you know 90% of Frond.

Templates live in `src/templates/`. Call `$response->render("page.html", $data)`. Frond loads the file, processes tags and expressions, returns final HTML.

This chapter covers every feature. After reading it, you build real pages.

---

2. Variables and Expressions

Double curly braces output a variable:

```
<h1>Hello, {{ name }}!</h1>
```

Route handler:

```
<?php
use Tina4\Router;

Router::get("/welcome", function ($request, $response) {
    return $response->render("welcome.html", [
        "name" => "Alice"
    ]);
});
```

Create `src/templates/welcome.html`:

```
<!DOCTYPE html>
<html>
<head><title>Welcome</title></head>
<body>
    <h1>Hello, {{ name }}!</h1>
</body>
</html>
```

Output:

```
Hello, Alice!
```

Accessing Nested Data

Dot notation reaches into nested arrays:

```
$data = [
    "user" => [
        "name" => "Alice",
        "email" => "alice@example.com",
        "address" => [
            "city" => "Cape Town",
            "country" => "South Africa"
```

```
        ]
    ]
];

return $response->render("profile.html", $data);
```

```
<p>{{ user.name }} lives in {{ user.address.city }}, {{ user.address.country }}.</p>
```

Output:

```
Alice lives in Cape Town, South Africa.
```

Method Calls on Values

When a variable is an object or an array containing a callable, you can call methods directly in dot notation:

```
<p>{{ user.getName() }}</p>
<p>{{ translator.t("welcome_message") }}</p>
<p>{{ cart.total() }}</p>
```

Arguments are passed as normal function arguments. This works on both objects (calls the method) and arrays (calls the callable stored at that key).

Expressions

Basic arithmetic and string concatenation work inside `{{ }}`:

```
<p>Total: ${{ price * quantity }}</p>
<p>Discounted: ${{ price * 0.9 }}</p>
<p>Full name: {{ first_name ~ " " ~ last_name }}</p>
```

The `~` operator concatenates strings.

---

3. Template Inheritance

The headline feature. Define a base layout once. Extend it everywhere.

Base Layout

Create `src/templates/base.twig`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}My App{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    {% block head %}{% endblock %}
</head>
<body>
    <nav>
        <a href="/">Home</a>
        <a href="/about">About</a>
        <a href="/contact">Contact</a>
    </nav>
```

```
    <main>
        {% block content %}{% endblock %}
    </main>

    <footer>
        <p>&copy; 2026 My App. All rights reserved.</p>
    </footer>

    <script src="/js/frond.js"></script>
    {% block scripts %}{% endblock %}
</body>
</html>
```

Four blocks: `title`, `head`, `content`, `scripts`. Child templates override only what they need.

Child Template

Create `src/templates/about.twig`:

```
{% extends "base.twig" %}

{% block title %}About Us{% endblock %}

{% block content %}
    <h1>About Us</h1>
    <p>We have been building things since {{ founded_year }}.</p>
    <p>Our team has {{ team_size }} members across {{ office_count }} offices.</p>
{% endblock %}
```

Route handler:

```
<?php
use Tina4\Router;

Router::get("/about", function ($request, $response) {
    return $response->render("about.twig", [
        "founded_year" => 2020,
        "team_size" => 12,
        "office_count" => 3
    ]);
});
```

Result: A full HTML page with the nav, the "About Us" content, and the footer. The `<title>` reads "About Us". The `head` and `scripts` blocks stay empty because the child did not override them.

Using `{{ parent() }}`

Add to a block instead of replacing it:

```
{% extends "base.twig" %}

{% block head %}
    {{ parent() }}
    <link rel="stylesheet" href="/css/contact-form.css">
{% endblock %}

{% block content %}
    <h1>Contact Us</h1>
    <form>...</form>
```

```
{% endblock %}
```

The `head` block now contains everything from the base plus the extra stylesheet.

---

4. Includes

Break templates into reusable pieces with `{% include %}`:

Create `src/templates/partials/header.twig`:

```
<header>
    <div class="logo">{{ site_name | default("My App") }}</div>
    <nav>
        <a href="/">Home</a>
        <a href="/products">Products</a>
        <a href="/contact">Contact</a>
    </nav>
</header>
```

Create `src/templates/partials/product-card.twig`:

```
<div class="product-card{{ product.featured ? ' featured' : '' }}">
    <h3>{{ product.name }}</h3>
    <p class="price">${{ product.price | number_format(2) }}</p>
    {% if product.in_stock %}
        <span class="badge-success">In Stock</span>
    {% else %}
        <span class="badge-danger">Out of Stock</span>
    {% endif %}
</div>
```

Use them in a page:

```
{% extends "base.twig" %}

{% block content %}
    {% include "partials/header.twig" %}

    <h1>Products</h1>
    {% for product in products %}
        {% include "partials/product-card.twig" %}
    {% endfor %}
{% endblock %}
```

The `product` variable is available inside the included template because it exists in the current scope -- the for loop.

Passing Variables to Includes

Explicit variable passing with `with`:

```
{% include "partials/header.twig" with {"site_name": "Cool Store"} %}
```

Isolate the included template from the parent scope with `only`:

```
{% include "partials/header.twig" with {"site_name": "Cool Store"} only %}
```

With `only`, the included template sees `site_name` and nothing else.

---

5. For Loops

Loop through arrays with `{% for %}`:

```
<ul>
{% for item in items %}
    <li>{{ item }}</li>
{% endfor %}
</ul>
```

The `loop` Variable

Inside every for loop, Frond provides a `loop` variable:

Property	Type	Description
`loop.index`	int	Current iteration (1-based)
`loop.index0`	int	Current iteration (0-based)
`loop.first`	bool	True on the first iteration
`loop.last`	bool	True on the last iteration
`loop.length`	int	Total number of items
`loop.revindex`	int	Iterations remaining (1-based)

```
<table>
    <thead>
        <tr><th>#</th><th>Name</th><th>Price</th></tr>
    </thead>
    <tbody>
    {% for product in products %}
        <tr class="{{ loop.index is odd ? 'row-light' : 'row-dark' }}">
            <td>{{ loop.index }}</td>
            <td>{{ product.name }}</td>
            <td>${{ product.price | number_format(2) }}</td>
        </tr>
    {% endfor %}
    </tbody>
</table>
```

Empty Lists

Handle empty lists with `{% else %}`:

```
{% for product in products %}
    <div class="product-card">
        <h3>{{ product.name }}</h3>
    </div>
{% else %}
    <p>No products found.</p>
{% endfor %}
```

If `products` is empty or undefined, the `else` block renders instead.

Looping Over Key-Value Pairs

```
{% for key, value in metadata %}
    <dt>{{ key }}</dt>
    <dd>{{ value }}</dd>
{% endfor %}
```

---

6. Conditionals

if / elseif / else

```
{% if user.role == "admin" %}
    <a href="/admin">Admin Panel</a>
{% elseif user.role == "editor" %}
    <a href="/editor">Editor Dashboard</a>
{% else %}
    <a href="/profile">My Profile</a>
{% endif %}
```

Ternary Operator

Inline conditionals:

```
<span class="{{ is_active ? 'text-green' : 'text-gray' }}">
    {{ is_active ? 'Active' : 'Inactive' }}
</span>
```

Testing for Existence

```
{% if error_message is defined %}
    <div class="alert alert-danger">{{ error_message }}</div>
{% endif %}
```

Truthiness

False values: `false`, `null`, `0`, `""` (empty string), `[]` (empty array). Everything else is true.

```
{% if items %}
    <p>{{ items | length }} items found.</p>
{% else %}
    <p>No items.</p>
{% endif %}
```

{% set %} -- Local Variables

Create or update a variable inside a template:

```
{% set greeting = "Hello" %}
{% set full_name = user.first_name ~ " " ~ user.last_name %}
{% set total = price * quantity %}
{% set discount = total - rebate %}

<p>{{ greeting }}, {{ full_name }}!</p>
<p>Total: {{ total }}, After discount: {{ discount }}</p>
```

The `~` operator concatenates strings. Arithmetic operators (`+`, `-`, `*`, `/`, `//`, `%`, `**`) work in `set` and expressions.

When combining filters with arithmetic, assign the filtered values first:

```
{% set dr = account.dr|default(0) %}
{% set cr = account.cr|default(0) %}
{% set balance = dr - cr %}
<p>Balance: {{ balance }}</p>
```

---

7. Filters

Filters transform values. Apply them with `|`:

```
{{ name | upper }}
```

Complete Filter Reference

String Filters

Filter	Example	Description	
`upper`	`{{ name | upper }}`		Convert to uppercase
`lower`	`{{ name | lower }}`		Convert to lowercase
`capitalize`	`{{ name | capitalize }}`		Capitalize first letter
`title`	`{{ name | title }}`		Capitalize each word
`trim`	`{{ name | trim }}`		Strip leading/trailing whitespace
`ltrim`	`{{ name | ltrim }}`		Strip leading whitespace
`rtrim`	`{{ name | rtrim }}`		Strip trailing whitespace
`slug`	`{{ title | slug }}`		Convert to URL-friendly slug
`wordwrap(80)`	`{{ text | wordwrap(80) }}`		Wrap text at N characters
`truncate(100)`	`{{ text | truncate(100) }}`		Truncate to N characters with ellipsis
`nl2br`	`{{ text | nl2br }}`		Convert newlines to ` ` tags
`striptags`	`{{ html | striptags }}`		Remove all HTML tags
`replace("a", "b")`	`{{ text | replace("old", "new") }}`		Replace occurrences of a substring

Array Filters

Filter	Example	Description	
`length`	`{{ items \	length }}`	Count items in array or string length
`reverse`	`{{ items \	reverse }}`	Reverse order of items
`sort`	`{{ items \	sort }}`	Sort items ascending
`shuffle`	`{{ items \	shuffle }}`	Randomly shuffle items
`first`	`{{ items \	first }}`	Get the first item
`last`	`{{ items \	last }}`	Get the last item
`join(", ")`	`{{ items \	join(", ") }}`	Join array items with separator
`split(",")`	`{{ csv \	split(",") }}`	Split string into array
`unique`	`{{ items \	unique }}`	Remove duplicate values
`filter`	`{{ items \	filter }}`	Remove falsy values from array
`map("name")`	`{{ items \	map("name") }}`	Extract a property from each item
`column("name")`	`{{ items \	column("name") }}`	Extract a column from array of objects
`batch(3)`	`{{ items \	batch(3) }}`	Group items into batches of N
`slice(0, 3)`	`{{ items \	slice(0, 3) }}`	Extract a slice from offset with length

Encoding Filters

Filter	Example	Description	
`escape` (`e`)	`{{ text | escape }}`	HTML-escape special characters
`raw` (`safe`)	`{{ html | raw }}`	Output without auto-escaping
`url_encode`	`{{ text | url_encode }}`	URL-encode a string
`base64_encode` (`base64encode`)	`{{ text | base64_encode }}`	Base64-encode a string
`base64_decode` (`base64decode`)	`{{ data | base64_decode }}`	Base64-decode a string
`md5`	`{{ text | md5 }}`	Compute MD5 hash
`sha256`	`{{ text | sha256 }}`	Compute SHA-256 hash

Numeric Filters

Filter	Example	Description	
`abs`	`{{ num | abs }}`	Absolute value
`round(2)`	`{{ price | round(2) }}`	Round to N decimal places
`number_format(2)`	`{{ price | number_format(2) }}`	Format with decimals and thousands separator
`int`	`{{ val | int }}`	Cast to integer
`float`	`{{ val | float }}`	Cast to float
`string`	`{{ val | string }}`	Cast to string

JSON Filters

Filter	Example	Description	
`json_encode`	`{{ data | json_encode }}`	Encode value as JSON string
`to_json` (`tojson`)	`{{ data | to_json }}`	Encode value as JSON string (alias)
`json_decode`	`{{ str | json_decode }}`	Decode JSON string to object
`js_escape`	`{{ text | js_escape }}`	Escape string for safe use in JavaScript

Dict Filters

Filter	Example	Description	
`keys`	`{{ obj | keys }}`		Get dictionary keys as array
`values`	`{{ obj | values }}`		Get dictionary values as array
`merge(other)`	`{{ defaults | merge(overrides) }}`		Merge two dictionaries

Other Filters

Filter	Example	Description	
`default("fallback")`	`{{ name | default("Guest") }}`		Fallback when value is empty or undefined
`date("Y-m-d")`	`{{ created | date("Y-m-d") }}`		Format a date value
`format(val)`	`{{ "%.2f" | format(price) }}`		Format string with value (sprintf-style)
`data_uri`	`{{ content | data_uri }}`		Convert to a data URI string
`dump`	`{{ var | dump }}` `or` `{{ dump(var) }}`		Debug output - gated on `TINA4_DEBUG=true` (see *Dumping Values*)
`form_token`	`{{ form_token() }}`		Generate a CSRF hidden input with token
`formTokenValue`	`{{ formTokenValue("context") }}`		Return just the raw JWT token string
`to_json`	`{{ data | to_json }}`		JSON-encode a value (safe, no double-escaping)
`js_escape`	`{{ text | js_escape }}`		Escape for safe use in JavaScript strings

Chaining Filters

Left to right:

```
{{ name | trim | lower | capitalize }}
{# "  ALICE SMITH  " -> "Alice smith" #}
```

Dumping Values for Debugging

The `dump` helper lets you inspect any variable mid-template. Two interchangeable forms are supported:

```
{{ user | dump }}
{{ dump($user) }}
```

Both produce the same `<pre>`-wrapped, HTML-escaped `var_dump()` of the value. Handles arrays, objects, class instances, and cyclic references (PHP's `var_dump` prints `*RECURSION*` for back-edges).

```
{{ dump($order) }}

{# Output: #}
{# <pre>object(Order)#42 (3) {                                #}
{#   ["id"]=> int(42)                                        #}
{#   ["items"]=> array(2) { ... }                            #}
{#   ["total"]=> float(99.99)                                #}
{# }</pre>                                                   #}
```

dump is gated on `TINA4_DEBUG=true`. In production (env var unset or `false`) **both** the filter and function form silently return an empty string. This prevents accidental leaks of internal state, object shapes, or sensitive values into rendered HTML if a developer leaves a `{{ dump($x) }}` call in a template.

```
# .env - dev
TINA4_DEBUG=true    # dump() outputs the value

# .env - production
TINA4_DEBUG=false   # dump() is a no-op
```

You can rely on this gate for safety, but treat `dump` as a development-only convenience. For structured output in production code paths, use `to_json`.

---

8. Macros

Macros are reusable template functions. Define once, call many times.

Defining a Macro

Create `src/templates/macros.twig`:

```
{% macro button(text, url, style) %}
    <a href="{{ url | default('#') }}" class="btn btn-{{ style | default('primary') }}">
        {{ text }}
    </a>
{% endmacro %}

{% macro alert(message, type) %}
    <div class="alert alert-{{ type | default('info') }}">
        {{ message }}
    </div>
{% endmacro %}

{% macro input(name, label, type, value) %}
    <div class="form-group">
        <label for="{{ name }}">{{ label | default(name | capitalize) }}</label>
        <input type="{{ type | default('text') }}" id="{{ name }}" name="{{ name }}" value="{{ v
    </div>
{% endmacro %}
```

Using Macros

Import and use:

```
{% from "macros.twig" import button, alert, input %}

{% extends "base.twig" %}

{% block content %}
    {{ alert("Your profile has been updated.", "success") }}

    <form method="POST" action="/profile">
        {{ input("name", "Full Name", "text", user.name) }}
        {{ input("email", "Email Address", "email", user.email) }}
        {{ input("phone", "Phone Number", "tel", user.phone) }}

        {{ button("Save Changes", "", "primary") }}
        {{ button("Cancel", "/dashboard", "secondary") }}
    </form>
{% endblock %}
```

Output (simplified):

```
<div class="alert alert-success">
    Your profile has been updated.
</div>

<form method="POST" action="/profile">
    <div class="form-group">
        <label for="name">Full Name</label>
        <input type="text" id="name" name="name" value="Alice">
    </div>
    <div class="form-group">
        <label for="email">Email Address</label>
        <input type="email" id="email" name="email" value="alice@example.com">
    </div>
    <div class="form-group">
        <label for="phone">Phone Number</label>
        <input type="tel" id="phone" name="phone" value="">
    </div>

    <a href="#" class="btn btn-primary">Save Changes</a>
    <a href="/dashboard" class="btn btn-secondary">Cancel</a>
</form>
```

---

9. Special Tags

{% raw %} -- Literal Output

Output literal `{{ }}` or `{% %}` without processing. Essential for Vue.js or Angular templates:

```
{% raw %}
    <div id="app">
        {{ message }}
    </div>
{% endraw %}
```

Outputs the literal text `{{ message }}`.

{% spaceless %} -- Remove Whitespace

Strip whitespace between HTML tags:

```
{% spaceless %}
    <div>
        <span>Hello</span>
    </div>
{% endspaceless %}
```

Output:

```
<div><span>Hello</span></div>
```

Useful for inline elements where whitespace creates unwanted gaps.

{% autoescape %} -- Control Escaping

Override auto-escaping for a block:

```
{% autoescape false %}
    {{ trusted_html }}
{% endautoescape %}
```

Everything inside outputs without HTML escaping. Equivalent to `| raw` on every variable, but more convenient for large blocks of trusted content.

Comments

Template comments are invisible in the output:

```
{# This comment will not appear in the HTML output #}

{#
    Multi-line comments work too.
    Use them to document template logic.
#}
```

---

10. tina4css Integration

Every Tina4 project includes `tina4.css` -- a built-in CSS utility framework. Available at `/css/tina4.css`. Layout, typography, common UI patterns. No external dependencies.

Include it in your base template:

```
<link rel="stylesheet" href="/css/tina4.css">
```

Layout Classes

```
<div class="container">
    <div class="row">
        <div class="col-6">Left half</div>
        <div class="col-6">Right half</div>
    </div>
</div>
```

Common Components

```
<!-- Buttons -->
<button class="btn btn-primary">Primary</button>
<button class="btn btn-secondary">Secondary</button>
<button class="btn btn-danger">Danger</button>

<!-- Cards -->
<div class="card">
    <div class="card-header">Title</div>
    <div class="card-body">Content here</div>
    <div class="card-footer">Footer</div>
</div>

<!-- Alerts -->
<div class="alert alert-success">Operation completed.</div>
<div class="alert alert-danger">Something went wrong.</div>
<div class="alert alert-warning">Please review your input.</div>

<!-- Forms -->
<div class="form-group">
    <label for="name">Name</label>
    <input type="text" id="name" class="form-control">
</div>
```

Utility Classes

```
<p class="text-center">Centered text</p>
<p class="text-right">Right-aligned text</p>
<div class="mt-4">Margin top</div>
<div class="p-3">Padding all around</div>
<span class="text-muted">Gray text</span>
<span class="text-primary">Primary color text</span>
```

No Bootstrap. No Tailwind. If you prefer those, swap the `<link>` tag. Tina4 does not care.

---

11. Exercise: Build a Product Catalog Page

Build a catalog page with a base layout, product cards, category filters, and a reusable card macro.

Requirements

- Create a base layout at `src/templates/catalog-base.twig` with blocks for `title`, `content`, and `scripts`
- Create a macro file at `src/templates/catalog-macros.twig` with:
- A `productCard(product)` macro that renders a styled card with name, category, price, stock status, and optional featured badge
- A `categoryFilter(categories, active)` macro that renders filter buttons
- Create a page template at `src/templates/catalog.twig` that:
- Extends the base layout
- Uses the macros

- Shows a heading with total product count
- Shows category filter buttons (All, plus one per unique category)
- Shows product cards in a grid
- Highlights featured products
- Handles empty filter results
- Create a route at `GET /catalog` that accepts an optional `?category=` filter

Data

Use this product list in your route handler:

```
$products = [
    ["name" => "Espresso Machine", "category" => "Kitchen", "price" => 299.99, "in_stock" => tru
    ["name" => "Yoga Mat", "category" => "Fitness", "price" => 29.99, "in_stock" => true, "featu
    ["name" => "Standing Desk", "category" => "Office", "price" => 549.99, "in_stock" => true, "
    ["name" => "Blender", "category" => "Kitchen", "price" => 89.99, "in_stock" => false, "featu
    ["name" => "Running Shoes", "category" => "Fitness", "price" => 119.99, "in_stock" => true,
    ["name" => "Desk Lamp", "category" => "Office", "price" => 39.99, "in_stock" => true, "featu
    ["name" => "Cast Iron Skillet", "category" => "Kitchen", "price" => 44.99, "in_stock" => tru
];
```

Test with:

```
http://localhost:7145/catalog
http://localhost:7145/catalog?category=Kitchen
http://localhost:7145/catalog?category=Fitness
```

---

12. Solution

Create `src/templates/catalog-base.twig`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}Product Catalog{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    <style>
        body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif; m
        .container { max-width: 1000px; margin: 0 auto; padding: 20px; }
        .header { background: #2c3e50; color: white; padding: 20px; margin-bottom: 24px; }
        .header h1 { margin: 0; }
        .header p { margin: 4px 0 0; opacity: 0.8; }
        .filters { margin-bottom: 20px; }
        .filter-btn { display: inline-block; padding: 6px 14px; margin: 0 6px 6px 0; border-radi
        .filter-btn.active { background: #2c3e50; color: white; border-color: #2c3e50; }
        .product-grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(280px, 1f
        .product-card { background: white; border: 2px solid #e9ecef; border-radius: 8px; paddin
        .product-card:hover { border-color: #adb5bd; }
        .product-card.featured { border-color: #f39c12; background: #fef9e7; }
        .product-name { font-size: 1.1em; font-weight: 600; margin: 0 0 4px; }
        .product-category { font-size: 0.85em; color: #6c757d; margin: 0 0 8px; }
```

```
        .product-price { font-size: 1.2em; font-weight: bold; color: #27ae60; }
        .badge { display: inline-block; padding: 2px 8px; border-radius: 4px; font-size: 0.75em;
        .badge-featured { background: #f39c12; color: white; }
        .badge-stock { background: #d4edda; color: #155724; }
        .badge-nostock { background: #f8d7da; color: #721c24; }
        .empty-state { text-align: center; padding: 40px; color: #6c757d; }
    </style>
</head>
<body>
    {% block content %}{% endblock %}

    <script src="/js/frond.js"></script>
    {% block scripts %}{% endblock %}
</body>
</html>
```

Create `src/templates/catalog-macros.twig`:

```
{% macro productCard(product) %}
    <div class="product-card{{ product.featured ? ' featured' : '' }}">
        <p class="product-name">
            {{ product.name }}
            {% if product.featured %}
                <span class="badge badge-featured">Featured</span>
            {% endif %}
        </p>
        <p class="product-category">{{ product.category }}</p>
        <p class="product-price">
            ${{ product.price | number_format(2) }}
            {% if product.in_stock %}
                <span class="badge badge-stock">In Stock</span>
            {% else %}
                <span class="badge badge-nostock">Out of Stock</span>
            {% endif %}
        </p>
    </div>
{% endmacro %}

{% macro categoryFilter(categories, active) %}
    <div class="filters">
        <a href="/catalog" class="filter-btn{{ active is not defined or active == '' ? ' active'
        {% for cat in categories %}
            <a href="/catalog?category={{ cat }}" class="filter-btn{{ active == cat ? ' active'
        {% endfor %}
    </div>
{% endmacro %}
```

Create `src/templates/catalog.twig`:

```
{% extends "catalog-base.twig" %}

{% from "catalog-macros.twig" import productCard, categoryFilter %}

{% block title %}{{ active_category | default("All") }} Products - Catalog{% endblock %}

{% block content %}
    <div class="header">
        <h1>Product Catalog</h1>
        <p>{{ products | length }} product{{ products | length != 1 ? 's' : '' }}{% if active_ca
    </div>
```

```
    <div class="container">
        {{ categoryFilter(categories, active_category) }}

        {% if products | length > 0 %}
            <div class="product-grid">
                {% for product in products %}
                    {{ productCard(product) }}
                {% endfor %}
            </div>
        {% else %}
            <div class="empty-state">
                <h2>No products found</h2>
                <p>Try a different category or <a href="/catalog">view all products</a>.</p>
            </div>
        {% endif %}
    </div>
{% endblock %}
```

Create `src/routes/catalog.php`:

```
<?php
use Tina4\Router;

Router::get("/catalog", function ($request, $response) {
    $allProducts = [
        ["name" => "Espresso Machine", "category" => "Kitchen", "price" => 299.99, "in_stock" =>
        ["name" => "Yoga Mat", "category" => "Fitness", "price" => 29.99, "in_stock" => true, "f
        ["name" => "Standing Desk", "category" => "Office", "price" => 549.99, "in_stock" => tru
        ["name" => "Blender", "category" => "Kitchen", "price" => 89.99, "in_stock" => false, "f
        ["name" => "Running Shoes", "category" => "Fitness", "price" => 119.99, "in_stock" => tr
        ["name" => "Desk Lamp", "category" => "Office", "price" => 39.99, "in_stock" => true, "f
        ["name" => "Cast Iron Skillet", "category" => "Kitchen", "price" => 44.99, "in_stock" =>
    ];

    // Get unique categories
    $categories = array_unique(array_column($allProducts, "category"));
    sort($categories);

    // Filter by category if specified
    $activeCategory = $request->params["category"] ?? "";
    if (!empty($activeCategory)) {
        $products = array_values(array_filter(
            $allProducts,
            fn($p) => strtolower($p["category"]) === strtolower($activeCategory)
        ));
    } else {
        $products = $allProducts;
    }

    return $response->render("catalog.twig", [
        "products" => $products,
        "categories" => $categories,
        "active_category" => $activeCategory
    ]);
});
```

Expected browser output for `/catalog`:

- A dark header with "Product Catalog" and "7 products"

- Filter buttons: All (active), Fitness, Kitchen, Office
- A grid of 7 product cards
- Three cards (Espresso Machine, Standing Desk, Desk Lamp) have a gold border and "Featured" badge
- The Blender card shows an "Out of Stock" badge in red

Expected browser output for `/catalog?category=Kitchen`:

- Header shows "3 products in Kitchen"
- The Kitchen filter button is active
- Three cards: Espresso Machine, Blender, Cast Iron Skillet

---

13. Gotchas

1. `{% extends %}` Must Be the First Tag

Problem: Template inheritance does not work. The page renders without the base layout.

Cause: `{% extends "base.twig" %}` must be the very first tag. Any text, whitespace, or comment before it breaks inheritance.

Fix: Put `{% extends %}` on the absolute first line. Move `{% from %}` imports after it.

2. Undefined Variables Show Nothing

Problem: `{{ username }}` renders as blank instead of an error.

Cause: Frond silently outputs nothing for undefined variables. By design, like Twig. But it hides bugs.

Fix: Use the `default` filter: `{{ username | default("Guest") }}`. Or check with `{% if username is defined %}`.

3. Auto-Escaping Prevents HTML Output

Problem: HTML content like `"<strong>bold</strong>"` appears as literal text.

Cause: Auto-escaping converts `<` to `<` and `>` to `>` for security.

Fix: Trusted content: `{{ content | raw }}`. Never use `raw` on user-supplied input.

4. Variable Scope in Includes

Problem: A variable defined inside a `{% for %}` loop is not accessible after the loop ends.

Cause: Loop variables are scoped to the loop. They do not leak.

Fix: Use `{% set %}` before the loop and update inside it. Or restructure to keep all logic within the loop.

5. Macro Arguments Are Positional

Problem: `{{ button("Click", style="danger") }}` does not work.

Cause: Frond macros use positional arguments. Order matters. Keyword arguments are not supported.

Fix: Pass arguments in definition order: `{{ button("Click", "/url", "danger") }}`. For many optional arguments, pass a single object.

6. Template File Extension Does Not Matter

Problem: Unsure whether to use `.html`, `.twig`, or `.tpl`.

Cause: Frond processes any file in `src/templates/` regardless of extension.

Fix: Pick one extension. Be consistent. This book uses `.twig` for templates with Twig syntax and `.html` for simple files. Both work identically.

7. Filters Are Not PHP Functions

Problem: `{{ items | count }}` or `{{ name | strtoupper }}` causes an error.

Cause: Frond filters follow Twig conventions, not PHP function names.

Fix: `{{ items | length }}` not `count`. `{{ name | upper }}` not `strtoupper`. `{{ text | lower }}` not `strtolower`. See the filter table in section 7.

Database

1. From Arrays to Real Data

In Chapters 2 and 3, data lived in PHP arrays. Every server restart wiped the slate. That works for learning routing and responses. Real applications need persistence. This chapter covers Tina4's database layer: raw queries, parameterised queries, transactions, schema inspection, helper methods, migrations, and seeding.

Tina4 supports five database engines: SQLite, PostgreSQL, MySQL, Microsoft SQL Server, and Firebird. The API stays identical across all of them. Switch databases by changing one line in `.env`.

---

2. Connecting to a Database

The Default: SQLite

`tina4 init` creates an empty `data/` directory but does not create a database file. The default `.env` contains:

```
TINA4_DEBUG=true
```

No explicit `TINA4_DATABASE_URL` means Tina4 defaults to `sqlite:///data/app.db`. The SQLite file is created automatically the first time Tina4 opens a database connection (for example, when you run a query, execute a migration, or the framework initialises the database layer at startup with a `TINA4_DATABASE_URL` configured).

Connection Strings for Other Databases

Set `TINA4_DATABASE_URL` in `.env`:

```
# SQLite (explicit)
TINA4_DATABASE_URL=sqlite:///data/app.db

# PostgreSQL
TINA4_DATABASE_URL=postgres://localhost:5432/myapp

# MySQL
TINA4_DATABASE_URL=mysql://localhost:3306/myapp

# Microsoft SQL Server
TINA4_DATABASE_URL=mssql://localhost:1433/myapp

# Firebird
TINA4_DATABASE_URL=firebird://localhost:3050/path/to/database.fdb
```

Firebird URL Forms

Firebird is the awkward one: every other engine has a server-side database name (`postgres://host:port/dbname`), but Firebird wants either an absolute file path on the server, a Windows drive-letter path, or an alias. The classic URI form needs a double slash to keep the leading `/` of an absolute path through `parse_url`, which is unintuitive.

Tina4 normalises five equivalent forms. Pick whichever reads best:

```
# Classic double-slash absolute path -- the URL spec way
TINA4_DATABASE_URL=firebird://SYSDBA:masterkey@localhost:3050//firebird/data/app.fdb

# Single-slash absolute path -- what most people instinctively type
TINA4_DATABASE_URL=firebird://SYSDBA:masterkey@localhost:3050/firebird/data/app.fdb

# Windows drive-letter path (also accepts /C%3A/Data/app.fdb)
TINA4_DATABASE_URL=firebird://SYSDBA:masterkey@host:3050/C:/Data/app.fdb

# Firebird alias (single token, no slashes)
TINA4_DATABASE_URL=firebird://SYSDBA:masterkey@localhost:3050/employee
```

For ops setups that keep the server URL and database location in separate config layers -- or for Windows backslash paths -- set `TINA4_DATABASE_FIREBIRD_PATH`:

```
TINA4_DATABASE_FIREBIRD_PATH=C:\firebird\data\app.fdb
TINA4_DATABASE_URL=firebird://SYSDBA:masterkey@localhost:3050/ignored
```

The env override wins over whatever path is in the URL.

Firebird: Dual-Driver Support

The Firebird adapter works with either the `ibase_*` or `fbird_*` PHP functions. It auto-detects which set is available at connection time. If you have `ext-interbase` installed, it uses `ibase_*`. If you have the newer `fbird_*` functions, it uses those instead. No configuration needed -- install whichever extension is available for your platform and the adapter picks it up.

Separate Credentials

Keep credentials out of the connection string. Better for production:

```
TINA4_DATABASE_URL=postgres://localhost:5432/myapp
TINA4_DATABASE_USERNAME=myuser
TINA4_DATABASE_PASSWORD=secretpassword
```

Tina4 merges these at startup. Separate variables take precedence over anything embedded in the URL.

Connection Pooling

For applications that handle many concurrent requests, enable connection pooling with the `pool` parameter:

```
$db = new Database("postgres://localhost/mydb", pool: 5);
```

The `pool` parameter controls how many database connections are maintained:

- `pool: 0` (the default) -- a single connection is used for all queries
- `pool: N` (where N > 0) -- N connections are created and rotated round-robin across queries

Pooled connections are thread-safe. Each query is dispatched to the next available connection in the pool. This eliminates contention when multiple route handlers query the database simultaneously.

Verifying the Connection

Update `.env`. Restart. Check:

```
curl http://localhost:7145/health
```

```
{
  "status": "ok",
  "version": "3.0.0",
  "uptime": 3.14,
  "framework": "tina4-php"
}
```

---

3. Getting the Database Object

Access the database through the global `Tina4\Database` class:

```
<?php
use Tina4\Router;
use Tina4\Database;

Router::get("/api/test-db", function ($request, $response) {
    $db = Database::getConnection();

    $result = $db->fetch("SELECT 1 + 1 AS answer");

    return $response->json($result);
});
```

```
curl http://localhost:7145/api/test-db
```

```
{"answer": 2}
```

`Database::getConnection()` returns the active connection. Call `fetch()`, `execute()`, and `fetchOne()` on it.

---

4. Raw Queries

fetch() -- Get Multiple Rows

```
$db = Database::getConnection();

// Returns an array of associative arrays
$products = $db->fetch("SELECT * FROM products WHERE price > 50");
```

Each row is an associative array:

```
// $products looks like:
[
    ["id" => 1, "name" => "Keyboard", "price" => 79.99],
    ["id" => 4, "name" => "Standing Desk", "price" => 549.99]
]
```

DatabaseResult

`fetch()` returns a `DatabaseResult` object. It behaves like an array but carries extra metadata about the query.

Properties

```
$result = $db->fetch("SELECT * FROM users WHERE active = ?", [1]);

$result->records;      // [["id" => 1, "name" => "Alice"], ["id" => 2, "name" => "Bob"]]
$result->columns;      // ["id", "name", "email", "active"]
$result->count;        // total number of matching rows
$result->limit;        // query limit (if set)
$result->offset;       // query offset (if set)
```

Iteration

A `DatabaseResult` is iterable. Use it directly in `foreach`:

```
foreach ($result as $user) {
    echo $user["name"];
}
```

Index Access

Access rows by index like a regular array:

```
$firstUser = $result[0];
```

Countable

`count()` works on the result:

```
echo count($result); // number of records in this result set
```

Conversion Methods

```
$result->toJson();      // JSON string of all records
$result->toCsv();       // CSV string with column headers
$result->toArray();     // plain array of associative arrays
$result->toPaginate();  // ["records" => [...], "count" => 42, "limit" => 10, "offset" => 0]
```

`toPaginate()` is designed for building paginated API responses. It bundles the records with the total count, limit, and offset in a single array.

Schema Metadata with columnInfo()

`columnInfo()` returns detailed metadata about the columns in the result set. The data is lazy-loaded -- it only queries the database schema when you call the method for the first time:

```
$info = $result->columnInfo();
// [
//     ["name" => "id", "type" => "INTEGER", "size" => null, "decimals" => null, "nullable" => f
//     ["name" => "name", "type" => "TEXT", "size" => null, "decimals" => null, "nullable" => fa
//     ["name" => "email", "type" => "TEXT", "size" => 255, "decimals" => null, "nullable" => tr
//     ...
// ]
```

Each column entry contains:

Field	Description
`name`	Column name
`type`	Database type (e.g. `INTEGER`, `TEXT`, `REAL`)
`size`	Maximum size (or `null` if not applicable)
`decimals`	Decimal places (or `null`)
`nullable`	Whether the column allows `NULL`
`primary_key`	Whether the column is part of the primary key

This is useful for building dynamic forms, generating documentation, or validating data before insert.

fetchOne() -- Get a Single Row

```
$product = $db->fetchOne("SELECT * FROM products WHERE id = 1");
// Returns: ["id" => 1, "name" => "Keyboard", "price" => 79.99]
```

No match returns `null`.

execute() -- Run a Statement

For INSERT, UPDATE, DELETE, and DDL -- statements that do not return rows:

```
$db->execute("INSERT INTO products (name, price) VALUES ('Widget', 9.99)");
$db->execute("UPDATE products SET price = 89.99 WHERE id = 1");
$db->execute("DELETE FROM products WHERE id = 5");
$db->execute("CREATE TABLE IF NOT EXISTS logs (id INTEGER PRIMARY KEY, message TEXT, created_at
```

Full Example: A Simple Query Route

```
<?php
use Tina4\Router;
use Tina4\Database;

Router::get("/api/products", function ($request, $response) {
    $db = Database::getConnection();

    $products = $db->fetch("SELECT * FROM products ORDER BY name");

    return $response->json([
        "products" => $products,
        "count" => count($products)
    ]);
});
```

```
curl http://localhost:7145/api/products
```

```
{
  "products": [
    {"id": 1, "name": "Keyboard", "price": 79.99, "in_stock": 1},
    {"id": 2, "name": "Mouse", "price": 29.99, "in_stock": 1},
    {"id": 3, "name": "Monitor", "price": 399.99, "in_stock": 0}
  ],
  "count": 3
```

```
}
```

---

5. Parameterised Queries

Never concatenate user input into SQL strings. That is how SQL injection happens:

```
// NEVER do this:
$db->fetch("SELECT * FROM products WHERE name = '" . $userInput . "'");
```

Use parameterised queries instead. Parameters go in the second argument:

```
$db = Database::getConnection();

// Named parameters
$product = $db->fetchOne(
    "SELECT * FROM products WHERE id = :id",
    ["id" => 42]
);

// Positional parameters
$products = $db->fetch(
    "SELECT * FROM products WHERE price BETWEEN ? AND ? ORDER BY price",
    [10.00, 100.00]
);
```

The database driver handles escaping. Your input never touches the SQL string.

A Safe Search Endpoint

```
<?php
use Tina4\Router;
use Tina4\Database;

Router::get("/api/products/search", function ($request, $response) {
    $db = Database::getConnection();

    $q = $request->params["q"] ?? "";
    $maxPrice = (float) ($request->params["max_price"] ?? 99999);

    if (empty($q)) {
        return $response->json(["error" => "Query parameter 'q' is required"], 400);
    }

    $products = $db->fetch(
        "SELECT * FROM products WHERE name LIKE :query AND price <= :maxPrice ORDER BY name",
        ["query" => "%" . $q . "%", "maxPrice" => $maxPrice]
    );

    return $response->json([
        "query" => $q,
        "max_price" => $maxPrice,
        "results" => $products,
        "count" => count($products)
    ]);
});
```

```
curl "http://localhost:7145/api/products/search?q=key&max_price=100"
```

```
{
  "query": "key",
  "max_price": 100,
  "results": [
    {"id": 1, "name": "Wireless Keyboard", "price": 79.99, "in_stock": 1}
  ],
  "count": 1
}
```

---

6. Transactions

Group operations that must succeed or fail as one unit:

```
<?php
use Tina4\Router;
use Tina4\Database;

Router::post("/api/orders", function ($request, $response) {
    $db = Database::getConnection();
    $body = $request->body;

    try {
        $db->startTransaction();

        // Create the order
        $db->execute(
            "INSERT INTO orders (customer_id, total, status) VALUES (:customerId, :total, 'pendi
            ["customerId" => $body["customer_id"], "total" => $body["total"]]
        );

        // Get the new order ID
        $order = $db->fetchOne("SELECT last_insert_rowid() AS id");
        $orderId = $order["id"];

        // Create order items
        foreach ($body["items"] as $item) {
            $db->execute(
                "INSERT INTO order_items (order_id, product_id, quantity, price) VALUES (:orderI
                [
                    "orderId" => $orderId,
                    "productId" => $item["product_id"],
                    "qty" => $item["quantity"],
                    "price" => $item["price"]
                ]
            );

            // Decrease stock
            $db->execute(
                "UPDATE products SET stock = stock - :qty WHERE id = :productId",
                ["qty" => $item["quantity"], "productId" => $item["product_id"]]
            );
        }

        $db->commit();

        return $response->json(["order_id" => $orderId, "status" => "created"], 201);
    } catch (\Exception $e) {
```

```
        $db->rollback();
        return $response->json(["error" => "Order failed: " . $e->getMessage()], 500);
    }
});
```

Any step fails. `rollback()` undoes everything. The database never lands in a half-finished state.

Call `commit()` to save. Forget it and the transaction rolls back when the connection closes.

---

7. Schema Inspection

Tina4 exposes methods to inspect your database structure at runtime. The `Database` object delegates to the underlying adapter, so these work across all five engines.

getTables()

```
$db = Database::getConnection();
$tables = $db->getTables();
```

Returns an array of table names:

```
["orders", "order_items", "products", "users"]
```

getColumns()

```
$columns = $db->getColumns("products");
```

Returns column definitions:

```
[
    ["name" => "id", "type" => "INTEGER", "nullable" => false, "primary" => true],
    ["name" => "name", "type" => "TEXT", "nullable" => false, "primary" => false],
    ["name" => "price", "type" => "REAL", "nullable" => true, "primary" => false],
    ["name" => "in_stock", "type" => "INTEGER", "nullable" => true, "primary" => false]
]
```

tableExists()

```
if ($db->tableExists("products")) {
    // Table exists, safe to query
}
```

getDatabaseType()

```
$engine = $db->getDatabaseType();
// "sqlite", "postgresql", "mysql", "mssql", or "firebird"
```

Returns a lowercase string identifying the active database engine. Use this when you need engine-specific SQL in a multi-database setup.

A Schema Info Endpoint

Combine these methods to build a schema browser:

```
<?php
use Tina4\Router;
use Tina4\Database;

Router::get("/api/schema", function ($request, $response) {
```

```
    $db = Database::getConnection();
    $tables = $db->getTables();

    $schema = [];
    foreach ($tables as $table) {
        $schema[$table] = $db->getColumns($table);
    }

    return $response->json([
        "engine" => $db->getDatabaseType(),
        "tables" => $schema
    ]);
});
```

Schema inspection powers admin dashboards, migration generators, and dynamic form builders. The database tells you its own structure -- no guesswork required.

---

8. Batch Operations with executeMany()

Insert or update many rows in one call:

```
$db = Database::getConnection();

$products = [
    ["name" => "Widget A", "price" => 9.99],
    ["name" => "Widget B", "price" => 14.99],
    ["name" => "Widget C", "price" => 19.99],
    ["name" => "Widget D", "price" => 24.99]
];

$db->executeMany(
    "INSERT INTO products (name, price) VALUES (:name, :price)",
    $products
);
```

`executeMany()` prepares the statement once and executes it for each item. The SQL is parsed once. Four rows inserted. Much faster than four separate `execute()` calls.

---

9. Helper Methods: insert(), update(), delete()

Shorthand methods for simple operations. No SQL needed.

insert()

```
$db = Database::getConnection();

// Insert a single row
$db->insert("products", [
    "name" => "Wireless Mouse",
    "price" => 34.99,
    "in_stock" => 1
]);

// Insert multiple rows
$db->insert("products", [
    ["name" => "USB Cable", "price" => 9.99, "in_stock" => 1],
    ["name" => "HDMI Cable", "price" => 14.99, "in_stock" => 1],
    ["name" => "DisplayPort Cable", "price" => 19.99, "in_stock" => 0]
]);
```

update()

```
// Update rows matching a filter
$db->update("products", ["price" => 39.99, "in_stock" => 1], "id = :id", ["id" => 7]);
```

Third argument: WHERE clause. Fourth argument: parameters.

delete()

```
// Delete rows matching a filter
$db->delete("products", "id = :id", ["id" => 7]);
```

These helpers generate SQL for you. Use them for simple CRUD. For joins, subqueries, or aggregations, reach for raw queries.

---

10. Migrations

Migrations are versioned SQL scripts that evolve your schema over time. Write migration files. Tina4 applies them in order and tracks what has run. No manual `CREATE TABLE` calls against production.

File Naming

Two naming patterns are supported:

- **Sequential**: `000001_create_products_table.sql`
- **Timestamp**: `20260322143000_create_products_table.sql` (YYYYMMDDHHMMSS)

Both work. Pick one pattern and stick with it throughout a project. Do not mix them.

Generating a Migration

```
tina4 generate migration create_products_table
```

```
Created migration: migrations/20260322143000_create_products_table.sql
Created migration: migrations/20260322143000_create_products_table.down.sql
```

The generator creates two files: the migration itself and a matching `.down.sql` file for rollback.

Writing the Migration

Edit `migrations/20260322143000_create_products_table.sql`:

```
CREATE TABLE products (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    category TEXT NOT NULL DEFAULT 'Uncategorized',
    price REAL NOT NULL DEFAULT 0.00,
    in_stock INTEGER NOT NULL DEFAULT 1,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT DEFAULT CURRENT_TIMESTAMP
);
```

Edit `migrations/20260322143000_create_products_table.down.sql`:

```
DROP TABLE IF EXISTS products;
```

The `.down.sql` file is optional but recommended for production projects. It contains the SQL that reverses the migration.

Running Migrations

```
tina4 migrate
# or
tina4 migrate
```

```
Running migrations...
  [APPLIED] 20260322143000_create_products_table.sql
Migrations complete. 1 applied.
```

Each call to `tina4 migrate` is a **batch**. All pending migrations applied in a single run share the same batch number. This matters for rollback.

Checking Migration Status

```
tina4 migrate:status
```

```
Migration                                    Status     Applied At
---------                                    ------     ----------
20260322143000_create_products_table.sql      applied    2026-03-22 14:30:00
20260322150000_create_orders_table.sql        pending    -
```

Shows which migrations have been applied and which are still pending.

Rolling Back

```
tina4 migrate:rollback
```

```
Rolling back last batch...
  [ROLLED BACK] 20260322150000_create_orders_table.sql
Rollback complete. 1 rolled back.
```

Rollback undoes the entire last batch. It finds each migration's `.down.sql` file and executes it, then removes the tracking records. If you applied three migrations in one `tina4 migrate` call, rollback undoes all three.

The `.down.sql` file must share the exact same base name as the migration. For `20260322150000_create_orders_table.sql`, the down file is `20260322150000_create_orders_table.down.sql`.

Tracking Table

Tina4 creates a `tina4_migration` table to track applied migrations. The table has four columns:

Column	Type	Description
`id`	integer	Auto-increment primary key
`migration`	VARCHAR(255)	The migration filename
`batch`	integer	Batch number (increments each `tina4 migrate` run)
`applied_at`	timestamp	When the migration was applied

You should not modify this table directly, but inspecting it can help debug migration issues.

Advanced SQL: Stored Procedures and Block Comments

The migration runner handles more than simple semicolon-delimited statements. It correctly parses:

- `$$` **delimited blocks** -- PostgreSQL stored procedures and functions
- `//` **blocks** -- alternative delimiter blocks
- `/* */` **block comments** -- skipped during parsing
- `--` **line comments** -- skipped during parsing

A PostgreSQL stored procedure migration works without issues:

```
CREATE OR REPLACE FUNCTION update_modified_column()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER set_updated_at
    BEFORE UPDATE ON products
    FOR EACH ROW
    EXECUTE FUNCTION update_modified_column();
```

No special configuration needed. The SQL splitter recognises `$$` boundaries and treats the block as a single statement.

A Real Migration Sequence

```
migrations/
■■■ 20260322143000_create_products_table.sql
■■■ 20260322143000_create_products_table.down.sql
■■■ 20260322143100_create_users_table.sql
■■■ 20260322143100_create_users_table.down.sql
■■■ 20260322143200_create_orders_table.sql
■■■ 20260322143200_create_orders_table.down.sql
■■■ 20260322143300_create_order_items_table.sql
■■■ 20260322143300_create_order_items_table.down.sql
■■■ 20260323091500_add_email_index_to_users.sql
```

The index migration without a `.down.sql` -- this is fine. Down migrations are optional. If you roll back and no `.down.sql` exists, the tracking record is removed but no reversal SQL runs.

The index migration:

```
CREATE INDEX idx_users_email ON users (email);
```

Its down file (`20260323091500_add_email_index_to_users.down.sql`), if you choose to create one:

```
DROP INDEX IF EXISTS idx_users_email;
```

Migrations run in filename order. Each runs only once.

---

11. Query Caching

For read-heavy applications:

```
TINA4_DB_CACHE=true
```

Tina4 caches results of `fetch()` and `fetchOne()` calls. Identical queries with identical parameters return cached results instead of hitting the database.

The cache invalidates when you call `execute()`, `insert()`, `update()`, or `delete()` on the same table.

Per-query control:

```
// Force a fresh query (bypass cache)
$products = $db->fetch("SELECT * FROM products", [], false); // third arg = use cache

// Clear the entire cache
$db->clearCache();
```

---

12. Exercise: Build a Notes App

A notes application backed by SQLite. Migration for the table. Full CRUD API.

Requirements

- Create a migration for a `notes` table:

- `id` -- integer, primary key, auto-increment
- `title` -- text, not null
- `content` -- text, not null
- `tag` -- text, default "general"
- `created_at` -- text, default current timestamp
- `updated_at` -- text, default current timestamp
- Build these endpoints:

Method	Path	Description
`GET`	`/api/notes`	List all notes. Support `?tag=` and `?search=` filters.
`GET`	`/api/notes/{id:int}`	Get a single note. 404 if not found.
`POST`	`/api/notes`	Create a note. Validate title and content are not empty.
`PUT`	`/api/notes/{id:int}`	Update a note. 404 if not found.
`DELETE`	`/api/notes/{id:int}`	Delete a note. 204 on success, 404 if not found.

Test with:

```
# Create
curl -X POST http://localhost:7145/api/notes \
  -H "Content-Type: application/json" \
  -d '{"title": "Shopping List", "content": "Milk, eggs, bread", "tag": "personal"}'

# List all
curl http://localhost:7145/api/notes

# Search
curl "http://localhost:7145/api/notes?search=shopping"

# Filter by tag
curl "http://localhost:7145/api/notes?tag=personal"

# Get one
curl http://localhost:7145/api/notes/1

# Update
curl -X PUT http://localhost:7145/api/notes/1 \
  -H "Content-Type: application/json" \
  -d '{"title": "Updated Shopping List", "content": "Milk, eggs, bread, butter"}'

# Delete
curl -X DELETE http://localhost:7145/api/notes/1
```

---

13. Solution

Migration

Generate the migration:

```
tina4 generate migration create_notes_table
```

Edit `migrations/20260322143000_create_notes_table.sql`:

```
CREATE TABLE notes (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    content TEXT NOT NULL,
    tag TEXT NOT NULL DEFAULT 'general',
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT DEFAULT CURRENT_TIMESTAMP
);
```

Edit `migrations/20260322143000_create_notes_table.down.sql`:

```
DROP TABLE IF EXISTS notes;
```

Run it:

```
tina4 migrate
```

```
Running migrations...
  [APPLIED] 20260322143000_create_notes_table.sql
Migrations complete. 1 applied.
```

Routes

Create `src/routes/notes.php`:

```
<?php
use Tina4\Router;
use Tina4\Database;

// List all notes with optional filters
Router::get("/api/notes", function ($request, $response) {
    $db = Database::getConnection();

    $tag = $request->params["tag"] ?? "";
    $search = $request->params["search"] ?? "";

    $sql = "SELECT * FROM notes";
    $params = [];
    $conditions = [];

    if (!empty($tag)) {
        $conditions[] = "tag = :tag";
        $params["tag"] = $tag;
    }

    if (!empty($search)) {
        $conditions[] = "(title LIKE :search OR content LIKE :search)";
        $params["search"] = "%" . $search . "%";
```

```
    }

    if (!empty($conditions)) {
        $sql .= " WHERE " . implode(" AND ", $conditions);
    }

    $sql .= " ORDER BY updated_at DESC";

    $notes = $db->fetch($sql, $params);

    return $response->json([
        "notes" => $notes,
        "count" => count($notes)
    ]);
});

// Get a single note
Router::get("/api/notes/{id:int}", function ($request, $response) {
    $db = Database::getConnection();
    $id = $request->params["id"];

    $note = $db->fetchOne("SELECT * FROM notes WHERE id = :id", ["id" => $id]);

    if ($note === null) {
        return $response->json(["error" => "Note not found", "id" => $id], 404);
    }

    return $response->json($note);
});

// Create a note
Router::post("/api/notes", function ($request, $response) {
    $db = Database::getConnection();
    $body = $request->body;

    // Validate
    $errors = [];
    if (empty($body["title"])) {
        $errors[] = "Title is required";
    }
    if (empty($body["content"])) {
        $errors[] = "Content is required";
    }
    if (!empty($errors)) {
        return $response->json(["errors" => $errors], 400);
    }

    $db->execute(
        "INSERT INTO notes (title, content, tag) VALUES (:title, :content, :tag)",
        [
            "title" => $body["title"],
            "content" => $body["content"],
            "tag" => $body["tag"] ?? "general"
        ]
    );

    $note = $db->fetchOne("SELECT * FROM notes WHERE id = last_insert_rowid()");

    return $response->json($note, 201);
```

```
});

// Update a note
Router::put("/api/notes/{id:int}", function ($request, $response) {
    $db = Database::getConnection();
    $id = $request->params["id"];
    $body = $request->body;

    $existing = $db->fetchOne("SELECT * FROM notes WHERE id = :id", ["id" => $id]);

    if ($existing === null) {
        return $response->json(["error" => "Note not found", "id" => $id], 404);
    }

    $db->execute(
        "UPDATE notes SET title = :title, content = :content, tag = :tag, updated_at = CURRENT_T
        [
            "title" => $body["title"] ?? $existing["title"],
            "content" => $body["content"] ?? $existing["content"],
            "tag" => $body["tag"] ?? $existing["tag"],
            "id" => $id
        ]
    );

    $note = $db->fetchOne("SELECT * FROM notes WHERE id = :id", ["id" => $id]);

    return $response->json($note);
});

// Delete a note
Router::delete("/api/notes/{id:int}", function ($request, $response) {
    $db = Database::getConnection();
    $id = $request->params["id"];

    $existing = $db->fetchOne("SELECT * FROM notes WHERE id = :id", ["id" => $id]);

    if ($existing === null) {
        return $response->json(["error" => "Note not found", "id" => $id], 404);
    }

    $db->execute("DELETE FROM notes WHERE id = :id", ["id" => $id]);

    return $response->json(null, 204);
});
```

Expected output for create:

```
{
  "id": 1,
  "title": "Shopping List",
  "content": "Milk, eggs, bread",
  "tag": "personal",
  "created_at": "2026-03-22 14:30:00",
  "updated_at": "2026-03-22 14:30:00"
}
```

(Status: `201 Created`)

Expected output for list:

```
{
  "notes": [
    {
      "id": 1,
      "title": "Shopping List",
      "content": "Milk, eggs, bread",
      "tag": "personal",
      "created_at": "2026-03-22 14:30:00",
      "updated_at": "2026-03-22 14:30:00"
    }
  ],
  "count": 1
}
```

Expected output for search:

```
{
  "notes": [
    {
      "id": 1,
      "title": "Shopping List",
      "content": "Milk, eggs, bread",
      "tag": "personal",
      "created_at": "2026-03-22 14:30:00",
      "updated_at": "2026-03-22 14:30:00"
    }
  ],
  "count": 1
}
```

Expected output for validation error:

```
{"errors": ["Title is required", "Content is required"]}
```

(Status: `400 Bad Request`)

---

14. Seeder -- Generating Test Data

Testing with an empty database tells you nothing. Testing with hand-typed rows is slow and brittle. The `FakeData` class generates realistic test data. The `seedTable()` function inserts it in bulk.

FakeData

```
use Tina4\FakeData;

$fake = new FakeData();

$fake->name();       // "Grace Lopez"
$fake->email();      // "bob.anderson@demo.net"
$fake->phone();      // "+1 (547) 382-9104"
$fake->sentence();   // "Magna exercitation lorem ipsum dolor sit amet consectetur."
$fake->paragraph();  // Four sentences of filler text
$fake->integer();    // 7342
$fake->decimal();    // 481.29
$fake->date();       // "2023-07-14"
$fake->uuid();       // "a3f1b2c4-d5e6-f7a8-b9c0-d1e2f3a4b5c6"
$fake->address();    // "742 Oak Ave, Tokyo"
$fake->boolean();    // true
```

Every method draws from built-in word banks. No network calls. No external packages.

Deterministic Output

Pass a seed to get reproducible results. The same seed produces the same sequence every time:

```
$fake = new FakeData(42);
$fake->name();   // Always "Wendy White" with seed 42
$fake->email();  // Always the same email with seed 42
```

Deterministic data means deterministic assertions. This matters for tests.

Seeding a Table

`seedTable()` combines `FakeData` with your database. Pass a field map -- an associative array where each key is a column name and each value is a callable that generates data:

```
use Tina4\FakeData;
use Tina4\Database;
use function Tina4\seedTable;

$db = Database::getConnection();
$fake = new FakeData(1);

seedTable($db, "users", 100, [
    "name" => [$fake, "name"],
    "email" => [$fake, "email"],
    "phone" => [$fake, "phone"],
    "bio" => [$fake, "sentence"],
]);
```

This inserts 100 rows into the `users` table. Each row calls `$fake->name()`, `$fake->email()`, and so on to generate values. The function commits after all rows insert.

Overrides

Static values that apply to every row go in the overrides array:

```
seedTable($db, "users", 50, [
    "name" => [$fake, "name"],
    "email" => [$fake, "email"],
], [
```

```
    "role" => "member",
    "active" => 1,
]);
```

Every row gets `role = "member"` and `active = 1`. The field map generates the rest.

Seeding ORM Models

Use `FakeData::seedOrm()` for one model and `FakeData::seedModels()` for a related model graph:

```
FakeData::seedOrm(User::class, count: 20, clear: true, seed: 42);
FakeData::seedModels([User::class, Order::class], count: 20, clear: true, seed: 42);
```

`seedModels()` orders parents before children and clears in reverse order so foreign keys remain valid. Its `seed` controls the `FakeData` instance it creates. `seedTable()` deliberately has no seed argument because its generators belong to the caller; create `new FakeData(42)` and pass its methods in the field map, as shown above.

FakeData is suitable for tests, development, demos, and load data. It is not cryptographically secure: never use it to create production passwords, tokens, API keys, or other secrets.

When to Use It

- Populating a development database with realistic data
- Writing integration tests that need rows in the database
- Load testing with thousands of records
- Demos and screenshots that look real without using real data

---

15. Gotchas

1. Forgetting commit()

Problem: Queries run inside `startTransaction()`, but changes vanish on the next request.

Cause: No `commit()`. The transaction rolls back when the connection closes.

Fix: Always call `$db->commit()` on success. Use try/catch with `$db->rollback()` in the catch.

2. Connection String Formats

Problem: Database refuses to connect. Cryptic error about the connection string.

Cause: Missing port. A common mistake is `mysql://user:pass@host/db` without the port number.

Fix: Always include the port:

Engine	Default Port
PostgreSQL	5432

MySQL	3306
MSSQL	1433
Firebird	3050
SQLite	(file path, no port)

3. SQLite File Paths

Problem: SQLite creates a new empty database instead of using the existing one.

Cause: Wrong slash count. `sqlite://` (two slashes) instead of `sqlite:///` (three slashes). Or a relative path resolving to the wrong directory.

Fix: Three slashes for a relative path: `sqlite:///data/app.db`. Four slashes for absolute: `sqlite:////var/data/app.db`. The third slash separates the scheme. The fourth starts the absolute path.

4. Parameterised Queries with LIKE

Problem: `WHERE name LIKE :q` with `["q" => "%search%"]` works. `WHERE name LIKE '%:q%'` does not.

Cause: Parameters inside quotes are literal text, not placeholders.

Fix: Put the `%` wildcards in the parameter value: `["q" => "%" . $search . "%"]`. The SQL should be `WHERE name LIKE :q`.

5. Boolean Values in SQLite

Problem: You insert `true` or `false`. The database stores `1` or `0`. Reading it back gives integers, not booleans.

Cause: SQLite has no native boolean type. Booleans become integers.

Fix: Cast in PHP: `"in_stock" => (bool) $row["in_stock"]`. Or accept that `1` and `0` work as truthy/falsy.

6. Migration Already Applied

Problem: You edited a migration file and ran `tina4 migrate` again. Nothing changed.

Cause: Tina4 tracks applied migrations by filename in the `tina4_migration` table. Once applied, a migration will not run again regardless of content changes.

Fix: Create a new migration for schema changes. Never edit applied migrations. For early development, use `tina4 migrate:rollback` first, then `tina4 migrate` to reapply.

7. Down Migration Not Found During Rollback

Problem: `tina4 migrate:rollback` removes the tracking record but does not reverse the schema change.

Cause: The `.down.sql` file is missing, or its name does not match the migration. For `20260322143000_create_products_table.sql`, the down file must be `20260322143000_create_products_table.down.sql` -- same base name with `.down.sql` appended before the extension.

Fix: Always generate migrations with `tina4 generate migration` which creates both files automatically. If you created migration files manually, add the `.down.sql` file with the exact matching name.

8. fetch() Returns Empty Array, Not Null

Problem: `if ($result === null)` never matches, even when the table is empty.

Cause: `fetch()` always returns an array. Empty result is `[]`, not `null`. Only `fetchOne()` returns `null`.

Fix: Check with `if (empty($result))` or `if (count($result) === 0)`.

ORM

1. From SQL to Objects

The last chapter was raw SQL. It works. It also gets repetitive. Every insert demands an INSERT statement. Every update demands an UPDATE. Every fetch maps column names to array keys. Over and over.

Tina4's ORM turns database rows into PHP objects. Define a model class with properties. The ORM writes the SQL. It stays SQL-first -- you can drop to raw SQL at any moment -- but for the 90% case of CRUD operations, the ORM handles the grunt work.

Picture a blog. Authors, posts, comments. Authors own many posts. Posts own many comments. Comments belong to posts. Modelling these relationships with raw SQL means JOINs and manual foreign key management. The ORM makes this declarative.

ORM at a Glance: Four Languages, One Shape

The ORM does the same job in every Tina4 book. Define a model. Save it. Query it. Each language wears its own clothes (PHP uses typed properties, Python uses field class instances, Ruby uses a DSL, Node uses config objects), but the operations line up. If you know the API in one book, you can read the others.

Defining a Model

The same `Post` model with `id`, `title`, `body`, and `created_at`:

Python - field class instances on the class body:

```
from tina4_python.orm import ORM, IntegerField, StringField, DateTimeField

class Post(ORM):
    table_name = "posts"

    id = IntegerField(primary_key=True, auto_increment=True)
    title = StringField(required=True, max_length=200)
    body = StringField(default="")
    created_at = DateTimeField()
```

PHP - native typed properties:

```
<?php
use Tina4\ORM;

class Post extends ORM
{
    public string $tableName = "posts";

    public int $id;
    public string $title;
    public string $body = "";
    public string $createdAt;
```

```
}
```

Ruby - class-level DSL declarations:

```
class Post < Tina4::ORM
  table_name "posts"

  integer_field :id, primary_key: true, auto_increment: true
  string_field :title, nullable: false, length: 200
  string_field :body, default: ""
  datetime_field :created_at
end
```

Node.js (TypeScript) - config objects in a `static fields` block:

```
import { BaseModel } from "tina4-nodejs/orm";

export default class Post extends BaseModel {
  static tableName = "posts";
  static fields = {
    id:        { type: "integer" as const, primaryKey: true, autoIncrement: true },
    title:     { type: "string"  as const, required: true, maxLength: 200 },
    body:      { type: "string"  as const, default: "" },
    createdAt: { type: "datetime" as const },
  };
}
```

Common Query Operations

Same operation, four shapes:

Operation	Python	PHP	Ruby	Node.js
Find by primary key	`Post.find_by_id(1)`	`Post::findById(1)`	`Post.find_by_id(1)`	`Post.findById(1)`
Filter by attributes	`Post.find({"title": "x"})`	`Post::find(["title" => "x"])`	`Post.find(title: "x")`	`Post.find({ title: "x" })`
Raw SQL where clause	`Post.where("title = ?", ["x"])`	`(new Post())->where("title = ?", ["x"])`	`Post.where("title = ?", ["x"])`	`Post.where("title = ?", ["x"])`
Build and save	`Post.create(title="x")`	`Post::create(["title" => "x"])`	`Post.create(title: "x")`	`Post.create({ title: "x" })`
Save an instance	`post.save()`	`$post->save()`	`post.save`	`post.save()`
Fetch every row	`Post.all()`	`(new Post())->all()`	`Post.all`	`Post.all()`

Delete a record	`post.delete()`	`$post->delete()`	`post.delete`	`post.delete()`
Count rows	`Post.count()`	`(new Post())->count()`	`Post.count`	`Post.count()`

A few details worth noting. `find()` takes attribute names and applies the field map; `where()` takes raw SQL and skips translation. PHP needs `(new Post())` for instance methods like `where()` and `all()`; the rest are static. Ruby methods drop the parentheses by convention.

For full detail on field options, relationships, eager loading, soft delete, validation, and Auto-CRUD, read the rest of this chapter, which shows the API for the language of this book.

---

2. Defining a Model

Create a model file in `src/orm/`. Every `.php` file in that directory is auto-loaded.

Create `src/orm/Note.php`:

```
<?php

use Tina4\ORM;

class Note extends ORM
{
    public string $tableName = "notes";
    public string $primaryKey = "id";

    public int $id;
    public string $title;
    public string $content = "";
    public string $category = "general";
    public bool $pinned = false;
    public string $createdAt;
    public string $updatedAt;
}
```

A complete model. Here is what each piece does:

- `$tableName` -- the database table this model maps to. If omitted, the ORM uses the lowercase class name (e.g. `Contact` maps to `contact`).
- `$primaryKey` -- the primary key column name. Defaults to `"id"` if not specified.
- Each property uses native PHP type hints (`int`, `string`, `float`, `bool`) to define the column type.

Property Types

PHP Type	SQL Type	Description
`int`	`INTEGER`	Whole numbers
`string`	`VARCHAR(255)`	Text strings
`float`	`REAL` / `DOUBLE PRECISION`	Decimal numbers
`bool`	`INTEGER` (0/1)	True/False

For foreign keys, use `int`. There is no separate foreign key type -- the relationship is defined through `$hasMany`, `$hasOne`, and `$belongsTo` array properties instead.

Field Mapping

When your PHP property names do not match the database column names, use `$fieldMapping` to define the translation:

```
<?php

use Tina4\ORM;

class User extends ORM
{
    public string $tableName = "user_accounts";
    public array $fieldMapping = [
        "firstName" => "fname",      // PHP property => DB column
        "lastName"  => "lname",
        "emailAddress" => "email",
    ];

    public int $id;
    public string $firstName;
    public string $lastName;
    public string $emailAddress;
}
```

With this mapping, `$user->firstName` reads from and writes to the `fname` column. The ORM handles the conversion in both directions -- on `findById()`, `save()`, `select()`, and `toDict()`. This is useful with legacy databases or third-party schemas where you cannot rename the columns.

autoMap and Case Conversion

The `$autoMap` flag converts camelCase PHP property names to snake*case database column names. This matters in PHP because convention uses camelCase for properties, but most databases use snake*case for columns.

```
<?php

use Tina4\ORM;

class User extends ORM
{
    public string $tableName = "users";
```

```
    public bool $autoMap = true;  // firstName <-> first_name

    public int $id;
    public string $firstName;    // maps to first_name column
    public string $lastName;     // maps to last_name column
    public string $emailAddress; // maps to email_address column
}
```

When `$autoMap` is `true`, the ORM generates the `$fieldMapping` entries from your camelCase properties. Explicit entries in `$fieldMapping` take precedence over auto-generated ones.

---

3. createTable -- Schema from Models

You can create the database table directly from your model:

```
$note = new Note($db);
$note->createTable([
    'id'         => 'INTEGER PRIMARY KEY AUTOINCREMENT',
    'title'      => 'VARCHAR(200) NOT NULL',
    'content'    => 'TEXT DEFAULT ""',
    'category'   => 'VARCHAR(50) DEFAULT "general"',
    'pinned'     => 'INTEGER DEFAULT 0',
    'created_at' => 'DATETIME',
    'updated_at' => 'DATETIME',
]);
```

This generates and runs the CREATE TABLE SQL. It is good for development and testing. For production, use migrations (Chapter 5) for version-controlled schema changes.

If you call `createTable()` with no arguments, it creates a minimal table with just the primary key column.

---

4. CRUD Operations

save -- Create or Update

```
Router::post("/api/notes", function (Request $request, Response $response) {
    $note = new Note();
    $note->title = $request->body["title"];
    $note->content = $request->body["content"] ?? "";
    $note->category = $request->body["category"] ?? "general";
    $note->pinned = $request->body["pinned"] ?? false;
    $note->save();

    return $response(["message" => "Note created", "note" => $note->toDict()], 201);
});
```

`save()` detects whether the record is new (INSERT) or existing (UPDATE) based on whether the primary key has a value and whether the record exists in the database. It returns `$this` on success (fluent chaining), or `false` on failure.

create -- Build and Save in One Step

When you have an array of data ready, `create()` builds the model and saves it in one call:

```
$note = Note::create([
    "title"    => "Quick Note",
    "content"  => "Created in one step",
    "category" => "general",
]);
```

`create()` is a static method. It creates a new instance, populates it from the array, calls `save()`, and returns the saved instance.

findById -- Fetch One Record by Primary Key

```
Router::get("/api/notes/{id}", function (Request $request, Response $response) {
    $note = Note::findById($request->params["id"]);

    if ($note === null) {
        return $response(["error" => "Note not found"], 404);
    }

    return $response($note->toDict());
});
```

`findById()` takes a primary key value and returns a model instance, or `null` if no row matches. If soft delete is enabled, it excludes soft-deleted records.

Use `findOrFail()` when you want a `RuntimeException` raised instead of `null`:

```
$note = Note::findOrFail($id);  // Throws RuntimeException if not found
```

find -- Query by Filter Array

The `find()` method accepts an associative array of column-value pairs and returns an array of matching records:

```
// Find all notes in the "work" category
$workNotes = Note::find(["category" => "work"]);

// Find with pagination and ordering
$recent = Note::find(["pinned" => true], limit: 10, orderBy: "created_at DESC");

// Find all records (no filter)
$allNotes = Note::find();
```

find() vs where() -- naming convention

The two query methods have a deliberate difference in how they handle column names:

- `find($filter)` uses **PHP property names**. The ORM translates them via `$fieldMapping` or `$autoMap`.
- `where($sql)` uses **raw DB column names** in the SQL string. No translation is done.

```
// find() -- use PHP property names
$accounts = (new Account())->find(["accountNo" => "A001"]);    // translates to ACCOUNTNO = ?

// where() -- use DB column names directly in the SQL
$accounts = (new Account())->where("ACCOUNTNO = ?", ["A001"]); // raw SQL, no translation
```

> ***Warning:*** *Mixing up the two is a common source of bugs. Use* `find()` *for portability and* `where()` *when you need full SQL control. Never pass DB column names to* `find()`*, and never use PHP property names as column names in* `where()`*.*

where -- Query with SQL Conditions

For more complex queries, `where()` takes a SQL WHERE clause with `?` placeholders:

```
$notes = (new Note())->where("category = ?", ["work"]);
```

delete -- Remove a Record

```
Router::delete("/api/notes/{id}", function (Request $request, Response $response) {
    $note = Note::findById($request->params["id"]);

    if ($note === null) {
        return $response(["error" => "Note not found"], 404);
    }

    $note->delete();

    return $response(null, 204);
});
```

Listing Records

```
Router::get("/api/notes", function (Request $request, Response $response) {
    $category = $request->query["category"] ?? null;

    if ($category) {
        $notes = (new Note())->where("category = ?", [$category]);
        return $response([
            "notes" => array_map(fn(Note $n) => $n->toDict(), $notes),
            "count" => count($notes),
        ]);
    }

    $notes = (new Note())->all();

    return $response([
        "notes" => array_map(fn(Note $n) => $n->toDict(), $notes),
        "count" => count($notes),
    ]);
});
```

`where()` takes a WHERE clause with `?` placeholders and an array of parameters. It returns a `ModelCollection` (covered just below). `all()` also returns a `ModelCollection`. Both support pagination:

```
// With pagination
$notes = (new Note())->where("category = ?", ["work"], limit: 20, offset: 40);

// Fetch all with pagination
$result = (new Note())->all(limit: 20, offset: 0);

// SQL-first query -- full control over the SQL
$notes = (new Note())->select(
    "SELECT * FROM notes WHERE pinned = ? ORDER BY created_at DESC",
    [1], limit: 20, offset: 0
```

```
);
```

ModelCollection -- The Page and the Total

Pagination hides an awkward gap. You ask for 20 rows and you get 20 rows, but a pager needs to say "page 3 of 13", and that means knowing the total number of matching rows, not the 20 sitting on this page. The old answer was a second `COUNT(*)` query with the same filter written out again by hand.

Tina4 closes the gap. `where()`, `select()`, `find()` (the filter form), `all()`, and `withTrashed()` all return a `ModelCollection`. It stays array-compatible: `foreach` it, index it with `$rows[0]`, `count()` it, and `json_encode()` it exactly as before. It just carries one extra thing: the total for the filter, independent of `limit` and `offset`.

```
$rows = (new User())->where("active = ?", [1], limit: 20, offset: 40);  // a page of up to 20 mo
$rows->getTotalRecords();   // 250 -- the whole matching set, ignoring limit/offset
```

That total is free. Every one of those methods already runs a `COUNT(*)` probe when it fetches the page, and the ORM used to hydrate the models and throw the count away. `ModelCollection` keeps it instead, so `getTotalRecords()` fires no second query. It is a method, not a `count` property, on purpose, and it reads the same as the accessor on `DatabaseResult`.

PHP is the one framework where this changes the return type. A PHP `array` cannot carry a property, so the read methods now return an object instead of a bare `array`. The four interfaces above keep that invisible to ordinary code, but native `array_map()`, `array_filter()`, and `is_array()` no longer apply to it. When you need a plain array for those, call `->toArray()`:

```
$names = array_map(fn ($u) => $u->name, $rows->toArray());
```

The single-record finders are untouched. The primary-key finders and `selectOne()` still return one model or `null`.

Call `toPaginate()` for a ready-made pagination envelope. It hands back the same seven keys as `$db->fetch(...)->toPaginate()`, so a route paginates the same way whether the data came through the ORM or through raw SQL:

```
Router::get("/api/notes", function (Request $request, Response $response) {
    $page = (new Note())->where("category = ?", ["work"], limit: 20, offset: 40);
    return $response->json($page->toPaginate());
    // {"records": [...20...], "total": 250, "page": 3, "per_page": 20,
    //  "total_pages": 13, "limit": 20, "offset": 40}
});
```

The keys (`records`, `total`, `page`, `per_page`, `total_pages`, `limit`, `offset`) are snake_case and identical in all four frameworks, so a client reads the same JSON everywhere.

selectOne -- Fetch a Single Record by SQL

When you need exactly one record from a custom SQL query:

```
$note = (new Note())->selectOne("SELECT * FROM notes WHERE slug = ?", ["my-note"]);
```

Returns a model instance or `null`.

load -- Populate an Existing Instance

The `load()` method fills an existing model instance from the database:

```
$note = new Note();
$note->id = 42;
$note->load();  // Loads data for id=42

// Or with a filter string
$note = new Note();
$note->load("slug = ?", ["my-note"]);
```

Returns `true` if a record was found, `false` otherwise.

count -- Count Records

```
$total = (new Note())->count();
$workCount = (new Note())->count("category = ?", ["work"]);
```

Respects soft delete -- only counts non-deleted records.

---

5. toDict, toJson, and Other Serialisation

toDict

Convert a model instance to an associative array (keyed by property names):

```
$note = Note::findById(1);

$data = $note->toDict();
// ["id" => 1, "title" => "Shopping List", "content" => "Milk, eggs", "category" => "personal",
```

The `$include` parameter adds relationship data to the output (see Eager Loading below). Pass an array of relationship names:

```
// Include relationships in the dict
$data = $note->toDict(["comments"]);
```

toJson

Convert directly to a JSON string:

```
$jsonString = $note->toJson();
// '{"id": 1, "title": "Shopping List", ...}'
```

toArray vs toDict

This distinction matters. `toDict()` returns an associative array with property names as keys. `toArray()` returns an indexed array of values with keys stripped:

```
$note = Note::findById(1);

$note->toDict();
// ["id" => 1, "title" => "Shopping List", "content" => "Milk, eggs"]

$note->toArray();
// [1, "Shopping List", "Milk, eggs"]
```

toObject() -- Returns a stdClass

`toObject()` returns a PHP `stdClass` object, not an array. This differs from `toDict()` which returns an associative array:

```
$user = (new User())->findById(1);

$obj = $user->toObject();
var_dump(is_object($obj));  // bool(true)
echo $obj->name;            // access as object property

$arr = $user->toDict();
var_dump(is_array($arr));   // bool(true)
echo $arr["name"];          // access as array key
```

All Serialisation Methods

Method	Returns	Description
`toDict($include)`	`array`	Associative array, keyed by PHP property names
`toAssoc($include)`	`array`	Alias for `toDict()`
`toObject()`	`stdClass`	PHP object (NOT an array) - properties match PHP property names
`toJson($include)`	`string`	JSON string
`toArray()`	`array`	Indexed array of values (no keys)
`toList()`	`array`	Alias for `toArray()`

---

6. Relationships

$foreignKeys - Auto-Wired Relationships

Declaring `public array $foreignKeys = ['user_id' => 'User']` on a model automatically wires both sides of the relationship. The declaring model gets a `belongsTo` accessor (the column name with `_id` stripped), and the referenced model gets a `hasMany` accessor (the declaring class name lowercased with `s` appended).

```
<?php
use Tina4\ORM;

class User extends ORM
{
    public string $tableName = "users";
    public string $primaryKey = "id";
}

class Post extends ORM
```

```
{
    public string $tableName = "posts";
    public string $primaryKey = "id";

    // Auto-wires $post->user (belongs_to) and $user->posts (has_many)
    public array $foreignKeys = [
        'user_id' => 'User',
    ];
}
```

With just the `$foreignKeys` array, both sides are accessible:

```
$post = new Post($db);
$post->load('id = 1');
echo $post->user->name;              // "Alice"

$user = new User($db);
$user->load('id = 1');
foreach ($user->posts as $post) {
    echo $post->title . "\n";
}
```

For a custom `has_many` key, use the extended form:

```
public array $foreignKeys = [
    'user_id' => ['model' => 'User', 'related_name' => 'blog_posts'],
];
// $user->blog_posts instead of $user->posts
```

hasMany

An author has many posts:

Create `src/orm/Author.php`:

```
<?php

use Tina4\ORM;

class Author extends ORM
{
    public string $tableName = "authors";

    public int $id;
    public string $name;
    public string $email;
    public string $bio = "";
    public string $createdAt;
}
```

Create `src/orm/BlogPost.php`:

```
<?php

use Tina4\ORM;

class BlogPost extends ORM
{
    public string $tableName = "posts";

    public int $id;
```

```
    public int $authorId;
    public string $title;
    public string $slug;
    public string $content = "";
    public string $status = "draft";
    public string $createdAt;
    public string $updatedAt;
}
```

Now use `hasMany()` to get an author's posts:

```
Router::get("/api/authors/{id}", function (Request $request, Response $response) {
    $author = Author::findById($request->params["id"]);

    if ($author === null) {
        return $response(["error" => "Author not found"], 404);
    }

    $posts = $author->hasMany(BlogPost::class, "author_id");

    $data = $author->toDict();
    $data["posts"] = array_map(fn(BlogPost $p) => $p->toDict(), $posts);

    return $response($data);
});
```

```
{
  "id": 1,
  "name": "Alice",
  "email": "alice@example.com",
  "bio": "Tech writer",
  "posts": [
    {"id": 1, "title": "Getting Started with Tina4", "slug": "getting-started", "status": "publi
    {"id": 2, "title": "Advanced Routing", "slug": "advanced-routing", "status": "draft"}
  ]
}
```

hasOne

A user has one profile:

```
$profile = $user->hasOne(Profile::class, "user_id");
```

Returns a single model instance or `null`.

belongsTo

A post belongs to an author:

```
Router::get("/api/posts/{id}", function (Request $request, Response $response) {
    $post = BlogPost::findById($request->params["id"]);

    if ($post === null) {
        return $response(["error" => "Post not found"], 404);
    }

    $author = $post->belongsTo(Author::class, "author_id");

    $data = $post->toDict();
    $data["author"] = $author ? $author->toDict() : null;
```

```
    return $response($data);
});
```

```
{
  "id": 1,
  "authorId": 1,
  "title": "Getting Started with Tina4",
  "slug": "getting-started",
  "content": "...",
  "status": "published",
  "author": {
    "id": 1,
    "name": "Alice",
    "email": "alice@example.com"
  }
}
```

Declarative Relationships

Instead of calling relationship methods in every route, declare them as array properties on the model. Each entry is a two-element array: `[ClassName, foreign_key_column]`.

```
<?php

use Tina4\ORM;

class Author extends ORM
{
    public string $tableName = "authors";
    public array $hasMany = [
        ["BlogPost", "author_id"],
    ];

    public int $id;
    public string $name;
    public string $email;
}
```

```
<?php

use Tina4\ORM;

class BlogPost extends ORM
{
    public string $tableName = "posts";
    public array $hasOne = [
        ["Author", "author_id"],
    ];
    public array $hasMany = [
        ["Comment", "post_id"],
    ];

    public int $id;
    public int $authorId;
    public string $title;
    public string $content = "";
}
```

With declarative relationships, accessing the magic property triggers lazy loading. The property name is the lowercase class name (or the lowercase plural for `hasMany`):

```
$author = (new Author())->findById(1);
$posts  = $author->posts;   // Lazy-loads BlogPost records via the hasMany declaration

$post   = (new BlogPost())->findById(1);
$author = $post->author;    // Lazy-loads the related Author via hasOne
```

---

7. Eager Loading

Calling relationship methods inside a loop creates the N+1 problem. Load 10 authors. Call `hasMany(BlogPost::class, "author_id")` for each one. That fires 11 queries -- 1 for authors, 10 for posts. The page drags.

The `$include` parameter on `toDict()` and `selectOne()` solves this. It eager-loads relationships in bulk.

For models with declarative relationships (`$hasOne`, `$hasMany`, `$belongsTo` array properties), pass a list of relationship names:

```
// Eager load posts when serialising an author
$author = Author::findById(1);
$data = $author->toDict(["posts"]);

// Eager load author and comments when serialising a post
$post = BlogPost::findById(1);
$data = $post->toDict(["author", "comments"]);
```

Nested Eager Loading

Dot notation loads multiple levels deep:

```
// Load author with posts, and each post with its comments
$data = $author->toDict(["posts", "posts.comments"]);
```

Authors, their posts, and each post's comments. Three queries total instead of hundreds.

toDict with Nested Includes

When eager loading is active, `toDict()` embeds the related data:

```
$post = BlogPost::findById(1);
$data = $post->toDict(["author", "comments"]);
```

```
{
  "id": 1,
  "title": "Getting Started with Tina4",
  "author": {
    "id": 1,
    "name": "Alice",
    "email": "alice@example.com"
  },
  "comments": [
    {"id": 1, "body": "Great post!", "authorName": "Bob"}
  ]
}
```

---

8. Soft Delete

Sometimes a record needs to disappear from queries without leaving the database. Soft delete handles this. The row stays. A flag marks it as deleted. Queries skip it.

```
<?php

use Tina4\ORM;

class Task extends ORM
{
    public string $tableName = "tasks";
    public bool $softDelete = true;  // Enable soft delete

    public int $id;
    public string $title;
    public bool $completed = false;
    public int $isDeleted = 0;  // Required for soft delete (0 = active, 1 = deleted)
    public string $createdAt;
}
```

When `$softDelete` is `true`, the ORM changes its behaviour:

- `$task->delete()` sets `is_deleted` to `1` instead of running a DELETE query
- `Task::find()`, `(new Task)->where()`, and `Task::findById()` filter out records where `is_deleted = 1`
- `$task->restore()` sets `is_deleted` back to `0` and makes the record visible again
- `$task->forceDelete()` permanently removes the row from the database
- `(new Task)->withTrashed()` includes soft-deleted records in query results

The soft delete column is `is_deleted` (integer, 0 or 1). There is no `deleted_at` timestamp column -- just the flag.

Deleting and Restoring

```
// Soft delete -- sets is_deleted = 1, row stays in the database
$task = Task::findById(1);
$task->delete();

// Restore -- sets is_deleted = 0, record is visible again
$task->restore();

// Permanently delete -- removes the row, no recovery possible
$task->forceDelete();
```

`restore()` is the inverse of `delete()`. It sets `is_deleted` back to `0` and commits the change. The record reappears in all standard queries.

Including Soft-Deleted Records

Standard queries (`all()`, `where()`, `findById()`) exclude soft-deleted records. When you need to see everything -- for admin dashboards, audit logs, or data recovery -- use `withTrashed()`:

```
// All tasks, including soft-deleted ones
$allTasks = (new Task())->withTrashed();
```

```
// Soft-deleted tasks matching a condition
$deletedTasks = (new Task())->withTrashed("completed = ?", [1]);
```

`withTrashed()` accepts the same filter parameters as `where()`. The only difference: it ignores the `is_deleted` filter that standard queries apply.

Counting with Soft Delete

The `count()` method respects soft delete. It only counts non-deleted records:

```
$activeCount = (new Task())->count();
$activeWork = (new Task())->count("category = ?", ["work"]);
```

When to Use Soft Delete

Soft delete suits data that users might want to recover -- emails, documents, user accounts. It also serves audit requirements where regulations demand retention. For temporary data (sessions, cache entries, logs), hard delete keeps the table lean.

---

9. Auto-CRUD

Writing the same five REST endpoints for every model gets tedious. Auto-CRUD generates them from your model class. Define the model. Register it. Five routes appear.

The autoCrud Flag

The simplest approach -- set `$autoCrud = true` on your model class:

```
<?php

use Tina4\ORM;

class Note extends ORM
{
    public string $tableName = "notes";
    public bool $autoCrud = true;  // Generates REST endpoints automatically

    public int $id;
    public string $title;
    public string $content = "";
}
```

The moment PHP loads this class, the ORM registers it with AutoCrud. Five routes appear.

Manual Registration

You can also register models explicitly using the `AutoCrud` class:

```
use Tina4\AutoCrud;
use Tina4\Database\Database;

$db = Database::create("sqlite:///path/to/app.db");
$crud = new AutoCrud($db);
$crud->register(Note::class);
$crud->generateRoutes();
```

Both approaches produce the same result:

Method	Path	Description
`GET`	`/api/notes`	List all with pagination (`limit`, `offset` params)
`GET`	`/api/notes/{id}`	Get one by primary key
`POST`	`/api/notes`	Create a new record
`PUT`	`/api/notes/{id}`	Update a record
`DELETE`	`/api/notes/{id}`	Delete a record

The endpoint prefix derives from the table name. The `notes` table becomes `/api/notes`. Pass a custom prefix to the `AutoCrud` constructor to change it:

```
$crud = new AutoCrud($db, prefix: "/api/v2");
// Routes: /api/v2/notes, /api/v2/notes/{id}, etc.
```

Auto-Discovering Models

Rather than registering each model by hand, point `discover()` at your models directory. It scans every `.php` file, finds ORM subclasses, and registers them all:

```
use Tina4\AutoCrud;

$crud = new AutoCrud($db);
$crud->discover("src/orm");
$crud->generateRoutes();
```

Every ORM model in `src/orm/` gets five REST endpoints. No route files needed.

What the Generated Routes Do

GET /api/notes returns paginated results:

```
curl "http://localhost:7145/api/notes?limit=10&offset=0"
```

```
{
  "data": [
    {"id": 1, "title": "Shopping List", "content": "Milk, eggs", "category": "personal", "pinned
    {"id": 2, "title": "Sprint Plan", "content": "Review backlog", "category": "work", "pinned":
  ],
  "total": 2,
  "limit": 10,
  "offset": 0
}
```

POST /api/notes creates a record:

```
curl -X POST http://localhost:7145/api/notes \
  -H "Content-Type: application/json" \
  -d '{"title": "New Note", "content": "Created via auto-CRUD"}'
```

DELETE /api/notes/1 respects soft delete. If the model has `$softDelete = true`, the record is marked deleted instead of removed.

Sorting and Filtering

The list endpoint accepts `sort` and `filter` query parameters:

```
# Sort by name descending, then created_at ascending
curl "http://localhost:7145/api/notes?sort=-name,created_at"

# Filter by column values
curl "http://localhost:7145/api/notes?filter[category]=work"
```

The `-` prefix on a sort field means descending order.

Custom Routes Alongside Auto-CRUD

If you need special logic for one endpoint (custom validation, side effects, complex queries), define that route yourself and let Auto-CRUD handle the rest. Don't count on load order to settle which one answers, because the router follows two rules:

- Registering the **same method and the same path string** again replaces the earlier route, so whichever registers last wins.
- A **different pattern** for the same URL (`/api/notes/{id:int}` against Auto-CRUD's `/api/notes/{id}`) replaces nothing. Both stay registered, and the one registered first answers.

The dependable choice is a path of its own for the custom route, or leaving `$autoCrud` off for that model and writing its routes yourself.

Introspection

Check which models are registered:

```
$crud = new AutoCrud($db);
$crud->register(Note::class);
$registered = $crud->getModels();
// ["notes" => "Note"]
```

---

10. Database Connection

Binding the Database

The ORM needs a database connection. Three ways to provide one:

```
use Tina4\ORM;
use Tina4\Database\Database;

// Option 1: Bind the default connection on ORM
$db = Database::create("sqlite:///path/to/app.db");
ORM::bindDatabase($db);

// Option 2: Set via App
App::setDatabase($db);

// Option 3: Set TINA4_DATABASE_URL in .env (auto-bound, no call needed)
// TINA4_DATABASE_URL=sqlite:///path/to/app.db
```

Once the default database is bound, all ORM models resolve it. You can also pass a database adapter to a specific instance:

```
$note = new Note($db);
```

The resolution order is: instance `$_db` -> default `ORM::bindDatabase()` binding -> `App::getDatabase()` -> `Database::fromEnv()`. If none is found, the ORM throws a `RuntimeException`.

Named (secondary) connections

`bindDatabase()` can also register a **named** connection that a specific model points at, so one app can talk to several databases:

```
ORM::bindDatabase(
    Database::create("postgres://localhost:5432/analytics", username: "u", password: "p"),
    name: "analytics"
);

class Visit extends \Tina4\ORM {
    // A string selects a named connection registered above.
    public \Tina4\Database\DatabaseAdapter|string|null $_db = "analytics";
}
```

A missing named connection throws a clear error.

---

11. Raw SQL and DatabaseResult

When you drop below the ORM to execute raw SQL, `$db->fetch()` returns a `DatabaseResult` object. This object has a `->records` property containing the rows -- it is **not** a plain array.

```
$result = $db->fetch("SELECT * FROM users WHERE active = ?", [1]);

// WRONG -- $result is not an array:
foreach ($result['data'] as $row) { ... }

// RIGHT -- iterate over ->records:
foreach ($result->records as $row) {
    echo $row->name;
}
```

`$result->records` is an array of `stdClass` objects, one per row. Column names map to object properties.

```
// Fetch a single row
$result = $db->fetch("SELECT * FROM users WHERE id = ?", [42]);
if (!empty($result->records)) {
    $user = $result->records[0];
    echo $user->email;
}
```

> ***Note:*** *ORM methods (`findById()`, `find()`, `where()`, etc.) return model instances or arrays of model instances -- not `DatabaseResult`. The `DatabaseResult` object only appears when you call `$db->fetch()` directly.*

---

12. Scopes

Scopes are reusable query filters baked into the model. Register them with the `scope()` method, then call them as static methods:

```
<?php

use Tina4\ORM;

class BlogPost extends ORM
{
    public string $tableName = "posts";

    public int $id;
    public string $title;
    public string $status = "draft";
    public string $createdAt;
}

// Register scopes
(new BlogPost())->scope("published", "status = ?", ["published"]);
(new BlogPost())->scope("drafts", "status = ?", ["draft"]);
```

Use them in your routes:

```
Router::get("/api/posts/published", function (Request $request, Response $response) {
    $posts = BlogPost::published();
    return $response(["posts" => array_map(fn($p) => $p->toDict(), $posts)]);
});

Router::get("/api/posts/drafts", function (Request $request, Response $response) {
    $posts = BlogPost::drafts();
    return $response(["posts" => array_map(fn($p) => $p->toDict(), $posts)]);
});
```

Scopes accept `$limit` and `$offset` as arguments:

```
$recentPublished = BlogPost::published(10, 0);  // limit 10, offset 0
```

Scopes keep query logic in the model where it belongs. Route handlers stay thin.

---

13. Input Validation

Override the `validate()` method on your model to add validation rules:

```
<?php

use Tina4\ORM;

class Product extends ORM
{
    public string $tableName = "products";

    public int $id;
```

```
    public string $name;
    public string $sku;
    public float $price;
    public string $category;

    public function validate(): array
    {
        $errors = [];

        if (empty($this->name) || strlen($this->name) < 2) {
            $errors[] = "name: Must be at least 2 characters";
        }
        if (strlen($this->name) > 200) {
            $errors[] = "name: Must be at most 200 characters";
        }
        if (!preg_match('/^[A-Z]{2}-\d{4}$/', $this->sku ?? '')) {
            $errors[] = "sku: Must match pattern XX-0000 (e.g., EL-1234)";
        }
        if (($this->price ?? 0) < 0.01 || ($this->price ?? 0) > 999999.99) {
            $errors[] = "price: Must be between 0.01 and 999999.99";
        }
        if (!in_array($this->category ?? '', ["Electronics", "Kitchen", "Office", "Fitness"])) {
            $errors[] = "category: Must be one of: Electronics, Kitchen, Office, Fitness";
        }

        return $errors;
    }
}

Router::post("/api/products", function (Request $request, Response $response) {
    $product = new Product();
    $product->name = $request->body["name"] ?? "";
    $product->sku = $request->body["sku"] ?? "";
    $product->price = $request->body["price"] ?? 0;
    $product->category = $request->body["category"] ?? "";

    $errors = $product->validate();
    if (!empty($errors)) {
        return $response(["errors" => $errors], 400);
    }

    $product->save();
    return $response(["product" => $product->toDict()], 201);
});
```

If validation fails, `validate()` returns an array of error messages:

```
{
  "errors": [
    "name: Must be at least 2 characters",
    "sku: Must match pattern XX-0000 (e.g., EL-1234)",
    "price: Must be between 0.01 and 999999.99",
    "category: Must be one of: Electronics, Kitchen, Office, Fitness"
  ]
}
```

---

14. QueryBuilder Integration

ORM models provide a `query()` static method that returns a `QueryBuilder` pre-configured with the model's table name and database connection. This gives you a fluent API for building complex queries without writing raw SQL:

```
// Fluent query builder from ORM
$results = User::query()
    ->select("id", "name", "email")
    ->where("active = ?", [1])
    ->orderBy("name")
    ->limit(50)
    ->get();

// First matching record
$user = User::query()
    ->where("email = ?", ["alice@example.com"])
    ->first();

// Count
$total = User::query()
    ->where("role = ?", ["admin"])
    ->count();

// Check existence
$exists = User::query()
    ->where("email = ?", ["test@example.com"])
    ->exists();
```

See the QueryBuilder chapter for the full fluent API including joins, grouping, having, and MongoDB support.

---

15. Exercise: Build a Blog with Relationships

Build a blog API with authors, posts, and comments.

Requirements

- Create these models:

Author: `id`, `name` (required), `email` (required), `bio`, `created_at`

Post: `id`, `author_id` (integer foreign key), `title` (required, max 300), `slug` (required), `content`, `status` (choices: draft/published/archived, default draft), `created_at`, `updated_at`

Comment: `id`, `post_id` (integer foreign key), `author_name` (required), `author_email` (required), `body` (required, min 5 chars), `created_at`

- Build these endpoints:

Method	Path	Description
`POST`	`/api/authors`	Create an author
`GET`	`/api/authors/{id}`	Get author with their posts

POST	/api/posts	Create a post (requires author_id)
GET	/api/posts	List published posts with author info
GET	/api/posts/{id}	Get post with author and comments
POST	/api/posts/{id}/comments	Add comment to a post

---

16. Solution

Create `src/orm/Author.php`:

```
<?php

use Tina4\ORM;

class Author extends ORM
{
    public string $tableName = "authors";

    public int $id;
    public string $name;
    public string $email;
    public string $bio = "";
    public string $createdAt;

    public function validate(): array
    {
        $errors = [];
        if (empty($this->name) || strlen($this->name) < 2) {
            $errors[] = "name: Must be at least 2 characters";
        }
        if (empty($this->email)) {
            $errors[] = "email: Required";
        }
        return $errors;
    }
}
```

Create `src/orm/BlogPost.php`:

```
<?php

use Tina4\ORM;

class BlogPost extends ORM
{
    public string $tableName = "posts";

    public int $id;
    public int $authorId;
```

```
    public string $title;
    public string $slug;
    public string $content = "";
    public string $status = "draft";
    public string $createdAt;
    public string $updatedAt;

    public function validate(): array
    {
        $errors = [];
        if (empty($this->title)) {
            $errors[] = "title: Required";
        }
        if (strlen($this->title ?? '') > 300) {
            $errors[] = "title: Must be at most 300 characters";
        }
        if (empty($this->slug)) {
            $errors[] = "slug: Required";
        }
        if (!in_array($this->status ?? 'draft', ["draft", "published", "archived"])) {
            $errors[] = "status: Must be one of: draft, published, archived";
        }
        return $errors;
    }
}
```

Register a scope for published posts:

```
(new BlogPost())->scope("published", "status = ?", ["published"]);
```

Create `src/orm/Comment.php`:

```
<?php

use Tina4\ORM;

class Comment extends ORM
{
    public string $tableName = "comments";

    public int $id;
    public int $postId;
    public string $authorName;
    public string $authorEmail;
    public string $body;
    public string $createdAt;

    public function validate(): array
    {
        $errors = [];
        if (empty($this->authorName)) {
            $errors[] = "authorName: Required";
        }
        if (empty($this->authorEmail)) {
            $errors[] = "authorEmail: Required";
        }
        if (empty($this->body) || strlen($this->body) < 5) {
            $errors[] = "body: Must be at least 5 characters";
        }
        return $errors;
```

```
    }
}
```

Create `src/routes/blog.php`:

```
<?php

use Tina4\Router;
use Tina4\Request;
use Tina4\Response;

Router::post("/api/authors", function (Request $request, Response $response) {
    $author = new Author();
    $author->name = $request->body["name"] ?? "";
    $author->email = $request->body["email"] ?? "";
    $author->bio = $request->body["bio"] ?? "";

    $errors = $author->validate();
    if (!empty($errors)) {
        return $response(["errors" => $errors], 400);
    }

    $author->save();
    return $response(["author" => $author->toDict()], 201);
});

Router::get("/api/authors/{id}", function (Request $request, Response $response) {
    $author = Author::findById($request->params["id"]);

    if ($author === null) {
        return $response(["error" => "Author not found"], 404);
    }

    $posts = $author->hasMany(BlogPost::class, "author_id");

    $data = $author->toDict();
    $data["posts"] = array_map(fn(BlogPost $p) => $p->toDict(), $posts);

    return $response($data);
});

Router::post("/api/posts", function (Request $request, Response $response) {
    $body = $request->body;

    // Verify author exists
    $author = Author::findById($body["author_id"] ?? 0);
    if ($author === null) {
        return $response(["error" => "Author not found"], 404);
    }

    $post = new BlogPost();
    $post->authorId = $body["author_id"];
    $post->title = $body["title"] ?? "";
    $post->slug = $body["slug"] ?? "";
    $post->content = $body["content"] ?? "";
    $post->status = $body["status"] ?? "draft";

    $errors = $post->validate();
    if (!empty($errors)) {
```

```
        return $response(["errors" => $errors], 400);
    }

    $post->save();
    return $response(["post" => $post->toDict()], 201);
});

Router::get("/api/posts", function (Request $request, Response $response) {
    $posts = BlogPost::published();
    $data = [];

    foreach ($posts as $p) {
        $postDict = $p->toDict();
        $author = $p->belongsTo(Author::class, "author_id");
        $postDict["author"] = $author ? $author->toDict() : null;
        $data[] = $postDict;
    }

    return $response(["posts" => $data, "count" => count($data)]);
});

Router::get("/api/posts/{id}", function (Request $request, Response $response) {
    $post = BlogPost::findById($request->params["id"]);

    if ($post === null) {
        return $response(["error" => "Post not found"], 404);
    }

    $author = $post->belongsTo(Author::class, "author_id");
    $comments = $post->hasMany(Comment::class, "post_id");

    $data = $post->toDict();
    $data["author"] = $author ? $author->toDict() : null;
    $data["comments"] = array_map(fn(Comment $c) => $c->toDict(), $comments);
    $data["commentCount"] = count($comments);

    return $response($data);
});

Router::post("/api/posts/{id}/comments", function (Request $request, Response $response) {
    $post = BlogPost::findById($request->params["id"]);

    if ($post === null) {
        return $response(["error" => "Post not found"], 404);
    }

    $comment = new Comment();
    $comment->postId = (int)$request->params["id"];
    $comment->authorName = $request->body["author_name"] ?? "";
    $comment->authorEmail = $request->body["author_email"] ?? "";
    $comment->body = $request->body["body"] ?? "";

    $errors = $comment->validate();
    if (!empty($errors)) {
        return $response(["errors" => $errors], 400);
    }

    $comment->save();
    return $response(["comment" => $comment->toDict()], 201);
```

```
});
```

---

17. Gotchas

1. Forgetting to call save()

Problem: You set properties on a model but the database does not change.

Cause: Setting `$note->title = "New Title"` only changes the PHP object. The database remains unchanged until you call `$note->save()`.

Fix: Call `save()` after modifying properties. Check the return value -- `save()` returns `$this` on success and `false` on failure.

2. findById() returns null

Problem: You call `Note::findById($id)` but get `null` instead of a note object.

Cause: `findById()` returns `null` when no row matches the given primary key. If soft delete is enabled, `findById()` also excludes soft-deleted records.

Fix: Check for `null` after `findById()`: `if ($note === null) { return $response(["error" => "Not found"], 404); }`. Use `findOrFail()` if you want a `RuntimeException` raised instead.

3. find() vs findById()

Problem: You call `Note::find(42)` expecting a single record, but get a type error.

Cause: `find()` takes an associative array filter (`find(["id" => 42])`), not a bare primary key value. For single-record lookups by primary key, use `findById(42)`.

Fix: Use `findById($id)` for primary key lookups. Use `find(["column" => $value])` for filter-based queries.

4. Static vs instance methods

Problem: You call `Note::where("category = ?", ["work"])` but get an error.

Cause: `where()`, `all()`, `select()`, `selectOne()`, `count()`, and `withTrashed()` are instance methods. `findById()`, `findOrFail()`, `find()`, `create()`, and `query()` are static methods.

Fix: For instance methods, create an instance first: `(new Note())->where(...)`. Static methods call directly: `Note::findById(...)`.

5. toDict() includes everything

Problem: `$user->toDict()` includes `passwordHash` in the API response.

Cause: `toDict()` includes all fields by default.

Fix: Build the response array manually, omitting sensitive fields: `["id" => $user->id, "name" => $user->name, "email" => $user->email]`. Or create a helper method on

your model class that returns only safe fields.

6. save() returns false on invalid data

Problem: `save()` returns `false` and the row never reaches the database, but nothing threw.

Cause: `save()` runs `validate()` first. If any field fails its rules, it refuses the write, logs the reason, and returns `false` instead of throwing. The same happens when the database rejects the write (a missing table, a `NOT NULL` column).

Fix: Check the return value. `$model->getError()` holds the reason. In a route handler, call `$errors = $model->validate()` before `save()` when you want to send the field errors back to the client as a `400`.

7. Soft delete column is is*deleted, not deleted*at

Problem: You add a `deleted_at` timestamp column expecting soft delete to use it.

Cause: Tina4's soft delete uses an `is_deleted` integer column (0 = active, 1 = deleted). It does not use a `deleted_at` timestamp.

Fix: Add an `is_deleted` integer column to your table with a default of `0`. Set `$softDelete = true` on the model.

8. N+1 query problem

Problem: Listing 100 authors with their posts runs 101 queries (1 for authors + 100 for posts), and the page loads slowly.

Cause: You call `$author->hasMany(BlogPost::class, "author_id")` inside a loop for each author.

Fix: Use declarative relationships with `$hasMany`/`$hasOne`/`$belongsTo` array properties and eager loading via `toDict(["posts"])`. Or fetch all posts in a single query and group them manually.

9. Auto-CRUD endpoint conflicts

Problem: Custom route at `/api/notes/{id}` stops working after registering Auto-CRUD for the Note model.

Cause: Both routes answer the same URL. With the identical path the route registered last replaces the other. With different patterns the route registered first answers.

Fix: Give the custom route a path of its own, or leave `$autoCrud` off for that model and write all its routes yourself.

10. Soft-deleted records appearing in queries

Problem: You soft-deleted a record, but queries still return it.

Cause: Soft delete requires the `$softDelete = true` flag on the model class and an `is_deleted` column in the database table. Without both, soft delete is inactive.

Fix: Verify both the `$softDelete = true` flag and the `is_deleted` column exist. The column stores `0` for active records and `1` for deleted ones.

11. $db->fetch() returns DatabaseResult, not an array

Problem: You call `$db->fetch(...)` and try to iterate over it as an array, getting a type error or empty results.

Cause: `$db->fetch()` returns a `DatabaseResult` object. The rows are in `$result->records`, not in `$result['data']` or `$result` itself.

Fix: Access `->records`:

```
// WRONG:
foreach ($result['data'] as $row) { ... }

// RIGHT:
foreach ($result->records as $row) { ... }
```

This only applies to raw `$db->fetch()` calls. ORM methods (`findById()`, `find()`, `where()`, etc.) return model instances directly.

QueryBuilder

1. Why a Query Builder?

Chapter 5 used raw SQL strings. Chapter 6 wrapped single-table CRUD in the ORM. Both work. Neither handles the messy middle ground well: multi-table joins, conditional filters, pagination, aggregation. You end up concatenating SQL fragments, juggling parameter arrays, and debugging mismatched question marks.

The QueryBuilder sits between raw SQL and the ORM. It builds SQL programmatically through a fluent, chainable API. Every method returns `$this`, so you compose queries left to right. When you need the SQL string, call `toSql()`. When you need results, call `get()`, `first()`, `count()`, or `exists()`.

No magic. No hidden queries. You see exactly what runs.

---

2. The Factory: `QueryBuilder::fromTable()`

The constructor is private. You create a QueryBuilder through the static `fromTable()` method:

```
<?php
use Tina4\QueryBuilder;
use Tina4\Database\Database;

$db = Database::create("sqlite:///data/app.db");

$qb = QueryBuilder::fromTable("users", $db);
```

Two arguments:

- `$table` -- The table name. This becomes the `FROM` clause.
- `$db` -- A `DatabaseAdapter` instance. Optional if you only need `toSql()`, required if you call any execution method.

The factory returns a fresh `QueryBuilder` instance. Chain methods directly:

```
$result = QueryBuilder::fromTable("users", $db)
    ->select("id", "name")
    ->where("active = ?", [1])
    ->get();
```

---

3. Selecting Columns: `select()`

By default the builder selects all columns (`*`). Narrow the selection with `select()`:

```
$qb = QueryBuilder::fromTable("products", $db)
    ->select("id", "name", "price");
```

Pass column names as separate string arguments. Expressions work too:

```
$qb = QueryBuilder::fromTable("orders", $db)
    ->select("customer_id", "SUM(total) as revenue");
```

If you never call `select()`, the builder uses `SELECT *`.

---

4. Filtering: `where()`

Add a `WHERE` condition with `where()`:

```
$qb = QueryBuilder::fromTable("users", $db)
    ->where("active = ?", [1]);
```

The first argument is the SQL condition. Use `?` placeholders for parameter binding. The second argument is an array of values that replace the placeholders in order.

Multiple `where()` Calls

Each call adds an `AND` condition:

```
$qb = QueryBuilder::fromTable("products", $db)
    ->where("price > ?", [10])
    ->where("category = ?", ["electronics"]);
```

Generates:

```
SELECT * FROM products WHERE price > ? AND category = ?
```

Parameters are accumulated in order: `[10, "electronics"]`.

Conditions Without Parameters

For conditions that do not need binding:

```
$qb = QueryBuilder::fromTable("users", $db)
    ->where("email IS NOT NULL");
```

The second argument defaults to an empty array.

---

5. OR Conditions: `orWhere()`

Use `orWhere()` to add an `OR` condition:

```
$qb = QueryBuilder::fromTable("products", $db)
    ->where("category = ?", ["books"])
    ->orWhere("category = ?", ["music"]);
```

Generates:

```
SELECT * FROM products WHERE category = ? OR category = ?
```

The first condition in the chain never has a connector prefix. The second gets `OR`. If you mix `where()` and `orWhere()`, they appear in order:

```
$qb = QueryBuilder::fromTable("products", $db)
    ->where("active = ?", [1])
    ->where("price > ?", [5])
```

```
    ->orWhere("featured = ?", [1]);
```

Generates:

```
SELECT * FROM products WHERE active = ? AND price > ? OR featured = ?
```

Standard SQL operator precedence applies. If you need grouping, write the grouped condition as a single string:

```
->where("(category = ? OR category = ?)", ["books", "music"])
```

---

6. Joins: `join()` and `leftJoin()`

Inner Join

```
$qb = QueryBuilder::fromTable("orders", $db)
    ->select("orders.id", "users.name", "orders.total")
    ->join("users", "users.id = orders.user_id");
```

Generates:

```
SELECT orders.id, users.name, orders.total FROM orders INNER JOIN users ON users.id = orders.use
```

Left Join

```
$qb = QueryBuilder::fromTable("users", $db)
    ->select("users.name", "orders.id as order_id")
    ->leftJoin("orders", "orders.user_id = users.id");
```

Generates:

```
SELECT users.name, orders.id as order_id FROM users LEFT JOIN orders ON orders.user_id = users.i
```

Multiple Joins

Chain as many as you need:

```
$qb = QueryBuilder::fromTable("orders", $db)
    ->select("orders.id", "users.name", "products.name as product")
    ->join("users", "users.id = orders.user_id")
    ->join("order_items", "order_items.order_id = orders.id")
    ->join("products", "products.id = order_items.product_id");
```

Joins appear in the SQL in the order you call them.

---

7. Grouping: `groupBy()`

```
$qb = QueryBuilder::fromTable("orders", $db)
    ->select("customer_id", "COUNT(*) as order_count")
    ->groupBy("customer_id");
```

Generates:

```
SELECT customer_id, COUNT(*) as order_count FROM orders GROUP BY customer_id
```

Call `groupBy()` multiple times for multiple columns:

```
$qb = QueryBuilder::fromTable("orders", $db)
    ->select("customer_id", "status", "COUNT(*) as cnt")
    ->groupBy("customer_id")
    ->groupBy("status");
```

Generates:

```
SELECT customer_id, status, COUNT(*) as cnt FROM orders GROUP BY customer_id, status
```

---

8. Filtering Groups: `having()`

`having()` filters aggregated results. It works like `where()` but applies after `GROUP BY`:

```
$qb = QueryBuilder::fromTable("orders", $db)
    ->select("customer_id", "SUM(total) as revenue")
    ->groupBy("customer_id")
    ->having("SUM(total) > ?", [1000]);
```

Generates:

```
SELECT customer_id, SUM(total) as revenue FROM orders GROUP BY customer_id HAVING SUM(total) > ?
```

Multiple `having()` calls are joined with `AND`:

```
$qb = QueryBuilder::fromTable("orders", $db)
    ->select("customer_id", "COUNT(*) as cnt", "SUM(total) as revenue")
    ->groupBy("customer_id")
    ->having("COUNT(*) > ?", [5])
    ->having("SUM(total) > ?", [500]);
```

Generates:

```
... HAVING COUNT(*) > ? AND SUM(total) > ?
```

---

9. Sorting: `orderBy()`

```
$qb = QueryBuilder::fromTable("products", $db)
    ->orderBy("price ASC");
```

Pass the column name and direction as a single string. Call multiple times for multi-column sorts:

```
$qb = QueryBuilder::fromTable("products", $db)
    ->orderBy("category ASC")
    ->orderBy("price DESC");
```

Generates:

```
SELECT * FROM products ORDER BY category ASC, price DESC
```

---

10. Pagination: `limit()`

```
$qb = QueryBuilder::fromTable("products", $db)
    ->limit(10);
```

With an offset for pagination:

```
$qb = QueryBuilder::fromTable("products", $db)
    ->limit(10, 20); // 10 rows, skip 20
```

The first argument is the maximum number of rows. The second is the offset (number of rows to skip). When you call `get()` without ever calling `limit()`, the builder defaults to 100 rows with offset 0.

---

11. Inspecting the SQL: `toSql()`

`toSql()` returns the constructed SQL string without executing it. Invaluable for debugging:

```
$qb = QueryBuilder::fromTable("orders", $db)
    ->select("customer_id", "SUM(total) as revenue")
    ->join("users", "users.id = orders.user_id")
    ->where("orders.status = ?", ["completed"])
    ->groupBy("customer_id")
    ->having("SUM(total) > ?", [1000])
    ->orderBy("revenue DESC")
    ->limit(10);

echo $qb->toSql();
```

Output:

```
SELECT customer_id, SUM(total) as revenue FROM orders INNER JOIN users ON users.id = orders.user
```

Note: `toSql()` does not include the `LIMIT`/`OFFSET` in the SQL string. Those values are passed to the database adapter's `fetch()` method as separate arguments.

`toSql()` does not require a database connection. Use it to build SQL strings for logging, testing, or manual inspection without connecting to a database.

---

12. Execution Methods

Four methods execute the query against the database. All require a `DatabaseAdapter` passed to `fromTable()`.

`get()` -- Fetch All Matching Rows

```
$result = QueryBuilder::fromTable("users", $db)
    ->where("active = ?", [1])
    ->orderBy("name ASC")
    ->limit(25)
    ->get();
```

Returns the raw result from `DatabaseAdapter::fetch()`. Access rows through `$result['data']`:

```
foreach ($result['data'] as $row) {
    echo $row['name'] . "\n";
}
```

`first()` -- Fetch a Single Row

```
$user = QueryBuilder::fromTable("users", $db)
    ->where("email = ?", ["alice@example.com"])
    ->first();

if ($user !== null) {
    echo $user['name'];
}
```

Returns an associative array for the first matching row, or `null` if nothing matches. Internally sets the limit to 1.

`count()` -- Count Matching Rows

```
$total = QueryBuilder::fromTable("orders", $db)
    ->where("status = ?", ["pending"])
    ->count();

echo "Pending orders: {$total}";
```

Returns an integer. Internally replaces your column list with `COUNT(*) as cnt`, executes, and restores the original columns.

`exists()` -- Check for Any Match

```
$hasAdmin = QueryBuilder::fromTable("users", $db)
    ->where("role = ?", ["admin"])
    ->exists();

if ($hasAdmin) {
    echo "Admin exists";
}
```

Returns a boolean. Calls `count()` internally and checks if the result is greater than zero.

---

13. Chaining It All Together

Every method returns `$this`. Chain them in any order (though a logical order improves readability):

```
$topCustomers = QueryBuilder::fromTable("orders", $db)
    ->select("users.name", "users.email", "COUNT(*) as order_count", "SUM(orders.total) as total
    ->join("users", "users.id = orders.user_id")
    ->where("orders.created_at > ?", ["2025-01-01"])
    ->where("orders.status = ?", ["completed"])
    ->groupBy("users.name")
    ->groupBy("users.email")
    ->having("SUM(orders.total) > ?", [500])
    ->orderBy("total_spent DESC")
    ->limit(10)
    ->get();
```

This single chain:

- Joins orders with users.

- Filters to completed orders in 2025.
- Groups by customer.
- Keeps only customers who spent more than 500.
- Sorts by spending, highest first.
- Returns the top 10.

Using QueryBuilder in Routes

```
<?php
use Tina4\Router;
use Tina4\Request;
use Tina4\Response;
use Tina4\QueryBuilder;

Router::get("/api/products", function (Request $request, Response $response) {
    global $db;

    $qb = QueryBuilder::fromTable("products", $db)
        ->select("id", "name", "price", "category")
        ->where("active = ?", [1])
        ->orderBy("name ASC")
        ->limit(50);

    $result = $qb->get();

    return $response($result['data']);
});
```

Dynamic Filters

Build queries conditionally based on request parameters:

```
Router::get("/api/products/search", function (Request $request, Response $response) {
    global $db;

    $qb = QueryBuilder::fromTable("products", $db)
        ->select("id", "name", "price", "category")
        ->where("active = ?", [1]);

    if (!empty($request->query["category"])) {
        $qb->where("category = ?", [$request->query["category"]]);
    }

    if (!empty($request->query["min_price"])) {
        $qb->where("price >= ?", [(float)$request->query["min_price"]]);
    }

    if (!empty($request->query["max_price"])) {
        $qb->where("price <= ?", [(float)$request->query["max_price"]]);
    }

    $sortBy = $request->query["sort"] ?? "name ASC";
    $qb->orderBy($sortBy);

    $limit = (int)($request->query["limit"] ?? 25);
    $offset = (int)($request->query["page"] ?? 0) * $limit;
    $qb->limit($limit, $offset);
```

```
    $result = $qb->get();

    return $response($result['data']);
});
```

This is where the builder shines. Conditional filters with raw SQL require clumsy `if` blocks that splice strings and shuffle parameter arrays. With the builder, each `->where()` call simply appends a condition. No index juggling. No string concatenation.

---

14. ORM Integration

ORM models can expose a `query()` static method that returns a QueryBuilder pre-configured with the model's table and the global database connection:

```
class User extends \Tina4\ORM
{
    public string $tableName = "users";
    public string $primaryKey = "id";

    public int $id;
    public string $name;
    public string $email;
    public bool $active = true;

    public static function query(): QueryBuilder
    {
        global $db;
        return QueryBuilder::fromTable("users", $db);
    }
}
```

Then query through the model:

```
$activeUsers = User::query()
    ->where("active = ?", [1])
    ->orderBy("name ASC")
    ->get();

$count = User::query()
    ->where("role = ?", ["admin"])
    ->count();

$first = User::query()
    ->where("email = ?", ["alice@example.com"])
    ->first();
```

This keeps the ORM for simple CRUD (`save()`, `load()`, `delete()`) and the QueryBuilder for complex reads. Both use the same table. Both use parameterised queries. Pick the tool that fits the job.

---

15. NoSQL: MongoDB Queries

The QueryBuilder can generate MongoDB-compatible query documents with `toMongo()`. This returns an associative array containing the filter, projection, sort, limit, and skip -- ready to pass to the MongoDB PHP driver.

Operator Mapping

SQL Operator	MongoDB Operator
`=`	Exact match
`!=`	`$ne`
`>`	`$gt`
`<`	`$lt`
`>=`	`$gte`
`<=`	`$lte`
`LIKE`	`$regex`
`IN`	`$in`
`IS NULL`	`$exists: false`
`IS NOT NULL`	`$exists: true`

Example

```
$query = QueryBuilder::fromTable("users")
    ->select("name", "email")
    ->where("age > ?", [25])
    ->where("status = ?", ["active"])
    ->orderBy("name ASC")
    ->limit(10, 5);

$mongo = $query->toMongo();
```

The returned array:

```
[
    "filter" => ["age" => ['$gt' => 25], "status" => "active"],
    "projection" => ["name" => 1, "email" => 1],
    "sort" => ["name" => 1],
    "limit" => 10,
    "skip" => 5,
]
```

Pass it directly to the MongoDB driver:

```
$collection = $client->selectCollection("mydb", "users");
$cursor = $collection->find(
    $mongo["filter"],
    [
        "projection" => $mongo["projection"],
```

```
        "sort" => $mongo["sort"],
        "limit" => $mongo["limit"],
        "skip" => $mongo["skip"],
    ]
);
```

---

16. Gotchas

No Database Adapter

Calling `get()`, `first()`, `count()`, or `exists()` without a database adapter throws a `RuntimeException`:

```
$qb = QueryBuilder::fromTable("users"); // no $db
$qb->get(); // RuntimeException: QueryBuilder: No database adapter provided.
```

Fix: pass a `DatabaseAdapter` as the second argument to `fromTable()`.

Parameter Order Matters

Parameters are accumulated in the order you call `where()`, `orWhere()`, and `having()`. The `?` placeholders in your conditions must align with the parameter arrays you pass:

```
// Correct
->where("price > ? AND price < ?", [10, 100])

// Also correct -- two separate calls
->where("price > ?", [10])
->where("price < ?", [100])
```

Both produce the same result. The second form is easier to read and easier to make conditional.

Default Limit

If you call `get()` without calling `limit()`, the builder defaults to fetching 100 rows with offset 0. This prevents accidental full-table scans on large datasets. Set an explicit `limit()` if you need more:

```
->limit(500) // fetch up to 500 rows
```

`toSql()` Does Not Include LIMIT

The `LIMIT` and `OFFSET` are not part of the SQL string returned by `toSql()`. They are passed as separate arguments to the database adapter's `fetch()` method. This is by design -- different database engines handle pagination differently, and the adapter takes care of the dialect-specific syntax.

`count()` Temporarily Replaces Columns

`count()` swaps your selected columns with `COUNT(*) as cnt` for the query, then restores the original columns. This means calling `toSql()` after `count()` still shows your original columns, not the count expression.

OR Precedence

SQL evaluates `AND` before `OR`. This query:

```
->where("active = ?", [1])
->where("price > ?", [5])
->orWhere("featured = ?", [1])
```

Means `(active = 1 AND price > 5) OR featured = 1`, not `active = 1 AND (price > 5 OR featured = 1)`. If you need the second interpretation, group the condition manually:

```
->where("active = ?", [1])
->where("(price > ? OR featured = ?)", [5, 1])
```

---

16. Exercise: Product Search API

Build a product search endpoint that uses QueryBuilder for all database access.

Setup

Create the migration file `migrations/003_create_products.sql`:

```
CREATE TABLE IF NOT EXISTS products (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name VARCHAR(255) NOT NULL,
    category VARCHAR(100),
    price REAL DEFAULT 0.00,
    active INTEGER DEFAULT 1,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO products (name, category, price) VALUES ('PHP in Action', 'books', 29.99);
INSERT INTO products (name, category, price) VALUES ('Mechanical Keyboard', 'electronics', 149.0
INSERT INTO products (name, category, price) VALUES ('Standing Desk', 'furniture', 499.00);
INSERT INTO products (name, category, price) VALUES ('USB-C Hub', 'electronics', 45.00);
INSERT INTO products (name, category, price) VALUES ('Code Complete', 'books', 35.00);
INSERT INTO products (name, category, price) VALUES ('Monitor Light', 'electronics', 65.00);
INSERT INTO products (name, category, price) VALUES ('Desk Mat', 'furniture', 25.00);
INSERT INTO products (name, category, price) VALUES ('Clean Code', 'books', 32.00);
```

The Route

Create `src/routes/productSearch.php`:

```
<?php
use Tina4\Router;
use Tina4\Request;
use Tina4\Response;
use Tina4\QueryBuilder;

/**
 * @description Search products with filters
 * @tags Products
 * @queryParam category string Filter by category
 * @queryParam min_price number Minimum price
 * @queryParam max_price number Maximum price
 * @queryParam search string Search by name
```

```
 * @queryParam sort string Sort field and direction (e.g. "price DESC")
 * @queryParam limit int Results per page (default 10)
 * @queryParam page int Page number (default 0)
 * @example /api/products/search?category=books&min_price=20&sort=price ASC
 */
Router::get("/api/products/search", function (Request $request, Response $response) {
    global $db;

    try {
        $qb = QueryBuilder::fromTable("products", $db)
            ->select("id", "name", "category", "price")
            ->where("active = ?", [1]);

        // Category filter
        if (!empty($request->query["category"])) {
            $qb->where("category = ?", [$request->query["category"]]);
        }

        // Price range
        if (!empty($request->query["min_price"])) {
            $qb->where("price >= ?", [(float)$request->query["min_price"]]);
        }
        if (!empty($request->query["max_price"])) {
            $qb->where("price <= ?", [(float)$request->query["max_price"]]);
        }

        // Name search
        if (!empty($request->query["search"])) {
            $qb->where("name LIKE ?", ["%" . $request->query["search"] . "%"]);
        }

        // Sorting
        $sort = $request->query["sort"] ?? "name ASC";
        $qb->orderBy($sort);

        // Pagination
        $limit = (int)($request->query["limit"] ?? 10);
        $page = (int)($request->query["page"] ?? 0);
        $qb->limit($limit, $page * $limit);

        // Get total count (separate query)
        $countQb = QueryBuilder::fromTable("products", $db)
            ->where("active = ?", [1]);

        if (!empty($request->query["category"])) {
            $countQb->where("category = ?", [$request->query["category"]]);
        }
        if (!empty($request->query["min_price"])) {
            $countQb->where("price >= ?", [(float)$request->query["min_price"]]);
        }
        if (!empty($request->query["max_price"])) {
            $countQb->where("price <= ?", [(float)$request->query["max_price"]]);
        }
        if (!empty($request->query["search"])) {
            $countQb->where("name LIKE ?", ["%" . $request->query["search"] . "%"]);
        }

        $total = $countQb->count();
        $result = $qb->get();
```

```
        return $response([
            "data" => $result['data'] ?? [],
            "total" => $total,
            "page" => $page,
            "limit" => $limit,
            "pages" => ceil($total / $limit),
        ]);
    } catch (\Throwable $e) {
        \Tina4\Debug::message($e->getMessage(), TINA4_LOG_ERROR);
        return $response(["error" => "Search failed"], 500);
    }
});
```

Test It

```
# All active products
curl "http://localhost:7145/api/products/search"

# Books only
curl "http://localhost:7145/api/products/search?category=books"

# Electronics under $100
curl "http://localhost:7145/api/products/search?category=electronics&max_price=100"

# Search by name, sorted by price
curl "http://localhost:7145/api/products/search?search=Code&sort=price%20DESC"

# Page 2, 3 per page
curl "http://localhost:7145/api/products/search?limit=3&page=1"
```

Verify the SQL

Add a debug line to inspect what the builder generates:

```
\Tina4\Debug::message("SQL: " . $qb->toSql(), TINA4_LOG_DEBUG);
```

Check the log output. The SQL should match your filters exactly. No extra conditions. No missing joins. What you chain is what you get.

Summary

Method	Purpose	Returns
`QueryBuilder::fromTable($table, $db)`	Create a builder	`QueryBuilder`
`->select(...$columns)`	Set columns	`$this`
`->where($condition, $params)`	AND filter	`$this`
`->orWhere($condition, $params)`	OR filter	`$this`
`->join($table, $on)`	INNER JOIN	`$this`
`->leftJoin($table, $on)`	LEFT JOIN	`$this`
`->groupBy($column)`	GROUP BY	`$this`
`->having($expression, $params)`	HAVING filter	`$this`
`->orderBy($expression)`	ORDER BY	`$this`
`->limit($count, $offset)`	LIMIT / OFFSET	`$this`
`->toSql()`	Get SQL string	`string`
`->get()`	Fetch rows	`mixed`
`->first()`	Fetch one row	`?array`
`->count()`	Count rows	`int`
`->exists()`	Check existence	`bool`

Authentication

1. Locking the Door

Every endpoint built so far is public. Anyone with the URL can read, create, update, delete. That works for a tutorial. A real application needs to know who is making a request and whether they are allowed.

This chapter covers Tina4's authentication system: JWT tokens, password hashing, middleware-based route protection, CSRF tokens for forms, and session management.

---

2. JWT Tokens

Tina4 uses JSON Web Tokens for authentication. A JWT is a signed string containing a payload -- user ID, role, expiry. The server creates it at login. The client sends it with every request. The server verifies it without touching the database.

Generating a Token

```
<?php
use Tina4\Auth;

$payload = [
    "user_id" => 42,
    "email" => "alice@example.com",
    "role" => "admin"
];

$token = Auth::getToken($payload, $secret);
```

`getToken()` signs the payload with HS256 (HMAC-SHA256) using the provided secret. The `$secret` parameter is **required**. Returns a JWT string:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjo0MiwiZW1haWwiOiJhbGljZUBleGFtcGxlLmNvbSIsInJ
```

Three parts separated by dots: header, payload, signature. The signature ensures the token has not been tampered with.

Token Expiry

Tokens expire after 60 minutes by default. Configure in `.env`:

```
TINA4_TOKEN_LIMIT=60
```

Value in **minutes** (default: 60):

Value	Duration
`15`	15 minutes
`60`	1 hour (default)
`1440`	24 hours

`10080`	7 days

Validating a Token

```
$payload = Auth::validToken($token, $secret);
// Returns the payload array if valid, null if invalid or expired
```

`validToken()` returns the decoded payload on success, not a boolean. This lets you validate and read the token in one step. Returns `null` if the token is invalid or expired.

Reading the Payload

```
$payload = Auth::getPayload($token);
```

Returns the decoded payload **without validation** -- it just decodes the token. Note that `getPayload()` takes only the token string -- no secret is needed because it does not verify the signature:

```
[
    "user_id" => 42,
    "email" => "alice@example.com",
    "role" => "admin",
    "iat" => 1711112600,  // issued at (Unix timestamp)
    "exp" => 1711116200   // expires at (Unix timestamp)
]
```

If the token cannot be decoded: `null`.

> ***Important:*** `getPayload()` *does not verify the signature or check expiry. Use* `validToken()` *when you need to confirm the token is trustworthy.*

The Secret Key and Algorithm

Tina4 PHP uses **HS256** (HMAC-SHA256) for JWT signing. It uses only the standard library -- zero external dependencies.

Set the secret key in `.env`:

```
TINA4_SECRET=my-super-secret-key-at-least-32-chars
```

The `$secret` parameter is **required** on `getToken()` and `validToken()`. Pass it explicitly -- there is no automatic fallback. Read it from your `.env` in your route handler:

```
$secret = $_ENV["TINA4_SECRET"] ?? getenv("TINA4_SECRET");
```

`getPayload()` does not take a secret at all -- it decodes without verifying.

Guard this key. Anyone who has it can forge tokens.

---

3. Password Hashing

Passwords in plain text are a breach waiting to happen. Tina4 provides two functions for secure password handling.

Hashing a Password

```
use Tina4\Auth;

$hash = Auth::hashPassword("my-secure-password");
// Returns: "$2y$10$abc123...long-hash-string..."
```

Uses PBKDF2 from the standard library -- no external dependencies. Each hash includes a random salt. Hashing the same password twice produces different results.

Checking a Password

```
$isCorrect = Auth::checkPassword("my-secure-password", $storedHash);
// true if the password matches
```

Registration Example

```
<?php
use Tina4\Router;
use Tina4\Auth;
use Tina4\Database;

Router::post("/api/register", function ($request, $response) {
    $body = $request->body;

    if (empty($body["name"]) || empty($body["email"]) || empty($body["password"])) {
        return $response->json(["error" => "Name, email, and password are required"], 400);
    }

    if (strlen($body["password"]) < 8) {
        return $response->json(["error" => "Password must be at least 8 characters"], 400);
    }

    $db = Database::getConnection();

    $existing = $db->fetchOne("SELECT id FROM users WHERE email = :email", ["email" => $body["em
    if ($existing !== null) {
        return $response->json(["error" => "Email already registered"], 409);
    }

    $passwordHash = Auth::hashPassword($body["password"]);

    $db->execute(
        "INSERT INTO users (name, email, password_hash) VALUES (:name, :email, :hash)",
        [
            "name" => $body["name"],
            "email" => $body["email"],
            "hash" => $passwordHash
        ]
    );

    $user = $db->fetchOne("SELECT id, name, email FROM users WHERE id = last_insert_rowid()");

    return $response->json([
        "message" => "Registration successful",
        "user" => $user
    ], 201);
});
```

```
curl -X POST http://localhost:7145/api/register \
```

```
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "password": "securePass123"}'
```

```
{
  "message": "Registration successful",
  "user": {"id": 1, "name": "Alice", "email": "alice@example.com"}
}
```

---

4. The Login Flow

Client sends credentials. Server validates them. Server returns a JWT.

```
<?php
use Tina4\Router;
use Tina4\Auth;
use Tina4\Database;

/**
 * @noauth
 */
Router::post("/api/login", function ($request, $response) {
    $body = $request->body;

    if (empty($body["email"]) || empty($body["password"])) {
        return $response->json(["error" => "Email and password are required"], 400);
    }

    $db = Database::getConnection();

    $user = $db->fetchOne(
        "SELECT id, name, email, password_hash FROM users WHERE email = :email",
        ["email" => $body["email"]]
    );

    if ($user === null) {
        return $response->json(["error" => "Invalid email or password"], 401);
    }

    if (!Auth::checkPassword($body["password"], $user["password_hash"])) {
        return $response->json(["error" => "Invalid email or password"], 401);
    }

    $token = Auth::getToken([
        "user_id" => $user["id"],
        "email" => $user["email"],
        "name" => $user["name"]
    ]);

    return $response->json([
        "message" => "Login successful",
        "token" => $token,
        "user" => [
            "id" => $user["id"],
            "name" => $user["name"],
            "email" => $user["email"]
        ]
    ]);
```

```
});
```

The `@noauth` annotation is critical. The login endpoint must be public. You cannot require a token to get a token.

```
curl -X POST http://localhost:7145/api/login \
  -H "Content-Type: application/json" \
  -d '{"email": "alice@example.com", "password": "securePass123"}'
```

```
{
  "message": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": 1,
    "name": "Alice",
    "email": "alice@example.com"
  }
}
```

The client stores the token and sends it with subsequent requests.

---

5. Using Tokens in Requests

The token travels in the `Authorization` header with the `Bearer` prefix:

```
curl http://localhost:7145/api/profile \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
```

---

6. Protecting Routes

Auth Middleware

A reusable gate:

```
<?php
use Tina4\Auth;

function authMiddleware($request, $response, $next) {
    $authHeader = $request->header("Authorization") ?? "";

    if (empty($authHeader) || !str_starts_with($authHeader, "Bearer ")) {
        return $response->json(["error" => "Authorization header required"], 401);
    }

    $token = substr($authHeader, 7); // Remove "Bearer " prefix

    $payload = Auth::validToken($token);
    if ($payload === null) {
        return $response->json(["error" => "Invalid or expired token"], 401);
    }

    $request->user = $payload; // Attach user data to the request

    return $next($request, $response);
}
```

Applying Middleware to Routes

```
<?php
use Tina4\Router;

Router::get("/api/profile", function ($request, $response) {
    return $response->json([
        "user_id" => $request->user["user_id"],
        "email" => $request->user["email"],
        "name" => $request->user["name"]
    ]);
}, "authMiddleware");
```

```
# Without token -- 401
curl http://localhost:7145/api/profile
```

```
{"error":"Authorization header required"}
```

```
# With valid token -- 200
curl http://localhost:7145/api/profile \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
```

```
{
  "user_id": 1,
  "email": "alice@example.com",
  "name": "Alice"
}
```

Applying Middleware to a Group

```
Router::group("/api/admin", function () {

    Router::get("/stats", function ($request, $response) {
        return $response->json(["active_users" => 42]);
    });

    Router::get("/logs", function ($request, $response) {
        return $response->json(["logs" => []]);
    });

}, "authMiddleware");
```

Every route in the group requires a valid token.

---

7. @noauth and @secured Decorators

Two decorators for route-level authentication control. Introduced in Chapter 2. Here they are in context.

@noauth -- Skip Authentication

Public endpoints that bypass global or group-level auth:

```
/**
 * @noauth
 */
Router::get("/api/public/health", function ($request, $response) {
    return $response->json(["status" => "ok"]);
```

```
});

/**
 * @noauth
 */
Router::post("/api/login", function ($request, $response) {
    // Login logic
});

/**
 * @noauth
 */
Router::post("/api/register", function ($request, $response) {
    // Registration logic
});
```

@secured -- Require Authentication for GET Routes

POST, PUT, PATCH, DELETE are secured by default. GET is public. Use `@secured` to protect a GET route:

```
/**
 * @secured
 */
Router::get("/api/me", function ($request, $response) {
    return $response->json($request->user);
});
```

Role-Based Authorization

Combine auth middleware with role checks:

```
<?php
use Tina4\Auth;

function requireRole($role) {
    return function ($request, $response, $next) use ($role) {
        $authHeader = $request->header("Authorization") ?? "";
        if (empty($authHeader) || !str_starts_with($authHeader, "Bearer ")) {
            return $response->json(["error" => "Authorization required"], 401);
        }

        $token = substr($authHeader, 7);
        $payload = Auth::validToken($token);
        if ($payload === null) {
            return $response->json(["error" => "Invalid or expired token"], 401);
        }

        if (($payload["role"] ?? "") !== $role) {
            return $response->json(["error" => "Forbidden. Required role: " . $role], 403);
        }

        $request->user = $payload;
        return $next($request, $response);
    };
}
```

`requireRole()` returns a closure. Register the returned function as middleware:

```
$adminOnly = requireRole("admin");
```

```
Router::delete("/api/users/{id:int}", function ($request, $response) {
    return $response->json(["deleted" => true]);
}, $adminOnly);
```

---

8. CSRF Protection

For traditional form-based applications (not SPAs), Tina4 provides CSRF protection.

Generating a Token

Include the CSRF token in every form:

```
<form method="POST" action="/profile/update">
    {{ form_token() }}

    <div class="form-group">
        <label for="name">Name</label>
        <input type="text" name="name" id="name" value="{{ user.name }}">
    </div>

    <button type="submit">Update Profile</button>
</form>
```

`{{ form_token() }}` renders a hidden input:

```
<input type="hidden" name="formToken" value="abc123randomtoken456">
```

Validating the Token

Register `CsrfMiddleware` and it validates the `formToken` on every state-changing request automatically, with no manual check in the handler:

```
<?php
use Tina4\Router;
use Tina4\Middleware\CsrfMiddleware;

// Enable globally (or set TINA4_CSRF=true in .env)
Router::use(CsrfMiddleware::class);

Router::post("/profile/update", function ($request, $response) {
    // CsrfMiddleware already rejected the request with a 403 if the
    // formToken was missing or invalid - just process the form.
    return $response->redirect("/profile");
});
```

The middleware reads the token from `request->body["formToken"]` (or the `X-Form-Token` header), validates it with `Auth::validToken()`, and returns a `403 CSRF_INVALID` error if it is missing or invalid. The token is tied to the user's session. A malicious site cannot forge it.

When to Use CSRF Tokens

Use them for:

- HTML forms submitted by browsers
- POST/PUT/DELETE from server-rendered pages

Skip them for:

- API endpoints using JWT (the Bearer token proves intent)
- SPAs using `fetch()` with custom headers (the `Authorization` header cannot be set by cross-origin forms)

---

9. Sessions

Server-side sessions store per-user state between requests. Use JWTs for API authentication. Use sessions for stateful web pages.

Session Configuration

Set the backend in `.env`:

```
# File-based sessions (default)
TINA4_SESSION_BACKEND=file

# Redis
TINA4_SESSION_BACKEND=redis
TINA4_SESSION_REDIS_HOST=localhost
TINA4_SESSION_REDIS_PORT=6379

# MongoDB
TINA4_SESSION_BACKEND=mongodb
TINA4_SESSION_REDIS_HOST=localhost
TINA4_SESSION_REDIS_PORT=27017

# Valkey
TINA4_SESSION_BACKEND=valkey
TINA4_SESSION_REDIS_HOST=localhost
TINA4_SESSION_REDIS_PORT=6379
```

File sessions work out of the box. Redis or Valkey for multi-server production deployments where sessions must be shared across instances.

Using Sessions

Access session data through `$request->session`:

```
<?php
use Tina4\Router;

Router::post("/login-form", function ($request, $response) {
    // After validating credentials...
    $request->session["user_id"] = 42;
    $request->session["user_name"] = "Alice";
    $request->session["logged_in"] = true;

    return $response->redirect("/dashboard");
});

Router::get("/dashboard", function ($request, $response) {
    if (empty($request->session["logged_in"])) {
        return $response->redirect("/login");
```

```
    }

    return $response->render("dashboard.html", [
        "user_name" => $request->session["user_name"]
    ]);
});

Router::post("/logout", function ($request, $response) {
    $request->session = [];

    return $response->redirect("/login");
});
```

Session Options

```
TINA4_SESSION_TTL=3600       # Expires after 1 hour of inactivity
TINA4_SESSION_NAME=tina4_session  # Cookie name for the session ID
```

---

10. Exercise: Build Login, Register, and Profile

A complete authentication system. Registration, login, profile viewing, password changing.

Requirements

- Create a `users` table migration: `id`, `name`, `email` (unique), `password_hash`, `role` (default "user"), `created_at`
- Build these endpoints:

Method	Path	Auth	Description
`POST`	`/api/register`	@noauth	Create account. Validate name, email, password (min 8 chars).
`POST`	`/api/login`	@noauth	Login. Return JWT.
`GET`	`/api/profile`	secured	Get profile from token.
`PUT`	`/api/profile`	secured	Update name and email.
`PUT`	`/api/profile/password`	secured	Change password. Require current password.

- Create auth middleware that extracts the user from the JWT and attaches it to `$request->user`.

Test with:

```
# Register
curl -X POST http://localhost:7145/api/register \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "password": "securePass123"}'

# Login
curl -X POST http://localhost:7145/api/login \
  -H "Content-Type: application/json" \
  -d '{"email": "alice@example.com", "password": "securePass123"}'

# Save the token, then:

# Get profile
curl http://localhost:7145/api/profile \
  -H "Authorization: Bearer YOUR_TOKEN_HERE"

# Update profile
curl -X PUT http://localhost:7145/api/profile \
  -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice Smith"}'

# Change password
curl -X PUT http://localhost:7145/api/profile/password \
  -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  -H "Content-Type: application/json" \
  -d '{"current_password": "securePass123", "new_password": "evenMoreSecure456"}'

# No token (should fail)
curl http://localhost:7145/api/profile
```

---

11. Solution

Migration

Create `migrations/20260322160000_create_auth_users_table.sql`:

```
-- UP
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    role TEXT NOT NULL DEFAULT 'user',
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

-- DOWN
DROP TABLE IF EXISTS users;
```

```
tina4 migrate
```

Middleware

Create `src/routes/middleware.php`:

```php
<?php
use Tina4\Auth;

function authMiddleware($request, $response, $next) {
    $authHeader = $request->header("Authorization") ?? "";

    if (empty($authHeader) || !str_starts_with($authHeader, "Bearer ")) {
        return $response->json(["error" => "Authorization required. Send: Authorization: Bearer
    }

    $token = substr($authHeader, 7);

    $payload = Auth::validToken($token);
    if ($payload === null) {
        return $response->json(["error" => "Invalid or expired token. Please login again."], 401
    }

    $request->user = $payload;

    return $next($request, $response);
}
```

Routes

Create `src/routes/auth.php`:

```php
<?php
use Tina4\Router;
use Tina4\Auth;
use Tina4\Database;

/**
 * @noauth
 */
Router::post("/api/register", function ($request, $response) {
    $body = $request->body;

    $errors = [];
    if (empty($body["name"])) $errors[] = "Name is required";
    if (empty($body["email"])) $errors[] = "Email is required";
    if (empty($body["password"])) {
        $errors[] = "Password is required";
    } elseif (strlen($body["password"]) < 8) {
        $errors[] = "Password must be at least 8 characters";
    }

    if (!empty($errors)) {
        return $response->json(["errors" => $errors], 400);
    }

    $db = Database::getConnection();

    $existing = $db->fetchOne("SELECT id FROM users WHERE email = :email", ["email" => $body["em
    if ($existing !== null) {
        return $response->json(["error" => "Email already registered"], 409);
    }

    $hash = Auth::hashPassword($body["password"]);

    $db->execute(
```

```
        "INSERT INTO users (name, email, password_hash) VALUES (:name, :email, :hash)",
        ["name" => $body["name"], "email" => $body["email"], "hash" => $hash]
    );

    $user = $db->fetchOne("SELECT id, name, email, role, created_at FROM users WHERE id = last_i

    return $response->json(["message" => "Registration successful", "user" => $user], 201);
});

/**
 * @noauth
 */
Router::post("/api/login", function ($request, $response) {
    $body = $request->body;

    if (empty($body["email"]) || empty($body["password"])) {
        return $response->json(["error" => "Email and password are required"], 400);
    }

    $db = Database::getConnection();

    $user = $db->fetchOne(
        "SELECT id, name, email, password_hash, role FROM users WHERE email = :email",
        ["email" => $body["email"]]
    );

    if ($user === null || !Auth::checkPassword($body["password"], $user["password_hash"])) {
        return $response->json(["error" => "Invalid email or password"], 401);
    }

    $token = Auth::getToken([
        "user_id" => $user["id"],
        "email" => $user["email"],
        "name" => $user["name"],
        "role" => $user["role"]
    ]);

    return $response->json([
        "message" => "Login successful",
        "token" => $token,
        "user" => ["id" => $user["id"], "name" => $user["name"], "email" => $user["email"], "rol
    ]);
});

Router::get("/api/profile", function ($request, $response) {
    $db = Database::getConnection();

    $user = $db->fetchOne(
        "SELECT id, name, email, role, created_at FROM users WHERE id = :id",
        ["id" => $request->user["user_id"]]
    );

    if ($user === null) {
        return $response->json(["error" => "User not found"], 404);
    }

    return $response->json($user);
}, "authMiddleware");
```

```
Router::put("/api/profile", function ($request, $response) {
    $db = Database::getConnection();
    $body = $request->body;
    $userId = $request->user["user_id"];

    if (!empty($body["email"])) {
        $existing = $db->fetchOne(
            "SELECT id FROM users WHERE email = :email AND id != :id",
            ["email" => $body["email"], "id" => $userId]
        );
        if ($existing !== null) {
            return $response->json(["error" => "Email already in use by another account"], 409);
        }
    }

    $current = $db->fetchOne("SELECT * FROM users WHERE id = :id", ["id" => $userId]);

    $db->execute(
        "UPDATE users SET name = :name, email = :email WHERE id = :id",
        ["name" => $body["name"] ?? $current["name"], "email" => $body["email"] ?? $current["ema
    );

    $updated = $db->fetchOne("SELECT id, name, email, role, created_at FROM users WHERE id = :id

    return $response->json(["message" => "Profile updated", "user" => $updated]);
}, "authMiddleware");

Router::put("/api/profile/password", function ($request, $response) {
    $db = Database::getConnection();
    $body = $request->body;
    $userId = $request->user["user_id"];

    if (empty($body["current_password"]) || empty($body["new_password"])) {
        return $response->json(["error" => "Current password and new password are required"], 40
    }

    if (strlen($body["new_password"]) < 8) {
        return $response->json(["error" => "New password must be at least 8 characters"], 400);
    }

    $user = $db->fetchOne("SELECT password_hash FROM users WHERE id = :id", ["id" => $userId]);

    if (!Auth::checkPassword($body["current_password"], $user["password_hash"])) {
        return $response->json(["error" => "Current password is incorrect"], 401);
    }

    $newHash = Auth::hashPassword($body["new_password"]);

    $db->execute("UPDATE users SET password_hash = :hash WHERE id = :id", ["hash" => $newHash, "

    return $response->json(["message" => "Password changed successfully"]);
}, "authMiddleware");
```

Expected output for register: `201 Created` with user data.

Expected output for login: `200 OK` with token and user data.

Expected output for profile without token: `401 Unauthorized`.

Expected output for profile with token: `200 OK` with user data.

Expected output for wrong current password: `401 Unauthorized`.

Expected output for successful password change: `{"message":"Password changed successfully"}`.

---

12. Gotchas

1. Token Expiry Confusion

Problem: Tokens that worked yesterday return 401 today.

Cause: Default lifetime is 60 minutes (`TINA4_TOKEN_LIMIT=60`). After that, the token is invalid.

Fix: Issue a new token at login. For long-lived sessions, use refresh tokens: a short-lived access token (15 minutes) paired with a long-lived refresh token (7 days).

2. Secret Key Management

Problem: Tokens from one server fail on another. Or tokens stop working after deployment.

Cause: Each server generated its own `secrets/jwt.key`. Or the key was regenerated during deployment.

Fix: Set `TINA4_SECRET` in `.env` and use the same value across all servers. Store it in your secrets manager. Not in version control. Key change invalidates all tokens. Users must log in again.

3. CORS with Authentication

Problem: Frontend requests with `Authorization` header fail with CORS error, even with `TINA4_CORS_ORIGINS=*`.

Cause: The browser sends a preflight `OPTIONS` request. The server must respond with CORS headers including `Access-Control-Allow-Headers: Authorization`.

Fix: Tina4 handles preflight requests. Verify `TINA4_CORS_ORIGINS` is set. Check that middleware is not overriding CORS headers.

4. Storing Tokens in localStorage

Problem: Token stolen via XSS because it lived in `localStorage`.

Cause: Any JavaScript on the page reads `localStorage`, including injected scripts.

Fix: Use `httpOnly` cookies when possible -- invisible to JavaScript. For SPAs that must use `localStorage`, enforce strict Content Security Policy headers. Sanitize all input.

5. Forgetting @noauth on Login

Problem: Login endpoint returns 401. Cannot log in.

Cause: Global auth middleware blocks the login endpoint. You need a token to get a token. A catch-22.

Fix: Add `@noauth` to login and register routes.

6. Password Hash Column Too Short

Problem: Registration fails with a database error about truncation.

Cause: PBKDF2 hashes can be long. `VARCHAR(50)` truncates them.

Fix: Use `TEXT` for the password hash column. Or at minimum `VARCHAR(255)`.

7. Token in URL Query Parameters

Problem: Tokens in URLs like `/api/profile?token=eyJ...` leak through browser history, server logs, and Referer headers.

Fix: Always send tokens in the `Authorization` header. The only exception: WebSocket connections where the upgrade request cannot carry custom headers. Use a short-lived token for that case.

Sessions & Cookies

1. State in a Stateless World

Your e-commerce site needs a shopping cart that survives page reloads, remembers the user's language, and flashes success messages after form submissions. HTTP has no memory. Every request is independent. Sessions and cookies give the server a way to remember who is asking and what they have been doing.

Chapter 8 introduced sessions for authentication. This chapter goes deeper: session backends, flash messages, cookies, remember-me tokens, and security configuration.

---

2. How Sessions Work

Sessions are auto-started. Every route handler receives `$request->session` ready to use. No manual setup required.

First visit. No session cookie. Tina4 generates a unique session ID -- a long random string. Stores it in a cookie named `tina4_session` (`HttpOnly`, `SameSite=Lax`). Creates server-side storage keyed by that ID. Every subsequent request carries the cookie. Tina4 looks up the data. Makes it available through `$request->session`.

The flow:

- Browser sends first request (no session cookie)
- Tina4 generates session ID: `abc123def456`
- Tina4 sets cookie: `tina4_session=abc123def456`
- Tina4 creates empty session storage for `abc123def456`
- Browser sends second request with cookie `tina4_session=abc123def456`
- Tina4 loads session data for `abc123def456`
- Your handler reads and writes `$request->session`
- Request ends. Tina4 saves updated session data.

The data lives server-side. The browser holds the session ID. Nothing else.

---

3. The Session API

Access session data through `$request->session`. It is available in every route handler with zero configuration.

Full API Reference

Method	Description
`$request->session->set(key, value)`	Store a value
`$request->session->get(key, default)`	Retrieve a value (with optional default)
`$request->session->delete(key)`	Remove a key
`$request->session->has(key)`	Check if a key exists
`$request->session->clear()`	Remove all session data
`$request->session->destroy()`	Destroy the session entirely
`$request->session->save()`	Persist session data (auto-called after response)
`$request->session->regenerate()`	Generate a new session ID, preserve data
`$request->session->flash(key, value)`	Set flash data (one-time read)
`$request->session->getFlash(key)`	Read and remove flash data
`$request->session->all()`	Get all session data as an array

---

4. File Sessions (Default)

No configuration needed. Sessions stored in files. Works out of the box.

```
<?php
use Tina4\Router;

Router::get("/visit-counter", function ($request, $response) {
    $count = ($request->session->get("visit_count", 0)) + 1;
    $request->session->set("visit_count", $count);

    return $response->json([
        "visit_count" => $count,
        "message" => "You have visited this page " . $count . " time" . ($count === 1 ? "" : "s"
    ]);
});
```

```
curl http://localhost:7145/visit-counter -c cookies.txt -b cookies.txt
```

```
{"visit_count":1,"message":"You have visited this page 1 time"}
```

```
curl http://localhost:7145/visit-counter -c cookies.txt -b cookies.txt
```

```
{"visit_count":2,"message":"You have visited this page 2 times"}
```

```
curl http://localhost:7145/visit-counter -c cookies.txt -b cookies.txt
```

```
{"visit_count":3,"message":"You have visited this page 3 times"}
```

The `-c` flag saves cookies. The `-b` flag sends them back. This simulates browser behavior.

File sessions work for single-server deployments. Simplest option. No extra software.

---

5. Redis Sessions

Multiple servers behind a load balancer need a shared session store. Redis is the standard choice.

```
TINA4_SESSION_BACKEND=redis
TINA4_SESSION_REDIS_HOST=localhost
TINA4_SESSION_REDIS_PORT=6379
TINA4_SESSION_REDIS_PASSWORD=your-redis-password
```

That is the only change. Your code stays identical. `$request->session` works the same whether backed by files, Redis, MongoDB, or Valkey. The storage backend is invisible to your handlers.

Why Redis

- Sessions shared across all server instances
- Sub-millisecond reads and writes
- Built-in key expiry (automatic cleanup)
- No disk I/O

Redis with a Prefix

Sharing a Redis instance with other applications:

```
TINA4_SESSION_BACKEND=redis
TINA4_SESSION_REDIS_HOST=localhost
TINA4_SESSION_REDIS_PORT=6379
TINA4_SESSION_REDIS_PREFIX=myapp:sess:
```

---

6. MongoDB Sessions

Already running MongoDB:

```
TINA4_SESSION_BACKEND=mongodb
TINA4_SESSION_REDIS_HOST=localhost
TINA4_SESSION_REDIS_PORT=27017
TINA4_SESSION_MONGO_DB=myapp
TINA4_SESSION_MONGO_COLLECTION=sessions
```

TTL indexes handle expired session cleanup.

---

7. Valkey Sessions

Valkey is the open-source Redis fork. Wire-compatible. Same client library:

```
TINA4_SESSION_BACKEND=valkey
TINA4_SESSION_REDIS_HOST=localhost
TINA4_SESSION_REDIS_PORT=6379
```

---

8. Database Sessions

```
TINA4_SESSION_BACKEND=database
```

Stores sessions in the `tina4_session` table using your existing database connection (`TINA4_DATABASE_URL`). The table is auto-created on first use. Works with all 5 database engines (SQLite, PostgreSQL, MySQL, MSSQL, Firebird).

---

9. Reading and Writing Session Data

A key-value store. Read and write through `$request->session`:

```
<?php
use Tina4\Router;

// Write
Router::post("/api/preferences", function ($request, $response) {
    $body = $request->body;

    $request->session->set("language", $body["language"] ?? "en");
    $request->session->set("theme", $body["theme"] ?? "light");
    $request->session->set("items_per_page", (int) ($body["items_per_page"] ?? 20));

    return $response->json([
        "message" => "Preferences saved",
        "preferences" => [
            "language" => $request->session->get("language"),
            "theme" => $request->session->get("theme"),
            "items_per_page" => $request->session->get("items_per_page")
        ]
    ]);
});

// Read
Router::get("/api/preferences", function ($request, $response) {
    return $response->json([
        "language" => $request->session->get("language", "en"),
        "theme" => $request->session->get("theme", "light"),
        "items_per_page" => $request->session->get("items_per_page", 20)
    ]);
});

// Delete a key
Router::delete("/api/preferences/{key}", function ($request, $response) {
    $key = $request->params["key"];
```

```
    $request->session->delete($key);

    return $response->json(["message" => "Preference '" . $key . "' removed"]);
});

// Clear everything
Router::post("/api/session/clear", function ($request, $response) {
    $request->session->clear();

    return $response->json(["message" => "Session cleared"]);
});
```

Storing Complex Data

Sessions hold arrays and nested structures:

```
Router::post("/api/cart/add", function ($request, $response) {
    $body = $request->body;

    $cart = $request->session->get("cart", []);

    $cart[] = [
        "product_id" => (int) $body["product_id"],
        "name" => $body["name"],
        "price" => (float) $body["price"],
        "quantity" => (int) ($body["quantity"] ?? 1),
        "added_at" => date("c")
    ];

    $request->session->set("cart", $cart);

    $total = array_sum(array_map(
        fn($item) => $item["price"] * $item["quantity"],
        $cart
    ));

    return $response->json([
        "message" => "Added to cart",
        "cart_items" => count($cart),
        "cart_total" => $total
    ]);
});
```

---

10. Flash Messages

Session data that lives for one request. Set it before redirecting. Read it on the next request. Gone after that.

The pattern: submit a form, redirect to a success page, show a message that disappears on refresh.

Setting a Flash Message

```
<?php
use Tina4\Router;

Router::post("/profile/update", function ($request, $response) {
    $body = $request->body;

    // Update the profile...

    $request->session->flash("message", "Profile updated successfully");
    $request->session->flash("message_type", "success");

    return $response->redirect("/profile");
});
```

Reading and Clearing

```
Router::get("/profile", function ($request, $response) {
    $flash_message = $request->session->getFlash("message");
    $flash_type = $request->session->getFlash("message_type") ?? "info";

    return $response->render("profile.html", [
        "user" => ["name" => "Alice", "email" => "alice@example.com"],
        "flash_message" => $flash_message,
        "flash_type" => $flash_type
    ]);
});
```

The `getFlash()` method reads the value and removes it in one step. The next request will not see it.

In Templates

```
{% extends "base.html" %}

{% block content %}
    {% if flash_message %}
        <div class="alert alert-{{ flash_type }}">
            {{ flash_message }}
        </div>
    {% endif %}

    <h1>Profile</h1>
    <p>Name: {{ user.name }}</p>
    <p>Email: {{ user.email }}</p>
{% endblock %}
```

The alert appears once. Refresh the page. Gone.

---

11. Setting and Reading Cookies

Cookies live in the browser. Unlike sessions, the data is client-side. Use cookies for non-sensitive preferences that should survive session expiry.

Setting a Cookie

```
<?php
use Tina4\Router;

Router::post("/api/set-language", function ($request, $response) {
    $language = $request->body["language"] ?? "en";

    $response->setCookie("language", $language, [
        "expires" => time() + (365 * 24 * 60 * 60),  // 1 year
        "path" => "/",
        "httpOnly" => false,  // JavaScript can read it
        "secure" => false,    // true in production
        "sameSite" => "Lax"
    ]);

    return $response->json(["message" => "Language set to " . $language]);
});
```

Reading a Cookie

```
Router::get("/api/get-language", function ($request, $response) {
    $language = $request->cookies["language"] ?? "en";

    return $response->json(["language" => $language]);
});
```

Deleting a Cookie

Set expiry in the past:

```
Router::post("/api/clear-language", function ($request, $response) {
    $response->setCookie("language", "", [
        "expires" => time() - 3600,
        "path" => "/"
    ]);

    return $response->json(["message" => "Language cookie cleared"]);
});
```

When to Use Cookies vs Sessions

Use Cookies For	Use Sessions For
Language preference	Shopping cart contents
Theme (light/dark)	Authentication state
"Remember this device" flag	Flash messages
Analytics consent	Form wizard progress
Non-sensitive, long-lived data	Sensitive, short-lived data

---

12. Remember Me Functionality

A long-lived cookie re-authenticates users after their session expires.

```
<?php
use Tina4\Router;
use Tina4\Auth;
use Tina4\Database;

/**
 * @noauth
 */
Router::post("/login", function ($request, $response) {
    $body = $request->body;
    $db = Database::getConnection();

    $user = $db->fetchOne(
        "SELECT id, name, email, password_hash FROM users WHERE email = :email",
        ["email" => $body["email"]]
    );

    if ($user === null || !Auth::checkPassword($body["password"], $user["password_hash"])) {
        return $response->json(["error" => "Invalid email or password"], 401);
    }

    $request->session->set("user_id", $user["id"]);
    $request->session->set("user_name", $user["name"]);

    if (!empty($body["remember_me"])) {
        $rememberToken = bin2hex(random_bytes(32));

        // Store hashed token in database
        $db->execute(
            "UPDATE users SET remember_token = :token WHERE id = :id",
            ["token" => hash("sha256", $rememberToken), "id" => $user["id"]]
        );

        // Set long-lived cookie with unhashed token
        $response->setCookie("remember_me", $rememberToken, [
            "expires" => time() + (30 * 24 * 60 * 60),  // 30 days
            "path" => "/",
            "httpOnly" => true,
            "secure" => true,
            "sameSite" => "Lax"
        ]);
    }

    return $response->json([
        "message" => "Login successful",
        "user" => ["id" => $user["id"], "name" => $user["name"]]
    ]);
});
```

The middleware that checks the cookie:

```
<?php
use Tina4\Database;

function rememberMeMiddleware($request, $response, $next) {
```

```
    if ($request->session->has("user_id")) {
        return $next($request, $response);
    }

    $rememberToken = $request->cookies["remember_me"] ?? "";

    if (empty($rememberToken)) {
        return $next($request, $response);
    }

    $db = Database::getConnection();
    $hashedToken = hash("sha256", $rememberToken);

    $user = $db->fetchOne(
        "SELECT id, name, email FROM users WHERE remember_token = :token",
        ["token" => $hashedToken]
    );

    if ($user !== null) {
        $request->session->set("user_id", $user["id"]);
        $request->session->set("user_name", $user["name"]);
    }

    return $next($request, $response);
}
```

The flow:

- User logs in with "remember me" checked
- Server stores a hashed token in the database. Sets the raw token in a cookie.
- Session expires.
- User returns. Session empty. Cookie still present.
- Middleware finds the cookie. Looks up the hashed token. Restores the session.
- User is logged in again. No credentials entered.

The database holds the hash. The cookie holds the raw token. If the database is breached, the attacker gets hashes. They cannot forge the cookie.

---

13. Session Security

Configuration Options

```
TINA4_SESSION_TTL=3600            # Expires after 1 hour of inactivity
TINA4_SESSION_SECURE=true         # HTTPS only
TINA4_SESSION_HTTPONLY=true       # JavaScript cannot access the cookie (default)
TINA4_SESSION_SAMESITE=Lax        # CSRF protection (default)
```

httpOnly

`TINA4_SESSION_HTTPONLY=true` (default) makes the session cookie invisible to JavaScript. XSS attacks cannot steal the session ID. Almost never a reason to set this to `false`.

secure

`TINA4_SESSION_SECURE=true` restricts the cookie to HTTPS connections. Set to `true` in production. During development with `http://localhost`, set to `false` or the browser will not send the cookie.

sameSite

Controls cross-site cookie behavior:

Value	Behavior
`Strict`	Never sent with cross-site requests. Safest. Breaks some flows (clicking links from email).
`Lax`	Sent with top-level navigations (clicking links) but not cross-site API calls. Good default.
`None`	Always sent. Requires `secure=true`. Only for cross-site cookie access.

Session Regeneration

After login, regenerate the session ID to prevent session fixation attacks:

```
Router::post("/login", function ($request, $response) {
    // Validate credentials...

    $request->session->regenerate();

    $request->session->set("user_id", $user["id"]);

    return $response->redirect("/dashboard");
});
```

Session fixation: attacker sets a known session ID on the victim's browser before login. After login, the attacker uses that same ID. Regeneration invalidates the old ID.

Destroy a Session

To completely destroy a session (not just clear its data):

```
Router::post("/logout", function ($request, $response) {
    $request->session->destroy();
    return $response->redirect("/login");
});
```

---

14. Exercise: Build a Shopping Cart with Session Storage

A cart stored entirely in session data. No database.

Requirements

Method	Path	Description
`POST`	`/api/cart/add`	Add item. Body: `{"product_id": 1, "name": "Widget", "price": 9.99, "quantity": 2}`
`GET`	`/api/cart`	View cart. Items, quantities, subtotals, total.
`PUT`	`/api/cart/{product_id:int}`	Update quantity. Body: `{"quantity": 3}`. Remove if 0.
`DELETE`	`/api/cart/{product_id:int}`	Remove item.
`DELETE`	`/api/cart`	Clear cart.

Business Rules

- Adding an existing product increments quantity instead of duplicating
- Total calculated dynamically
- Full cart state returned after every operation

Test with:

```
# Add first item
curl -X POST http://localhost:7145/api/cart/add \
  -H "Content-Type: application/json" \
  -d '{"product_id": 1, "name": "Wireless Keyboard", "price": 79.99, "quantity": 1}' \
  -c cookies.txt -b cookies.txt

# Add second item
curl -X POST http://localhost:7145/api/cart/add \
  -H "Content-Type: application/json" \
  -d '{"product_id": 2, "name": "USB-C Hub", "price": 49.99, "quantity": 2}' \
  -c cookies.txt -b cookies.txt

# Add more of item 1 (increments, no duplicate)
curl -X POST http://localhost:7145/api/cart/add \
  -H "Content-Type: application/json" \
  -d '{"product_id": 1, "name": "Wireless Keyboard", "price": 79.99, "quantity": 1}' \
  -c cookies.txt -b cookies.txt

# View cart
curl http://localhost:7145/api/cart -b cookies.txt

# Update quantity
curl -X PUT http://localhost:7145/api/cart/2 \
  -H "Content-Type: application/json" \
  -d '{"quantity": 5}' \
  -c cookies.txt -b cookies.txt

# Remove item
curl -X DELETE http://localhost:7145/api/cart/1 -b cookies.txt -c cookies.txt

# Clear cart
curl -X DELETE http://localhost:7145/api/cart -b cookies.txt -c cookies.txt
```

---

15. Solution

Create `src/routes/cart.php`:

```
<?php
use Tina4\Router;

function getCart($session) {
    return $session->get("cart", []);
}

function cartResponse($cart) {
    $total = 0;
    $itemCount = 0;
    $items = [];

    foreach ($cart as $item) {
        $subtotal = $item["price"] * $item["quantity"];
        $total += $subtotal;
        $itemCount += $item["quantity"];
        $items[] = array_merge($item, ["subtotal" => $subtotal]);
    }
```

```
    return [
        "items" => $items,
        "item_count" => $itemCount,
        "unique_items" => count($cart),
        "total" => round($total, 2)
    ];
}

Router::post("/api/cart/add", function ($request, $response) {
    $body = $request->body;

    if (empty($body["product_id"]) || empty($body["name"]) || !isset($body["price"])) {
        return $response->json(["error" => "product_id, name, and price are required"], 400);
    }

    $cart = getCart($request->session);
    $productId = (int) $body["product_id"];
    $quantity = (int) ($body["quantity"] ?? 1);
    $found = false;

    foreach ($cart as $index => $item) {
        if ($item["product_id"] === $productId) {
            $cart[$index]["quantity"] += $quantity;
            $found = true;
            break;
        }
    }

    if (!$found) {
        $cart[] = [
            "product_id" => $productId,
            "name" => $body["name"],
            "price" => (float) $body["price"],
            "quantity" => $quantity
        ];
    }

    $request->session->set("cart", $cart);

    return $response->json(cartResponse($cart));
});

Router::get("/api/cart", function ($request, $response) {
    $cart = getCart($request->session);
    return $response->json(cartResponse($cart));
});

Router::put("/api/cart/{product_id:int}", function ($request, $response) {
    $productId = $request->params["product_id"];
    $quantity = (int) ($request->body["quantity"] ?? 0);
    $cart = getCart($request->session);
    $found = false;

    foreach ($cart as $index => $item) {
        if ($item["product_id"] === $productId) {
            if ($quantity <= 0) {
                array_splice($cart, $index, 1);
            } else {
                $cart[$index]["quantity"] = $quantity;
```

```
            }
            $found = true;
            break;
        }
    }

    if (!$found) {
        return $response->json(["error" => "Product not in cart"], 404);
    }

    $request->session->set("cart", $cart);

    return $response->json(cartResponse($cart));
});

Router::delete("/api/cart/{product_id:int}", function ($request, $response) {
    $productId = $request->params["product_id"];
    $cart = getCart($request->session);
    $found = false;

    foreach ($cart as $index => $item) {
        if ($item["product_id"] === $productId) {
            array_splice($cart, $index, 1);
            $found = true;
            break;
        }
    }

    if (!$found) {
        return $response->json(["error" => "Product not in cart"], 404);
    }

    $request->session->set("cart", $cart);

    return $response->json(cartResponse($cart));
});

Router::delete("/api/cart", function ($request, $response) {
    $request->session->set("cart", []);

    return $response->json(cartResponse([]));
});
```

After adding two items and adding more of item 1:

```
{
  "items": [
    {"product_id": 1, "name": "Wireless Keyboard", "price": 79.99, "quantity": 2, "subtotal": 15
    {"product_id": 2, "name": "USB-C Hub", "price": 49.99, "quantity": 2, "subtotal": 99.98}
  ],
  "item_count": 4,
  "unique_items": 2,
  "total": 259.96
}
```

Keyboard has `quantity: 2` (1 + 1). Not two separate entries.

After clearing:

```
{"items":[],"item_count":0,"unique_items":0,"total":0}
```

---

16. Gotchas

1. Sessions Do Not Work with curl Without Cookie Flags

Problem: Every curl request sees an empty session.

Cause: curl does not save or send cookies by default.

Fix: Use `-c cookies.txt -b cookies.txt`. Browsers handle this automatically.

2. Session Data Disappears After Server Restart

Problem: All session data gone.

Cause: File sessions in the temp directory. Cleared on restart.

Fix: Set `TINA4_SESSION_PATH` to a persistent directory. For production, use Redis or Valkey.

3. Session Cookie Not Sent in Production

Problem: Sessions work locally. Fail in production.

Cause: `TINA4_SESSION_SECURE=true` but the app sees HTTP (behind a proxy that terminates SSL).

Fix: Ensure the reverse proxy sets `X-Forwarded-Proto: https`. Or verify the connection is HTTPS end-to-end.

4. Flash Messages Show Twice

Problem: Flash message appears. Appears again on the next page.

Cause: Read it but did not clear it.

Fix: Use `$request->session->getFlash("message")` instead of `$request->session->get()`. The `getFlash()` method reads and deletes in one step.

5. Large Session Data Causes Slow Requests

Problem: Pages load slowly. Performance degrades over time.

Cause: Storing too much in the session. Entire result sets. File contents. Large arrays. Session data serializes and deserializes on every request.

Fix: Keep sessions small. Store IDs and references. Not entire objects.

6. Remember Me Token Not Invalidated on Password Change

Problem: After password change, other devices still logged in.

Cause: Remember token not cleared.

Fix: Clear the token on password change: `$db->execute("UPDATE users SET remember_token = NULL WHERE id = :id", ["id" => $userId])`. Forces all devices to re-authenticate.

7. Session Fixation

Problem: Attacker hijacks a session by setting a known ID before login.

Cause: Session ID not regenerated after authentication.

Fix: Call `$request->session->regenerate()` after login. New ID. Old one useless.

Middleware & Security

1. The Gatekeepers

Your API needs CORS headers for the React frontend, rate limiting for public endpoints, and auth checks for admin routes. You could paste the same 10 lines of CORS code into every handler. You will forget one. You could pile every check into a giant `if` tree. The business logic disappears under boilerplate.

Middleware solves this. Wrap routes with reusable logic. Each middleware does one job -- check a token, set CORS headers, log the request, enforce rate limits -- and the framework decides whether to continue to the next layer. Route handlers stay focused on their purpose.

Chapter 2 introduced middleware briefly. This chapter goes deep: built-in middleware, custom middleware, execution order, short-circuiting, and real-world patterns.

---

2. What Middleware Is

Middleware is code that runs before or after your route handler. It sits in the HTTP pipeline between the incoming request and the response. Every request can pass through multiple middleware layers before reaching the handler.

Tina4 PHP supports two styles of middleware:

Route-level middleware (function-based) receives `$request` and `$response`. To continue the chain, return nothing (null/void). To short-circuit, return a `Response` object directly. To block with a 403, return `false`.

```
<?php

// This middleware continues to the next layer (returns nothing)
function passthrough($request, $response) {
    // do something with the request
}
```

```
<?php

// This middleware short-circuits with a response
function blockUnauthorized($request, $response) {
    if (!$request->bearerToken()) {
        return $response->json(["error" => "Unauthorized"], 401);
    }
    // return nothing to continue
}
```

Class-based middleware (global) uses naming conventions. Static methods prefixed with `before` run before the handler. Methods prefixed with `after` run after it. Each method receives `($request, $response)` and returns `[$request, $response]`.

```
<?php
use Tina4\Request;
use Tina4\Response;
```

```
class MyMiddleware
{
    public static function beforeCheck(Request $request, Response $response): array
    {
        // Runs before the route handler
        return [$request, $response];
    }

    public static function afterCleanup(Request $request, Response $response): array
    {
        // Runs after the route handler
        return [$request, $response];
    }
}
```

If a `before*` method sets the response status to >= 400, the handler is skipped (short-circuit).

Register class-based middleware globally with `Middleware::use()`:

```
<?php
use Tina4\Middleware;
use Tina4\Middleware\CorsMiddleware;
use Tina4\Middleware\RequestLogger;

Middleware::use(CorsMiddleware::class);
Middleware::use(RequestLogger::class);
```

Global middleware runs on every request, in the order registered.

---

3. Built-in CorsMiddleware

CORS controls which domains can call your API. When React at `http://localhost:3000` calls your Tina4 API at `http://localhost:7145`, the browser sends a preflight `OPTIONS` request first. Wrong headers: the browser blocks everything.

Tina4 provides `CorsMiddleware`. Configure in `.env`:

```
TINA4_CORS_ORIGINS=http://localhost:3000,https://myapp.com
TINA4_CORS_METHODS=GET,POST,PUT,PATCH,DELETE,OPTIONS
TINA4_CORS_HEADERS=Content-Type,Authorization,X-API-Key
TINA4_CORS_MAX_AGE=86400
```

Register it globally:

```
<?php
use Tina4\Middleware;
use Tina4\Middleware\CorsMiddleware;

Middleware::use(CorsMiddleware::class);
```

The `beforeCors` method sets CORS response headers on every request. For `OPTIONS` preflight requests, it sets a 204 status which short-circuits the handler.

Test the preflight:

```
curl -X OPTIONS http://localhost:7145/api/products \
  -H "Origin: http://localhost:3000" \
```

```
  -H "Access-Control-Request-Method: POST" \
  -H "Access-Control-Request-Headers: Content-Type,Authorization" \
  -v
```

Response headers:

```
Access-Control-Allow-Origin: http://localhost:3000
Access-Control-Allow-Methods: GET,POST,PUT,PATCH,DELETE,OPTIONS
Access-Control-Allow-Headers: Content-Type,Authorization,X-API-Key
Access-Control-Max-Age: 86400
```

The `OPTIONS` request returns `204 No Content` with those headers. The browser caches the preflight for 86400 seconds (24 hours). Subsequent requests skip the preflight.

Wildcard Origins

During development:

```
TINA4_CORS_ORIGINS=*
```

The default. Do not use `*` in production. Specify your domains.

CORS Without Middleware

Set `TINA4_CORS_ORIGINS` in `.env` and Tina4 applies CORS headers globally. The middleware approach gives finer control -- CORS on specific groups, none on internal routes.

---

4. Built-in RateLimiter

Prevents a single client from flooding your API. Tracks requests per IP. Returns `429 Too Many Requests` when exceeded.

Configure in `.env`:

```
TINA4_RATE_LIMIT=60
TINA4_RATE_WINDOW=60
```

60 requests per 60 seconds per IP. Register it globally:

```
<?php
use Tina4\Middleware;
use Tina4\Middleware\RateLimiter;

Middleware::use(RateLimiter::class);
```

When exceeded, the `beforeRateLimit` method sets a 429 status on the response, which short-circuits the handler. The response includes rate limit headers:

```
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 0
Retry-After: 42
```

Built-in RequestLogger

The `RequestLogger` middleware logs every request with its timing. It uses two hooks:

- `beforeLog` stamps the start time before the handler runs

- `afterLog` calculates elapsed time and writes an info-level log entry

Register it globally:

```
<?php
use Tina4\Middleware;
use Tina4\Middleware\RequestLogger;

Middleware::use(RequestLogger::class);
```

The log output looks like:

```
GET /api/users 12.34ms
POST /api/products 45.67ms
```

Built-in SecurityHeadersMiddleware

The `SecurityHeadersMiddleware` adds standard security headers to every response. Register it globally:

```
<?php
use Tina4\Middleware;
use Tina4\Middleware\SecurityHeadersMiddleware;

Middleware::use(SecurityHeadersMiddleware::class);
```

It sets the following headers by default:

Header	Default Value
`X-Frame-Options`	`SAMEORIGIN`
`Content-Security-Policy`	`default-src 'self'`, plus `style-src` and `script-src` limited to `'self'` and a per-request nonce
`Strict-Transport-Security`	Not sent. With `TINA4_HSTS` set to a number of seconds: `max-age=<TINA4_HSTS>; includeSubDomains`, on HTTPS requests only
`Referrer-Policy`	`strict-origin-when-cross-origin`
`Permissions-Policy`	`camera=(), microphone=(), geolocation=()`
`X-Content-Type-Options`	`nosniff`

Override any header via environment variables in `.env`:

```
TINA4_FRAME_OPTIONS=SAMEORIGIN
TINA4_CSP=default-src 'self'; script-src 'self' https://cdn.example.com
TINA4_HSTS=63072000
TINA4_REFERRER_POLICY=no-referrer
TINA4_PERMISSIONS_POLICY=camera=(), microphone=(), geolocation=(self)
```

Combining All Four Built-In Middleware

A common production setup registers all four globally:

```
<?php
use Tina4\Middleware;
use Tina4\Middleware\CorsMiddleware;
use Tina4\Middleware\RateLimiter;
use Tina4\Middleware\RequestLogger;
use Tina4\Middleware\SecurityHeadersMiddleware;

Middleware::use(CorsMiddleware::class);
Middleware::use(RateLimiter::class);
Middleware::use(RequestLogger::class);
Middleware::use(SecurityHeadersMiddleware::class);
```

Order matters. CORS handles `OPTIONS` preflight first. The rate limiter only counts real requests (not preflight). The logger measures total time including the other middleware. Security headers are added to every response.

---

5. Writing Custom Middleware

Route-level middleware functions receive `$request` and `$response`. Return nothing to continue the chain. Return a `Response` to short-circuit. Return `false` to block with 403.

Request Logging Middleware

```
<?php

function logRequest($request, $response) {
    $method = $request->method;
    $path = $request->path;
    $ip = $request->ip;

    error_log("[" . date("Y-m-d H:i:s") . "] " . $method . " " . $path . " from " . $ip);
    // return nothing to continue
}
```

Save in `src/routes/middleware.php`. Apply it to a route:

```
Router::get("/api/products", function ($request, $response) {
    return $response->json(["products" => []]);
})->middleware([function ($request, $response) {
    error_log($request->method . " " . $request->path);
}]);
```

Or apply the named function to a group:

```
Router::group("/api", function () {
    Router::get("/products", function ($request, $response) {
        return $response->json(["products" => []]);
    });
}, [$logRequest]);
```

Note: `Router::group()` takes an **array** of middleware callables as its third argument.

IP Whitelist Middleware

```
<?php

function ipWhitelist($request, $response) {
    $allowedIps = explode(",", $_ENV["ALLOWED_IPS"] ?? "127.0.0.1");

    if (!in_array($request->ip, $allowedIps)) {
        return $response->json([
            "error" => "Access denied",
            "your_ip" => $request->ip
        ], 403);
    }
    // return nothing to continue
}
```

Configure in `.env`:

```
ALLOWED_IPS=127.0.0.1,10.0.0.5,192.168.1.100
```

When the IP is not in the list, the middleware returns a `Response` object. This short-circuits the chain -- the handler never runs.

Request Validation Middleware

```
<?php

function requireJson($request, $response) {
    if (in_array($request->method, ["POST", "PUT", "PATCH"])) {
        $contentType = $request->headers["Content-Type"] ?? "";

        if (strpos($contentType, "application/json") === false) {
            return $response->json([
                "error" => "Content-Type must be application/json",
                "received" => $contentType
            ], 415);
        }
    }
    // return nothing to continue
}
```

Ensures all write requests send JSON. Status: `415 Unsupported Media Type`.

> ***Note on header lookups*** *(v3.13.4+):* `$request->headers` *is case-insensitive per RFC 7230 §3.2.* `$request->headers["Content-Type"]`, `["content-type"]`, *and* `["CONTENT-TYPE"]` *all return the same value. Internally headers are a* `Tina4\CaseInsensitiveArray`*, so you can pass it to anything that expects an* `ArrayAccess` *and it behaves like a normal hash, just without the casing footgun.*

Continuation Middleware (the `$next` callable)

The two-argument form above is a *filter*: it runs before the handler and either short-circuits or steps aside. For middleware that needs to run code **after** the handler (timing, response transformation, cleanup), use the three-argument **continuation** form. Tina4 detects it via reflection: any callable with three or more parameters is dispatched Express-style.

```
<?php

$timer = function ($request, $response, $next) {
```

```
    $start = microtime(true);

    $result = $next($request, $response);          // descend into the next layer

    $elapsedMs = (int)((microtime(true) - $start) * 1000);
    \Tina4\Log::info("{$request->method} {$request->path} → {$elapsedMs}ms");
    return $result;
};

Router::post("/api/orders", function ($request, $response) {
    return $response->json(["created" => true]);
})->middleware([$timer]);
```

Inside the middleware, calling `$next($request, $response)` invokes the next middleware in the chain, or the route handler if this is the innermost layer. Omitting the call short-circuits, just like returning a `Response` from a filter middleware does.

Russian-doll order

When multiple continuation middlewares are attached, the first declared wraps everything else:

```
$outer = function ($req, $resp, $next) {
    error_log("outer.before");
    $result = $next($req, $resp);
    error_log("outer.after");
    return $result;
};

$inner = function ($req, $resp, $next) {
    error_log("inner.before");
    $result = $next($req, $resp);
    error_log("inner.after");
    return $result;
};

Router::get("/api/x", $handler)->middleware([$outer, $inner]);
//   request → outer.before → inner.before → handler → inner.after → outer.after → response
```

Short-circuiting without calling `$next`

```
$gate = function ($request, $response, $next) {
    if (!$request->headers["X-Internal-Token"] ?? null) {
        return $response->json(["error" => "Forbidden"], 403);
        // $next is never called - handler does not run
    }
    return $next($request, $response);
};
```

This is the same short-circuit pattern as the filter form, but with the explicit choice of whether to continue made visible at the call site.

When to pick which

Need	Use
Block or allow based on headers, then continue	2-arg filter (terser)
Add request metadata, then continue	2-arg filter
Run code **after** the handler returns	3-arg continuation
Time the request	3-arg continuation
Transform the response object	3-arg continuation
Catch exceptions from inner layers	3-arg continuation (wrap `$next` in try/catch)

Both forms can be mixed in the same `->middleware([...])` array. The dispatcher applies the 2-arg filters first, then folds the 3-arg continuations into a chain around the handler.

Writing Class-Based Middleware

For more complex middleware, use the class-based pattern with `before*` and `after*` static methods. This style is for **global** middleware registered via `Middleware::use()`:

```
<?php
use Tina4\Request;
use Tina4\Response;

class InputSanitizer
{
    public static function beforeSanitize(Request $request, Response $response): array
    {
        if (is_array($request->body)) {
            $request->body = self::sanitize($request->body);
        }
        return [$request, $response];
    }

    private static function sanitize(array $data): array
    {
        $clean = [];
        foreach ($data as $key => $value) {
            if (is_string($value)) {
                $clean[$key] = htmlspecialchars($value, ENT_QUOTES, 'UTF-8');
            } elseif (is_array($value)) {
                $clean[$key] = self::sanitize($value);
            } else {
                $clean[$key] = $value;
            }
        }
        return $clean;
    }
}
```

Register it globally:

```
Middleware::use(InputSanitizer::class);
```

JWT Authentication Middleware (Class-Based)

A real-world authentication middleware that verifies JWT tokens:

```
<?php
use Tina4\Auth;
use Tina4\Request;
use Tina4\Response;

class JwtAuthMiddleware
{
    public static function beforeVerifyToken(Request $request, Response $response): array
    {
        $authHeader = $request->headers["Authorization"] ?? "";

        if (empty($authHeader) || !str_starts_with($authHeader, "Bearer ")) {
            $response->status(401);
            return [$request, $response->json(["error" => "Authorization header required"])];
        }

        $token = substr($authHeader, 7);

        if (!Auth::validToken($token)) {
            $response->status(401);
            return [$request, $response->json(["error" => "Invalid or expired token"])];
        }

        $request->user = Auth::getPayload($token);
        return [$request, $response];
    }
}
```

Register it globally so all routes require authentication:

```
Middleware::use(JwtAuthMiddleware::class);
```

The middleware short-circuits with 401 if the token is missing or invalid. When `beforeVerifyToken` sets the response status to >= 400, the framework skips the route handler. The decoded payload is available as `$request->user` in the handler.

---

6. Applying Middleware to Individual Routes

Use the `->middleware()` method to attach route-level middleware. Pass an array of callables:

```
<?php
use Tina4\Router;

Router::get("/api/data", function ($request, $response) {
    return $response->json(["data" => [1, 2, 3]]);
})->middleware([function ($request, $response) {
    error_log("Accessing /api/data");
}]);
```

Multiple middleware functions run in listed order:

```
Router::post("/api/data", function ($request, $response) {
    return $response->json(["created" => true], 201);
```

```
})->middleware([$logRequest, $requireJson]);
```

You can also use named functions defined elsewhere:

```
$checkAuth = function ($request, $response) {
    if (!$request->bearerToken()) {
        return $response->json(["error" => "Unauthorized"], 401);
    }
};

Router::get("/api/secret", function ($request, $response) {
    return $response->json(["secret" => "data"]);
})->middleware([$checkAuth]);
```

`->middleware()` is purely additive

> ***Behaviour change in v3.13.4**: `->middleware()` does **not** change a route's auth requirement. POST/PUT/PATCH/DELETE routes stay Bearer-token-gated by default. To open a write route, chain `->noAuth()` explicitly. To lock a read route, chain `->secure()`.*

Before v3.13.4, applying `->middleware()` to a write route silently flipped its auth gate off: the framework assumed any custom middleware was handling auth itself. That assumption was undocumented and unsafe: a developer adding a logger or rate-limiter to an admin endpoint would silently open it to unauthenticated callers.

The current behaviour is mechanical: middleware adds layers; `->noAuth()` and `->secure()` are the only knobs that touch `auth_required`.

```
// POST routes are auth-gated by default - middleware doesn't change that
Router::post("/api/admin/users", $handler)
    ->middleware([$logger]);                        // STILL requires Bearer

// To open a write route, say so explicitly
Router::post("/api/webhook", $handler)
    ->middleware([$logger])
    ->noAuth();                                     // public

// To lock a read route, say so explicitly
Router::get("/api/admin/stats", $handler)
    ->middleware([$logger])
    ->secure();                                     // requires Bearer
```

The Bearer check still runs *before* your middleware, so a 401 short-circuits the chain before the logger sees the request. If you need to handle auth yourself (OAuth cookie sessions, custom token formats), open the route with `->noAuth()` and put the auth logic inside your middleware.

---

7. Route Groups with Shared Middleware

Groups apply middleware to every route inside. The third argument is an **array** of middleware callables:

```
<?php
use Tina4\Router;
```

```
$checkAuth = function ($request, $response) {
    if (!$request->bearerToken()) {
        return $response->json(["error" => "Unauthorized"], 401);
    }
};

$logRequest = function ($request, $response) {
    error_log($request->method . " " . $request->path);
};

// Public API -- no auth needed
Router::group("/api/public", function () {

    Router::get("/products", function ($request, $response) {
        return $response->json(["products" => []]);
    });

    Router::get("/categories", function ($request, $response) {
        return $response->json(["categories" => []]);
    });

}, [$logRequest]);

// Admin API -- auth required, IP restricted, logged
Router::group("/api/admin", function () {

    Router::get("/users", function ($request, $response) {
        return $response->json(["users" => []]);
    });

    Router::delete("/users/{id:int}", function ($request, $response) {
        $id = $request->params["id"];
        return $response->json(["deleted" => $id]);
    });

}, [$logRequest, $ipWhitelist, $checkAuth]);
```

Routes inside a group can add their own middleware. Group middleware runs first, then route-specific:

```
Router::group("/api", function () {

    // Execution: $logRequest -> $requireJson -> handler
    Router::post("/upload", function ($request, $response) {
        return $response->json(["uploaded" => true]);
    })->middleware([$requireJson]);

}, [$logRequest]);
```

---

8. Middleware Execution Order

Route-level middleware runs in listed order. Each middleware either returns nothing (continue) or returns a `Response` (stop).

```
<?php

$middlewareA = function ($request, $response) {
```

```
    error_log("A: running");
    // returns nothing -- continues to B
};

$middlewareB = function ($request, $response) {
    error_log("B: running");
    // returns nothing -- continues to C
};

$middlewareC = function ($request, $response) {
    error_log("C: running");
    // returns nothing -- continues to handler
};

Router::get("/api/test", function ($request, $response) {
    error_log("Handler");
    return $response->json(["ok" => true]);
})->middleware([$middlewareA, $middlewareB, $middlewareC]);
```

Server log:

```
A: running
B: running
C: running
Handler
```

The request flows through the middleware array in order: A, B, C, then the handler.

Placement matters:

- Authentication goes early. Catch unauthorized requests before doing work.
- Validation goes before the handler. Reject bad input early.
- Logging can go first to capture every request, even blocked ones.

Group + Route Middleware Order

```
Router::group("/api", function () {

    Router::get("/data", function ($request, $response) {
        return $response->json(["data" => true]);
    })->middleware([$middlewareC]);

}, [$middlewareA, $middlewareB]);
```

Execution: `$middlewareA` -> `$middlewareB` -> `$middlewareC` -> handler. Group middleware always runs before route middleware.

---

9. Short-Circuiting

When middleware returns a `Response` object, the chain stops. No subsequent middleware runs. The handler never executes.

Authentication Short-Circuit

```
<?php

function requireAuth($request, $response) {
    $token = $request->headers["Authorization"] ?? "";

    if (empty($token)) {
        return $response->json(["error" => "Authentication required"], 401);
    }
    // return nothing to continue
}
```

Missing header: `401` returned. Handler never runs. No database query. No business logic. Resources saved.

Blocking with false

Returning `false` from middleware tells the framework to respond with a generic `403 Forbidden`:

```
<?php

function requireAdmin($request, $response) {
    if (!isAdmin($request)) {
        return false; // framework sends 403 Forbidden
    }
    // return nothing to continue
}
```

Maintenance Mode

```
<?php

function maintenanceMode($request, $response) {
    $isMaintenanceMode = ($_ENV["MAINTENANCE_MODE"] ?? "false") === "true";

    if ($isMaintenanceMode) {
        return $response->json([
            "error" => "Service is undergoing maintenance",
            "retry_after" => 300
        ], 503);
    }
    // return nothing to continue
}
```

Add to `.env`:

```
MAINTENANCE_MODE=true
```

Every request gets `503 Service Unavailable`.

Read-Only Mode

```
<?php

function readOnly($request, $response) {
    if (in_array($request->method, ["POST", "PUT", "PATCH", "DELETE"])) {
        return $response->json(["error" => "API is in read-only mode"], 405);
    }
    // return nothing to continue
}
```

GET requests pass. Write operations blocked. Useful for standby replicas or demo environments.

---

10. Modifying Requests in Middleware

Middleware can attach data to the request before the handler sees it:

```
<?php
use Tina4\Auth;

function attachUser($request, $response) {
    $authHeader = $request->headers["Authorization"] ?? "";

    if (!empty($authHeader) && str_starts_with($authHeader, "Bearer ")) {
        $token = substr($authHeader, 7);
        if (Auth::validToken($token)) {
            $request->user = Auth::getPayload($token);
        }
    }

    // return nothing -- this middleware does not block
}
```

Different from blocking auth middleware. This attaches user data if present but does not reject unauthenticated requests. Some routes need user data for personalization but remain accessible without it.

Adding Request Metadata

```
<?php

function addRequestId($request, $response) {
    $requestId = bin2hex(random_bytes(8));
    $request->requestId = $requestId;
    $response->addHeader("X-Request-Id", $requestId);
    // return nothing to continue
}
```

The handler accesses `$request->requestId` for logging and correlation:

```
Router::get("/api/data", function ($request, $response) {
    error_log("[" . $request->requestId . "] Processing data request");
    return $response->json(["request_id" => $request->requestId, "data" => []]);
})->middleware([$addRequestId]);
```

---

11. Real-World Middleware Stack

A realistic production setup combining global class-based middleware with route-level function middleware:

```
<?php
use Tina4\Middleware;
use Tina4\Middleware\CorsMiddleware;
use Tina4\Middleware\RateLimiter;
use Tina4\Middleware\RequestLogger;

// Global middleware -- runs on every request
Middleware::use(CorsMiddleware::class);
Middleware::use(RateLimiter::class);
Middleware::use(RequestLogger::class);
```

```
<?php
use Tina4\Router;

// src/routes/middleware.php

$addRequestId = function ($request, $response) {
    $request->requestId = bin2hex(random_bytes(8));
    $response->addHeader("X-Request-Id", $request->requestId);
};

$requireApiKey = function ($request, $response) {
    $apiKey = $request->headers["X-API-Key"] ?? "";
    $validKeys = explode(",", $_ENV["API_KEYS"] ?? "");

    if (!in_array($apiKey, $validKeys)) {
        return $response->json([
            "error" => "Invalid or missing API key",
            "request_id" => $request->requestId ?? null
        ], 401);
    }
};
```

```
<?php
use Tina4\Router;

// src/routes/api.php

Router::group("/api/v1", function () {

    Router::get("/products", function ($request, $response) {
        return $response->json(["products" => [
            ["id" => 1, "name" => "Widget", "price" => 9.99],
            ["id" => 2, "name" => "Gadget", "price" => 19.99]
        ]]);
    });

    Router::get("/products/{id:int}", function ($request, $response) {
        $id = $request->params["id"];
        return $response->json(["id" => $id, "name" => "Widget", "price" => 9.99]);
    });

    Router::post("/products", function ($request, $response) {
        $body = $request->body;
```

```
        return $response->json([
            "id" => 3,
            "name" => $body["name"] ?? "Unknown",
            "price" => (float) ($body["price"] ?? 0)
        ], 201);
    });

}, [$addRequestId, $requireApiKey]);
```

Without API key:

```
{"error":"Invalid or missing API key","request_id":"a1b2c3d4e5f6a7b8"}
```

With valid key:

```
{"products":[{"id":1,"name":"Widget","price":9.99},{"id":2,"name":"Gadget","price":19.99}]}
```

---

12. Exercise: Build an API Key Middleware

Build API key middleware:

- Check for `X-API-Key` header
- Validate against a comma-separated list in `API_KEYS` env variable
- Missing key: `401` with `{"error": "API key required"}`
- Invalid key: `403` with `{"error": "Invalid API key"}`
- Valid key: attach to `$request->apiKey` and continue
- Apply to a group with at least two endpoints

Setup

```
API_KEYS=key-alpha-001,key-beta-002,key-gamma-003
```

Test with:

```
# No key -- 401
curl http://localhost:7145/api/partner/data

# Invalid key -- 403
curl http://localhost:7145/api/partner/data \
  -H "X-API-Key: wrong-key"

# Valid key -- 200
curl http://localhost:7145/api/partner/data \
  -H "X-API-Key: key-alpha-001"

# Valid key on another endpoint
curl http://localhost:7145/api/partner/stats \
  -H "X-API-Key: key-beta-002"
```

---

13. Solution

Create `src/routes/api-key-middleware.php`:

```
<?php
use Tina4\Router;

$validateApiKey = function ($request, $response) {
    $apiKey = $request->headers["X-API-Key"] ?? "";

    if (empty($apiKey)) {
        return $response->json(["error" => "API key required"], 401);
    }

    $validKeys = array_map("trim", explode(",", $_ENV["API_KEYS"] ?? ""));

    if (!in_array($apiKey, $validKeys)) {
        return $response->json(["error" => "Invalid API key"], 403);
    }

    $request->apiKey = $apiKey;
    // return nothing to continue
};

Router::group("/api/partner", function () {

    Router::get("/data", function ($request, $response) {
        return $response->json([
            "authenticated_with" => $request->apiKey,
            "data" => [
                ["id" => 1, "value" => "alpha"],
                ["id" => 2, "value" => "beta"]
            ]
        ]);
    });

    Router::get("/stats", function ($request, $response) {
        return $response->json([
            "authenticated_with" => $request->apiKey,
            "stats" => [
                "total_requests" => 1423,
                "avg_response_ms" => 42
            ]
        ]);
    });

}, [$validateApiKey]);
```

No key: `401` -- `{"error":"API key required"}`

Invalid key: `403` -- `{"error":"Invalid API key"}`

Valid key:

```
{
  "authenticated_with": "key-alpha-001",
  "data": [
    {"id": 1, "value": "alpha"},
    {"id": 2, "value": "beta"}
```

```
    ]
}
```

---

14. Gotchas

1. Route Middleware Must Be Callable

Problem: Passing a string name instead of a callable to `->middleware()` or `Router::group()`.

Cause: Route-level middleware expects an array of callable functions (closures or function references), not string names.

Fix: Pass callables: `->middleware([$myFunction])` or `->middleware([function ($request, $response) { ... }])`. Use variables holding closures, not strings.

2. Returning the Wrong Thing from Middleware

Problem: Middleware runs but does not behave as expected -- handler runs when it should be blocked, or chain stops when it should continue.

Cause: Confusion about return values. Route-level middleware uses a three-way contract:

- Return nothing (null/void) to continue
- Return a `Response` to short-circuit
- Return `false` to block with 403

Fix: To continue, just do not return anything. To block, return `$response->json([...], 401)`. Do not return `true` or other values.

3. Middleware Order Matters

Problem: Logging middleware does not see the request ID.

Cause: `$logRequest` runs before `$addRequestId`.

Fix: Put `$addRequestId` first: `[$addRequestId, $logRequest]`. Think: what needs to happen first.

4. CORS Preflight Returns 404

Problem: Browser `OPTIONS` request gets 404. `GET` and `POST` work with curl.

Cause: No `CorsMiddleware` registered. `OPTIONS` is not handled.

Fix: Register `Middleware::use(CorsMiddleware::class)`. It handles `OPTIONS` and returns correct CORS headers. Or set `TINA4_CORS_ORIGINS` in `.env` for global handling.

5. Rate Limiter Counts Preflight Requests

Problem: Frontend hits rate limit faster than expected. Every `POST` counts as two (OPTIONS + POST).

Fix: Register `CorsMiddleware` before `RateLimiter` globally. CORS handles `OPTIONS` and short-circuits with 204. The rate limiter only sees real requests.

6. Middleware File Not Auto-Loaded

Problem: Middleware variable is undefined when used in a route file.

Cause: File is not in `src/routes/`. Tina4 auto-loads `.php` files from that directory only.

Fix: Put middleware in `src/routes/middleware.php`. Filename does not matter. Location does.

7. Group Middleware Expects an Array

Problem: Passing a single callable instead of an array to `Router::group()`.

Cause: The third parameter of `Router::group()` is typed as `array $middleware = []`.

Fix: Always wrap in an array: `Router::group("/api", $callback, [$myMiddleware])`, not `Router::group("/api", $callback, $myMiddleware)`.

8. Mixing Up the Two Middleware Styles

Problem: Trying to use a class-based middleware in `->middleware()` or a function in `Middleware::use()`.

Cause: The two styles serve different purposes. Route-level middleware (`->middleware()` and `Router::group()`) takes callable functions. Global middleware (`Middleware::use()`) takes class names with `before*`/`after*` static methods.

Fix: Use `Middleware::use(MyClass::class)` for class-based global middleware. Use `->middleware([$callable])` for route-level function middleware.

---

15. Security

Every route you write is a door. Chapter 8 gave you locks. Chapter 9 gave you session keys. The first half of this chapter gave you guards. This section ties them together into a defence that works without thinking about it.

Tina4 ships secure by default. POST routes require authentication. CSRF tokens protect forms. Security headers harden every response. The framework does the boring security work so you focus on building features. But you need to understand what it does, and why, so you don't accidentally undo it.

---

1. Secure-by-Default Routing

Every POST, PUT, PATCH, and DELETE route requires a valid `Authorization: Bearer` token. No configuration needed. No method to call. The framework enforces this before your handler runs.

```
use Tina4\Router;
```

```
use Tina4\Request;
use Tina4\Response;

Router::post("/api/orders", function (Request $request, Response $response) {
    // This handler ONLY runs if the request carries a valid Bearer token.
    // Without one, the framework returns 401 before your code executes.
    return $response(["created" => true], 201);
});
```

Test it without a token:

```
curl -X POST http://localhost:7145/api/orders \
  -H "Content-Type: application/json" \
  -d '{"product": "widget"}'
# 401 Unauthorized
```

Test it with a valid token:

```
curl -X POST http://localhost:7145/api/orders \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiJ9..." \
  -d '{"product": "widget"}'
# 201 Created
```

GET routes are public by default. Anyone can read. Writing requires proof of identity.

Making a Write Route Public

Some endpoints need to accept unauthenticated writes: webhooks, registration forms, public contact forms. Use `->noAuth()`:

```
use Tina4\Router;
use Tina4\Request;
use Tina4\Response;

Router::post("/api/webhooks/stripe", function (Request $request, Response $response) {
    // No token required. Stripe can POST here freely.
    return $response(["received" => true]);
})->noAuth();
```

Protecting a GET Route

Admin dashboards, user profiles, account settings: some pages need protection even though they only read data. Use `->secure()`:

```
use Tina4\Router;
use Tina4\Request;
use Tina4\Response;

Router::get("/api/admin/users", function (Request $request, Response $response) {
    // Requires a valid Bearer token, even though it's a GET.
    return $response(["users" => []]);
})->secure();
```

The Rule

Method	Default	Override
GET, HEAD, OPTIONS	Public	`->secure()` to protect
POST, PUT, PATCH, DELETE	Auth required	`->noAuth()` to open

Two methods. One rule. No surprises.

---

2. CSRF Protection

Cross-Site Request Forgery tricks a user's browser into submitting a form to your server. The browser sends cookies automatically, including session cookies. Without CSRF protection, an attacker's page can submit forms as your logged-in user.

Tina4 blocks this with form tokens.

How It Works

- Your template renders a hidden token using `{{ form_token() }}`.
- The browser submits the token with the form data.
- The `CsrfMiddleware` validates the token before the route handler runs.
- Invalid or missing tokens receive a `403 Forbidden` response.

The Template

```
<form method="POST" action="/api/profile">
    {{ form_token() }}
    <input type="text" name="name" placeholder="Your name">
    <button type="submit">Save</button>
</form>
```

The `{{ form_token() }}` call generates a hidden input field containing a signed JWT. The token is bound to the current session, so a token from one session cannot be used in another.

The Middleware

CSRF protection is on by default. Every POST, PUT, PATCH, and DELETE request must include a valid form token. The middleware checks two places:

- **Request body** - `$request->body["formToken"]`
- **Request header** - `X-Form-Token`

If the token is missing or invalid, the middleware returns 403 before your handler runs.

AJAX Requests

For JavaScript-driven forms, send the token as a header:

```
// frond.min.js handles this automatically via saveForm()
// For manual AJAX, extract the token from the hidden field:
```

```
const token = document.querySelector('input[name="formToken"]').value;

fetch("/api/profile", {
    method: "POST",
    headers: {
        "Content-Type": "application/json",
        "X-Form-Token": token
    },
    body: JSON.stringify({ name: "Alice" })
});
```

Tokens in Query Strings - Blocked

Tokens must never appear in URLs. Query strings are logged in server access logs, browser history, and referrer headers. A token in the URL is a token anyone can steal.

Tina4 rejects any request that carries `formToken` in the query string and logs a warning:

```
CSRF token found in query string - rejected for security.
Use POST body or X-Form-Token header instead.
```

Skipping CSRF Validation

Three scenarios skip CSRF checks automatically:

- **GET, HEAD, OPTIONS** - Safe methods don't modify state.
- **Routes with** `->noAuth()` - Public write endpoints don't need CSRF (they have no session to protect).
- **Requests with a valid Bearer token** - API clients authenticate with tokens, not cookies. CSRF only matters for cookie-based sessions.

Disabling CSRF Globally

For internal microservices behind a firewall, where no browser ever touches the API, you can disable CSRF entirely:

```
TINA4_CSRF=false
```

Leave it enabled for anything a browser can reach. The cost is one hidden field per form. The protection is worth it.

---

3. Session-Bound Tokens

A form token alone prevents cross-site forgery. But what if someone steals a token from a form? Session binding stops them.

When `{{ form_token() }}` generates a token, it embeds the current session ID in the JWT payload. The CSRF middleware checks that the session ID in the token matches the session ID of the request. A token stolen from one session cannot be replayed in another.

This happens automatically. No configuration. No extra code.

How Stolen Tokens Fail

- Attacker visits your site, gets a form token for session `abc-123`.

- Attacker sends that token from their own session `xyz-789`.
- The middleware compares: `abc-123 != xyz-789` - rejected with 403.

The token is cryptographically valid. But it belongs to the wrong session. The binding catches it.

---

4. Security Headers

Every response from Tina4 carries security headers. The `SecurityHeadersMiddleware` injects them before the response reaches the browser. No opt-in required.

Header	Default Value	Purpose
`X-Frame-Options`	`SAMEORIGIN`	Prevents clickjacking - your pages cannot be embedded in iframes on other domains.
`X-Content-Type-Options`	`nosniff`	Stops browsers from guessing content types. A script is a script, not HTML.
`Content-Security-Policy`	`default-src 'self'`	Controls which resources the browser loads. Blocks inline scripts from injected HTML.
`Referrer-Policy`	`strict-origin-when-cross-origin`	Limits referrer data sent to external sites. Protects internal URLs from leaking.
`X-XSS-Protection`	`0`	Disabled. Modern CSP replaces this legacy header. Keeping it on can introduce vulnerabilities.
`Permissions-Policy`	`camera=(), microphone=(), geolocation=()`	Disables browser APIs your app does not need.

HSTS - Enforcing HTTPS

Strict Transport Security tells the browser to always use HTTPS. Disabled by default (it breaks local development on HTTP). Enable it in production:

```
TINA4_HSTS=31536000
```

This sets a one-year HSTS policy with `includeSubDomains`. Once a browser sees this header, it refuses to connect over HTTP, even if the user types `http://`.

Customising Headers

Override any header via environment variables:

```
TINA4_FRAME_OPTIONS=DENY
TINA4_CSP=default-src 'self'; script-src 'self' https://cdn.example.com
TINA4_REFERRER_POLICY=no-referrer
TINA4_PERMISSIONS_POLICY=camera=(), microphone=(), geolocation=(), payment=()
```

---

5. SameSite Cookies

Session cookies control who can send them. The `SameSite` attribute tells the browser when to include the cookie in requests.

Value	Behaviour
`Strict`	Cookie sent only on same-site requests. Even clicking a link from email to your site won't include the cookie. The user must navigate directly.
`Lax`	Cookie sent on same-site requests and top-level navigations (clicking a link). Not sent on cross-site AJAX or form POSTs from other domains.
`None`	Cookie sent on all requests, including cross-site. Requires `Secure` flag (HTTPS only).

Tina4 defaults to `Lax`. This blocks cross-site form submissions (CSRF) while allowing normal link navigation. Users click a link to your site from an email and they stay logged in. An attacker's page submits a hidden form and the cookie is withheld.

For most applications, `Lax` is the right choice. Change it only if you understand the trade-offs.

---

6. Login Flow - Complete Example

Authentication, sessions, tokens, and security converge in the login flow. Here is a complete implementation.

The Login Route

```
use Tina4\Router;
use Tina4\Request;
use Tina4\Response;
use Tina4\Auth;

Router::post("/api/login", function (Request $request, Response $response) {
    $email = $request->body["email"] ?? "";
    $password = $request->body["password"] ?? "";

    if (empty($email) || empty($password)) {
        return $response(["error" => "Email and password required"], 400);
    }

    // Look up user (replace with your database query)
    $user = $DBA->fetch(
        "SELECT id, email, password_hash, role FROM users WHERE email = ?",
        [$email]
    )->asObject()[0] ?? null;

    if (!$user) {
        return $response(["error" => "Invalid credentials"], 401);
    }

    // Verify password
    if (!Auth::checkPassword($password, $user->password_hash)) {
        return $response(["error" => "Invalid credentials"], 401);
    }

    // Generate token with user claims
    $secret = $_ENV["TINA4_SECRET"] ?? getenv("TINA4_SECRET");
    $token = Auth::getToken([
        "sub" => $user->id,
        "email" => $user->email,
        "role" => $user->role,
    ], $secret);

    // Store user in session
    $request->session->set("user_id", $user->id);
    $request->session->set("email", $user->email);
    $request->session->set("role", $user->role);
    $request->session->save();

    return $response([
        "token" => $token,
        "user" => ["id" => $user->id, "email" => $user->email],
    ]);
})->noAuth();
```

The `->noAuth()` call opens this route to unauthenticated requests. The handler validates credentials and issues a token. The session stores the user identity for server-side lookups.

The Login Form

```
{% extends "base.twig" %}
{% block content %}
<div class="container mt-5" style="max-width: 400px;">
    <h2>Login</h2>
    <form id="loginForm">
        {{ form_token() }}
        <div class="mb-3">
            <label for="email">Email</label>
            <input type="email" name="email" id="email" class="form-control"
                   placeholder="you@example.com" required>
        </div>
        <div class="mb-3">
            <label for="password">Password</label>
            <input type="password" name="password" id="password" class="form-control"
                   placeholder="Your password" required>
        </div>
        <button type="button" class="btn btn-primary w-100"
                onclick="saveForm('loginForm', '/api/login', 'loginMsg', handleLogin)">
            Sign In
        </button>
        <div id="loginMsg" class="mt-3"></div>
    </form>
</div>

<script>
function handleLogin(result) {
    if (result.token) {
        localStorage.setItem("token", result.token);
        window.location.href = "/dashboard";
    }
}
</script>
{% endblock %}
```

Protected Pages - Checking the Session

```
use Tina4\Router;
use Tina4\Request;
use Tina4\Response;

Router::get("/dashboard", function (Request $request, Response $response) {
    $userId = $request->session->get("user_id");

    if (!$userId) {
        return $response->redirect("/login");
    }

    return $response->render("dashboard.twig", [
        "email" => $request->session->get("email"),
        "role" => $request->session->get("role"),
    ]);
});
```

Logout - Destroying the Session

```
use Tina4\Router;
use Tina4\Request;
use Tina4\Response;

Router::post("/api/logout", function (Request $request, Response $response) {
    $request->session->destroy();
    return $response(["logged_out" => true]);
})->noAuth();
```

---

7. Handling Expired Sessions

Sessions expire. Tokens expire. The user clicks a link and finds themselves staring at a broken page or a cryptic error. A good security implementation handles expiry gracefully.

The Pattern: Redirect to Login, Then Back

When a session expires mid-use, the user should:

- See a login page - not an error.
- Log in again.
- Land on the page they were trying to reach - not the home page.

```
use Tina4\Router;
use Tina4\Request;
use Tina4\Response;

Router::get("/account/settings", function (Request $request, Response $response) {
    $userId = $request->session->get("user_id");

    if (!$userId) {
        // Remember where they wanted to go
        $returnUrl = urlencode($request->url);
        return $response->redirect("/login?redirect={$returnUrl}");
    }

    return $response->render("settings.twig", ["user_id" => $userId]);
});
```

The login handler reads the `redirect` parameter after successful authentication:

```
Router::post("/api/login", function (Request $request, Response $response) {
    // ... validate credentials ...

    $redirectUrl = $request->query["redirect"] ?? "/dashboard";

    return $response([
        "token" => $token,
        "redirect" => $redirectUrl,
    ]);
})->noAuth();
```

The login form JavaScript redirects to the saved URL:

```
function handleLogin(result) {
    if (result.token) {
```

```
        localStorage.setItem("token", result.token);
        window.location.href = result.redirect || "/dashboard";
    }
}
```

Token Refresh

Tokens expire based on `TINA4_TOKEN_LIMIT` (default: 60 minutes). The `frond.min.js` frontend library handles token refresh automatically: every response includes a `FreshToken` header with a new token. The client stores it and uses it for the next request.

For custom AJAX code, read the header yourself:

```
const res = await fetch("/api/data", {
    headers: { "Authorization": "Bearer " + localStorage.getItem("token") }
});

const freshToken = res.headers.get("FreshToken");
if (freshToken) {
    localStorage.setItem("token", freshToken);
}
```

---

8. Rate Limiting

Brute-force login attempts. Credential stuffing. API abuse. Rate limiting stops all of them.

Tina4 includes a sliding-window rate limiter that tracks requests per IP address. It activates automatically.

```
TINA4_RATE_LIMIT=100
TINA4_RATE_WINDOW=60
```

One hundred requests per sixty seconds per IP. Exceed the limit and the server returns `429 Too Many Requests` with headers telling the client when to retry:

```
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 45
```

For login routes, consider a stricter limit:

```
use Tina4\Router;
use Tina4\Request;
use Tina4\Response;
use Tina4\Middleware;

class LoginRateLimit extends Middleware
{
    public static function beforeRateCheck(Request $request, Response $response): array
    {
        // Custom rate limiter: 5 attempts per 60 seconds for login
        $ip = $request->ip;
        // ... implement per-route rate limiting ...
        return [$request, $response];
    }
}

Router::post("/api/login", function (Request $request, Response $response) {
```

```
    // ... login logic ...
})->noAuth()->middleware(LoginRateLimit::class);
```

---

9. CORS and Credentials

When your frontend runs on a different origin than your API (common in development), CORS controls whether the browser sends cookies and auth headers.

Tina4 handles CORS automatically. The relevant security settings:

```
TINA4_CORS_ORIGINS=*
TINA4_CORS_CREDENTIALS=true
```

Two rules to remember:

- `TINA4_CORS_ORIGINS=*` **with** `TINA4_CORS_CREDENTIALS=true` is invalid per the CORS spec. Tina4 handles this: when origin is `*`, the credentials header is not sent. But in production, list your actual origins.
- **Cookies need** `SameSite=None; Secure` for true cross-origin requests. If your API is on `api.example.com` and your frontend is on `app.example.com`, the default `Lax` cookie works because they share the same registrable domain. Different domains need `SameSite=None`.

Production CORS:

```
TINA4_CORS_ORIGINS=https://app.example.com,https://admin.example.com
TINA4_CORS_CREDENTIALS=true
```

---

10. Security Checklist

Before you deploy, verify:

- [ ] `TINA4_SECRET` is set to a long, random string - not the default.
- [ ] `TINA4_DEBUG=false` in production.
- [ ] `TINA4_HSTS=31536000` if serving over HTTPS.
- [ ] `TINA4_CORS_ORIGINS` lists your actual domains - not `*`.
- [ ] `TINA4_CSRF=true` (the default) for any browser-facing application.
- [ ] Login route uses `->noAuth()` and validates credentials before issuing tokens.
- [ ] Session is regenerated after login (prevents session fixation).
- [ ] Passwords are hashed with `Auth::hashPassword()` - never stored in plain text.
- [ ] File uploads are validated and size-limited (`TINA4_MAX_UPLOAD_SIZE`).
- [ ] Rate limiting is active on login and registration routes.
- [ ] Expired sessions redirect to login with a return URL.

Gotchas

1. "My POST route returns 401 but I didn't add auth"

Cause: Tina4 requires authentication on all write routes by default.

Fix: Chain `->noAuth()` on the route definition if the endpoint should be public. Otherwise, send a valid Bearer token with the request.

2. "CSRF validation fails on AJAX requests"

Cause: The form token is not included in the request.

Fix: Send the token as an `X-Form-Token` header. If using `frond.min.js`, call `saveForm()`; it handles tokens automatically.

3. "I disabled CSRF but forms still fail"

Cause: The route still requires Bearer auth (separate from CSRF). CSRF and auth are independent checks.

Fix: Either send a Bearer token or chain `->noAuth()` on the route.

4. "My Content-Security-Policy blocks inline scripts"

Cause: The default CSP is `default-src 'self'`, which blocks inline `<script>` tags and `onclick` handlers.

Fix: Move scripts to external `.js` files (the right approach) or relax the CSP:

```
TINA4_CSP=default-src 'self'; script-src 'self' 'unsafe-inline'
```

Prefer external scripts. Inline scripts are an XSS vector.

5. "User stays logged in after session expires"

Cause: The frontend stores a JWT in localStorage. The token is still valid even after the session is destroyed server-side.

Fix: Check the session on every page load. If the session is gone, redirect to login regardless of the token. Tokens authenticate API calls; sessions track server-side state. Both must be valid.

Exercise: Secure Contact Form

Build a public contact form that:

- Does not require login (`->noAuth()`).
- Validates CSRF tokens (form includes `{{ form_token() }}`).
- Rate-limits submissions to 3 per minute per IP.
- Stores messages in the database.
- Returns a success message.

Solution

```
// src/routes/contact.php
use Tina4\Router;
use Tina4\Request;
use Tina4\Response;

Router::get("/contact", function (Request $request, Response $response) {
    return $response->render("contact.twig", ["title" => "Contact Us"]);
});

Router::post("/api/contact", function (Request $request, Response $response) {
    $name = trim($request->body["name"] ?? "");
    $email = trim($request->body["email"] ?? "");
    $message = trim($request->body["message"] ?? "");

    if (empty($name) || empty($email) || empty($message)) {
        return $response(["error" => "All fields are required"], 400);
    }

    global $DBA;

    $DBA->exec(
        "INSERT INTO contact_messages (name, email, message) VALUES (?, ?, ?)",
        [$name, $email, $message]
    );
    $DBA->commit();

    return $response(["success" => true, "message" => "Thank you for your message"]);
})->noAuth();
```

```
{# src/templates/contact.twig #}
{% extends "base.twig" %}
{% block title %}Contact Us{% endblock %}
{% block content %}
<div class="container mt-5" style="max-width: 500px;">
    <h2>{{ title }}</h2>
    <form id="contactForm">
        {{ form_token() }}
        <div class="mb-3">
            <label for="name">Name</label>
            <input type="text" name="name" id="name" class="form-control"
                   placeholder="Jane Smith" required>
        </div>
        <div class="mb-3">
            <label for="email">Email</label>
            <input type="email" name="email" id="email" class="form-control"
                   placeholder="jane@example.com" required>
        </div>
        <div class="mb-3">
            <label for="message">Message</label>
            <textarea name="message" id="message" class="form-control" rows="4"
                      placeholder="How can we help?" required></textarea>
        </div>
        <button type="button" class="btn btn-primary"
                onclick="saveForm('contactForm', '/api/contact', 'contactMsg')">
            Send Message
        </button>
        <div id="contactMsg" class="mt-3"></div>
    </form>
```

```
</div>
{% endblock %}
```

The form is public. The CSRF token is present. The `->noAuth()` call opens the route. The middleware validates the token. The database stores the message. The user sees confirmation.

Five moving parts. Zero security holes. The framework handles the rest.

Caching

1. From 800ms to 3ms

Your product catalog page runs 12 database queries. It takes 800 milliseconds to render. Every visitor triggers the same queries, the same template rendering, the same JSON serialization -- for data that changes once a day.

Add caching. The first request takes 800ms. The next 10,000 take 3ms each. A 266x improvement. One line of configuration.

Caching stores the result of expensive operations for reuse. Tina4 gives you three layers, and you reach for them in this order:

- **Request-scoped query cache** -- on by default. Dedupes identical SELECTs inside a single request.
- **Persistent database cache** -- opt-in. Shares query results across requests with a TTL.
- **Response cache** -- the `ResponseCache` middleware. Stores whole HTTP responses and skips your handler entirely.

This chapter walks each layer, then shows the direct key/value API and how to combine the layers.

---

2. Layer 1: Request-Scoped Query Cache (On by Default)

Tina4 wraps every database connection in a query cache. The first layer runs automatically. When a request fires the same SELECT twice, the second call returns the first result instead of hitting the database again.

```
# These are the defaults -- you do not need to set them
TINA4_AUTO_CACHING=true
TINA4_AUTO_CACHING_TTL=5
```

The cache key derives from the SQL and its parameters. The dispatcher clears this cache at the **start** of every HTTP request, so it never serves stale rows across requests. Any write (`insert`, `update`, `delete`, `execute`) flushes it immediately. The 5-second TTL is a safety net for long-running CLI scripts and workers that live outside the request cycle.

```
<?php
use Tina4\Router;
use Tina4\Database\Database;

Router::get("/api/report", function ($request, $response) {
    $db = Database::getConnection();

    // First call hits the database.
    $totals = $db->fetchAll("SELECT category, count(*) c FROM products GROUP BY category");

    // Same SQL again in this request -- served from the request-scoped cache.
    $alsoTotals = $db->fetchAll("SELECT category, count(*) c FROM products GROUP BY category");
```

```
    return $response->json(["totals" => $totals]);
});
```

To turn this layer off, set `TINA4_AUTO_CACHING=false`.

---

3. Layer 2: Persistent Database Cache (Opt-In)

The request-scoped cache dies at the end of each request. For results that stay fresh across requests -- reference data, category lists, slow aggregates -- enable the persistent layer:

```
TINA4_DB_CACHE=true
TINA4_DB_CACHE_TTL=30
```

Now identical queries return cached results across requests until the TTL expires:

```
<?php
use Tina4\Router;
use Tina4\Database\Database;

Router::get("/api/categories", function ($request, $response) {
    $db = Database::getConnection();

    // First call: executes the query (20ms)
    // Subsequent calls within 30 seconds: returns the cached result (0.1ms)
    $categories = $db->fetchAll("SELECT * FROM categories ORDER BY name");

    return $response->json(["categories" => $categories]);
});
```

The persistent cache routes through the unified backend. Pick where it lives and point it at a server:

```
TINA4_DB_CACHE=true
TINA4_DB_CACHE_TTL=30
TINA4_DB_CACHE_BACKEND=redis              # memory (default), file, redis, valkey, memcached, mo
TINA4_DB_CACHE_URL=redis://localhost:6379
```

With a network backend like Redis, multiple server instances share one cache, and a write on any instance invalidates the shared cache for all of them.

When to Use the Persistent Cache

- Read-heavy applications where the same queries run repeatedly
- Reference data that changes rarely (categories, countries, settings)
- Dashboard queries that aggregate large datasets

When Not to Use It

- Write-heavy applications where data changes constantly
- Queries with real-time requirements (inventory counts, live prices)
- Queries that must return the latest data

Bypassing the Cache for One Query

Every read method takes a final `$noCache` flag. Pass `true` to skip both the lookup and the store for that one call -- it runs straight against the database, in either cache mode:

```
$db = Database::getConnection();

// fetch($sql, $params, $limit, $offset, $noCache)
$fresh = $db->fetch("SELECT * FROM products", [], 100, 0, true);

// fetchOne($sql, $params, $noCache)
$row = $db->fetchOne("SELECT * FROM products WHERE id = :id", ["id" => 42], true);

// fetchAll($sql, $params, $limit, $offset, $noCache)
$all = $db->fetchAll("SELECT * FROM products", [], 0, 0, true);
```

Inspecting the Query Cache

`$db->cacheStats()` reports the live state of the query cache:

```
Router::get("/api/db-cache/stats", function ($request, $response) {
    $db = \Tina4\Database\Database::getConnection();
    return $response->json($db->cacheStats());
});
```

```
{
  "enabled": true,
  "mode": "persistent",
  "hits": 1842,
  "misses": 96,
  "size": 12,
  "ttl": 30,
  "backend": "redis"
}
```

`mode` is `"request"` (layer 1 only), `"persistent"` (layer 2 on), or `"off"`. Call `$db->cacheClear()` to flush the query cache and reset the counters.

---

4. Layer 3: Response Caching with ResponseCache Middleware

The fastest cache is at the HTTP response level. The `ResponseCache` middleware stores the complete response -- body, status, and content type -- and serves it on subsequent GET requests without calling your route handler at all.

Attach it as a string spec with a TTL:

```
<?php
use Tina4\Router;

Router::get("/api/products", function ($request, $response) {
    // This handler runs 12 database queries and takes 800ms
    // With ResponseCache, it only runs once every 5 minutes

    error_log("Handler called -- this should only appear once every 5 minutes");

    $products = [
```

```
        ["id" => 1, "name" => "Wireless Keyboard", "price" => 79.99],
        ["id" => 2, "name" => "USB-C Hub", "price" => 49.99],
        ["id" => 3, "name" => "Monitor Stand", "price" => 129.99]
    ];

    return $response->json(["products" => $products, "generated_at" => date("c")]);
})->middleware(["ResponseCache:300"]);
```

`"ResponseCache:300"` caches the response for 300 seconds (5 minutes). During those 5 minutes:

- The first request runs the handler (800ms)
- The next 10,000 requests serve the cached response (3ms each)
- After 300 seconds, the cache expires and the next request runs the handler again

```
curl http://localhost:7145/api/products
```

```
{
  "products": [
    {"id": 1, "name": "Wireless Keyboard", "price": 79.99},
    {"id": 2, "name": "USB-C Hub", "price": 49.99},
    {"id": 3, "name": "Monitor Stand", "price": 129.99}
  ],
  "generated_at": "2026-03-22T14:30:00+00:00"
}
```

Call it again within 5 minutes:

```
curl http://localhost:7145/api/products
```

```
{
  "products": [
    {"id": 1, "name": "Wireless Keyboard", "price": 79.99},
    {"id": 2, "name": "USB-C Hub", "price": 49.99},
    {"id": 3, "name": "Monitor Stand", "price": 129.99}
  ],
  "generated_at": "2026-03-22T14:30:00+00:00"
}
```

Same `generated_at` timestamp. The handler did not run. The response came from cache.

Two Ways to Attach It

The string form parses a TTL after the colon. The class-array form uses the default TTL (`TINA4_CACHE_TTL`, 60 seconds). Both work:

```
// String spec -- per-route TTL
Router::get("/api/products", $handler)->middleware(["ResponseCache:300"]);

// Class array -- default TTL from TINA4_CACHE_TTL
Router::get("/api/categories", $handler)->middleware([\Tina4\Middleware\ResponseCache::class]);
```

Two env vars tune the response cache globally: `TINA4_CACHE_TTL` (default 60, set 0 to disable) and `TINA4_CACHE_MAX_ENTRIES` (default 1000).

Cache Headers

The `ResponseCache` middleware stamps two headers on every cacheable response:

```
X-Cache: HIT
X-Cache-TTL: 300
```

- `X-Cache: HIT` or `X-Cache: MISS` -- whether the response came from cache
- `X-Cache-TTL` -- the configured TTL in seconds

The middleware does not set a `Cache-Control` header.

Caching with Query Parameters

The cache key is the request method plus the full URL, so `/api/products?page=1` and `/api/products?page=2` are cached separately:

```
Router::get("/api/products", function ($request, $response) {
    $page = (int) ($request->params["page"] ?? 1);
    $limit = 20;
    $offset = ($page - 1) * $limit;

    // Simulate database query
    return $response->json([
        "page" => $page,
        "products" => [],
        "generated_at" => date("c")
    ]);
})->middleware(["ResponseCache:300"]);
```

```
curl "http://localhost:7145/api/products?page=1"  # Cache MISS, stores for page=1
curl "http://localhost:7145/api/products?page=2"  # Cache MISS, stores for page=2
curl "http://localhost:7145/api/products?page=1"  # Cache HIT
```

What Not to Cache

`ResponseCache` only caches GET requests that return a 200. Even so, do not attach it to:

- **User-specific endpoints**: `/api/profile` returns different data for each user
- **Real-time data**: stock prices, live scores, chat messages
- **Authenticated endpoints**: unless every user shares the same response

```
// GOOD: Public, stable data
Router::get("/api/categories", function ($request, $response) {
    return $response->json(["categories" => []]);
})->middleware(["ResponseCache:3600"]); // Cache for 1 hour

// BAD: User-specific data -- do NOT cache
Router::get("/api/profile", function ($request, $response) {
    return $response->json($request->user);
})->middleware(["authMiddleware"]); // No ResponseCache here
```

Inspecting the Response Cache

`ResponseCache::cacheStats()` reports hits, misses, and the active backend:

```
<?php
use Tina4\Router;
use Tina4\Middleware\ResponseCache;

Router::get("/api/cache/stats", function ($request, $response) {
    return $response->json(ResponseCache::cacheStats());
});
```

```
{
  "hits": 15234,
  "misses": 891,
  "size": 42,
  "backend": "memory",
  "keys": []
}
```

Call `ResponseCache::clearCache()` to flush every cached response and reset the counters.

---

5. Cache Backends

All three layers share one set of backends, selected by `TINA4_CACHE_BACKEND`. Seven are available:

Backend	Notes
`memory`	In-process array. The default. Fastest, zero dependencies, lost on restart.
`file`	JSON files under `data/cache/`. Survives restarts, no external service.
`redis`	Redis over the wire. Shared across instances, server-side expiry.
`valkey`	Valkey (Redis wire protocol).
`memcached`	Memcached, zero-dependency text protocol.
`mongodb`	MongoDB TTL collection.
`database`	A `tina4_cache` table in any Tina4-supported database.

If a configured network or driver backend is unreachable -- the driver is missing, the service is down, the credentials are wrong -- Tina4 logs a warning and falls back to the **file** backend. It never silently degrades to a no-op, so you always get a real cache.

Memory (Default)

```
# This is the default -- you do not need to set it
TINA4_CACHE_BACKEND=memory
```

Memory cache is the fastest option. No disk I/O. No network calls. But it resets when the server restarts. Ideal for development and single-server deployments where losing the cache on restart is acceptable.

File

```
TINA4_CACHE_BACKEND=file
TINA4_CACHE_DIR=data/cache
```

Each cache entry becomes a file on disk. Slower than memory, but survives restarts without extra infrastructure. Use it when you need persistence but cannot run Redis -- shared hosting, no external services.

Redis (and Valkey, Memcached, MongoDB)

For production, you usually want a cache that survives restarts and is shared across server instances behind a load balancer. Point the backend at one URL:

```
TINA4_CACHE_BACKEND=redis
TINA4_CACHE_URL=redis://localhost:6379
```

Credentials can ride in the URL (`redis://user:pass@host:6379`) or live in their own variables:

```
TINA4_CACHE_USERNAME=cacheuser
TINA4_CACHE_PASSWORD=secret
```

Your code does not change. `cache_get`, `cache_set`, and `ResponseCache` all work the same. Only the storage backend changes.

---

6. Direct Key/Value Cache API

For custom caching logic, use the namespace-level helpers. They live in `\Tina4\Middleware\`, so import them from there:

```
use function Tina4\Middleware\cache_get;
use function Tina4\Middleware\cache_set;
use function Tina4\Middleware\cache_delete;
```

The full surface:

```
\Tina4\Middleware\cache_get(string $key): mixed
\Tina4\Middleware\cache_set(string $key, mixed $value, int $ttl = 0): void
\Tina4\Middleware\cache_delete(string $key): bool
\Tina4\Middleware\cache_clear(): void
\Tina4\Middleware\cache_stats(): array
```

cache_set

```
<?php
use function Tina4\Middleware\cache_set;

// Cache a value for 300 seconds
cache_set("product:42", [
    "id" => 42,
    "name" => "Wireless Keyboard",
    "price" => 79.99,
    "in_stock" => true
], 300);

// Cache a string
cache_set("exchange_rate:USD_EUR", "0.92", 3600);

// Cache with the backend's default TTL (pass no TTL)
cache_set("app:config", ["theme" => "dark", "lang" => "en"]);
```

cache_get

```
<?php
use function Tina4\Middleware\cache_get;
use function Tina4\Middleware\cache_set;

$product = cache_get("product:42");
// Returns the cached value, or null if not found or expired

if ($product === null) {
    // Cache miss -- fetch from database
    $product = fetchProductFromDatabase(42);
    cache_set("product:42", $product, 300);
}

return $response->json($product);
```

cache_delete

```
<?php
use function Tina4\Middleware\cache_delete;

// Delete a specific key
cache_delete("product:42");

// Delete multiple keys
cache_delete("product:42");
cache_delete("product:43");
cache_delete("product:44");
```

Real-World Pattern: Cache-Aside

The most common caching pattern. Also called lazy loading:

```
<?php
use Tina4\Router;
use Tina4\Database\Database;
use function Tina4\Middleware\cache_get;
use function Tina4\Middleware\cache_set;

Router::get("/api/products/{id:int}", function ($request, $response) {
    $id = $request->params["id"];
    $cacheKey = "product:" . $id;

    // 1. Try the cache first
    $product = cache_get($cacheKey);

    if ($product !== null) {
        // Cache hit -- return immediately
        return $response->json(array_merge($product, ["source" => "cache"]));
    }

    // 2. Cache miss -- fetch from database
    $db = Database::getConnection();
    $product = $db->fetchOne(
        "SELECT id, name, category, price, in_stock FROM products WHERE id = :id",
        ["id" => $id]
    );

    if ($product === null) {
```

```
        return $response->json(["error" => "Product not found"], 404);
    }

    // 3. Store in cache for next time
    cache_set($cacheKey, $product, 600); // Cache for 10 minutes

    return $response->json(array_merge($product, ["source" => "database"]));
});
```

```
curl http://localhost:7145/api/products/42
```

First call (cache miss):

```
{
  "id": 42,
  "name": "Wireless Keyboard",
  "category": "Electronics",
  "price": 79.99,
  "in_stock": true,
  "source": "database"
}
```

Second call (cache hit):

```
{
  "id": 42,
  "name": "Wireless Keyboard",
  "category": "Electronics",
  "price": 79.99,
  "in_stock": true,
  "source": "cache"
}
```

---

7. Cache Invalidation Strategies

The hardest problem in caching: knowing when to clear the cache. Stale cache serves outdated data. Premature invalidation reduces cache effectiveness.

Strategy 1: Time-Based Expiry (TTL)

The simplest approach. Set a TTL. Let the cache expire on its own:

```
cache_set("products:featured", $featuredProducts, 600); // Expires in 10 minutes
```

Good for data where near-real-time accuracy is acceptable. A 10-minute delay in updating the featured products list is usually fine.

Strategy 2: Event-Based Invalidation

Clear the cache when the underlying data changes:

```
<?php
use Tina4\Router;
use Tina4\Database\Database;
use function Tina4\Middleware\cache_delete;

// Update a product
Router::put("/api/products/{id:int}", function ($request, $response) {
```

```
    $id = $request->params["id"];
    $body = $request->body;
    $db = Database::getConnection();

    $db->execute(
        "UPDATE products SET name = :name, price = :price WHERE id = :id",
        ["name" => $body["name"], "price" => $body["price"], "id" => $id]
    );

    // Invalidate the cache for this product
    cache_delete("product:" . $id);

    // Also invalidate any list caches that might include this product
    cache_delete("products:all");
    cache_delete("products:featured");

    $updated = $db->fetchOne("SELECT * FROM products WHERE id = :id", ["id" => $id]);

    return $response->json($updated);
});
```

The most accurate strategy. The cache is always fresh after a write. The downside: you must remember to invalidate everywhere the data could be cached.

Strategy 3: Write-Through Cache

Update the cache at the same time as the database:

```
Router::put("/api/products/{id:int}", function ($request, $response) {
    $id = $request->params["id"];
    $body = $request->body;
    $db = Database::getConnection();

    $db->execute(
        "UPDATE products SET name = :name, price = :price WHERE id = :id",
        ["name" => $body["name"], "price" => $body["price"], "id" => $id]
    );

    $updated = $db->fetchOne("SELECT * FROM products WHERE id = :id", ["id" => $id]);

    // Write the new data directly to cache (instead of deleting)
    cache_set("product:" . $id, $updated, 600);

    return $response->json($updated);
});
```

The cache always has the latest data. No cache miss after an update. The next read comes from the already-warm cache.

---

8. TTL Management

Choosing the right TTL depends on change frequency and tolerance for stale data:

Data Type	Suggested TTL	Reasoning

Static config (categories, countries)	3600 (1 hour)	Changes rarely, stale data is harmless
Product catalog	300 (5 min)	Updates several times per day
User profile	60 (1 min)	Users expect changes to appear fast
Search results	120 (2 min)	Balance between freshness and performance
Dashboard stats	30 (30 sec)	Near-real-time but expensive to compute
Exchange rates	60 (1 min)	Updates frequently, slight delay is acceptable
Shopping cart	0 (no cache)	Must reflect current state

Dynamic TTL

Adjust TTL based on data characteristics:

```
<?php
use function Tina4\Middleware\cache_get;
use function Tina4\Middleware\cache_set;

function getCachedProduct($id) {
    $cacheKey = "product:" . $id;
    $product = cache_get($cacheKey);

    if ($product !== null) {
        return $product;
    }

    $product = fetchProductFromDatabase($id);

    // Popular products: shorter TTL (more likely to change)
    // Inactive products: longer TTL (rarely change)
    $ttl = ($product["view_count"] > 1000) ? 60 : 3600;

    cache_set($cacheKey, $product, $ttl);

    return $product;
}
```

---

9. Cache Statistics

Monitor cache performance. Verify that caching helps.

For the response cache, use `cache_stats()` (or `ResponseCache::cacheStats()`):

```
<?php
```

```
use Tina4\Router;
use function Tina4\Middleware\cache_stats;

Router::get("/api/cache/stats", function ($request, $response) {
    return $response->json(cache_stats());
});
```

```
curl http://localhost:7145/api/cache/stats
```

```
{
  "hits": 15234,
  "misses": 891,
  "size": 42,
  "backend": "memory",
  "keys": []
}
```

A high hit-to-miss ratio means your caching strategy works. Lots of misses suggests TTLs are too short, or you are caching data that is accessed too infrequently to benefit.

For the database query cache, use `$db->cacheStats()` instead -- it reports `enabled`, `mode`, `hits`, `misses`, `size`, `ttl`, and `backend` (see Section 3).

---

10. Combining Cache Layers

For maximum performance, layer multiple cache strategies:

```
<?php
use Tina4\Router;
use Tina4\Database\Database;
use function Tina4\Middleware\cache_get;
use function Tina4\Middleware\cache_set;

Router::get("/api/catalog", function ($request, $response) {
    $page = (int) ($request->params["page"] ?? 1);
    $cacheKey = "catalog:page:" . $page;

    // Layer A: Check the key/value cache
    $catalog = cache_get($cacheKey);

    if ($catalog !== null) {
        return $response->json(array_merge($catalog, ["cache" => "application"]));
    }

    // Layer B: Database query (with the query cache underneath)
    $db = Database::getConnection();
    $limit = 20;
    $offset = ($page - 1) * $limit;

    $products = $db->fetchAll(
        "SELECT p.*, c.name as category_name
         FROM products p
         JOIN categories c ON p.category_id = c.id
         WHERE p.active = 1
         ORDER BY p.created_at DESC
         LIMIT :limit OFFSET :offset",
        ["limit" => $limit, "offset" => $offset]
```

```
    );

    $total = $db->fetchOne("SELECT COUNT(*) as count FROM products WHERE active = 1");

    $catalog = [
        "products" => $products,
        "page" => $page,
        "total" => $total["count"],
        "pages" => (int) ceil($total["count"] / $limit),
        "generated_at" => date("c")
    ];

    // Store in the key/value cache
    cache_set($cacheKey, $catalog, 300);

    return $response->json(array_merge($catalog, ["cache" => "none"]));
})->middleware(["ResponseCache:60"]); // Response cache (60 seconds)
```

Three cache layers stack here:

- **ResponseCache** (60 seconds): the entire HTTP response is cached. No PHP code runs.
- **Key/value cache** (300 seconds): if the response cache expired but the KV entry is fresh, skip the database queries.
- **Query cache** (request-scoped by default, or persistent if `TINA4_DB_CACHE=true`): individual query results are cached even when the KV entry missed.

The first visitor after a full cache expiry waits 800ms. Everyone else gets the response in under 5ms.

---

11. Exercise: Cache an Expensive Product Listing Endpoint

Build a product listing endpoint that uses caching at multiple levels.

Requirements

- Create a `GET /api/store/products` endpoint that:
- Accepts query parameters: `category`, `page`, `limit`
- Returns a list of products with pagination metadata
- Uses the direct cache API (`cache_get`/`cache_set`) with a 5-minute TTL
- Includes a `source` field in the response (`"cache"` or `"database"`)
- Create a `POST /api/store/products` endpoint that:
- Creates a new product
- Invalidates the relevant cache entries
- Create a `GET /api/store/cache-stats` endpoint that shows cache statistics

Test with:

```
# First call -- cache miss, slow
curl "http://localhost:7145/api/store/products?category=Electronics&page=1"

# Second call -- cache hit, fast
curl "http://localhost:7145/api/store/products?category=Electronics&page=1"

# Different category -- cache miss
curl "http://localhost:7145/api/store/products?category=Fitness&page=1"

# Create a product -- should invalidate cache
curl -X POST http://localhost:7145/api/store/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Smart Watch", "category": "Electronics", "price": 299.99}'

# Same query again -- cache miss (invalidated by the POST)
curl "http://localhost:7145/api/store/products?category=Electronics&page=1"

# Check cache stats
curl http://localhost:7145/api/store/cache-stats
```

---

12. Solution

Create `src/routes/store-cached.php`:

```
<?php
use Tina4\Router;
use function Tina4\Middleware\cache_get;
use function Tina4\Middleware\cache_set;
use function Tina4\Middleware\cache_delete;
use function Tina4\Middleware\cache_stats;

// Simulated product database
function getProductStore() {
    return [
        ["id" => 1, "name" => "Wireless Keyboard", "category" => "Electronics", "price" => 79.99
        ["id" => 2, "name" => "Yoga Mat", "category" => "Fitness", "price" => 29.99, "in_stock"
        ["id" => 3, "name" => "Coffee Grinder", "category" => "Kitchen", "price" => 49.99, "in_s
        ["id" => 4, "name" => "Standing Desk", "category" => "Electronics", "price" => 549.99, "
        ["id" => 5, "name" => "Running Shoes", "category" => "Fitness", "price" => 119.99, "in_s
        ["id" => 6, "name" => "Bluetooth Speaker", "category" => "Electronics", "price" => 39.99
        ["id" => 7, "name" => "Resistance Bands", "category" => "Fitness", "price" => 14.99, "in
        ["id" => 8, "name" => "French Press", "category" => "Kitchen", "price" => 34.99, "in_sto
    ];
}

// List products with caching
Router::get("/api/store/products", function ($request, $response) {
    $category = $request->params["category"] ?? null;
    $page = (int) ($request->params["page"] ?? 1);
    $limit = (int) ($request->params["limit"] ?? 20);

    // Build cache key from query parameters
    $cacheKey = "store:products:" . md5(json_encode([
        "category" => $category,
        "page" => $page,
```

```
        "limit" => $limit
    ]));

    // Try cache first
    $cached = cache_get($cacheKey);

    if ($cached !== null) {
        return $response->json(array_merge($cached, ["source" => "cache"]));
    }

    // Simulate expensive database query
    usleep(100000); // 100ms delay to simulate a slow query

    $products = getProductStore();

    // Filter by category
    if ($category !== null) {
        $products = array_values(array_filter(
            $products,
            fn($p) => strtolower($p["category"]) === strtolower($category)
        ));
    }

    $total = count($products);
    $offset = ($page - 1) * $limit;
    $products = array_slice($products, $offset, $limit);

    $result = [
        "products" => $products,
        "page" => $page,
        "limit" => $limit,
        "total" => $total,
        "pages" => (int) ceil($total / $limit),
        "generated_at" => date("c")
    ];

    // Cache for 5 minutes
    cache_set($cacheKey, $result, 300);

    return $response->json(array_merge($result, ["source" => "database"]));
});

// Create product and invalidate cache
Router::post("/api/store/products", function ($request, $response) {
    $body = $request->body;

    if (empty($body["name"])) {
        return $response->json(["error" => "Name is required"], 400);
    }

    $product = [
        "id" => rand(100, 9999),
        "name" => $body["name"],
        "category" => $body["category"] ?? "General",
        "price" => (float) ($body["price"] ?? 0),
        "in_stock" => true
    ];

    // Invalidate all product list caches
```

```
    // In a real app with many cache keys, you would track which keys to invalidate
    // For now, we delete known cache patterns
    $categories = ["Electronics", "Fitness", "Kitchen", "General"];
    foreach ($categories as $cat) {
        for ($p = 1; $p <= 5; $p++) {
            $key = "store:products:" . md5(json_encode([
                "category" => $cat,
                "page" => $p,
                "limit" => 20
            ]));
            cache_delete($key);
        }
    }

    // Also invalidate the unfiltered list
    for ($p = 1; $p <= 5; $p++) {
        $key = "store:products:" . md5(json_encode([
            "category" => null,
            "page" => $p,
            "limit" => 20
        ]));
        cache_delete($key);
    }

    return $response->json([
        "message" => "Product created",
        "product" => $product,
        "cache_invalidated" => true
    ], 201);
});

// Cache statistics
Router::get("/api/store/cache-stats", function ($request, $response) {
    return $response->json(cache_stats());
});
```

Expected output -- first call (cache miss):

```
curl "http://localhost:7145/api/store/products?category=Electronics&page=1"
```

```
{
  "products": [
    {"id": 1, "name": "Wireless Keyboard", "category": "Electronics", "price": 79.99, "in_stock"
    {"id": 4, "name": "Standing Desk", "category": "Electronics", "price": 549.99, "in_stock": t
    {"id": 6, "name": "Bluetooth Speaker", "category": "Electronics", "price": 39.99, "in_stock"
  ],
  "page": 1,
  "limit": 20,
  "total": 3,
  "pages": 1,
  "generated_at": "2026-03-22T14:30:00+00:00",
  "source": "database"
}
```

Expected output -- second call (cache hit):

```
{
  "products": [
    {"id": 1, "name": "Wireless Keyboard", "category": "Electronics", "price": 79.99, "in_stock"
    {"id": 4, "name": "Standing Desk", "category": "Electronics", "price": 549.99, "in_stock": t
```

```
    {"id": 6, "name": "Bluetooth Speaker", "category": "Electronics", "price": 39.99, "in_stock"
  ],
  "page": 1,
  "limit": 20,
  "total": 3,
  "pages": 1,
  "generated_at": "2026-03-22T14:30:00+00:00",
  "source": "cache"
}
```

Same `generated_at`. Source changed from `"database"` to `"cache"`. The handler did not run.

Expected output -- after creating a product:

```
curl -X POST http://localhost:7145/api/store/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Smart Watch", "category": "Electronics", "price": 299.99}'
```

```
{
  "message": "Product created",
  "product": {
    "id": 4721,
    "name": "Smart Watch",
    "category": "Electronics",
    "price": 299.99,
    "in_stock": true
  },
  "cache_invalidated": true
}
```

The next request to the same URL is a cache miss again. The POST invalidated the cache.

---

13. Gotchas

1. Caching Authenticated Responses

Problem: User A's profile is served to User B because the response was cached.

Cause: `ResponseCache` keys by method and URL only. `/api/profile` returns different data per user, but all requests hit the same URL. The first user's response is served to everyone.

Fix: Do not attach `ResponseCache` to user-specific endpoints. Use the direct cache API with user-specific keys: `cache_set("profile:" . $userId, $data, 300)`.

2. Cache Stampede

Problem: A popular cache key expires. Hundreds of requests hit the database at once. All experience a cache miss at the same moment.

Cause: All requests see the miss. All try to rebuild the cache independently.

Fix: Shorten the window where the key is missing -- pre-warm hot keys on a schedule, or stagger TTLs so popular keys do not all expire together. Tina4 does not lock or serve stale entries for you, so plan for the miss.

3. Memory Cache Lost on Restart

Problem: After restarting the server, performance drops until the cache warms up.

Cause: The memory backend dies when the process restarts. Every request is a cache miss until data populates again.

Fix: For production, use a persistent backend (`TINA4_CACHE_BACKEND=redis` or `file`). Redis also survives across multiple instances. Or run a cache warmup script that pre-populates frequently accessed data.

4. Stale Data After Database Update

Problem: You updated a product's price in the database. The API still returns the old price.

Cause: The key/value cache holds the old data. It has not expired yet. (The query cache flushes itself on writes; the KV cache does not.)

Fix: Always invalidate or update the KV cache when you modify the underlying data. Use `cache_delete("product:" . $id)` after an update, or write the new value with `cache_set()`.

5. Cache Key Collisions

Problem: Two different queries return the same cached data.

Cause: Cache keys are not specific enough. Using `"products"` as a key for both the full list and a filtered list causes collisions.

Fix: Include all relevant parameters in the cache key: `"products:category:Electronics:page:1:limit:20"`. Or use an MD5 hash of the parameters: `"products:" . md5(json_encode($params))`.

6. Serialization Overhead

Problem: Caching makes certain requests slower, not faster.

Cause: The cached object is large. Serializing and deserializing it takes more time than re-computing it.

Fix: Only cache data that is expensive to compute. If the original operation takes 5ms and cache serialization takes 10ms, caching is counterproductive. Profile before and after to verify the improvement.

7. Importing the Helpers from the Wrong Namespace

Problem: `Call to undefined function Tina4\cache_set()`.

Cause: The key/value helpers live in `\Tina4\Middleware\`, not the root `Tina4\` namespace.

Fix: Import them from the right place: `use function Tina4\Middleware\cache_set;`. Or call them fully qualified: `\Tina4\Middleware\cache_set("key", $value, 300)`.

Queue System

1. Do Not Make the User Wait

Your app sends welcome emails on signup, generates PDF invoices, and resizes uploaded images. Each task takes 2 to 30 seconds. Do them inside the HTTP request and the user stares at a spinner while the server processes. That is a broken experience.

Queues move slow work to a background process. The handler drops a job onto a queue and responds immediately. A separate consumer picks it up. The user sees "Welcome -- check your email." in under 100 milliseconds. The email arrives 5 seconds later.

Tina4 has a built-in queue system. It works out of the box with a file-based backend. No Redis. No RabbitMQ. No external services. Add jobs. Process them.

The providers share delivery semantics, but they do not pretend every broker supports every management operation:

Provider	Supported contract	Explicitly unavailable
File / lite	Every queue operation	None
MongoDB	Every queue operation	None
RabbitMQ	Push, consume, complete, fail/reject, stable non-destructive dead letters, broker size where available, close	Priority, delay, pop by ID, purge by status, failed-job listing, retry failed
Kafka	Delivery, acknowledge, fail, stable non-destructive dead letters; `size()` honestly returns `0`	Priority, delay, pop by ID, failed-job listing, retry failed

This capability contract is defined by ADR-0022, ADR-0023, and ADR-0024. An unsupported operation raises an error naming the backend and operation. It never succeeds as a no-op or returns a misleading empty result.

---

2. Why Queues Matter

Without queues:

```
User clicks "Sign Up"
  -> Server validates input (10ms)
  -> Server creates user in database (20ms)
  -> Server sends welcome email (3000ms)
  -> Server generates PDF welcome kit (2000ms)
  -> Server resizes avatar (1500ms)
```

```
   -> User sees response (6530ms later)
```

With queues:

```
User clicks "Sign Up"
   -> Server validates input (10ms)
   -> Server creates user in database (20ms)
   -> Server queues: send welcome email (1ms)
   -> Server queues: generate PDF (1ms)
   -> Server queues: resize avatar (1ms)
   -> User sees response (33ms later)

Meanwhile, in the background:
   -> Consumer sends welcome email
   -> Consumer generates PDF
   -> Consumer resizes avatar
```

6.5 seconds becomes 33 milliseconds. The work still happens. Just not during the HTTP request.

Beyond speed, queues give you:

- **Automatic retries**: The email server is down. The job retries on its own, then lands in dead letters if it keeps failing.
- **Priority**: Password resets jump ahead of newsletters.
- **Fault isolation**: A failed PDF does not crash the signup request.
- **Scaling**: Run more consumers for higher load.

---

3. File Queue (Default)

The file-based backend is the default. No configuration needed. The first job creates the queue storage automatically under `data/queue`.

Creating a Queue and Pushing a Job

```php
<?php
use Tina4\Queue;

$queue = new Queue(topic: 'emails');

// Push a job
$queue->push([
    "to" => "alice@example.com",
    "subject" => "Order Confirmation",
    "body" => "Your order #1234 has been confirmed."
]);
```

You can also use the longer constructor form. The signature is `new Queue($backend, $config, $topic)`:

```php
$queue = new Queue('file', [], 'emails');
```

The `$config` array accepts `path`, `maxRetries`, and `retryBackoff` (covered in section 7):

```php
$queue = new Queue('file', ['maxRetries' => 5, 'retryBackoff' => 10], 'emails');
```

Convenience Method: produce

The `produce` method pushes to a specific topic without creating a separate Queue instance:

```
$queue = new Queue(topic: 'emails');
$queue->produce('invoices', ["order_id" => 101, "format" => "pdf"]);
```

Push with Priority

Jobs default to priority 0 (normal). Higher numbers are popped first. Within the same priority, the oldest job goes first:

```
// Normal priority (default)
$queue->push(["to" => "alice@example.com", "subject" => "Newsletter"]);

// High priority -- popped before the newsletter above
$queue->push(["to" => "alice@example.com", "subject" => "Password Reset"], priority: 10);
```

Push with Delay

Pass a delay (in seconds) to hold a job back until the delay elapses:

```
// Becomes available 60 seconds from now
$queue->push(["to" => "alice@example.com", "subject" => "Reminder"], priority: 0, delay: 60);
```

Queue Size

Check how many pending messages are in the queue:

```
$count = $queue->size();
```

Pass a status string to count jobs in a specific state. The file backend tracks three states: `pending`, `failed` (retrying), and `dead` (exhausted retries):

```
$pending = $queue->size("pending");
$retrying = $queue->size("failed");
$dead = $queue->size("dead");
```

---

4. Pushing from Route Handlers

The most common pattern is pushing messages from route handlers:

```
<?php
use Tina4\Router;
use Tina4\Queue;

Router::post("/api/register", function ($request, $response) {
    $body = $request->body;

    // Create the user (database logic)
    $userId = 42; // Simulated

    $queue = new Queue(topic: 'emails');

    // Queue a welcome email
    $queue->push([
        "user_id" => $userId,
        "to" => $body["email"],
```

```
        "name" => $body["name"],
        "subject" => "Welcome!"
    ]);

    return $response->json([
        "message" => "Registration successful. Welcome email will arrive shortly.",
        "user_id" => $userId
    ], 201);
});
```

```
curl -X POST http://localhost:7145/api/register \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "password": "securePass123"}'
```

```
{
  "message": "Registration successful. Welcome email will arrive shortly.",
  "user_id": 42
}
```

The response returns immediately. The email job waits in the queue.

---

5. Consuming Jobs

The `consume` method is a generator that yields `Job` objects one at a time. Each `Job` carries the payload and the lifecycle methods you call on it:

```
<?php
use Tina4\Queue;

$queue = new Queue(topic: 'emails');

foreach ($queue->consume('emails') as $job) {
    try {
        sendEmail($job->payload['to'], $job->payload['subject'], $job->payload['body']);
        $job->complete();
    } catch (\Throwable $e) {
        $job->fail($e->getMessage());
    }
}
```

When a job fails, `$job->fail()` re-queues it automatically and the next iteration picks it up again. After `maxRetries` attempts the job moves to dead letters. You do not call anything manually -- the loop above is the whole retry mechanism. Section 7 covers the lifecycle in detail.

consume Is a Long-Running Poll

By default `consume` never returns. When the queue drains it sleeps for `pollInterval` seconds (default `1.0`) and polls again, so a worker loop keeps running and waiting for new jobs. That is exactly what you want for a long-lived background worker.

When you want a **single pass** -- drain everything currently queued, then stop -- pass `pollInterval` of `0`. The generator returns as soon as the queue is empty:

```
// Drain the queue once and stop (useful in tests and one-shot scripts)
foreach ($queue->consume('emails', null, 0) as $job) {
    sendEmail($job->payload['to'], $job->payload['subject'], $job->payload['body']);
```

```
    $job->complete();
}
```

The `consume` signature is `consume($topic, $id, $pollInterval, $iterations, $batchSize)`. Pass `$iterations` to stop after a fixed number of jobs, or `$id` to consume one specific job by ID.

Retry with a Manual Delay

`$job->fail()` already retries automatically. If instead you want to push a job back yourself with a specific cooldown -- regardless of the retry limit -- call `$job->retry($delaySeconds)`:

```
foreach ($queue->consume('emails') as $job) {
    try {
        sendEmail($job->payload['to'], $job->payload['subject'], $job->payload['body']);
        $job->complete();
    } catch (\Throwable $e) {
        // Manual override: re-queue after 30 seconds (does not consult maxRetries)
        $job->retry(30);
    }
}
```

Manual Pop

For lower-level control, `pop` returns the next job as a **plain array** (not a `Job` object), or `null` when the queue is empty. Use array access on the payload, and do not call lifecycle methods on it -- the file backend already removed the job from the pending queue when you popped it:

```
$job = $queue->pop();

if ($job !== null) {
    // $job is an array: ['id' => ..., 'payload' => [...], 'priority' => ..., ...]
    sendEmail($job['payload']['to'], $job['payload']['subject']);
}
```

If you need the convenient `Job` object with `->payload`, `->complete()`, and `->fail()`, use `consume` instead. To grab several jobs at once, `popBatch($count)` returns an array of job arrays (highest priority first).

---

6. The Job Object

`consume` yields `Job` objects. Here are the methods and properties you use.

Job Methods

- `$job->complete()` -- mark the job as done. Terminal -- the job is finished and gone.
- `$job->fail($reason)` -- record a failed attempt. Increments `attempts`, stores the error, and **automatically re-queues** the job while retries remain, or moves it to dead letters once they are exhausted.
- `$job->reject($reason)` -- alias for `fail`.
- `$job->retry($delaySeconds)` -- a manual override that always re-queues the job after the delay, regardless of the retry limit. Use this when you want to schedule a retry yourself

rather than rely on the automatic path.

Job Properties

- `$job->payload` -- the data you pushed.
- `$job->topic` -- the topic this job belongs to. Useful when consuming from multiple topics.
- `$job->priority` -- the job's priority.
- `$job->attempts` -- how many times this job has been attempted.
- `$job->id` -- the unique job ID.
- `$job->error` -- the last failure reason, if any.

You can also call `$job->toArray()`, `$job->toHash()`, or `$job->toJson()` to serialize a job.

---

7. Automatic Retry and Dead Letters

This is the part the queue handles for you. When a `Job` from `consume` fails, you do not schedule the retry -- `$job->fail()` does.

How the Lifecycle Works

Every job moves through these states:

```
push() -> PENDING -> consume() yields Job -> $job->complete() -> done (removed)
                                          -> $job->fail()
                                                  |
                                          attempts < maxRetries ?
                                             |              |
                                           yes             no
                                             |              |
                                re-queued to PENDING      DEAD LETTER
                                (after retryBackoff)   (deadLetters() returns it)
```

When you call `$job->fail()`:

- `attempts` is incremented and the error is stored.
- If `attempts` is still **below** `maxRetries`, the job is re-queued to PENDING (after the `retryBackoff` delay, if set). The next `consume`/`pop` picks it up again.
- Once `attempts` reaches `maxRetries`, the job is moved to the dead-letter store, where `deadLetters()` returns it.

So a `consume` loop that calls `$job->fail($e)` in its `catch` block retries each job `maxRetries` times on its own, then dead-letters it. **There is no manual `retryFailed()` step in this path.**

maxRetries

The default `maxRetries` is 3. Override it in the constructor config:

```
$queue = new Queue('file', ['maxRetries' => 5], 'emails');
```

retryBackoff

`retryBackoff` (in seconds) delays the automatic re-enqueue after a failure. With `0` (the default), a failed job is available again on the very next poll. Set it to space retries out -- handy when an external service needs time to recover:

```
// Failed jobs wait 10 seconds before becoming available to retry again
$queue = new Queue('file', ['maxRetries' => 5, 'retryBackoff' => 10], 'emails');
```

Inspecting Retrying and Dead Jobs

These management methods operate on the file backend's store:

- `$queue->failed()` returns jobs that have failed at least once but are **still being retried** (`0 < attempts < maxRetries`). They live in the pending queue.
- `$queue->deadLetters()` returns jobs that **exhausted their retries** (`attempts >= maxRetries`).

```
$deadJobs = $queue->deadLetters();

foreach ($deadJobs as $job) {
    // Items from deadLetters() are arrays, not Job objects
    error_log("Dead job: " . $job['id']);
    error_log("  Payload: " . json_encode($job['payload']));
    error_log("  Error: " . ($job['error'] ?? ''));
}
```

Manually Reviving Jobs

Beyond the automatic path, you can revive jobs by hand:

```
// Revive a specific dead-letter job by ID (always re-queues it)
$queue->retry($jobId);

// Revive all dead-letter jobs
$queue->retry();

// Re-queue failed jobs that are still under maxRetries
$queue->retryFailed();
```

Purging Jobs

Remove jobs by status. `purge` accepts `pending`, `failed`, and `dead`, and returns the number removed:

```
$queue->purge("dead");      // clear the dead-letter store
$queue->purge("failed");    // clear retrying jobs
$queue->purge("pending");   // clear the pending queue
```

`$queue->clear()` is a shortcut that removes all pending jobs and returns the count.

---

8. Producing Multiple Jobs

One action. Multiple background tasks:

```
<?php
```

```
use Tina4\Router;
use Tina4\Queue;

Router::post("/api/orders", function ($request, $response) {
    $body = $request->body;

    $orderId = 101;
    $queue = new Queue(topic: 'emails');

    $queue->push([
        "order_id" => $orderId,
        "to" => $body["email"],
        "subject" => "Order Confirmation"
    ]);

    $queue->produce("invoices", [
        "order_id" => $orderId,
        "format" => "pdf"
    ]);

    $queue->produce("inventory", [
        "items" => $body["items"]
    ]);

    $queue->produce("warehouse", [
        "order_id" => $orderId,
        "shipping_address" => $body["shipping_address"]
    ]);

    return $response->json([
        "message" => "Order placed successfully",
        "order_id" => $orderId
    ], 201);
});
```

Four jobs queued in under 5 milliseconds. Instant response.

---

9. Switching Backends

Switching backends is a config change, not a code change. The file backend handles the retry and dead-letter lifecycle described above; external brokers (RabbitMQ, Kafka, MongoDB) manage their own delivery and retry semantics.

Default: File

```
# No config needed -- file is the default
# Optionally set a custom storage path (defaults to data/queue)
TINA4_QUEUE_PATH=./data/queue
```

RabbitMQ

```
TINA4_QUEUE_BACKEND=rabbitmq
TINA4_QUEUE_URL=amqp://user:pass@localhost:5672
```

Kafka

```
TINA4_QUEUE_BACKEND=kafka
TINA4_QUEUE_URL=kafka://localhost:9092
```

MongoDB

```
TINA4_QUEUE_BACKEND=mongodb
TINA4_QUEUE_URL=mongodb://user:pass@localhost:27017/tina4
```

`TINA4_QUEUE_BACKEND` selects the backend, `TINA4_QUEUE_PATH` sets the file backend's storage directory, and `TINA4_QUEUE_URL` carries the connection string for rabbitmq, kafka, and mongodb. Your queue code does not change -- the same `$queue->push()` and `$queue->consume()` calls work with every backend.

---

10. Exercise: Build an Email Queue

Build a queue-based email system with failure handling.

Requirements

- Create these endpoints:

Method	Path	Description
`POST`	`/api/emails/send`	Queue an email for sending
`GET`	`/api/emails/queue`	Show pending email count
`GET`	`/api/emails/dead`	List dead letter jobs
`POST`	`/api/emails/retry`	Revive dead-letter jobs

- The email payload should include: `to` (required), `subject` (required), `body` (required)
- Create a consumer that processes the queue, simulating occasional failures
- When an email fails repeatedly, it should end up in dead letters

Test with:

```
# Queue an email
curl -X POST http://localhost:7145/api/emails/send \
  -H "Content-Type: application/json" \
  -d '{"to": "alice@example.com", "subject": "Welcome!", "body": "Thanks for signing up."}'

# Check queue size
curl http://localhost:7145/api/emails/queue

# Check dead letters
curl http://localhost:7145/api/emails/dead

# Revive dead letters
curl -X POST http://localhost:7145/api/emails/retry
```

---

11. Solution

Create `src/routes/email-queue.php`:

```
<?php
use Tina4\Router;
use Tina4\Queue;

$queue = new Queue(topic: 'emails');

/**
 * @noauth
 */
Router::post("/api/emails/send", function ($request, $response) use ($queue) {
    $body = $request->body;

    $errors = [];
    if (empty($body["to"])) $errors[] = "'to' is required";
    if (empty($body["subject"])) $errors[] = "'subject' is required";
    if (empty($body["body"])) $errors[] = "'body' is required";

    if (!empty($errors)) {
        return $response->json(["errors" => $errors], 400);
    }

    $messageId = $queue->push([
        "to" => $body["to"],
        "subject" => $body["subject"],
        "body" => $body["body"]
    ]);

    return $response->json([
        "message" => "Email queued for sending",
        "message_id" => $messageId
    ], 201);
});

Router::get("/api/emails/queue", function ($request, $response) use ($queue) {
    $count = $queue->size();
    return $response->json(["pending" => $count]);
});

Router::get("/api/emails/dead", function ($request, $response) use ($queue) {
    $deadJobs = $queue->deadLetters();
    $items = [];
    foreach ($deadJobs as $job) {
        $items[] = [
            "id" => $job["id"],
            "payload" => $job["payload"],
            "error" => $job["error"] ?? null
        ];
    }
    return $response->json(["dead_letters" => $items, "count" => count($items)]);
});

Router::post("/api/emails/retry", function ($request, $response) use ($queue) {
    // Revive every dead-letter job back to the pending queue
    $queue->retry();
    return $response->json(["message" => "Dead-letter emails re-queued"]);
```

```
});
```

Create a separate consumer file `src/workers/email_worker.php`:

```
<?php
use Tina4\Queue;

$queue = new Queue(topic: 'emails');

foreach ($queue->consume('emails') as $job) {
    $payload = $job->payload;

    echo "Sending email to {$payload['to']}...\n";
    echo "  Subject: {$payload['subject']}\n";

    try {
        // Simulate sending (replace with real email logic)
        sleep(1);

        // Simulate failure for a specific address
        if ($payload['to'] === 'bad@example.com') {
            throw new \RuntimeException("SMTP connection refused");
        }

        echo "  Email sent to {$payload['to']} successfully!\n";
        $job->complete();

    } catch (\Throwable $e) {
        echo "  Failed: {$e->getMessage()}\n";
        // Automatic retry -> dead-letter. No manual re-queue needed.
        $job->fail($e->getMessage());
    }
}
```

The consumer calls `$job->fail()` whenever an email fails. The queue re-queues that job automatically, so the same worker retries it on its next pass. After `maxRetries` attempts (3 by default) the job to `bad@example.com` lands in dead letters, where `$queue->deadLetters()` returns it. The `/api/emails/dead` endpoint shows it. You investigate, fix the address, and call `/api/emails/retry` to revive it.

> *The consumer above is a long-running poll -- it keeps waiting for new jobs and never returns on its own. Run it as a dedicated worker process. For a one-shot drain (for example in a test), use* `$queue->consume('emails', null, 0)`*.*

---

12. Gotchas

1. consume runs forever by default

Problem: Your script calls `consume` and never reaches the code after the loop.

Cause: `consume` is a long-running poll. With the default `pollInterval` of `1.0`, it sleeps and keeps polling when the queue is empty -- it does not return.

Fix: That is correct for a background worker. For a single-pass drain (tests, one-shot jobs), pass `pollInterval` of `0`: `$queue->consume('emails', null, 0)`.

2. Calling complete or fail on a pop() result

Problem: `$job->complete()` or `$job->payload` errors after `pop()`.

Cause: `pop()` returns a plain **array**, not a `Job` object. There is no `->payload` property and no lifecycle methods on it.

Fix: Use array access (`$job['payload']`) for `pop()` results, and do not call `complete()`/`fail()` -- the job is already removed from the pending queue. If you want `Job` objects with lifecycle methods, use `consume` instead.

3. Letting jobs fail silently

Problem: Jobs that error are lost instead of retried.

Cause: Your consumer catches the exception but never calls `$job->fail()`, so the queue never records the failure or schedules a retry.

Fix: In a `consume` loop, call `$job->complete()` on success and `$job->fail($reason)` on failure. `fail()` is what drives the automatic retry -> dead-letter lifecycle.

4. Worker not picking up messages

Problem: Messages are pushed but nothing happens.

Cause: No consumer process is running, or the consumer is listening on a different topic.

Fix: Make sure the consumer is running. Check that the topic name in `$queue->push()` matches the topic in `$queue->consume()`.

5. Payload must be JSON-serializable

Problem: Your payload comes back wrong or empty.

Cause: You passed an object, database connection, file handle, or other non-serializable value. Jobs are stored as JSON.

Fix: Payload must contain only simple types: strings, numbers, booleans, and arrays of these. Pass IDs, not objects. The consumer looks up records by ID.

6. Dead letters pile up

Problem: Dead letters accumulate and nobody notices.

Cause: Jobs that exhaust `maxRetries` become dead letters and stay there until you act on them.

Fix: Monitor dead letters with `$queue->deadLetters()` or `$queue->size("dead")`. Alert when the count crosses a threshold. Fix the root cause, then `$queue->retry()` to revive them or `$queue->purge("dead")` to clear them.

7. File backend for production

Problem: Many workers contend on the file backend.

Cause: The file backend is designed for single-worker setups.

Fix: For production with multiple workers, switch to RabbitMQ, Kafka, or MongoDB via the `TINA4_QUEUE_BACKEND` environment variable.

Events

1. Decouple Your Code

A user registers. You need to send a welcome email, create a default profile, assign a free trial, and log the event. Without events, your registration handler does all of it directly. It grows. It imports five different services. It becomes impossible to test in isolation.

Events invert that. The registration handler emits one event: `user.registered`. Five listeners respond to it independently. Each listener is small, single-purpose, and testable on its own. Adding a sixth action means adding one new listener. No changes to the registration handler.

Tina4 has a built-in events system. No external message broker. No configuration. Zero dependencies.

---

2. Listening for Events

Register a listener with `Events::on()`. The listener fires every time the event is emitted.

```
<?php
use Tina4\Events;

Events::on('user.registered', function (array $data): void {
    echo "Send welcome email to: {$data['email']}\n";
});

Events::on('user.registered', function (array $data): void {
    echo "Create default profile for user {$data['id']}\n";
});
```

Multiple listeners on the same event all run. Registration order is the default execution order.

---

3. Emitting Events

Emit an event with `Events::emit()`. Every registered listener for that event fires synchronously in registration order.

```
<?php
use Tina4\Events;

// Register listeners (usually done at app boot)
Events::on('user.registered', function (array $data): void {
    echo "Welcome email -> {$data['email']}\n";
});

Events::on('user.registered', function (array $data): void {
    echo "Default profile -> user {$data['id']}\n";
});

Events::on('user.registered', function (array $data): void {
```

```
    echo "Free trial started for {$data['name']}\n";
});

// Emit the event (usually done inside a route handler or service)
Events::emit('user.registered', [
    'id'    => 42,
    'name'  => 'Alice',
    'email' => 'alice@example.com',
    'plan'  => 'free'
]);
```

Output:

```
Welcome email -> alice@example.com
Default profile -> user 42
Free trial started for Alice
```

All three listeners ran. The emitter does not know they exist.

---

4. One-Shot Listeners with once()

`Events::once()` registers a listener that fires exactly once, then removes itself.

```
<?php
use Tina4\Events;

// This listener fires on the first emit only
Events::once('app.boot', function (): void {
    echo "Database connection pool initialized\n";
});

Events::emit('app.boot');   // Fires: "Database connection pool initialized"
Events::emit('app.boot');   // Nothing -- listener was already removed
Events::emit('app.boot');   // Nothing
```

Useful for one-time initialization, cache warmup, or any setup that must run exactly once regardless of how many times the event fires.

---

5. Removing Listeners with off()

`Events::off()` removes a specific listener or all listeners for an event.

```
<?php
use Tina4\Events;

$auditListener = function (array $data): void {
    echo "Audit log: order {$data['id']} placed\n";
};

// Register
Events::on('order.placed', $auditListener);

// Works
Events::emit('order.placed', ['id' => 101]);
```

```
// Output: Audit log: order 101 placed

// Remove this specific listener
Events::off('order.placed', $auditListener);

// Listener no longer fires
Events::emit('order.placed', ['id' => 102]);
// Output: (nothing)
```

Remove all listeners for an event:

```
Events::off('order.placed');
```

After this call, `Events::emit('order.placed', ...)` fires nothing.

---

6. Priority

Listeners with a higher priority run before those with lower priority. Default priority is 0. Pass priority as the third argument to `Events::on()`.

```
<?php
use Tina4\Events;

Events::on('payment.received', function (array $data): void {
    echo "3. Send receipt email\n";
}, 0);   // Priority 0 -- runs third

Events::on('payment.received', function (array $data): void {
    echo "1. Record payment in ledger\n";
}, 10);  // Priority 10 -- runs first

Events::on('payment.received', function (array $data): void {
    echo "2. Update subscription status\n";
}, 5);   // Priority 5 -- runs second

Events::emit('payment.received', ['amount' => 99.00, 'currency' => 'USD']);
```

Output:

```
1. Record payment in ledger
2. Update subscription status
3. Send receipt email
```

Higher number = higher priority = runs earlier.

---

7. Events in Route Handlers

The typical pattern: emit events from route handlers, define listeners in a separate boot file.

`src/boot/events.php` - register all listeners at startup:

```
<?php
use Tina4\Events;

// User events
```

```
Events::on('user.registered', function (array $data): void {
    // Send welcome email via Messenger
    \Tina4\Messenger::send(
        to: $data['email'],
        subject: 'Welcome to the app!',
        body: "Hi {$data['name']}, your account is ready."
    );
});

Events::on('user.registered', function (array $data): void {
    // Queue a follow-up drip email for day 3
    $queue = new \Tina4\Queue(topic: 'drip-emails');
    $queue->push([
        'user_id'   => $data['id'],
        'template'  => 'day3-followup',
        'send_at'   => time() + (3 * 86400)
    ]);
});

// Order events
Events::on('order.completed', function (array $data): void {
    error_log("[order] completed: #{$data['id']} total={$data['total']}");
});
```

`src/routes/users.php` - emit from the route handler:

```
<?php
use Tina4\Router;
use Tina4\Events;

Router::post('/api/users/register', function ($request, $response) {
    $body = $request->body;

    if (empty($body['email']) || empty($body['name'])) {
        return $response->json(['error' => 'email and name are required'], 400);
    }

    // Create user (simplified)
    $user = [
        'id'    => rand(1000, 9999),
        'name'  => $body['name'],
        'email' => $body['email'],
        'plan'  => $body['plan'] ?? 'free'
    ];

    // Emit - listeners handle the rest
    Events::emit('user.registered', $user);

    return $response->json([
        'message' => 'Registration successful',
        'user_id' => $user['id']
    ], 201);
});
```

```
curl -X POST http://localhost:7145/api/users/register \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com"}'
```

```
{
  "message": "Registration successful",
```

```
  "user_id": 4721
}
```

The route handler responds immediately. The listeners handle the rest.

---

8. Checking Registered Listeners

Inspect which events have listeners (useful in tests and during development):

```
<?php
use Tina4\Events;

Events::on('order.placed', function (): void {});
Events::on('order.placed', function (): void {});
Events::on('payment.failed', function (): void {});

$count = count(Events::listeners('order.placed')); // 2
$names = Events::events();                          // ['order.placed', 'payment.failed']
$has   = Events::listeners('order.placed') !== []; // true
```

In tests, reset all events between test cases:

```
Events::clear();
```

---

9. Exercise: Order Lifecycle Events

Build an order system where each state change emits an event and multiple listeners respond.

Requirements

- Define listeners for: `order.created`, `order.paid`, `order.shipped`
- Create these endpoints:

Method	Path	Description
`POST`	`/api/orders`	Create order, emit `order.created`
`POST`	`/api/orders/{id}/pay`	Mark paid, emit `order.paid`
`POST`	`/api/orders/{id}/ship`	Mark shipped, emit `order.shipped`

- Each event should have at least two listeners (e.g., logging + notification)

Test with:

```
curl -X POST http://localhost:7145/api/orders \
  -H "Content-Type: application/json" \
  -d '{"customer": "Alice", "items": ["Widget A", "Widget B"], "total": 59.98}'

curl -X POST http://localhost:7145/api/orders/1001/pay
curl -X POST http://localhost:7145/api/orders/1001/ship
```

---

10. Solution

src/boot/order-events.php:

```
<?php
use Tina4\Events;

Events::on('order.created', function (array $order): void {
    error_log("[order] created #{$order['id']} for {$order['customer']}");
}, 10);

Events::on('order.created', function (array $order): void {
    // Notify warehouse
    error_log("[warehouse] new order #{$order['id']} -> " . implode(', ', $order['items']));
}, 5);

Events::on('order.paid', function (array $order): void {
    error_log("[payment] received for order #{$order['id']} - \${$order['total']}");
}, 10);

Events::on('order.paid', function (array $order): void {
    // Trigger fulfillment
    error_log("[fulfillment] queue pick-and-pack for order #{$order['id']}");
}, 5);

Events::on('order.shipped', function (array $order): void {
    error_log("[shipping] order #{$order['id']} dispatched -> {$order['customer']}");
}, 10);

Events::on('order.shipped', function (array $order): void {
    // Send tracking email
    error_log("[email] tracking notification sent for order #{$order['id']}");
}, 5);
```

src/routes/orders.php:

```
<?php
use Tina4\Router;
use Tina4\Events;

$orders = [];

Router::post('/api/orders', function ($request, $response) use (&$orders) {
    $body = $request->body;
    $order = [
        'id'       => rand(1000, 9999),
        'customer' => $body['customer'] ?? 'Unknown',
        'items'    => $body['items'] ?? [],
        'total'    => $body['total'] ?? 0,
        'status'   => 'created'
    ];
    $orders[$order['id']] = $order;
    Events::emit('order.created', $order);
    return $response->json(['message' => 'Order created', 'order' => $order], 201);
});

Router::post('/api/orders/{id:int}/pay', function ($request, $response) use (&$orders) {
```

```
    $id = $request->params['id'];
    if (!isset($orders[$id])) {
        return $response->json(['error' => 'Order not found'], 404);
    }
    $orders[$id]['status'] = 'paid';
    Events::emit('order.paid', $orders[$id]);
    return $response->json(['message' => 'Order marked as paid', 'order' => $orders[$id]]);
});

Router::post('/api/orders/{id:int}/ship', function ($request, $response) use (&$orders) {
    $id = $request->params['id'];
    if (!isset($orders[$id])) {
        return $response->json(['error' => 'Order not found'], 404);
    }
    $orders[$id]['status'] = 'shipped';
    Events::emit('order.shipped', $orders[$id]);
    return $response->json(['message' => 'Order shipped', 'order' => $orders[$id]]);
});
```

---

11. Gotchas

1. Listeners fire synchronously

Problem: A slow listener blocks the HTTP response.

Cause: `Events::emit()` runs all listeners in the same thread before returning. A listener that takes 2 seconds delays the response by 2 seconds.

Fix: For slow work (emails, PDF generation), push to a queue inside the listener instead of doing the work inline.

2. Uncaught exceptions in listeners crash the emit

Problem: One bad listener prevents the remaining listeners from running.

Cause: An exception propagates up from the listener and aborts the emit loop.

Fix: Wrap listener bodies in try/catch, or register a global error handler on the event system. At minimum, log the error and continue.

3. Forgetting to reset in tests

Problem: Tests interfere with each other because a listener registered in test A fires during test B.

Cause: Events are global state. Listeners accumulate across tests.

Fix: Call `Events::clear()` in your test `setUp` or `tearDown` method.

4. Using closures with off()

Problem: `Events::off('event', $listener)` does not remove the listener.

Cause: You registered an anonymous closure inline and are trying to pass a different closure reference to `off()`.

Fix: Assign the closure to a named variable before registering it, then pass that same variable to `off()`.

Localization

1. One App, Many Languages

Your app goes live. Users arrive from Germany, Brazil, Japan. Everything is in English. They bounce.

Localization (L10n) makes the same app speak every language. UI strings, error messages, date formats, and number formatting adapt to the user's locale. The code stays the same.

Tina4 provides a built-in `I18n` class that loads JSON locale files, interpolates variables, falls back to a default locale when a key is missing, and requires no external packages.

---

2. Locale Files

Store translations as JSON files. One file per locale. Place them in `src/locales/` by convention.

`src/locales/en.json`:

```
{
  "welcome": "Welcome, {name}!",
  "goodbye": "Goodbye, {name}.",
  "errors": {
    "required": "The {field} field is required.",
    "min_length": "The {field} must be at least {min} characters.",
    "not_found": "The requested resource was not found."
  },
  "orders": {
    "count": "You have {count} order(s).",
    "empty": "You have no orders yet."
  },
  "nav": {
    "home": "Home",
    "products": "Products",
    "account": "My Account",
    "logout": "Log Out"
  }
}
```

`src/locales/de.json`:

```
{
  "welcome": "Willkommen, {name}!",
  "goodbye": "Auf Wiedersehen, {name}.",
  "errors": {
    "required": "Das Feld {field} ist erforderlich.",
    "min_length": "Das Feld {field} muss mindestens {min} Zeichen lang sein.",
    "not_found": "Die angeforderte Ressource wurde nicht gefunden."
  },
  "orders": {
    "count": "Sie haben {count} Bestellung(en).",
    "empty": "Sie haben noch keine Bestellungen."
  },
```

```
  "nav": {
    "home": "Startseite",
    "products": "Produkte",
    "account": "Mein Konto",
    "logout": "Abmelden"
  }
}
```

`src/locales/pt.json`:

```
{
  "welcome": "Bem-vindo, {name}!",
  "goodbye": "Até logo, {name}.",
  "errors": {
    "required": "O campo {field} é obrigatório.",
    "not_found": "O recurso solicitado não foi encontrado."
  },
  "nav": {
    "home": "Início",
    "products": "Produtos",
    "account": "Minha Conta",
    "logout": "Sair"
  }
}
```

---

3. Loading and Using I18n

```
<?php
use Tina4\I18n;

// Load locale files
$i18n = new I18n('en', 'src/locales');

// Set the active locale
$i18n->setLocale('de');

// Translate a key
echo $i18n->t('welcome', ['name' => 'Alice']);
// Output: Willkommen, Alice!

echo $i18n->t('errors.not_found');
// Output: Die angeforderte Ressource wurde nicht gefunden.

echo $i18n->t('orders.count', ['count' => 3]);
// Output: Sie haben 3 Bestellung(en).
```

Dot notation navigates nested keys. `'errors.required'` accesses `$translations['errors']['required']`.

---

4. Fallback Locale

When a key exists in `en.json` but not in the active locale file, `I18n` falls back to the default locale automatically.

```php
<?php
use Tina4\I18n;

$i18n = new I18n('en', 'src/locales');
$i18n->setLocale('pt');

// 'orders.count' missing from pt.json -- falls back to en.json
echo $i18n->t('orders.count', ['count' => 2]);
// Output: You have 2 order(s).

// 'nav.home' exists in pt.json
echo $i18n->t('nav.home');
// Output: Início
```

Fallback prevents blank strings from appearing in partially translated apps. Ship the English version first. Translations can come later without breaking anything.

---

5. Locale Switching in API Routes

Detect the locale from the `Accept-Language` header, a query parameter, or the user's profile. Then set it on the `I18n` instance.

```php
<?php
use Tina4\Router;
use Tina4\I18n;

$i18n = new I18n('en', 'src/locales');

function resolveLocale($request, I18n $i18n): string {
    // 1. Query parameter: ?lang=de
    if (!empty($request->params['lang'])) {
        return $request->params['lang'];
    }

    // 2. Accept-Language header: Accept-Language: de-DE,de;q=0.9
    $header = $request->headers['Accept-Language'] ?? 'en';
    $primary = explode(',', $header)[0];
    $lang = explode('-', $primary)[0];

    // 3. Fall back to default
    return $i18n->hasLocale($lang) ? $lang : 'en';
}

/**
 * @noauth
 */
Router::get('/api/greeting', function ($request, $response) use ($i18n) {
    $locale = resolveLocale($request, $i18n);
    $i18n->setLocale($locale);

    $name = $request->params['name'] ?? 'stranger';

    return $response->json([
        'locale'  => $locale,
        'message' => $i18n->t('welcome', ['name' => $name]),
        'nav'     => [
```

```
            'home'     => $i18n->t('nav.home'),
            'products' => $i18n->t('nav.products'),
            'account'  => $i18n->t('nav.account')
        ]
    ]);
});
```

```
curl "http://localhost:7145/api/greeting?name=Alice&lang=de"
```

```
{
  "locale": "de",
  "message": "Willkommen, Alice!",
  "nav": {
    "home": "Startseite",
    "products": "Produkte",
    "account": "Mein Konto"
  }
}
```

```
curl "http://localhost:7145/api/greeting?name=Alice&lang=pt"
```

```
{
  "locale": "pt",
  "message": "Bem-vindo, Alice!",
  "nav": {
    "home": "Início",
    "products": "Produtos",
    "account": "Minha Conta"
  }
}
```

---

6. Interpolation

Placeholders use `{key}` syntax. Pass values as an associative array.

```
<?php
use Tina4\I18n;

$i18n = new I18n('en', 'src/locales');
$i18n->setLocale('en');

// Single value
echo $i18n->t('welcome', ['name' => 'Bob']);
// Welcome, Bob!

// Multiple values
echo $i18n->t('errors.min_length', [
    'field' => 'password',
    'min'   => 8
]);
// The password must be at least 8 characters.

// Missing placeholder value -- left as-is
echo $i18n->t('welcome');
// Welcome, {name}!
```

No special escaping needed. Values are inserted as-is.

---

7. Checking Available Locales

```
<?php
use Tina4\I18n;

$i18n = new I18n('en', 'src/locales');

// List all loaded locales
$locales = $i18n->getLocales();
// ['en', 'de', 'pt']

// Check if a locale exists
$has = $i18n->hasLocale('fr');  // false
$has = $i18n->hasLocale('de');  // true

// Get the current active locale
$current = $i18n->getLocale();  // 'en' (default)
```

Expose available locales via API for frontend locale switchers:

```
Router::get('/api/locales', function ($request, $response) use ($i18n) {
    return $response->json([
        'available' => $i18n->getLocales(),
        'default'   => 'en'
    ]);
});
```

---

8. Localized Validation Errors

Return validation errors in the user's language:

```
<?php
use Tina4\Router;
use Tina4\I18n;

$i18n = new I18n('en', 'src/locales');

Router::post('/api/users', function ($request, $response) use ($i18n) {
    $lang = $request->params['lang'] ?? 'en';
    $i18n->setLocale($lang);
    $body = $request->body;

    $errors = [];

    if (empty($body['name'])) {
        $errors[] = $i18n->t('errors.required', ['field' => 'name']);
    }

    if (empty($body['email'])) {
        $errors[] = $i18n->t('errors.required', ['field' => 'email']);
    }

    if (!empty($body['password']) && strlen($body['password']) < 8) {
        $errors[] = $i18n->t('errors.min_length', ['field' => 'password', 'min' => 8]);
```

```
    }

    if (!empty($errors)) {
        return $response->json(['errors' => $errors], 422);
    }

    return $response->json(['message' => 'User created'], 201);
});
```

```
curl -X POST "http://localhost:7145/api/users?lang=de" \
  -H "Content-Type: application/json" \
  -d '{"password": "abc"}'
```

```
{
  "errors": [
    "Das Feld name ist erforderlich.",
    "Das Feld email ist erforderlich.",
    "Das Feld password muss mindestens 8 Zeichen lang sein."
  ]
}
```

---

9. Adding Locales at Runtime

Load additional locale data programmatically (useful when translations come from a database):

```
<?php
use Tina4\I18n;

$i18n = new I18n('en', 'src/locales');

// Add or extend a locale at runtime - one key at a time
$i18n->addTranslation('fr', 'welcome', 'Bienvenue, {name} !');
$i18n->addTranslation('fr', 'goodbye', 'Au revoir, {name}.');
$i18n->addTranslation('fr', 'nav.home', 'Accueil');
$i18n->addTranslation('fr', 'nav.products', 'Produits');
$i18n->addTranslation('fr', 'nav.account', 'Mon Compte');
$i18n->addTranslation('fr', 'nav.logout', 'Déconnexion');

$i18n->setLocale('fr');
echo $i18n->t('welcome', ['name' => 'Alice']);
// Bienvenue, Alice !
```

---

10. Exercise: Multilingual API

Build a product API that returns localized content.

Requirements

- Create locale files for `en`, `de`, and one other language of your choice with keys for:
- `product.in_stock`, `product.out_of_stock`
- `product.price` (e.g., "Price: {amount}")
- `errors.not_found`

- Create `GET /api/products/{id}` that:
- Accepts `?lang=` query parameter
- Returns product data with localized status and label strings
- Create `GET /api/locales` that returns the list of available locales

Test with:

```
curl "http://localhost:7145/api/products/1?lang=en"
curl "http://localhost:7145/api/products/1?lang=de"
curl "http://localhost:7145/api/products/999?lang=de"
curl "http://localhost:7145/api/locales"
```

---

11. Gotchas

1. Missing keys return the key name

Problem: The UI shows `errors.not_found` instead of a message.

Cause: The key does not exist in the active locale, and the default locale (the first constructor argument) also lacks it.

Fix: Pass `'en'` as the default locale (`new I18n('en', 'src/locales')`) and ensure the English locale file has all keys. It is the source of truth.

2. Locale files not loaded

Problem: `$i18n->t('welcome')` returns the raw key on all locales.

Cause: The locale directory path is wrong, or JSON files have a syntax error.

Fix: Verify the path passed to `new I18n()` is correct relative to the project root. Validate JSON files with `php -r "echo json_encode(json_decode(file_get_contents('src/locales/en.json')));"`.

3. Placeholder not replaced

Problem: Output shows `Welcome, {name}!` with the literal `{name}`.

Cause: You called `$i18n->t('welcome')` without passing the values array.

Fix: Always pass replacement values: `$i18n->t('welcome', ['name' => $user['name']])`.

4. Locale not resetting between requests

Problem: After one request with `?lang=de`, subsequent requests without a `lang` parameter return German text.

Cause: The `I18n` instance is shared and locale state persists across the request lifecycle if the instance lives in a service container.

Fix: Always call `$i18n->setLocale($locale)` at the start of each request before calling `t()`. Do not assume the locale from a previous request.

Structured Logging

1. Stop Using error_log

`error_log("Something happened")` works. It also produces an unreadable wall of text in production. Searching for a specific error across 500,000 lines of plain text is painful. Correlating a log line with a request, a user, and a timestamp is manual detective work.

Structured logging writes JSON. Every log entry is a machine-readable object with a timestamp, level, message, and whatever context you attach. Log aggregators (Datadog, Grafana Loki, AWS CloudWatch, Papertrail) can query, filter, and alert on structured logs.

Tina4 provides `Tina4\Log` for structured logging. Each level has its own method: `Log::debug()`, `Log::info()`, `Log::warning()`, `Log::error()`, `Log::critical()`. Output is JSON by default. Zero external packages.

The historical `Tina4\Debug::message()` API still works as a compatibility shim that forwards to `Tina4\Log`; use it if you're upgrading from a v3.12.x codebase. New code should use the level-specific `Log` methods.

---

2. Log Levels

Tina4 has five log levels, each with its own `Log` method:

Method	When to use
`Log::debug($msg, $context)`	Verbose detail for development
`Log::info($msg, $context)`	Normal operations, confirmations
`Log::warning($msg, $context)`	Something unexpected but recoverable
`Log::error($msg, $context)`	A failure that needs attention
`Log::critical($msg, $context)`	System is failing. Immediate action required

Higher levels are always visible. Lower levels are filtered by `TINA4_LOG_LEVEL`.

---

3. Basic Logging

```
<?php
use Tina4\Log;

Log::info("Application started");
Log::debug("Cache miss for key: product:42");
Log::warning("Payment gateway responded slowly");
Log::error("Database query failed");
Log::critical("Out of disk space");
```

Output (JSON format):

```
{"timestamp":"2026-04-02T14:30:01Z","level":"INFO","message":"Application started"}
{"timestamp":"2026-04-02T14:30:01Z","level":"DEBUG","message":"Cache miss for key: product:42"}
{"timestamp":"2026-04-02T14:30:01Z","level":"WARNING","message":"Payment gateway responded slowl
{"timestamp":"2026-04-02T14:30:01Z","level":"ERROR","message":"Database query failed"}
{"timestamp":"2026-04-02T14:30:01Z","level":"CRITICAL","message":"Out of disk space"}
```

Each line is a complete JSON object. One per log entry.

---

4. Log Level Filtering

Set the minimum log level via environment variable. Only messages at or above that level appear in the output.

```
# .env
TINA4_LOG_LEVEL=WARNING
```

With this setting:

```
<?php
use Tina4\Log;

Log::debug("Detailed SQL query");        // Suppressed
Log::info("User logged in");             // Suppressed
Log::warning("Slow query detected");     // Appears
Log::error("Auth service unreachable");  // Appears
Log::critical("Disk full");              // Appears
```

Recommended levels by environment:

Environment	TINA4_LOG_LEVEL
Development	`DEBUG`
Staging	`INFO`
Production	`WARNING`

---

5. Logging with Context

Pass a context array as the second argument. Context values are merged into the JSON log entry.

```
<?php
use Tina4\Log;

Log::info("User login attempt", [
    'user_id'    => 42,
    'email'      => 'alice@example.com',
    'ip'         => '203.0.113.5',
    'user_agent' => 'Mozilla/5.0 ...'
]);
```

Output:

```
{
  "timestamp": "2026-04-02T14:30:01Z",
  "level": "INFO",
  "message": "User login attempt",
  "user_id": 42,
  "email": "alice@example.com",
  "ip": "203.0.113.5",
  "user_agent": "Mozilla/5.0 ..."
}
```

Context makes log lines searchable: `level:INFO AND user_id:42` finds every action by that user.

---

6. Logging in Route Handlers

Log the lifecycle of an HTTP request. Incoming requests, decisions made, and outcomes:

```
<?php
use Tina4\Router;
use Tina4\Log;

Router::post('/api/payments', function ($request, $response) {
    $body = $request->body;

    Log::info("Payment request received", [
        'amount'   => $body['amount'] ?? null,
        'currency' => $body['currency'] ?? 'USD',
        'ip'       => $request->server['REMOTE_ADDR'] ?? null
    ]);

    if (empty($body['amount']) || $body['amount'] <= 0) {
        Log::warning("Payment rejected: invalid amount", [
            'amount' => $body['amount'] ?? 'missing'
        ]);
        return $response->json(['error' => 'Invalid payment amount'], 400);
    }

    // Simulate payment processing
    $success = true;
    $transactionId = 'txn_' . uniqid();

    if ($success) {
        Log::info("Payment processed successfully", [
            'transaction_id' => $transactionId,
            'amount'         => $body['amount'],
            'currency'       => $body['currency'] ?? 'USD'
        ]);
        return $response->json(['transaction_id' => $transactionId], 201);
    } else {
        Log::error("Payment gateway failure", [
            'amount'   => $body['amount'],
            'gateway'  => 'stripe',
            'reason'   => 'Gateway timeout'
        ]);
```

```
        return $response->json(['error' => 'Payment failed'], 502);
    }
});
```

---

7. Logging Exceptions

Log exceptions with full context. Include the message, file, line, and trace:

```
<?php
use Tina4\Log;

function processOrder(array $order): array {
    try {
        // Simulate work that might fail
        if (empty($order['items'])) {
            throw new \InvalidArgumentException("Order must have at least one item");
        }

        return ['status' => 'processed', 'order_id' => $order['id']];

    } catch (\InvalidArgumentException $e) {
        Log::warning("Order validation failed", [
            'order_id' => $order['id'] ?? null,
            'error'    => $e->getMessage()
        ]);
        throw $e;

    } catch (\Throwable $e) {
        Log::error("Unexpected error processing order", [
            'order_id' => $order['id'] ?? null,
            'error'    => $e->getMessage(),
            'file'     => $e->getFile(),
            'line'     => $e->getLine(),
            'trace'    => $e->getTraceAsString()
        ]);
        throw $e;
    }
}
```

---

8. Request Correlation

Attach a request ID to every log message within a request. All log lines from the same request share one ID. This lets you filter a full request trace from thousands of concurrent requests.

```
<?php
use Tina4\Router;
use Tina4\Log;

Router::any('*', function ($request, $response) {
    // Generate a request ID and attach to all logs in this request
    $requestId = bin2hex(random_bytes(8));
    $GLOBALS['request_id'] = $requestId;

    Log::debug("Request started", [
```

```
        'request_id' => $requestId,
        'method'     => $request->method,
        'path'       => $request->server['REQUEST_URI'] ?? '/'
    ]);

    return null; // Pass through to the real handler
}, 'next');

function logWithRequest(string $level, string $message, array $ctx = []): void {
    $merged = array_merge(['request_id' => $GLOBALS['request_id'] ?? 'none'], $ctx);
    Log::{$level}($message, $merged);
}
```

With request correlation, a single `grep "request_id:a3f8c1d2"` finds every log line for one HTTP request.

---

9. Performance Logging

Log slow operations automatically. Time your expensive calls:

```
<?php
use Tina4\Log;

function timedQuery(callable $query, string $label, float $warnThresholdMs = 100): mixed {
    $start = microtime(true);
    $result = $query();
    $durationMs = (microtime(true) - $start) * 1000;

    $context = [
        'label'        => $label,
        'duration_ms'  => round($durationMs, 2),
        'threshold_ms' => $warnThresholdMs,
        'slow'         => $durationMs > $warnThresholdMs,
    ];

    if ($durationMs > $warnThresholdMs) {
        Log::warning("Query timing: {$label}", $context);
    } else {
        Log::debug("Query timing: {$label}", $context);
    }

    return $result;
}

// Usage
$products = timedQuery(
    fn() => fetchAllProducts(),
    'fetchAllProducts',
    warnThresholdMs: 50
);
```

Any query slower than 50ms emits a `WARNING`. All others emit `DEBUG` and are suppressed in production.

---

10. Environment Configuration

```
# Minimum level to log (DEBUG | INFO | WARNING | ERROR | CRITICAL)
TINA4_LOG_LEVEL=WARNING

# Log output destination (stderr | stdout | file)
TINA4_LOG_OUTPUT=stderr

# Log file path (only used when TINA4_LOG_OUTPUT=file)
TINA4_LOG_FILE=./logs/app.log

# Log format (json | text)
TINA4_LOG_FORMAT=json
```

In development, use `DEBUG` level with `text` format for human-readable output. In production, use `WARNING` or `ERROR` level with `json` format for log aggregators.

---

11. Gotchas

1. Logging sensitive data

Problem: User passwords and payment card numbers appear in log files.

Cause: You logged the entire request body without filtering.

Fix: Never log raw `$request->body` or `$_POST`. Explicitly list the fields you want to log, excluding sensitive ones:

```
Log::info("User updated", [
    'user_id' => $body['id'],
    'email'   => $body['email'],
    // NOT: 'password' => $body['password']
]);
```

2. Logging too much in production

Problem: Log volume is enormous. Storage costs spike. Log aggregator rate limits trigger.

Cause: `TINA4_LOG_LEVEL=DEBUG` in production logs every cache miss, DB query, and template render.

Fix: Set `TINA4_LOG_LEVEL=WARNING` in production. Use `DEBUG` only in development.

3. Forgetting to log errors before rethrowing

Problem: A caught exception is rethrown but never logged. It disappears silently or appears only in a generic error handler with no context.

Fix: Always log before rethrowing. The context (order ID, user ID, inputs) is available inside the catch block. It is gone by the time the exception reaches the top-level handler.

Email with Messenger

1. Every App Sends Email

Your SaaS app needs signup confirmations. Password resets. Weekly digests. Attachments. HTML templates. Reliable delivery.

Email is plumbing. Every application needs it. Nobody wants to build it. SMTP configuration. Plain text fallbacks. Attachment encoding. Connection timeouts. Bounce handling. The details pile up.

Tina4's `Messenger` class handles all of it. Configure in `.env`. Create an instance. Send. Without an SMTP host, Messenger captures mail in the dev mailbox. An SMTP host enables real delivery, so staging needs an explicit safety setting.

---

2. Messenger Configuration via .env

All email configuration lives in `.env`:

```
TINA4_MAIL_HOST=smtp.example.com
TINA4_MAIL_PORT=587
TINA4_MAIL_USERNAME=your-email@example.com
TINA4_MAIL_PASSWORD=your-app-password
TINA4_MAIL_ENCRYPTION=tls
TINA4_MAIL_FROM=noreply@example.com
TINA4_MAIL_FROM_NAME=My Store
```

Variable	Description	Common Values
`TINA4_MAIL_HOST`	SMTP server hostname	`smtp.gmail.com`, `smtp.mailgun.org`, `smtp.sendgrid.net`
`TINA4_MAIL_PORT`	SMTP port	`587` (TLS), `465` (SSL), `25` (unencrypted)
`TINA4_MAIL_USERNAME`	Login username	Usually your email address
`TINA4_MAIL_PASSWORD`	Login password or app-specific password	App passwords for Gmail
`TINA4_MAIL_ENCRYPTION`	Encryption method	`tls` (recommended), `ssl`, `none`
`TINA4_MAIL_FROM`	Default "From" address	`noreply@yourdomain.com`
`TINA4_MAIL_FROM_NAME`	Default "From" display name	`My Store`, `Acme Corp`
`TINA4_MAIL_CAPTURE`	Disable SMTP and capture each message locally	`true` on staging when no external delivery is allowed

`TINA4_MAIL_REDIRECT_TO`	Replace every SMTP recipient with a safety list	`qa@example.com,product@example.com`

Staging delivery safety

`TINA4_DEBUG` controls developer tools. It does not disable SMTP delivery. Pick one explicit mail policy for staging:

```
# No message leaves the application. Inspect it in the dev mailbox.
TINA4_MAIL_CAPTURE=true
```

Capture wins even when an SMTP host exists. A missing SMTP host also captures mail. `TINA4_MAILBOX_DIR` only selects the dev mailbox directory; it does not turn capture on or off.

Use real SMTP without risking customer delivery by replacing all recipients:

```
TINA4_MAIL_REDIRECT_TO=qa@example.com,product@example.com
```

Tina4 trims each comma-separated address and drops blank entries. On the SMTP path, the safety list replaces `To`; Tina4 clears `Cc` and `Bcc` and stores all original recipients in `X-Tina4-Original-To`. An unset or empty list leaves delivery unchanged. If capture and redirect are both set, capture wins and nothing reaches SMTP.

Common Provider Configurations

Gmail:

```
TINA4_MAIL_HOST=smtp.gmail.com
TINA4_MAIL_PORT=587
TINA4_MAIL_USERNAME=your-email@gmail.com
TINA4_MAIL_PASSWORD=your-app-password
TINA4_MAIL_ENCRYPTION=tls
```

Note: Gmail requires an "App Password" (not your regular password) when two-factor authentication is enabled.

Mailgun:

```
TINA4_MAIL_HOST=smtp.mailgun.org
TINA4_MAIL_PORT=587
TINA4_MAIL_USERNAME=postmaster@mg.yourdomain.com
TINA4_MAIL_PASSWORD=your-mailgun-smtp-password
TINA4_MAIL_ENCRYPTION=tls
```

SendGrid:

```
TINA4_MAIL_HOST=smtp.sendgrid.net
TINA4_MAIL_PORT=587
TINA4_MAIL_USERNAME=apikey
TINA4_MAIL_PASSWORD=your-sendgrid-api-key
TINA4_MAIL_ENCRYPTION=tls
```

---

3. Constructor Override Pattern

Different purposes need different accounts. Transactional emails from one sender. Marketing from another. Override the configuration in the constructor:

```
<?php
use Tina4\Messenger;

// Uses .env defaults
$mailer = new Messenger();

// Override specific settings
$marketingMailer = new Messenger([
    "host" => "smtp.mailgun.org",
    "port" => 587,
    "username" => "marketing@mg.yourdomain.com",
    "password" => "marketing-smtp-password",
    "encryption" => "tls",
    "from_address" => "newsletter@yourdomain.com",
    "from_name" => "My Store Newsletter"
]);
```

The constructor accepts an associative array. Override any `.env` value. Unspecified keys fall back to the `.env` configuration.

---

4. Sending Plain Text Email

The simplest email:

```
<?php
use Tina4\Router;
use Tina4\Messenger;

Router::post("/api/contact", function ($request, $response) {
    $body = $request->body;

    $mailer = new Messenger();

    $result = $mailer->send(
        $body["email"],                    // To
        "Contact Form Submission",         // Subject
        "Name: " . $body["name"] . "\n" .  // Body (plain text)
        "Email: " . $body["email"] . "\n" .
        "Message:\n" . $body["message"]
    );

    if ($result["success"]) {
        return $response->json(["message" => "Email sent successfully"]);
    }

    return $response->json(["error" => "Failed to send email", "details" => $result["error"]], 5
});
```

```
curl -X POST http://localhost:7145/api/contact \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "message": "I love your products!"}'
```

```
{"message":"Email sent successfully"}
```

The `send()` method returns an array with `"success"` (boolean) and `"error"` (string, present only on failure).

The send() Method

```
$mailer->send($to, $subject, $body, $options = []);
```

- **$to**: Recipient email address (string)
- **$subject**: Email subject line (string)
- **$body**: Email body content (string -- plain text or HTML)
- **$options**: Optional settings (array)

---

5. Sending HTML Email with Text Fallback

Most emails should be HTML with a plain text fallback. Some email clients do not render HTML:

```
<?php
use Tina4\Messenger;

$mailer = new Messenger();

$htmlBody = '
<html>
<body style="font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto;">
    <div style="background: #1a1a2e; color: white; padding: 20px; text-align: center;">
        <h1 style="margin: 0;">Welcome to My Store!</h1>
    </div>
    <div style="padding: 20px;">
        <p>Hi Alice,</p>
        <p>Thank you for creating your account. We are excited to have you!</p>
        <p>Here is what you can do next:</p>
        <ul>
            <li>Browse our <a href="https://mystore.com/products">product catalog</a></li>
            <li>Set up your <a href="https://mystore.com/profile">profile</a></li>
            <li>Check out our <a href="https://mystore.com/deals">current deals</a></li>
        </ul>
        <p>If you have any questions, reply to this email and we will get back to you within 24
        <p>Cheers,<br>The My Store Team</p>
    </div>
    <div style="background: #f5f5f5; padding: 12px; text-align: center; font-size: 12px; color:
        <p>You received this because you signed up at mystore.com</p>
    </div>
</body>
</html>';

$textBody = "Hi Alice,\n\n" .
    "Thank you for creating your account. We are excited to have you!\n\n" .
    "Here is what you can do next:\n" .
    "- Browse our product catalog: https://mystore.com/products\n" .
    "- Set up your profile: https://mystore.com/profile\n" .
    "- Check out our current deals: https://mystore.com/deals\n\n" .
    "If you have any questions, reply to this email.\n\n" .
    "Cheers,\nThe My Store Team";
```

```
$result = $mailer->send(
    "alice@example.com",
    "Welcome to My Store!",
    $htmlBody,
    ["text_body" => $textBody]
);
```

The `text_body` option provides the plain text fallback. Email clients that cannot render HTML show the text version instead.

---

6. Adding Attachments

Attach files by path:

```
<?php
use Tina4\Messenger;

$mailer = new Messenger();

$result = $mailer->send(
    "accounting@example.com",
    "Monthly Invoice #1042",
    "<h2>Invoice #1042</h2><p>Please find the invoice attached.</p>",
    [
        "attachments" => [
            "/path/to/invoices/invoice-1042.pdf",
            "/path/to/reports/monthly-summary.csv"
        ]
    ]
);
```

Each attachment is an absolute file path. Tina4 reads the file, determines the MIME type, and encodes it for transmission.

Inline Attachments with Custom Names

```
$result = $mailer->send(
    "alice@example.com",
    "Your Export",
    "<p>Here is your data export.</p>",
    [
        "attachments" => [
            [
                "path" => "/tmp/export-20260322.csv",
                "name" => "my-store-export.csv"  // Custom filename
            ]
        ]
    ]
);
```

The recipient sees `my-store-export.csv` regardless of the actual filename on disk.

---

7. CC and BCC

```
<?php
use Tina4\Messenger;

$mailer = new Messenger();

$result = $mailer->send(
    "alice@example.com",
    "Team Meeting Notes",
    "<p>Here are the notes from today's meeting.</p>",
    [
        "cc" => ["bob@example.com", "charlie@example.com"],
        "bcc" => ["manager@example.com"],
        "reply_to" => "alice@example.com"
    ]
);
```

- **cc**: Array of email addresses to carbon copy. All recipients see CC addresses.
- **bcc**: Array of email addresses to blind carbon copy. Recipients cannot see BCC addresses.
- **reply_to**: When the recipient clicks "Reply," this address is used instead of the "From" address.

---

8. Custom Headers

Add custom headers to outgoing emails with `addHeader()`:

```
<?php
use Tina4\Messenger;

$mailer = new Messenger();

$mailer->addHeader("X-Priority", "1");
$mailer->addHeader("X-Mailer", "Tina4 Messenger");
$mailer->addHeader("List-Unsubscribe", "<https://mystore.com/unsubscribe?token=abc123>");

$result = $mailer->send(
    "alice@example.com",
    "Important Update",
    "<p>Your account requires attention.</p>"
);
```

Headers persist for the lifetime of the `Messenger` instance. Create a fresh instance if you need different headers for the next email.

Common custom headers:

Header	Purpose
`X-Priority`	`1` (high), `3` (normal), `5` (low)
`X-Mailer`	Identifies the sending application

`List-Unsubscribe`	One-click unsubscribe link (required by some providers)
`X-Entity-Ref-ID`	Prevents email threading in some clients
`Precedence`	`bulk` for mass emails, `list` for mailing lists

---

9. Reading Inbox via IMAP

Tina4's Messenger reads emails via IMAP:

```
TINA4_MAIL_IMAP_HOST=imap.example.com
TINA4_MAIL_IMAP_PORT=993
TINA4_MAIL_IMAP_USERNAME=support@example.com
TINA4_MAIL_IMAP_PASSWORD=your-imap-password
TINA4_MAIL_IMAP_ENCRYPTION=ssl
```

```
<?php
use Tina4\Router;
use Tina4\Messenger;

Router::get("/api/inbox", function ($request, $response) {
    $mailer = new Messenger();

    $emails = $mailer->getInbox([
        "limit" => 20,
        "unread_only" => true
    ]);

    $messages = [];
    foreach ($emails as $email) {
        $messages[] = [
            "id" => $email["id"],
            "from" => $email["from"],
            "subject" => $email["subject"],
            "date" => $email["date"],
            "preview" => substr($email["text_body"], 0, 200),
            "has_attachments" => !empty($email["attachments"])
        ];
    }

    return $response->json(["messages" => $messages, "count" => count($messages)]);
});
```

```
curl http://localhost:7145/api/inbox
```

```
{
  "messages": [
    {
      "id": "12345",
      "from": "customer@example.com",
      "subject": "Order question",
      "date": "2026-03-22T10:30:00+00:00",
      "preview": "Hi, I have a question about my recent order...",
      "has_attachments": false
    }
```

```
  ],
  "count": 1
}
```

Reading a Specific Email

```
Router::get("/api/inbox/{id}", function ($request, $response) {
    $mailer = new Messenger();
    $emailId = $request->params["id"];

    $email = $mailer->getMessage($emailId);

    if ($email === null) {
        return $response->json(["error" => "Email not found"], 404);
    }

    return $response->json([
        "id" => $email["id"],
        "from" => $email["from"],
        "to" => $email["to"],
        "subject" => $email["subject"],
        "date" => $email["date"],
        "html_body" => $email["html_body"],
        "text_body" => $email["text_body"],
        "attachments" => array_map(fn($a) => [
            "name" => $a["name"],
            "size" => $a["size"],
            "type" => $a["type"]
        ], $email["attachments"] ?? [])
    ]);
});
```

Searching Emails

Search the inbox with IMAP search criteria:

```
$mailer = new Messenger();

// Search by subject
$results = $mailer->searchInbox("SUBJECT \"invoice\"");

// Search by sender
$results = $mailer->searchInbox("FROM \"alice@example.com\"");

// Search by date range
$results = $mailer->searchInbox("SINCE \"22-Mar-2026\" BEFORE \"29-Mar-2026\"");

// Unread messages from a specific sender
$results = $mailer->searchInbox("UNSEEN FROM \"support@example.com\"");
```

The search string follows IMAP search syntax. Common operators: `FROM`, `TO`, `SUBJECT`, `BODY`, `SINCE`, `BEFORE`, `SEEN`, `UNSEEN`.

Marking Messages

```
$mailer = new Messenger();

// Mark as read
$mailer->markRead("12345");

// Mark as unread
$mailer->markUnread("12345");
```

Pass the message ID returned by `getInbox()` or `searchInbox()`.

Deleting Messages

```
$mailer = new Messenger();
$mailer->deleteMessage("12345");
```

The message moves to the Trash folder (or is marked for deletion, depending on the IMAP server).

Listing Folders

```
$mailer = new Messenger();
$folders = $mailer->getFolders();
// ["INBOX", "Sent", "Drafts", "Trash", "Spam", "Archive"]
```

Reading from a Specific Folder

```
$mailer = new Messenger();
$sentEmails = $mailer->getInbox([
    "folder" => "Sent",
    "limit" => 10
]);
```

Pass the `folder` option to read from any IMAP folder. The default is `"INBOX"`.

---

10. Dev Mailbox Capture

Messenger captures outgoing mail when no SMTP host exists or when `TINA4_MAIL_CAPTURE=true`. `TINA4_DEBUG` only makes the dev dashboard available; it does not change delivery.

Navigate to `/__dev` and find the "Mail" section. You see:

- Every email "sent" during the current session
- The To, CC, and BCC addresses
- The subject and body (HTML and plain text)
- Attachments (viewable inline)
- The timestamp

Test email without configuring a real SMTP server. Enable `TINA4_DEBUG=true` to inspect captured messages in the dashboard.

Sending from a development environment

Configure an SMTP host and leave `TINA4_MAIL_CAPTURE` unset or false. The `TINA4_DEBUG` value does not matter to delivery. To test SMTP without reaching customers, set `TINA4_MAIL_REDIRECT_TO` to controlled inboxes.

---

11. Using Templates for Email Content

Hardcoding HTML in PHP strings is fragile and hard to read. Use Frond templates for email content.

Create `src/templates/emails/welcome.html`:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
</head>
<body style="font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto; background: #f5f5
    <div style="background: #1a1a2e; color: white; padding: 24px; text-align: center; border-rad
        <h1 style="margin: 0; font-size: 24px;">Welcome, {{ name }}!</h1>
    </div>
    <div style="background: white; padding: 24px; border-radius: 0 0 8px 8px;">
        <p>Hi {{ name }},</p>
        <p>Your account has been created successfully. Here are your details:</p>

        <table style="width: 100%; border-collapse: collapse; margin: 16px 0;">
            <tr>
                <td style="padding: 8px; border-bottom: 1px solid #eee; font-weight: bold;">Emai
                <td style="padding: 8px; border-bottom: 1px solid #eee;">{{ email }}</td>
            </tr>
            <tr>
                <td style="padding: 8px; border-bottom: 1px solid #eee; font-weight: bold;">Acco
                <td style="padding: 8px; border-bottom: 1px solid #eee;">#{{ user_id }}</td>
            </tr>
            <tr>
                <td style="padding: 8px; border-bottom: 1px solid #eee; font-weight: bold;">Sign
                <td style="padding: 8px; border-bottom: 1px solid #eee;">{{ signed_up_at }}</td>
            </tr>
        </table>

        <p>Get started by exploring:</p>
        <ul>
            <li><a href="{{ base_url }}/products" style="color: #1a1a2e;">Our product catalog</a
            <li><a href="{{ base_url }}/profile" style="color: #1a1a2e;">Your profile settings</
        </ul>

        {% if promo_code %}
            <div style="background: #d4edda; padding: 16px; border-radius: 4px; margin: 16px 0;"
                <strong>Special offer!</strong> Use code <code>{{ promo_code }}</code> for 10% o
            </div>
        {% endif %}

        <p>Cheers,<br>The {{ app_name }} Team</p>
    </div>
```

```
    <div style="text-align: center; padding: 12px; color: #888; font-size: 12px;">
        <p>You received this because you signed up at {{ app_name }}.</p>
        <p><a href="{{ base_url }}/unsubscribe?token={{ unsubscribe_token }}" style="color: #888
    </div>
</body>
</html>
```

Rendering and Sending

```
<?php
use Tina4\Router;
use Tina4\Messenger;
use Tina4\Frond;

Router::post("/api/register", function ($request, $response) {
    $body = $request->body;

    // Create user (database logic)
    $userId = 42;

    // Render the email template
    $emailData = [
        "name" => $body["name"],
        "email" => $body["email"],
        "user_id" => $userId,
        "signed_up_at" => date("F j, Y"),
        "base_url" => $_ENV["APP_URL"] ?? "http://localhost:7145",
        "app_name" => "My Store",
        "promo_code" => "WELCOME10",
        "unsubscribe_token" => bin2hex(random_bytes(16))
    ];

    $htmlBody = Frond::render("emails/welcome.html", $emailData);

    // Send the email
    $mailer = new Messenger();
    $result = $mailer->send(
        $body["email"],
        "Welcome to My Store, " . $body["name"] . "!",
        $htmlBody,
        [
            "text_body" => "Hi " . $body["name"] . ",\n\nWelcome to My Store! Your account (#" .
        ]
    );

    return $response->json([
        "message" => "Registration successful",
        "email_sent" => $result["success"],
        "user_id" => $userId
    ], 201);
});
```

```
curl -X POST http://localhost:7145/api/register \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "password": "securePass123"}'
```

```
{
  "message": "Registration successful",
  "email_sent": true,
  "user_id": 42
```

```
}
```

With `TINA4_MAIL_CAPTURE=true`, the email stays in the dev mailbox instead of reaching a real inbox. Enable `TINA4_DEBUG=true` to inspect the rendered HTML and verify the template values in the dashboard.

---

12. Sending Email via Queues

In production, do not send email inside a route handler. The SMTP handshake takes time. The user waits. Push email work to the queue system (Chapter 12):

```
<?php
use Tina4\Router;
use Tina4\Queue;
use Tina4\Messenger;
use Tina4\Frond;

// In the route handler, just queue the email
Router::post("/api/register", function ($request, $response) {
    $body = $request->body;
    $userId = 42; // Simulated

    Queue::produce("emails", [
        "template" => "emails/welcome.html",
        "to" => $body["email"],
        "subject" => "Welcome to My Store, " . $body["name"] . "!",
        "data" => [
            "name" => $body["name"],
            "email" => $body["email"],
            "user_id" => $userId,
            "signed_up_at" => date("F j, Y"),
            "base_url" => $_ENV["APP_URL"] ?? "http://localhost:7145",
            "app_name" => "My Store",
            "promo_code" => "WELCOME10"
        ]
    ]);

    return $response->json(["message" => "Registration successful", "user_id" => $userId], 201);
});

// The consumer sends the actual email
Queue::consume("emails", function ($job) {
    $payload = $job->payload;

    $htmlBody = Frond::render($payload["template"], $payload["data"]);

    $mailer = new Messenger();
    $result = $mailer->send(
        $payload["to"],
        $payload["subject"],
        $htmlBody
    );

    if (!$result["success"]) {
        error_log("Email failed: " . $result["error"]);
        return false; // Retry
```

```
    }

    error_log("Email sent to " . $payload["to"]);
    return true;
});
```

The route handler returns in under 50 milliseconds. The queue worker sends the email in the background. Automatic retries handle temporary SMTP failures.

---

13. Exercise: Build a Contact Form with Email Notification

Build a contact form that sends an email notification when submitted.

Requirements

- Create a `GET /contact` page that renders a contact form with fields: name, email, subject, and message
- Create a `POST /contact` endpoint that:
- Validates all fields are present
- Sends an email notification to the site admin (`admin@example.com`)
- The email should include all form fields, formatted with HTML
- Shows a flash message on success
- Redirects back to the contact page
- Create an email template at `src/templates/emails/contact-notification.html` that formats the contact submission

Test with:

```
# View the form
curl http://localhost:7145/contact

# Submit the form
curl -X POST http://localhost:7145/contact \
  -H "Content-Type: application/json" \
  -d '{"name": "Bob", "email": "bob@example.com", "subject": "Product inquiry", "message": "Do y

# Check the dev dashboard for the intercepted email
# Navigate to http://localhost:7145/__dev
```

---

14. Solution

Create `src/templates/emails/contact-notification.html`:

```
<!DOCTYPE html>
<html>
<head><meta charset="UTF-8"></head>
<body style="font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto;">
    <div style="background: #1a1a2e; color: white; padding: 16px; text-align: center;">
```

```
        <h2 style="margin: 0;">New Contact Form Submission</h2>
    </div>
    <div style="padding: 20px; background: white;">
        <table style="width: 100%; border-collapse: collapse;">
            <tr>
                <td style="padding: 10px; border-bottom: 1px solid #eee; font-weight: bold; widt
                <td style="padding: 10px; border-bottom: 1px solid #eee;">{{ name }} ({{ email }
            </tr>
            <tr>
                <td style="padding: 10px; border-bottom: 1px solid #eee; font-weight: bold;">Sub
                <td style="padding: 10px; border-bottom: 1px solid #eee;">{{ subject }}</td>
            </tr>
            <tr>
                <td style="padding: 10px; border-bottom: 1px solid #eee; font-weight: bold;">Dat
                <td style="padding: 10px; border-bottom: 1px solid #eee;">{{ submitted_at }}</td
            </tr>
        </table>

        <div style="margin-top: 16px; padding: 16px; background: #f8f9fa; border-radius: 4px;">
            <h3 style="margin-top: 0;">Message</h3>
            <p style="white-space: pre-wrap;">{{ message }}</p>
        </div>

        <p style="margin-top: 16px; color: #888; font-size: 12px;">
            Reply directly to this email to respond to {{ name }} at {{ email }}.
        </p>
    </div>
</body>
</html>
```

Create `src/templates/contact.html`:

```
{% extends "base.html" %}

{% block title %}Contact Us{% endblock %}

{% block content %}
    <h1>Contact Us</h1>

    {% if flash %}
        <div style="padding: 12px; border-radius: 4px; margin-bottom: 16px;
            {% if flash.type == 'success' %}background: #d4edda; color: #155724;{% endif %}
            {% if flash.type == 'error' %}background: #f8d7da; color: #721c24;{% endif %}">
            {{ flash.message }}
        </div>
    {% endif %}

    <form method="POST" action="/contact" style="max-width: 500px;">
        <div style="margin-bottom: 12px;">
            <label for="name" style="display: block; margin-bottom: 4px; font-weight: bold;">Nam
            <input type="text" name="name" id="name" required
                   style="width: 100%; padding: 8px; border: 1px solid #ddd; border-radius: 4px;
        </div>

        <div style="margin-bottom: 12px;">
            <label for="email" style="display: block; margin-bottom: 4px; font-weight: bold;">Em
            <input type="email" name="email" id="email" required
                   style="width: 100%; padding: 8px; border: 1px solid #ddd; border-radius: 4px;
        </div>
```

```
        <div style="margin-bottom: 12px;">
            <label for="subject" style="display: block; margin-bottom: 4px; font-weight: bold;">
            <input type="text" name="subject" id="subject" required
                   style="width: 100%; padding: 8px; border: 1px solid #ddd; border-radius: 4px;
        </div>

        <div style="margin-bottom: 12px;">
            <label for="message" style="display: block; margin-bottom: 4px; font-weight: bold;">
            <textarea name="message" id="message" rows="6" required
                      style="width: 100%; padding: 8px; border: 1px solid #ddd; border-radius: 4
        </div>

        <button type="submit"
                style="padding: 10px 20px; background: #1a1a2e; color: white; border: none; bord
            Send Message
        </button>
    </form>
{% endblock %}
```

Create `src/routes/contact.php`:

```
<?php
use Tina4\Router;
use Tina4\Messenger;
use Tina4\Frond;

Router::get("/contact", function ($request, $response) {
    $flash = $request->session["_flash"] ?? null;
    unset($request->session["_flash"]);

    return $response->render("contact.html", [
        "flash" => $flash
    ]);
});

Router::post("/contact", function ($request, $response) {
    $body = $request->body;

    // Validate
    $errors = [];
    if (empty($body["name"])) $errors[] = "Name is required";
    if (empty($body["email"])) $errors[] = "Email is required";
    if (empty($body["subject"])) $errors[] = "Subject is required";
    if (empty($body["message"])) $errors[] = "Message is required";

    if (!empty($errors)) {
        $request->session["_flash"] = [
            "type" => "error",
            "message" => "Please fill in all fields: " . implode(", ", $errors)
        ];
        return $response->redirect("/contact");
    }

    // Render the email template
    $htmlBody = Frond::render("emails/contact-notification.html", [
        "name" => $body["name"],
        "email" => $body["email"],
        "subject" => $body["subject"],
        "message" => $body["message"],
```

```
        "submitted_at" => date("F j, Y \a\\t g:i A")
    ]);

    // Send the email
    $mailer = new Messenger();
    $adminEmail = $_ENV["ADMIN_EMAIL"] ?? "admin@example.com";

    $result = $mailer->send(
        $adminEmail,
        "Contact Form: " . $body["subject"],
        $htmlBody,
        [
            "reply_to" => $body["email"],
            "text_body" => "Contact form submission from " . $body["name"] . " (" . $body["email
                           "Subject: " . $body["subject"] . "\n\n" .
                           "Message:\n" . $body["message"]
        ]
    );

    if ($result["success"]) {
        $request->session["_flash"] = [
            "type" => "success",
            "message" => "Thank you for your message! We will get back to you shortly."
        ];
    } else {
        $request->session["_flash"] = [
            "type" => "error",
            "message" => "Sorry, there was a problem sending your message. Please try again late
        ];
    }

    return $response->redirect("/contact");
});
```

Testing:

- Open `http://localhost:7145/contact` in your browser
- Fill in the form and submit
- You should see a green "Thank you" flash message
- Open `http://localhost:7145/__dev` to see the intercepted email
- The email should show the sender details, subject, message, and formatted HTML

API test:

```
curl -X POST http://localhost:7145/contact \
  -H "Content-Type: application/json" \
  -d '{"name": "Bob", "email": "bob@example.com", "subject": "Product inquiry", "message": "Do y
  -c cookies.txt -b cookies.txt
```

The response is a `302` redirect to `/contact`. Follow the redirect to see the flash message:

```
curl http://localhost:7145/contact -b cookies.txt
```

The HTML response includes the success flash message.

---

15. Gotchas

1. Gmail Blocks "Less Secure" Apps

Problem: Sending via Gmail fails with "Authentication failed" or "Username and Password not accepted."

Cause: Gmail blocks SMTP access from apps that do not use OAuth2 by default. Your regular password will not work with two-factor authentication enabled.

Fix: Generate an "App Password" in your Google Account settings (Security > 2-Step Verification > App Passwords). Use this 16-character password as `TINA4_MAIL_PASSWORD`. It is separate from your regular Google password.

2. Emails Go to Spam

Problem: Emails are delivered but land in the spam folder.

Cause: Your sending domain lacks proper DNS records (SPF, DKIM, DMARC), or you send from a free email provider (Gmail, Yahoo).

Fix: Use a dedicated sending domain with proper DNS records. Set up SPF, DKIM, and DMARC. Or use a transactional email service -- Mailgun, SendGrid, Amazon SES -- that manages email reputation for you.

3. HTML Email Looks Broken

Problem: The email renders in Gmail but breaks in Outlook or Apple Mail.

Cause: Email clients have wildly different HTML/CSS support. CSS flexbox, grid, and many modern properties do not work in email.

Fix: Use inline styles. Not external stylesheets, not `<style>` blocks. Use table-based layouts for complex designs. Test with an email preview tool. Keep it simple. Most transactional emails do not need elaborate designs.

4. Attachment File Not Found

Problem: `$mailer->send()` returns an error about a missing file.

Cause: The attachment path is relative or incorrect. The file does not exist at the specified location.

Fix: Use absolute paths for attachments. Verify the file exists before calling `send()`: `if (!file_exists($path)) { ... }`. If the file is generated at runtime (a PDF, for example), make sure the generation completes before sending.

5. Dev Mode Intercepts Emails Silently

Problem: You set up SMTP. No emails arrive. No errors either.

Cause: `TINA4_MAIL_CAPTURE=true` or a missing SMTP host sends mail to the dev mailbox. `TINA4_DEBUG` does not control delivery.

Fix: Check `/__dev` for captured mail. To use SMTP, configure `TINA4_MAIL_HOST` and remove `TINA4_MAIL_CAPTURE`. On staging, set `TINA4_MAIL_REDIRECT_TO` so tests cannot reach customer addresses.

6. Email Template Variables Not Substituted

Problem: The email body shows `{{ name }}` instead of the user's name.

Cause: You passed the raw template file content instead of rendering it through Frond. The template engine never ran.

Fix: Use `Frond::render("emails/template.html", $data)` to render the template with variables substituted. Do not use `file_get_contents()`. That gives you the raw template source.

7. Connection Timeout on Send

Problem: `$mailer->send()` hangs for 30 seconds and then fails with a timeout error.

Cause: The SMTP server is unreachable. The port is blocked by a firewall. The hostname is wrong.

Fix: Test SMTP connectivity: `telnet smtp.example.com 587`. Verify the hostname, port, and encryption settings. Check that your firewall allows outbound connections on the SMTP port. Corporate firewalls often block ports 587 and 465. Ask your network administrator.

Frontend with tina4css

1. The Problem with Frontend Toolchains

Your client wants a dashboard. You know the drill. Install Node.js. Run `npm install`. Watch 200MB of `node_modules` download. Configure webpack or Vite. Set up PostCSS. Add a CSS framework. Maybe Tailwind with its purge config. Pray nothing breaks when you upgrade a dependency six months later.

Tina4 takes a different path. The framework ships with **tina4css** -- a Bootstrap-compatible CSS framework -- and **frond.js** -- a lightweight JavaScript helper library. Both arrive when you scaffold a project. No npm. No webpack. No build step. Link the files. Start building.

By the end of this chapter, you have a complete admin dashboard with sidebar, navigation, cards, tables, modals, and dark mode support. Zero npm dependencies.

---

2. What Ships with Tina4

Run `tina4 init`. Two files appear in your project:

```
src/public/
■■■ css/
■   ■■■ tina4.css        # The CSS framework
■■■ js/
■   ■■■ frond.js         # AJAX helpers, form submission, token management
■■■ scss/
    ■■■ tina4.scss       # SCSS source (optional, for customization)
```

Include them in any template:

```
<link rel="stylesheet" href="/css/tina4.css">
<script src="/js/frond.js"></script>
```

That is everything. No CDN. No package manager. No version conflicts.

---

3. The Grid System

tina4css uses a 12-column responsive grid, compatible with Bootstrap's class names. Know Bootstrap? You know tina4css.

```
<div class="container">
    <div class="row">
        <div class="col-md-4">
            <p>One third</p>
        </div>
        <div class="col-md-4">
            <p>One third</p>
        </div>
        <div class="col-md-4">
            <p>One third</p>
        </div>
```

```
    </div>
</div>
```

Breakpoints:

Class Prefix	Screen Width	Typical Device
`col-`	All sizes	Phones and up
`col-sm-`	>= 576px	Large phones
`col-md-`	>= 768px	Tablets
`col-lg-`	>= 992px	Desktops
`col-xl-`	>= 1200px	Large desktops

Columns stack vertically on screens smaller than their breakpoint. A `col-md-6` element takes half the row on tablets and up, but full width on phones.

---

4. Components

Navbar

```
<nav class="navbar navbar-dark bg-dark">
    <div class="container">
        <a class="navbar-brand" href="/">My Dashboard</a>
        <ul class="navbar-nav">
            <li class="nav-item">
                <a class="nav-link active" href="/dashboard">Dashboard</a>
            </li>
            <li class="nav-item">
                <a class="nav-link" href="/users">Users</a>
            </li>
            <li class="nav-item">
                <a class="nav-link" href="/settings">Settings</a>
            </li>
        </ul>
    </div>
</nav>
```

Use `navbar-light bg-light` for a light theme, or `navbar-dark bg-primary` for a colored background.

Buttons

```
<button class="btn btn-primary">Save</button>
<button class="btn btn-secondary">Cancel</button>
<button class="btn btn-danger">Delete</button>
<button class="btn btn-success">Approve</button>
<button class="btn btn-warning">Warning</button>
<button class="btn btn-outline-primary">Outlined</button>
<button class="btn btn-sm btn-primary">Small</button>
<button class="btn btn-lg btn-primary">Large</button>
```

Cards

```
<div class="card">
    <div class="card-header">
        Monthly Revenue
    </div>
    <div class="card-body">
        <h5 class="card-title">$12,450</h5>
        <p class="card-text">Up 8% from last month</p>
    </div>
    <div class="card-footer text-muted">
        Updated 5 minutes ago
    </div>
</div>
```

Forms

```
<form>
    <div class="form-group">
        <label for="email">Email address</label>
        <input type="email" class="form-control" id="email"
               placeholder="you@example.com">
    </div>
    <div class="form-group">
        <label for="password">Password</label>
        <input type="password" class="form-control" id="password">
    </div>
    <div class="form-group form-check">
        <input type="checkbox" class="form-check-input" id="remember">
        <label class="form-check-label" for="remember">Remember me</label>
    </div>
    <button type="submit" class="btn btn-primary">Sign In</button>
</form>
```

Tables

```
<table class="table table-striped table-hover">
    <thead>
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Email</th>
            <th>Role</th>
            <th>Actions</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <td>1</td>
            <td>Alice Johnson</td>
            <td>alice@example.com</td>
            <td><span class="badge badge-primary">Admin</span></td>
            <td>
                <button class="btn btn-sm btn-outline-primary">Edit</button>
                <button class="btn btn-sm btn-outline-danger">Delete</button>
            </td>
        </tr>
        <tr>
            <td>2</td>
            <td>Bob Smith</td>
            <td>bob@example.com</td>
            <td><span class="badge badge-secondary">User</span></td>
            <td>
                <button class="btn btn-sm btn-outline-primary">Edit</button>
                <button class="btn btn-sm btn-outline-danger">Delete</button>
            </td>
        </tr>
    </tbody>
</table>
```

Table variants: `table-bordered`, `table-striped`, `table-hover`, `table-sm` (compact), `table-responsive` (wraps in a scrollable container on small screens). Mix and match.

Badges

```
<span class="badge badge-primary">Primary</span>
<span class="badge badge-success">Active</span>
<span class="badge badge-warning">Pending</span>
<span class="badge badge-danger">Overdue</span>
<span class="badge badge-info">Info</span>
<span class="badge badge-dark">Dark</span>
```

Alerts

```
<div class="alert alert-success">
    Product saved successfully.
</div>

<div class="alert alert-danger">
    Error: could not connect to the database.
</div>

<div class="alert alert-warning alert-dismissible">
    Your trial expires in 3 days.
    <button type="button" class="close" data-dismiss="alert">&times;</button>
</div>

<div class="alert alert-info">
    Tip: you can drag and drop items to reorder them.
</div>
```

Modals

```
<button class="btn btn-primary" data-toggle="modal" data-target="#confirmModal">
    Delete User
</button>

<div class="modal" id="confirmModal">
    <div class="modal-dialog">
        <div class="modal-content">
            <div class="modal-header">
                <h5 class="modal-title">Confirm Deletion</h5>
                <button type="button" class="close" data-dismiss="modal">&times;</button>
            </div>
            <div class="modal-body">
                <p>Are you sure you want to delete this user? This action cannot be undone.</p>
            </div>
            <div class="modal-footer">
                <button class="btn btn-secondary" data-dismiss="modal">Cancel</button>
                <button class="btn btn-danger">Delete</button>
            </div>
        </div>
    </div>
</div>
```

tina4css includes the JavaScript needed for modal toggling. No jQuery required.

Progress Bars

```
<div class="progress">
    <div class="progress-bar bg-success" style="width: 75%">75%</div>
</div>

<div class="progress">
    <div class="progress-bar bg-warning progress-bar-striped" style="width: 45%">
        45% - Uploading...
    </div>
</div>
```

---

5. SCSS Customization

The SCSS source lives in `src/public/scss/tina4.scss`. Customize colors, fonts, or spacing here. Compile to CSS when ready.

Variables

At the top of `tina4.scss`, you will find variables you can override:

```
// Colors
$primary:   #007bff;
$secondary: #6c757d;
$success:   #28a745;
$danger:    #dc3545;
$warning:   #ffc107;
$info:      #17a2b8;
$dark:      #343a40;
$light:     #f8f9fa;

// Typography
$font-family-base: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
$font-size-base: 1rem;
$line-height-base: 1.5;

// Spacing
$spacer: 1rem;

// Border radius
$border-radius: 0.25rem;

// Breakpoints
$breakpoint-sm: 576px;
$breakpoint-md: 768px;
$breakpoint-lg: 992px;
$breakpoint-xl: 1200px;
```

Compiling SCSS

If you edit the SCSS files, compile them with the Tina4 CLI:

```
tina4 scss
```

```
Compiling SCSS...
  src/public/scss/tina4.scss → src/public/css/tina4.css
Done.
```

This reads all `.scss` files from `src/public/scss/` and writes compiled CSS to `src/public/css/`. No Sass gem. No Node. No additional dependencies. The Tina4 CLI includes a built-in SCSS compiler.

You can also run it in watch mode during development:

```
tina4 scss --watch
```

```
Watching src/public/scss/ for changes...
  [14:30:05] Compiled tina4.scss → tina4.css (42ms)
```

Every time you save a `.scss` file, it recompiles automatically.

Custom Theme Example

Create `src/public/scss/custom.scss`:

```
// Override variables before importing tina4
$primary: #6f42c1;
$font-family-base: "Inter", sans-serif;
$border-radius: 0.5rem;

// Import the base framework
@import "tina4";

// Add your own styles
.sidebar {
    background: $dark;
    color: $light;
    min-height: 100vh;
    padding: 1rem;

    .nav-link {
        color: rgba(255, 255, 255, 0.7);
        padding: 0.5rem 1rem;
        border-radius: $border-radius;

        &:hover, &.active {
            color: #fff;
            background: rgba(255, 255, 255, 0.1);
        }
    }
}
```

Compile:

```
tina4 scss
```

Then reference your custom CSS in your template:

```
<link rel="stylesheet" href="/css/custom.css">
```

---

6. frond.js -- The JavaScript Helper

frond.js is a lightweight JavaScript library that ships with Tina4. AJAX helpers. Form submission. JWT token management. No jQuery. No Axios. No other library.

AJAX Requests

```
// GET request
frond.get("/api/products", function (data) {
    console.log(data.products);
});

// GET with error handling
frond.get("/api/products", function (data) {
    console.log(data);
}, function (error) {
    console.error("Failed:", error);
});

// POST request
frond.post("/api/products", {
    name: "Widget",
    price: 9.99
}, function (data) {
    console.log("Created:", data);
});

// PUT request
frond.put("/api/products/1", {
    name: "Updated Widget",
    price: 12.99
}, function (data) {
    console.log("Updated:", data);
});

// DELETE request
frond.delete("/api/products/1", function (data) {
    console.log("Deleted");
});
```

Form Submission

frond.js can intercept form submissions and send them via AJAX instead of a full page reload:

```
<form id="createProduct" data-frond-submit="/api/products" data-frond-method="POST">
    <div class="form-group">
        <label for="name">Product Name</label>
        <input type="text" class="form-control" name="name" id="name">
    </div>
    <div class="form-group">
        <label for="price">Price</label>
        <input type="number" class="form-control" name="price" id="price" step="0.01">
    </div>
    <button type="submit" class="btn btn-primary">Create Product</button>
</form>

<div id="result"></div>

<script>
    frond.onSubmit("createProduct", function (response) {
        document.getElementById("result").innerHTML =
            '<div class="alert alert-success">Product created: ' + response.name + '</div>';
    }, function (error) {
        document.getElementById("result").innerHTML =
            '<div class="alert alert-danger">Error: ' + error.message + '</div>';
```

```
    });
</script>
```

The `data-frond-submit` attribute tells frond.js the URL to POST to. The `data-frond-method` attribute specifies the HTTP method. frond.js serializes all form fields as JSON automatically.

Token Management

When your application uses JWT authentication (Chapter 8), frond.js manages tokens automatically:

```
// Store the token after login
frond.setToken("eyJhbGciOiJIUzI1NiIs...");

// All subsequent requests include the Authorization header automatically
frond.get("/api/profile", function (data) {
    // The request included: Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
    console.log(data);
});

// Clear the token on logout
frond.clearToken();
```

frond.js stores the token in `localStorage` and includes it as a `Bearer` token in the `Authorization` header on every request.

Loading Indicators

```
// Show a loading state while fetching
frond.get("/api/products", function (data) {
    renderProducts(data.products);
}, null, {
    loading: "#loadingSpinner"  // CSS selector for loading element
});
```

The element with id `loadingSpinner` will be shown while the request is in flight and hidden when it completes.

---

7. Building a Dashboard Layout

A real admin dashboard. Template layout with sidebar, top navbar, and content area.

The Base Layout

Create `src/templates/admin/layout.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}Admin Dashboard{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    <style>
        * { margin: 0; padding: 0; box-sizing: border-box; }
```

```
body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif; }

.app-wrapper { display: flex; min-height: 100vh; }

.sidebar {
    width: 250px;
    background: #1a1a2e;
    color: #e0e0e0;
    padding: 0;
    flex-shrink: 0;
}
.sidebar-header {
    padding: 20px;
    font-size: 1.2em;
    font-weight: bold;
    color: #fff;
    border-bottom: 1px solid rgba(255,255,255,0.1);
}
.sidebar-nav { list-style: none; padding: 10px 0; }
.sidebar-nav li a {
    display: block;
    padding: 10px 20px;
    color: rgba(255,255,255,0.7);
    text-decoration: none;
    transition: all 0.2s;
}
.sidebar-nav li a:hover,
.sidebar-nav li a.active {
    color: #fff;
    background: rgba(255,255,255,0.1);
}
.sidebar-nav li a .badge {
    float: right;
    margin-top: 2px;
}

.main-content { flex: 1; background: #f5f5f5; }

.topbar {
    background: #fff;
    padding: 12px 24px;
    border-bottom: 1px solid #e0e0e0;
    display: flex;
    justify-content: space-between;
    align-items: center;
}
.topbar-title { font-size: 1.1em; font-weight: 600; }
.topbar-actions { display: flex; gap: 12px; align-items: center; }

.content-area { padding: 24px; }

.stats-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
    gap: 20px;
    margin-bottom: 24px;
}

.stat-card {
```

```
            background: #fff;
            border-radius: 8px;
            padding: 20px;
            box-shadow: 0 1px 3px rgba(0,0,0,0.1);
        }
        .stat-card .stat-value { font-size: 2em; font-weight: bold; }
        .stat-card .stat-label { color: #888; margin-top: 4px; }
        .stat-card .stat-change { font-size: 0.85em; margin-top: 8px; }
        .stat-card .stat-change.up { color: #28a745; }
        .stat-card .stat-change.down { color: #dc3545; }

        @media (max-width: 768px) {
            .sidebar { display: none; }
            .stats-grid { grid-template-columns: 1fr; }
        }
    </style>
    {% block extra_css %}{% endblock %}
</head>
<body>
    <div class="app-wrapper">
        <aside class="sidebar">
            <div class="sidebar-header">Admin Panel</div>
            <ul class="sidebar-nav">
                <li><a href="/admin" class="{% if active_page == 'dashboard' %}active{% endif %}
                <li><a href="/admin/users" class="{% if active_page == 'users' %}active{% endif
                <li><a href="/admin/products" class="{% if active_page == 'products' %}active{%
                <li>
                    <a href="/admin/orders" class="{% if active_page == 'orders' %}active{% endi
                        Orders
                        {% if pending_orders > 0 %}
                            <span class="badge badge-warning">{{ pending_orders }}</span>
                        {% endif %}
                    </a>
                </li>
                <li><a href="/admin/settings" class="{% if active_page == 'settings' %}active{%
            </ul>
        </aside>

        <div class="main-content">
            <div class="topbar">
                <span class="topbar-title">{% block page_title %}Dashboard{% endblock %}</span>
                <div class="topbar-actions">
                    <span>Welcome, {{ user_name | default("Admin") }}</span>
                    <button class="btn btn-sm btn-outline-danger">Logout</button>
                </div>
            </div>

            <div class="content-area">
                {% block content %}{% endblock %}
            </div>
        </div>
    </div>

    <script src="/js/frond.js"></script>
    {% block extra_js %}{% endblock %}
</body>
</html>
```

The Dashboard Page

Create `src/templates/admin/dashboard.html`:

```
{% extends "admin/layout.html" %}

{% block title %}Dashboard - Admin{% endblock %}
{% block page_title %}Dashboard{% endblock %}

{% block content %}
    <div class="stats-grid">
        {% for stat in stats %}
            <div class="stat-card">
                <div class="stat-value">{{ stat.value }}</div>
                <div class="stat-label">{{ stat.label }}</div>
                <div class="stat-change {{ stat.direction }}">
                    {{ stat.direction == "up" ? "+" : "" }}{{ stat.change }}% from last month
                </div>
            </div>
        {% endfor %}
    </div>

    <div class="row">
        <div class="col-md-8">
            <div class="card">
                <div class="card-header">Recent Orders</div>
                <div class="card-body" style="padding: 0;">
                    <table class="table table-hover" style="margin: 0;">
                        <thead>
                            <tr>
                                <th>Order ID</th>
                                <th>Customer</th>
                                <th>Total</th>
                                <th>Status</th>
                            </tr>
                        </thead>
                        <tbody>
                            {% for order in recent_orders %}
                                <tr>
                                    <td>#{{ order.id }}</td>
                                    <td>{{ order.customer }}</td>
                                    <td>${{ order.total | number_format(2) }}</td>
                                    <td>
                                        {% if order.status == "completed" %}
                                            <span class="badge badge-success">Completed</span>
                                        {% elseif order.status == "pending" %}
                                            <span class="badge badge-warning">Pending</span>
                                        {% elseif order.status == "cancelled" %}
                                            <span class="badge badge-danger">Cancelled</span>
                                        {% endif %}
                                    </td>
                                </tr>
                            {% endfor %}
                        </tbody>
                    </table>
                </div>
            </div>
        </div>

        <div class="col-md-4">
```

```
            <div class="card">
                <div class="card-header">Quick Actions</div>
                <div class="card-body">
                    <button class="btn btn-primary btn-block" style="margin-bottom: 8px;">
                        Add New Product
                    </button>
                    <button class="btn btn-outline-primary btn-block" style="margin-bottom: 8px;
                        View Reports
                    </button>
                    <button class="btn btn-outline-secondary btn-block">
                        Export Data
                    </button>
                </div>
            </div>

            <div class="card" style="margin-top: 20px;">
                <div class="card-header">System Health</div>
                <div class="card-body">
                    <p><strong>CPU:</strong></p>
                    <div class="progress" style="margin-bottom: 12px;">
                        <div class="progress-bar bg-success" style="width: {{ cpu_usage }}%">
                            {{ cpu_usage }}%
                        </div>
                    </div>
                    <p><strong>Memory:</strong></p>
                    <div class="progress" style="margin-bottom: 12px;">
                        <div class="progress-bar bg-info" style="width: {{ memory_usage }}%">
                            {{ memory_usage }}%
                        </div>
                    </div>
                    <p><strong>Disk:</strong></p>
                    <div class="progress">
                        <div class="progress-bar bg-warning" style="width: {{ disk_usage }}%">
                            {{ disk_usage }}%
                        </div>
                    </div>
                </div>
            </div>
        </div>
    </div>
{% endblock %}
```

The Dashboard Route

Create `src/routes/admin.php`:

```
<?php
use Tina4\Router;

Router::get("/admin", function ($request, $response) {
    $stats = [
        ["label" => "Total Users", "value" => "1,247", "change" => 12, "direction" => "up"],
        ["label" => "Revenue", "value" => "$34,500", "change" => 8, "direction" => "up"],
        ["label" => "Orders", "value" => "456", "change" => 3, "direction" => "down"],
        ["label" => "Conversion Rate", "value" => "3.2%", "change" => 0.5, "direction" => "up"]
    ];

    $recentOrders = [
        ["id" => 1042, "customer" => "Alice Johnson", "total" => 149.99, "status" => "completed"
```

```
        ["id" => 1041, "customer" => "Bob Smith", "total" => 89.50, "status" => "pending"],
        ["id" => 1040, "customer" => "Carol Davis", "total" => 234.00, "status" => "completed"],
        ["id" => 1039, "customer" => "David Lee", "total" => 45.99, "status" => "cancelled"],
        ["id" => 1038, "customer" => "Eva Martinez", "total" => 178.25, "status" => "pending"]
    ];

    return $response->render("admin/dashboard.html", [
        "active_page" => "dashboard",
        "user_name" => "Admin",
        "pending_orders" => 2,
        "stats" => $stats,
        "recent_orders" => $recentOrders,
        "cpu_usage" => 42,
        "memory_usage" => 68,
        "disk_usage" => 55
    ]);
});
```

Start the server. Visit `http://localhost:7145/admin`. You see:

- Dark sidebar on the left with navigation links
- Top bar with a welcome message and logout button
- Four stat cards showing users, revenue, orders, and conversion rate
- Recent orders table with color-coded status badges
- Quick action buttons in a card on the right
- System health progress bars

Zero npm dependencies.

---

8. Dark Mode Support

tina4css includes built-in dark mode. One attribute on the `<html>` element:

```
<html lang="en" data-theme="dark">
```

Light backgrounds become dark. Dark text becomes light. All components adapt.

Toggle with JavaScript

Add a dark mode toggle button to your layout:

```
<button class="btn btn-sm btn-outline-secondary" onclick="toggleDarkMode()">
    Toggle Dark Mode
</button>

<script>
    function toggleDarkMode() {
        const html = document.documentElement;
        const current = html.getAttribute("data-theme");
        const next = current === "dark" ? "light" : "dark";
        html.setAttribute("data-theme", next);
        localStorage.setItem("theme", next);
    }
```

```
    // Load saved preference
    const saved = localStorage.getItem("theme");
    if (saved) {
        document.documentElement.setAttribute("data-theme", saved);
    }
</script>
```

Respecting System Preference

If the user has not set a preference, respect their operating system setting:

```
const saved = localStorage.getItem("theme");
if (saved) {
    document.documentElement.setAttribute("data-theme", saved);
} else if (window.matchMedia("(prefers-color-scheme: dark)").matches) {
    document.documentElement.setAttribute("data-theme", "dark");
}
```

Custom Dark Mode Colors

In your SCSS, you can customize the dark mode palette:

```
[data-theme="dark"] {
    --bg-primary: #1a1a2e;
    --bg-secondary: #16213e;
    --text-primary: #e0e0e0;
    --text-secondary: #a0a0a0;
    --border-color: #2d2d4a;
    --card-bg: #16213e;
}
```

---

9. Responsive Design

tina4css is mobile-first. Components stack on small screens. They expand on larger screens.

Responsive Utilities

Show and hide elements at different breakpoints:

```
<!-- Only visible on medium screens and up -->
<div class="d-none d-md-block">
    Desktop sidebar content
</div>

<!-- Only visible on small screens -->
<div class="d-block d-md-none">
    Mobile navigation
</div>
```

Responsive Tables

Wrap tables in a `.table-responsive` container to make them scroll horizontally on small screens:

```
<div class="table-responsive">
    <table class="table">
        <!-- Wide table content -->
    </table>
```

```
</div>
```

Spacing Utilities

Use margin and padding utilities that respond to breakpoints:

```
<div class="p-2 p-md-4">Padding: 0.5rem on mobile, 1.5rem on desktop</div>
<div class="mt-2 mt-lg-5">Margin-top: 0.5rem on mobile, 3rem on large screens</div>
```

Spacing scale:

Class	Size
`*-0`	0
`*-1`	0.25rem
`*-2`	0.5rem
`*-3`	1rem
`*-4`	1.5rem
`*-5`	3rem

Directions: `m` (margin), `p` (padding), `t` (top), `b` (bottom), `l` (left), `r` (right), `x` (horizontal), `y` (vertical).

---

10. Building a Users Page with AJAX

A users management page. Data loads via AJAX using frond.js.

Create `src/templates/admin/users.html`:

```
{% extends "admin/layout.html" %}

{% block title %}Users - Admin{% endblock %}
{% block page_title %}User Management{% endblock %}

{% block content %}
    <div class="card">
        <div class="card-header" style="display: flex; justify-content: space-between; align-ite
            <span>All Users</span>
            <button class="btn btn-sm btn-primary" data-toggle="modal" data-target="#addUserModa
                Add User
            </button>
        </div>
        <div class="card-body" style="padding: 0;">
            <div id="loadingSpinner" class="text-center p-4" style="display: none;">
                Loading...
            </div>
            <table class="table table-hover" id="usersTable" style="margin: 0;">
                <thead>
                    <tr>
                        <th>ID</th>
                        <th>Name</th>
                        <th>Email</th>
```

```
                        <th>Created</th>
                        <th>Actions</th>
                    </tr>
                </thead>
                <tbody id="usersBody">
                    <!-- Populated by JavaScript -->
                </tbody>
            </table>
        </div>
    </div>

    <!-- Add User Modal -->
    <div class="modal" id="addUserModal">
        <div class="modal-dialog">
            <div class="modal-content">
                <div class="modal-header">
                    <h5 class="modal-title">Add New User</h5>
                    <button type="button" class="close" data-dismiss="modal">&times;</button>
                </div>
                <div class="modal-body">
                    <form id="addUserForm">
                        <div class="form-group">
                            <label for="userName">Name</label>
                            <input type="text" class="form-control" name="name" id="userName" re
                        </div>
                        <div class="form-group">
                            <label for="userEmail">Email</label>
                            <input type="email" class="form-control" name="email" id="userEmail"
                        </div>
                    </form>
                </div>
                <div class="modal-footer">
                    <button class="btn btn-secondary" data-dismiss="modal">Cancel</button>
                    <button class="btn btn-primary" onclick="createUser()">Create User</button>
                </div>
            </div>
        </div>
    </div>

    <div id="alertArea" style="margin-top: 16px;"></div>
{% endblock %}

{% block extra_js %}
<script>
    function loadUsers() {
        frond.get("/api/users", function (data) {
            var tbody = document.getElementById("usersBody");
            tbody.innerHTML = "";

            if (data.data && data.data.length > 0) {
                data.data.forEach(function (user) {
                    var row = '<tr>'
                        + '<td>' + user.id + '</td>'
                        + '<td>' + user.name + '</td>'
                        + '<td>' + user.email + '</td>'
                        + '<td>' + user.created_at + '</td>'
                        + '<td>'
                        + '<button class="btn btn-sm btn-outline-primary" onclick="editUser(' +
                        + '<button class="btn btn-sm btn-outline-danger" onclick="deleteUser(' +
```

```
                        + '</td>'
                        + '</tr>';
                    tbody.innerHTML += row;
                });
            } else {
                tbody.innerHTML = '<tr><td colspan="5" class="text-center p-4">No users found</t
            }
        }, null, { loading: "#loadingSpinner" });
    }

    function createUser() {
        var name = document.getElementById("userName").value;
        var email = document.getElementById("userEmail").value;

        frond.post("/api/users", { name: name, email: email }, function (data) {
            document.getElementById("alertArea").innerHTML =
                '<div class="alert alert-success">User "' + data.name + '" created.</div>';
            loadUsers();
        }, function (error) {
            document.getElementById("alertArea").innerHTML =
                '<div class="alert alert-danger">Error creating user.</div>';
        });
    }

    function deleteUser(id) {
        if (confirm("Are you sure you want to delete this user?")) {
            frond.delete("/api/users/" + id, function () {
                loadUsers();
            });
        }
    }

    // Load users on page load
    loadUsers();
</script>
{% endblock %}
```

Create the route for the users page:

```
Router::get("/admin/users", function ($request, $response) {
    return $response->render("admin/users.html", [
        "active_page" => "users",
        "user_name" => "Admin",
        "pending_orders" => 2
    ]);
});
```

This page loads users via an AJAX call to `/api/users` (provided by auto-CRUD on the User model from Chapter 6). The table populates dynamically, and users can be added through a modal form and deleted with confirmation -- all without full page reloads.

---

11. Exercise: Build a Complete Admin Dashboard

Build an admin dashboard with the following features:

Requirements

• **Layout** -- Sidebar with navigation links for Dashboard, Products, and Orders. Top bar with the current user name and a dark mode toggle button.

• **Dashboard Page** (`GET /admin`) -- Four stat cards (Total Products, Total Orders, Revenue, Low Stock Items). A table of the 5 most recent orders. A progress bar showing inventory health (percentage of products in stock).

• **Products Page** (`GET /admin/products`) -- A table listing all products with columns: ID, Name, Category, Price, Stock Status. An "Add Product" button that opens a modal with a form. The form submits via frond.js AJAX to `POST /api/products`. After submission, the table reloads without a page refresh.

• **Dark Mode** -- The toggle button switches between light and dark themes. The preference persists in localStorage.

Seed Data

Create a route that populates sample data:

```
Router::get("/admin/seed", function ($request, $response) {
    $products = [
        ["name" => "Wireless Keyboard", "category" => "Electronics", "price" => 79.99, "in_stock
        ["name" => "USB-C Hub", "category" => "Electronics", "price" => 49.99, "in_stock" => tru
        ["name" => "Standing Desk", "category" => "Furniture", "price" => 549.99, "in_stock" =>
        ["name" => "Monitor Light", "category" => "Electronics", "price" => 39.99, "in_stock" =>
        ["name" => "Ergonomic Chair", "category" => "Furniture", "price" => 399.99, "in_stock" =
    ];

    foreach ($products as $data) {
        $product = new Product();
        $product->name = $data["name"];
        $product->category = $data["category"];
        $product->price = $data["price"];
        $product->inStock = $data["in_stock"];
        $product->save();
    }

    return $response->json(["message" => "Seeded " . count($products) . " products"]);
});
```

Expected Result

Visit `http://localhost:7145/admin` and see:

• A sidebar with navigation (Dashboard highlighted)

• Four stat cards showing product and order counts

• A recent orders table with status badges

• A dark mode toggle that switches the entire UI

Visit `http://localhost:7145/admin/products` and see:

• A table of all products loaded via AJAX

• An "Add Product" button that opens a modal

- Form submission creates the product and refreshes the table

---

12. Solution

The solution follows the same patterns shown in sections 7 and 10. Create the layout template extending the base layout from section 7. Create the dashboard page template with stat cards using `{% for stat in stats %}`. Create the products page template using the AJAX pattern from section 10, but for products instead of users.

For the products page, use auto-CRUD on the Product model (`$autoCrud = true`) so the API endpoints are available at `/api/products`. Load the table with `frond.get("/api/products", ...)` and handle form submission with `frond.post("/api/products", ...)`.

For the dark mode toggle, add the JavaScript from section 8 to the base layout template inside the `{% block extra_js %}` block.

The key route handlers are:

```
<?php
use Tina4\Router;

Router::get("/admin", function ($request, $response) {
    $product = new Product();
    $allProducts = $product->select("*");
    $inStockCount = count(array_filter($allProducts, fn($p) => $p->inStock));
    $totalProducts = count($allProducts);
    $stockHealth = $totalProducts > 0 ? round(($inStockCount / $totalProducts) * 100) : 0;

    $stats = [
        ["label" => "Total Products", "value" => $totalProducts, "change" => 5, "direction" => "
        ["label" => "Total Orders", "value" => 156, "change" => 12, "direction" => "up"],
        ["label" => "Revenue", "value" => "$8,450", "change" => 3, "direction" => "up"],
        ["label" => "Low Stock", "value" => $totalProducts - $inStockCount, "change" => 2, "dire
    ];

    return $response->render("admin/dashboard.html", [
        "active_page" => "dashboard",
        "stats" => $stats,
        "stock_health" => $stockHealth,
        "recent_orders" => [],
        "pending_orders" => 0
    ]);
});

Router::get("/admin/products", function ($request, $response) {
    return $response->render("admin/products.html", [
        "active_page" => "products",
        "pending_orders" => 0
    ]);
});
```

---

13. Gotchas

1. CSS File Not Loading

Problem: The browser shows unstyled HTML. The CSS file returns a 404.

Cause: The CSS file must be inside `src/public/css/`. If you put it in `src/css/` or `css/` at the root, Tina4 will not serve it.

Fix: Make sure the file is at `src/public/css/tina4.css`. Static files are served from `src/public/`.

2. SCSS Changes Not Reflected

Problem: You edited `tina4.scss` but the browser still shows the old styles.

Cause: SCSS must be compiled to CSS. Editing the `.scss` file does not automatically update the `.css` file unless you are running `tina4 scss --watch`.

Fix: Run `tina4 scss` to compile, or use `tina4 scss --watch` during development.

3. Modal Does Not Open

Problem: Clicking a button with `data-toggle="modal"` does nothing.

Cause: frond.js is not loaded, or the `data-target` value does not match the modal's `id`.

Fix: Check that `<script src="/js/frond.js"></script>` is included before `</body>`. Verify that `data-target="#myModal"` matches `<div class="modal" id="myModal">` (the `#` prefix is required in `data-target`).

4. frond.js AJAX Returns HTML Instead of JSON

Problem: `frond.get("/api/products", ...)` receives an HTML page instead of JSON data.

Cause: The route handler returns `$response->render(...)` or `$response->html(...)` instead of `$response->json(...)`. Or the URL is wrong and hitting a catch-all route that returns HTML.

Fix: Verify the API route returns JSON. Check the URL in the frond.js call matches the route path exactly.

5. Dark Mode Flickers on Load

Problem: The page loads in light mode for a split second before switching to dark mode.

Cause: The theme is set by JavaScript after the page renders. The initial HTML does not include `data-theme="dark"`.

Fix: Move the theme detection script to the `<head>` section so it runs before the body renders:

```
<head>
    <script>
        (function() {
            var t = localStorage.getItem("theme");
```

```
            if (t) document.documentElement.setAttribute("data-theme", t);
            else if (window.matchMedia("(prefers-color-scheme: dark)").matches)
                document.documentElement.setAttribute("data-theme", "dark");
        })();
    </script>
</head>
```

6. Form Data Not Reaching the Server

Problem: `frond.post()` sends the request but `$request->body` is empty on the server.

Cause: frond.js sends JSON by default. If you are building the data object incorrectly or passing a FormData object, the server may not parse it correctly.

Fix: Pass a plain JavaScript object to `frond.post()`. Do not use `new FormData()` -- frond.js handles serialization internally. Make sure your object keys match what the server expects:

```
frond.post("/api/products", {
    name: document.getElementById("name").value,
    price: parseFloat(document.getElementById("price").value)
}, callback);
```

7. Responsive Grid Not Working

Problem: Columns do not stack on mobile. They overflow horizontally instead.

Cause: Missing the viewport meta tag. Without it, mobile browsers render the page at desktop width.

Fix: Add this to your `<head>`:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

This tag is included in all the examples above but is easy to forget when creating templates from scratch.

---

14. HtmlElement - Programmatic HTML Builder

Build HTML in PHP without string concatenation:

```
$el = new HtmlElement("div", ["class" => "card"], ["Hello"]);
echo $el; // <div class="card">Hello</div>

// Nesting
$card = new HtmlElement("div", ["class" => "card"], [
    new HtmlElement("h2", [], ["Title"]),
    new HtmlElement("p", [], ["Content"]),
]);

// Helper functions
extract(HtmlElement::helpers());
echo $_div(["class" => "card"], $_p("Hello"), $_a(["href" => "/"], "Home"));
```

Void tags (`<br>`, `<img>`, `<input>`) render without closing tags. Boolean attributes render as bare names.

Testing

1. Why Tests Matter More Than You Think

Friday afternoon. Your client reports a critical bug in production. You fix it. One line of code. But did that fix break anything else? You have 47 routes, 12 ORM models, and 3 middleware functions. Manually clicking through every page takes an hour. Running the test suite takes 2 seconds.

```
tina4 test

Running tests...

  ProductTest
    [PASS] test_create_product
    [PASS] test_load_product
    [PASS] test_update_product
    [PASS] test_delete_product
    [PASS] test_list_products_with_filter

  AuthTest
    [PASS] test_login_with_valid_credentials
    [PASS] test_login_with_invalid_password
    [PASS] test_protected_route_without_token
    [PASS] test_protected_route_with_valid_token

  9 tests, 9 passed, 0 failed (0.34s)
```

Everything still works. Deploy with confidence. Enjoy your weekend.

Tina4 includes an inline testing framework. No external packages. No PHPUnit configuration. Write a test. Run it. Done.

---

2. Your First Test

Tests live in the `tests/` directory. Every `.php` file there is auto-discovered when you run `tina4 test`.

Create `tests/BasicTest.php`:

```
<?php
use Tina4\Test;

class BasicTest extends Test
{
    public function testAddition()
    {
        $this->assertEqual(2 + 2, 4, "Basic addition should work");
    }

    public function testStringContains()
    {
        $greeting = "Hello, World!";
        $this->assertTrue(str_contains($greeting, "World"), "Greeting should contain 'World'");
```

```
    }

    public function testArrayLength()
    {
        $items = [1, 2, 3, 4, 5];
        $this->assertEqual(count($items), 5, "Array should have 5 items");
    }
}
```

Run it:

```
tina4 test

Running tests...

  BasicTest
    [PASS] test_addition
    [PASS] test_string_contains
    [PASS] test_array_length

  3 tests, 3 passed, 0 failed (0.02s)
```

How It Works

- Your test class extends `Tina4\Test`.
- Every method that starts with `test` is a test case. The method name is converted to a readable label: `testAddition` becomes `test_addition`.
- Inside each test, you call assertion methods to verify behavior.
- If all assertions pass, the test passes. If any assertion fails, the test fails and you see the failure message.

---

3. Assertion Methods

Tina4's Test class provides these assertion methods:

assertEqual($actual, $expected, $message)

Checks that two values are equal.

```
$this->assertEqual(4, 4, "Should be equal");              // PASS
$this->assertEqual("hello", "hello", "Strings match");    // PASS
$this->assertEqual(4, 5, "Not equal");                     // FAIL
```

assertTrue($value, $message)

Checks that a value is truthy.

```
$this->assertTrue(true, "Should be true");                // PASS
$this->assertTrue(1, "1 is truthy");                      // PASS
$this->assertTrue("yes", "Non-empty string is truthy");   // PASS
$this->assertTrue(false, "This fails");                   // FAIL
$this->assertTrue(0, "Zero is falsy");                    // FAIL
```

assertFalse($value, $message)

Checks that a value is falsy.

```
$this->assertFalse(false, "Should be false");            // PASS
$this->assertFalse(0, "Zero is falsy");                  // PASS
$this->assertFalse("", "Empty string is falsy");         // PASS
$this->assertFalse(true, "This fails");                  // FAIL
```

assertRaises($callable, $exceptionClass, $message)

Checks that a function throws a specific exception.

```
$this->assertRaises(function () {
    throw new \InvalidArgumentException("Bad input");
}, \InvalidArgumentException::class, "Should throw InvalidArgumentException");

$this->assertRaises(function () {
    $result = 10 / 0;
}, \DivisionByZeroError::class, "Should throw on division by zero");
```

assertNotEqual($actual, $expected, $message)

Checks that two values are not equal.

```
$this->assertNotEqual("hello", "world", "Strings differ");  // PASS
$this->assertNotEqual(4, 4, "Same values");                  // FAIL
```

assertNull($value, $message)

Checks that a value is null.

```
$this->assertNull(null, "Should be null");     // PASS
$this->assertNull("hello", "Not null");         // FAIL
```

assertNotNull($value, $message)

Checks that a value is not null.

```
$this->assertNotNull("hello", "Has value");    // PASS
$this->assertNotNull(null, "Is null");          // FAIL
```

---

4. Testing ORM Models

Test a Product model. Create records. Load them. Update them. Delete them.

Create `tests/ProductTest.php`:

```
<?php
use Tina4\Test;

class ProductTest extends Test
{
    private ?int $testProductId = null;

    public function testCreateProduct()
    {
        $product = new Product();
```

```
        $product->name = "Test Widget";
        $product->category = "Testing";
        $product->price = 19.99;
        $product->inStock = true;
        $product->save();

        $this->assertNotNull($product->id, "Product should have an ID after save");
        $this->assertTrue($product->id > 0, "Product ID should be positive");

        $this->testProductId = $product->id;
    }

    public function testLoadProduct()
    {
        // Create a product to load
        $product = new Product();
        $product->name = "Load Test Widget";
        $product->category = "Testing";
        $product->price = 29.99;
        $product->save();

        // Load it back
        $loaded = new Product();
        $loaded->load($product->id);

        $this->assertEqual($loaded->name, "Load Test Widget", "Name should match");
        $this->assertEqual($loaded->category, "Testing", "Category should match");
        $this->assertEqual($loaded->price, 29.99, "Price should match");
        $this->assertTrue($loaded->inStock, "Should be in stock by default");
    }

    public function testUpdateProduct()
    {
        $product = new Product();
        $product->name = "Update Test Widget";
        $product->price = 10.00;
        $product->save();

        $id = $product->id;

        // Update it
        $product->name = "Updated Widget";
        $product->price = 15.00;
        $product->save();

        // Reload and verify
        $reloaded = new Product();
        $reloaded->load($id);

        $this->assertEqual($reloaded->name, "Updated Widget", "Name should be updated");
        $this->assertEqual($reloaded->price, 15.00, "Price should be updated");
    }

    public function testDeleteProduct()
    {
        $product = new Product();
        $product->name = "Delete Me";
        $product->price = 5.00;
        $product->save();
```

```
        $id = $product->id;
        $product->delete();

        // Try to load the deleted product
        $gone = new Product();
        $gone->load($id);

        $this->assertTrue(empty($gone->id), "Deleted product should not be loadable");
    }

    public function testSelectWithFilter()
    {
        // Create products in different categories
        $p1 = new Product();
        $p1->name = "Filter Test A";
        $p1->category = "FilterCat";
        $p1->price = 10.00;
        $p1->save();

        $p2 = new Product();
        $p2->name = "Filter Test B";
        $p2->category = "FilterCat";
        $p2->price = 20.00;
        $p2->save();

        $p3 = new Product();
        $p3->name = "Other Product";
        $p3->category = "Other";
        $p3->price = 30.00;
        $p3->save();

        // Query by category
        $product = new Product();
        $results = $product->select("*", "category = :cat", ["cat" => "FilterCat"]);

        $this->assertTrue(count($results) >= 2, "Should find at least 2 FilterCat products");

        $names = array_map(fn($p) => $p->name, $results);
        $this->assertTrue(in_array("Filter Test A", $names), "Should include Filter Test A");
        $this->assertTrue(in_array("Filter Test B", $names), "Should include Filter Test B");
    }
}
```

Run it:

```
tina4 test

Running tests...

  ProductTest
    [PASS] test_create_product
    [PASS] test_load_product
    [PASS] test_update_product
    [PASS] test_delete_product
    [PASS] test_select_with_filter

  5 tests, 5 passed, 0 failed (0.18s)
```

Test Database

By default, `tina4 test` uses a separate test database so your development data is not affected. The test database is created at `data/test.db` (SQLite) and is reset before each test run. If you want to use a different database for tests, set it in `.env`:

```
TINA4_DATABASE_URL=sqlite:///data/test.db
```

---

5. Testing Routes

Tina4 provides a test client for HTTP requests to your routes. No server needed.

Create `tests/RouteTest.php`:

```
<?php
use Tina4\Test;

class RouteTest extends Test
{
    public function testHealthEndpoint()
    {
        $response = $this->get("/health");

        $this->assertEqual($response->status, 200, "Health check should return 200");

        $body = json_decode($response->body, true);
        $this->assertEqual($body["status"], "ok", "Status should be 'ok'");
        $this->assertNotNull($body["version"], "Should include version");
    }

    public function testGetProducts()
    {
        $response = $this->get("/api/products");

        $this->assertEqual($response->status, 200, "Should return 200");

        $body = json_decode($response->body, true);
        $this->assertTrue(isset($body["data"]) || isset($body["products"]), "Should contain prod
    }

    public function testCreateProduct()
    {
        $response = $this->post("/api/products", [
            "name" => "Route Test Product",
            "category" => "Testing",
            "price" => 42.00
        ]);

        $this->assertEqual($response->status, 201, "Should return 201 Created");

        $body = json_decode($response->body, true);
        $this->assertEqual($body["name"], "Route Test Product", "Name should match");
        $this->assertEqual($body["price"], 42.00, "Price should match");
    }

    public function testGetProductNotFound()
```

```
    {
        $response = $this->get("/api/products/99999");

        $this->assertEqual($response->status, 404, "Should return 404 for missing product");
    }

    public function testCreateProductValidation()
    {
        $response = $this->post("/api/products", []);

        $this->assertEqual($response->status, 400, "Should return 400 for empty body");
    }

    public function testDeleteProduct()
    {
        // Create a product first
        $createResponse = $this->post("/api/products", [
            "name" => "To Be Deleted",
            "price" => 1.00
        ]);
        $body = json_decode($createResponse->body, true);
        $id = $body["id"];

        // Delete it
        $deleteResponse = $this->delete("/api/products/" . $id);
        $this->assertEqual($deleteResponse->statusCode, 204, "Should return 204 No Content");

        // Verify it is gone
        $getResponse = $this->get("/api/products/" . $id);
        $this->assertEqual($getResponse->statusCode, 404, "Should return 404 after deletion");
    }
}
```

Test Client Methods

The test client provides methods for all HTTP verbs:

```
// GET request
$response = $this->get("/api/products");

// GET with query parameters
$response = $this->get("/api/products?category=Electronics&page=2");

// POST with JSON body
$response = $this->post("/api/products", ["name" => "Widget", "price" => 9.99]);

// PUT with JSON body
$response = $this->put("/api/products/1", ["name" => "Updated Widget"]);

// PATCH with JSON body
$response = $this->patch("/api/products/1", ["price" => 12.99]);

// DELETE
$response = $this->delete("/api/products/1");

// Request with custom headers
$response = $this->get("/api/profile", [
    "Authorization" => "Bearer eyJhbGciOiJIUzI1NiIs..."
]);
```

Response Object

The response object has these properties:

```
$response->status;        // HTTP status code (200, 201, 404, etc.)
$response->body;          // Response body as a string
$response->headers;       // Response headers as an associative array (lowercase keys)
$response->contentType;   // Content-Type header value
$response->json();        // Parse body as associative array (or null if not JSON)
$response->text();        // Body as a string (alias for ->body)
```

---

6. Testing Authentication

Create `tests/AuthTest.php`:

```
<?php
use Tina4\Test;

class AuthTest extends Test
{
    private ?string $token = null;

    public function testLoginWithValidCredentials()
    {
        $response = $this->post("/api/auth/login", [
            "email" => "admin@example.com",
            "password" => "correct-password"
        ]);

        $this->assertEqual($response->status, 200, "Login should succeed");

        $body = json_decode($response->body, true);
        $this->assertNotNull($body["token"], "Should return a JWT token");
        $this->assertTrue(strlen($body["token"]) > 50, "Token should be a substantial string");

        $this->token = $body["token"];
    }

    public function testLoginWithInvalidPassword()
    {
        $response = $this->post("/api/auth/login", [
            "email" => "admin@example.com",
            "password" => "wrong-password"
        ]);

        $this->assertEqual($response->status, 401, "Should reject invalid password");
    }

    public function testLoginWithMissingFields()
    {
        $response = $this->post("/api/auth/login", [
            "email" => "admin@example.com"
        ]);

        $this->assertTrue(
            $response->status === 400 || $response->status === 401,
            "Should reject missing password"
```

```
        );
    }

    public function testProtectedRouteWithoutToken()
    {
        $response = $this->get("/api/profile");

        $this->assertEqual($response->status, 401, "Should reject unauthenticated request");
    }

    public function testProtectedRouteWithValidToken()
    {
        // Login first to get a token
        $loginResponse = $this->post("/api/auth/login", [
            "email" => "admin@example.com",
            "password" => "correct-password"
        ]);
        $loginBody = json_decode($loginResponse->body, true);
        $token = $loginBody["token"];

        // Access protected route with token
        $response = $this->get("/api/profile", [
            "Authorization" => "Bearer " . $token
        ]);

        $this->assertEqual($response->status, 200, "Should allow authenticated request");

        $body = json_decode($response->body, true);
        $this->assertEqual($body["user"]["email"], "admin@example.com", "Should return user data
    }

    public function testProtectedRouteWithInvalidToken()
    {
        $response = $this->get("/api/profile", [
            "Authorization" => "Bearer invalid.token.here"
        ]);

        $this->assertEqual($response->status, 401, "Should reject invalid token");
    }
}
```

---

7. Setup and Teardown

Use `setUp()` and `tearDown()` methods to run code before and after each test:

```
<?php
use Tina4\Test;

class UserTest extends Test
{
    private ?int $userId = null;

    public function setUp(): void
    {
        // Runs before each test
        $user = new User();
        $user->name = "Test User";
```

```
        $user->email = "test-" . time() . "@example.com";
        $user->save();
        $this->userId = $user->id;
    }

    public function tearDown(): void
    {
        // Runs after each test
        if ($this->userId) {
            $user = new User();
            $user->load($this->userId);
            if (!empty($user->id)) {
                $user->delete();
            }
        }
    }

    public function testUserExists()
    {
        $user = new User();
        $user->load($this->userId);
        $this->assertNotNull($user->id, "User should exist");
        $this->assertEqual($user->name, "Test User", "Name should match");
    }

    public function testUpdateUser()
    {
        $user = new User();
        $user->load($this->userId);
        $user->name = "Updated Name";
        $user->save();

        $reloaded = new User();
        $reloaded->load($this->userId);
        $this->assertEqual($reloaded->name, "Updated Name", "Name should be updated");
    }
}
```

`setUp()` runs before every test method. `tearDown()` runs after every test method. Pass or fail, the cleanup runs. Each test starts with a clean state.

---

8. Running Tests

Run All Tests

```
tina4 test
```

`tina4 test` runs the full PHPUnit suite under `tests/`. For more targeted runs, invoke PHPUnit directly; the framework's `tina4 test` is a convenience wrapper that shells out to `vendor/bin/phpunit tests --verbose --color`.

Run a Specific Test File

```
vendor/bin/phpunit tests/ProductTest.php
```

Run a Specific Test Method

```
vendor/bin/phpunit --filter testCreateProduct tests/ProductTest.php
```

Verbose Output

```
vendor/bin/phpunit tests --verbose --color

Running tests...

  ProductTest
    [PASS] test_create_product (0.03s)
      assertEqual: Product should have an ID after save
      assertTrue: Product ID should be positive
    [PASS] test_load_product (0.02s)
      assertEqual: Name should match
      assertEqual: Category should match
      assertEqual: Price should match
      assertTrue: Should be in stock by default
    [PASS] test_update_product (0.04s)
      assertEqual: Name should be updated
      assertEqual: Price should be updated
    [PASS] test_delete_product (0.02s)
      assertTrue: Deleted product should not be loadable
    [PASS] test_select_with_filter (0.05s)
      assertTrue: Should find at least 2 FilterCat products
      assertTrue: Should include Filter Test A
      assertTrue: Should include Filter Test B

  5 tests, 5 passed, 0 failed (0.16s)
```

Verbose mode shows each assertion within each test, along with timing information.

Failed Test Output

When a test fails, you see exactly what went wrong:

```
  ProductTest
    [PASS] test_create_product
    [FAIL] test_load_product
      assertEqual FAILED: Name should match
        Expected: "Load Test Widget"
        Actual:   "Wrong Name"
        File: tests/ProductTest.php:34
    [PASS] test_update_product

  3 tests, 2 passed, 1 failed (0.12s)
```

The failure message shows the assertion that failed, what was expected, what was actually received, and the file and line number.

---

9. PHPUnit Integration

If your team already uses PHPUnit, Tina4 tests can run alongside it. Install PHPUnit via Composer:

```
composer require --dev phpunit/phpunit
```

Create `phpunit.xml` at the project root:

```
<?xml version="1.0" encoding="UTF-8"?>
<phpunit bootstrap="vendor/autoload.php"
         colors="true"
         stopOnFailure="false">
    <testsuites>
        <testsuite name="Application">
            <directory>tests</directory>
        </testsuite>
    </testsuites>
</phpunit>
```

Tina4's `Test` class is compatible with PHPUnit. Your test files work with both `tina4 test` and `vendor/bin/phpunit` without modification.

```
vendor/bin/phpunit
```

```
PHPUnit 10.5.0 by Sebastian Bergmann and contributors.

.....                                                               5 / 5 (100%)

Time: 00:00.340, Memory: 8.00 MB

OK (5 tests, 12 assertions)
```

Code Coverage

With PHPUnit, you can generate code coverage reports:

```
vendor/bin/phpunit --coverage-text
```

```
Code Coverage Report:
  Summary:
    Classes:  60.00% (3/5)
    Methods:  75.00% (15/20)
    Lines:    82.14% (115/140)

  Product
    Methods:  100.00% ( 5/ 5)   Lines: 100.00% ( 28/ 28)
  User
    Methods:  66.67% ( 4/ 6)   Lines:  78.95% ( 30/ 38)
```

For HTML coverage reports:

```
vendor/bin/phpunit --coverage-html coverage/
```

Open `coverage/index.html` in your browser to see a visual breakdown of which lines are covered by tests.

Note: Code coverage requires the Xdebug or PCOV extension. Install one of them:

```
# Xdebug
pecl install xdebug

# PCOV (faster for coverage only)
pecl install pcov
```

---

10. Testing Best Practices

Test One Thing Per Test

Each test method verifies one behavior. When it fails, you know what broke.

```
// Good: each test verifies one thing
public function testCreateProductReturns201()
{
    $response = $this->post("/api/products", ["name" => "Widget", "price" => 9.99]);
    $this->assertEqual($response->status, 201, "Should return 201");
}

public function testCreateProductReturnsCreatedProduct()
{
    $response = $this->post("/api/products", ["name" => "Widget", "price" => 9.99]);
    $body = json_decode($response->body, true);
    $this->assertEqual($body["name"], "Widget", "Should return the product name");
}

// Avoid: testing multiple unrelated things
public function testEverything()
{
    // Creates, reads, updates, deletes, checks auth, validates input...
    // When this fails, you don't know which part broke
}
```

Use Descriptive Assertion Messages

```
// Good: tells you what went wrong
$this->assertEqual($user->name, "Alice", "User name should be 'Alice' after creation");

// Bad: unhelpful when it fails
$this->assertEqual($user->name, "Alice", "Failed");
```

Isolate Tests

Each test creates its own data and cleans up after itself. Never depend on data from another test or from the development database.

```
// Good: creates its own data
public function testDeleteProduct()
{
    $product = new Product();
    $product->name = "Temporary";
    $product->price = 1.00;
    $product->save();

    $product->delete();

    $check = new Product();
    $check->load($product->id);
    $this->assertTrue(empty($check->id), "Product should be deleted");
}

// Bad: depends on data from another test or the dev database
public function testDeleteProduct()
{
    $product = new Product();
```

```
    $product->load(1);  // Assumes product with ID 1 exists
    $product->delete();
}
```

---

11. Exercise: Test a User Model and Authentication Flow

Write a complete test suite for a User model and authentication system.

Requirements

Create `tests/UserModelTest.php` with these tests:

- **testCreateUser** -- Create a user with name, email. Verify the user gets an ID and all fields are correct.
- **testDuplicateEmail** -- Try to create two users with the same email. Verify the second one raises an exception or returns an error.
- **testUpdateUser** -- Create a user, update their name, reload and verify.
- **testDeleteUser** -- Create a user, delete them, verify they cannot be loaded.
- **testSelectUsers** -- Create 3 users, query all, verify count is at least 3.

Create `tests/AuthFlowTest.php` with these tests:

- **testRegisterNewUser** -- POST to `/api/auth/register` with name, email, password. Verify 201 status.
- **testRegisterDuplicateEmail** -- Register the same email twice. Verify the second returns 409 Conflict.
- **testLoginSuccess** -- Login with correct credentials. Verify you get a JWT token.
- **testLoginFailure** -- Login with wrong password. Verify 401 status.
- **testAccessProtectedRoute** -- Use the token from login to access `/api/profile`. Verify 200 status and correct user data.
- **testAccessWithExpiredToken** -- Use a manually crafted expired token. Verify 401 status.

Expected output:

```
tina4 test

Running tests...

  UserModelTest
    [PASS] test_create_user
    [PASS] test_duplicate_email
    [PASS] test_update_user
    [PASS] test_delete_user
    [PASS] test_select_users

  AuthFlowTest
    [PASS] test_register_new_user
    [PASS] test_register_duplicate_email
    [PASS] test_login_success
```

```
  [PASS] test_login_failure
  [PASS] test_access_protected_route
  [PASS] test_access_with_expired_token

11 tests, 11 passed, 0 failed (0.52s)
```

---

12. Solution

tests/UserModelTest.php

```
<?php
use Tina4\Test;

class UserModelTest extends Test
{
    public function testCreateUser()
    {
        $user = new User();
        $user->name = "Test User";
        $user->email = "testuser-" . uniqid() . "@example.com";
        $user->save();

        $this->assertNotNull($user->id, "User should have an ID after save");
        $this->assertEqual($user->name, "Test User", "Name should match");
        $this->assertTrue(str_contains($user->email, "@example.com"), "Email should be set");
    }

    public function testDuplicateEmail()
    {
        $email = "duplicate-" . uniqid() . "@example.com";

        $user1 = new User();
        $user1->name = "First User";
        $user1->email = $email;
        $user1->save();

        $this->assertRaises(function () use ($email) {
            $user2 = new User();
            $user2->name = "Second User";
            $user2->email = $email;
            $user2->save();
        }, \Exception::class, "Should reject duplicate email");
    }

    public function testUpdateUser()
    {
        $user = new User();
        $user->name = "Original Name";
        $user->email = "update-" . uniqid() . "@example.com";
        $user->save();

        $id = $user->id;
        $user->name = "New Name";
        $user->save();

        $reloaded = new User();
        $reloaded->load($id);
        $this->assertEqual($reloaded->name, "New Name", "Name should be updated");
    }

    public function testDeleteUser()
    {
        $user = new User();
        $user->name = "Delete Me";
        $user->email = "delete-" . uniqid() . "@example.com";
```

```
        $user->save();

        $id = $user->id;
        $user->delete();

        $gone = new User();
        $gone->load($id);
        $this->assertTrue(empty($gone->id), "Deleted user should not exist");
    }

    public function testSelectUsers()
    {
        for ($i = 0; $i < 3; $i++) {
            $user = new User();
            $user->name = "Select Test User " . $i;
            $user->email = "select-test-" . $i . "-" . uniqid() . "@example.com";
            $user->save();
        }

        $user = new User();
        $results = $user->select("*");

        $this->assertTrue(count($results) >= 3, "Should have at least 3 users");
    }
}
```

tests/AuthFlowTest.php

```
<?php
use Tina4\Test;

class AuthFlowTest extends Test
{
    private string $testEmail;
    private string $testPassword = "SecurePassword123!";

    public function setUp(): void
    {
        $this->testEmail = "auth-test-" . uniqid() . "@example.com";
    }

    public function testRegisterNewUser()
    {
        $response = $this->post("/api/auth/register", [
            "name" => "Auth Test User",
            "email" => $this->testEmail,
            "password" => $this->testPassword
        ]);

        $this->assertEqual($response->status, 201, "Registration should return 201");

        $body = json_decode($response->body, true);
        $this->assertEqual($body["name"], "Auth Test User", "Should return user name");
        $this->assertNotNull($body["id"], "Should return user ID");
    }

    public function testRegisterDuplicateEmail()
    {
        $email = "dup-" . uniqid() . "@example.com";

        // Register first time
        $this->post("/api/auth/register", [
            "name" => "First",
            "email" => $email,
            "password" => $this->testPassword
        ]);

        // Register again with same email
        $response = $this->post("/api/auth/register", [
            "name" => "Second",
            "email" => $email,
            "password" => $this->testPassword
        ]);

        $this->assertEqual($response->status, 409, "Duplicate email should return 409");
    }

    public function testLoginSuccess()
    {
        $email = "login-" . uniqid() . "@example.com";

        // Register
        $this->post("/api/auth/register", [
            "name" => "Login User",
            "email" => $email,
```

```
        "password" => $this->testPassword
    ]);

    // Login
    $response = $this->post("/api/auth/login", [
        "email" => $email,
        "password" => $this->testPassword
    ]);

    $this->assertEqual($response->status, 200, "Login should return 200");

    $body = json_decode($response->body, true);
    $this->assertNotNull($body["token"], "Should return a token");
}

public function testLoginFailure()
{
    $response = $this->post("/api/auth/login", [
        "email" => "nobody@example.com",
        "password" => "wrong"
    ]);

    $this->assertEqual($response->status, 401, "Invalid login should return 401");
}

public function testAccessProtectedRoute()
{
    $email = "profile-" . uniqid() . "@example.com";

    // Register and login
    $this->post("/api/auth/register", [
        "name" => "Profile User",
        "email" => $email,
        "password" => $this->testPassword
    ]);

    $loginResponse = $this->post("/api/auth/login", [
        "email" => $email,
        "password" => $this->testPassword
    ]);

    $token = json_decode($loginResponse->body, true)["token"];

    // Access protected route
    $response = $this->get("/api/profile", [
        "Authorization" => "Bearer " . $token
    ]);

    $this->assertEqual($response->status, 200, "Should allow access with valid token");

    $body = json_decode($response->body, true);
    $this->assertEqual($body["user"]["email"], $email, "Should return correct user");
}

public function testAccessWithExpiredToken()
{
    $response = $this->get("/api/profile", [
        "Authorization" => "Bearer expired.invalid.token"
    ]);
```

```
        $this->assertEqual($response->status, 401, "Should reject invalid token");
    }
}
```

---

13. Gotchas

1. Tests Run in Order

Problem: Test B depends on data created in Test A, but sometimes Test B fails.

Cause: While tests within a class run in the order they are defined, test classes may run in any order. If Test B depends on Test A being in a different class, this is fragile.

Fix: Each test should create its own data. Never depend on another test's side effects. Use `setUp()` to create the data each test needs.

2. Test Database vs Development Database

Problem: Running tests deletes your development data.

Cause: Tests are running against the same database as your development server.

Fix: Tina4 uses a separate test database by default (`data/test.db`). If your tests are modifying development data, check that `TINA4_DATABASE_URL` is set correctly in `.env`, or that you are running `tina4 test` (not executing test files manually).

3. Unique Constraint Failures

Problem: Tests fail with "UNIQUE constraint failed" on the email field.

Cause: Previous test runs left data in the test database, or two tests create records with the same email.

Fix: Use `uniqid()` or `time()` to generate unique values in tests:

```
$user->email = "test-" . uniqid() . "@example.com";
```

4. Test Method Not Discovered

Problem: You wrote a test method but it does not appear in the output.

Cause: The method name does not start with `test`. Tina4 only runs methods that begin with `test` (case-sensitive).

Fix: Rename the method to start with `test`: `testCreateProduct`, `testLoginFailure`, etc.

5. Assertion Arguments Reversed

Problem: The failure message shows "Expected: 404, Actual: 200" but that seems backward.

Cause: You passed the arguments in the wrong order. `assertEqual($actual, $expected)` -- actual comes first, expected comes second.

Fix: Follow the convention: `$this->assertEqual($response->status, 200, "message")`. The first argument ~~is what you got, the second is~~ what you expected.

6. Cannot Test Routes That Require Authentication Setup

Problem: Tests for protected routes fail because the authentication middleware expects a database table or JWT secret that does not exist in the test environment.

Cause: The test database is empty -- it does not have the users table or the JWT secret is not configured.

Fix: Use `setUp()` to create the necessary data:

```
public function setUp(): void
{
    // Create the users table if it does not exist
    $user = new User();
    $user->createTable();

    // Create a test user
    $user->name = "Test Admin";
    $user->email = "admin@test.com";
    $user->passwordHash = password_hash("password", PASSWORD_DEFAULT);
    $user->save();
}
```

7. Tests Pass Locally but Fail in CI

Problem: All tests pass on your machine but fail in continuous integration.

Cause: The CI environment may have a different PHP version, missing extensions, or different filesystem permissions. Also, tests that depend on time or random values can be flaky.

Fix: Make tests deterministic. Avoid relying on the current time, filesystem state, or external services. If you must test time-dependent behavior, mock the clock or use fixed timestamps. Check that your CI environment matches your local PHP version and has all required extensions.

Scaffolding

One command. Six files. A working CRUD feature with routes, templates, tests, and Swagger docs -- ready to run.

That is Tina4's scaffolding system. It generates the boilerplate you write by hand in every project: models, migrations, routes, forms, views, and tests. You describe what you want. The generators produce it.

The CRUD Generator

This is the generator most developers reach for first. It creates everything a feature needs in one shot.

```
tina4 generate crud Product --fields "name:string,price:float"
```

That single command creates six files:

#	File	Purpose
1	`src/orm/Product.php`	ORM model with typed fields
2	`migrations/20260401_create_product.sql`	UP migration (CREATE TABLE)
3	`migrations/20260401_create_product.down.sql`	DOWN migration (DROP TABLE)
4	`src/routes/products.php`	CRUD routes with Swagger annotations
5	`src/templates/products/form.html`	Form template with typed inputs and `form_token`
6	`src/templates/products/view.html`	List and detail templates
7	`tests/ProductTest.php`	PHPUnit stubs for all CRUD operations

What Each File Contains

The **model** maps the `product` table to a PHP class:

```
<?php

use Tina4\ORM;

class Product extends ORM
{
```

```
    public string $tableName = "product";
    public ?int $id = null;
    public ?string $name = null;
    public ?float $price = null;
    public ?string $createdAt = null;
    public ?string $updatedAt = null;
}
```

The **migration** creates the table:

```
-- migrations/20260401_create_product.sql
CREATE TABLE product (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name VARCHAR(255),
    price FLOAT,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

The **down migration** reverses it:

```
-- migrations/20260401_create_product.down.sql
DROP TABLE IF EXISTS product;
```

The **routes** file wires up five endpoints with Swagger docs:

```
<?php

use Tina4\Router;

Router::get("/api/products", function ($request, $response) {
    $products = (new Product())->select();
    return $response($products);
})->swagger(['summary' => 'List all products', 'tags' => ['products']]);

Router::get("/api/products/{id}", function ($request, $response) {
    $product = new Product();
    $product->id = $request->params["id"];
    $product->load();
    return $response($product);
})->swagger(['summary' => 'Get a product by ID', 'tags' => ['products']]);

Router::post("/api/products", function ($request, $response) {
    $product = new Product($request->body);
    $product->save();
    return $response($product, 201);
})->swagger(['summary' => 'Create a product', 'tags' => ['products']]);

Router::put("/api/products/{id}", function ($request, $response) {
    $product = new Product($request->body);
    $product->id = $request->params["id"];
    $product->save();
    return $response($product);
})->swagger(['summary' => 'Update a product', 'tags' => ['products']]);

Router::delete("/api/products/{id}", function ($request, $response) {
    $product = new Product();
    $product->id = $request->params["id"];
    $product->delete();
    return $response(null, 204);
```

```
})->swagger(['summary' => 'Delete a product', 'tags' => ['products']]);
```

The route handler signature is `function ($request, $response)`: `$request` first, `$response` second. Use `->swagger([...])` for OpenAPI metadata; the keys are pass-through to the generator (`summary`, `tags`, `description`, `parameters`, etc.).

The **form template** renders typed inputs with CSRF protection:

```
<form method="POST" action="/api/products">
    <input type="hidden" name="form_token" value="{{ form_token }}">
    <label>Name</label>
    <input type="text" name="name" required>
    <label>Price</label>
    <input type="number" step="0.01" name="price" required>
    <button type="submit">Save</button>
</form>
```

The **test file** stubs out CRUD assertions:

```
<?php

use PHPUnit\Framework\TestCase;
use Tina4\App;

class ProductTest extends TestCase
{
    private $client;

    protected function setUp(): void
    {
        $app = new App();
        $this->client = $app->testClient();
    }

    public function testCreateProduct(): void
    {
        $response = $this->client->post("/api/products", ["name" => "Widget", "price" => 9.99]);
        $this->assertEquals(201, $response->statusCode);
    }

    public function testListProducts(): void
    {
        $response = $this->client->get("/api/products");
        $this->assertEquals(200, $response->statusCode);
    }

    public function testGetProduct(): void
    {
        $response = $this->client->get("/api/products/1");
        $this->assertEquals(200, $response->statusCode);
    }

    public function testUpdateProduct(): void
    {
        $response = $this->client->put("/api/products/1", ["name" => "Updated Widget"]);
        $this->assertEquals(200, $response->statusCode);
    }

    public function testDeleteProduct(): void
```

```
    {
        $response = $this->client->delete("/api/products/1");
        $this->assertEquals(204, $response->statusCode);
    }
}
```

Run It

After generating, run the migration and start the server:

```
tina4 migrate
tina4 serve
```

Open Swagger UI at `http://localhost:7145/swagger` and test every endpoint. The scaffolded code works out of the box.

Individual Generators

The CRUD generator calls several smaller generators under the hood. You can call each one directly when you need a single piece.

Model

```
tina4 generate model Product --fields "name:string,price:float"
```

Creates three files: the ORM model (`src/orm/Product.php`), the UP migration, and the DOWN migration. No routes, no templates, no tests.

Route

```
tina4 generate route products --model Product
```

Creates one file: `src/routes/products.php` with CRUD endpoints and Swagger annotations. The model must exist first.

Migration

```
tina4 generate migration add_category_to_product
```

Creates two files: `migrations/20260401_add_category_to_product.sql` and `migrations/20260401_add_category_to_product.down.sql`. Both are empty stubs. You write the SQL.

Middleware

```
tina4 generate middleware AuthLog
```

Creates one file with before and after stubs:

```
<?php

use Tina4\Middleware;

Middleware::add("AuthLogBefore", function ($request) {
    echo "Request: {$request->method} {$request->url}\n";
    return $request;
}, true); // before
```

```
Middleware::add("AuthLogAfter", function ($request, $response) {
    echo "Response: {$response->statusCode}\n";
    return $response;
}, false); // after
```

Test

```
tina4 generate test products --model Product
```

Creates one file: `tests/ProductTest.php` with PHPUnit CRUD stubs.

Form

```
tina4 generate form Product --fields "name:string,price:float"
```

Creates one file: `src/templates/products/form.html` with typed inputs and `form_token`.

View

```
tina4 generate view Product --fields "name:string,price:float"
```

Creates two templates: a list view and a detail view in `src/templates/products/`.

CRUD

```
tina4 generate crud Product --fields "name:string,price:float"
```

Shorthand for running all generators at once: model, migration, route, form, view, and test.

Auth

```
tina4 generate auth
```

Generates the full authentication scaffold: User model, migrations, login/register/logout routes, templates, and tests.

AutoCRUD

AutoCRUD automatically generates REST API endpoints from your ORM models:

- `GET /api/{table}` - List with pagination (`?limit=10&offset=0`)
- `GET /api/{table}/{id}` - Get single record
- `POST /api/{table}` - Create record
- `PUT /api/{table}/{id}` - Update record
- `DELETE /api/{table}/{id}` - Delete record

Usage

```
$crud = new AutoCrud($db, '/api');
$crud->register(User::class);
$crud->generateRoutes();
```

`AutoCrud::register()` wires up a single model class. `generateRoutes()` mounts the five standard endpoints automatically, with no route files needed.

The Auth Generator

Authentication needs more than one file. The auth generator creates seven:

```
tina4 generate auth
```

#	File	Purpose
1	`src/orm/User.php`	User model with hashed password field
2	`migrations/20260401_create_user.sql`	UP migration
3	`migrations/20260401_create_user.down.sql`	DOWN migration
4	`src/routes/auth.php`	Login, register, logout routes
5	`src/templates/auth/login.html`	Login form
6	`src/templates/auth/register.html`	Registration form
7	`tests/AuthTest.php`	Auth flow tests

The generated routes handle password hashing, JWT token creation, and session management. The templates include CSRF tokens. The tests cover registration, login, invalid credentials, and logout.

Run the migration, start the server, and you have working auth:

```
tina4 migrate
tina4 serve
```

Field Types

Generators accept these field types. Each type maps to a specific column type in migrations, input type in forms, and display format in views.

Field Type	Migration Column	Form Input	View Display
`string`	`VARCHAR(255)`	`<input type="text">`	Plain text
`int`	`INTEGER`	`<input type="number">`	Number

`float`	`FLOAT`	`<input type="number" step="0.01">`	Decimal
`bool`	`BOOLEAN`	`<input type="checkbox">`	Yes / No
`text`	`TEXT`	`<textarea>`	Paragraph
`datetime`	`DATETIME`	`<input type="datetime-local">`	Formatted date
`blob`	`BLOB`	`<input type="file">`	Download link

Table Naming Convention

Tina4 uses singular table names. The model name `Product` maps to the table `product`. The model name `OrderItem` maps to `order_item`. The generator handles the conversion.

Combining Generators

Sometimes you want a model with routes but no form. Or a model with a migration but no test. The `--with` flags let you compose:

```
tina4 generate model Product --fields "name:string,price:float" --with-route --with-migration
```

Available flags:

Flag	Adds
`--with-route`	CRUD route file
`--with-migration`	Migration files (included by default with model)
`--with-test`	Test file
`--with-form`	Form template
`--with-view`	View templates

The `generate crud` command is equivalent to using all `--with` flags at once.

Exercise: Scaffold a Blog

Build a blog with three resources using generators.

Step 1: Generate the auth system.

```
tina4 generate auth
```

Step 2: Scaffold the Post resource.

```
tina4 generate crud Post --fields "title:string,body:text,published:bool"
```

Step 3: Scaffold the Category resource.

```
tina4 generate crud Category --fields "name:string,description:text"
```

Step 4: Add a migration to link posts to categories.

```
tina4 generate migration add_category_id_to_post
```

Edit the migration to add the foreign key:

```
ALTER TABLE post ADD COLUMN category_id INTEGER REFERENCES category(id);
```

Step 5: Run all migrations and start the server.

```
tina4 migrate
tina4 serve
```

You now have a working blog with authentication, posts, categories, and Swagger documentation. Total commands: five. Total hand-written SQL: one line.

Gotchas

Generators Do Not Overwrite

If a file exists, the generator skips it and prints a warning. This protects your edits. To regenerate, delete the file first.

Run Migrate After Generate

The model generator creates migration files. Those files do nothing until you run `tina4 migrate`. Generate and migrate are separate steps by design.

File Naming Matters

The generator derives file names from the model name. `Product` becomes `Product.php` for the model and `products.php` for routes. Do not rename generated files unless you update all imports.

Field Changes Need New Migrations

Changing `--fields` and re-running the generator does not update existing migrations. Create a new migration with `generate migration` and write the ALTER TABLE by hand.

Singular Table Names

Tina4 uses singular table names: `product`, not `products`. The route paths use plural (`/api/products`), but the table stays singular. The generator handles this split.

API Documentation with Swagger

1. The 47-Endpoint Problem

Your team ships 47 API endpoints. The frontend developer asks what each one accepts. You email a spreadsheet. It goes stale in a week. You write a wiki page. Nobody updates it. You add code comments. Nobody reads them.

Swagger solves this for good. It generates interactive API documentation from annotations in your route files. The docs stay current because they live in the code. Your frontend developer browses every endpoint, sees request and response shapes, and tests calls from the browser.

Tina4 auto-generates a Swagger UI at `/swagger` from doc-block annotations. No build step. No extra tooling. Write the annotations. The documentation appears.

---

2. What Swagger/OpenAPI Is

OpenAPI is a specification for describing REST APIs. Swagger is the toolset that reads OpenAPI specs and produces documentation, client SDKs, and server stubs.

An OpenAPI spec describes:

- Every endpoint (path + HTTP method)
- Parameters (path, query, header, body)
- Responses (status codes, body schemas)
- Data schemas (what a "User" or "Product" looks like)
- Authentication requirements
- Grouping and tagging

Tina4 builds this spec from doc-block comments in your PHP. No JSON or YAML by hand.

---

3. Enabling Swagger

Available out of the box when `TINA4_DEBUG=true`. Navigate to:

```
http://localhost:7145/swagger
```

The Swagger UI appears with all defined routes. No annotations yet means default descriptions.

For production, control it explicitly:

```
TINA4_SWAGGER_ENABLED=true
```

The Swagger JSON Endpoint

Raw OpenAPI spec:

```
http://localhost:7145/swagger/openapi.json
```

```
curl http://localhost:7145/swagger/openapi.json
```

```
{
  "openapi": "3.0.3",
  "info": {
    "title": "My Store API",
    "version": "1.0.0"
  },
  "paths": {
    "/api/products": {
      "get": {
        "summary": "List all products",
        "responses": {
          "200": {
            "description": "Successful response"
          }
        }
      }
    }
  }
}
```

Import this JSON into Postman, Insomnia, or use it to generate client SDKs.

---

4. Adding Descriptions to Routes

Doc-block comments above route definitions:

```
<?php
use Tina4\Router;

/**
 * List all products
 * @description Returns a paginated list of all products in the catalog
 * @tags Products
 */
Router::get("/api/products", function ($request, $response) {
    return $response->json(["products" => []]);
});
```

First line becomes the `summary`. `@description` provides detail. `@tags` groups the endpoint.

Documenting Path Parameters

```
/**
 * Get a product by ID
 * @description Returns a single product with full details including inventory status
 * @tags Products
 * @param int $id The unique product identifier
 */
Router::get("/api/products/{id:int}", function ($request, $response) {
    $id = $request->params["id"];
    return $response->json([
        "id" => $id,
        "name" => "Wireless Keyboard",
        "price" => 79.99
    ]);
});
```

`@param` tells Swagger about the path parameter. Shows as a required field with type information.

Documenting Query Parameters

```
/**
 * Search products
 * @description Search the product catalog by name, category, or price range
 * @tags Products
 * @query string $q Search query (searches product name and description)
 * @query string $category Filter by category name
 * @query float $min_price Minimum price filter
 * @query float $max_price Maximum price filter
 * @query int $page Page number (default: 1)
 * @query int $limit Items per page (default: 20, max: 100)
 */
Router::get("/api/products/search", function ($request, $response) {
    $q = $request->query["q"] ?? "";
    $page = (int) ($request->query["page"] ?? 1);
    $limit = min((int) ($request->query["limit"] ?? 20), 100);

    return $response->json([
        "query" => $q,
        "page" => $page,
        "limit" => $limit,
        "results" => [],
        "total" => 0
    ]);
});
```

Each `@query` adds a parameter to the docs with type and description.

---

5. Documenting Request and Response Schemas

Request Body

```
/**
 * Create a new product
 * @description Creates a product in the catalog. Requires admin authentication.
 * @tags Products
 * @body {"name": "string", "category": "string", "price": "float", "in_stock": "bool", "descrip
 * @response 201 {"id": "int", "name": "string", "category": "string", "price": "float", "in_sto
 * @response 400 {"error": "string"}
 */
Router::post("/api/products", function ($request, $response) {
    $body = $request->body;

    if (empty($body["name"])) {
        return $response->json(["error" => "Name is required"], 400);
    }

    return $response->json([
        "id" => 1,
        "name" => $body["name"],
        "category" => $body["category"] ?? "Uncategorized",
        "price" => (float) ($body["price"] ?? 0),
        "in_stock" => (bool) ($body["in_stock"] ?? true),
        "created_at" => date("c")
    ], 201);
});
```

`@body` describes the expected JSON. `@response` documents each status code and its payload.

Multiple Response Codes

```
/**
 * Update a product
 * @description Update an existing product by ID. Only provided fields are updated.
 * @tags Products
 * @param int $id Product ID
 * @body {"name": "string", "category": "string", "price": "float", "in_stock": "bool"}
 * @response 200 {"id": "int", "name": "string", "category": "string", "price": "float", "in_sto
 * @response 404 {"error": "string", "id": "int"}
 * @response 400 {"error": "string"}
 */
Router::put("/api/products/{id:int}", function ($request, $response) {
    $id = $request->params["id"];
    $body = $request->body;

    if ($id > 100) {
        return $response->json(["error" => "Product not found", "id" => $id], 404);
    }

    return $response->json([
        "id" => $id,
        "name" => $body["name"] ?? "Widget",
        "category" => $body["category"] ?? "General",
        "price" => (float) ($body["price"] ?? 9.99),
        "in_stock" => (bool) ($body["in_stock"] ?? true),
        "updated_at" => date("c")
    ]);
});
```

Swagger UI shows each response code with its schema. A `200` versus a `404` at a glance.

---

6. Tags for Grouping Endpoints

Tags organize the Swagger UI. Without tags: one flat list. With tags: collapsible sections.

```
/**
 * List all users
 * @tags Users
 */
Router::get("/api/users", function ($request, $response) {
    return $response->json(["users" => []]);
});

/**
 * Get user by ID
 * @tags Users
 */
Router::get("/api/users/{id:int}", function ($request, $response) {
    return $response->json(["id" => $request->params["id"], "name" => "Alice"]);
});

/**
 * List all orders
 * @tags Orders
 */
Router::get("/api/orders", function ($request, $response) {
```

```
    return $response->json(["orders" => []]);
});

/**
 * Create an order
 * @tags Orders
 */
Router::post("/api/orders", function ($request, $response) {
    return $response->json(["order_id" => 1], 201);
});

/**
 * List all products
 * @tags Products
 */
Router::get("/api/products", function ($request, $response) {
    return $response->json(["products" => []]);
});
```

Three sections in the UI: "Users", "Orders", "Products". Each expands to show its endpoints.

Multiple Tags

An endpoint in multiple groups:

```
/**
 * Get user's orders
 * @tags Users, Orders
 */
Router::get("/api/users/{id:int}/orders", function ($request, $response) {
    return $response->json(["orders" => []]);
});
```

Appears in both the "Users" and "Orders" sections.

---

7. Example Values

Realistic data instead of type names:

```
/**
 * Create a new product
 * @description Creates a product in the catalog
 * @tags Products
 * @example request {"name": "Ergonomic Keyboard", "category": "Electronics", "price": 89.99, "i
 * @example response {"id": 42, "name": "Ergonomic Keyboard", "category": "Electronics", "price"
 */
Router::post("/api/products", function ($request, $response) {
    $body = $request->body;

    return $response->json([
        "id" => 42,
        "name" => $body["name"],
        "category" => $body["category"] ?? "Uncategorized",
        "price" => (float) ($body["price"] ?? 0),
        "in_stock" => (bool) ($body["in_stock"] ?? true),
        "created_at" => date("c")
    ], 201);
```

```
});
```

The `@example` annotations populate the Swagger UI. When a developer clicks "Try it out", the example request is pre-filled.

---

8. Try-It-Out from the Swagger UI

Every endpoint has a "Try it out" button. Click it:

- Input fields expand
- Example values pre-fill (if provided)
- Edit parameters, headers, body
- The actual HTTP request fires against your running server
- Response appears: status, headers, body

A live testing tool inside your documentation. No Postman needed.

Authentication in Try-It-Out

Endpoints requiring auth show a lock icon. Click "Authorize" at the top of the Swagger UI. Enter your JWT or API key. All subsequent requests include the header.

Tina4 auto-detects auth requirements from `@secured` and `@noauth` annotations.

---

9. Customizing the Swagger Info Block

Configure in `.env`:

```
TINA4_SWAGGER_TITLE=My Store API
TINA4_SWAGGER_DESCRIPTION=API for managing products, orders, and users
TINA4_SWAGGER_VERSION=1.0.0
```

This appears in the Swagger UI header and the OpenAPI spec:

```
{
  "openapi": "3.0.3",
  "info": {
    "title": "My Store API",
    "description": "API for managing products, orders, and users",
    "version": "1.0.0",
    "contact": {
      "email": "api@mystore.com"
    },
    "license": {
      "name": "MIT"
    }
  }
}
```

---

10. Generating Client SDKs from the Spec

The OpenAPI spec at `/swagger/openapi.json` feeds code generation tools. Client libraries in any language.

Using OpenAPI Generator

```
npm install -g @openapitools/openapi-generator-cli

# TypeScript client
openapi-generator-cli generate \
  -i http://localhost:7145/swagger/openapi.json \
  -g typescript-fetch \
  -o ./frontend/api-client

# Python client
openapi-generator-cli generate \
  -i http://localhost:7145/swagger/openapi.json \
  -g python \
  -o ./python-client
```

The generated code is typed. IDE autocompletion works:

```
const api = new ProductsApi();

const product = await api.getProductById({ id: 42 });
console.log(product.name);  // TypeScript knows this is a string

const newProduct = await api.createProduct({
    name: "Widget",
    category: "General",
    price: 9.99,
    inStock: true
});
```

Update annotations. Regenerate. Client stays in sync.

---

11. A Complete Documented API

All annotation features together:

```
<?php
use Tina4\Router;

/**
 * List all users
 * @description Returns a paginated list of users. Supports filtering by role and searching by n
 * @tags Users
 * @query int $page Page number (default: 1)
 * @query int $limit Items per page (default: 20)
 * @query string $role Filter by role (admin, user, moderator)
 * @query string $search Search by name or email
 * @response 200 {"users": [{"id": "int", "name": "string", "email": "string", "role": "string"}
 * @example response {"users": [{"id": 1, "name": "Alice", "email": "alice@example.com", "role":
 */
Router::get("/api/users", function ($request, $response) {
```

```
    $page = (int) ($request->query["page"] ?? 1);
    $limit = (int) ($request->query["limit"] ?? 20);

    return $response->json([
        "users" => [
            ["id" => 1, "name" => "Alice", "email" => "alice@example.com", "role" => "admin"],
            ["id" => 2, "name" => "Bob", "email" => "bob@example.com", "role" => "user"]
        ],
        "total" => 42,
        "page" => $page,
        "pages" => (int) ceil(42 / $limit)
    ]);
});

/**
 * Get user by ID
 * @description Returns full user profile including account creation date
 * @tags Users
 * @param int $id User ID
 * @response 200 {"id": "int", "name": "string", "email": "string", "role": "string", "created_a
 * @response 404 {"error": "string"}
 * @example response {"id": 1, "name": "Alice", "email": "alice@example.com", "role": "admin", "
 */
Router::get("/api/users/{id:int}", function ($request, $response) {
    $id = $request->params["id"];

    if ($id > 100) {
        return $response->json(["error" => "User not found"], 404);
    }

    return $response->json([
        "id" => $id,
        "name" => "Alice",
        "email" => "alice@example.com",
        "role" => "admin",
        "created_at" => "2026-01-15T10:30:00+00:00"
    ]);
});

/**
 * Create a new user
 * @description Creates a user account. Email must be unique.
 * @tags Users
 * @body {"name": "string", "email": "string", "password": "string", "role": "string"}
 * @response 201 {"id": "int", "name": "string", "email": "string", "role": "string", "created_a
 * @response 400 {"errors": ["string"]}
 * @response 409 {"error": "string"}
 * @example request {"name": "Charlie", "email": "charlie@example.com", "password": "securePass1
 * @example response {"id": 3, "name": "Charlie", "email": "charlie@example.com", "role": "user"
 */
Router::post("/api/users", function ($request, $response) {
    $body = $request->body;

    $errors = [];
    if (empty($body["name"])) $errors[] = "Name is required";
    if (empty($body["email"])) $errors[] = "Email is required";
    if (empty($body["password"])) $errors[] = "Password is required";

    if (!empty($errors)) {
```

```
        return $response->json(["errors" => $errors], 400);
    }

    return $response->json([
        "id" => 3,
        "name" => $body["name"],
        "email" => $body["email"],
        "role" => $body["role"] ?? "user",
        "created_at" => date("c")
    ], 201);
});

/**
 * Delete a user
 * @description Permanently deletes a user account. Requires admin role.
 * @tags Users
 * @param int $id User ID
 * @response 204
 * @response 404 {"error": "string"}
 */
Router::delete("/api/users/{id:int}", function ($request, $response) {
    $id = $request->params["id"];

    if ($id > 100) {
        return $response->json(["error" => "User not found"], 404);
    }

    return $response->json(null, 204);
});
```

Visit `/swagger`. Four endpoints under "Users". Full parameter documentation, examples, multiple response codes.

---

12. Exercise: Document a Complete User API

Extend the User API with three more endpoints. Full Swagger annotations on each.

Requirements

Method	Path	Description
`PUT`	`/api/users/{id}`	Update a user. Body: name, email, role. Response: updated user.
`GET`	`/api/users/{id}/orders`	List a user's orders. Query: status filter, pagination.
`POST`	`/api/users/{id}/avatar`	Upload user avatar. Body: avatar_url string.

Each endpoint needs:

- Summary (first line)

- `@description`
- `@tags`
- `@param` for path parameters
- `@query` for query parameters (where applicable)
- `@body` for request body (where applicable)
- `@response` for each status code
- `@example` for request and response (where applicable)

Verify at:

```
http://localhost:7145/swagger
```

---

13. Solution

Create `src/routes/user-api-documented.php`:

```
<?php
use Tina4\Router;

/**
 * Update a user
 * @description Updates an existing user's profile information. Only provided fields are updated
 * @tags Users
 * @param int $id User ID
 * @body {"name": "string", "email": "string", "role": "string"}
 * @response 200 {"id": "int", "name": "string", "email": "string", "role": "string", "updated_a
 * @response 404 {"error": "string"}
 * @response 409 {"error": "string"}
 * @example request {"name": "Alice Smith", "email": "alice.smith@example.com", "role": "admin"}
 * @example response {"id": 1, "name": "Alice Smith", "email": "alice.smith@example.com", "role"
 */
Router::put("/api/users/{id:int}", function ($request, $response) {
    $id = $request->params["id"];
    $body = $request->body;

    if ($id > 100) {
        return $response->json(["error" => "User not found"], 404);
    }

    return $response->json([
        "id" => $id,
        "name" => $body["name"] ?? "Alice",
        "email" => $body["email"] ?? "alice@example.com",
        "role" => $body["role"] ?? "user",
        "updated_at" => date("c")
    ]);
});

/**
 * List user orders
 * @description Returns a paginated list of orders for a specific user. Supports filtering by or
 * @tags Users, Orders
```

```
 * @param int $id User ID
 * @query string $status Filter by order status (pending, processing, shipped, delivered, cancel
 * @query int $page Page number (default: 1)
 * @query int $limit Items per page (default: 20)
 * @response 200 {"orders": [{"id": "int", "product": "string", "quantity": "int", "total": "flo
 * @response 404 {"error": "string"}
 * @example response {"orders": [{"id": 101, "product": "Wireless Keyboard", "quantity": 2, "tot
 */
Router::get("/api/users/{id:int}/orders", function ($request, $response) {
    $id = $request->params["id"];
    $status = $request->query["status"] ?? null;
    $page = (int) ($request->query["page"] ?? 1);

    if ($id > 100) {
        return $response->json(["error" => "User not found"], 404);
    }

    $orders = [
        ["id" => 101, "product" => "Wireless Keyboard", "quantity" => 2, "total" => 159.98, "sta
        ["id" => 102, "product" => "USB-C Hub", "quantity" => 1, "total" => 49.99, "status" => "
    ];

    if ($status !== null) {
        $orders = array_values(array_filter($orders, fn($o) => $o["status"] === $status));
    }

    return $response->json(["orders" => $orders, "total" => count($orders), "page" => $page]);
});

/**
 * Upload user avatar
 * @description Sets or updates the avatar URL for a user. The avatar should be hosted on a CDN
 * @tags Users
 * @param int $id User ID
 * @body {"avatar_url": "string"}
 * @response 200 {"id": "int", "avatar_url": "string", "updated_at": "string"}
 * @response 400 {"error": "string"}
 * @response 404 {"error": "string"}
 * @example request {"avatar_url": "https://cdn.example.com/avatars/alice-2026.jpg"}
 * @example response {"id": 1, "avatar_url": "https://cdn.example.com/avatars/alice-2026.jpg", "
 */
Router::post("/api/users/{id:int}/avatar", function ($request, $response) {
    $id = $request->params["id"];
    $body = $request->body;

    if ($id > 100) {
        return $response->json(["error" => "User not found"], 404);
    }

    if (empty($body["avatar_url"])) {
        return $response->json(["error" => "avatar_url is required"], 400);
    }

    return $response->json([
        "id" => $id,
        "avatar_url" => $body["avatar_url"],
        "updated_at" => date("c")
    ]);
});
```

Visit `http://localhost:7145/swagger`. Verify:

- "Users" section has six endpoints (list, get, create, update, delete, avatar)
- "Orders" section shows "List user orders" (dual-tagged)
- Each endpoint has summary, description, parameters, request body, response codes
- "Try it out" works
- Examples pre-fill

---

14. Gotchas

1. Annotations Must Be Directly Above the Route

Problem: Swagger annotations missing from the docs.

Cause: Blank line or code between the doc-block and `Router::`. Tina4 reads only doc-blocks immediately above the route definition.

Fix: `*/` must be on the line directly before `Router::get(...)`. No blank lines.

2. Missing @tags Makes Endpoints Hard to Find

Problem: All endpoints in one flat list.

Cause: No `@tags`. Everything grouped under "default".

Fix: Add `@tags ResourceName` to every route.

3. @body Must Be Valid JSON

Problem: Body schema shows empty or broken.

Cause: Malformed JSON. Trailing comma, missing quotes, unescaped characters.

Fix: Validate the JSON. Double quotes on every key and string value. No trailing commas.

4. Swagger Shows Unannotated Routes

Problem: Routes without annotations appear with minimal docs.

Cause: Tina4 includes all registered routes. By design. Nothing hidden.

Fix: Add annotations for quality. Hide a route with `@hidden` in its doc-block.

5. Response Examples Do Not Match Reality

Problem: Example response shows different fields than the actual API.

Cause: Annotations written once, never updated. They are comments. Not validated against code.

Fix: Treat annotations as code. When the handler changes, update annotations. Consider integration tests that compare responses to documented schemas.

6. Swagger UI Not Available in Production

Problem: `/swagger` returns 404 in production.

Cause: Swagger disabled when `TINA4_DEBUG=false`.

Fix: Set `TINA4_SWAGGER_ENABLED=true` in `.env` for staging servers. Be aware that public documentation reveals implementation details.

7. SDK Generation Produces Incorrect Types

Problem: Generated TypeScript client has `any` types.

Cause: Annotations use generic strings instead of typed schemas.

Fix: Use correct OpenAPI type format: `"name": "string"` (lowercase), `"price": "number"`, `"in_stock": "boolean"`, `"tags": ["string"]` (array of strings).

API Client

1. Calling External APIs Without Dependencies

Every app calls external services. Weather APIs. Payment gateways. CRM systems. Shipping providers. The default PHP approach is cURL: verbose, error-prone, and full of boilerplate.

Tina4 provides a built-in `Api` class that wraps cURL with a clean interface. It supports GET, POST, PUT, DELETE, and PATCH with custom headers, basic auth, JSON or form payloads, and timeout control, with no Guzzle and no Composer dependencies.

---

2. Creating an API Client

```
<?php
use Tina4\Api;

// Base URL -- all requests are relative to this
$api = new Api('https://api.example.com');
```

The `Api` class stores the base URL and shared configuration. Individual requests specify the path.

---

3. sendRequest()

All requests go through `sendRequest()`. It returns an associative array with:

- `status` - HTTP status code (integer)
- `body` - parsed response body (array if JSON, string otherwise)
- `headers` - response headers
- `error` - error message string on failure, `null` on success

```
<?php
use Tina4\Api;

$api = new Api('https://jsonplaceholder.typicode.com');

// GET request
$result = $api->sendRequest('GET', '/posts/1');

echo $result['status'];        // 200
print_r($result['body']);      // ['id' => 1, 'title' => '...', ...]
```

---

4. GET Requests

Fetch a resource:

```
<?php
```

```php
use Tina4\Api;

$api = new Api('https://jsonplaceholder.typicode.com');

$result = $api->sendRequest('GET', '/users/1');

if ($result['status'] === 200) {
    $user = $result['body'];
    echo "Name: {$user['name']}\n";
    echo "Email: {$user['email']}\n";
} else {
    echo "Error {$result['status']}: " . ($result['error'] ?? 'Unknown error');
}
```

---

5. POST, PUT, PATCH, DELETE

Pass the payload as the fourth argument to `sendRequest()`.

POST - Create a resource

```php
<?php
use Tina4\Api;

$api = new Api('https://jsonplaceholder.typicode.com');

$result = $api->sendRequest('POST', '/posts', [], [
    'title'  => 'My New Post',
    'body'   => 'This is the content of the post.',
    'userId' => 1
]);

echo $result['status'];              // 201
echo $result['body']['id'];          // 101 (new resource ID)
```

PUT - Full update

```php
$result = $api->sendRequest('PUT', '/posts/1', [], [
    'id'     => 1,
    'title'  => 'Updated Title',
    'body'   => 'Updated content.',
    'userId' => 1
]);

echo $result['status'];   // 200
```

PATCH - Partial update

```php
$result = $api->sendRequest('PATCH', '/posts/1', [], [
    'title' => 'Just the title changed'
]);

echo $result['status'];   // 200
```

DELETE - Remove a resource

```php
$result = $api->sendRequest('DELETE', '/posts/1');

echo $result['status'];   // 200 or 204
```

---

6. Query Parameters

Pass query parameters as the third argument:

```
<?php
use Tina4\Api;

$api = new Api('https://jsonplaceholder.typicode.com');

$result = $api->sendRequest('GET', '/posts', [
    'userId' => 1,
    '_limit' => 5,
    '_sort'  => 'id',
    '_order' => 'desc'
]);

// Resolves to: GET /posts?userId=1&_limit=5&_sort=id&_order=desc

echo count($result['body']);   // 5
```

---

7. Custom Headers

Use `addCustomHeaders()` to set headers that apply to all subsequent requests from this client instance.

```
<?php
use Tina4\Api;

$api = new Api('https://api.stripe.com/v1');

$api->addCustomHeaders([
    'Authorization' => 'Bearer sk_test_YOUR_STRIPE_KEY_HERE',
    'Stripe-Version' => '2023-10-16',
    'Idempotency-Key' => bin2hex(random_bytes(16))
]);

$result = $api->sendRequest('POST', '/payment_intents', [], [
    'amount'   => 2000,   // $20.00 in cents
    'currency' => 'usd',
    'payment_method_types' => ['card']
]);

echo $result['status'];          // 200
echo $result['body']['id'];      // pi_3OWL...
echo $result['body']['status'];  // requires_payment_method
```

---

8. Basic Authentication

Use `setUsernamePassword()` for HTTP Basic Auth:

```
<?php
```

```
use Tina4\Api;

$api = new Api('https://api.example.com');
$api->setUsernamePassword('my-api-key', 'my-api-secret');

$result = $api->sendRequest('GET', '/account');

echo $result['body']['plan'];   // 'enterprise'
```

The credentials are Base64-encoded and sent as an `Authorization: Basic ...` header.

---

9. Bearer Token Auth

Use a custom header for token-based auth (OAuth2, JWT):

```
<?php
use Tina4\Api;

// Step 1: Get an access token
$authApi = new Api('https://auth.example.com');
$tokenResult = $authApi->sendRequest('POST', '/oauth/token', [], [
    'grant_type'    => 'client_credentials',
    'client_id'     => getenv('CLIENT_ID'),
    'client_secret' => getenv('CLIENT_SECRET')
]);

$accessToken = $tokenResult['body']['access_token'];

// Step 2: Use the token for API calls
$api = new Api('https://api.example.com');
$api->addCustomHeaders([
    'Authorization' => "Bearer {$accessToken}",
    'Accept'        => 'application/json'
]);

$result = $api->sendRequest('GET', '/resources');
```

---

10. Error Handling

Always check `status` before accessing `body`. Handle network errors via `error`:

```
<?php
use Tina4\Api;

function fetchUser(int $id): ?array {
    $api = new Api('https://api.example.com');
    $result = $api->sendRequest('GET', "/users/{$id}");

    // Network or cURL failure
    if ($result['error'] !== null) {
        error_log("API network error: {$result['error']}");
        return null;
    }

    // HTTP error responses
```

```
    if ($result['status'] === 404) {
        return null;   // Not found -- expected, return null
    }

    if ($result['status'] === 429) {
        // Rate limited -- retry after delay
        sleep((int) ($result['headers']['Retry-After'] ?? 5));
        return fetchUser($id);  // Retry once
    }

    if ($result['status'] >= 500) {
        error_log("API server error {$result['status']} for user {$id}");
        throw new \RuntimeException("External API unavailable");
    }

    if ($result['status'] !== 200) {
        error_log("Unexpected status {$result['status']}");
        return null;
    }

    return $result['body'];
}
```

---

11. Calling External APIs from Routes

Wrap the `Api` client in route handlers to build aggregator or proxy endpoints:

```
<?php
use Tina4\Router;
use Tina4\Api;

/**
 * @noauth
 */
Router::get('/api/weather/{city}', function ($request, $response) {
    $city = $request->params['city'];
    $apiKey = getenv('OPENWEATHER_API_KEY');

    $api = new Api('https://api.openweathermap.org/data/2.5');

    $result = $api->sendRequest('GET', '/weather', [
        'q'     => $city,
        'appid' => $apiKey,
        'units' => 'metric'
    ]);

    if ($result['status'] === 404) {
        return $response->json(['error' => "City '{$city}' not found"], 404);
    }

    if ($result['status'] !== 200) {
        return $response->json(['error' => 'Weather service unavailable'], 502);
    }

    $weather = $result['body'];

    return $response->json([
```

```
        'city'        => $weather['name'],
        'country'     => $weather['sys']['country'],
        'temperature' => $weather['main']['temp'],
        'description' => $weather['weather'][0]['description'],
        'humidity'    => $weather['main']['humidity'],
        'wind_speed'  => $weather['wind']['speed']
    ]);
});
```

```
curl http://localhost:7145/api/weather/London
```

```
{
  "city": "London",
  "country": "GB",
  "temperature": 12.5,
  "description": "overcast clouds",
  "humidity": 81,
  "wind_speed": 5.2
}
```

---

12. Exercise: GitHub API Proxy

Build a proxy that fetches GitHub repository information.

Requirements

- Create these endpoints:

Method	Path	Description
`GET`	`/api/github/{owner}/{repo}`	Repo details (stars, forks, description)
`GET`	`/api/github/{owner}/{repo}/releases`	Latest 5 releases

- Use the GitHub public API at `https://api.github.com`
- Set the required `User-Agent` header (GitHub rejects requests without it)
- Handle 404 gracefully

Test with:

```
curl http://localhost:7145/api/github/tina4/tina4-python
curl http://localhost:7145/api/github/tina4/tina4-python/releases
curl http://localhost:7145/api/github/nobody/nonexistent-repo
```

---

13. Gotchas

1. No timeout set

Problem: A slow external API hangs your request for 30+ seconds.

Cause: cURL default timeout is 0 (infinite wait).

Fix: Pass a timeout in the options: `$api->sendRequest('GET', '/path', [], [], ['timeout' => 5])`. Always set a reasonable timeout for external calls.

2. Not checking status before accessing body

Problem: Code crashes with "Undefined index" when the body is an error message, not the expected structure.

Cause: The API returned a 4xx/5xx error. The body is an error object, not the expected resource.

Fix: Always check `$result['status']` before accessing `$result['body']`.

3. Credentials in source code

Problem: API keys and secrets are committed to the repository.

Cause: Credentials hard-coded in the `Api` instantiation call.

Fix: Always read credentials from environment variables: `getenv('TINA4_API_KEY')`. Never commit `.env` files.

4. Not handling rate limits

Problem: After many requests, the API returns 429 and all subsequent calls fail.

Cause: No rate limit detection or backoff logic.

Fix: Check for `status === 429`, read `Retry-After` from the response headers, and sleep before retrying.

GraphQL

1. The Problem GraphQL Solves

Your mobile app needs a list of products. Each product has a name, price, 20 image URLs, a full description, 15 review objects, and 8 other fields you do not need right now. REST sends all of it. 50KB of JSON when you needed 2KB. On a spotty mobile connection, that wasted bandwidth matters.

Now your web dashboard needs the same products plus the category, stock status, and supplier info. REST forces you into three requests (products, categories, suppliers) stitched together on the client, or a custom endpoint with query parameter parsing on the server.

GraphQL solves both problems. The client asks for the fields it needs. The server returns those fields. One endpoint. One request. One response with the right shape.

Tina4 includes a built-in GraphQL engine. No external packages. No Apollo Server. No `graphql-php` library. Part of the framework.

---

2. GraphQL vs REST -- A Quick Comparison

Aspect	REST	GraphQL
Endpoints	One per resource (`/products`, `/users`)	One endpoint (`/graphql`)
Data shape	Server decides	Client decides
Over-fetching	Common (get all fields)	Never (get only requested fields)
Under-fetching	Common (multiple requests needed)	Never (get related data in one query)
Versioning	`/api/v1/`, `/api/v2/`	Schema evolves, no versioning needed
Caching	HTTP caching works naturally	Requires client-side cache management
Learning curve	Low	Moderate

REST is still great for simple APIs. GraphQL shines when clients have diverse data needs -- mobile apps, dashboards, third-party integrations.

---

3. Enabling GraphQL

GraphQL is available by default. The engine serves requests at `/graphql`. Change the endpoint in `.env` if needed:

```
TINA4_GRAPHQL_ENDPOINT=/graphql
```

To verify it is working, start the server and send a test query:

```
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ __schema { queryType { name } } }"}'
```

```
{
  "data": {
    "__schema": {
      "queryType": {
        "name": "Query"
      }
    }
  }
}
```

If you see that response, GraphQL is running.

---

4. Defining a Schema

A GraphQL schema defines the data types your API can return and the operations clients can execute.

Create `src/graphql/schema.graphql`:

```
type Product {
    id: Int!
    name: String!
    category: String!
    price: Float!
    inStock: Boolean!
    createdAt: String
}

type Query {
    products: [Product!]!
    product(id: Int!): Product
}
```

This schema says:

- A `Product` has six fields. The `!` means the field is non-nullable.
- The `Query` type has two operations: `products` returns a list of products, and `product(id)` returns a single product (or null if not found).

Tina4 auto-discovers `.graphql` files in `src/graphql/`. You do not need to register them.

---

5. Writing Resolvers

A resolver is the function that runs when a query or mutation executes. Resolvers live in `src/graphql/` as PHP files.

Create `src/graphql/resolvers.php`:

```
<?php
use Tina4\GraphQL;

// Resolve the "products" query
GraphQL::resolve("Query", "products", function ($root, $args) {
    $product = new Product();
    $products = $product->select("*", "", [], "name ASC");

    return array_map(fn($p) => $p->toArray(), $products);
});

// Resolve the "product" query
GraphQL::resolve("Query", "product", function ($root, $args) {
    $product = new Product();
    $product->load($args["id"]);

    if (empty($product->id)) {
        return null;
    }

    return $product->toArray();
});
```

The `GraphQL::resolve()` method takes three arguments:

- **Type name** -- The GraphQL type this resolver belongs to (`Query`, `Mutation`, or a custom type).
- **Field name** -- The field within the type this resolver handles.
- **Resolver function** -- Receives `$root` (the parent value, if any) and `$args` (the query arguments).

Testing the Queries

List all products:

```
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ products { id name price inStock } }"}'
```

```
{
  "data": {
    "products": [
      {"id": 1, "name": "Wireless Keyboard", "price": 79.99, "inStock": true},
      {"id": 2, "name": "USB-C Hub", "price": 49.99, "inStock": true},
      {"id": 3, "name": "Standing Desk", "price": 549.99, "inStock": false}
    ]
  }
}
```

Notice: the response only contains the four fields we asked for (`id`, `name`, `price`, `inStock`), not `category` or `createdAt`. That is GraphQL in action.

Get a single product with all fields:

```
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ product(id: 1) { id name category price inStock createdAt } }"}'
```

```
{
  "data": {
    "product": {
      "id": 1,
      "name": "Wireless Keyboard",
      "category": "Electronics",
      "price": 79.99,
      "inStock": true,
      "createdAt": "2026-03-22 14:30:00"
    }
  }
}
```

Request a product that does not exist:

```
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ product(id: 999) { id name } }"}'
```

```
{
  "data": {
    "product": null
  }
}
```

---

6. Mutations

Mutations create, update, or delete data. They live in the schema alongside queries.

Update `src/graphql/schema.graphql`:

```
type Product {
    id: Int!
    name: String!
    category: String!
    price: Float!
    inStock: Boolean!
    createdAt: String
}

input ProductInput {
    name: String!
    category: String
    price: Float!
    inStock: Boolean
}

input ProductUpdateInput {
    name: String
```

```
    category: String
    price: Float
    inStock: Boolean
}

type DeleteResult {
    success: Boolean!
    message: String!
}

type Query {
    products: [Product!]!
    product(id: Int!): Product
    productsByCategory(category: String!): [Product!]!
}

type Mutation {
    createProduct(input: ProductInput!): Product!
    updateProduct(id: Int!, input: ProductUpdateInput!): Product
    deleteProduct(id: Int!): DeleteResult!
}
```

Key concepts:

- `input` **types** define the shape of data the client sends. They are like regular types but used for arguments.
- `Mutation` **type** lists all write operations. By convention, mutations are named with verbs: `createProduct`, `updateProduct`, `deleteProduct`.
- Fields in `ProductUpdateInput` are all optional (no `!`) because the client may only want to update one field.

Mutation Resolvers

Add to `src/graphql/resolvers.php`:

```
// Create a product
GraphQL::resolve("Mutation", "createProduct", function ($root, $args) {
    $input = $args["input"];

    $product = new Product();
    $product->name = $input["name"];
    $product->category = $input["category"] ?? "Uncategorized";
    $product->price = (float) $input["price"];
    $product->inStock = (bool) ($input["inStock"] ?? true);
    $product->save();

    return $product->toArray();
});

// Update a product
GraphQL::resolve("Mutation", "updateProduct", function ($root, $args) {
    $product = new Product();
    $product->load($args["id"]);

    if (empty($product->id)) {
        return null;
    }
```

```
    $input = $args["input"];
    if (isset($input["name"])) $product->name = $input["name"];
    if (isset($input["category"])) $product->category = $input["category"];
    if (isset($input["price"])) $product->price = (float) $input["price"];
    if (isset($input["inStock"])) $product->inStock = (bool) $input["inStock"];
    $product->save();

    return $product->toArray();
});

// Delete a product
GraphQL::resolve("Mutation", "deleteProduct", function ($root, $args) {
    $product = new Product();
    $product->load($args["id"]);

    if (empty($product->id)) {
        return ["success" => false, "message" => "Product not found"];
    }

    $product->delete();
    return ["success" => true, "message" => "Product deleted"];
});

// Query: products by category
GraphQL::resolve("Query", "productsByCategory", function ($root, $args) {
    $product = new Product();
    $products = $product->select("*", "category = :category", [
        "category" => $args["category"]
    ]);

    return array_map(fn($p) => $p->toArray(), $products);
});
```

Testing Mutations

Create a product:

```
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{
    "query": "mutation { createProduct(input: { name: \"Desk Lamp\", category: \"Office\", price
  }'
```

```
{
  "data": {
    "createProduct": {
      "id": 4,
      "name": "Desk Lamp",
      "price": 39.99
    }
  }
}
```

Update a product:

```
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{
    "query": "mutation { updateProduct(id: 4, input: { price: 44.99, inStock: false }) { id name
  }'
```

```
{
  "data": {
    "updateProduct": {
      "id": 4,
      "name": "Desk Lamp",
      "price": 44.99,
      "inStock": false
    }
  }
}
```

Delete a product:

```
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{
    "query": "mutation { deleteProduct(id: 4) { success message } }"
  }'
```

```
{
  "data": {
    "deleteProduct": {
      "success": true,
      "message": "Product deleted"
    }
  }
}
```

---

7. Nested Types and Relationships

GraphQL's real power: traversing relationships in a single query. Authors and comments in a blog schema.

Update `src/graphql/schema.graphql`:

```
type User {
    id: Int!
    name: String!
    email: String!
    posts: [Post!]!
}

type Post {
    id: Int!
    title: String!
    body: String!
    published: Boolean!
    author: User!
    comments: [Comment!]!
    commentCount: Int!
}

type Comment {
    id: Int!
    authorName: String!
    body: String!
    post: Post!
```

```
}

type Product {
    id: Int!
    name: String!
    category: String!
    price: Float!
    inStock: Boolean!
    createdAt: String
}

input ProductInput {
    name: String!
    category: String
    price: Float!
    inStock: Boolean
}

type DeleteResult {
    success: Boolean!
    message: String!
}

type Query {
    products: [Product!]!
    product(id: Int!): Product
    productsByCategory(category: String!): [Product!]!
    posts: [Post!]!
    post(id: Int!): Post
    user(id: Int!): User
}

type Mutation {
    createProduct(input: ProductInput!): Product!
    deleteProduct(id: Int!): DeleteResult!
}
```

Add resolvers for the nested types:

```
// Posts query
GraphQL::resolve("Query", "posts", function ($root, $args) {
    $post = new Post();
    $posts = $post->select("*", "published = :pub", ["pub" => 1], "created_at DESC");
    return array_map(fn($p) => $p->toArray(), $posts);
});

// Single post query
GraphQL::resolve("Query", "post", function ($root, $args) {
    $post = new Post();
    $post->load($args["id"]);
    return empty($post->id) ? null : $post->toArray();
});

// Resolve Post.author (nested field)
GraphQL::resolve("Post", "author", function ($post, $args) {
    $user = new User();
    $user->load($post["user_id"]);
    return $user->toArray();
});
```

```
// Resolve Post.comments (nested field)
GraphQL::resolve("Post", "comments", function ($post, $args) {
    $comment = new Comment();
    $comments = $comment->select("*", "post_id = :postId", ["postId" => $post["id"]]);
    return array_map(fn($c) => $c->toArray(), $comments);
});

// Resolve Post.commentCount (computed field)
GraphQL::resolve("Post", "commentCount", function ($post, $args) {
    $comment = new Comment();
    $comments = $comment->select("count(*) as cnt", "post_id = :postId", ["postId" => $post["id"
    return $comments[0]->cnt ?? 0;
});

// Resolve User.posts (nested field)
GraphQL::resolve("User", "posts", function ($user, $args) {
    $post = new Post();
    $posts = $post->select("*", "user_id = :userId", ["userId" => $user["id"]]);
    return array_map(fn($p) => $p->toArray(), $posts);
});
```

Now clients can query across relationships in a single request:

```
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{
    "query": "{ posts { id title author { name email } comments { authorName body } commentCount
  }'
```

```
{
  "data": {
    "posts": [
      {
        "id": 1,
        "title": "My First Post",
        "author": {
          "name": "Alice",
          "email": "alice@example.com"
        },
        "comments": [
          {"authorName": "Bob", "body": "Great post!"}
        ],
        "commentCount": 1
      }
    ]
  }
}
```

One request. The fields the client asked for. Nothing more. Nothing less.

---

8. Auto-Generating Schema from ORM Models

If your ORM models have `$autoCrud = true`, Tina4 can auto-generate GraphQL types and basic CRUD resolvers for them. Enable this in `.env`:

```
TINA4_GRAPHQL_AUTO_SCHEMA=true
```

With this setting, every ORM model with `$autoCrud = true` automatically gets:

- A GraphQL type with fields matching the model properties
- A query to list all records (e.g., `products`)
- A query to get one by ID (e.g., `product(id: Int!)`)
- A mutation to create (e.g., `createProduct(input: ProductInput!)`)
- A mutation to update (e.g., `updateProduct(id: Int!, input: ProductUpdateInput!)`)
- A mutation to delete (e.g., `deleteProduct(id: Int!)`)

You can still define custom resolvers that override the auto-generated ones. Custom resolvers always take precedence.

---

9. The GraphiQL Playground

When `TINA4_DEBUG=true`, Tina4 serves a GraphiQL interactive playground at:

```
http://localhost:7145/graphql/playground
```

GraphiQL gives you:

- A query editor with syntax highlighting and auto-completion
- A documentation explorer showing all types, queries, and mutations
- A results panel showing the response
- Query history

This is the fastest way to explore and test your GraphQL API during development. Type a query on the left, click the play button, and see the results on the right. The documentation explorer on the right sidebar shows every type, field, and argument available in your schema.

GraphiQL is only available when `TINA4_DEBUG=true`. In production, it is disabled automatically.

---

10. Query Variables

For production, clients send query variables separately from the query string. This prevents injection and allows query caching.

```
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{
    "query": "query GetProduct($id: Int!) { product(id: $id) { id name price } }",
    "variables": {"id": 1}
  }'
```

```
{
  "data": {
    "product": {
      "id": 1,
```

```
      "name": "Wireless Keyboard",
      "price": 79.99
    }
  }
}
```

The query uses `$id` as a placeholder, and the actual value comes from the `variables` object. This is safer and more efficient than string interpolation.

---

11. Exercise: Build a GraphQL API for a Blog

Build a complete GraphQL API for a blog with posts, authors, and comments.

Requirements

- **Schema** -- Define types for `User`, `Post`, and `Comment` with the following fields:
- `User`: id, name, email, posts (nested)
- `Post`: id, title, body, published, author (nested), comments (nested), commentCount (computed)
- `Comment`: id, authorName, body, createdAt
- **Queries**:
- `posts` -- List all published posts
- `post(id: Int!)` -- Get a single post by ID
- `user(id: Int!)` -- Get a user with their posts
- **Mutations**:
- `createPost(userId: Int!, title: String!, body: String!, published: Boolean)` -- Create a new post
- `addComment(postId: Int!, authorName: String!, body: String!)` -- Add a comment to a post
- Use the ORM models from Chapter 6 (User, Post, Comment).

Test with these queries:

```
# List published posts with author names
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ posts { id title author { name } commentCount } }"}'

# Get a specific post with all comments
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ post(id: 1) { title body author { name email } comments { authorName body } }

# Create a post
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "mutation { createPost(userId: 1, title: \"GraphQL is great\", body: \"Here is w

# Add a comment
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "mutation { addComment(postId: 1, authorName: \"Carol\", body: \"Nice article!\"

# Get a user with all their posts
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ user(id: 1) { name email posts { title published } } }"}'
```

---

12. Solution

Schema

Create `src/graphql/blog-schema.graphql`:

```
type User {
    id: Int!
    name: String!
    email: String!
    posts: [Post!]!
}

type Post {
    id: Int!
    title: String!
    body: String!
    published: Boolean!
    author: User!
    comments: [Comment!]!
    commentCount: Int!
    createdAt: String
}

type Comment {
    id: Int!
    authorName: String!
    body: String!
    createdAt: String
}
```

```
type Query {
    posts: [Post!]!
    post(id: Int!): Post
    user(id: Int!): User
}

type Mutation {
    createPost(userId: Int!, title: String!, body: String!, published: Boolean): Post!
    addComment(postId: Int!, authorName: String!, body: String!): Comment!
}
```

Resolvers

Create `src/graphql/blog-resolvers.php`:

```
<?php
use Tina4\GraphQL;

// Query: list published posts
GraphQL::resolve("Query", "posts", function ($root, $args) {
    $post = new Post();
    $posts = $post->select("*", "published = :pub", ["pub" => 1], "created_at DESC");
    return array_map(fn($p) => $p->toArray(), $posts);
});

// Query: single post
GraphQL::resolve("Query", "post", function ($root, $args) {
    $post = new Post();
    $post->load($args["id"]);
    return empty($post->id) ? null : $post->toArray();
});

// Query: single user
GraphQL::resolve("Query", "user", function ($root, $args) {
    $user = new User();
    $user->load($args["id"]);
    return empty($user->id) ? null : $user->toArray();
});

// Post.author resolver
GraphQL::resolve("Post", "author", function ($post, $args) {
    $user = new User();
    $user->load($post["user_id"]);
    return $user->toArray();
});

// Post.comments resolver
GraphQL::resolve("Post", "comments", function ($post, $args) {
    $comment = new Comment();
    $comments = $comment->select("*", "post_id = :postId", [
        "postId" => $post["id"]
    ], "created_at ASC");
    return array_map(fn($c) => $c->toArray(), $comments);
});

// Post.commentCount resolver
GraphQL::resolve("Post", "commentCount", function ($post, $args) {
    $comment = new Comment();
    $results = $comment->select("count(*) as cnt", "post_id = :postId", [
```

```
        "postId" => $post["id"]
    ]);
    return (int) ($results[0]->cnt ?? 0);
});

// User.posts resolver
GraphQL::resolve("User", "posts", function ($user, $args) {
    $post = new Post();
    $posts = $post->select("*", "user_id = :userId", [
        "userId" => $user["id"]
    ], "created_at DESC");
    return array_map(fn($p) => $p->toArray(), $posts);
});

// Mutation: create a post
GraphQL::resolve("Mutation", "createPost", function ($root, $args) {
    $user = new User();
    $user->load($args["userId"]);

    if (empty($user->id)) {
        throw new \Exception("User not found");
    }

    $post = new Post();
    $post->userId = $args["userId"];
    $post->title = $args["title"];
    $post->body = $args["body"];
    $post->published = (bool) ($args["published"] ?? false);
    $post->save();

    return $post->toArray();
});

// Mutation: add a comment
GraphQL::resolve("Mutation", "addComment", function ($root, $args) {
    $post = new Post();
    $post->load($args["postId"]);

    if (empty($post->id)) {
        throw new \Exception("Post not found");
    }

    $comment = new Comment();
    $comment->postId = $args["postId"];
    $comment->authorName = $args["authorName"];
    $comment->body = $args["body"];
    $comment->save();

    return $comment->toArray();
});
```

Expected output for `{ posts { id title author { name } commentCount } }`:

```
{
  "data": {
    "posts": [
      {
        "id": 1,
        "title": "My First Post",
```

```
        "author": {"name": "Alice"},
        "commentCount": 1
      },
      {
        "id": 2,
        "title": "GraphQL is great",
        "author": {"name": "Alice"},
        "commentCount": 0
      }
    ]
  }
}
```

---

13. Input Validation

Tina4's GraphQL engine validates every argument against its declared type before your resolver runs. You never need to check whether a required argument is present or whether a string arrived where an integer was expected -- the engine rejects the query before your code executes.

The built-in validator covers:

- **Non-null enforcement** -- Arguments marked with `!` must be present and non-null.
- **Scalar type checking** -- `Int`, `Float`, `String`, `Boolean`, and `ID` values are verified against their declared type.
- **Type coercion** -- When safe, the engine coerces compatible values automatically (e.g., an integer `42` passed to a `Float` field becomes `42.0`, or a string `"7"` passed to an `Int` field becomes `7`).
- **List item validation** -- For `[Int!]!` arguments, the engine checks that the value is a list, that no item is null, and that every item is an integer.

Example: Missing Required Argument

Suppose you define a query that requires an `id`:

```
<?php
use Tina4\GraphQL;

GraphQL::resolve("Query", "user", function ($root, $args) {
    $user = new User();
    $user->load($args["id"]);
    return empty($user->id) ? null : $user->toArray();
});
```

With the schema declaring `user(id: ID!): User`, sending a query without the required argument:

```
{
  user {
    name
    email
  }
}
```

Returns an error before the resolver runs:

```
{
  "errors": [
    {
      "message": "Argument 'id' of type 'ID!' is required but not provided.",
      "locations": [{"line": 2, "column": 3}]
    }
  ]
}
```

Your resolver never executes. No null-check needed inside the function.

---

14. Field-Level Auth Directives

Tina4 supports three directives that control field-level access in your GraphQL schema:

Directive	Effect
`@auth`	Field resolves only when `ctx["user"]` is present (any authenticated user)
`@role(role: "admin")`	Field resolves only when `ctx["user"]["role"]` matches the specified role
`@guest`	Field resolves only when `ctx["user"]` is absent (unauthenticated visitors)

Passing Auth Context

Pass user information through the context parameter of `execute()`:

```
<?php
$result = $gql->execute($query, $variables, [
    'user' => ['id' => 1, 'role' => 'admin']
]);
```

When no user is logged in, pass an empty context or omit the `user` key:

```
<?php
$result = $gql->execute($query, $variables, []);
```

Schema Example

```
type User {
    id: ID!
    name: String!
    email: String! @auth
    role: String! @role(role: "admin")
}

type Query {
    me: User @auth
    publicStats: Stats @guest
    users: [User!]! @role(role: "admin")
}
```

Behavior on Failed Auth

When a directive check fails, the field resolves to `null` silently -- no error is added to the response. The rest of the query executes normally. This prevents leaking information about which fields exist behind authentication.

For example, if a non-admin user queries:

```
{
  users {
    name
    email
    role
  }
}
```

The response is:

```
{
  "data": {
    "users": null
  }
}
```

No error message. No hint that the field requires admin access. The field is simply excluded.

---

15. Gotchas

1. Schema File Not Found

Problem: Queries return errors about unknown types or fields.

Cause: The `.graphql` file is not in `src/graphql/`, or has a syntax error that prevents parsing.

Fix: Make sure schema files are in `src/graphql/` and end with `.graphql`. Check for syntax errors -- a missing `!` or unmatched brace will cause the entire schema to fail silently.

2. Resolver Not Called

Problem: A query returns `null` even though data exists.

Cause: The resolver is not registered, or the type/field names do not match the schema exactly. GraphQL is case-sensitive.

Fix: Verify that `GraphQL::resolve("Query", "products", ...)` matches `type Query { products: ... }` exactly. Check capitalization.

3. Nested Resolver Returns Wrong Data

Problem: `post.author` returns the entire post instead of the user.

Cause: The nested resolver receives the parent object as its first argument (`$post`), not a fresh model. If you return the parent instead of loading the related record, you get the wrong data.

Fix: In a nested resolver, always load the related record from the database using the foreign key from the parent:

```
GraphQL::resolve("Post", "author", function ($post, $args) {
    $user = new User();
    $user->load($post["user_id"]); // Use the foreign key from the parent
    return $user->toArray();
});
```

4. Mutation Input Not Parsed

Problem: `$args["input"]` is null inside a mutation resolver.

Cause: The mutation signature in the schema uses an `input` type, but the client query passes arguments directly instead of wrapping them in an `input` object.

Fix: Match the query to the schema. If the schema says `createProduct(input: ProductInput!)`, the client must send `createProduct(input: { name: "Widget", price: 9.99 })`, not `createProduct(name: "Widget", price: 9.99)`.

5. N+1 Query Problem

Problem: Loading 50 posts with their authors generates 51 database queries (1 for posts + 50 for authors).

Cause: Each nested resolver runs independently. When you resolve `author` for each post, that is one query per post.

Fix: Use data loader patterns or batch loading. For simple cases, you can pre-load all related records in the parent resolver and pass them down. Check `$ctx["__selections"]` to see which nested fields the client requested -- if `author` is not in the selection set, skip the join. For ORM queries, use the `include` parameter to eager-load relationships in a single query. For complex cases, consider using the REST API with eager loading instead of GraphQL for that particular endpoint.

6. GraphQL Playground Returns 404

Problem: `/graphql/playground` returns a 404 error.

Cause: `TINA4_DEBUG` is set to `false`. The playground is only available in debug mode.

Fix: Set `TINA4_DEBUG=true` in your `.env` file. The playground is intentionally disabled in production.

7. Type Mismatch Between Schema and Resolver

Problem: A field declared as `Int!` in the schema receives a string from the resolver, causing a type error.

Cause: PHP does not enforce types as strictly as GraphQL. If your resolver returns `"42"` (a string) for an `Int!` field, GraphQL rejects it.

Fix: The input validation layer (see section 13) now coerces compatible types automatically -- a string `"42"` passed as an argument to an `Int` parameter is converted before your resolver runs. However, resolver *return values* still need correct types. Cast values explicitly with `(int)`, `(float)`, `(bool)`:

```
return [
    "id" => (int) $product->id,
    "price" => (float) $product->price,
    "inStock" => (bool) $product->inStock
];
```

The `toArray()` method handles this automatically for model properties with type declarations, but computed or derived fields need manual casting.

Real-time with WebSocket

1. The Refresh Button Problem

Your project management app needs live updates. Someone moves a card from "In Progress" to "Done." Everyone else should see it. No page refresh. No polling. No waiting.

HTTP is a conversation with amnesia. The client asks, the server answers, the connection dies. The server cannot reach back.

WebSocket fixes this. It opens a persistent, two-way channel between browser and server. Either side sends messages at any time. The connection holds until someone closes it.

Tina4 treats WebSocket the same as routing. Define a handler. Assign a path. Done.

---

2. What WebSocket Is

HTTP works this way:

```
Client: "Give me /api/products"
Server: "Here are the products" (connection closes)
Client: "Give me /api/products" (new connection)
Server: "Here are the products" (connection closes)
```

WebSocket works this way:

```
Client: "I want to upgrade to WebSocket on /ws/chat"
Server: "Upgrade accepted. Connection open."
Client: "Hello everyone!"
Server: "Alice says: Hello everyone!"  (pushed to all clients)
Server: "Bob joined the chat"  (pushed to all clients, at any time)
Client: "Goodbye!"
Server: "Alice says: Goodbye!"
Client: (closes connection)
```

Four differences matter:

- **Persistent**: The connection stays open. No repeated handshakes.
- **Bi-directional**: The server pushes data without the client asking.
- **Low overhead**: After the initial handshake, messages are tiny. No HTTP headers per message.
- **Real-time**: Messages arrive within milliseconds.

---

3. Router::websocket() -- WebSocket as a Route

Define WebSocket handlers with `Router::websocket()`:

```
<?php
use Tina4\Router;
```

```
Router::websocket("/ws/echo", function ($connection, $event, $data) {
    if ($event === "message") {
        $connection->send("Echo: " . $data);
    }
});
```

The simplest handler. It receives a message and sends it back with "Echo: " prepended. Three arguments arrive:

- **$connection**: The WebSocket connection object. Send messages through it.
- **$event**: The event type: `"open"`, `"message"`, or `"close"`.
- **$data**: The message data. Present only for `"message"` events.

Starting the Server

WebSocket runs alongside your HTTP server:

```
tina4 serve
```

```
  Tina4 PHP v3.0.0
  HTTP server running at http://0.0.0.0:7145
  WebSocket server running at ws://0.0.0.0:7145
  Press Ctrl+C to stop
```

Both protocols share the same port. Tina4's built-in server uses PHP's `stream_select()` to multiplex HTTP and WebSocket connections in a single process -- no extensions like Swoole or ReactPHP required. The server detects the upgrade request and routes to the correct handler.

---

4. Connection Events

Every WebSocket connection passes through three lifecycle events.

Open

Fires when a client connects:

```
<?php
use Tina4\Router;

Router::websocket("/ws/notifications", function ($connection, $event, $data) {
    if ($event === "open") {
        error_log("Client connected: " . $connection->id);
        $connection->send(json_encode([
            "type" => "welcome",
            "message" => "Connected to notifications",
            "connection_id" => $connection->id
        ]));
    }
});
```

Message

Fires when a client sends data:

```
Router::websocket("/ws/notifications", function ($connection, $event, $data) {
    if ($event === "open") {
```

```
        $connection->send(json_encode([
            "type" => "welcome",
            "connection_id" => $connection->id
        ]));
    }

    if ($event === "message") {
        $message = json_decode($data, true);
        error_log("Received from " . $connection->id . ": " . $data);

        // Process the message based on its type
        if (($message["type"] ?? "") === "ping") {
            $connection->send(json_encode([
                "type" => "pong",
                "timestamp" => date("c")
            ]));
        }
    }
});
```

Close

Fires when a client disconnects:

```
Router::websocket("/ws/notifications", function ($connection, $event, $data) {
    if ($event === "open") {
        error_log("Client connected: " . $connection->id);
    }

    if ($event === "message") {
        error_log("Message from " . $connection->id . ": " . $data);
    }

    if ($event === "close") {
        error_log("Client disconnected: " . $connection->id);
        // Clean up: remove from tracking, notify others, etc.
    }
});
```

A Complete Handler

All three events in one handler:

```
<?php
use Tina4\Router;

Router::websocket("/ws/chat", function ($connection, $event, $data) {
    switch ($event) {
        case "open":
            error_log("[Chat] New connection: " . $connection->id);
            $connection->send(json_encode([
                "type" => "system",
                "message" => "Welcome to the chat!",
                "your_id" => $connection->id
            ]));
            break;

        case "message":
            $message = json_decode($data, true);
            error_log("[Chat] " . $connection->id . ": " . ($message["text"] ?? $data));
```

```
            // Echo back with sender info
            $connection->send(json_encode([
                "type" => "message",
                "from" => $connection->id,
                "text" => $message["text"] ?? $data,
                "timestamp" => date("c")
            ]));
            break;

        case "close":
            error_log("[Chat] Disconnected: " . $connection->id);
            break;
    }
});
```

---

5. Sending to a Single Client

`$connection->send()` targets the specific client that triggered the event:

```
Router::websocket("/ws/private", function ($connection, $event, $data) {
    if ($event === "message") {
        $message = json_decode($data, true);
        $action = $message["action"] ?? "";

        if ($action === "get-time") {
            $connection->send(json_encode([
                "type" => "time",
                "server_time" => date("c"),
                "timezone" => date_default_timezone_get()
            ]));
        }

        if ($action === "get-status") {
            $connection->send(json_encode([
                "type" => "status",
                "uptime" => 3600,
                "connections" => 42,
                "memory_mb" => round(memory_get_usage() / 1024 / 1024, 2)
            ]));
        }
    }
});
```

Only the sender receives the response. Other connected clients see nothing.

Closing a Connection

`$connection->close()` terminates the connection from the server side:

```
Router::websocket("/ws/secure", function ($connection, $event, $data) {
    if ($event === "open") {
        // Reject unauthenticated connections
        $token = $connection->params["token"] ?? "";
        if (!Auth::validToken($token)) {
            $connection->send(json_encode([
                "type" => "error",
                "message" => "Invalid token"
```

```
            ]));
            $connection->close();
            return;
        }
    }
});
```

The client receives the close event. Use this for kicking users, enforcing authentication, or cleaning up idle connections.

---

6. Broadcasting to All Clients

`$connection->broadcast()` sends a message to every client connected to the same WebSocket path:

```
<?php
use Tina4\Router;

Router::websocket("/ws/announcements", function ($connection, $event, $data) {
    if ($event === "open") {
        // Tell everyone about the new connection
        $connection->broadcast(json_encode([
            "type" => "system",
            "message" => "A new user joined",
            "online_count" => $connection->connectionCount()
        ]));
    }

    if ($event === "message") {
        $message = json_decode($data, true);

        // Broadcast the message to everyone (including sender)
        $connection->broadcast(json_encode([
            "type" => "announcement",
            "from" => $connection->id,
            "text" => $message["text"] ?? "",
            "timestamp" => date("c")
        ]));
    }

    if ($event === "close") {
        $connection->broadcast(json_encode([
            "type" => "system",
            "message" => "A user left",
            "online_count" => $connection->connectionCount()
        ]));
    }
});
```

One client sends a message. Every client connected to `/ws/announcements` receives it. Chat rooms, live dashboards, collaborative editing -- they all start here.

Broadcast Excluding Sender

Send to everyone except the client that triggered the event:

```
if ($event === "message") {
    $message = json_decode($data, true);

    // Send to sender (confirmation)
    $connection->send(json_encode([
        "type" => "sent",
        "text" => $message["text"]
    ]));

    // Send to everyone else
    $connection->broadcast(json_encode([
        "type" => "message",
        "from" => $message["username"] ?? "Anonymous",
        "text" => $message["text"],
        "timestamp" => date("c")
    ]), true); // true = exclude sender
}
```

The second argument to `broadcast()` excludes the sender when set to `true`. The sender gets a "sent" confirmation. Everyone else gets the "message."

---

7. Path-Scoped Isolation

Different WebSocket paths are walls. Clients connected to `/ws/chat/room-1` never see messages from `/ws/chat/room-2`:

```
<?php
use Tina4\Router;

Router::websocket("/ws/chat/{room}", function ($connection, $event, $data) {
    $room = $connection->params["room"];

    if ($event === "open") {
        error_log("[Room " . $room . "] New connection: " . $connection->id);
        $connection->broadcast(json_encode([
            "type" => "system",
            "message" => "Someone joined room " . $room,
            "room" => $room,
            "online" => $connection->connectionCount()
        ]));
    }

    if ($event === "message") {
        $message = json_decode($data, true);

        $connection->broadcast(json_encode([
            "type" => "message",
            "room" => $room,
            "from" => $message["username"] ?? "Anonymous",
            "text" => $message["text"] ?? "",
            "timestamp" => date("c")
        ]));
    }

    if ($event === "close") {
        $connection->broadcast(json_encode([
```

```
            "type" => "system",
            "message" => "Someone left room " . $room,
            "room" => $room,
            "online" => $connection->connectionCount()
        ]));
    }
});
```

Connect to different rooms:

```
ws://localhost:7145/ws/chat/general    -- general chat
ws://localhost:7145/ws/chat/random     -- random chat
ws://localhost:7145/ws/chat/dev-team   -- dev team chat
```

Broadcasting in `/ws/chat/general` reaches only clients in `/ws/chat/general`. The `dev-team` and `random` rooms are separate worlds.

Chat rooms. Project-specific notifications. Per-user channels. No extra configuration. The URL path is the isolation boundary.

---

8. Building a Live Chat

A complete chat application with usernames, typing indicators, and message history.

WebSocket Handler

Create `src/routes/chat-ws.php`:

```
<?php
use Tina4\Router;

$chatUsers = [];

Router::websocket("/ws/livechat/{room}", function ($connection, $event, $data) use (&$chatUsers)
    $room = $connection->params["room"];

    if ($event === "open") {
        // User has not set their name yet
        $chatUsers[$connection->id] = [
            "id" => $connection->id,
            "username" => "Anonymous",
            "room" => $room,
            "joined_at" => date("c")
        ];

        $connection->send(json_encode([
            "type" => "welcome",
            "message" => "Connected to room: " . $room,
            "your_id" => $connection->id,
            "online" => $connection->connectionCount()
        ]));
    }

    if ($event === "message") {
        $message = json_decode($data, true);
        $type = $message["type"] ?? "message";
```

```
        if ($type === "set-username") {
            $oldName = $chatUsers[$connection->id]["username"];
            $chatUsers[$connection->id]["username"] = $message["username"];

            $connection->broadcast(json_encode([
                "type" => "system",
                "message" => $oldName . " is now known as " . $message["username"]
            ]));
        }

        if ($type === "message") {
            $username = $chatUsers[$connection->id]["username"];

            $connection->broadcast(json_encode([
                "type" => "message",
                "from" => $username,
                "from_id" => $connection->id,
                "text" => $message["text"] ?? "",
                "timestamp" => date("c")
            ]));
        }

        if ($type === "typing") {
            $username = $chatUsers[$connection->id]["username"];

            $connection->broadcast(json_encode([
                "type" => "typing",
                "username" => $username
            ]), true); // Exclude sender
        }
    }

    if ($event === "close") {
        $username = $chatUsers[$connection->id]["username"] ?? "Unknown";
        unset($chatUsers[$connection->id]);

        $connection->broadcast(json_encode([
            "type" => "system",
            "message" => $username . " left the chat",
            "online" => $connection->connectionCount()
        ]));
    }
});
```

---

9. Live Notifications

WebSocket pushes notifications to users in real time:

```
<?php
use Tina4\Router;
use Tina4\Queue;

// WebSocket handler for notifications
Router::websocket("/ws/notifications/{userId}", function ($connection, $event, $data) {
    $userId = $connection->params["userId"];

    if ($event === "open") {
```

```
        error_log("[Notifications] User " . $userId . " connected");
        $connection->send(json_encode([
            "type" => "connected",
            "message" => "Listening for notifications"
        ]));
    }

    if ($event === "message") {
        // Client can send acknowledgments or mark notifications as read
        $message = json_decode($data, true);
        if (($message["type"] ?? "") === "mark-read") {
            error_log("[Notifications] User " . $userId . " read notification " . $message["id"]
        }
    }
});

// Inside the WebSocket handler, push to everyone on this path
Router::websocket("/ws/notifications/{userId}", function ($connection, $event, $data) {
    if ($event === "message") {
        $payload = json_decode($data, true);
        if (($payload["type"] ?? "") === "order-shipped") {
            // Broadcast to every client connected on this path
            $connection->broadcast(json_encode([
                "type" => "notification",
                "title" => "Order Shipped",
                "message" => "Your order #" . ($payload["orderId"] ?? "") . " has been shipped!"
                "action_url" => "/orders/" . ($payload["orderId"] ?? ""),
                "timestamp" => date("c")
            ]), includeSelf: true);
        }
    }
});
```

Broadcasting happens from inside the `Router::websocket()` handler through the `$connection` object: `$connection->send()` for one client, `$connection->broadcast()` for everyone on the same path, and `$connection->broadcastToRoom()` for a room. Request-response meets real-time.

---

10. Connecting from JavaScript

Tina4 ships `frond.js`, a built-in JavaScript helper library with WebSocket support.

Basic Connection

```
<script src="/js/frond.js"></script>
<script>
    const ws = frond.ws("/ws/chat/general");

    ws.on("open", function () {
        console.log("Connected to chat");
    });

    ws.on("message", function (data) {
        const message = JSON.parse(data);
        console.log("Received:", message);

        if (message.type === "message") {
            addMessageToUI(message.from, message.text, message.timestamp);
        }

        if (message.type === "system") {
            addSystemMessage(message.message);
        }
    });

    ws.on("close", function () {
        console.log("Disconnected from chat");
    });

    // Send a message
    function sendMessage(text) {
        ws.send(JSON.stringify({
            type: "message",
            text: text
        }));
    }

    // Set username
    function setUsername(name) {
        ws.send(JSON.stringify({
            type: "set-username",
            username: name
        }));
    }
</script>
```

Auto-Reconnect

`frond.js` reconnects when the connection drops:

```
const ws = frond.ws("/ws/notifications/42", {
    reconnect: true,           // Enable auto-reconnect (default: true)
    reconnectInterval: 3000,   // Retry every 3 seconds
    maxReconnectAttempts: 10   // Give up after 10 attempts
});

ws.on("reconnect", function (attempt) {
    console.log("Reconnecting... attempt " + attempt);
});

ws.on("reconnect_failed", function () {
    console.log("Failed to reconnect after maximum attempts");
```

```
});
```

Using Native WebSocket

The native WebSocket API works too:

```
const ws = new WebSocket("ws://localhost:7145/ws/chat/general");

ws.onopen = function () {
    console.log("Connected");
    ws.send(JSON.stringify({ type: "set-username", username: "Alice" }));
};

ws.onmessage = function (event) {
    const message = JSON.parse(event.data);
    console.log("Received:", message);
};

ws.onclose = function () {
    console.log("Disconnected");
};

ws.onerror = function (error) {
    console.error("WebSocket error:", error);
};
```

The advantage of `frond.js`: auto-reconnect, message buffering during reconnection, a cleaner event API.

---

11. A Complete Chat Page

A full chat page using templates and WebSocket.

Create `src/templates/chat.html`:

```
{% extends "base.html" %}

{% block title %}Chat - {{ room }}{% endblock %}

{% block content %}
    <h1>Chat Room: {{ room }}</h1>
    <p id="status">Connecting...</p>
    <p id="online">Online: 0</p>

    <div id="messages" style="border: 1px solid #ddd; height: 400px; overflow-y: scroll; padding
    </div>

    <div id="typing" style="height: 20px; color: #888; font-style: italic; margin-bottom: 4px;">

    <form id="chat-form" style="display: flex; gap: 8px;">
        <input type="text" id="message-input" placeholder="Type a message..."
               style="flex: 1; padding: 8px; border: 1px solid #ddd; border-radius: 4px;">
        <button type="submit" style="padding: 8px 16px; background: #333; color: white; border:
            Send
        </button>
    </form>
```

```
<script src="/js/frond.js"></script>
<script>
    const room = "{{ room }}";
    const ws = frond.ws("/ws/livechat/" + room);
    const messagesDiv = document.getElementById("messages");
    const statusEl = document.getElementById("status");
    const onlineEl = document.getElementById("online");
    const typingEl = document.getElementById("typing");

    let username = prompt("Enter your username:") || "Anonymous";

    ws.on("open", function () {
        statusEl.textContent = "Connected";
        ws.send(JSON.stringify({ type: "set-username", username: username }));
    });

    ws.on("message", function (data) {
        const msg = JSON.parse(data);

        if (msg.type === "message") {
            const div = document.createElement("div");
            div.style.marginBottom = "8px";
            div.innerHTML = "<strong>" + msg.from + ":</strong> " + msg.text;
            messagesDiv.appendChild(div);
            messagesDiv.scrollTop = messagesDiv.scrollHeight;
        }

        if (msg.type === "system") {
            const div = document.createElement("div");
            div.style.cssText = "color: #888; font-style: italic; margin-bottom: 8px;";
            div.textContent = msg.message;
            messagesDiv.appendChild(div);
            messagesDiv.scrollTop = messagesDiv.scrollHeight;
        }

        if (msg.type === "typing") {
            typingEl.textContent = msg.username + " is typing...";
            setTimeout(function () { typingEl.textContent = ""; }, 2000);
        }

        if (msg.online !== undefined) {
            onlineEl.textContent = "Online: " + msg.online;
        }
    });

    ws.on("close", function () {
        statusEl.textContent = "Disconnected. Reconnecting...";
    });

    document.getElementById("chat-form").addEventListener("submit", function (e) {
        e.preventDefault();
        const input = document.getElementById("message-input");
        if (input.value.trim()) {
            ws.send(JSON.stringify({ type: "message", text: input.value }));
            input.value = "";
        }
    });

    document.getElementById("message-input").addEventListener("input", function () {
```

```
            ws.send(JSON.stringify({ type: "typing" }));
        });
    </script>
{% endblock %}
```

Create the route to serve the page:

```
<?php
use Tina4\Router;

Router::get("/chat/{room}", function ($request, $response) {
    $room = $request->params["room"];
    return $response->render("chat.html", ["room" => $room]);
});
```

Open `http://localhost:7145/chat/general` in two browser tabs. Type in one. Watch the message appear in the other.

---

12. Exercise: Build a Real-Time Chat Room

Build a WebSocket chat room with the following features:

Requirements

- WebSocket endpoint at `/ws/room/{roomName}` that handles:
- `open`: Send welcome message with connection count
- `message` with type `set-name`: Set the user's display name
- `message` with type `chat`: Broadcast the message to all clients with the sender's name
- `close`: Broadcast that the user left
- HTTP endpoint at `GET /room/{roomName}` that serves an HTML chat page
- The chat page should:
- Prompt for a username on load
- Display messages in real time
- Show system messages (join/leave) in a different style
- Show the count of online users

Test by:

- Open `http://localhost:7145/room/test` in two browser tabs
- Set different usernames in each
- Send messages from both tabs and verify they appear in both
- Close one tab and verify the "user left" message appears in the other

---

13. Solution

Create `src/routes/chat-room.php`:

```
<?php
use Tina4\Router;

$roomUsers = [];

Router::websocket("/ws/room/{roomName}", function ($connection, $event, $data) use (&$roomUsers)
    $room = $connection->params["roomName"];
    $key = $room . ":" . $connection->id;

    if ($event === "open") {
        $roomUsers[$key] = "Anonymous";

        $connection->send(json_encode([
            "type" => "system",
            "message" => "Welcome to room: " . $room . ". Set your name with: {\"type\": \"set-n
            "online" => $connection->connectionCount()
        ]));

        $connection->broadcast(json_encode([
            "type" => "system",
            "message" => "A new user joined",
            "online" => $connection->connectionCount()
        ]), true);
    }

    if ($event === "message") {
        $msg = json_decode($data, true);
        $type = $msg["type"] ?? "chat";

        if ($type === "set-name") {
            $oldName = $roomUsers[$key] ?? "Anonymous";
            $newName = $msg["name"] ?? "Anonymous";
            $roomUsers[$key] = $newName;

            $connection->broadcast(json_encode([
                "type" => "system",
                "message" => $oldName . " changed their name to " . $newName
            ]));
        }

        if ($type === "chat") {
            $username = $roomUsers[$key] ?? "Anonymous";

            $connection->broadcast(json_encode([
                "type" => "chat",
                "from" => $username,
                "text" => $msg["text"] ?? "",
                "timestamp" => date("H:i:s")
            ]));
        }
    }

    if ($event === "close") {
        $username = $roomUsers[$key] ?? "Anonymous";
        unset($roomUsers[$key]);
```

```
        $connection->broadcast(json_encode([
            "type" => "system",
            "message" => $username . " left the room",
            "online" => $connection->connectionCount()
        ]));
    }
});

Router::get("/room/{roomName}", function ($request, $response) {
    $room = $request->params["roomName"];
    return $response->render("room.html", ["room" => $room]);
});
```

Create `src/templates/room.html`:

```
{% extends "base.html" %}

{% block title %}Room: {{ room }}{% endblock %}

{% block content %}
    <h1>Room: {{ room }}</h1>
    <p id="online" style="color: #666;">Online: 0</p>

    <div id="messages" style="border: 1px solid #ddd; height: 400px; overflow-y: auto; padding:
    </div>

    <form id="form" style="display: flex; gap: 8px;">
        <input type="text" id="input" placeholder="Type a message..."
               style="flex: 1; padding: 10px; border: 1px solid #ddd; border-radius: 4px; font-s
        <button type="submit"
                style="padding: 10px 20px; background: #1a1a2e; color: white; border: none; bord
            Send
        </button>
    </form>

    <script src="/js/frond.js"></script>
    <script>
        const room = "{{ room }}";
        const username = prompt("Choose a username:") || "Anonymous";
        const ws = frond.ws("/ws/room/" + room);
        const msgs = document.getElementById("messages");

        ws.on("open", function () {
            ws.send(JSON.stringify({ type: "set-name", name: username }));
        });

        ws.on("message", function (raw) {
            const msg = JSON.parse(raw);
            const div = document.createElement("div");
            div.style.marginBottom = "8px";

            if (msg.type === "chat") {
                div.innerHTML = "<strong>" + msg.from + "</strong> <span style='color:#999;font-
            } else if (msg.type === "system") {
                div.style.color = "#888";
                div.style.fontStyle = "italic";
                div.textContent = msg.message;
            }

            msgs.appendChild(div);
```

```
            msgs.scrollTop = msgs.scrollHeight;

            if (msg.online !== undefined) {
                document.getElementById("online").textContent = "Online: " + msg.online;
            }
        });

        document.getElementById("form").addEventListener("submit", function (e) {
            e.preventDefault();
            const input = document.getElementById("input");
            if (input.value.trim()) {
                ws.send(JSON.stringify({ type: "chat", text: input.value }));
                input.value = "";
            }
        });
    </script>
{% endblock %}
```

Open `http://localhost:7145/room/test` in two browser tabs. Set different usernames. Send messages from one tab and watch them appear in both. Close one tab and verify the "left the room" message appears.

---

14. Scaling with a Backplane

When you run a single server instance, `broadcast()` reaches every connected client. But in production you often run multiple instances behind a load balancer. Each instance only knows about its own connections. A message broadcast on instance A never reaches clients connected to instance B.

A backplane solves this. It relays WebSocket messages across all instances using the shared `tina4:ws` pub/sub channel. Tina4 supports Redis and NATS adapters. Local delivery remains the default.

Configuration

Set two environment variables in your `.env`:

```
TINA4_WS_BACKPLANE=redis
TINA4_WS_BACKPLANE_URL=redis://localhost:6379
```

For NATS:

```
TINA4_WS_BACKPLANE=nats
TINA4_WS_BACKPLANE_URL=nats://localhost:4222
```

Every `broadcast()` call delivers to local connections first, then publishes an envelope to the backplane. Sibling instances discard the sender's echo and relay the message to their own local connections. Your WebSocket routes do not change.

Requirements

Redis needs no Composer package. Tina4 uses the `redis` language extension when present and otherwise speaks RESP over a socket. The extension is a capability, not a package dependency.

NATS needs its protocol client in the application:

```
composer require basis-company/nats
```

The NATS client is an application dependency, not a Tina4 core dependency. Tina4 loads it only when the application selects NATS. An explicit selection is a deployment contract: an unknown adapter, missing package, or unreachable broker fails startup with `Configured WebSocket backplane failed`. Tina4 never claims distributed delivery while running local-only.

When to Choose a Backplane

Deployment	Choice
One Tina4 process or container	Leave `TINA4_WS_BACKPLANE` unset
Multiple instances and Redis is already operated	`redis`
Multiple instances and NATS is already operated	`nats`
Unsure which broker to introduce	Redis is usually the simpler starting point

The signal is the process count, not traffic volume: once clients can land on different Tina4 instances, broadcasts need a shared backplane. Do not install NATS solely because Tina4 supports it; choose it when the deployment already uses NATS or needs NATS as its wider event bus.

Verify the setup with two Tina4 processes. Connect one WebSocket client to each process, broadcast from the first, and confirm the second client receives the frame. A single-process test proves only local delivery.

If `TINA4_WS_BACKPLANE` is not set, Tina4 broadcasts only to local connections. This is the correct default for a single instance.

---

15. Gotchas

1. WebSocket Needs a Persistent Server

Problem: WebSocket connections drop or fail to connect.

Cause: You are running behind a traditional PHP-FPM + Nginx setup. PHP-FPM processes terminate after each HTTP request. No persistent process exists to maintain WebSocket connections.

Fix: Use `tina4 serve`. It runs a persistent server handling both HTTP and WebSocket. For production, run the Tina4 server directly -- not behind PHP-FPM. Nginx can still serve as a reverse proxy, but it must be configured for WebSocket: `proxy_set_header Upgrade $http_upgrade` and `proxy_set_header Connection "upgrade"`.

2. Messages Are Strings, Not Objects

Problem: `$data` in the message handler is a string, not a PHP array.

Cause: WebSocket transmits raw strings. JSON from the client arrives as a JSON string, not a decoded array.

Fix: Always `json_decode($data, true)` when you expect JSON. Always `json_encode()` when you send structured data. WebSocket does not know about content types. It moves bytes.

3. Connection Count Is Per-Path

Problem: `$connection->connectionCount()` returns a lower number than expected.

Cause: Connection count is scoped to the WebSocket path. Clients on `/ws/chat/room-1` are counted separately from `/ws/chat/room-2`.

Fix: This is by design. Each path is an isolated group. For a global connection count across all paths, maintain a counter in a shared variable.

4. Broadcasting Does Not Scale Across Servers

Problem: Users connected to different server instances do not see each other's messages.

Cause: `TINA4_WS_BACKPLANE` is unset, so `$connection->broadcast()` correctly uses local-only mode. Multiple instances behind a load balancer each maintain their own connection sets.

Fix: Use a pub/sub backend. Redis works well. Each server subscribes to a Redis channel. Broadcast messages publish to Redis. All servers receive them.

5. Large Messages Cause Disconnects

Problem: The connection drops when sending a large message.

Cause: WebSocket servers enforce a maximum message size. The default is around 1MB. Exceed it, and the server closes the connection.

Fix: Keep messages small. Under 64KB is a good target. For large data transfers, use HTTP endpoints. WebSocket is built for small, frequent messages -- not bulk data.

6. Memory Leak from Tracking Connected Users

Problem: Server memory grows over time and crashes.

Cause: User data stored in a PHP array (like `$chatUsers`) is not cleaned up when users disconnect. The `close` handler does not remove them. The array grows without limit.

Fix: Always clean up in the `close` handler. Remove disconnected users from tracking arrays: `unset($chatUsers[$connection->id])`. Test by connecting and disconnecting repeatedly. Verify memory stays stable.

7. No Authentication on WebSocket Connect

Problem: Anyone can connect to your WebSocket endpoint and see all messages.

Cause: The WebSocket upgrade request does not carry your JWT token in the `Authorization` header. Browsers do not support custom headers on WebSocket connections.

Fix: Pass the token as a query parameter: `ws://localhost:7145/ws/chat?token=eyJ...`. In your `open` handler, validate the token and disconnect if invalid. Use a short-lived token for WebSocket connections. Or authenticate via HTTP first, store the session, and check the session cookie during the upgrade.

Server-Sent Events (SSE)

1. The Polling Problem

Your monitoring dashboard needs live metrics. CPU usage. Memory. Active connections. The numbers change every few seconds.

The obvious solution: poll. A timer fires every three seconds. The browser sends a GET request. The server queries the database. The server responds. The browser updates the page. Repeat.

That works. It also wastes resources. Nineteen out of twenty polls return the same data. Each one opens a connection, sends headers, waits for a response, and closes the connection. The server does the same work whether the data changed or not.

WebSocket solves this. A persistent, bi-directional connection. The server pushes when data changes. But WebSocket is designed for two-way communication. Chat rooms. Collaborative editing. Live gaming. Your dashboard only needs one direction: server to client.

Server-Sent Events (SSE) is the middle ground. One-way streaming over plain HTTP. The server pushes. The client listens. The browser reconnects automatically if the connection drops. No upgrade handshake. No special protocol. Just HTTP with a twist.

---

2. What SSE Is

HTTP works like this:

```
Client: "Give me /api/metrics"
Server: "Here are the metrics" (connection closes)
Client: "Give me /api/metrics" (new connection, 3 seconds later)
Server: "Here are the metrics" (connection closes)
```

WebSocket works like this:

```
Client: "Upgrade to WebSocket on /ws/metrics"
Server: "Upgrade accepted. Connection open."
Client: "Subscribe to CPU metrics"
Server: "CPU: 42%"
Server: "CPU: 38%"  (pushed at any time)
Client: "Unsubscribe"
```

SSE works like this:

```
Client: "Give me /events (keep the connection open)"
Server: "data: CPU 42%"
Server: "data: CPU 38%"  (pushed, no client request)
Server: "data: CPU 45%"  (keeps flowing)
Client: (just listens)
```

The key differences:

Feature	HTTP Polling	WebSocket	SSE
Direction	Client to server	Both ways	Server to client

Connection	New each time	Persistent	Persistent
Protocol	HTTP	WebSocket (upgrade)	HTTP
Auto-reconnect	No	No (manual)	Yes (built-in)
Binary data	Yes	Yes	No (text only)
Browser support	Universal	Universal	Universal (except IE)
Complexity	Simple	Moderate	Simple

SSE uses plain HTTP. No protocol upgrade. No special server support. The server sets `Content-Type: text/event-stream` and keeps the connection open. The browser's `EventSource` API handles reconnection, event parsing, and last-event-ID tracking.

---

3. Your First Stream

Create `src/routes/events.php`:

```
<?php
use Tina4\Router;
use Tina4\Request;
use Tina4\Response;

Router::get("/events", function (Request $request, Response $response) {
    $generator = function () {
        for ($i = 0; $i < 10; $i++) {
            yield "data: " . json_encode(["count" => $i, "message" => "tick"]) . "\n\n";
            sleep(1);
        }
        yield "data: " . json_encode(["done" => true]) . "\n\n";
    };

    return $response->stream($generator);
});
```

Three things happen here:

- `$generator` is a closure that returns a PHP generator. Each `yield` produces one SSE message.
- `$response->stream()` tells Tina4 to keep the connection open and send each chunk as the generator yields it.
- The framework sets `Content-Type: text/event-stream`, `Cache-Control: no-cache`, `Connection: keep-alive`, and `X-Accel-Buffering: no` for you.

Start the server and test with curl:

```
curl -N http://localhost:7145/events
```

You see one message per second:

```
data: {"count":0,"message":"tick"}

data: {"count":1,"message":"tick"}

data: {"count":2,"message":"tick"}
```

Each message arrives the moment the server yields it. No buffering. No waiting for the full response.

---

4. The SSE Protocol

SSE messages are plain text with a simple format. Each message is one or more field lines followed by a blank line.

The `data:` Field

The most common field. Contains the message payload.

```
data: Hello, world
```

Multiple `data:` lines in one message get joined with newlines:

```
data: line one
data: line two
```

The client receives this as `"line one\nline two"`.

The `event:` Field

Names the event type. Without it, the browser fires `onmessage`. With it, the browser fires a named event listener.

```
yield "event: metrics\ndata: " . json_encode(["cpu" => 42]) . "\n\n";
yield "event: alert\ndata: " . json_encode(["level" => "high", "message" => "CPU spike"]) . "\n\
```

On the client:

```
source.addEventListener("metrics", (e) => {
    const data = JSON.parse(e.data);
    updateDashboard(data);
});

source.addEventListener("alert", (e) => {
    const data = JSON.parse(e.data);
    showAlert(data.message);
});
```

Named events let you multiplex different data types on a single connection.

The `id:` Field

Sets the last event ID. If the connection drops, the browser sends `Last-Event-ID` in the reconnection request. Your server can resume from where it left off.

```
yield "id: {$i}\ndata: " . json_encode(["count" => $i]) . "\n\n";
```

The `retry:` Field

Tells the browser how many milliseconds to wait before reconnecting after a disconnect.

```
yield "retry: 5000\n\n"; // reconnect after 5 seconds
```

The Delimiter

Every SSE message ends with two newlines: `\n\n`. This tells the browser that the message is complete. A single `\n` separates fields within one message.

```
// One complete SSE message:
yield "event: update\nid: 42\ndata: " . json_encode(["value" => 100]) . "\n\n";

// Broken down:
// event: update\n        <- event type
// id: 42\n               <- event ID
// data: {"value":100}\n  <- payload
// \n                     <- end of message (blank line)
```

---

5. Frontend: EventSource

The browser provides `EventSource` for consuming SSE streams. It handles connection management, reconnection, and message parsing.

Basic Usage

```
const source = new EventSource("/events");

source.onmessage = function (event) {
    const data = JSON.parse(event.data);
    console.log("Received:", data);
};

source.onerror = function () {
    console.log("Connection lost. Reconnecting...");
};

source.onopen = function () {
    console.log("Connected");
};
```

`EventSource` reconnects automatically when the connection drops. The `onerror` handler fires, the browser waits (default 3 seconds), then reconnects. Your server receives a new request with the `Last-Event-ID` header if you sent event IDs.

Named Events

```
const source = new EventSource("/events");

source.addEventListener("metrics", function (event) {
    updateChart(JSON.parse(event.data));
});

source.addEventListener("notification", function (event) {
    showToast(JSON.parse(event.data));
});

source.addEventListener("heartbeat", function (event) {
    // Connection is alive
});
```

Closing the Connection

```
source.close();
```

The browser stops reconnecting. The server receives a client disconnect.

With Authentication

`EventSource` does not support custom headers. Pass tokens as query parameters:

```
const token = getAuthToken();
const source = new EventSource(`/events?token=${token}`);
```

On the server:

```
<?php
use Tina4\Router;
use Tina4\Request;
use Tina4\Response;
use Tina4\Auth;

Router::get("/events", function (Request $request, Response $response) {
    $token = $request->query["token"] ?? "";
    $payload = Auth::validToken($token);
    if (!$payload) {
        return $response->json(["error" => "Unauthorized"], 401);
    }

    $generator = function () use ($payload) {
        while (true) {
            yield "data: " . json_encode(["user" => $payload["email"]]) . "\n\n";
            sleep(5);
        }
    };

    return $response->stream($generator);
});
```

---

6. Real Examples

Live Dashboard

Stream database metrics every five seconds:

```
<?php
use Tina4\Router;
use Tina4\Request;
use Tina4\Response;
use Tina4\Database\Database;

Router::get("/events/dashboard", function (Request $request, Response $response) {
    $generator = function () {
        $db = Database::fromEnv();
        while (true) {
            $orders = $db->fetch("SELECT count(*) as total FROM orders")->records[0] ?? ["total"
            $revenue = $db->fetch("SELECT sum(total) as revenue FROM orders WHERE date(created_a

            yield "data: " . json_encode([
                "orders" => (int) $orders["total"],
                "revenue" => (float) ($revenue["revenue"] ?? 0),
            ]) . "\n\n";
            sleep(5);
        }
    };

    return $response->stream($generator);
});
```

The client updates the dashboard numbers without polling. One connection. One query every five seconds. No wasted requests.

Build Progress

Stream deployment status as it happens:

```
<?php
use Tina4\Router;
use Tina4\Request;
use Tina4\Response;

Router::get("/events/deploy/{buildId}", function (Request $request, Response $response, string $
    $generator = function () use ($buildId) {
        $steps = [
            ["pull", "Pulling latest code..."],
            ["install", "Installing dependencies..."],
            ["test", "Running tests..."],
            ["build", "Building application..."],
            ["deploy", "Deploying to production..."],
            ["done", "Deployment complete"],
        ];

        foreach ($steps as [$step, $message]) {
            yield "event: progress\ndata: " . json_encode([
                "step" => $step,
                "message" => $message,
                "build_id" => $buildId,
            ]) . "\n\n";
```

```
            sleep(2); // simulate work
        }
    };

    return $response->stream($generator);
});
```

The browser shows a progress bar that updates in real time. Each step arrives as it completes. The user watches the deployment unfold instead of staring at a spinner.

LLM Chat Streaming

Stream AI responses token by token. This is how ChatGPT works:

```
<?php
use Tina4\Router;
use Tina4\Request;
use Tina4\Response;

Router::get("/events/chat", function (Request $request, Response $response) {
    $prompt = $request->query["q"] ?? "Hello";

    $generator = function () use ($prompt) {
        // Call the LLM API with streaming enabled
        $body = json_encode([
            "model" => "claude-sonnet-4-20250514",
            "max_tokens" => 512,
            "stream" => true,
            "messages" => [["role" => "user", "content" => $prompt]],
        ]);

        $context = stream_context_create([
            "http" => [
                "method" => "POST",
                "header" => implode("\r\n", [
                    "Content-Type: application/json",
                    "x-api-key: " . getenv("ANTHROPIC_API_KEY"),
                    "anthropic-version: 2023-06-01",
                ]),
                "content" => $body,
            ],
        ]);

        $stream = fopen("https://api.anthropic.com/v1/messages", "r", false, $context);
        while (!feof($stream)) {
            $line = trim(fgets($stream));
            if (str_starts_with($line, "data: ")) {
                $chunk = json_decode(substr($line, 6), true);
                if (($chunk["type"] ?? "") === "content_block_delta") {
                    $text = $chunk["delta"]["text"] ?? "";
                    yield "data: " . json_encode(["token" => $text]) . "\n\n";
                }
            }
        }
        fclose($stream);

        yield "data: " . json_encode(["done" => true]) . "\n\n";
    };

    return $response->stream($generator);
```

```
});
```

The frontend appends each token as it arrives:

```
const source = new EventSource(`/events/chat?q=${encodeURIComponent(question)}`);
const output = document.getElementById("response");

source.onmessage = function (event) {
    const data = JSON.parse(event.data);
    if (data.done) {
        source.close();
        return;
    }
    output.textContent += data.token;
};
```

The response builds character by character. The user reads as the AI writes. No waiting for the full answer.

File Processing Progress

Upload a CSV, stream the processing status:

```
<?php
use Tina4\Router;
use Tina4\Request;
use Tina4\Response;

Router::post("/api/import", function (Request $request, Response $response) {
    $file = $request->files["csv"] ?? null;
    if (!$file) {
        return $response->json(["error" => "No file"], 400);
    }

    $lines = explode("\n", trim($file["content"]));
    $total = count($lines) - 1; // minus header

    $generator = function () use ($lines, $total) {
        $rows = array_slice($lines, 1);
        foreach ($rows as $i => $line) {
            // Process each row
            $cols = explode(",", $line);
            // ... insert into database ...
            $processed = $i + 1;
            yield "data: " . json_encode([
                "processed" => $processed,
                "total" => $total,
                "percent" => (int) round($processed / $total * 100),
            ]) . "\n\n";
        }

        yield "data: " . json_encode(["done" => true, "total" => $total]) . "\n\n";
    };

    return $response->stream($generator, "text/event-stream");
});
```

The browser shows a progress bar that fills as each row is processed. The user sees exactly where the import stands.

---

7. Custom Content Types

SSE defaults to `text/event-stream`, but `$response->stream()` accepts any content type as the second argument. Use this for newline-delimited JSON (NDJSON), chunked binary, or custom protocols.

NDJSON Streaming

```
<?php
use Tina4\Router;
use Tina4\Request;
use Tina4\Response;
use Tina4\Database\Database;

Router::get("/api/export", function (Request $request, Response $response) {
    $generator = function () {
        $db = Database::fromEnv();
        $result = $db->fetch("SELECT * FROM products", 1000);
        foreach ($result->records as $row) {
            yield json_encode($row) . "\n";
        }
    };

    return $response->stream($generator, "application/x-ndjson");
});
```

Each line is a complete JSON object. The client reads line by line. No need to parse a giant array. Memory stays flat regardless of how many rows you export.

Consuming NDJSON on the Client

```
async function streamProducts() {
    const response = await fetch("/api/export");
    const reader = response.body.getReader();
    const decoder = new TextDecoder();
    let buffer = "";

    while (true) {
        const { done, value } = await reader.read();
        if (done) break;

        buffer += decoder.decode(value, { stream: true });
        const lines = buffer.split("\n");
        buffer = lines.pop(); // keep incomplete line

        for (const line of lines) {
            if (line.trim()) {
                const product = JSON.parse(line);
                addToTable(product);
            }
        }
    }
}
```

---

8. SSE vs WebSocket

Both keep a persistent connection. Both push data in real time. The choice depends on the direction of communication.

Use Case	Choose	Why
Live dashboard	SSE	Server pushes metrics. Client only listens.
Chat application	WebSocket	Both sides send messages.
Notification feed	SSE	Server pushes. Client marks as read via separate HTTP POST.
Collaborative editor	WebSocket	Both sides send cursor positions and edits.
Build/deploy progress	SSE	Server streams status. Client watches.
LLM token streaming	SSE	Server streams tokens. Client displays them.
Online gaming	WebSocket	Low-latency, bi-directional input and state.
Stock ticker	SSE	Server pushes prices. Client displays.
File upload progress	HTTP (native)	Browser tracks upload. Server tracks processing via SSE.

Rule of thumb: If the client only listens, use SSE. If the client sends data back on the same connection, use WebSocket.

SSE has one advantage WebSocket does not: automatic reconnection. The browser's `EventSource` reconnects without any code. WebSocket requires manual reconnection logic (or `frond.js` which handles it for you).

---

9. Production Considerations

Nginx Proxy Buffering

Nginx buffers responses by default. SSE messages accumulate in the buffer and arrive in batches instead of one at a time. Tina4 sets `X-Accel-Buffering: no` on streaming responses, which tells Nginx to disable buffering for that response.

If you configure Nginx manually:

```
location /events/ {
    proxy_pass http://localhost:7145;
    proxy_buffering off;
    proxy_cache off;
    proxy_set_header Connection '';
    proxy_http_version 1.1;
    chunked_transfer_encoding off;
}
```

Connection Limits

Each SSE connection is a persistent HTTP connection. Most servers limit concurrent connections. Tina4's built-in server uses `stream_select()` to multiplex thousands of connections in a single process. In production:

- Monitor open connections
- Set timeouts on long-running streams
- Close streams when the client no longer needs them

```
<?php
use Tina4\Router;
use Tina4\Request;
use Tina4\Response;

Router::get("/events/metrics", function (Request $request, Response $response) {
    $generator = function () {
        for ($i = 0; $i < 3600; $i++) { // max 1 hour (one message per second)
            yield "data: " . json_encode(["cpu" => get_cpu()]) . "\n\n";
            sleep(1);
        }
        yield "event: timeout\ndata: " . json_encode(["message" => "Stream expired. Reconnect."]
    };

    return $response->stream($generator);
});
```

`$response->stream()` checks `connection_aborted()` after each chunk and breaks the loop when the client disconnects, so dropped clients release their slot immediately.

Heartbeat Messages

Some proxies and load balancers close idle connections after a timeout (typically 30-60 seconds). Send a heartbeat comment to keep the connection alive:

```
$generator = function () {
    while (true) {
        if (has_new_data()) {
            yield "data: " . json_encode(get_data()) . "\n\n";
        } else {
            yield ": heartbeat\n\n"; // SSE comment (colon prefix) -- ignored by EventSource
        }
        sleep(5);
    }
};
```

Lines starting with `:` are SSE comments. The browser ignores them, but they keep the connection alive through proxies.

---

10. Exercise: Build a Live Metrics Dashboard

Build a dashboard that streams server metrics to the browser.

Requirements

- SSE endpoint at `GET /events/server-metrics` that streams every 3 seconds:
- Current timestamp
- A random CPU percentage (simulate with `random_int(10, 90)`)
- A random memory percentage
- A random request count
- HTML page at `GET /dashboard` that:
- Connects to the SSE endpoint
- Displays the four metrics in cards
- Updates the values in real time (no page refresh)
- Shows a "Connected" / "Reconnecting..." status indicator

Test by:

- Open `http://localhost:7145/dashboard`
- Watch the numbers update every 3 seconds
- Stop the server and verify "Reconnecting..." appears
- Restart the server and verify it reconnects and resumes updating

---

11. Solution

Create `src/routes/server_metrics.php`:

```
<?php
use Tina4\Router;
use Tina4\Request;
use Tina4\Response;

Router::get("/events/server-metrics", function (Request $request, Response $response) {
    $generator = function () {
        yield "retry: 3000\n\n";
        $counter = 0;
        while (true) {
            $payload = json_encode([
                "timestamp" => gmdate("c"),
                "cpu" => random_int(10, 90),
                "memory" => random_int(30, 85),
```

```
                "requests" => random_int(100, 5000),
            ]);
            yield "id: {$counter}\ndata: {$payload}\n\n";
            $counter++;
            sleep(3);
        }
    };

    return $response->stream($generator);
});


Router::get("/dashboard", function (Request $request, Response $response) {
    $html = <<<HTML
    <!DOCTYPE html>
    <html>
    <head><title>Live Dashboard</title></head>
    <body style="font-family: system-ui; max-width: 600px; margin: 40px auto; padding: 0 20px;">
        <h1>Server Metrics</h1>
        <p id="status" style="color: #888;">Connecting...</p>
        <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 16px; margin-top: 20px;"
            <div style="background: #f5f5f5; padding: 20px; border-radius: 8px;">
                <div style="color: #888; font-size: 14px;">CPU</div>
                <div id="cpu" style="font-size: 32px; font-weight: 700;">--</div>
            </div>
            <div style="background: #f5f5f5; padding: 20px; border-radius: 8px;">
                <div style="color: #888; font-size: 14px;">Memory</div>
                <div id="memory" style="font-size: 32px; font-weight: 700;">--</div>
            </div>
            <div style="background: #f5f5f5; padding: 20px; border-radius: 8px;">
                <div style="color: #888; font-size: 14px;">Requests/min</div>
                <div id="requests" style="font-size: 32px; font-weight: 700;">--</div>
            </div>
            <div style="background: #f5f5f5; padding: 20px; border-radius: 8px;">
                <div style="color: #888; font-size: 14px;">Last Update</div>
                <div id="time" style="font-size: 16px; font-weight: 500;">--</div>
            </div>
        </div>
        <script>
            const source = new EventSource("/events/server-metrics");
            const status = document.getElementById("status");

            source.onopen = function () {
                status.textContent = "Connected";
                status.style.color = "#22c55e";
            };

            source.onmessage = function (event) {
                const data = JSON.parse(event.data);
                document.getElementById("cpu").textContent = data.cpu + "%";
                document.getElementById("memory").textContent = data.memory + "%";
                document.getElementById("requests").textContent = data.requests.toLocaleString()
                document.getElementById("time").textContent = new Date(data.timestamp).toLocaleT
            };

            source.onerror = function () {
                status.textContent = "Reconnecting...";
                status.style.color = "#ef4444";
            };
```

```
        </script>
    </body>
    </html>
    HTML;
    return $response->html($html);
});
```

Open `http://localhost:7145/dashboard`. The numbers update every three seconds. Stop the server. The status turns red: "Reconnecting..." Start it again. The status turns green. The numbers resume.

No polling. No WebSocket. No JavaScript timers. The browser handles everything.

---

12. What You Learned

- **SSE streams data from server to client** over a single HTTP connection. No upgrade. No special protocol.
- `$response->stream($generator)` keeps the connection open and flushes each chunk as the generator yields it.
- **The SSE protocol** uses `data:`, `event:`, `id:`, and `retry:` fields. Messages end with `\n\n`.
- `EventSource` on the client handles connection, parsing, and automatic reconnection.
- **Named events** let you multiplex different data types on one stream.
- **Custom content types** let you stream NDJSON, binary, or any format -- pass the type as the second argument to `stream()`.
- **SSE is for one-way server pushes.** Use WebSocket when the client needs to send data back on the same connection.
- **Heartbeat comments** keep connections alive through proxies. Tina4 sets `X-Accel-Buffering: no` to disable nginx buffering and breaks the loop on `connection_aborted()` to release dropped clients.

One direction. One connection. Real-time data. The server speaks. The client listens. The rest is infrastructure.

WSDL / SOAP

1. When You Need SOAP

Most new APIs use REST and JSON. But enterprise systems (banks, insurers, government portals, ERP platforms) often expose SOAP services. You need to interoperate. Sometimes you are also required to expose a SOAP interface for an existing client that cannot change.

Tina4 provides a `WSDL` base class. Extend it. Annotate methods with the `#[WSDLOperation([...])]` PHP attribute. The WSDL document is generated automatically and served at `?wsdl`. Your service handles both SOAP 1.1 and SOAP 1.2.

---

2. Creating a SOAP Service

Extend `Tina4\WSDL`. Each public method marked with the `#[WSDLOperation([...])]` attribute becomes a SOAP operation. The attribute's array declares the response shape, a map of field names to XSD types.

```
<?php
use Tina4\WSDL;
use Tina4\WSDLOperation;

class CalculatorService extends WSDL
{
    protected string $serviceName = 'Calculator';

    #[WSDLOperation(['Result' => 'float'])]
    public function Add(float $a, float $b): array
    {
        return ['Result' => $a + $b];
    }

    #[WSDLOperation(['Result' => 'float'])]
    public function Subtract(float $a, float $b): array
    {
        return ['Result' => $a - $b];
    }

    #[WSDLOperation(['Result' => 'float'])]
    public function Multiply(float $a, float $b): array
    {
        return ['Result' => $a * $b];
    }

    #[WSDLOperation(['Result' => 'float'])]
    public function Divide(float $a, float $b): array
    {
        if ($b == 0) {
            throw new \RuntimeException('Division by zero is not allowed');
        }
        return ['Result' => $a / $b];
    }
}
```

Each operation is a public method on the subclass. The `#[WSDLOperation([...])]` PHP attribute declares the response shape, a map of field names to XSD types. The method returns an associative array whose keys match the attribute's spec. Parameter types come from PHP's native type hints (`float`, `int`, `string`, `bool`).

---

3. Registering the Service

Mount the service on a URL path. The same handler answers both the WSDL document request (`GET ?wsdl`) and SOAP invocations (`POST`): `(new CalculatorService($request))->handle()` inspects the request and returns the right thing.

```
<?php
use Tina4\Router;
use Tina4\Request;
use Tina4\Response;

Router::any('/calculator', function (Request $request, Response $response) {
    $service = new CalculatorService($request);
    return $service->handle();
});
```

The service is now live at:

- `POST /calculator` - accepts SOAP envelopes, dispatches to the matching `#[WSDLOperation]` method
- `GET /calculator?wsdl` - returns the auto-generated WSDL document

---

4. Accessing the Auto-Generated WSDL

```
curl http://localhost:7145/calculator?wsdl
```

Response:

```
<?xml version="1.0" encoding="UTF-8"?>
<wsdl:definitions
    name="CalculatorService"
    targetNamespace="http://localhost:7145/calculator"
    xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
    xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
    xmlns:tns="http://localhost:7145/calculator"
    xmlns:xsd="http://www.w3.org/2001/XMLSchema">

  <wsdl:types>
    <xsd:schema targetNamespace="http://localhost:7145/calculator">
      <xsd:element name="Add">
        <xsd:complexType>
          <xsd:sequence>
            <xsd:element name="a" type="xsd:float"/>
            <xsd:element name="b" type="xsd:float"/>
          </xsd:sequence>
        </xsd:complexType>
```

```
      </xsd:element>
      <!-- ... other operations ... -->
    </xsd:schema>
  </wsdl:types>

  <wsdl:portType name="CalculatorServicePortType">
    <wsdl:operation name="Add">
      <wsdl:documentation>Add two numbers together.</wsdl:documentation>
      <wsdl:input message="tns:AddSoapIn"/>
      <wsdl:output message="tns:AddSoapOut"/>
    </wsdl:operation>
    <!-- ... -->
  </wsdl:portType>

  <!-- binding, service, port elements follow -->
</wsdl:definitions>
```

The WSDL is derived from your PHP type hints and docblock comments. You write PHP. Tina4 writes the WSDL.

---

5. Calling the Service with PHP SoapClient

Test your service using PHP's built-in `SoapClient`:

```
<?php

$client = new SoapClient('http://localhost:7145/calculator?wsdl', [
    'trace'      => true,
    'exceptions' => true
]);

// Add
$result = $client->Add(['a' => 10.5, 'b' => 4.5]);
echo $result->AddResult;   // 15.0

// Subtract
$result = $client->Subtract(['a' => 100, 'b' => 37]);
echo $result->SubtractResult;   // 63.0

// Multiply
$result = $client->Multiply(['a' => 6, 'b' => 7]);
echo $result->MultiplyResult;   // 42.0

// Divide
$result = $client->Divide(['a' => 22, 'b' => 7]);
echo $result->DivideResult;     // 3.142857...

// SOAP Fault on divide by zero
try {
    $client->Divide(['a' => 1, 'b' => 0]);
} catch (\SoapFault $e) {
    echo $e->getMessage();  // Division by zero is not allowed
}
```

---

6. A Real-World Service: Currency Conversion

```php
<?php
use Tina4\WSDL;

class CurrencyService extends WSDL
{
    private array $rates = [
        'USD' => 1.0,
        'EUR' => 0.92,
        'GBP' => 0.79,
        'JPY' => 149.50,
        'AUD' => 1.54
    ];

    #[WSDLOperation(['Converted' => 'float'])]
    public function Convert(float $amount, string $fromCurrency, string $toCurrency): array
    {
        $from = strtoupper($fromCurrency);
        $to   = strtoupper($toCurrency);

        if (!isset($this->rates[$from])) {
            throw new \RuntimeException("Unsupported currency: {$from}");
        }
        if (!isset($this->rates[$to])) {
            throw new \RuntimeException("Unsupported currency: {$to}");
        }

        // Convert via USD as the base
        $usdAmount = $amount / $this->rates[$from];
        return ['Converted' => round($usdAmount * $this->rates[$to], 2)];
    }

    #[WSDLOperation(['Currencies' => 'string'])]
    public function GetSupportedCurrencies(): array
    {
        return ['Currencies' => implode(',', array_keys($this->rates))];
    }

    #[WSDLOperation(['Rate' => 'float'])]
    public function GetRate(string $fromCurrency, string $toCurrency): array
    {
        $result = $this->Convert(1.0, $fromCurrency, $toCurrency);
        return ['Rate' => $result['Converted']];
    }
}
```

Register:

```php
Router::any('/currency', function (Request $request, Response $response) {
    return (new CurrencyService($request))->handle();
});
```

Call:

```php
$client = new SoapClient('http://localhost:7145/currency?wsdl');

$converted = $client->Convert(['amount' => 100, 'fromCurrency' => 'USD', 'toCurrency' => 'EUR']);
echo $converted->Converted;          // 92.0
```

```
$currencies = $client->GetSupportedCurrencies();
echo $currencies->Currencies;        // USD,EUR,GBP,JPY,AUD
```

The SOAP response field names mirror the `#[WSDLOperation([...])]` attribute keys.

---

7. Type Annotations

Tina4 reads PHP 8 type hints on parameters to generate XSD request types, and the `#[WSDLOperation([fieldName => 'xsd-type'])]` attribute to generate response types. The mapping is:

PHP type / attribute value	WSDL/XSD type
`int` / `'int'` / `'integer'`	`xsd:integer`
`float` / `'float'` / `'double'`	`xsd:float`
`string` / `'string'`	`xsd:string`
`bool` / `'boolean'`	`xsd:boolean`
`array` (parameter)	`xsd:anyType`

Always use explicit PHP 8 type declarations on method parameters. The response field names AND types come from the attribute; the method returns an associative array matching the attribute's spec.

---

8. SOAP Faults

Throw `SoapFault` to return a structured SOAP error to the client. The first argument is the fault code, the second is the fault message.

Standard fault codes:

Code	When to use
`'Client'`	The request is malformed or missing required data
`'Receiver'`	The server encountered an error processing a valid request
`'VersionMismatch'`	SOAP version mismatch

```
#[WSDLOperation(['Email' => 'string'])]
public function GetUserEmail(int $userId): array
{
    if ($userId <= 0) {
        throw new \SoapFault('Client', 'User ID must be a positive integer');
    }
```

```
    $user = $this->db->fetchOne("SELECT email FROM users WHERE id = ?", [$userId]);

    if ($user === null) {
        throw new \SoapFault('Receiver', "User {$userId} not found");
    }

    return ['Email' => $user['email']];
}
```

---

9. Testing with curl

Send a raw SOAP envelope to test without a SOAP client:

```
curl -X POST http://localhost:7145/calculator \
  -H "Content-Type: text/xml; charset=utf-8" \
  -H "SOAPAction: \"Add\"" \
  -d '<?xml version="1.0" encoding="utf-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <Add xmlns="http://localhost:7145/calculator">
      <a>10.5</a>
      <b>4.5</b>
    </Add>
  </soap:Body>
</soap:Envelope>'
```

Response:

```
<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <AddResponse xmlns="http://localhost:7145/calculator">
      <AddResult>15</AddResult>
    </AddResponse>
  </soap:Body>
</soap:Envelope>
```

---

10. Gotchas

1. Method not appearing in WSDL

Problem: A method exists on the class but does not appear in the generated WSDL.

Cause: Missing `#[WSDLOperation([...])]` PHP attribute on the method.

Fix: Add `#[WSDLOperation(['FieldName' => 'xsdType'])]` immediately above the method declaration. All WSDL-exposed methods must have it. Make sure you `use Tina4\WSDLOperation;` at the top of the file.

2. Wrong XSD types

Problem: The client receives strings instead of numbers, or booleans instead of integers.

Cause: PHP type hints are missing on method parameters, OR the `#[WSDLOperation([...])]` response-shape array uses the wrong XSD type name.

Fix: Always declare explicit parameter types using PHP 8 type hints (`int`, `float`, `string`, `bool`). For response types, use the XSD names in the attribute array: `'int'`, `'float'`, `'string'`, `'boolean'`.

3. SOAP Fault not reaching the client

Problem: Throwing `SoapFault` inside the service does not produce a proper SOAP fault response. The client receives a generic HTTP 500.

Cause: The exception is caught by a top-level error handler before Tina4 can serialize it as a SOAP fault.

Fix: Only throw `\SoapFault`. Do not throw generic exceptions. Tina4's SOAP dispatcher catches `SoapFault` and serializes it correctly.

4. WSDL URL is wrong in generated document

Problem: The generated WSDL contains `localhost` as the service URL. Clients in other environments cannot use it.

Cause: The WSDL generator reads the `Host` header from the current request.

Fix: Set `TINA4_HOST_NAME` in your environment to override the auto-detected host: `TINA4_HOST_NAME=https://api.example.com`.

DI Container

1. The Problem with Global State

Without dependency injection, your route handlers create their own service instances:

```
Router::post('/api/orders', function ($request, $response) {
    $db    = new Database(getenv('TINA4_DATABASE_URL'));
    $mailer = new Mailer(getenv('TINA4_MAIL_HOST'), getenv('TINA4_MAIL_PORT'));
    $logger = new Logger('/var/log/app.log');
    // ...
});
```

Every handler creates fresh instances. Configuration is duplicated. Testing requires mocking at the class level. The code is tightly coupled to concrete implementations.

Dependency injection inverts this. You register services once. The container creates and manages them. Handlers receive what they need without knowing how it was constructed.

Tina4 provides a built-in `Container` class. No Reflection-based auto-wiring complexity. Register factories. Retrieve by name. Singletons for shared instances.

---

2. Creating a Container

```
<?php
use Tina4\Container;

$container = new Container();
```

The container is an empty registry. Register services into it. Retrieve them by name.

---

3. register() - Transient Services

`register()` stores a factory callable. Each call to `get()` invokes the factory and returns a new instance.

```
<?php
use Tina4\Container;

$container = new Container();

// Register a factory for a database connection
$container->register('db', function () {
    return new \Tina4\Database(getenv('TINA4_DATABASE_URL'));
});

// Register a logger
$container->register('logger', function () {
    return new \Psr\Log\NullLogger();
});

// Each get() creates a new instance
```

```
$db1 = $container->get('db');
$db2 = $container->get('db');

var_dump($db1 === $db2);   // bool(false) -- different instances
```

Use transient services when you need isolated instances per request or per operation.

---

4. singleton() - Shared Instances

`singleton()` stores a factory that runs once. Every call to `get()` returns the same instance.

```
<?php
use Tina4\Container;

$container = new Container();

// Register as singleton -- created once, reused everywhere
$container->singleton('mailer', function () {
    $mailer = new \Tina4\Messenger();
    $mailer->setHost(getenv('TINA4_MAIL_HOST'));
    $mailer->setPort((int) getenv('TINA4_MAIL_PORT'));
    $mailer->setUsername(getenv('SMTP_USER'));
    $mailer->setPassword(getenv('SMTP_PASS'));
    return $mailer;
});

$container->singleton('cache', function () {
    return new \Tina4\Cache(backend: getenv('TINA4_CACHE_BACKEND', 'memory'));
});

// Same instance every time
$m1 = $container->get('mailer');
$m2 = $container->get('mailer');

var_dump($m1 === $m2);   // bool(true) -- same instance
```

Use singletons for services that are expensive to create (database pools, HTTP clients, email configuration) or must share state (caches, event buses).

---

5. get() and has()

`get()` retrieves a service by name. `has()` checks existence before retrieving.

```
<?php
use Tina4\Container;

$container = new Container();
$container->singleton('db', fn() => new \Tina4\Database(getenv('TINA4_DATABASE_URL')));

// Check before retrieving
if ($container->has('db')) {
    $db = $container->get('db');
}
```

```
// Throws \Tina4\ContainerException if name not registered
try {
    $missing = $container->get('nonexistent');
} catch (\Tina4\ContainerException $e) {
    echo $e->getMessage();  // "Service 'nonexistent' is not registered"
}
```

---

6. Services Depending on Other Services

Factories receive the container instance as their argument. Use it to resolve dependencies:

```
<?php
use Tina4\Container;

$container = new Container();

$container->singleton('logger', function () {
    return new \Tina4\Debug();
});

$container->singleton('db', function () {
    return new \Tina4\Database(getenv('TINA4_DATABASE_URL'));
});

// OrderService depends on db and logger
$container->singleton('order_service', function () use ($container) {
    return new OrderService(
        db:     $container->get('db'),
        logger: $container->get('logger')
    );
});

class OrderService {
    public function __construct(
        private \Tina4\Database $db,
        private \Tina4\Debug $logger
    ) {}

    public function createOrder(array $data): array {
        $this->logger->message("Creating order", TINA4_LOG_INFO, $data);
        // $this->db->execute(...);
        return ['id' => rand(1000, 9999), ...$data];
    }
}

$orders = $container->get('order_service');
$order  = $orders->createOrder(['customer' => 'Alice', 'total' => 59.99]);
```

Services are composed through the container. Each service is unaware of how its dependencies were built.

---

7. Binding a Container to Route Handlers

Pass the container into your route handlers via `use`:

```php
<?php
use Tina4\Router;
use Tina4\Container;

// Bootstrap container in app.php or index.php
$container = new Container();

$container->singleton('db', fn() => new \Tina4\Database(getenv('TINA4_DATABASE_URL')));

$container->singleton('order_service', function () use ($container) {
    return new OrderService($container->get('db'));
});

// Route uses the container
Router::post('/api/orders', function ($request, $response) use ($container) {
    $orderService = $container->get('order_service');
    $body = $request->body;

    $order = $orderService->createOrder([
        'customer' => $body['customer'],
        'items'    => $body['items'],
        'total'    => $body['total']
    ]);

    return $response->json(['order' => $order], 201);
});

Router::get('/api/orders/{id:int}', function ($request, $response) use ($container) {
    $orderService = $container->get('order_service');
    $order = $orderService->findOrder($request->params['id']);

    if ($order === null) {
        return $response->json(['error' => 'Order not found'], 404);
    }

    return $response->json(['order' => $order]);
});
```

---

8. reset() - Testing

`reset()` clears all registrations. Use it between test cases to prevent state leakage:

```php
<?php
use Tina4\Container;

// In your test setUp
$container = new Container();
$container->singleton('db', fn() => new MockDatabase());

// Run the test
$service = $container->get('db');
assert($service instanceof MockDatabase);

// Tear down
$container->reset();

// Container is empty again
```

```
assert($container->has('db') === false);
```

---

9. A Full App Bootstrap

`src/bootstrap.php` - all services registered in one place:

```
<?php
use Tina4\Container;

$container = new Container();

// Infrastructure
$container->singleton('db', function () {
    return new \Tina4\Database(getenv('TINA4_DATABASE_URL'));
});

$container->singleton('cache', function () {
    return new \Tina4\Cache(getenv('TINA4_CACHE_BACKEND') ?: 'memory');
});

$container->singleton('queue', function () {
    return new \Tina4\Queue(topic: 'default');
});

$container->singleton('mailer', function () {
    $m = new \Tina4\Messenger();
    $m->setHost(getenv('TINA4_MAIL_HOST'));
    return $m;
});

// Domain services
$container->singleton('user_service', function () use ($container) {
    return new UserService(
        db:     $container->get('db'),
        mailer: $container->get('mailer'),
        cache:  $container->get('cache')
    );
});

$container->singleton('order_service', function () use ($container) {
    return new OrderService(
        db:    $container->get('db'),
        queue: $container->get('queue')
    );
});

return $container;
```

In `index.php`:

```
$container = require 'src/bootstrap.php';

// Pass to all routes
require 'src/routes/users.php';
require 'src/routes/orders.php';
```

---

10. Gotchas

1. Circular dependencies

Problem: Service A depends on B, and B depends on A. The container loops forever.

Cause: A circular dependency in your service graph.

Fix: Introduce a third service that both A and B depend on, or restructure to break the cycle. The container does not detect cycles automatically; you will see a PHP stack overflow.

2. Forgetting to use singleton for stateful services

Problem: Two calls to `$container->get('mailer')` produce two SMTP connections.

Cause: Registered with `register()` instead of `singleton()`.

Fix: Use `singleton()` for any service where a single shared instance is correct. Use `register()` only when you explicitly need a new instance each time.

3. Not resetting between tests

Problem: A singleton registered in test A bleeds into test B.

Cause: Tests share the same container instance without calling `reset()`.

Fix: Create a fresh `new Container()` or call `$container->reset()` in `setUp()` in each test class.

4. Storing request-scoped data in a singleton

Problem: User A's request data appears in User B's response.

Cause: A singleton service stores per-request state (current user, request context) on the instance itself.

Fix: Singletons must be stateless or explicitly scoped. Pass per-request data as method arguments rather than storing it on the service instance.

Service Runner

1. Work That Runs Outside HTTP Requests

Not everything fits in an HTTP request. A queue consumer that processes emails. A cache warmer that pre-populates product data on a schedule. These run continuously in the background, independent of web traffic.

Tina4's service runner provides a pattern for long-running background workers. They start with the application, restart on failure, and shut down cleanly.

---

2. The Service Interface

A background service implements a simple contract: a `run()` method that loops indefinitely, and a `stop()` method that signals it to exit.

```
<?php
use Tina4\Service;

class EmailQueueWorker extends Service
{
    private bool $running = true;
    private \Tina4\Queue $queue;

    public function __construct()
    {
        $this->queue = new \Tina4\Queue(topic: 'emails');
    }

    public function run(): void
    {
        echo "[EmailQueueWorker] Starting...\n";

        while ($this->running) {
            $job = $this->queue->pop();

            if ($job === null) {
                // No jobs pending -- sleep and check again
                sleep(1);
                continue;
            }

            try {
                $this->processEmail($job->payload);
                $job->complete();
                echo "[EmailQueueWorker] Email sent to {$job->payload['to']}\n";
            } catch (\Throwable $e) {
                $job->fail($e->getMessage());
                echo "[EmailQueueWorker] Failed: {$e->getMessage()}\n";
            }
        }

        echo "[EmailQueueWorker] Stopped.\n";
```

```
    }

    public function stop(): void
    {
        $this->running = false;
    }

    private function processEmail(array $payload): void
    {
        // Actual email sending logic here
        // \Tina4\Messenger::send($payload['to'], $payload['subject'], $payload['body']);
        echo "  Sending: {$payload['subject']} -> {$payload['to']}\n";
    }
}
```

---

3. Registering and Starting Services

```
<?php
use Tina4\ServiceRunner;

// Register services
ServiceRunner::registerService('email_queue_worker', new EmailQueueWorker());
ServiceRunner::registerService('report_generator', new ReportGenerator());
ServiceRunner::registerService('cache_warmer', new CacheWarmer());

// Start all services
ServiceRunner::start();
```

`start()` launches each service in its own process or thread (depending on the platform). Services run concurrently and independently.

---

4. A Report Generator Service

```
<?php
use Tina4\Service;

class ReportGenerator extends Service
{
    private bool $running = true;

    public function run(): void
    {
        echo "[ReportGenerator] Starting...\n";

        while ($this->running) {
            $now = new \DateTimeImmutable();
            $hour = (int) $now->format('G');

            // Generate daily summary at 02:00
            if ($hour === 2 && (int) $now->format('i') === 0) {
                $this->generateDailySummary($now->format('Y-m-d'));
                sleep(61); // Prevent double-trigger within the same minute
                continue;
            }

            sleep(30); // Check every 30 seconds
        }

        echo "[ReportGenerator] Stopped.\n";
    }

    public function stop(): void
    {
        $this->running = false;
    }

    private function generateDailySummary(string $date): void
    {
        echo "[ReportGenerator] Generating summary for {$date}...\n";
        // Build and email/store the report
    }
}
```

---

5. A Cache Warmer Service

Pre-populate the cache at startup and refresh it before it expires:

```
<?php
use Tina4\Service;
use function Tina4\cache_set;

class CacheWarmer extends Service
{
    private bool $running = true;
    private int $ttl = 300;         // 5 minutes
    private int $refreshBefore = 60; // Refresh 60 seconds before expiry

    public function run(): void
```

```
    {
        echo "[CacheWarmer] Starting...\n";

        // Warm on start
        $this->warmAll();

        while ($this->running) {
            // Refresh $refreshBefore seconds before expiry
            sleep($this->ttl - $this->refreshBefore);
            $this->warmAll();
        }

        echo "[CacheWarmer] Stopped.\n";
    }

    public function stop(): void
    {
        $this->running = false;
    }

    private function warmAll(): void
    {
        echo "[CacheWarmer] Warming cache...\n";

        $db = new \Tina4\Database(getenv('TINA4_DATABASE_URL'));

        $categories = $db->fetchAll("SELECT * FROM categories ORDER BY name");
        cache_set('categories:all', $categories, $this->ttl);

        $featured = $db->fetchAll(
            "SELECT * FROM products WHERE featured = 1 ORDER BY rank LIMIT 20"
        );
        cache_set('products:featured', $featured, $this->ttl);

        echo "[CacheWarmer] Cache warmed (" . count($categories) . " categories, " . count($feat
    }
}
```

---

6. Graceful Shutdown

Signal handling lets your service exit cleanly when the server receives SIGTERM (e.g., during a deployment):

```
<?php
use Tina4\Service;

class RobustWorker extends Service
{
    private bool $running = true;

    public function run(): void
    {
        // Register signal handlers
        if (function_exists('pcntl_signal')) {
            pcntl_signal(SIGTERM, function () {
                echo "[RobustWorker] SIGTERM received, stopping...\n";
```

```
                $this->stop();
            });

            pcntl_signal(SIGINT, function () {
                echo "[RobustWorker] SIGINT received, stopping...\n";
                $this->stop();
            });
        }

        echo "[RobustWorker] Running...\n";

        while ($this->running) {
            if (function_exists('pcntl_signal_dispatch')) {
                pcntl_signal_dispatch();
            }

            $this->doWork();
            sleep(5);
        }

        echo "[RobustWorker] Graceful shutdown complete.\n";
    }

    public function stop(): void
    {
        $this->running = false;
    }

    private function doWork(): void
    {
        // Business logic here
    }
}
```

---

7. Running Services as a Standalone Script

For simple deployments, run services directly as a PHP CLI script:

`src/workers/run.php`:

```
<?php
require __DIR__ . '/../../vendor/autoload.php';

use Tina4\ServiceRunner;

ServiceRunner::registerService('email_queue_worker', new EmailQueueWorker());
ServiceRunner::registerService('report_generator', new ReportGenerator());
ServiceRunner::start();
```

Run it:

```
php src/workers/run.php
```

Keep it alive with a process manager. With Supervisor:

```
[program:tina4-workers]
command=php /var/www/app/src/workers/run.php
```

```
directory=/var/www/app
autostart=true
autorestart=true
stderr_logfile=/var/log/tina4-workers.err.log
stdout_logfile=/var/log/tina4-workers.out.log
user=www-data
```

With systemd:

```
[Unit]
Description=Tina4 Background Workers
After=network.target

[Service]
Type=simple
User=www-data
WorkingDirectory=/var/www/app
ExecStart=/usr/bin/php src/workers/run.php
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

---

8. Running with Docker

```
FROM php:8.3-cli

WORKDIR /app
COPY . .
RUN composer install --no-dev

CMD ["php", "src/workers/run.php"]
```

```
# docker-compose.yml (excerpt)
services:
  web:
    build: .
    # `tina4 serve` requires the Rust CLI; inside Docker we run the framework
    # directly with TINA4_OVERRIDE_CLIENT=true so the framework boots without it.
    command: sh -c "TINA4_OVERRIDE_CLIENT=true php index.php"

  workers:
    build: .
    command: php src/workers/run.php
    environment:
      - TINA4_DATABASE_URL=sqlite:./data/app.db
      - TINA4_MAIL_HOST=mailhog
    depends_on:
      - db
```

The web service and workers service share the same image. They run independently.

---

9. Gotchas

1. Services blocking each other

Problem: Two services are registered but only the first one runs. The second never starts.

Cause: The first service's `run()` method blocks the main thread forever. The second service never gets control.

Fix: Either use `ServiceRunner` with proper multi-process support (each service in its own process), or run each service in a separate PHP process. Do not put multiple blocking loops in the same process without concurrency support.

2. Memory leak in long-running workers

Problem: The worker process grows in memory until the server runs out.

Cause: Variables accumulate in the loop. Large result sets are never freed.

Fix: `unset()` large variables inside the loop. Call `gc_collect_cycles()` periodically. Keep loop iterations small: process one job at a time, not thousands.

3. Database connection dropped

Problem: The worker fails after several hours with "server has gone away" or "connection lost."

Cause: The database server closes idle connections. The worker holds a connection for hours and then tries to use a dead connection.

Fix: Reconnect on failure. Wrap your database calls in a try/catch. On `PDOException` matching "gone away" or "connection lost," create a new connection and retry.

4. Not logging worker errors

Problem: The worker silently fails. Nothing appears in logs. Jobs pile up.

Cause: Exceptions inside `processEmail()` or `doWork()` are caught by a bare catch block that does nothing.

Fix: Always log exceptions inside the worker loop using `Debug::message()`. At minimum, log the error and continue to the next job.

MCP Dev Tools

1. What is MCP?

The Model Context Protocol connects AI coding tools to your running application. Claude Code, Cursor, and other assistants speak this protocol. When they connect, they see your database, routes, templates, and files -- not as static text, but as live, queryable tools.

Tina4 ships a built-in MCP server. It starts automatically in dev mode. No configuration needed.

---

2. How It Works

Set `TINA4_DEBUG=true` in your `.env` file and start the server:

```
php app.php
```

The console prints:

```
MCP server available at http://localhost:7145/__dev/mcp
```

Tina4 writes the connection details to `.claude/settings.json` automatically. Claude Code discovers the server on its next restart. Cursor reads the same file.

Localhost-Only Security

The built-in MCP server activates only when two conditions hold:

- `TINA4_DEBUG=true`
- The server runs on `localhost`, `127.0.0.1`, or `0.0.0.0`

Deploy to a remote server and the MCP endpoint vanishes -- even with debug enabled. This prevents accidental exposure of database queries and file editing in production.

To enable on a staging server (rare, intentional), set:

```
TINA4_MCP_REMOTE=true
```

---

3. Connecting AI Tools

Claude Code

Tina4 auto-generates `.claude/settings.json` with the current Streamable HTTP transport:

```
{
  "mcpServers": {
    "tina4-dev": {
      "type": "http",
      "url": "http://localhost:7145/__dev/mcp"
    }
  }
}
```

Restart Claude Code. It connects and lists the tools.

Prefer the command line? Register the same server in one step:

```
claude mcp add --transport http tina4-dev http://localhost:7145/__dev/mcp
```

Cursor

Copy the same MCP config into Cursor's settings. The `type` and `url` are identical.

Manual Connection

Any modern MCP client talks to a single endpoint:

- **Streamable HTTP**: `POST http://localhost:7145/__dev/mcp` -- send JSON-RPC 2.0 and read the response inline. `initialize` returns an `Mcp-Session-Id` header; send it back on later calls. `DELETE` on the same URL ends the session.

Older clients that speak the 2024-11-05 HTTP+SSE transport keep working:

- **SSE handshake** (legacy): `GET http://localhost:7145/__dev/mcp/sse` -- returns the message endpoint URL
- **Message endpoint** (legacy): `POST http://localhost:7145/__dev/mcp/message` -- accepts JSON-RPC 2.0 messages

---

4. Built-in Tools

The MCP server exposes 48 tools organized by category. The exact list lives in the framework's `Tina4\Mcp\Tools` registration block -- that is authoritative.

Database

Tool	Description
`database_query`	Execute a read-only SQL query (SELECT)
`database_execute`	Execute arbitrary SQL (INSERT, UPDATE, DELETE, DDL)
`database_tables`	List all tables in the database
`database_columns`	Get column definitions for a specific table

Example -- an AI assistant queries your database directly:

```
Tool: database_query
Arguments: {"sql": "SELECT id, name, email FROM users WHERE active = 1"}
```

The `database_execute` tool handles write operations. It commits automatically after each statement.

Routes

Tool	Description
`route_list`	List all registered routes with methods and auth status
`route_test`	Call a route and return status, body, and headers
`swagger_spec`	Return the full OpenAPI 3.0.3 specification

The `route_test` tool lets an AI assistant verify endpoints without leaving the conversation:

```
Tool: route_test
Arguments: {"method": "GET", "path": "/api/users"}
```

Templates

Tool	Description
`template_render`	Render a Frond template string with provided data

Useful for testing template snippets:

```
Tool: template_render
Arguments: {"template": "Hello {{ name }}!", "data": "{\"name\": \"Alice\"}"}
```

Files

Tool	Description
`file_read`	Read a project file (relative to project root)
`file_write`	Write or update a project file (full overwrite)
`file_patch`	Targeted edit -- replace `old_string` with `new_string` in a file
`file_list`	List files in a directory
`asset_upload`	Upload a file to `src/public/`

File operations are sandboxed. Paths that escape the project directory are rejected. An AI assistant cannot read `/etc/passwd` or write outside your project.

Migrations

Tool	Description
`migration_status`	List pending and completed migrations
`migration_create`	Create a new migration file
`migration_run`	Run all pending migrations

Data and ORM

Tool	Description
`orm_describe`	List all ORM models with fields, types, and relationships
`seed_table`	Seed a table with fake data

Infrastructure

Tool	Description
`queue_status`	Queue size by status (pending, completed, failed)
`session_list`	Active sessions with data
`cache_stats`	Response cache hit/miss statistics

Debugging

Tool	Description
`log_tail`	Read recent log entries
`error_log`	Recent errors and exceptions with stack traces
`env_list`	Environment variables (sensitive values redacted)
`system_info`	Framework version, PHP version, project info

The `env_list` tool redacts any variable containing "secret", "password", "token", "key", or "credential" in its name.

Project introspection

Tool	Description
`git_status`	Show current branch, modified/untracked files, and recent commits
`deps_list`	List the project's declared Composer dependencies (from `composer.json`)
`project_overview`	One-shot snapshot: dependencies, routes, models, tables, errors, git

Persistent code index

Tool	Description
`index_rebuild`	Refresh the persistent project index (lazy, mtime-based)
`index_search`	Find files by path, symbol, route, or summary -- use FIRST for "where is X"
`index_file`	Full index entry for one file: symbols, routes, imports
`index_overview`	Project shape: files by language, routes, models, recent edits

Framework documentation (prose)

Tool	Description
`docs_list`	List framework documentation files
`docs_search`	Search Tina4 framework docs for a query string -- use before guessing
`docs_section`	Return a full markdown section from a framework doc file

Live API RAG (reflection)

`api_*` tools query the live framework reflection -- actual class/method signatures, file/line locations, with a `framework` vs `user` source tag. Use these for "what's the signature of X?" questions; `docs_*` is for "explain queues conceptually".

Tool	Description
`api_search`	Live API search -- finds framework + user classes/methods by name/summary

`api_class`	Live class reflection -- returns full method list and signatures for an FQN
`api_method`	Live method reflection -- returns signature, params, return type, file, line

Plan management

The `plan_*` family lets an AI assistant cooperate with a markdown plan file in `plan/`. Tina4 uses these to keep a steady record of in-progress refactors and multi-session work.

Tool	Description
`plan_current`	The active plan: title, steps (done/not), next step, progress
`plan_list`	All plans in `plan/` with progress and which one is active
`plan_create`	Create a new markdown plan and make it active
`plan_switch_to`	Make a different plan the active one
`plan_complete_step`	Tick a step as done (call the moment the step finishes)
`plan_add_step`	Append a new unchecked step to the current plan
`plan_note`	Append a timestamped note/breadcrumb to the current plan
`plan_archive`	Move a finished plan to `plan/done/` and clear the current pointer
`plan_read`	Full structured view of any plan by filename
`plan_flesh`	Auto-generate concrete build steps via qwen and append them to a plan

---

5. Protocol Details

The MCP server speaks JSON-RPC 2.0 over the current Streamable HTTP transport (protocol versions `2025-06-18` and `2025-03-26`). It still answers the legacy `2024-11-05` HTTP+SSE transport so older clients keep working. The server negotiates: it echoes the version a client asks for when it can speak it, and otherwise falls back to the newest one.

Streamable HTTP (current)

One endpoint carries the whole conversation:

```
POST /__dev/mcp
Content-Type: application/json
```

`initialize` mints a session and returns it in a header:

```
Mcp-Session-Id: 0f9a...c31
```

Send that header on every later request. A request bearing an unknown session gets a `404`, the client's cue to initialize again. A notification (a message with no `id`) returns `202` with an empty body. Every other request answers inline with the JSON-RPC response as `application/json`. `GET /__dev/mcp` returns `405` with an `Allow: POST, DELETE` header; `DELETE /__dev/mcp` ends the session.

Legacy HTTP+SSE (still supported)

```
GET /__dev/mcp/sse
Content-Type: text/event-stream
```

Returns the message endpoint URL as a server-sent event:

```
event: endpoint
data: http://localhost:7145/__dev/mcp/message
```

Then POST JSON-RPC messages to that endpoint:

```
POST /__dev/mcp/message
Content-Type: application/json
```

Messages

Both transports carry the same JSON-RPC 2.0 requests:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/list",
  "params": {}
}
```

Returns JSON-RPC 2.0 responses:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "tools": [
      {"name": "database_query", "description": "Execute a read-only SQL query", "inputSchema":
    ]
  }
}
```

Lifecycle

- Client sends `initialize` -- server responds with capabilities
- Client sends `notifications/initialized` -- server acknowledges

- Client calls `tools/list` -- server returns all registered tools with schemas
- Client calls `tools/call` with tool name and arguments -- server executes and returns results
- Client calls `resources/list` and `resources/read` for data resources

---

6. Troubleshooting

MCP server not starting:

- Check `TINA4_DEBUG=true` in `.env`
- Verify the server is running on localhost (not a remote host)

Claude Code not detecting the server:

- Restart Claude Code after starting the Tina4 server
- Check `.claude/settings.json` exists with the correct URL
- Verify the port matches your server's port

Tools returning errors:

- Database tools require `$db = new Database(...)` in `app.php`
- File tools are sandboxed to the project directory
- `database_execute` only works on localhost

Remote server - MCP disabled:

- This is intentional. Set `TINA4_MCP_REMOTE=true` only on staging environments you trust
- Never enable MCP on production servers

Building Custom MCP Servers

1. Beyond Dev Tools

Chapter 28 covered the built-in MCP server that ships with Tina4. It exposes framework internals for AI-assisted development. This chapter goes further: you build your own MCP servers that expose your application's business logic.

A CRM system exposes customer lookup. An accounting system exposes invoice queries. A warehouse system exposes inventory checks. Any domain logic that an AI assistant should access becomes an MCP tool.

---

2. Creating an MCP Server

Import `McpServer` and create an instance on any path:

```
use Tina4\McpServer;

$mcp = new McpServer("/api/my-tools", name: "My App Tools", version: "1.0.0");
```

The server registers HTTP endpoints at:

- `POST /api/my-tools` -- Streamable HTTP (the current transport): send JSON-RPC and read the response inline. `initialize` returns an `Mcp-Session-Id` header, `DELETE` ends the session.
- `POST /api/my-tools/message` -- legacy JSON-RPC message handler
- `GET /api/my-tools/sse` -- legacy SSE discovery handshake

Register it with the router in `app.php` before `run()`:

```
$mcp->registerRoutes($router);
```

---

3. Registering Tools with #[McpTool]

The `#[McpTool]` attribute turns a method into an MCP tool. Type hints become the input schema automatically:

```
use Tina4\McpServer;
use Tina4\McpTool;

$mcp = new McpServer("/crm/mcp", name: "CRM Tools");

class CrmTools
{
    #[McpTool("lookup_customer", description: "Find a customer by email", server: "crm")]
    public function lookupCustomer(string $email): array
    {
        global $db;
        return $db->fetchOne("SELECT * FROM customers WHERE email = ?", [$email]);
```

```
    }

    #[McpTool("recent_orders", description: "Get recent orders for a customer", server: "crm")]
    public function recentOrders(int $customerId, int $limit = 10): array
    {
        global $db;
        $result = $db->fetch(
            "SELECT * FROM orders WHERE customer_id = ? ORDER BY created_at DESC",
            [$customerId], $limit
        );
        return $result->toArray();
    }
}
```

The attribute extracts:

- **Parameter names** from the method signature
- **Types** from type hints (`string` -> `"string"`, `int` -> `"integer"`, etc.)
- **Required vs optional** -- parameters with defaults are optional
- **Description** from the `description` argument or the docblock

An AI assistant sees these tools and their schemas:

```
{
  "name": "lookup_customer",
  "description": "Find a customer by email",
  "inputSchema": {
    "type": "object",
    "properties": {
      "email": {"type": "string"}
    },
    "required": ["email"]
  }
}
```

---

4. Registering Resources with #[McpResource]

Resources are read-only data endpoints. They expose reference data that AI assistants can browse:

```
use Tina4\McpResource;

class CrmResources
{
    #[McpResource("crm://product-catalog", description: "All active products", server: "crm")]
    public function productCatalog(): array
    {
        global $db;
        return $db->fetch("SELECT id, name, price, category FROM products WHERE active = 1")->to
    }

    #[McpResource("crm://tax-rates", description: "Current tax rates by region", server: "crm")]
    public function taxRates(): array
    {
        global $db;
```

```
        return $db->fetch("SELECT region, rate FROM tax_rates")->toArray();
    }
}
```

Resources are accessed via `resources/list` and `resources/read` in the MCP protocol.

---

5. Class-Based MCP Services

Group related tools into a service class. Each method with `#[McpTool]` becomes a tool:

```
use Tina4\McpServer;
use Tina4\McpTool;

$mcp = new McpServer("/accounting/mcp", name: "Accounting Tools");

class AccountingService
{
    private $db;

    public function __construct($db)
    {
        $this->db = $db;
    }

    #[McpTool("invoice_lookup", description: "Find an invoice by number", server: "accounting")]
    public function lookup(string $invoiceNo): ?array
    {
        return $this->db->fetchOne(
            "SELECT * FROM invoices WHERE invoice_no = ?", [$invoiceNo]
        );
    }

    #[McpTool("outstanding_balances", description: "List all unpaid invoices", server: "accounti
    public function balances(float $minAmount = 0.0): array
    {
        return $this->db->fetch(
            "SELECT * FROM invoices WHERE paid = 0 AND total >= ?", [$minAmount]
        )->toArray();
    }

    #[McpTool("monthly_summary", description: "Revenue summary for a month", server: "accounting
    public function summary(int $year, int $month): ?array
    {
        return $this->db->fetchOne(
            "SELECT SUM(total) as revenue, COUNT(*) as invoice_count "
            . "FROM invoices WHERE strftime('%Y', created_at) = ? "
            . "AND strftime('%m', created_at) = ?",
            [(string)$year, str_pad($month, 2, '0', STR_PAD_LEFT)]
        );
    }
}

// Create the service instance - methods are already registered via attributes
$accounting = new AccountingService($db);
```

---

6. Securing MCP Endpoints

By default, developer MCP servers are public. Add authentication using the standard Tina4 middleware:

```
use Tina4\Secured;
use Tina4\Middleware;

// Secure the entire MCP path
#[Secured]
#[Middleware(AuthMiddleware::class)]
function registerMcp() {
    global $mcp, $router;
    $mcp->registerRoutes($router);
}
```

Or check the bearer token inside individual tools:

```
#[McpTool("sensitive_data", description: "Access restricted data", server: "crm")]
public function sensitiveData(string $token): array
{
    global $db;
    $payload = Auth::validToken($token);
    if (!$payload) {
        return ["error" => "Unauthorized"];
    }
    return $db->fetch("SELECT * FROM sensitive_table")->toArray();
}
```

---

7. Testing MCP Tools

Use `TestClient` to test MCP endpoints without starting a server, or test tool methods directly:

```
// Test the tool method directly
public function testLookupCustomer(): void
{
    $tools = new CrmTools();
    $result = $tools->lookupCustomer("alice@example.com");
    $this->assertNotNull($result);
    $this->assertEquals("alice@example.com", $result["email"]);
}

// Test via MCP protocol
public function testMcpToolCall(): void
{
    $resp = json_decode($mcp->handleMessage([
        "jsonrpc" => "2.0",
        "id" => 1,
        "method" => "tools/call",
        "params" => [
            "name" => "lookup_customer",
            "arguments" => ["email" => "alice@example.com"]
        ]
    ]), true);
    $this->assertArrayHasKey("result", $resp);
    $content = $resp["result"]["content"][0]["text"];
```

```
    $this->assertStringContainsString("alice", strtolower($content));
}
```

---

8. Complete Example: CRM MCP Server

Here is a full working example -- a CRM system with customer, order, and product tools:

```
<?php
// app.php
require_once "vendor/autoload.php";

use Tina4\McpServer;
use Tina4\McpTool;
use Tina4\McpResource;
use Tina4\Database;
use Tina4\ORM;

$db = new Database("sqlite:///crm.db");
ORM::bind($db);

// Create MCP server
$crmMcp = new McpServer("/crm/mcp", name: "CRM Assistant", version: "1.0.0");

class CrmService
{
    private $db;

    public function __construct($db)
    {
        $this->db = $db;
    }

    // Tools
    #[McpTool("find_customer", description: "Search customers by name or email", server: "crm")]
    public function findCustomer(string $query): array
    {
        return $this->db->fetch(
            "SELECT * FROM customers WHERE name LIKE ? OR email LIKE ?",
            ["%{$query}%", "%{$query}%"]
        )->toArray();
    }

    #[McpTool("customer_orders", description: "Get all orders for a customer", server: "crm")]
    public function customerOrders(int $customerId): array
    {
        return $this->db->fetch(
            "SELECT o.*, GROUP_CONCAT(oi.product_name) as items "
            . "FROM orders o LEFT JOIN order_items oi ON o.id = oi.order_id "
            . "WHERE o.customer_id = ? GROUP BY o.id ORDER BY o.created_at DESC",
            [$customerId]
        )->toArray();
    }

    #[McpTool("create_note", description: "Add a note to a customer record", server: "crm")]
    public function createNote(int $customerId, string $note): array
    {
        $this->db->insert("customer_notes", ["customer_id" => $customerId, "note" => $note]);
```

```
        return ["success" => true];
    }

    // Resources
    #[McpResource("crm://products", description: "Product catalog", server: "crm")]
    public function products(): array
    {
        return $this->db->fetch("SELECT * FROM products WHERE active = 1")->toArray();
    }

    #[McpResource("crm://stats", description: "CRM statistics", server: "crm")]
    public function stats(): array
    {
        $customers = $this->db->fetchOne("SELECT COUNT(*) as count FROM customers");
        $orders = $this->db->fetchOne("SELECT COUNT(*) as count, SUM(total) as revenue FROM orde
        return [
            "customers" => $customers["count"],
            "orders" => $orders["count"],
            "revenue" => $orders["revenue"],
        ];
    }
}

// Create the service and register routes
$crm = new CrmService($db);
$crmMcp->registerRoutes($router);

Tina4\run();
```

Connect Claude Code to `http://localhost:7145/crm/mcp` (Streamable HTTP) and ask:

> *"Find all customers named Smith and show their recent orders"*

The AI calls `find_customer` with `query: "Smith"`, then `customer_orders` for each result. No custom API needed. The MCP protocol handles it.

---

9. Best Practices

- **One server per domain** -- CRM tools on `/crm/mcp`, accounting on `/accounting/mcp`
- **Keep tools focused** -- one query per tool, not a Swiss-army-knife tool
- **Use type hints** -- they become the schema. An AI assistant cannot call a tool correctly without knowing the parameter types
- **Return structured data** -- arrays and objects, not formatted strings. Let the AI format for the user
- **Secure production endpoints** -- use `#[Secured]` or middleware for any MCP server that runs outside localhost
- **Test tools directly** -- call the PHP method in your test suite, not just through the MCP protocol

Dev Tools

1. Debugging at 2am

2am. Production monitoring pings you. A 500 error on checkout. You pull up the dev dashboard. The request inspector shows the failing request. The stack trace points to line 47 of `src/routes/checkout.php` -- a null reference on the shipping address because the user skipped the form field. You add a null check. Push the fix. Back to sleep. Total time: 30 seconds.

Tina4's dev tools ship with the framework from day one. When `TINA4_DEBUG=true`, you get a development dashboard, an error overlay with source code, hot reload, a request inspector with replay, a SQL query runner, a queue monitor, and system info -- all without installing extra packages.

---

2. Enabling the Dev Dashboard

Set `TINA4_DEBUG=true` in your `.env`:

```
TINA4_DEBUG=true
```

Restart your server and navigate to:

```
http://localhost:7145/__dev
```

No token or additional environment variables needed. The dashboard runs only when debug mode is on. Set `TINA4_DEBUG=false` in production and the entire dashboard disappears.

---

3. Dashboard Overview

The dev dashboard has several sections. Navigation tabs at the top:

System Overview

The landing page shows at a glance:

- **Framework version** -- The installed Tina4 PHP version
- **PHP version** -- The running PHP version and loaded extensions
- **Uptime** -- How long the server has been running
- **Memory usage** -- Current and peak memory consumption
- **Database status** -- Connection status, database engine, file size (for SQLite)
- **Environment** -- Current `.env` variables (sensitive values are masked)
- **Project structure** -- Directory listing of your project with file counts

Check here first when something feels off. Is the database connected? Is the right PHP version running? Are the environment variables loaded?

---

4. The Dev Toolbar

Visit any HTML page in your application (like `/products` or `/admin`). A debug toolbar appears at the bottom. Thin bar. Expands when you click on it.

The toolbar shows:

Field	What It Means
Request	HTTP method and URL of the current request
Status	HTTP response status code
Time	Total request processing time in milliseconds
DB	Number of database queries executed and total query time
Memory	PHP memory used for this request
Template	The template rendered and how long rendering took
Session	Session ID and session data summary
Route	Which route handler matched this request

Click any section to expand it and see details. For example, clicking "DB" shows every SQL query that ran during the request, with the query text, parameters, execution time, and number of rows returned.

Disabling the Toolbar

The toolbar is automatically hidden when `TINA4_DEBUG=false`. You can also hide it for specific routes by returning a response with the `X-Debug-Toolbar: off` header:

```
Router::get("/api/data", function ($request, $response) {
    return $response->json(["data" => "no toolbar"], 200, [
        "X-Debug-Toolbar" => "off"
    ]);
});
```

This is useful for API endpoints that return JSON -- the toolbar is only meaningful for HTML pages.

---

5. Error Overlay

An unhandled exception occurs. Tina4 does not show a generic "500 Internal Server Error" page. It shows a detailed error overlay with:

- **Exception type and message** -- What went wrong, in plain language

- **Stack trace** -- Every function call that led to the error, from the entry point to the crash
- **Source code** -- The actual PHP code around the line that threw the exception, with the failing line highlighted
- **Request data** -- The HTTP method, URL, headers, body, and query parameters of the request that triggered the error
- **Environment** -- Relevant `.env` variables at the time of the error

Example Error

If you accidentally write:

```
Router::get("/api/broken", function ($request, $response) {
    $user = null;
    return $response->json(["name" => $user->name]);
});
```

Visiting `/api/broken` shows the error overlay:

```
TypeError: Cannot access property "name" on null

  File: src/routes/api.php
  Line: 4

  2 | Router::get("/api/broken", function ($request, $response) {
  3 |     $user = null;
> 4 |     return $response->json(["name" => $user->name]);
  5 | });

  Stack Trace:
    #0 src/routes/api.php:4 - {closure}()
    #1 vendor/tina4/tina4-php/src/Router.php:142 - call_user_func()
    #2 vendor/tina4/tina4-php/src/Server.php:89 - Router->dispatch()

  Request: GET /api/broken
  Headers: Accept: */*
  Query: (none)
  Body: (none)
```

The highlighted line (line 4) makes it immediately obvious: `$user` is null and you are trying to access `->name` on it.

Error Overlay in Production

When `TINA4_DEBUG=false`, the error overlay is disabled. Users see a clean error page (`src/templates/errors/500.html` if it exists, or a generic message). Full error details go to `logs/error.log`.

---

6. Request Inspector

The request inspector records every HTTP request to your application:

- **Method and URL** -- GET /api/products, POST /api/users, etc.

- **Status code** -- 200, 201, 404, 500, etc. (color-coded: green for 2xx, yellow for 4xx, red for 5xx)
- **Response time** -- How many milliseconds the request took
- **Request ID** -- A unique identifier for correlating logs
- **Timestamp** -- When the request was received

Click on any request to see its full details:

Request Details Panel

- **Headers** -- All request headers (Accept, Content-Type, Authorization, etc.)
- **Body** -- The request body (for POST/PUT/PATCH), formatted as JSON if applicable
- **Query parameters** -- Parsed URL query parameters
- **Route match** -- Which route definition matched this request
- **Middleware** -- Which middleware ran and how long each took
- **Database queries** -- Every SQL query executed during this request, with timing
- **Template renders** -- Which templates were rendered and how long each took
- **Response headers** -- The response headers sent back to the client
- **Response body** -- The first 1000 characters of the response body

Filtering Requests

The inspector supports filtering:

- **By status**: Click the status code badges at the top to filter (e.g., show only 5xx errors)
- **By method**: Filter by GET, POST, PUT, DELETE
- **By path**: Search for a URL pattern (e.g., `/api/` to show only API requests)
- **By time range**: Show requests from the last 5 minutes, 1 hour, or all time

Request Replay

Click "Replay" on any request to re-send it. Reproduce an error without constructing the curl command by hand.

---

7. SQL Query Runner

The dev dashboard includes an interactive SQL query runner. Navigate to the "SQL" tab:

- **Execute queries** directly against your database
- **See results** in a formatted table
- **View query timing** -- how long each query took
- **Browse tables** -- a sidebar lists all tables in your database with their column definitions
- **Export results** -- download query results as CSV

Example

Type into the query editor:

```
SELECT name, price, in_stock FROM products WHERE price > 50 ORDER BY price DESC
```

Click "Run" and see:

```
Results (3 rows, 2.1ms):

| name              | price  | in_stock |
|-------------------|--------|----------|
| Standing Desk     | 549.99 | 0        |
| Ergonomic Chair   | 399.99 | 1        |
| Wireless Keyboard | 79.99  | 1        |
```

Faster than opening a separate database client. Test queries, check data, debug issues -- all without leaving the browser.

Safety

The query runner is read-write in development. You can run INSERT, UPDATE, and DELETE statements. Be careful -- there is no undo. In shared environments, consider using a read-only database user for the dev dashboard.

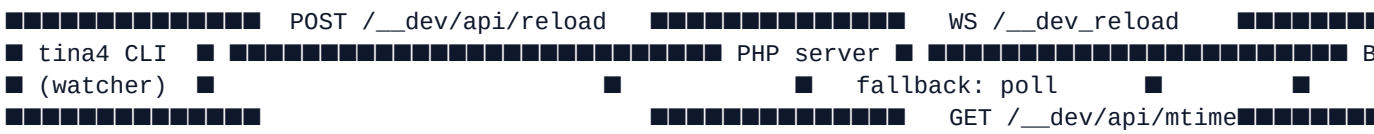
---

8. Hot Reload

Edit a file. Save it. The browser updates. No manual refresh. No external tools.

How It Works

The `tina4` Rust CLI is the sole file watcher for the Tina4 stack; PHP has no internal watcher (there never was one, unlike Python/Ruby/Node which had their own watchers that were removed in 3.11.x). The flow:



- The CLI watches `src/`, `migrations/`, `.env` with the `notify` crate. Events are filtered to real source changes: Access and Metadata-only events are dropped; `__pycache__`, `.git`, `.venv`, `node_modules`, `vendor`, `logs`, `.log`/`.db*`/`.swp`/`.pyc` files are ignored. A real mtime check also defeats overlayfs / polling-mode spurious events (Podman, distrobox).
- On a real change the CLI POSTs `/__dev/api/reload` to your running PHP server. The server keeps running; there is no process restart for file edits.
- `DevAdmin` bumps its in-memory `$reloadMtime` counter and broadcasts `{type: "reload"}` over WebSocket at `/__dev_reload`. `GET /__dev/api/mtime` returns the counter for browsers using the polling fallback.
- The inline reload script injected by the dev toolbar reloads the browser. SCSS/CSS changes are signalled as `type: "css"` and swap the stylesheet without a full reload.

No file watcher inside PHP. No sentinel file on disk. No daemon. No Node.js process. The Rust CLI watches, the framework relays, the browser reloads.

Watched Directories and File Types

The file watcher scans three locations:

- `src/` -- Your application code (routes, ORM models, templates, assets)
- `migrations/` -- Database migration files
- `.env` -- Environment configuration

What Happens on Reload

- **PHP source** (`src/routes/*.php`, `src/orm/*.php`, etc.) -- PHP loads these fresh on every request, so once the browser reloads, the next request sees the updated code. The server process stays up.
- **Template files** (`.twig`, `.html`) -- Re-read from disk on the next render.
- `.env` -- Re-read on next request.
- **SCSS/CSS** -- The browser swaps the stylesheet; no full page reload.

The PHP server never restarts for file edits. It only restarts if it actually crashes (unlikely in dev mode thanks to the error overlay catching most issues).

Activation

Hot reload is only active when `TINA4_DEBUG=true`. In production (`TINA4_DEBUG=false`), the file watcher is disabled, the `/__dev_reload` WebSocket endpoint does not exist, and routes and templates are cached for performance.

Disabling Hot Reload

If hot reload interferes with your workflow (e.g., you are filling out a form and do not want it to reset), disable it in `.env`:

```
TINA4_DEBUG=false
```

This disables the browser-side reload while keeping `TINA4_DEBUG=true` for the rest of the dev tools.

---

9. Gallery -- Interactive Examples

The dev dashboard includes a Gallery section where you can deploy interactive code examples. This is useful for:

- Demonstrating API endpoints to team members
- Creating runnable documentation
- Prototyping features quickly

Creating a Gallery Entry

Create a file in `src/gallery/`:

```
<?php
// src/gallery/product-search.php
// title: Product Search
// description: Search products by name and category

use Tina4\Router;

Router::get("/gallery/product-search", function ($request, $response) {
    $q = $request->params["q"] ?? "";
    $category = $request->params["category"] ?? "";

    $product = new Product();
    $conditions = [];
    $params = [];

    if (!empty($q)) {
        $conditions[] = "name LIKE :q";
        $params["q"] = "%" . $q . "%";
    }

    if (!empty($category)) {
        $conditions[] = "category = :category";
        $params["category"] = $category;
    }

    $filter = !empty($conditions) ? implode(" AND ", $conditions) : "";
    $results = $product->select("*", $filter, $params, "name ASC");

    return $response->json([
        "query" => $q,
        "category" => $category,
        "results" => array_map(fn($p) => $p->toArray(), $results),
        "count" => count($results)
    ]);
});
```

The comments at the top (`// title:` and `// description:`) are parsed by the gallery and shown in the dashboard. Navigate to the Gallery tab to see all your interactive examples, with a "Try It" button that opens the endpoint in a new tab.

---

10. Queue Monitor

If your application uses background job queues (Chapter 12 -- Queues), the dev dashboard includes a queue monitor showing:

- **Pending jobs** -- Jobs waiting to be processed
- **Active jobs** -- Jobs currently being processed
- **Completed jobs** -- Recently completed jobs with timing
- **Failed jobs** -- Jobs that threw exceptions, with error details

- **Dead-letter queue** -- Jobs that failed too many times and were moved to the dead-letter queue

For each job, you can see:

- The job class or function name
- The payload (serialized arguments)
- When it was enqueued
- When it started processing (if active)
- The error message and stack trace (if failed)
- How many times it has been retried

You can also:

- **Retry a failed job** -- Click "Retry" to move it back to the pending queue
- **Delete a job** -- Remove it from any queue
- **Pause/resume the queue** -- Temporarily stop processing without losing jobs

---

11. System Info

The System Info tab shows detailed information about your environment:

- **PHP Configuration** -- Version, loaded extensions, php.ini location, memory limit, max execution time, upload limits
- **Database Info** -- Engine, version, connection details, table sizes, index information
- **Server Info** -- OS, hostname, server software, document root
- **Tina4 Config** -- All loaded `.env` variables (sensitive values masked), auto-discovered routes, registered middleware, ORM models
- **Disk Usage** -- Size of your project directory, data directory, logs directory, and vendor directory

This is especially useful when debugging environment-specific issues. If a colleague says "it works on my machine," compare the System Info output between both environments to spot differences.

---

12. Exercise: Debug a Failing Route

Set up a project with the following intentionally broken route, then use the dev tools to find and fix the bug.

Setup

Create `src/routes/orders.php` with this intentionally broken code:

```
<?php
```

```
use Tina4\Router;

Router::get("/api/orders/summary", function ($request, $response) {
    $product = new Product();
    $products = $product->select("*");

    $total = 0;
    foreach ($products as $p) {
        $total += $p->price * $p->quantity; // Bug: Product has no "quantity" field
    }

    $orderCount = count($products);
    $averageValue = $total / $orderCount; // Bug: division by zero if no products

    return $response->json([
        "total_value" => $total,
        "order_count" => $orderCount,
        "average_value" => round($averageValue, 2)
    ]);
});
```

Task

- Start the dev server with `tina4 serve`.
- Visit `http://localhost:7145/api/orders/summary`.
- Observe the error overlay.
- Use the error overlay to identify the exact line and cause of the error.
- Fix both bugs:
- Replace `$p->quantity` with `1` (or remove the multiplication) since products do not have a quantity field.
- Add a check for zero `$orderCount` before dividing.
- Reload the page and verify the response is correct.
- Open the dev dashboard at `/__dev` and find the failed request in the request inspector.
- Verify the fixed request now appears as a 200 in the inspector.

Expected Steps

- The error overlay shows: `Undefined property: Product::$quantity` on the line `$total += $p->price * $p->quantity;`
- You fix the line to `$total += $p->price;`
- You reload and get a new error (if there are zero products): `Division by zero`
- You add `if ($orderCount > 0)` before the division
- The endpoint returns valid JSON

---

13. Solution

The fixed route:

```
<?php
use Tina4\Router;

Router::get("/api/orders/summary", function ($request, $response) {
    $product = new Product();
    $products = $product->select("*");

    $total = 0.0;
    foreach ($products as $p) {
        $total += $p->price;
    }

    $orderCount = count($products);
    $averageValue = $orderCount > 0 ? $total / $orderCount : 0;

    return $response->json([
        "total_value" => round($total, 2),
        "order_count" => $orderCount,
        "average_value" => round($averageValue, 2)
    ]);
});
```

Expected output (with 5 seeded products):

```
{
  "total_value": 909.95,
  "order_count": 5,
  "average_value": 181.99
}
```

What You Practiced

- Reading the error overlay to find the exact line and error type
- Understanding stack traces
- Using the request inspector to view failed and successful requests
- Iterative debugging -- fixing one error, reloading, fixing the next

---

14. Gotchas

1. Dev Dashboard Returns 404

Problem: Navigating to `/__dev` returns a 404 page.

Cause: `TINA4_DEBUG` is not set to `true`.

Fix: Add to your `.env`:

```
TINA4_DEBUG=true
```

Restart the server after changing `.env`.

3. Hot Reload Not Working

Problem: You save a file but the browser does not reload.

Cause: Hot reload uses a WebSocket connection to the `/__dev_reload` endpoint. If your browser blocks WebSocket connections (some corporate proxies do), or if you are accessing the site via a reverse proxy that does not forward WebSocket, hot reload will not work.

Fix: Check the browser console for WebSocket connection errors to `/__dev_reload`. If you are behind a proxy, configure it to forward WebSocket connections. As a workaround, manually refresh with `Ctrl+R`.

4. Error Overlay Shows in Production

Problem: Users see stack traces and source code on error pages.

Cause: `TINA4_DEBUG=true` in the production `.env`.

Fix: Always set `TINA4_DEBUG=false` in production. This hides all debug information and shows the custom error page from `src/templates/errors/500.html` instead.

5. Request Inspector Shows Too Many Requests

Problem: The request inspector is flooded with entries, making it hard to find the one you care about.

Cause: Static file requests (CSS, JS, images, favicon) are all recorded.

Fix: Use the path filter to narrow down the list. Type `/api/` in the filter box to show only API requests. You can also configure the dev dashboard to exclude static file requests from the inspector.

6. SQL Runner Modifies Data Accidentally

Problem: You ran a DELETE query in the SQL runner and lost data.

Cause: The SQL runner executes queries directly against the database with full read-write access.

Fix: There is no undo. For safety, always include a WHERE clause with DELETE and UPDATE statements. If you are worried about accidental modifications, use a read-only database connection for the dev dashboard.

7. Debug Toolbar Breaks Layout

Problem: The debug toolbar at the bottom of the page overlaps with your content or breaks the layout.

Cause: The toolbar adds a fixed-position element at the bottom of the page. If your page uses `position: fixed` elements at the bottom (like a footer or chat widget), they may conflict.

Fix: The toolbar adds a `data-tina4-debug` attribute to the body. Use this in your CSS to adjust:

```
body[data-tina4-debug] .my-footer {
    bottom: 40px; /* Make room for the debug toolbar */
```

```
}
```

Alternatively, collapse the toolbar by clicking on it -- it minimizes to a thin line.

Tina4 CLI

1. Getting a New Developer Up to Speed

A new developer joins your team Monday morning. You hand them the repo URL. By 10am they have a running project, a new database model, CRUD routes, a migration, and a deployment to staging. All from the command line. No hunting through documentation. No copy-pasting boilerplate from old projects. The CLI handles scaffolding.

The Tina4 CLI is a single Rust binary. It manages all four Tina4 frameworks (PHP, Python, Ruby, Node.js). Commands stay identical across languages. Learn the CLI for PHP. You know it for Python.

---

2. tina4 init -- Project Scaffolding

You saw this in Chapter 1. Now the details.

```
tina4 init my-project
```

```
Creating Tina4 project in ./my-project ...
  Detected language: PHP (composer.json)
  Created .env
  Created .env.example
  Created .gitignore
  Created src/routes/
  Created src/orm/
  Created migrations/
  Created src/seeds/
  Created src/templates/
  Created src/templates/errors/
  Created src/public/
  Created src/public/js/
  Created src/public/css/
  Created src/public/scss/
  Created src/public/images/
  Created src/public/icons/
  Created src/locales/
  Created data/
  Created logs/
  Created secrets/
  Created tests/

Project created! Next steps:
  cd my-project
  composer install
  tina4 serve
```

Language Detection

The CLI detects the language from existing files in the directory:

File Present	Language

`composer.json`	PHP
`pyproject.toml` or `requirements.txt`	Python
`Gemfile`	Ruby
`package.json`	Node.js

If no language-specific file exists, the CLI asks you:

```
tina4 init my-project

No language detected. Which language?
  1. PHP
  2. Python
  3. Ruby
  4. Node.js
> 1

Creating Tina4 PHP project in ./my-project ...
```

Init into an Existing Directory

Already have a project? Add Tina4 structure:

```
cd existing-project
tina4 init .
```

The CLI creates only files and directories that do not exist. It never overwrites.

---

3. tina4 serve -- Dev Server

```
tina4 serve

 _____ _              _  _
|_   _(_)_ __   __ _| || |
  | | | | '_ \ / _` | || |_
  | | | | | | | (_| |__   _|
  |_| |_|_| |_|\__,_|  |_|

  Tina4 PHP v3.0.0
  Server running at http://0.0.0.0:7145
  Debug mode: ON
  Database: sqlite:///data/app.db
  Press Ctrl+C to stop
```

Options

Flag	Description	Example
`-p`, `--port`	Custom port (default: 7145)	`tina4 serve -p 8080`
`--host`	Bind address (default: 0.0.0.0)	`tina4 serve --host 127.0.0.1`
`--no-browser`	Don't open a browser tab on startup	`tina4 serve --no-browser`
`--no-reload`	Disable live reload	`tina4 serve --no-reload`
`--dev`	Force the dev server even if a production server is installed	`tina4 serve --dev`
`--production`	Production server, debug off (see below)	`tina4 serve --production`

Serving a Project by Name

You don't have to `cd` into a project first. Pass its folder name and the CLI will find it, change into it, and serve it:

```
tina4 serve my-app             # ./my-app, or my-app in your projects folder
tina4 serve my-app -p 7150     # Options still apply
```

The CLI looks for `./my-app` in the current folder first, then in the projects folder `tina4 setup` recorded. If neither has it, it stops with `Project 'my-app' not found` and names the folder it searched. Run a bare `tina4 serve` outside any project and it will look in the projects folder too: one project there gets served, and several get listed so you can pick one by name.

Production Mode

```
tina4 serve --production
```

When the `--production` flag is passed, the CLI:

- Sets `TINA4_DEBUG=false` for the server, so the dev toolbar, the error overlay and template recompilation are off
- Sets `TINA4_PRODUCTION=true`, the framework's signal that you asked for production
- Compiles SCSS once and skips the live-reload watcher

In practice, you rarely use `tina4 serve --production` directly. Instead, you use Docker or a process manager (Chapter 34). But this flag is useful for quick production testing.

---

4. tina4 generate model -- ORM Scaffolding

Generate a new ORM model:

```
tina4 generate model Order
```

```
Created src/orm/Order.php
Created migrations/20260322100000_create_orders_table.sql
```

The generated model:

```
<?php
use Tina4\ORM;

class Order extends ORM
{
    public int $id;
    public string $createdAt;
    public string $updatedAt;

    public string $tableName = "orders";
    public string $primaryKey = "id";
}
```

The generated migration:

```
-- UP
CREATE TABLE orders (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT DEFAULT CURRENT_TIMESTAMP
);

-- DOWN
DROP TABLE IF EXISTS orders;
```

Adding Fields

You can specify fields on the command line:

```
tina4 generate model Order --fields "userId:int,total:float,status:string,paid:bool"
```

```
Created src/orm/Order.php
Created migrations/20260322100000_create_orders_table.sql
```

The generated model now includes all the fields:

```
<?php
use Tina4\ORM;

class Order extends ORM
{
    public int $id;
    public int $userId;
    public float $total;
    public string $status;
    public bool $paid;
    public string $createdAt;
    public string $updatedAt;

    public string $tableName = "orders";
    public string $primaryKey = "id";
}
```

And the migration:

```
-- UP
CREATE TABLE orders (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    total REAL NOT NULL DEFAULT 0,
    status TEXT NOT NULL DEFAULT '',
    paid INTEGER NOT NULL DEFAULT 0,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT DEFAULT CURRENT_TIMESTAMP
);

-- DOWN
DROP TABLE IF EXISTS orders;
```

Field Types

CLI Type	PHP Type	SQLite Column
`string`	`string`	`TEXT`
`int`	`int`	`INTEGER`
`float`	`float`	`REAL`
`bool`	`bool`	`INTEGER`
`text`	`string`	`TEXT`
`date`	`string`	`TEXT`

Options

Flag	Description	Example
`--fields`	Comma-separated field definitions	`--fields "name:string,price:float"`
`--auto-crud`	Enable auto-CRUD on the model	`--auto-crud`
`--soft-delete`	Add soft delete support	`--soft-delete`
`--no-migration`	Skip migration generation	`--no-migration`

---

5. tina4 generate route -- CRUD Route Scaffolding

Generate CRUD routes for a model:

```
tina4 generate route Order
```

```
Created src/routes/orders.php
```

The generated route file:

```
<?php
```

```
use Tina4\Router;

Router::group("/api", function () {

    // List all orders
    Router::get("/orders", function ($request, $response) {
        $order = new Order();
        $page = (int) ($request->params["page"] ?? 1);
        $perPage = (int) ($request->params["per_page"] ?? 20);
        $offset = ($page - 1) * $perPage;

        $orders = $order->select("*", "", [], "created_at DESC", $perPage, $offset);
        $results = array_map(fn($o) => $o->toArray(), $orders);

        return $response->json([
            "data" => $results,
            "page" => $page,
            "per_page" => $perPage,
            "count" => count($results)
        ]);
    });

    // Get a single order
    Router::get("/orders/{id:int}", function ($request, $response) {
        $order = new Order();
        $order->load($request->params["id"]);

        if (empty($order->id)) {
            return $response->json(["error" => "Order not found"], 404);
        }

        return $response->json($order->toArray());
    });

    // Create an order
    Router::post("/orders", function ($request, $response) {
        $body = $request->body;
        $order = new Order();
        $order->userId = (int) ($body["user_id"] ?? 0);
        $order->total = (float) ($body["total"] ?? 0);
        $order->status = $body["status"] ?? "";
        $order->paid = (bool) ($body["paid"] ?? false);
        $order->save();

        return $response->json($order->toArray(), 201);
    });

    // Update an order
    Router::put("/orders/{id:int}", function ($request, $response) {
        $order = new Order();
        $order->load($request->params["id"]);

        if (empty($order->id)) {
            return $response->json(["error" => "Order not found"], 404);
        }

        $body = $request->body;
        if (isset($body["user_id"])) $order->userId = (int) $body["user_id"];
        if (isset($body["total"])) $order->total = (float) $body["total"];
```

```
        if (isset($body["status"])) $order->status = $body["status"];
        if (isset($body["paid"])) $order->paid = (bool) $body["paid"];
        $order->save();

        return $response->json($order->toArray());
    });

    // Delete an order
    Router::delete("/orders/{id:int}", function ($request, $response) {
        $order = new Order();
        $order->load($request->params["id"]);

        if (empty($order->id)) {
            return $response->json(["error" => "Order not found"], 404);
        }

        $order->delete();
        return $response->json(null, 204);
    });
});
```

The generator reads the model's properties and creates routes with type casting and null checks. Customize the generated code immediately. It is regular PHP, not magic.

Options

Flag	Description	Example
`--prefix`	Custom route prefix (default: `/api`)	`--prefix /api/v2`
`--middleware`	Add middleware to all routes	`--middleware requireAuth`

---

6. tina4 generate migration -- Migration Scaffolding

Create an empty migration file:

```
tina4 generate migration "add email to orders"
```

```
Created migrations/20260322101500_add_email_to_orders.sql
```

The generated file:

```
-- UP
-- Add your forward migration SQL here


-- DOWN
-- Add your rollback migration SQL here
```

You fill in the SQL yourself:

```
-- UP
ALTER TABLE orders ADD COLUMN email TEXT DEFAULT '';
```

```
-- DOWN
ALTER TABLE orders DROP COLUMN email;
```

The timestamp prefix (`20260322101500`) ensures migrations run in order. Each migration runs once. The framework tracks which ones have been applied.

---

7. tina4 generate middleware -- Middleware Scaffolding

```
tina4 generate middleware rateLimiter
```

```
Created src/middleware/rateLimiter.php
```

The generated file:

```
<?php

function rateLimiter($request, $response, $next)
{
    // Add your middleware logic here

    // Continue to the next middleware or route handler
    return $next($request, $response);

    // Or return early to block the request:
    // return $response->json(["error" => "Rate limit exceeded"], 429);
}
```

The middleware is a named function that you can reference in route definitions:

```
Router::get("/api/data", function ($request, $response) {
    return $response->json(["data" => "protected"]);
}, "rateLimiter");
```

---

8. tina4 doctor -- Environment Health Check

```
tina4 doctor
```

```
Tina4 Doctor - Environment Health Check

  PHP Version ............. 8.3.4       [OK]
  Composer ................ 2.7.2       [OK]
  ext-json ................ loaded      [OK]
  ext-mbstring ............ loaded      [OK]
  ext-openssl ............. loaded      [OK]
  ext-sqlite3 ............. loaded      [OK]
  ext-fileinfo ............ loaded      [OK]
  ext-pdo_sqlite .......... loaded      [OK]
  .env file ............... found       [OK]
  data/ directory ......... writable    [OK]
  logs/ directory ......... writable    [OK]
  secrets/ directory ...... writable    [OK]
  Database connection ..... connected   [OK]
  Routes discovered ....... 14 routes   [OK]
  ORM models discovered .... 5 models    [OK]
  Templates directory ..... found       [OK]
```

```
  tina4.css ................ found       [OK]
  frond.js ................. found       [OK]

  All checks passed. Your environment is ready.
```

If something is wrong, the doctor tells you:

```
  ext-mbstring ............. missing     [FAIL]
    Fix: apt install php8.3-mbstring (Ubuntu)
         brew install php (macOS, includes mbstring)

  data/ directory .......... not writable [FAIL]
    Fix: chmod 755 data/

  Database connection ...... failed      [FAIL]
    Error: could not open database file
    Fix: Check TINA4_DATABASE_URL in .env. Current value: sqlite:///data/app.db
         Make sure the data/ directory exists and is writable.
```

The doctor checks everything a new developer might get wrong. Specific fix instructions for each issue.

---

9. tina4 test -- Running Tests

```
tina4 test
```

```
Running tests...

  ProductTest
    [PASS] test_create_product
    [PASS] test_load_product

  AuthTest
    [PASS] test_login

  3 tests, 3 passed, 0 failed (0.21s)
```

See Chapter 18 for the full testing guide. The CLI command discovers all test files in `tests/` and runs them.

Options

Flag	Description	Example
`--file`	Run a specific test file	`--file tests/ProductTest.php`
`--method`	Run a specific test method	`--method testCreateProduct`
`--verbose`	Show all assertions	`--verbose`

---

10. tina4 routes -- List All Routes

```
tina4 routes
```

```
Method   Path                          Middleware          Auth
------   ----                          ----------          ----
GET      /health                       -                   public
GET      /api/products                 -                   public
GET      /api/products/{id:int}        -                   public
POST     /api/products                 -                   secured
PUT      /api/products/{id:int}        -                   secured
DELETE   /api/products/{id:int}        -                   secured
GET      /api/orders                   -                   public
POST     /api/orders                   requireAuth         secured
GET      /admin                        -                   public
GET      /admin/users                  -                   public
```

Filtering

```
# Filter by HTTP method
tina4 routes --method POST

# Filter by path pattern
tina4 routes --filter orders

# Filter by middleware
tina4 routes --middleware requireAuth
```

When debugging routing issues, check here first. If a route is not matching, `tina4 routes` shows whether it was registered and what middleware is attached.

---

11. tina4 migrate -- Run Database Migrations

```
tina4 migrate
```

```
Running migrations...
  [APPLIED] 20260322100000_create_orders_table.sql
  [APPLIED] 20260322101500_add_email_to_orders.sql
  [SKIPPED] 20260322080000_create_products_table.sql (already applied)
Migrations complete. 2 applied, 1 skipped.
```

Options

Flag	Description	Example
`--rollback`	Undo the last migration	`tina4 migrate --rollback`
`--rollback-all`	Undo all migrations	`tina4 migrate --rollback`
`--status`	Show migration status	`tina4 migrate --status`

Migration Status

```
tina4 migrate --status
```

```
Migration Status:

  [APPLIED]  20260322080000_create_products_table.sql     (applied 2026-03-22 08:15:00)
  [APPLIED]  20260322100000_create_orders_table.sql       (applied 2026-03-22 10:05:00)
  [APPLIED]  20260322101500_add_email_to_orders.sql       (applied 2026-03-22 10:20:00)
  [PENDING]  20260322120000_add_status_to_products.sql    (not yet applied)

  3 applied, 1 pending
```

---

12. Exercise: Scaffold a Complete Feature in 5 Commands

You need to add a "Tasks" feature to your project. Each task has a title, description, priority (string), completed status (boolean), and belongs to a user.

Requirements

Using only CLI commands, do the following:

- Generate the Task model with fields
- Generate CRUD routes for the Task model
- Run the migration to create the table
- Verify the routes are registered
- Run the doctor to make sure everything is healthy

Expected Commands

Complete this in exactly 5 commands. After running them, test the API with curl to verify everything works.

Test with:

```
# Create a task
curl -X POST http://localhost:7145/api/tasks \
  -H "Content-Type: application/json" \
  -d '{"title": "Write chapter 19", "description": "CLI and scaffolding", "priority": "high", "u

# List tasks
curl http://localhost:7145/api/tasks

# Get one task
curl http://localhost:7145/api/tasks/1
```

---

13. Solution

The five commands:

```
# 1. Generate the model with fields (creates model + migration)
tina4 generate model Task --fields "userId:int,title:string,description:text,priority:string,com

# 2. Generate CRUD routes
tina4 generate route Task
```

```
# 3. Run the migration
tina4 migrate

# 4. Verify routes are registered
tina4 routes --filter tasks

# 5. Health check
tina4 doctor
```

Command 1 output:

```
Created src/orm/Task.php
Created migrations/20260322140000_create_tasks_table.sql
```

Command 2 output:

```
Created src/routes/tasks.php
```

Command 3 output:

```
Running migrations...
  [APPLIED] 20260322140000_create_tasks_table.sql
Migrations complete. 1 applied.
```

Command 4 output:

```
Method   Path                          Middleware          Auth
------   ----                          ----------          ----
GET      /api/tasks                    -                   public
GET      /api/tasks/{id:int}           -                   public
POST     /api/tasks                    -                   secured
PUT      /api/tasks/{id:int}           -                   secured
DELETE   /api/tasks/{id:int}           -                   secured
```

Command 5 output:

```
Tina4 Doctor - Environment Health Check
  ...
  All checks passed. Your environment is ready.
```

Test - create a task:

```
curl -X POST http://localhost:7145/api/tasks \
  -H "Content-Type: application/json" \
  -d '{"title": "Write chapter 19", "description": "CLI and scaffolding", "priority": "high", "u
```

```
{
  "id": 1,
  "user_id": 1,
  "title": "Write chapter 19",
  "description": "CLI and scaffolding",
  "priority": "high",
  "completed": false,
  "created_at": "2026-03-22 14:05:00",
  "updated_at": "2026-03-22 14:05:00"
}
```

From zero to a working CRUD API in 5 commands and under 2 minutes.

---

14. Gotchas

1. Model Name Must Be PascalCase

Problem: `tina4 generate model order_item` creates a model class named `order_item` which is not valid PHP.

Cause: The CLI uses the argument as-is for the class name if it cannot determine the PascalCase form.

Fix: Always use PascalCase for model names: `tina4 generate model OrderItem`. The CLI converts it to snake_case for the table name (`order_items`).

2. Migration Already Applied

Problem: You regenerated a model and the migration has the same name as an existing one, but the CLI says it was already applied.

Cause: Tina4 tracks applied migrations by filename. If the filename matches, it considers it already applied.

Fix: Each migration should have a unique timestamp prefix. If you need to modify a table after the initial migration, create a new migration: `tina4 generate migration "add status to tasks"`.

3. Generated Routes Conflict with Auto-CRUD

Problem: You generated routes for a model that has `$autoCrud = true`, and now requests hit the wrong handler.

Cause: Both the generated routes and auto-CRUD routes register at the same paths. The first one registered wins.

Fix: Either use generated routes or auto-CRUD, not both for the same model. If you use `tina4 generate route`, set `$autoCrud = false` on the model (or do not set it at all -- it defaults to false).

4. Port Already in Use

Problem: `tina4 serve` fails with "Address already in use."

Cause: Another process is using port 7145 (or whichever port you configured).

Fix: Find and stop the other process, or use a different port:

```
tina4 serve --port 8080
```

To find what is using the port:

```
lsof -i :7145
```

5. CLI Not Found After Installation

Problem: `tina4: command not found` after running the install script.

Cause: The CLI binary is not in your `PATH`. The install script puts it in `~/.tina4/bin/` which may not be in your shell's PATH.

Fix: Add the binary location to your PATH. For zsh (macOS default):

```
echo 'export PATH="$HOME/.tina4/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

For bash:

```
echo 'export PATH="$HOME/.tina4/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc
```

---

15. Documentation

```
tina4 docs      # Download framework-specific book chapters to .tina4-docs/
tina4 books     # Download the complete Tina4 book (all languages) to tina4-book/
```

`tina4 docs` detects your project language and downloads only the relevant chapters. The documentation is available in Markdown format, optimised for AI tools and local reference.

---

16. Test Port (Dual-Port Development)

When `TINA4_DEBUG=true`, Tina4 automatically starts a second HTTP server on `port + 1000`:

- **Main port** (e.g. 7145) - hot-reload enabled, for AI dev tools
- **Test port** (e.g. 8146) - stable, no hot-reload, for user testing

This prevents the browser from refreshing mid-test when AI tools edit files.

Setting	Effect
`TINA4_NO_AI_PORT=true`	Disables the test port entirely
`TINA4_NO_RELOAD=true`	Disables hot-reload on the main port too
`--no-reload`	CLI flag equivalent of TINA4*NO*RELOAD

Vibe Coding with AI

Why Tina4 is Built for AI-Assisted Development

Tell your AI assistant: "Add a product catalog with search, pagination, and category filtering." In most frameworks, the AI needs to know which packages to install, which config files to create, which naming conventions to follow, and how to wire everything together. It hallucinates half of it.

With Tina4, the AI reads one file -- `CLAUDE.md` -- and knows everything. Every method signature. Every import path. Every convention. It generates correct, runnable code on the first try because there is one way to do things in Tina4.

This is not accidental. Tina4 was designed from the ground up for AI-assisted development.

This chapter explains coding assistants and the `tina4 ai` skills installer. To call a model from your application, see Chapter 40: AI Client.

The Zero-Dependency Advantage

When an AI generates code for Laravel, it might suggest `use Illuminate\Support\Facades\Cache;` or `use Illuminate\Cache\CacheManager;`. Both exist. Both work differently. The AI guesses.

With Tina4, there is one cache. One queue. One ORM. One template engine. No ambiguity. No alternatives. The AI does not need to ask. It knows.

```
// There's only one way to cache in Tina4
$cache = cache_get("products");

// There's only one way to queue in Tina4
$queue = new Queue(topic: "emails");
$queue->push(["to" => "user@test.com"]);

// There's only one way to send email in Tina4
$mail = new Messenger();
$mail->send(to: "user@test.com", subject: "Welcome");
```

Zero dependencies means zero confusion for AI.

CLAUDE.md -- The AI's Instruction Manual

Every Tina4 project includes a `CLAUDE.md` file. It tells AI assistants how the framework works:

- Every method signature with parameters and return types
- The project structure convention (src/routes/, src/orm/, src/templates/)
- The .env variable reference
- Code style rules (routes return `$response()`, ORM uses `toArray()`)

- Database connection string formats
- Common gotchas and how to avoid them

Open a Tina4 project in Claude Code, Cursor, or GitHub Copilot. The AI reads this file first and writes correct Tina4 code from the start.

Skills -- Deep Framework Knowledge

Beyond CLAUDE.md, Tina4 ships with three AI skill files:

tina4-maintainer

For framework maintenance -- porting features between languages, reviewing PRs, running benchmarks, checking parity.

tina4-developer

For application development -- creating routes, defining models, writing templates, setting up auth, configuring queues.

tina4-js

For frontend development -- tina4-js signals, Tina4Element components, reactive templates, WebSocket client.

These skills install when AI tools are detected:

```
// In your project root
AI::installContext('.'); // Detects Claude, Cursor, Copilot, etc.
```

The Convention Advantage

AI thrives on convention. When every Tina4 project follows the same structure, the AI never asks "where should I put this?"

```
src/
  routes/hello.php     → AI knows: this is a route file
  orm/Product.php      → AI knows: this is an ORM model
  templates/home.twig  → AI knows: this is a template
  middleware/Auth.php   → AI knows: this is middleware
```

Tell the AI "add a User model." It creates `src/orm/User.php` with the correct field definitions, the correct namespace, the correct method stubs. Every time.

Real-World Vibe Coding Session

An actual conversation with Claude Code in a Tina4 PHP project:

You: "Add a blog with posts and comments. Posts belong to users. Comments belong to posts. I want a REST API and a template page that lists posts."

Claude Code generates:

- `src/orm/Post.php` -- Model with hasMany comments, belongsTo user
- `src/orm/Comment.php` -- Model with belongsTo post
- `src/routes/api/posts.php` -- Full CRUD API (list, get, create, update, delete)
- `src/routes/api/comments.php` -- Nested comments API
- `src/routes/blog.php` -- Template route for the blog page
- `src/templates/blog.twig` -- Page with tina4css cards for each post
- `migrations/20260322_create_posts_and_comments.sql` -- Database schema

All correct. All runnable. First try.

You: "Add pagination to the posts list"

Claude adds `$request->params['page']` handling and `toArray(page: $page, perPage: 10)` -- because CLAUDE.md told it how Tina4 pagination works.

You: "Cache the post list for 5 minutes"

Claude adds `cache_get("posts_page_$page")` with `cache_set("posts_page_$page", $data, 300)` -- because it knows the cache API.

You: "Send an email when a new comment is posted"

Claude adds `$mail = new Messenger(); $mail->send(...)` -- because it knows Messenger reads from .env.

No research. No Stack Overflow. Describe what you want. The AI builds it.

Supported AI Tools

Tina4 auto-detects and installs context for eight tools. The exact list lives in the framework's `Tina4\Ai` class:

Tool	Detection (context file)	Context installed
Claude Code	`CLAUDE.md`	CLAUDE.md + `.claude/` skills
Cursor	`.cursorules`	`.cursorules` (+ `.cursor/`)
GitHub Copilot	`.github/copilot-instructions.md`	`.github/copilot-instructions.md`
Windsurf	`.windsurfrules`	`.windsurfrules`
Aider	`CONVENTIONS.md`	`CONVENTIONS.md`

Cline	`.clinerules`	`.clinerules`
OpenAI Codex	`AGENTS.md`	`AGENTS.md`
Google Antigravity	`.antigravity/context.md`	`.antigravity/context.md`

Run `tina4 ai` to see which tools are detected in your project and install context files via the menu.

The AI Chat in Dev Dashboard

The dev dashboard at `/__dev` includes an AI chat tab. By default it talks to a local **qwen2.5-coder** model served via Ollama, grounded by Tina4's built-in RAG index of the framework source. Nothing leaves your machine.

Configure it in `.env`:

```
TINA4_AI_URL=http://localhost:11434      # Ollama HTTP endpoint
TINA4_AI_MODEL=qwen2.5-coder             # Default coding model
TINA4_RAG_URL=http://localhost:11434     # RAG embedding endpoint (defaults to TINA4_AI_URL)
```

If you prefer a remote provider, point `TINA4_AI_URL` at any OpenAI-compatible endpoint and set `TINA4_AI_MODEL` to the model name that endpoint serves.

The AI has full context of your Tina4 project -- routes, models, templates, configuration. Ask it:

- "Why is my /api/users route returning 500?"
- "How do I add WebSocket support?"
- "Write a migration to add an 'avatar' column to users"

The "Ask about this error" button on the error overlay sends the full stack trace and source code to the AI for instant debugging.

Tips for Effective Vibe Coding with Tina4

- **Be specific about what you want** -- "Add a product API with search by name and price range" works better than "add products"
- **Let the AI use tina4 generate** -- "Run tina4 generate model Product then add price and category fields" gives the AI a starting point
- **Reference the .env** -- "Configure the queue to use RabbitMQ" -- the AI knows to update .env with `TINA4_QUEUE_BACKEND=rabbitmq`
- **Ask for tests** -- "Write tests for the Product API" -- the AI uses Tina4's inline testing framework
- **Ask for the gallery** -- "Deploy the auth gallery example" -- the AI clicks Try It and you have a working JWT demo

- **Review, do not rewrite** -- The AI's first attempt is usually 90% correct. Tweak the details instead of starting over.

Exercise

Open your Tina4 PHP project in Claude Code (or your preferred AI tool) and try these prompts:

- "Add a Contact model with name, email, phone, and message fields"
- "Create a contact form page at /contact with a Twig template using tina4css"
- "Add an API endpoint that accepts the form submission and saves it to the database"
- "Send an email notification when a new contact is submitted"
- "Add rate limiting to the contact form -- max 3 submissions per minute"

Each prompt should generate correct, runnable code on the first try. If it does not, check that CLAUDE.md is in your project root.

Gotchas

- **No CLAUDE.md?** Run `tina4 init` or `AI::installContext('.')` to generate it
- **AI suggests wrong import paths?** The CLAUDE.md might be outdated -- regenerate it with `AI::installContext('.', force: true)`
- **AI hallucinates a package?** Remind it: "Tina4 has zero dependencies, use only built-in features"
- **AI creates files in wrong location?** Tell it: "Follow the src/routes, src/orm, src/templates convention"
- **AI code does not run?** Check that routes return `$response->json()` not `echo` -- Tina4 convention matters

---

The Philosophy

Tina4 is not just compatible with AI coding tools. It was designed to make AI-assisted development effortless.

Convention over configuration means the AI always knows where things go. Zero dependencies means the AI never chooses between packages. A single CLAUDE.md file means the AI has complete framework knowledge. Identical APIs across 4 languages means the AI's knowledge transfers instantly.

The future of web development is collaborative. You describe the intent. The AI writes the code. You review and refine. Tina4 is built for that future.

The Intelligent Native Application 4ramework. Built for AI. Built for you.

Environment Variables

> **■■ *BREAKING CHANGE - Tina4 v3.12.0***
>
> *Every framework env var now requires the `TINA4_` prefix. The legacy un-prefixed names (`DATABASE_URL`, `SECRET`, `SMTP_HOST`, `HOST_NAME`, etc.) no longer work. Setting them at startup makes the framework refuse to boot with a list of renames.*
>
> *Run `tina4 env --migrate` to rewrite your existing `.env` automatically, or rename manually using the table below. The runtime guard prints the same mapping if it detects legacy names.*
>
> ***Conventional names stay un-prefixed:*** `PORT`, `HOST`, `NODE_ENV`, `RACK_ENV`, `RUBY_ENV`, `ENVIRONMENT`. *These are runtime/PaaS conventions, not framework config.*

Tina4 is configured through environment variables, read from `.env` at the project root. Every variable has a sensible default; most projects set three or four values and leave the rest alone.

This chapter lists every variable the PHP framework reads, grouped by subsystem. Start with the minimum-config examples at the end, then come back here when you need to tune something specific.

Core Server

Variable	Default	Description
`HOST`	`0.0.0.0`	Bind address. `0.0.0.0` listens on every interface. `127.0.0.1` restricts to localhost.
`TINA4_HOST`	`0.0.0.0`	Tina4-specific bind override consulted by `App::serve()` and `Server` when no `$host` argument is passed. Wins over the conventional `HOST` for the framework's own server.
`PORT`	`7145`	HTTP server port. The Rust CLI prefers `TINA4_PORT` but falls back to `PORT`.
`TINA4_PORT`	*(inherits `PORT`)*	Explicit Tina4-specific port override. Takes precedence over `PORT` when both are set.
`TINA4_WS_PORT`	*(inherits port)*	Separate port for the WebSocket server. Leave unset to share the HTTP port.
`TINA4_HOST_NAME`	`localhost:7145`	Fully-qualified host used in generated absolute URLs (Swagger, OAuth redirects, emails).
`TINA4_DEBUG`	`false`	Master debug toggle. Enables Swagger UI, dev dashboard, live reload, template dump filter, error overlay. Never set to `true` in production.
`TINA4_LOG_LEVEL`	`INFO`	Console severity threshold. Options: `ALL`, `DEBUG`, `INFO`, `WARNING`, `ERROR`, `CRITICAL`, `NONE`.
`TINA4_NO_BROWSER`	`false`	Stops `tina4 serve` from opening your browser on every restart. Recommended during active development.

`TINA4_NO_RELOAD`	`false`	Disables the dev hot-reload signal from the Rust CLI. Use when you want a stable server for debugging.
`TINA4_SUPPRESS`	`false`	Hides the Tina4 startup banner. Useful in CI and systemd units where stdout is ingested.
`TINA4_VERSION`	*(framework)*	Override the version string reported by `/__dev/api/system`. Mostly for testing.
`TINA4_CLI_SERVE`	*(none)*	Set internally by the Rust CLI to signal managed mode. Do not set manually.
`TINA4_PUBLIC_DIR`	*(empty)*	Override directory served as static files at `/`. When unset, the static-file middleware searches `src/public/` and the framework's bundled public assets.
`TINA4_TEMPLATE_ROUTING`	`on`	Auto-routing of Twig templates from `src/templates/`. Set to `off`, `false`, `0`, `no`, or `disabled` to require explicit `Router::get()` for every URL.
`TINA4_ALLOW_LEGACY_ENV`	`false`	Bypass the v3.12 boot guard that rejects un-prefixed legacy env vars (`DATABASE_URL`, `SECRET`, `SMTP_HOST`, etc.). Use only in CI / migration scripts during the transition window.

`TINA4_ENV_FILE`	`.env`	Alternate path for the dotenv file loaded at boot. Resolved relative to the project root unless absolute. Useful for per-environment files (e.g. `.env.production`) without symlinking.
`TINA4_HEALTH_PATH`	`/health`	Path the built-in health-check route registers under. Set `/__health` to align with the v3 documented alias used by the other frameworks; the default keeps `/health` for back-compat with existing probes.

Routing

Variable	Default	Description
`TINA4_TRAILING_SLASH_REDIRECT`	`false`	When truthy, any request path ending in `/` (other than the bare root) is 301-redirected to the slash-stripped form. The query string is preserved. Lets you normalise URLs without writing per-route handlers.

Secrets and Authentication

Variable	Default	Description
`TINA4_SECRET`	*(empty)*	JWT signing secret. Must be long, random, and unique per environment. **Never commit.**
`TINA4_JWT_ALGORITHM`	`HS256`	JWT signing algorithm. Supports `HS256`, `HS384`, `HS512`.
`TINA4_TOKEN_LIMIT`	`60`	JWT token lifetime in minutes.
`TINA4_API_KEY`	*(empty)*	Static API key used by `Auth::validateApiKey()` as a fallback to JWT.

Database

Variable	Default	Description
`TINA4_DATABASE_URL`	`sqlite:///data/app.db`	Connection URL. Scheme selects the driver: `sqlite`, `postgres`, `mysql`, `mssql`, `sqlserver`, `firebird`.
`TINA4_DATABASE_USERNAME`	*(empty)*	Overrides the username embedded in `TINA4_DATABASE_URL`.
`TINA4_DATABASE_PASSWORD`	*(empty)*	Overrides the password embedded in `TINA4_DATABASE_URL`.
`TINA4_DATABASE_FIREBIRD_PATH`	*(empty)*	Overrides the database path/alias parsed from `TINA4_DATABASE_URL` for Firebird. Useful for Windows backslash paths and split-config setups.
`TINA4_DATABASE_URL`	*(empty)*	Legacy alias for `TINA4_DATABASE_URL`. Prefer `TINA4_DATABASE_URL` in new projects.
`TINA4_AUTOCOMMIT`	`true`	Standalone writes auto-commit on their own connection (durable + visible across a pool); explicit transactions stay atomic. Set `false` for strict manual-commit mode.
`TINA4_DB_CACHE`	`false`	Enables in-memory query-result caching for read queries.
`TINA4_DB_CACHE_TTL`	`60`	Query cache TTL in seconds when `TINA4_DB_CACHE=true`.

`TINA4_DB_POOL`	`0`	Default connection-pool size used when a caller constructs `Database` without passing `$pool`. `0` keeps the single-connection behaviour; values above zero enable pooled adapters lazily. An explicit constructor argument always wins.
`TINA4_MIGRATION_ID`	*(timestamp)*	Override the migration ID used when recording applied migrations.

CORS

Variable	Default	Description
`TINA4_CORS_ORIGINS`	*(empty)*	Comma-separated allowed origins. Empty denies every cross-origin request; set the origins your front end is served from. `*` allows any origin and has to be set explicitly.
`TINA4_CORS_METHODS`	`GET,POST,PUT,PATCH,DELETE,OPTIONS`	Allowed request methods.
`TINA4_CORS_HEADERS`	`Content-Type,Authorization,X-Requested-With`	Allowed request headers.
`TINA4_CORS_CREDENTIALS`	`false`	Send `Access-Control-Allow-Credentials: true`. Required for cross-origin cookies.
`TINA4_CORS_MAX_AGE`	`600`	Preflight cache lifetime in seconds.

Security Headers

Variable	Default	Description
`TINA4_CSP`	`default-src 'self'`	`Content-Security-Policy` header.
`TINA4_CSRF`	`true`	CSRF token validation on POST/PUT/PATCH/DELETE. Requires `_csrf` in the body or `X-CSRF-Token` header.
`TINA4_HSTS`	*(empty/off)*	`Strict-Transport-Security` max-age in seconds. Set `31536000` in production with HTTPS.
`TINA4_FRAME_OPTIONS`	`DENY`	`X-Frame-Options` header. Set `SAMEORIGIN` if you embed your own app in an iframe.
`TINA4_REFERRER_POLICY`	`strict-origin-when-cross-origin`	`Referrer-Policy` header.
`TINA4_PERMISSIONS_POLICY`	*(empty)*	`Permissions-Policy` header. Example: `geolocation=(), microphone=()`.

Rate Limiting

Variable	Default	Description
`TINA4_RATE_LIMIT`	`100`	Maximum requests per window per IP. Set `0` to disable.
`TINA4_RATE_WINDOW`	`60`	Rate-limit window in seconds.

Sessions

Variable	Default	Description
`TINA4_SESSION_BACKEND`	`file`	Storage backend. Options: `file`, `redis`, `valkey`, `mongo`, `database`.
`TINA4_SESSION_HANDLER`	*(inherits `_BACKEND`)*	Alternate handler class name. Overrides `TINA4_SESSION_BACKEND`.
`TINA4_SESSION_NAME`	`tina4_session`	Cookie name written by the framework session manager. Distinct from `TINA4_PHP_SESSION_NAME`, which controls the native PHP session cookie.
`TINA4_SESSION_TTL`	`3600`	Session expiry in seconds.
`TINA4_SESSION_SAMESITE`	`Lax`	SameSite cookie attribute. Options: `Strict`, `Lax`, `None`.
`TINA4_SESSION_HTTPONLY`	`true`	Emit the `HttpOnly` attribute on the session cookie. Leave on unless JavaScript genuinely needs to read the cookie.
`TINA4_SESSION_SECURE`	`false`	Emit the `Secure` attribute on the session cookie. Setting `TINA4_SESSION_SAMESITE=None` forces this to `true` regardless of the value here.
`TINA4_SESSION_PATH`	`data/sessions`	Filesystem path for the file backend.
`TINA4_PHP_SESSION_NAME`	`PHPSESSID`	Cookie name used by native PHP `$_SESSION` (separate from the framework session).
`TINA4_PHP_SESSION_PATH`	*(system temp)*	`session.save_path` for native PHP sessions. The framework configures it before `session_start()` so the SAPI and the built-in server share session storage.

Redis/Valkey session backend

Variable	Default	Description
`TINA4_SESSION_REDIS_HOST`	`localhost`	Redis host.
`TINA4_SESSION_REDIS_PORT`	`6379`	Redis port.
`TINA4_SESSION_REDIS_PASSWORD`	*(none)*	Redis auth password.
`TINA4_SESSION_REDIS_DB`	`0`	Redis database number.
`TINA4_SESSION_REDIS_URL`	*(none)*	Full `redis://` URL. Overrides the individual fields when set.
`TINA4_SESSION_VALKEY_HOST`	`localhost`	Valkey host.
`TINA4_SESSION_VALKEY_PORT`	`6379`	Valkey port.
`TINA4_SESSION_VALKEY_PASSWORD`	*(none)*	Valkey auth password.
`TINA4_SESSION_VALKEY_DB`	`0`	Valkey database number.

MongoDB session backend

Variable	Default	Description
`TINA4_SESSION_MONGO_URL`	`mongodb://localhost:27017`	MongoDB connection string.
`TINA4_SESSION_MONGO_DB`	`tina4`	MongoDB database name.

Templates

Variable	Default	Description
`TINA4_TEMPLATE_CACHE_TTL`	`0`	Lifetime of compiled Frond templates in production, in seconds. `0` means cache forever (no filesystem checks per render); a positive value re-reads and re-tokenises a template once it has been cached for that long. Ignored when `TINA4_DEBUG=true`; debug mode always re-reads.

Cache

Variable	Default	Description
`TINA4_CACHE_BACKEND`	`memory`	Response cache backend. Options: `memory`, `file`, `redis`, `valkey`, `memcached`, `mongodb`, `database`. Falls back to `file` if the configured backend is unreachable.
`TINA4_CACHE_DIR`	`data/cache`	Cache directory for the file backend.
`TINA4_CACHE_TTL`	`60`	Default cache TTL in seconds.
`TINA4_CACHE_MAX_ENTRIES`	`1000`	Maximum cache entries. Oldest entries evicted first.
`TINA4_CACHE_URL`	*(none)*	Connection URL for remote cache backends (e.g. `redis://localhost:6379`). For `database`, falls back to `TINA4_DATABASE_URL` when unset.
`TINA4_CACHE_USERNAME`	*(none)*	Username for the cache backend. May also be embedded in `TINA4_CACHE_URL`.
`TINA4_CACHE_PASSWORD`	*(none)*	Password for the cache backend. May also be embedded in `TINA4_CACHE_URL` (e.g. `redis://:pass@host`). Memcached is unauthenticated.

Queues

Variable	Default	Description
`TINA4_QUEUE_BACKEND`	`file`	Queue backend. Options: `file`, `kafka`, `rabbitmq`, `mongo`, `database`.
`TINA4_QUEUE_PATH`	`data/queue`	Filesystem path for the file backend.
`TINA4_QUEUE_URL`	*(none)*	Connection URL for remote backends.

Kafka queue backend

Variable	Default	Description
`TINA4_KAFKA_BROKERS`	`localhost:9092`	Comma-separated broker list.
`TINA4_KAFKA_GROUP_ID`	`tina4_consumer_group`	Kafka consumer group ID.

RabbitMQ queue backend

Variable	Default	Description
`TINA4_RABBITMQ_HOST`	`localhost`	RabbitMQ host.
`TINA4_RABBITMQ_PORT`	`5672`	RabbitMQ port.
`TINA4_RABBITMQ_USERNAME`	`guest`	RabbitMQ username.
`TINA4_RABBITMQ_PASSWORD`	`guest`	RabbitMQ password.
`TINA4_RABBITMQ_VHOST`	`/`	RabbitMQ virtual host.

MongoDB queue backend

Variable	Default	Description
`TINA4_MONGO_URI`	*(none)*	Full MongoDB connection string. Overrides host/port when set.
`TINA4_MONGO_HOST`	`localhost`	MongoDB host.
`TINA4_MONGO_PORT`	`27017`	MongoDB port.
`TINA4_MONGO_USERNAME`	*(none)*	MongoDB username.
`TINA4_MONGO_PASSWORD`	*(none)*	MongoDB password.
`TINA4_MONGO_DB`	`tina4`	MongoDB database name.
`TINA4_MONGO_COLLECTION`	`tina4_queue`	MongoDB collection name for jobs.

WebSocket Backplane

Variable	Default	Description
`TINA4_WS_BACKPLANE`	*(none)*	Backplane type. Set `redis` for multi-instance broadcasts.
`TINA4_WS_BACKPLANE_URL`	`redis://localhost:6379`	Connection URL for the backplane.

Email

Variable	Default	Description
`TINA4_MAIL_HOST`	*(none)*	SMTP server hostname.
`TINA4_MAIL_PORT`	`587`	SMTP server port.
`TINA4_MAIL_USERNAME`	*(none)*	SMTP authentication username.
`TINA4_MAIL_PASSWORD`	*(none)*	SMTP authentication password.
`TINA4_MAIL_FROM`	*(none)*	Default sender email address.
`TINA4_MAIL_FROM_NAME`	*(none)*	Default sender display name.
`TINA4_MAIL_ENCRYPTION`	`tls`	Connection encryption. Options: `tls`, `ssl`, `none`.
`TINA4_MAIL_IMAP_HOST`	*(none)*	IMAP server for inbound mail.
`TINA4_MAIL_IMAP_PORT`	`993`	IMAP server port.
`TINA4_MAIL_IMAP_USERNAME`	*(none)*	IMAP authentication username.
`TINA4_MAIL_IMAP_PASSWORD`	*(none)*	IMAP authentication password.
`TINA4_MAIL_IMAP_ENCRYPTION`	`tls`	IMAP transport encryption. Accepts `tls`, `starttls`, or `none`; any other value is silently coerced back to `tls`. Independent of `TINA4_MAIL_ENCRYPTION` so Gmail-style setups can use IMAPS on 993 alongside SMTP STARTTLS on 587.
`TINA4_MAIL_CAPTURE`	`false`	Force local DevMailbox capture and skip SMTP, even when an SMTP host exists. A missing host also captures.
`TINA4_MAIL_REDIRECT_TO`	*(none)*	Comma-separated safety recipients for real SMTP delivery. Replaces To, Cc, and Bcc.

`TINA4_MAILBOX_DIR`	`data/mailbox`	Storage directory for captured mail. This setting does not enable or disable capture.

> `TINA4_DEBUG` *does not suppress email. On staging, set* `TINA4_MAIL_CAPTURE=true` *to keep every message local, or set* `TINA4_MAIL_REDIRECT_TO=qa@example.com,product@example.com` *to exercise SMTP without reaching original recipients. Capture takes precedence when both are set.*
>
> `TINA4_MAIL_HOST`, `TINA4_MAIL_PORT`, `TINA4_MAIL_USERNAME`, `TINA4_MAIL_PASSWORD` *are also accepted as aliases for the* `TINA4_MAIL_*` *equivalents. New projects should use the* `TINA4_MAIL_*` *names.*

Logging

Variable	Default	Description
`TINA4_LOG_LEVEL`	`INFO`	Console severity threshold. Options: `ALL`, `DEBUG`, `INFO`, `WARNING`, `ERROR`, `CRITICAL`, `NONE`.
`TINA4_LOG_FILE_LEVEL`	`ALL`	Independent severity threshold for the file sink.
`TINA4_LOG_STRICT`	`false`	Raise when a configured log sink fails instead of continuing with the remaining sinks. Accepts only `true` or `false`.
`TINA4_LOG_DEBUG`	`0`	Numeric flag for debug-level messages. Used internally by `Debug::message()`.
`TINA4_LOG_INFO`	`1`	Numeric flag for info-level messages.
`TINA4_LOG_ERROR`	`3`	Numeric flag for error-level messages.
`TINA4_LOG_MAX_SIZE`	`10485760`	Per-file log size limit in bytes (10 MB). Rotated when exceeded.
`TINA4_LOG_KEEP`	`5`	Number of rotated log files to retain.

Log pipeline (sink, format, rotation)

Logs default to stdout. Set `TINA4_LOG_OUTPUT=file` plus `TINA4_LOG_FILE=app.log` to write to disk; the framework rotates at `TINA4_LOG_ROTATE_SIZE` bytes and keeps `TINA4_LOG_ROTATE_KEEP` backups.

Variable	Default	Description
`TINA4_LOG_FILE`	*(empty = stdout only)*	Primary log filename. A relative value is joined with `TINA4_LOG_DIR`; an absolute path overrides both directory and filename. Leave empty to skip file output entirely.
`TINA4_LOG_DIR`	`logs`	Directory log files are written into when `TINA4_LOG_OUTPUT` includes `file`. Created on first write if missing.
`TINA4_LOG_FORMAT`	`text`	Wire format for emitted lines. `text` is human-readable; `json` emits one structured object per line for ingestion by Loki, ELK, etc.
`TINA4_LOG_OUTPUT`	`stdout`	Where lines are sent. `stdout` writes to the process stream (great for systemd / containers), `file` writes only to `TINA4_LOG_FILE`, `both` does both.
`TINA4_LOG_CRITICAL`	`false`	Enable `Log::critical()` emission. Off by default so security-relevant criticals can be intentionally surfaced rather than buried in routine output.
`TINA4_LOG_ROTATE_SIZE`	`10485760`	Rotation threshold in bytes (10 MB default). Set `0` to disable rotation entirely, useful when an external tool (logrotate, Docker driver) owns the file.

`TINA4_LOG_ROTATE_KEEP`	`5`	Number of rotated backups to retain. Older files are pruned on rotation.

Uploads

Variable	Default	Description
`TINA4_MAX_UPLOAD_SIZE`	`10485760`	Maximum multipart upload size in bytes (10 MB).

Localisation

Variable	Default	Description
`TINA4_LOCALE`	`en`	Default locale for `I18n`.
`TINA4_LOCALE_DIR`	`src/locale`	Directory containing locale JSON files.

Services (background tasks)

Variable	Default	Description
`TINA4_SERVICE_DIR`	`src/services`	Directory scanned for service classes.
`TINA4_SERVICE_SLEEP`	`1`	Default tick interval (seconds) when a service does not specify one.

Application AI Client and Developer AI Tools

The app-facing `Ai` client and the developer dashboard share the `TINA4_AI_*` namespace. The client supports local OpenAI-compatible servers, OpenAI, and Anthropic. See Chapter 40: AI Client for code examples.

Variable	Default	Description

`TINA4_AI_PROVIDER`	`local`	Provider used by the app-facing client: `local`, `openai`, or `anthropic`.
`TINA4_AI_URL`	Provider-specific	Base URL or full endpoint. Local defaults to `http://localhost:11437`; OpenAI and Anthropic use their public API bases.
`TINA4_AI_MODEL`	Provider-specific	Default model. Local uses `llama3.2`, OpenAI uses `gpt-4o-mini`, and Anthropic uses `claude-3-5-haiku-latest`.
`TINA4_AI_KEY`	*(none)*	Required before an OpenAI or Anthropic request. Local requests need no key.
`TINA4_AI_TIMEOUT`	`60`	Total request deadline in seconds, including retries and response reads.
`TINA4_AI_CONNECT_TIMEOUT`	`10`	Connection-establishment deadline in seconds.
`TINA4_AI_MAX_RETRIES`	`2`	Retries before output for connection failures, HTTP 429, and HTTP 5xx.
`TINA4_EMBED_URL`	*(inherits `TINA4_AI_URL`)*	Optional embedding base URL or full endpoint.
`TINA4_RAG_URL`	`http://localhost:11438`	RAG service used by the developer dashboard.
`TINA4_RAG_TOPK`	`4`	Number of RAG matches returned per query.
`TINA4_VISION_URL`	`http://localhost:11437/api/chat`	Vision endpoint used by developer tools, not the app-facing AI client.
`TINA4_IMAGE_URL`	`http://localhost:11437/api/generate`	Image endpoint used by developer tools, not the app-facing AI client.
`TINA4_SUPERVISOR_URL`	`http://localhost:9999`	Rust supervisor URL used by developer tools.

`TINA4_MCP_REMOTE`	`false`	Allows MCP to bind beyond localhost. Do not enable it on a public production interface.
`TINA4_NO_AI_PORT`	`false`	Disables the developer AI port listener.
`TINA4_OVERRIDE_CLIENT`	`false`	Lets the framework start without the Rust client in containers and CI.

Swagger / OpenAPI

Variable	Default	Description
`TINA4_SWAGGER_TITLE`	`Tina4 API`	OpenAPI spec title shown in the Swagger UI.
`TINA4_SWAGGER_DESCRIPTION`	`Auto-generated from Tina4 routes`	OpenAPI spec description.
`TINA4_SWAGGER_VERSION`	`1.0.0`	OpenAPI spec version.
`TINA4_SWAGGER_ENABLED`	*(inherits `TINA4_DEBUG`)*	Master switch for Swagger UI and the `/swagger`/`/swagger.json` routes. When unset, follows `TINA4_DEBUG` so Swagger is on in dev and off in prod automatically. Set explicitly to expose docs in production.
`TINA4_SWAGGER_CONTACT_EMAIL`	*(empty)*	Contact email written into the generated OpenAPI `info.contact` block.
`TINA4_SWAGGER_LICENSE`	*(empty)*	License name written into the generated OpenAPI `info.license` block.

GraphQL

Variable	Default	Description
`TINA4_GRAPHQL_ENDPOINT`	`/graphql`	URL path the framework binds the GraphQL handler to. The same value is used when generating `__schema` links and the dev-dashboard query console URL.
`TINA4_GRAPHQL_AUTO_SCHEMA`	`true`	Auto-generate the GraphQL schema from registered ORM models. Set to `false` to skip introspection scaffolding when you supply a hand-written schema.

HTTP Status Constants

For use in route handlers instead of raw integers:

```
return $response->json($data, \Tina4\HTTP_CREATED);
return $response("<error/>", \Tina4\HTTP_BAD_REQUEST, \Tina4\APPLICATION_XML);
```

See Chapter 3: Request and Response for the full table.

Log-Level Constants

Passed to `Debug::message()` to tag severity:

Constant	Description
`TINA4_LOG_DEBUG`	Verbose developer messages.
`TINA4_LOG_INFO`	Normal operational messages.
`TINA4_LOG_WARNING`	Non-fatal anomalies.
`TINA4_LOG_ERROR`	Recoverable errors.
`TINA4_LOG_CRITICAL`	Fatal or security-relevant events.

```
\Tina4\Debug::message("User " . $id . " missed the cache", TINA4_LOG_INFO);
```

Configuration Recipes

The tables above list every knob. These are the setups most apps actually reach for, ready to paste into `.env`. Each block sets only what the feature needs. Everything else keeps its default.

PostgreSQL in production

One URL points the ORM, the migrations, and the query builder at Postgres. Credentials can ride in the URL or sit in their own variables, which keeps the password out of your shell history.

```
TINA4_DATABASE_URL=postgresql://localhost:5432/myapp
TINA4_DATABASE_USERNAME=myapp
TINA4_DATABASE_PASSWORD=changeme
TINA4_DB_POOL=4
```

`TINA4_DB_POOL=4` opens four connections and rotates across them. Leave it at `0` for a single connection on a small app.

A shared cache on Redis

The response cache and the cross-request query cache both speak to the same Redis. Point them at it and every instance shares one cache, invalidated globally on every write.

```
TINA4_CACHE_BACKEND=redis
TINA4_CACHE_URL=redis://localhost:6379
TINA4_DB_CACHE=true
TINA4_DB_CACHE_BACKEND=redis
TINA4_DB_CACHE_URL=redis://localhost:6379
```

If Redis is down or the driver is missing, the cache logs a warning and falls back to the file backend. It never silently stops caching.

Sessions that survive more than one box

The file backend is fine for a single server. Move sessions to Redis the moment you run more than one instance, so a user stays logged in whichever instance answers the next request.

```
TINA4_SESSION_BACKEND=redis
TINA4_SESSION_REDIS_HOST=localhost
TINA4_SESSION_REDIS_PORT=6379
TINA4_SESSION_SECURE=true
TINA4_SESSION_SAMESITE=Strict
```

`TINA4_SESSION_SECURE=true` keeps the cookie off plain HTTP. Turn it on once you have TLS.

A queue on RabbitMQ or Kafka

One URL is enough for RabbitMQ; the per-field variables only exist for split configs. The queue API stays identical whichever backend you pick.

```
TINA4_QUEUE_BACKEND=rabbitmq
TINA4_QUEUE_URL=amqp://guest:guest@localhost:5672/
```

Kafka reads a broker list instead:

```
TINA4_QUEUE_BACKEND=kafka
TINA4_KAFKA_BROKERS=localhost:9092
```

```
TINA4_KAFKA_GROUP_ID=myapp_workers
```

WebSocket broadcasts across instances

A single server broadcasts in memory. Add a backplane and a message sent on one instance reaches clients connected to every other instance.

```
TINA4_WS_BACKPLANE=redis
TINA4_WS_BACKPLANE_URL=redis://localhost:6379
TINA4_WS_ALLOWED_ORIGINS=https://myapp.com
```

Set the origin allow-list in production. Empty allows every origin, which is fine in dev and risky on the public internet.

Locked-down production headers

The defaults are already safe. These four tighten the screws for a public site on HTTPS.

```
TINA4_CORS_ORIGINS=https://myapp.com,https://www.myapp.com
TINA4_HSTS=31536000
TINA4_FRAME_OPTIONS=DENY
TINA4_SESSION_SECURE=true
```

Never pair `TINA4_CORS_CREDENTIALS=true` with `TINA4_CORS_ORIGINS=*`. Name your real origins instead.

The dev dashboard AI, kept local

The dashboard AI talks to a local model through Ollama by default, so nothing leaves your machine. Point the URLs elsewhere only when you run the hosted Tina4 AI services.

```
TINA4_AI_URL=http://localhost:11437/api/chat
TINA4_AI_MODEL=qwen2.5-coder:14b
TINA4_RAG_URL=http://localhost:11438
```

Minimal `.env` for Development

```
TINA4_DEBUG=true
TINA4_LOG_LEVEL=DEBUG
TINA4_NO_BROWSER=true
```

That is it. Debug mode lights up the Swagger UI, the dev dashboard, detailed error pages, and live reload. Keeping the browser flag on stops a new tab opening every time you save a file.

Minimal `.env` for Production

```
TINA4_SECRET=your-long-random-secret-here
TINA4_DATABASE_URL=postgresql://user:password@db-host:5432/myapp
TINA4_CORS_ORIGINS=https://myapp.com,https://www.myapp.com
TINA4_HSTS=31536000
TINA4_MAIL_HOST=smtp.example.com
TINA4_MAIL_PORT=587
TINA4_MAIL_USERNAME=noreply@myapp.com
TINA4_MAIL_PASSWORD=your-smtp-password
TINA4_MAIL_FROM=noreply@myapp.com
```

No `TINA4_DEBUG`. It defaults to `false`, which is what you want in production. Set a real secret, a real database, locked-down CORS origins, HSTS, and SMTP credentials if you send email. Everything else has a production-appropriate default.

Feature Module Settings

These settings are read by focused framework modules rather than the server bootstrap. They remain part of the public configuration surface.

Variable	Default	Description
`TINA4_ENV`	`development`	Runtime environment name. Set `production` to disable development-only behaviour.
`TINA4_FEEDBACK_DEV_USER`	*(empty)*	Local-development identity for testing the feedback widget without authentication. Never use this as production authentication.
`TINA4_SERVE_FORK`	`true` when supported	Set `false` to keep request handling in one process for a debugger or profiler. Systems without `pcntl` and `posix` remain serial.
`TINA4_SESSION_MEMCACHED_HOST`	`localhost`	Memcached host for the session backend.
`TINA4_SESSION_MEMCACHED_PORT`	`11211`	Memcached port for the session backend.
`TINA4_SESSION_MEMCACHED_PREFIX`	`tina4:session:`	Key prefix for Memcached sessions. Give each application a distinct prefix when sharing a server.
`TINA4_SESSION_VALKEY_PREFIX`	`tina4:session:`	Key prefix for Valkey sessions.

`TINA4_SWAGGER_CONTACT_TEAM`	*(empty)*	Contact name placed in the generated OpenAPI `info.contact` object.
`TINA4_SWAGGER_CONTACT_URL`	*(empty)*	Contact URL placed in the generated OpenAPI `info.contact` object.
`TINA4_TRUSTED_PROXIES`	*(empty)*	Comma-separated IP addresses and CIDR ranges allowed to supply forwarding headers. Empty trusts no proxy.

Deployment

1. From Development to Production

The app works on `localhost:7145`. Now it needs to run around the clock on a real server. Handle thousands of concurrent users. Survive restarts. Hold steady on memory. The gap between "works on my machine" and "works in production" is where projects stumble.

This chapter covers everything for a production deployment: environment configuration, Docker packaging, web server setup, SSL/TLS, scaling, monitoring, and graceful shutdown.

When you run `tina4 init`, the framework generates a production-ready `Dockerfile` and `.dockerignore` in your project root. The Dockerfile uses a multi-stage build: the first stage installs Composer dependencies and the second stage copies only the runtime artifacts into a slim image. You do not need to write a Dockerfile from scratch -- the generated one is a solid starting point that you can customise as needed.

---

2. Production .env Configuration

Development defaults optimize for debugging. Production defaults optimize for performance and security. The first deployment step: configure `.env` for production.

Create a production `.env`:

```
# Core
TINA4_DEBUG=false
TINA4_LOG_LEVEL=WARNING
TINA4_PORT=7145

# Database
TINA4_DATABASE_URL=sqlite:///data/app.db

# Security
TINA4_CORS_ORIGINS=https://yourdomain.com
TINA4_SECRET=your-long-random-secret-at-least-32-characters
TINA4_RATE_LIMIT=120

# Performance
TINA4_TEMPLATE_CACHE_TTL=true
```

Key Differences from Development

Setting	Development	Production	Why
`TINA4_DEBUG`	`true`	`false`	Hides stack traces, disables toolbar
`TINA4_LOG_LEVEL`	`ALL`	`WARNING`	Reduces log noise
`TINA4_CORS_ORIGINS`	Your dev front end (for example `http://localhost:3000`), or empty	Your domain	Only listed origins can call the API from a browser

Sensitive Values

Production secrets never go into version control. The `.env` file is gitignored by default. For deployment, use environment variables from your hosting platform, CI/CD secrets, or a secrets manager.

```
# Docker: pass env vars at runtime
docker run -e JWT_SECRET=your-secret -e TINA4_DATABASE_URL=sqlite:///data/app.db my-app

# Fly.io: set secrets
fly secrets set JWT_SECRET=your-secret

# Railway: use the dashboard or CLI
railway variables set JWT_SECRET=your-secret
```

---

3. FrankenPHP Auto-Detection

Tina4 PHP auto-detects FrankenPHP at startup. FrankenPHP is a modern PHP application server built on Caddy:

- Worker mode (keeps PHP in memory between requests)
- Built-in HTTPS with automatic certificate management
- HTTP/2 and HTTP/3 support
- Dramatically better performance than PHP-FPM for long-running applications

How Auto-Detection Works

When you run `tina4 serve --production`, the CLI checks for FrankenPHP:

- If `frankenphp` is in your PATH, it uses FrankenPHP in worker mode
- If not, it falls back to PHP's built-in server with production optimizations

```
# With FrankenPHP installed
tina4 serve --production

  Tina4 PHP v3.0.0
  Server: FrankenPHP (worker mode)
```

```
  Workers: 4
  Running at https://0.0.0.0:7145
  TLS: automatic (Let's Encrypt)
```

```
# Without FrankenPHP
tina4 serve --production
```

```
  Tina4 PHP v3.0.0
  Server: PHP built-in
  Running at http://0.0.0.0:7145
  Warning: For production, consider FrankenPHP or PHP-FPM + nginx
```

Installing FrankenPHP

```
# macOS
brew install dunglas/tap/frankenphp

# Linux
curl -fsSL https://frankenphp.dev/install.sh | bash

# Docker (no installation needed)
docker pull dunglas/frankenphp
```

---

4. Docker Deployment

Docker is the most portable deployment path. Your app runs the same way on your laptop, in CI, and on the production server.

Choosing a Runtime

PHP is the one Tina4 language where the process model is a deployment decision rather than a fixed property of the runtime. `tina4 deploy docker` writes a different image for each choice:

```
tina4 deploy docker                    # cli (default)
tina4 deploy docker --runtime fpm      # nginx + php-fpm
tina4 deploy docker --runtime swoole   # openswoole
```

Runtime	Server	Pick it when
`cli` (default)	The framework's own server	You want one image, no extra moving parts. Since 3.13.95 it forks per request, so a slow handler no longer blocks the rest.
`fpm`	nginx in front of php-fpm	You want the boring, safe option. Every request gets fresh process state, so nothing leaks between requests and a fatal cannot poison the next one. This is what most PHP hosting already runs.

`swoole`	openswoole	You are serving an API and per-request bootstrap is your bottleneck. The app stays resident, so there is no rebuild per request, plus coroutines and long-lived connections.

Each runtime writes its own companion files, because the generated Dockerfile copies them. The `swoole` image adds `server.php`, the `fpm` image adds `nginx.fpm.conf` and `docker-entrypoint.fpm.sh`. Generate, read them, then commit them with your app.

The flag applies to PHP only. Ask for it on a Python, Ruby or Node project and the CLI refuses rather than quietly writing the default image.

Swoole: What Changes About Your Code

Swoole keeps your application resident between requests, and that one fact is the whole trade. Anything you write to a static or a global lives for the life of the **worker**, not the request. A cache you never bound is a memory leak that grows all day.

Three rules follow:

- Keep `TINA4_DEBUG=false`. The dev toolbar keeps its message log and request inspector in static arrays that only ever grow, so a debug-mode Swoole worker climbs until something kills it. The generated image pins this for you.
- Bound anything you cache in a static yourself. Nothing else will.
- Leave `max_request` alone unless you have a reason. It recycles a worker after a set number of requests, which caps the damage from a leak you have not found yet.

If none of that appeals, the `cli` or `fpm` images cannot leak between requests, because they do not keep anything between requests.

`App::__invoke()` is the seam that makes this work. It takes a Swoole request and returns a Tina4 response, so your routes, ORM, middleware and templates are untouched. Only the transport changes.

Official Base Image

Tina4 PHP provides an official Docker Hub base image: `tina4stack/tina4-php:v3`. It is an Alpine-based image (~154MB) with PHP 8.4, SQLite, OPcache, and the Tina4 framework pre-installed. Your app Dockerfile extends it and adds only your application code and Composer dependencies.

The base image includes these environment variables pre-configured:

- `TINA4_OVERRIDE_CLIENT=true`, bypasses the CLI guard for Docker
- `TINA4_DEBUG=false`, production mode by default

OPcache is pre-configured with production settings (timestamps disabled, 10000 accelerated files).

Dockerfile

Create `Dockerfile` at the project root:

```
FROM tina4stack/tina4-php:v3
WORKDIR /app

# Install Composer and app dependencies
COPY --from=composer:2 /usr/bin/composer /usr/bin/composer
COPY composer.json composer.lock* ./
RUN composer install --no-dev --optimize-autoloader --no-scripts \
    && rm /usr/bin/composer

# Copy application code
COPY index.php .
COPY .env .
COPY migrations/ migrations/
COPY src/ src/

# Create data directories for SQLite, sessions, queue, and mailbox
RUN mkdir -p data data/sessions data/queue data/mailbox

EXPOSE 7145
CMD ["php", "index.php", "0.0.0.0:7145"]
```

.dockerignore

Create `.dockerignore` to keep the image small:

```
.git
.env.development
data/
logs/
secrets/
node_modules/
vendor/
tests/
.DS_Store
.tina4/
```

Build and Run

```
# Build the image
docker build -t my-tina4-app .

# Run it
docker run -d \
  --name my-app \
  -p 7145:7145 \
  -e JWT_SECRET=your-production-secret \
  -v $(pwd)/data:/app/data \
  my-tina4-app
```

Verify

```
curl http://localhost:7145/health
```

```
{
  "status": "ok",
  "database": "connected",
  "uptime_seconds": 5,
  "version": "3.0.0",
  "framework": "tina4-php"
}
```

Adding Database Drivers

The base image ships with SQLite only. To use PostgreSQL, MySQL, MSSQL, or Firebird, install the driver in your Dockerfile using the `install-php-extensions` tool from `mlocati`.

PostgreSQL:

```
FROM tina4stack/tina4-php:v3
WORKDIR /app
COPY --from=mlocati/php-extension-installer /usr/bin/install-php-extensions /usr/bin/
RUN install-php-extensions pdo_pgsql && rm /usr/bin/install-php-extensions
COPY --from=composer:2 /usr/bin/composer /usr/bin/composer
COPY composer.json composer.lock* ./
RUN composer install --no-dev --optimize-autoloader --no-scripts && rm /usr/bin/composer
COPY index.php .
COPY .env .
COPY migrations/ migrations/
COPY src/ src/
RUN mkdir -p data data/sessions data/queue data/mailbox
EXPOSE 7145
CMD ["php", "index.php", "0.0.0.0:7145"]
```

MySQL:

```
FROM tina4stack/tina4-php:v3
WORKDIR /app
COPY --from=mlocati/php-extension-installer /usr/bin/install-php-extensions /usr/bin/
RUN install-php-extensions pdo_mysql && rm /usr/bin/install-php-extensions
COPY --from=composer:2 /usr/bin/composer /usr/bin/composer
COPY composer.json composer.lock* ./
RUN composer install --no-dev --optimize-autoloader --no-scripts && rm /usr/bin/composer
COPY index.php .
COPY .env .
COPY migrations/ migrations/
COPY src/ src/
RUN mkdir -p data data/sessions data/queue data/mailbox
EXPOSE 7145
CMD ["php", "index.php", "0.0.0.0:7145"]
```

MSSQL:

```
FROM tina4stack/tina4-php:v3
WORKDIR /app
COPY --from=mlocati/php-extension-installer /usr/bin/install-php-extensions /usr/bin/
RUN install-php-extensions pdo_sqlsrv && rm /usr/bin/install-php-extensions
COPY --from=composer:2 /usr/bin/composer /usr/bin/composer
COPY composer.json composer.lock* ./
RUN composer install --no-dev --optimize-autoloader --no-scripts && rm /usr/bin/composer
COPY index.php .
COPY .env .
COPY migrations/ migrations/
COPY src/ src/
```

```
RUN mkdir -p data data/sessions data/queue data/mailbox
EXPOSE 7145
CMD ["php", "index.php", "0.0.0.0:7145"]
```

Firebird:

Firebird requires system-level libraries not available in Alpine. Use a Debian-based PHP image instead:

```
FROM php:8.4-cli-bookworm
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends \
    firebird-dev libfbclient2 && \
    docker-php-ext-install pdo_firebird interbase && \
    apt-get purge -y firebird-dev && apt-get autoremove -y && \
    rm -rf /var/lib/apt/lists/*
COPY --from=composer:2 /usr/bin/composer /usr/bin/composer
COPY composer.json composer.lock* ./
RUN composer install --no-dev --optimize-autoloader --no-scripts && rm /usr/bin/composer
COPY index.php .
COPY .env .
COPY migrations/ migrations/
COPY src/ src/
RUN mkdir -p data data/sessions data/queue data/mailbox
EXPOSE 7145
ENV TINA4_OVERRIDE_CLIENT=true
ENV TINA4_DEBUG=false
CMD ["php", "index.php", "0.0.0.0:7145"]
```

> ***Note:*** *The Firebird Dockerfile cannot use* `tina4stack/tina4-php:v3` *because the base image is Alpine and Firebird's* `fbclient` *library requires glibc (Debian).*

Docker Compose

Create `docker-compose.yml`:

```
services:
  app:
    build: .
    ports:
      - "7145:7145"
    environment:
      - TINA4_DEBUG=false
      - TINA4_LOG_LEVEL=WARNING
      - JWT_SECRET=${JWT_SECRET}
      - TINA4_DATABASE_URL=sqlite:///data/app.db
    volumes:
      - app-data:/app/data
    restart: unless-stopped

volumes:
  app-data:
```

```
# Start everything
docker compose up -d

# View logs
docker compose logs -f app

# Stop everything
```

```
docker compose down
```

---

5. PHP-FPM + nginx Configuration

If you prefer a traditional setup without Docker, use PHP-FPM with nginx.

PHP-FPM Pool Configuration

Create `/etc/php/8.3/fpm/pool.d/tina4.conf`:

```
[tina4]
user = www-data
group = www-data
listen = /run/php/tina4.sock
listen.owner = www-data
listen.group = www-data

pm = dynamic
pm.max_children = 50
pm.start_servers = 5
pm.min_spare_servers = 5
pm.max_spare_servers = 35
pm.max_requests = 1000

env[TINA4_DATABASE_URL] = sqlite:///var/www/tina4-app/data/app.db
env[JWT_SECRET] = your-production-secret
env[TINA4_DEBUG] = false
env[TINA4_LOG_LEVEL] = WARNING
```

nginx Configuration

Create `/etc/nginx/sites-available/tina4`:

```
server {
    listen 80;
    server_name yourdomain.com;
    root /var/www/tina4-app/src/public;
    index index.php;

    # Security headers
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-XSS-Protection "1; mode=block" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;

    # Static files
    location ~* \.(css|js|png|jpg|jpeg|gif|ico|svg|woff|woff2|ttf|eot)$ {
        expires 30d;
        add_header Cache-Control "public, immutable";
        try_files $uri =404;
    }

    # All other requests go to Tina4
    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }
```

```
    # PHP processing
    location ~ \.php$ {
        fastcgi_pass unix:/run/php/tina4.sock;
        fastcgi_param SCRIPT_FILENAME $realpath_root$fastcgi_script_name;
        include fastcgi_params;
        fastcgi_buffers 16 16k;
        fastcgi_buffer_size 32k;
    }

    # Block access to sensitive files
    location ~ /\.(env|git|htaccess) {
        deny all;
    }

    location ~ ^/(data|logs|secrets|vendor)/ {
        deny all;
    }
}
```

Enable the site and restart:

```
sudo ln -s /etc/nginx/sites-available/tina4 /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl restart nginx
sudo systemctl restart php8.3-fpm
```

---

6. Health Check Endpoint

Tina4 includes a built-in health check endpoint at `/health`:

```
curl http://localhost:7145/health
```

```
{
  "status": "ok",
  "database": "connected",
  "uptime_seconds": 3600,
  "version": "3.0.0",
  "framework": "tina4-php"
}
```

If the database is disconnected or the application is in a bad state, the health check returns a non-200 status:

```
{
  "status": "error",
  "database": "disconnected",
  "error": "Could not connect to database",
  "uptime_seconds": 3600,
  "version": "3.0.0",
  "framework": "tina4-php"
}
```

Use this endpoint for:

- **Docker HEALTHCHECK** (shown in the Dockerfile above)
- **Load balancer health checks** (AWS ALB, nginx upstream, HAProxy)

- **Monitoring tools** (Uptime Robot, Pingdom, custom scripts)
- **Kubernetes liveness and readiness probes**

Custom Health Checks

You can extend the health check to include your own checks:

```
<?php
use Tina4\Router;

Router::get("/health/detailed", function ($request, $response) {
    $checks = [
        "database" => false,
        "cache" => false,
        "disk_space" => false
    ];

    // Check database
    try {
        $product = new Product();
        $product->select("count(*) as cnt");
        $checks["database"] = true;
    } catch (\Exception $e) {
        // Database check failed
    }

    // Check disk space
    $freeBytes = disk_free_space("/");
    $checks["disk_space"] = $freeBytes > 100 * 1024 * 1024; // 100MB minimum

    $allOk = !in_array(false, $checks);

    return $response->json([
        "status" => $allOk ? "ok" : "degraded",
        "checks" => $checks,
        "free_disk_mb" => round($freeBytes / 1024 / 1024)
    ], $allOk ? 200 : 503);
});
```

---

7. Graceful Shutdown

Deploy a new version. The old process must stop. Graceful shutdown finishes active requests before terminating.

Tina4 handles this when it receives a `SIGTERM` signal (the standard shutdown signal from Docker and systemd):

- Stop accepting new connections
- Wait for active requests to complete (up to a configurable timeout)
- Close database connections
- Flush logs
- Exit

Shutdown Timeout

Set the maximum time to wait for in-flight requests in `.env`:

```
TINA4_SHUTDOWN_TIMEOUT=30
```

If requests are still processing after 30 seconds, the server force-kills them and exits. The default is 30 seconds, which is enough for most applications.

Docker Stop Behavior

Docker sends SIGTERM, waits 10 seconds (by default), then sends SIGKILL. Match the Docker timeout to your shutdown timeout:

```
services:
  app:
    stop_grace_period: 35s   # Slightly more than TINA4_SHUTDOWN_TIMEOUT
```

---

8. Log Rotation

In production, log files grow without limit unless rotated. Tina4 writes to `logs/app.log`.

Using logrotate (Linux)

Create `/etc/logrotate.d/tina4`:

```
/var/www/tina4-app/logs/*.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    create 640 www-data www-data
    postrotate
        # Tina4 re-opens log files automatically
    endscript
}
```

This rotates logs daily, keeps 14 days of compressed history, and creates new log files with the correct permissions.

Docker Logging

In Docker, application logs go to stdout/stderr by default. Configure Docker's logging driver to manage rotation:

```
services:
  app:
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "5"
```

This keeps a maximum of 50MB of logs (5 files of 10MB each).

---

9. Environment-Specific Configuration

Use different `.env` files for different environments:

```
.env                    # Development (gitignored)
.env.example            # Template (committed to git)
.env.staging            # Staging config (gitignored)
.env.production         # Production config (gitignored)
```

Loading a Specific .env File

Set the `TINA4_ENV_FILE` environment variable before starting:

```
# Staging
TINA4_ENV_FILE=.env.staging tina4 serve

# Production
TINA4_ENV_FILE=.env.production tina4 serve --production
```

In Docker, pass environment variables directly:

```
docker run -e TINA4_DEBUG=false -e JWT_SECRET=xxx my-app
```

Environment variables set via `docker run -e` or `docker-compose environment:` take precedence over `.env` file values. This lets you use a generic `.env` file in the image and override specific values at runtime.

---

10. SSL/TLS with Let's Encrypt

With FrankenPHP (Automatic)

FrankenPHP handles SSL automatically. Just set your domain:

```
TINA4_HOST=yourdomain.com
```

FrankenPHP uses Let's Encrypt to provision and renew certificates automatically. No manual configuration needed.

With nginx (Manual)

Install Certbot:

```
sudo apt install certbot python3-certbot-nginx
```

Obtain a certificate:

```
sudo certbot --nginx -d yourdomain.com
```

Certbot modifies your nginx configuration to include SSL settings and sets up automatic renewal. Verify auto-renewal:

```
sudo certbot renew --dry-run
```

With Docker (Using a Reverse Proxy)

Use Traefik or Caddy as a reverse proxy in front of your Tina4 container:

```
version: "3.8"

services:
  reverse-proxy:
    image: traefik:v3.0
    command:
      - "--providers.docker=true"
      - "--entrypoints.web.address=:80"
      - "--entrypoints.websecure.address=:443"
      - "--certificatesresolvers.letsencrypt.acme.tlschallenge=true"
      - "--certificatesresolvers.letsencrypt.acme.email=you@example.com"
      - "--certificatesresolvers.letsencrypt.acme.storage=/letsencrypt/acme.json"
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro
      - letsencrypt:/letsencrypt

  app:
    build: .
    labels:
      - "traefik.http.routers.app.rule=Host(`yourdomain.com`)"
      - "traefik.http.routers.app.tls=true"
      - "traefik.http.routers.app.tls.certresolver=letsencrypt"
    environment:
      - TINA4_DEBUG=false
      - JWT_SECRET=${JWT_SECRET}

volumes:
  letsencrypt:
```

---

11. Scaling

Multiple Workers

FrankenPHP runs multiple worker threads by default. Configure the count in `.env`:

A good starting point is the number of CPU cores on your server. For I/O-heavy applications (database queries, API calls), you can go higher -- 2x to 4x the core count.

Load Balancing with nginx

If you run multiple Tina4 instances, use nginx as a load balancer:

```
upstream tina4_backend {
    server 127.0.0.1:7145;
    server 127.0.0.1:7145;
    server 127.0.0.1:7147;
    server 127.0.0.1:7148;
}

server {
```

```
    listen 80;
    server_name yourdomain.com;

    location / {
        proxy_pass http://tina4_backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Start four instances on different ports:

```
TINA4_PORT=7145 tina4 serve --production &
TINA4_PORT=7145 tina4 serve --production &
TINA4_PORT=7147 tina4 serve --production &
TINA4_PORT=7148 tina4 serve --production &
```

Docker Scaling

With Docker Compose, scale horizontally:

```
docker compose up -d --scale app=4
```

Use a load balancer (Traefik, nginx, or a cloud load balancer) in front of the containers.

---

12. Monitoring

Log Aggregation

In production, send logs to a centralized service:

```
TINA4_LOG_FORMAT=json
```

JSON-formatted logs can be ingested by services like:

- Elastic Stack (ELK)
- Grafana Loki
- Datadog
- AWS CloudWatch

Example JSON log entry:

```
{
  "timestamp": "2026-03-22T14:30:00.123Z",
  "level": "WARNING",
  "message": "Rate limit exceeded",
  "request_id": "req-abc123",
  "ip": "203.0.113.42",
  "path": "/api/products",
  "method": "GET"
}
```

Uptime Monitoring

Point an external monitoring service at your health endpoint:

```
https://yourdomain.com/health
```

Services like Uptime Robot, Pingdom, or Better Uptime will ping this endpoint every 30-60 seconds and alert you if it stops responding or returns a non-200 status.

Application Performance Monitoring (APM)

For deeper insights, add APM instrumentation. Since Tina4 is standard PHP, any PHP APM agent works:

- New Relic APM
- Datadog APM
- Elastic APM

These agents track request latency, database query performance, error rates, and memory usage automatically.

---

13. Exercise: Deploy a Tina4 PHP App with Docker

Deploy the task management application you have been building throughout this book.

Requirements

- Create a `Dockerfile` that:
- Uses `tina4stack/tina4-php:v3` as the base image
- Installs Composer dependencies
- Copies the application code
- Creates data directories
- Exposes port 7145
- Create a `docker-compose.yml` that:
- Builds and runs the application
- Maps port 7145
- Uses environment variables for secrets
- Persists the database via volumes
- Includes restart policy
- Create a production `.env.production` with:
- Debug mode off
- Warning-level logging
- Template caching enabled

- Appropriate CORS settings
- Build and run the container.
- Verify the health check works.
- Verify you can create and list products through the containerized API.

Test with:

```
# Build
docker compose build

# Start
docker compose up -d

# Health check
curl http://localhost:7145/health

# Create a product
curl -X POST http://localhost:7145/api/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Docker Widget", "price": 19.99}'

# List products
curl http://localhost:7145/api/products

# View logs
docker compose logs app

# Stop
docker compose down
```

---

14. Solution

Dockerfile

```
FROM tina4stack/tina4-php:v3
WORKDIR /app

COPY --from=composer:2 /usr/bin/composer /usr/bin/composer
COPY composer.json composer.lock* ./
RUN composer install --no-dev --optimize-autoloader --no-scripts \
    && rm /usr/bin/composer

COPY index.php .
COPY .env .
COPY migrations/ migrations/
COPY src/ src/

RUN mkdir -p data data/sessions data/queue data/mailbox

EXPOSE 7145
CMD ["php", "index.php", "0.0.0.0:7145"]
```

docker-compose.yml

```
services:
  app:
    build: .
    ports:
      - "7145:7145"
    environment:
      - TINA4_DEBUG=false
      - TINA4_LOG_LEVEL=WARNING
      - TINA4_TEMPLATE_CACHE_TTL=true
      - JWT_SECRET=change-this-to-a-real-secret
      - TINA4_DATABASE_URL=sqlite:///data/app.db
      - TINA4_CORS_ORIGINS=http://localhost:7145
    volumes:
      - app-data:/app/data
    restart: unless-stopped
    stop_grace_period: 35s

volumes:
  app-data:
```

.env.production

```
TINA4_DEBUG=false
TINA4_LOG_LEVEL=WARNING
TINA4_TEMPLATE_CACHE_TTL=true
TINA4_RATE_LIMIT=120
TINA4_CORS_ORIGINS=https://yourdomain.com
```

Expected output for health check:

```
{
  "status": "ok",
  "database": "connected",
  "uptime_seconds": 8,
  "version": "3.0.0",
  "framework": "tina4-php"
}
```

Expected output for creating a product:

```
{
  "id": 1,
  "name": "Docker Widget",
  "price": 19.99,
  "in_stock": true,
  "created_at": "2026-03-22 14:30:00",
  "updated_at": "2026-03-22 14:30:00"
}
```

HTTP Caching, Compression, and ETags

Tina4 applies gzip to eligible dynamic responses and then computes a strong ETag from the final post-compression bytes. The identity and gzip representations therefore have different ETags, and `Vary: Accept-Encoding` keeps intermediary caches from mixing them. A matching `If-None-Match` returns `304 Not Modified` with an empty body while preserving the validators.

Static files use a weak validator derived from file size and modification time. Streaming responses bypass dynamic compression and ETag generation because their complete representation is not buffered. Test both identity and gzip requests when validating cache behavior.

15. Gotchas

1. SQLite Concurrency in Production

Problem: Under high load, SQLite throws "database is locked" errors.

Cause: SQLite uses file-level locking. Multiple concurrent writes block each other.

Fix: For low-to-medium traffic (under 100 concurrent users), SQLite works fine. For higher traffic, switch to PostgreSQL or MySQL:

```
TINA4_DATABASE_URL=postgres://user:pass@localhost:5432/myapp
```

If you must use SQLite under load, enable WAL mode by adding this to your application startup:

```
$db = Tina4\Database::getConnection();
$db->exec("PRAGMA journal_mode=WAL");
```

2. Data Volume Not Persisted

Problem: You restart the Docker container and the database is empty.

Cause: The `data/` directory is inside the container. When the container is recreated, the data is lost.

Fix: Use a Docker volume to persist the data directory, as shown in the docker-compose.yml above:

```
volumes:
  - app-data:/app/data
```

3. .env File Not Loaded in Docker

Problem: Environment variables from `.env` are not available in the container.

Cause: Docker does not automatically load `.env` files inside the container. The `.env` file is for the host machine (used by `docker compose`).

Fix: Pass environment variables via the `environment:` section in `docker-compose.yml`, or use `env_file:`:

```
services:
  app:
    env_file:
```

```
      - .env.production
```

4. Permission Denied on data/ Directory

Problem: The application cannot write to `data/` or `logs/` inside the container.

Cause: The directories were created by root during the Docker build, but the application runs as `www-data`.

Fix: Add `chown` to your Dockerfile:

```
RUN mkdir -p data logs secrets \
    && chown -R www-data:www-data data logs secrets
```

5. Health Check Fails During Startup

Problem: The container restarts in a loop because the health check fails before the application is ready.

Cause: The health check starts immediately, but the application needs a few seconds to initialize.

Fix: Use `start_period` in the health check to give the application time to start:

```
HEALTHCHECK --start-period=10s --interval=30s --timeout=5s --retries=3 \
    CMD curl -f http://localhost:7145/health || exit 1
```

The `start-period` tells Docker to ignore health check failures during the first 10 seconds.

6. CORS Errors in Production

Problem: Your frontend gets "No 'Access-Control-Allow-Origin' header" errors.

Cause: `TINA4_CORS_ORIGINS` is set to `*` in development but to a specific domain in production. If your frontend is served from a different domain or port, CORS blocks the requests.

Fix: Set `TINA4_CORS_ORIGINS` to include all domains that need access:

```
TINA4_CORS_ORIGINS=https://yourdomain.com,https://admin.yourdomain.com
```

7. SSL Certificate Not Renewing

Problem: Your HTTPS certificate expires and the site goes down.

Cause: The auto-renewal process (Certbot or FrankenPHP) failed silently. Common reasons: DNS changes, firewall blocking port 80 (needed for ACME challenge), or the renewal service crashed.

Fix: Set up monitoring for certificate expiry. Use a tool like `ssl-cert-check` or an online service that alerts you before the certificate expires. Check renewal logs:

```
# Certbot
sudo certbot renew --dry-run

# Check crontab for renewal
sudo systemctl list-timers | grep certbot
```

For FrankenPHP, ensure port 80 and 443 are accessible from the internet for ACME challenges.

Building a Complete App

1. Putting It All Together

Twenty chapters of building blocks. Routing. Templates. Databases. ORM. Authentication. Middleware. Queues. WebSocket. Caching. Frontend. GraphQL. Testing. Dev tools. CLI scaffolding. Deployment. Now all of it works together in one application.

TaskFlow -- a task management system with:

- User registration and JWT authentication
- Task creation, assignment, and tracking
- A dashboard with real-time updates
- Email notifications when tasks are assigned
- Caching for dashboard performance
- A full test suite
- Docker deployment

Not a toy project. A production-ready application. Every major Tina4 feature in one codebase.

---

2. Planning the App

Models

Model	Table	Fields
User	users	id, name, email, passwordHash, role, createdAt
Task	tasks	id, title, description, status, priority, createdBy, assignedTo, dueDate, completedAt, createdAt, updatedAt

Relationships

- A User has many Tasks (created by them)
- A User has many Tasks (assigned to them)
- A Task belongs to a User (creator)
- A Task belongs to a User (assignee)

Routes

Method	Path	Description	Auth
POST	/api/auth/register	Register a new user	public
POST	/api/auth/login	Login, get JWT	public
GET	/api/profile	Get current user profile	secured
GET	/api/tasks	List tasks (with filters)	secured
GET	/api/tasks/{id}	Get a single task	secured
POST	/api/tasks	Create a task	secured
PUT	/api/tasks/{id}	Update a task	secured
DELETE	/api/tasks/{id}	Delete a task	secured
GET	/api/dashboard/stats	Dashboard statistics	secured
GET	/admin	Dashboard HTML page	public

Templates

- `base.html` -- Base layout with sidebar and topbar
- `dashboard.html` -- Dashboard with stats, task list, quick actions
- `login.html` -- Login page
- `register.html` -- Registration page

---

3. Step 1: Init Project and Set Up Database

```
tina4 init taskflow
cd taskflow
composer install
```

Update `.env`:

```
TINA4_DEBUG=true
JWT_SECRET=taskflow-dev-secret-change-in-production
JWT_EXPIRY=86400
```

Create Migrations

Create `migrations/20260322000100_create_users_table.sql`:

```
-- UP
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
```

```
    password_hash TEXT NOT NULL,
    role TEXT NOT NULL DEFAULT 'user',
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_users_email ON users(email);

-- DOWN
DROP TABLE IF EXISTS users;
```

Create `migrations/20260322000200_create_tasks_table.sql`:

```
-- UP
CREATE TABLE tasks (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    description TEXT DEFAULT '',
    status TEXT NOT NULL DEFAULT 'todo',
    priority TEXT NOT NULL DEFAULT 'medium',
    created_by INTEGER NOT NULL,
    assigned_to INTEGER,
    due_date TEXT,
    completed_at TEXT,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (created_by) REFERENCES users(id),
    FOREIGN KEY (assigned_to) REFERENCES users(id)
);

CREATE INDEX idx_tasks_status ON tasks(status);
CREATE INDEX idx_tasks_assigned_to ON tasks(assigned_to);
CREATE INDEX idx_tasks_created_by ON tasks(created_by);

-- DOWN
DROP TABLE IF EXISTS tasks;
```

Run the migrations:

```
tina4 migrate
```

```
Running migrations...
  [APPLIED] 20260322000100_create_users_table.sql
  [APPLIED] 20260322000200_create_tasks_table.sql
Migrations complete. 2 applied.
```

---

4. Step 2: User Model with Registration and Login

Create `src/orm/User.php`:

```
<?php
use Tina4\ORM;

class User extends ORM
{
    public int $id;
    public string $name;
    public string $email;
    public string $passwordHash;
```

```
    public string $role = "user";
    public string $createdAt;

    public string $tableName = "users";
    public string $primaryKey = "id";

    /**
     * Get tasks created by this user
     */
    public function createdTasks(): array
    {
        return $this->hasMany(Task::class, "created_by");
    }

    /**
     * Get tasks assigned to this user
     */
    public function assignedTasks(): array
    {
        return $this->hasMany(Task::class, "assigned_to");
    }

    /**
     * Hash a password
     */
    public static function hashPassword(string $password): string
    {
        return password_hash($password, PASSWORD_DEFAULT);
    }

    /**
     * Verify a password against the stored hash
     */
    public function verifyPassword(string $password): bool
    {
        return password_verify($password, $this->passwordHash);
    }

    /**
     * Convert to dict without the password hash
     */
    public function toSafeDict(): array
    {
        $dict = $this->toArray();
        unset($dict["password_hash"]);
        return $dict;
    }
}
```

Authentication Routes

Create `src/routes/auth.php`:

```
<?php
use Tina4\Router;
use Tina4\Auth;

/**
 * @noauth
```

```
 */
Router::post("/api/auth/register", function ($request, $response) {
    $body = $request->body;

    // Validate input
    if (empty($body["name"]) || empty($body["email"]) || empty($body["password"])) {
        return $response->json(["error" => "name, email, and password are required"], 400);
    }

    if (strlen($body["password"]) < 8) {
        return $response->json(["error" => "Password must be at least 8 characters"], 400);
    }

    if (!filter_var($body["email"], FILTER_VALIDATE_EMAIL)) {
        return $response->json(["error" => "Invalid email address"], 400);
    }

    // Check for existing user
    $existing = new User();
    $found = $existing->select("*", "email = :email", ["email" => $body["email"]]);
    if (count($found) > 0) {
        return $response->json(["error" => "Email already registered"], 409);
    }

    // Create user
    $user = new User();
    $user->name = $body["name"];
    $user->email = $body["email"];
    $user->passwordHash = User::hashPassword($body["password"]);
    $user->role = "user";
    $user->save();

    return $response->json($user->toSafeDict(), 201);
});

/**
 * @noauth
 */
Router::post("/api/auth/login", function ($request, $response) {
    $body = $request->body;

    if (empty($body["email"]) || empty($body["password"])) {
        return $response->json(["error" => "email and password are required"], 400);
    }

    // Find user by email
    $user = new User();
    $found = $user->select("*", "email = :email", ["email" => $body["email"]]);

    if (count($found) === 0) {
        return $response->json(["error" => "Invalid email or password"], 401);
    }

    $user = $found[0];

    // Verify password
    if (!$user->verifyPassword($body["password"])) {
        return $response->json(["error" => "Invalid email or password"], 401);
    }
```

```
    // Generate JWT token
    $token = Auth::getToken([
        "user_id" => $user->id,
        "email" => $user->email,
        "role" => $user->role
    ]);

    return $response->json([
        "token" => $token,
        "user" => $user->toSafeDict()
    ]);
});

/**
 * @secured
 */
Router::get("/api/profile", function ($request, $response) {
    $userId = $request->user["user_id"];

    $user = new User();
    $user->load($userId);

    if (empty($user->id)) {
        return $response->json(["error" => "User not found"], 404);
    }

    return $response->json(["user" => $user->toSafeDict()]);
});
```

Test Registration and Login

```
# Start the server
tina4 serve
```

```
# Register a user
curl -X POST http://localhost:7145/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice Johnson", "email": "alice@example.com", "password": "securepass123"}'
```

```
{
  "id": 1,
  "name": "Alice Johnson",
  "email": "alice@example.com",
  "role": "user",
  "created_at": "2026-03-22 10:00:00"
}
```

```
# Login
curl -X POST http://localhost:7145/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email": "alice@example.com", "password": "securepass123"}'
```

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": 1,
    "name": "Alice Johnson",
    "email": "alice@example.com",
    "role": "user"
  }
}
```

```
# Access protected route
curl http://localhost:7145/api/profile \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."

{
  "user": {
    "id": 1,
    "name": "Alice Johnson",
    "email": "alice@example.com",
    "role": "user"
  }
}
```

---

5. Step 3: Task Model with CRUD

Create `src/orm/Task.php`:

```
<?php
use Tina4\ORM;

class Task extends ORM
{
    public int $id;
    public string $title;
    public string $description = "";
    public string $status = "todo";
    public string $priority = "medium";
    public int $createdBy;
    public ?int $assignedTo = null;
    public ?string $dueDate = null;
    public ?string $completedAt = null;
    public string $createdAt;
    public string $updatedAt;

    public string $tableName = "tasks";
    public string $primaryKey = "id";

    /**
     * Get the user who created this task
     */
    public function creator(): ?User
    {
        return $this->belongsTo(User::class, "created_by");
    }

    /**
     * Get the user this task is assigned to
     */
    public function assignee(): ?User
    {
        return $this->belongsTo(User::class, "assigned_to");
    }

    /**
     * Convert to dict with nested user data
     */
    public function toDetailedDict(): array
```

```
    {
        $dict = $this->toArray();

        $creator = $this->creator();
        $dict["creator"] = $creator ? $creator->toSafeDict() : null;

        $assignee = $this->assignee();
        $dict["assignee"] = $assignee ? $assignee->toSafeDict() : null;

        return $dict;
    }
}
```

Task Routes

Create `src/routes/tasks.php`:

```
<?php
use Tina4\Router;

Router::group("/api", function () {

    // List tasks with filters
    /**
     * @secured
     */
    Router::get("/tasks", function ($request, $response) {
        $userId = $request->user["user_id"];

        $status = $request->params["status"] ?? "";
        $priority = $request->params["priority"] ?? "";
        $assigned = $request->params["assigned"] ?? "";
        $page = (int) ($request->params["page"] ?? 1);
        $perPage = (int) ($request->params["per_page"] ?? 20);

        $conditions = [];
        $params = [];

        // Show tasks created by or assigned to the current user
        $conditions[] = "(created_by = :userId OR assigned_to = :userId2)";
        $params["userId"] = $userId;
        $params["userId2"] = $userId;

        if (!empty($status)) {
            $conditions[] = "status = :status";
            $params["status"] = $status;
        }

        if (!empty($priority)) {
            $conditions[] = "priority = :priority";
            $params["priority"] = $priority;
        }

        if ($assigned === "me") {
            $conditions[] = "assigned_to = :assignedTo";
            $params["assignedTo"] = $userId;
        } elseif ($assigned === "unassigned") {
            $conditions[] = "assigned_to IS NULL";
        }
```

```
    $filter = implode(" AND ", $conditions);
    $offset = ($page - 1) * $perPage;

    $task = new Task();
    $tasks = $task->select("*", $filter, $params, "created_at DESC", $perPage, $offset);

    $results = array_map(fn($t) => $t->toDetailedDict(), $tasks);

    return $response->json([
        "tasks" => $results,
        "page" => $page,
        "per_page" => $perPage,
        "count" => count($results)
    ]);
});

// Get a single task
/**
 * @secured
 */
Router::get("/tasks/{id:int}", function ($request, $response) {
    $task = new Task();
    $task->load($request->params["id"]);

    if (empty($task->id)) {
        return $response->json(["error" => "Task not found"], 404);
    }

    return $response->json($task->toDetailedDict());
});

// Create a task
Router::post("/tasks", function ($request, $response) {
    $userId = $request->user["user_id"];
    $body = $request->body;

    if (empty($body["title"])) {
        return $response->json(["error" => "title is required"], 400);
    }

    $validStatuses = ["todo", "in_progress", "review", "done"];
    $validPriorities = ["low", "medium", "high", "urgent"];

    $status = $body["status"] ?? "todo";
    if (!in_array($status, $validStatuses)) {
        return $response->json([
            "error" => "Invalid status. Must be one of: " . implode(", ", $validStatuses)
        ], 400);
    }

    $priority = $body["priority"] ?? "medium";
    if (!in_array($priority, $validPriorities)) {
        return $response->json([
            "error" => "Invalid priority. Must be one of: " . implode(", ", $validPriorities
        ], 400);
    }

    $task = new Task();
    $task->title = $body["title"];
```

```
    $task->description = $body["description"] ?? "";
    $task->status = $status;
    $task->priority = $priority;
    $task->createdBy = $userId;
    $task->dueDate = $body["due_date"] ?? null;

    // Handle assignment
    if (!empty($body["assigned_to"])) {
        $assignee = new User();
        $assignee->load((int) $body["assigned_to"]);
        if (empty($assignee->id)) {
            return $response->json(["error" => "Assigned user not found"], 400);
        }
        $task->assignedTo = $assignee->id;
    }

    $task->save();

    return $response->json($task->toDetailedDict(), 201);
});

// Update a task
Router::put("/tasks/{id:int}", function ($request, $response) {
    $task = new Task();
    $task->load($request->params["id"]);

    if (empty($task->id)) {
        return $response->json(["error" => "Task not found"], 404);
    }

    $body = $request->body;

    if (isset($body["title"])) $task->title = $body["title"];
    if (isset($body["description"])) $task->description = $body["description"];
    if (isset($body["priority"])) $task->priority = $body["priority"];
    if (isset($body["due_date"])) $task->dueDate = $body["due_date"];

    // Handle status change
    if (isset($body["status"])) {
        $oldStatus = $task->status;
        $task->status = $body["status"];

        // Mark completion time
        if ($body["status"] === "done" && $oldStatus !== "done") {
            $task->completedAt = date("Y-m-d H:i:s");
        } elseif ($body["status"] !== "done") {
            $task->completedAt = null;
        }
    }

    // Handle reassignment
    if (isset($body["assigned_to"])) {
        if ($body["assigned_to"] === null) {
            $task->assignedTo = null;
        } else {
            $assignee = new User();
            $assignee->load((int) $body["assigned_to"]);
            if (empty($assignee->id)) {
                return $response->json(["error" => "Assigned user not found"], 400);
```

```
                }
                $task->assignedTo = $assignee->id;
            }
        }

        $task->save();

        return $response->json($task->toDetailedDict());
    });

    // Delete a task
    Router::delete("/tasks/{id:int}", function ($request, $response) {
        $task = new Task();
        $task->load($request->params["id"]);

        if (empty($task->id)) {
            return $response->json(["error" => "Task not found"], 404);
        }

        $task->delete();
        return $response->json(null, 204);
    });
});
```

Test the Task API

```
# Get token (from login)
TOKEN="eyJhbGciOiJIUzI1NiIs..."

# Create tasks
curl -X POST http://localhost:7145/api/tasks \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"title": "Design database schema", "priority": "high", "status": "done"}'

curl -X POST http://localhost:7145/api/tasks \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"title": "Build API endpoints", "priority": "high", "status": "in_progress"}'

curl -X POST http://localhost:7145/api/tasks \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"title": "Write documentation", "priority": "medium", "due_date": "2026-04-01"}'

# List all tasks
curl http://localhost:7145/api/tasks \
  -H "Authorization: Bearer $TOKEN"

# Filter by status
curl "http://localhost:7145/api/tasks?status=in_progress" \
  -H "Authorization: Bearer $TOKEN"
```

---

6. Step 4: Dashboard Template with tina4css

Create `src/templates/app/layout.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}TaskFlow{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    <script>
        (function() {
            var t = localStorage.getItem("theme");
            if (t) document.documentElement.setAttribute("data-theme", t);
            else if (window.matchMedia("(prefers-color-scheme: dark)").matches)
                document.documentElement.setAttribute("data-theme", "dark");
        })();
    </script>
    <style>
        * { margin: 0; padding: 0; box-sizing: border-box; }
        body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif; }
        .app { display: flex; min-height: 100vh; }
        .sidebar {
            width: 240px; background: #1a1a2e; color: #ccc; flex-shrink: 0;
        }
        .sidebar-brand {
            padding: 20px; font-size: 1.3em; font-weight: bold; color: #fff;
            border-bottom: 1px solid rgba(255,255,255,0.1);
        }
        .sidebar-nav { list-style: none; padding: 12px 0; }
        .sidebar-nav a {
            display: block; padding: 10px 20px; color: rgba(255,255,255,0.6);
            text-decoration: none; transition: 0.2s;
        }
        .sidebar-nav a:hover, .sidebar-nav a.active {
            color: #fff; background: rgba(255,255,255,0.08);
        }
        .main { flex: 1; background: #f4f5f7; display: flex; flex-direction: column; }
        .topbar {
            background: #fff; padding: 14px 24px; border-bottom: 1px solid #e1e4e8;
            display: flex; justify-content: space-between; align-items: center;
        }
        .topbar-left { font-size: 1.1em; font-weight: 600; }
        .topbar-right { display: flex; gap: 12px; align-items: center; }
        .content { padding: 24px; flex: 1; }
        .stats-row { display: grid; grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
        .stat-card {
            background: #fff; border-radius: 8px; padding: 20px;
            box-shadow: 0 1px 3px rgba(0,0,0,0.08);
        }
        .stat-card .value { font-size: 2em; font-weight: bold; }
        .stat-card .label { color: #6c757d; margin-top: 4px; font-size: 0.9em; }
        .task-row {
            background: #fff; border-radius: 6px; padding: 14px 18px; margin-bottom: 8px;
            display: flex; align-items: center; gap: 12px;
            box-shadow: 0 1px 2px rgba(0,0,0,0.05);
        }
        .task-row .task-title { flex: 1; font-weight: 500; }
        .priority-dot {
            width: 10px; height: 10px; border-radius: 50%; flex-shrink: 0;
        }
        .priority-dot.urgent { background: #dc3545; }
```

```
        .priority-dot.high { background: #fd7e14; }
        .priority-dot.medium { background: #ffc107; }
        .priority-dot.low { background: #28a745; }
        @media (max-width: 768px) {
            .sidebar { display: none; }
            .stats-row { grid-template-columns: 1fr 1fr; }
        }
    </style>
    {% block extra_css %}{% endblock %}
</head>
<body>
    <div class="app">
        <aside class="sidebar">
            <div class="sidebar-brand">TaskFlow</div>
            <ul class="sidebar-nav">
                <li><a href="/admin" class="{% if page == 'dashboard' %}active{% endif %}">Dashb
                <li><a href="/admin/tasks" class="{% if page == 'tasks' %}active{% endif %}">All
                <li><a href="/admin/my-tasks" class="{% if page == 'my-tasks' %}active{% endif %
            </ul>
        </aside>
        <div class="main">
            <div class="topbar">
                <div class="topbar-left">{% block page_title %}Dashboard{% endblock %}</div>
                <div class="topbar-right">
                    <button class="btn btn-sm btn-outline-secondary" onclick="toggleTheme()">The
                    <span id="userName">{{ user_name | default("User") }}</span>
                    <button class="btn btn-sm btn-outline-danger" onclick="logout()">Logout</but
                </div>
            </div>
            <div class="content">
                {% block content %}{% endblock %}
            </div>
        </div>
    </div>
    <script src="/js/frond.js"></script>
    <script>
        function toggleTheme() {
            var html = document.documentElement;
            var next = html.getAttribute("data-theme") === "dark" ? "light" : "dark";
            html.setAttribute("data-theme", next);
            localStorage.setItem("theme", next);
        }
        function logout() {
            frond.clearToken();
            window.location.href = "/login";
        }
    </script>
    {% block extra_js %}{% endblock %}
</body>
</html>
```

Create `src/templates/app/dashboard.html`:

```
{% extends "app/layout.html" %}

{% block title %}Dashboard - TaskFlow{% endblock %}
{% block page_title %}Dashboard{% endblock %}

{% block content %}
```

```
<div class="stats-row">
    <div class="stat-card">
        <div class="value" id="statTotal">--</div>
        <div class="label">Total Tasks</div>
    </div>
    <div class="stat-card">
        <div class="value" id="statTodo">--</div>
        <div class="label">To Do</div>
    </div>
    <div class="stat-card">
        <div class="value" id="statInProgress">--</div>
        <div class="label">In Progress</div>
    </div>
    <div class="stat-card">
        <div class="value" id="statDone">--</div>
        <div class="label">Completed</div>
    </div>
</div>

<div style="display: flex; justify-content: space-between; align-items: center; margin-botto
    <h3>Recent Tasks</h3>
    <button class="btn btn-primary btn-sm" data-toggle="modal" data-target="#newTaskModal">
        New Task
    </button>
</div>

<div id="taskList">
    <p class="text-muted">Loading tasks...</p>
</div>

<!-- New Task Modal -->
<div class="modal" id="newTaskModal">
    <div class="modal-dialog">
        <div class="modal-content">
            <div class="modal-header">
                <h5 class="modal-title">Create Task</h5>
                <button type="button" class="close" data-dismiss="modal">&times;</button>
            </div>
            <div class="modal-body">
                <div class="form-group">
                    <label>Title</label>
                    <input type="text" class="form-control" id="taskTitle">
                </div>
                <div class="form-group">
                    <label>Description</label>
                    <textarea class="form-control" id="taskDescription" rows="3"></textarea>
                </div>
                <div class="form-group">
                    <label>Priority</label>
                    <select class="form-control" id="taskPriority">
                        <option value="low">Low</option>
                        <option value="medium" selected>Medium</option>
                        <option value="high">High</option>
                        <option value="urgent">Urgent</option>
                    </select>
                </div>
                <div class="form-group">
                    <label>Due Date</label>
                    <input type="date" class="form-control" id="taskDueDate">
```

```
                    </div>
                </div>
                <div class="modal-footer">
                    <button class="btn btn-secondary" data-dismiss="modal">Cancel</button>
                    <button class="btn btn-primary" onclick="createTask()">Create</button>
                </div>
            </div>
        </div>
    </div>

    <div id="alertArea" style="position: fixed; top: 80px; right: 24px; width: 300px; z-index: 1
{% endblock %}

{% block extra_js %}
<script>
    function loadStats() {
        frond.get("/api/dashboard/stats", function (data) {
            document.getElementById("statTotal").textContent = data.total;
            document.getElementById("statTodo").textContent = data.todo;
            document.getElementById("statInProgress").textContent = data.in_progress;
            document.getElementById("statDone").textContent = data.done;
        });
    }

    function loadTasks() {
        frond.get("/api/tasks?per_page=10", function (data) {
            var html = "";
            if (data.tasks.length === 0) {
                html = '<p class="text-muted">No tasks yet. Create your first task!</p>';
            } else {
                data.tasks.forEach(function (task) {
                    html += '<div class="task-row">'
                        + '<span class="priority-dot ' + task.priority + '"></span>'
                        + '<span class="task-title">' + task.title + '</span>'
                        + '<span class="badge badge-' + statusColor(task.status) + '">' + task.s
                        + (task.due_date ? '<span class="text-muted" style="font-size:0.85em;">'
                        + '</div>';
                });
            }
            document.getElementById("taskList").innerHTML = html;
        });
    }

    function statusColor(status) {
        var colors = { "todo": "secondary", "in_progress": "primary", "review": "warning", "done
        return colors[status] || "secondary";
    }

    function createTask() {
        var data = {
            title: document.getElementById("taskTitle").value,
            description: document.getElementById("taskDescription").value,
            priority: document.getElementById("taskPriority").value,
            due_date: document.getElementById("taskDueDate").value || null
        };

        frond.post("/api/tasks", data, function (result) {
            showAlert("Task created: " + result.title, "success");
            loadTasks();
```

```
            loadStats();
            // Clear form
            document.getElementById("taskTitle").value = "";
            document.getElementById("taskDescription").value = "";
        }, function (error) {
            showAlert("Error creating task", "danger");
        });
    }

    function showAlert(message, type) {
        var area = document.getElementById("alertArea");
        area.innerHTML = '<div class="alert alert-' + type + '">' + message + '</div>';
        setTimeout(function () { area.innerHTML = ""; }, 3000);
    }

    // Initial load
    loadStats();
    loadTasks();

    // Auto-refresh every 30 seconds
    setInterval(function () {
        loadStats();
        loadTasks();
    }, 30000);
</script>
{% endblock %}
```

Dashboard Stats Route

Create `src/routes/dashboard.php`:

```
<?php
use Tina4\Router;

/**
 * @secured
 */
Router::get("/api/dashboard/stats", function ($request, $response) {
    $userId = $request->user["user_id"];

    $task = new Task();
    $filter = "(created_by = :uid OR assigned_to = :uid2)";
    $params = ["uid" => $userId, "uid2" => $userId];

    $all = $task->select("*", $filter, $params);

    $stats = [
        "total" => count($all),
        "todo" => 0,
        "in_progress" => 0,
        "review" => 0,
        "done" => 0
    ];

    foreach ($all as $t) {
        if (isset($stats[$t->status])) {
            $stats[$t->status]++;
        }
    }
```

```
    return $response->json($stats);
});

/**
 * @noauth
 */
Router::get("/admin", function ($request, $response) {
    return $response->render("app/dashboard.html", [
        "page" => "dashboard"
    ]);
});
```

---

7. Step 5: Real-Time Updates via WebSocket

Real-time task updates. All connected dashboard users see changes the moment they happen.

Create `src/routes/websocket.php`:

```
<?php
use Tina4\WebSocket;

WebSocket::on("connect", function ($client) {
    error_log("Client connected: " . $client->id);
});

WebSocket::on("disconnect", function ($client) {
    error_log("Client disconnected: " . $client->id);
});

WebSocket::on("message", function ($client, $message) {
    $data = json_decode($message, true);

    if ($data["type"] === "subscribe" && $data["channel"] === "tasks") {
        WebSocket::subscribe($client, "tasks");
    }
});
```

Broadcast task changes by updating the task routes. Add this helper function at the top of `src/routes/tasks.php`:

```
function broadcastTaskUpdate(string $event, array $taskData): void
{
    \Tina4\WebSocket::broadcast("tasks", json_encode([
        "type" => "task_update",
        "event" => $event,
        "task" => $taskData
    ]));
}
```

Call it after each write operation:

```
// After creating a task
$task->save();
broadcastTaskUpdate("created", $task->toDetailedDict());

// After updating a task
$task->save();
```

```
broadcastTaskUpdate("updated", $task->toDetailedDict());

// After deleting a task
$task->delete();
broadcastTaskUpdate("deleted", ["id" => $request->params["id"]]);
```

Add WebSocket client code to the dashboard template. Add this inside the `{% block extra_js %}` block:

```
// WebSocket connection for real-time updates
var ws = new WebSocket("ws://localhost:7145/ws");

ws.onopen = function () {
    ws.send(JSON.stringify({ type: "subscribe", channel: "tasks" }));
};

ws.onmessage = function (event) {
    var data = JSON.parse(event.data);
    if (data.type === "task_update") {
        loadTasks();
        loadStats();
        showAlert("Task " + data.event + ": " + (data.task.title || ""), "info");
    }
};

ws.onclose = function () {
    // Reconnect after 3 seconds
    setTimeout(function () {
        ws = new WebSocket("ws://localhost:7145/ws");
    }, 3000);
};
```

Any user creates, updates, or deletes a task. All connected dashboard users see the change.

---

8. Step 6: Email Notifications on Task Assignment

A task is assigned to a user. Send them an email notification.

Create `src/routes/notifications.php`:

```
<?php
use Tina4\Mail;

function sendTaskAssignmentEmail(Task $task, User $assignee, User $assigner): void
{
    $subject = "New task assigned: " . $task->title;

    $body = "Hi " . $assignee->name . ",\n\n"
        . $assigner->name . " has assigned you a new task:\n\n"
        . "Title: " . $task->title . "\n"
        . "Priority: " . strtoupper($task->priority) . "\n"
        . "Due: " . ($task->dueDate ?? "No due date") . "\n\n"
        . "Description:\n" . ($task->description ?: "(No description)") . "\n\n"
        . "View it at: http://localhost:7145/admin\n";

    Mail::send(
        $assignee->email,
```

```
        $subject,
        $body
    );
}
```

In the task creation and update routes, call this function when `assigned_to` is set:

```
// After setting assigned_to and saving
if ($task->assignedTo && $task->assignedTo !== $userId) {
    $assignee = new User();
    $assignee->load($task->assignedTo);

    $assigner = new User();
    $assigner->load($userId);

    if (!empty($assignee->id) && !empty($assigner->id)) {
        sendTaskAssignmentEmail($task, $assignee, $assigner);
    }
}
```

Configure email in `.env`:

```
TINA4_MAIL_HOST=smtp.example.com
TINA4_MAIL_PORT=587
TINA4_MAIL_USERNAME=notifications@example.com
TINA4_MAIL_PASSWORD=your-email-password
TINA4_MAIL_FROM=notifications@example.com
TINA4_MAIL_FROM_NAME=TaskFlow
```

For development, you can use a local mail trap like MailHog or Mailtrap.io so emails are captured without actually sending.

---

9. Step 7: Add Caching for the Dashboard

The dashboard stats query runs on every page load. With many tasks, this gets slow. Cache the stats. Compute once. Serve from cache for subsequent requests.

Update the dashboard stats route:

```
/**
 * @secured
 */
Router::get("/api/dashboard/stats", function ($request, $response) {
    $userId = $request->user["user_id"];
    $cacheKey = "dashboard_stats_" . $userId;

    // Try cache first
    $cached = \Tina4\Cache::get($cacheKey);
    if ($cached !== null) {
        return $response->json($cached);
    }

    // Compute stats
    $task = new Task();
    $filter = "(created_by = :uid OR assigned_to = :uid2)";
    $params = ["uid" => $userId, "uid2" => $userId];
```

```
    $all = $task->select("*", $filter, $params);

    $stats = [
        "total" => count($all),
        "todo" => 0,
        "in_progress" => 0,
        "review" => 0,
        "done" => 0
    ];

    foreach ($all as $t) {
        if (isset($stats[$t->status])) {
            $stats[$t->status]++;
        }
    }

    // Cache for 60 seconds
    \Tina4\Cache::set($cacheKey, $stats, 60);

    return $response->json($stats);
});
```

Invalidate the cache when tasks change. Add this to the `broadcastTaskUpdate` function:

```
function broadcastTaskUpdate(string $event, array $taskData): void
{
    // Invalidate dashboard cache for all affected users
    if (isset($taskData["created_by"])) {
        \Tina4\Cache::delete("dashboard_stats_" . $taskData["created_by"]);
    }
    if (isset($taskData["assigned_to"]) && $taskData["assigned_to"]) {
        \Tina4\Cache::delete("dashboard_stats_" . $taskData["assigned_to"]);
    }

    \Tina4\WebSocket::broadcast("tasks", json_encode([
        "type" => "task_update",
        "event" => $event,
        "task" => $taskData
    ]));
}
```

---

10. Step 8: Write Tests

Create `tests/TaskFlowTest.php`:

```
<?php
use Tina4\Test;

class TaskFlowTest extends Test
{
    private ?string $token = null;
    private ?int $userId = null;

    public function setUp(): void
    {
        // Register a test user
        $email = "test-" . uniqid() . "@taskflow.test";
```

```
        $regResponse = $this->post("/api/auth/register", [
            "name" => "Test User",
            "email" => $email,
            "password" => "testpassword123"
        ]);

        $regBody = json_decode($regResponse->body, true);
        $this->userId = $regBody["id"] ?? null;

        // Login to get token
        $loginResponse = $this->post("/api/auth/login", [
            "email" => $email,
            "password" => "testpassword123"
        ]);

        $loginBody = json_decode($loginResponse->body, true);
        $this->token = $loginBody["token"] ?? null;
    }

    // --- Auth Tests ---

    public function testRegistrationReturns201()
    {
        $response = $this->post("/api/auth/register", [
            "name" => "New User",
            "email" => "new-" . uniqid() . "@test.com",
            "password" => "securepassword"
        ]);
        $this->assertEqual($response->statusCode, 201, "Registration should return 201");
    }

    public function testRegistrationRejectsShortPassword()
    {
        $response = $this->post("/api/auth/register", [
            "name" => "New User",
            "email" => "short-" . uniqid() . "@test.com",
            "password" => "abc"
        ]);
        $this->assertEqual($response->statusCode, 400, "Should reject short password");
    }

    public function testLoginReturnsToken()
    {
        $this->assertNotNull($this->token, "Login should return a token");
        $this->assertTrue(strlen($this->token) > 20, "Token should be substantial");
    }

    public function testProfileRequiresAuth()
    {
        $response = $this->get("/api/profile");
        $this->assertEqual($response->statusCode, 401, "Profile should require auth");
    }

    public function testProfileWithToken()
    {
        $response = $this->get("/api/profile", [
            "Authorization" => "Bearer " . $this->token
        ]);
        $this->assertEqual($response->statusCode, 200, "Profile should work with token");
```

```
}

// --- Task Tests ---

public function testCreateTask()
{
    $response = $this->post("/api/tasks", [
        "title" => "Test Task",
        "priority" => "high"
    ], ["Authorization" => "Bearer " . $this->token]);

    $this->assertEqual($response->statusCode, 201, "Should create task");

    $body = json_decode($response->body, true);
    $this->assertEqual($body["title"], "Test Task", "Title should match");
    $this->assertEqual($body["priority"], "high", "Priority should match");
    $this->assertEqual($body["status"], "todo", "Default status should be todo");
}

public function testCreateTaskRequiresTitle()
{
    $response = $this->post("/api/tasks", [
        "priority" => "low"
    ], ["Authorization" => "Bearer " . $this->token]);

    $this->assertEqual($response->statusCode, 400, "Should reject task without title");
}

public function testListTasks()
{
    // Create a task first
    $this->post("/api/tasks", [
        "title" => "List Test Task"
    ], ["Authorization" => "Bearer " . $this->token]);

    $response = $this->get("/api/tasks", [
        "Authorization" => "Bearer " . $this->token
    ]);

    $this->assertEqual($response->statusCode, 200, "Should list tasks");

    $body = json_decode($response->body, true);
    $this->assertTrue($body["count"] > 0, "Should have at least one task");
}

public function testUpdateTaskStatus()
{
    // Create a task
    $createResponse = $this->post("/api/tasks", [
        "title" => "Status Test Task"
    ], ["Authorization" => "Bearer " . $this->token]);

    $taskId = json_decode($createResponse->body, true)["id"];

    // Update status to done
    $updateResponse = $this->put("/api/tasks/" . $taskId, [
        "status" => "done"
    ], ["Authorization" => "Bearer " . $this->token]);
```

```
        $this->assertEqual($updateResponse->statusCode, 200, "Should update task");

        $body = json_decode($updateResponse->body, true);
        $this->assertEqual($body["status"], "done", "Status should be done");
        $this->assertNotNull($body["completed_at"], "Should have completion timestamp");
    }

    public function testDeleteTask()
    {
        // Create a task
        $createResponse = $this->post("/api/tasks", [
            "title" => "Delete Me"
        ], ["Authorization" => "Bearer " . $this->token]);

        $taskId = json_decode($createResponse->body, true)["id"];

        // Delete it
        $deleteResponse = $this->delete("/api/tasks/" . $taskId, [
            "Authorization" => "Bearer " . $this->token
        ]);

        $this->assertEqual($deleteResponse->statusCode, 204, "Should return 204");

        // Verify it is gone
        $getResponse = $this->get("/api/tasks/" . $taskId, [
            "Authorization" => "Bearer " . $this->token
        ]);

        $this->assertEqual($getResponse->statusCode, 404, "Should return 404 after deletion");
    }

    public function testDashboardStats()
    {
        // Create tasks with different statuses
        $this->post("/api/tasks", ["title" => "Todo Task", "status" => "todo"],
            ["Authorization" => "Bearer " . $this->token]);
        $this->post("/api/tasks", ["title" => "In Progress Task", "status" => "in_progress"],
            ["Authorization" => "Bearer " . $this->token]);
        $this->post("/api/tasks", ["title" => "Done Task", "status" => "done"],
            ["Authorization" => "Bearer " . $this->token]);

        $response = $this->get("/api/dashboard/stats", [
            "Authorization" => "Bearer " . $this->token
        ]);

        $this->assertEqual($response->statusCode, 200, "Should return stats");

        $body = json_decode($response->body, true);
        $this->assertTrue($body["total"] >= 3, "Should have at least 3 tasks");
        $this->assertTrue($body["todo"] >= 1, "Should have at least 1 todo");
        $this->assertTrue($body["in_progress"] >= 1, "Should have at least 1 in progress");
        $this->assertTrue($body["done"] >= 1, "Should have at least 1 done");
    }
}
```

Run the tests:

```
tina4 test

Running tests...
```

```
  TaskFlowTest
    [PASS] test_registration_returns_201
    [PASS] test_registration_rejects_short_password
    [PASS] test_login_returns_token
    [PASS] test_profile_requires_auth
    [PASS] test_profile_with_token
    [PASS] test_create_task
    [PASS] test_create_task_requires_title
    [PASS] test_list_tasks
    [PASS] test_update_task_status
    [PASS] test_delete_task
    [PASS] test_dashboard_stats

  11 tests, 11 passed, 0 failed (0.62s)
```

---

11. Step 9: Deploy with Docker

Create `Dockerfile`:

```
FROM dunglas/frankenphp:latest-php8.3-alpine

RUN install-php-extensions \
    pdo_sqlite \
    mbstring \
    openssl \
    fileinfo

WORKDIR /app

COPY composer.json composer.lock ./
RUN curl -sS https://getcomposer.org/installer | php -- --install-dir=/usr/local/bin --filename=
    && composer install --no-dev --optimize-autoloader --no-interaction

COPY . .

RUN mkdir -p data logs secrets \
    && chown -R www-data:www-data data logs secrets

EXPOSE 7145

HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
    CMD curl -f http://localhost:7145/health || exit 1

CMD ["tina4", "serve", "--production"]
```

Create `docker-compose.yml`:

```
version: "3.8"

services:
  app:
    build: .
    ports:
      - "7145:7145"
    environment:
      - TINA4_DEBUG=false
      - TINA4_LOG_LEVEL=WARNING
```

```
      - TINA4_TEMPLATE_CACHE_TTL=true
      - JWT_SECRET=${JWT_SECRET:-change-me-in-production}
      - TINA4_DATABASE_URL=sqlite:///data/app.db
    volumes:
      - taskflow-data:/app/data
      - taskflow-logs:/app/logs
    restart: unless-stopped
    stop_grace_period: 35s

volumes:
  taskflow-data:
  taskflow-logs:
```

Build and deploy:

```
# Build
docker compose build

# Start
docker compose up -d

# Run migrations inside the container
docker compose exec app tina4 migrate

# Verify
curl http://localhost:7145/health
```

```
{
  "status": "ok",
  "database": "connected",
  "uptime_seconds": 5,
  "version": "3.0.0",
  "framework": "tina4-php"
}
```

---

12. The Complete Project Structure

```
taskflow/
■■■ .env
■■■ .env.example
■■■ .gitignore
■■■ composer.json
■■■ composer.lock
■■■ Dockerfile
■■■ docker-compose.yml
■■■ vendor/
■■■ src/
■   ■■■ routes/
■   ■   ■■■ auth.php              # Registration, login, profile
■   ■   ■■■ tasks.php             # Task CRUD
■   ■   ■■■ dashboard.php         # Dashboard stats + page
■   ■   ■■■ notifications.php     # Email notification helpers
■   ■   ■■■ websocket.php         # WebSocket event handlers
■   ■■■ orm/
■   ■   ■■■ User.php              # User model with auth methods
■   ■   ■■■ Task.php              # Task model with relationships
■   ■■■ migrations/
■   ■   ■■■ 20260322000100_create_users_table.sql
■   ■   ■■■ 20260322000200_create_tasks_table.sql
■   ■■■ templates/
■   ■   ■■■ app/
■   ■   ■   ■■■ layout.html       # Base layout with sidebar
■   ■   ■   ■■■ dashboard.html    # Dashboard page
■   ■   ■■■ errors/
■   ■       ■■■ 404.html
■   ■       ■■■ 500.html
■   ■■■ public/
■   ■   ■■■ css/
■   ■   ■   ■■■ tina4.css
■   ■   ■■■ js/
■   ■       ■■■ frond.js
■   ■■■ locales/
■       ■■■ en.json
■■■ data/
■   ■■■ app.db
■■■ logs/
■■■ secrets/
■■■ tests/
    ■■■ TaskFlowTest.php
```

Every file has a purpose. Every directory follows the convention. A new developer looks at this structure and knows where to find things.

---

13. What to Build Next

TaskFlow is a solid foundation. Ideas for extending it:

Features:

- **Task comments** -- Add a Comment model with a `task_id` foreign key. Display comments on the task detail page.

- **File attachments** -- Let users upload files to tasks. Store them in `data/uploads/` and serve them via a route.
- **Team management** -- Add a Team model. Users belong to teams. Tasks are scoped to teams.
- **Task labels/tags** -- Many-to-many relationship between tasks and labels for categorization.
- **Due date reminders** -- Use the queue system to schedule reminder emails 24 hours before a task's due date.
- **Activity log** -- Record every change to a task (who changed what, when) for audit trails.
- **Search** -- Full-text search across task titles and descriptions.
- **Calendar view** -- Render tasks on a calendar based on their due dates.
- **Mobile API** -- The API already works for mobile apps. Add push notification support via Firebase Cloud Messaging.

Technical improvements:

- **Rate limiting per user** -- Replace the global rate limiter with per-user limits.
- **Database upgrade** -- Switch from SQLite to PostgreSQL for better concurrency.
- **CI/CD pipeline** -- Add GitHub Actions to run tests automatically on every push.
- **API documentation** -- Generate OpenAPI/Swagger docs from your route definitions.
- **Internationalization** -- Add `src/locales/` files for multiple languages.

---

14. Closing Thoughts -- The Tina4 Philosophy

You built a complete application. User auth. CRUD. Real-time updates. Email. Caching. Tests. Deployment. Your project has one dependency: `tina4/tina4-php`.

No ORM package. No template engine package. No authentication library. No WebSocket server. No caching library. No testing framework. No CLI tool. No CSS framework. No JavaScript helpers. All built in.

One framework. Zero dependencies. Everything you need.

The same patterns work in Python, Ruby, and Node.js. Same project structure. Same CLI commands. Same `.env` variables. Same template syntax. Learn Tina4 once. Use it everywhere.

Your `vendor/` directory is small. Your `composer.lock` has one entry. When PHP 9.0 ships, you update one package. Everything works. No dependency tree to untangle. No abandoned transitive dependency to replace. No security advisory for a package four levels deep that you never knew you were using.

Simple does not mean limited. TaskFlow has authentication, real-time WebSocket, email, caching, GraphQL, and a test suite. It deploys in a Docker container. It handles thousands of concurrent users. All of this runs on a single zero-dependency framework package.

Build things. Ship them. Keep it simple.

Release Notes

v3.13.138 (2026-09-24) - Safer requests, templates, and database operations

Licensing from this release. Tina4 is available under MPL-2.0, with separate commercial terms available from Code Infinity. Copyright Code Infinity. Previously published releases retain their original licences.

This release brings together the framework hardening work, database correctness fixes, measured-time guidance for the AI skills, and verifiable release artifacts.

Request and template boundaries. Frond escapes output by default and preserves only explicitly trusted output. Review templates that intentionally render HTML when upgrading; never mark untrusted input as safe. Header validation, request-body limits, security headers, CORS handling, model-field validation, and diagnostic redaction follow the shared contracts. Explicit Content-Type headers survive response conversion and binary bodies preserve bytes.

Credentials and development access. API clients retain credentials only for their configured origin, including uploads, downloads and streams. Use a separate client when authenticating to another service. Forwarded host and protocol require a trusted proxy. Development reads and writes enforce the raw-peer, Host and browser-origin boundary; remote non-MCP development access requires the dedicated token. The documented MCP transport API-key fallback is unchanged. File access checks resolved paths so symlinks cannot expose secrets or escape the project.

Database and service correctness. Statements that return rows retain those rows, filtered counts and write results agree across adapters, SQL placeholders preserve literal text, and ORM fields mapped to database columns read back correctly. Database operations and explicit transactions reserve their connection; a fully occupied pool fails clearly instead of sharing an in-use connection. Session transports reuse MongoDB connections and handle replica-set replies. Mail transports verify certificates and require STARTTLS when configured. Optional driver errors identify the needed install, and unconfirmed broker writes fail clearly.

Skills and release evidence. The maintainer skill estimates work from measured agent durations. Release packages carry checksums, an SPDX software inventory, and build provenance; release Actions are pinned to commits. The updated PowerShell skills installer is digitally signed. The ISO programme remains in progress; this release makes no certification or conformance claim.

The accompanying CLI 3.8.91 verifies update checksums before replacing an installed binary, refuses missing checksum manifests, and corrects generated Node.js package imports. It also includes the browser-opening and CI behavior fixes. Tina4 JS 1.7.2 ships the merged client hardening and verified browser bundle. Refresh the AI skills with `curl -fsSL https://tina4.com/install-skills.sh | sh` or the documented PowerShell installer.

v3.13.105 (2026-08-19) - Cross-API invariants, honest logs, portable tests

A bug release with teeth. Five audit bugs closed in the queue and ORM cache seams — the places where two spellings of the same intent quietly disagreed. Route inspection stops booting the app. The startup log stops lying about `@noauth`. Hot reload reaches the modules that captured the changed one. Firebird's migration ledger tolerates any case the driver hands back. And every framework developer skill gained a spine: it announces every step before taking it, and it warns with ■ when it is out of date.

Queue and ORM audit bugs — five fixed

- `Model.clear_cache()` **invalidates the DB layer too.** When both

`TINA4_AUTO_CACHING` and `TINA4_DB_CACHE` are enabled, a manual `clear_cache()` used to leave the DB-layer cache holding stale rows. It now cascades to `db.cache_clear()` on this model's bound connection.

- `Queue.retry()` **revives every dead letter.** The no-arg form used a

generator inside `any(...)`, which short-circuits after the first success. Now it materialises the loop so all N dead letters revive, not just one.

- `Job.retry()` **removes the dead-letter file.** Iterating `dead_letters()`

and calling `.retry()` on each used to leave the failed directory carrying every revived job. The next `dead_letters()` call re-reported them and consumers processed each job twice.

- **MongoDB** `retry_job(id)` **finds the dead letter.** The old filter looked

for `{_id: job_id, topic: self._topic, status: "failed"}` — three reasons that combination could never match a real dead-letter doc (fresh `_id`, `.dead_letter` topic namespace, `dead` status). The new filter reads the dead-letter namespace by `data.id`, deletes the record, and upserts the original back to pending.

- **MongoDB** `purge(status)` **returns a count.** Used to return `None` and

honour only the `pending` status. Now returns `deleted_count`, honours every status, and scopes the delete so `purge("pending")` no longer nukes completed or reserved docs under the same topic.

Route inspection scans, never boots

`tina4 routes` walks canonical route files without executing `app.py` or starting the server. The prior implementation booted the app on the same port and terminated whichever process was holding it — a nasty surprise for anyone running `tina4 routes` in a live shell. Closes tina4-python #104.

Startup log tells the truth about @noauth / @secured

Python decorators apply bottom-up, so `@post()` logged the route as `auth=required` a microsecond before the outer `@noauth()` flipped the flag. `@noauth()` and `@secured()` now emit a corrective `Route auth updated:` line whenever they change the auth default on a route another decorator already logged. Closes tina4-python #103, where a public login endpoint appeared to log its own security bug. Python-only — PHP, Ruby and Node do not log an auth field at registration time.

Hot reload reaches every module that imports the changed one

A route file that did `from src.orm.Todo import Todo` kept its stale `Todo` reference after the model changed — the API then lied about the schema on the next request. `_auto_discover` now cascades a re-import to every module that captured a binding from the changed one, recursive with a visited set, bounded to the discovery scope, fail-loud. Closes tina4-python #102.

Firebird migration ledger is case-agnostic

`tina4_migration` reads and writes work regardless of the case the Firebird driver returns for the `migration_name` column. Uses the atomic sequence pattern the other engines already have.

Test-harness fixes

- **SIGTERM port probe checks both interfaces.** The graceful-shutdown test

probed only `0.0.0.0` with `SO_REUSEADDR`, which succeeds on macOS while the framework's default `127.0.0.1` listener holds the port — a silent false negative on every Mac dev machine. Now probes both interfaces and refuses to conclude the port is free until both agree.

- **MySQL connect-timeout assertion tolerates driver cleanup time.** The

assertion compared the outer clock to the wrapper's rendered message. mysql-connector adds around 0.7 seconds of cleanup after aborting, so the two stopwatches legitimately differed. The assertion now parses the reported elapsed via regex and checks it against the configured bound, not against the outer clock.

Skills grow a spine

- **Announce before you act.** Every framework developer skill now instructs

itself to say what it is about to do — one line before each file write, migration, or dependency install — so the developer can stop the work before it spends their afternoon. Same rhythm across Python, PHP, Ruby and Node.

- ■ **stale-skill detection.** Every framework developer skill also checks

whether a newer skill is available (compared against the latest published skill version at `https://tina4.com/skills/<name>/version`, never against the project's framework version) and prepends ■ to every reply if it is behind. Your framework version stays your call; the skill just refuses to pretend its advice is current.

Upgrade notes

- No API changes. All fixes preserve existing signatures.
- Consumers iterating `dead_letters()` and calling `job.retry()` on each — now safe. Previously it produced duplicates on the next `dead_letters()` call.
- `Model.clear_cache()` under `TINA4_DB_CACHE=true` now also flushes the DB-layer cache. This is the expected behaviour.
- Lab full-suite verified: 25,263 tests across all four frameworks, zero failures, zero skips.

v3.13.104 (2026-08-17) - OIDC SSO and PostGIS land

Two big features, uniform across all four frameworks: a provider-neutral OIDC handoff for SSO logins, and a shared `Point` contract for PostGIS geospatial queries. Both wire through the same small API in Python, PHP, Ruby and Node.

OIDC / OpenID Connect — provider-neutral, session handoff

`Sso.from_issuer(...)` discovers a real OIDC provider (Keycloak, Auth0, Okta, anything speaking the standard) via its `.well-known/openid-configuration` endpoint and hands a validated identity into an existing Tina4 `Session`. The `login` / `callback` / `refresh` / `logout` cycle runs against the real provider — no mock. PKCE with S256 by default. Reserved SSO session keys (`Sso.PENDING_KEY`, `Sso.SESSION_KEY`) never leak through `session.all()`.

Opt in with `TINA4_SSO_ISSUER` plus the client id, secret and redirect URI.

PostGIS spatial support

`PointField` on an ORM model creates the right column for whichever engine you are on — PostGIS `geometry(Point, SRID)` on PostgreSQL, JSON on the others. `select_distance`, `intersects` and `bbox` predicates read the same way across engines but use each engine's native functions where available. GeoJSON in, GeoJSON out. Refuses hostile input (garbage geometry, injected column names, NaN or Inf ordinates) before any SQL runs.

Opt in by declaring a `PointField` on a model. Existing models unchanged.

Swagger is SSO-aware

The Swagger generator recognises OIDC-secured routes and emits the right `security` scheme in the OpenAPI 3.0.3 spec.

v3.13.102 (2026-08-15) - Unified client and CI hardening

Small release. Framework developer skills teach the unified `tina4 init` / `tina4 serve` workflow consistently across the four languages, and CI now verifies the downloaded native CLI asset before running the metrics handoff.

v3.13.103 (2026-08-16) - Metrics you can trust, releases that cannot lie

This release restores one public version across Python, PHP, Ruby, and Node.js. Version 3.13.102 reached PyPI and Packagist, but its RubyGems and npm packages still contained 3.13.101. Skip 3.13.102 and upgrade to 3.13.103.

What changed

• Dev-admin hands metrics work to the signed Tina4 client 3.8.76. Frameworks do

not keep a second metrics analyser or silently fall back to one.

• `has_referencing_test` means that test source refers to the file. It does not

claim that the test ran, passed, or covered the file.

• Frond expression parsing and evaluation are split into smaller internal steps,

while public APIs and the shared 84-case expression corpus remain unchanged.

• Tina4 skills lead with `tina4 init` and `tina4 serve`, and keep scaffolding

prominent in every language.

• Release workflows now reject a tag that differs from the source package

version before publishing.

No runtime packages were added. Language extensions remain extensions, not framework dependencies.

PHP connection timeout diagnosis

PostgreSQL's explicit `timeout expired` result is now classified as the configured connection timeout at the clock boundary. Immediate connection refusals remain ordinary connection errors.

+## v3.13.101 (2026-08-14) - One AI client, four languages

One class. Three methods. No provider SDK. Tina4 applications can now call local models, OpenAI, and Anthropic through the same small API: `AI::chat`, `AI::complete`, and `AI::embed`.

What ships

- The native `tina4 metrics` CLI now owns every code-health calculation. Framework-level metrics commands and quick census implementations are gone.
- Dev-admin keeps its metrics tab through two thin native-JSON adapters: full analysis and file detail.
- `AI::chat` returns one `ChatResponse` with text, model, token usage, finish reason, and the raw provider response.
- `AI::complete` sends one user message and returns its text.
- `AI::embed` accepts one string or a list and preserves that input shape in its output.

- `stream: true` yields ordered text deltas from OpenAI-compatible and Anthropic streams.

Failure is data

Hosted providers refuse to send a request without `TINA4_AI_KEY`. Connection and total-request deadlines stay separate. Retries cover connection failures, HTTP 429, and HTTP 5xx responses, but stop after the first streamed delta. Errors never include the key, prompt, or provider response body.

Breaking name change

PHP class names ignore case, so the old developer-tool installer could not remain `Tina4\AI` beside the new application client. The installer is now `Tina4\AITools`. Application code uses `Tina4\AI`. The transport uses PHP's built-in streams; your dependency tree does not grow.

v3.13.100 (2026-08-14) - Frond keeps the whole page

This fast-follow release fixes template inheritance and puts a ceiling on Frond's caches. The template engine now rejects a second `{% extends %}` tag, preserves nested root blocks, and evicts stale compiled state instead of letting a long-running process grow without limit.

Template inheritance

- A template may declare one parent. A second `{% extends %}` now raises a clear error instead of changing the parent without warning (`c7739aa0`)
- PHP's AST substitution already preserved nested root blocks. A shared regression now locks that correct behavior across all four frameworks (`955c66e0`)

Bounded caches

- Template, fragment, and expression caches now have size bounds and TTL sweeps. Long-running workers release stale entries while rendered output stays unchanged (`236dbabc`)

Frond extension scope

- Class calls to `addFilter`, `addGlobal`, and `addTest` remain process-global. Instance calls now affect that renderer only, so one request or test cannot seed every Frond instance created later (`f6fb7f23`)

AI skill installer

- Skill downloads retry transient network and HTTP failures. A short GitHub raw-content outage no longer drops the whole install (`aaeba4fc`, `45730d7f`)

Release consistency

- The runtime version and AI-facing guide now agree on `3.13.100`. The existing version guard caught the stale guide before release (`aaf5ffa9`)

Upgrade notes

No valid template syntax changes. Code that relied on an instance extension leaking globally must register on `Frond` itself. A template with two `{% extends %}` tags was already contradictory; it now fails at the line that needs fixing. Cache eviction may trigger a reparse, but it does not change the rendered bytes.

The Frond AOT compiler is not part of this release. Parser and compiler parity for Ruby and Node.js remains planned for `3.13.101`.

v3.13.99 (2026-08-13) - Stability, parity, and secure by default

The biggest step yet on the road to **3.14 stable**. This release closes out Phases 1 through 5 of the v3 parity audit: roughly thirty breaking changes, and every one of them is a security fix, a parity fix, or a correctness fix, never a change made for its own sake. Two conformance grids, logging and database adapters, are now fully proven against real services in all four frameworks, and a small batch of genuinely new bugs gets fixed alongside them, including one that broke first-time login under the built-in server. Read "Possible breaking" before you upgrade: several of these changes correct a footgun that was silently doing the wrong thing.

Security

Security now defaults on instead of requiring opt-in.

- Security headers emit by default, including `Content-Security-Policy: default-src 'self'`, which blocks inline scripts and third-party CDNs. Relax the policy with `TINA4_CSP`; HSTS emits only on HTTPS, when `TINA4_HSTS` is set (`853c47c`)
- `TINA4_CSRF=true` now actually attaches the CSRF middleware, where it used to be inert. A blank `TINA4_SECRET` fails closed now instead of minting a forgeable public-default token (`001f966a`)
- The dev server binds `127.0.0.1` by default; set `TINA4_HOST=0.0.0.0` to expose it. A cross-origin `/__dev` mutation is refused, and `.env` is never served through the file endpoints (`698e6a6`)
- A symlink whose real path escapes the public directory is refused, and dotfiles (`.env`, `.git`) 404 instead of serving. The public-directory search order is now `public` before `src/public` (`c422f4b`)
- `{% include %}`, `{% extends %}`, and `{% import %}` in a Frond template are confined to the templates directory. A path that escapes it, with `..`, an absolute path, or a symlink, now raises (`42a0723`)
- `tina4 serve` on a busy port no longer kills whatever holds it. It reclaims only a port held by an identifiable Tina4 dev server, refuses a foreign holder, and honours `TINA4_NO_TAKEOVER` / `--no-kill` (`40cd4c0`)
- A hostile inbound `X-Request-ID` (CRLF, illegal characters, an over-long value) is sanitized to a fresh id instead of echoed back raw, closing a response-header and log-injection path (`1d9d607`)

• The dead `renderProductionError` function is removed; nothing called it. The dev error overlay now redacts `Authorization`, `Cookie`, and `Set-Cookie` headers plus secret-looking body fields, and caps the rendered stack at 50 frames (`df860de`)

• A repeated multipart file field now yields a list instead of silently dropping every upload but the last. The new safe-save helper rejects `..` and absolute filenames, and an over-limit upload now answers `413` mid-stream, per chunk, instead of after buffering the whole body (`a6bb4b3`)

Data integrity

Footguns that used to fail silently now fail loud, or stop failing at all.

• An unparseable or unsupported MongoDB WHERE clause now raises instead of silently matching every document. A DELETE or UPDATE with no WHERE is rejected outright. Write an explicit WHERE, or call `truncate()` for the whole collection (`baf1af5`)

• `truncate()` on a Mongo collection was already correct in PHP; the other three frameworks now match it (`7ac2c0b`)

• MongoDB's next-id generator now raises on error instead of silently returning `1`, which used to produce duplicate ids (`833cceb`)

• Firebird write results are correct now: `$db->insert()->lastId` and `$db->update()` / `$db->delete()` `->affectedRows` return real values instead of a missing or zero value (`27dd66e`)

• MSSQL pagination converges on `OFFSET`/`FETCH` in all four frameworks; binary parameters travel as `0x` literals (`1deb9d6`)

• `App::background()` now returns a `Tina4\BackgroundTask` handle instead of `$this` (it used to be fluent). Split a chained `->background(a)->background(b)` into two separate calls (`9f62bde`)

ORM and validation

Eight fixes bring the ORM's write path, validation, and generated schema into agreement across all four frameworks.

• `datetime` name-inference is anchored now, so a column whose name only substring-matched (`runtime`, `downtime`, `updated_by`) is no longer typed as a datetime. Declare an explicit `\DateTime` property, or an `*_at` / `*_date` / `*_time` name, to keep it (`d97a405`)

• Ruby ORM `validate()` now enforces length/type/format on save, matching PHP; the two validators now speak one canonical message vocabulary, and PHP's own messages drop the colon (`"name: is required"` becomes `"name is required"`) (`cd9ab0d`)

• `create_table()` now injects the soft-delete column (`is_deleted INTEGER DEFAULT 0`) for a `softDelete = true` model that does not declare it itself (`3d10d5e`)

• A soft-deleted child is no longer returned through relationship traversal, lazy or eager (`94ccf77`)

• The imperative `hasMany` default row cap changes from a silent 100 to the whole result set. A caller relying on the old truncation now gets every row (`4e1d5df`)

• A REST list `?page` was already clamped to a minimum of 1 in PHP; an oversized `?limit` or `?per_page` is now capped at 100 across all four frameworks (`a8088ec`)

• AutoCrud returns `422` with field errors for an invalid create or update, where it used to return `500`. The write body is now allow-listed: `is_deleted` is never client-writable, and a client-supplied primary key is stripped on both create and update, closing a mass-assignment hole and an IDOR-shaped redirect (`51cd524`)

• `seed_table` now routes through the parameterized adapter insert instead of hand-written MySQL/SQLite backtick SQL. The generated SQL changes, and the dev-admin seed tool now works on PostgreSQL, MSSQL, and Firebird, where it used to be broken (`6208fbb`)

Request model - the big one

`$request->params` is route-params-only now, in all four frameworks. Client input lives only in `$request->query` and `$request->body`; `$request->params` holds route params and nothing else, closing a param-pollution surface where Ruby's equivalent used to merge query and body values in with the route params. A handler or middleware reading the old merged `params` must read `query` or `body` explicitly (`88edd95`).

Migrations

• `rollback` is fail-safe now: a missing `.down.sql` file or a failed down script raises and leaves the `tina4_migration` ledger row in place, instead of deleting the tracking row and leaving the schema applied but untracked (`49b404a`)

HTTP

• Your responses now gzip-compress when eligible (body over 1024 bytes, `Accept-Encoding: gzip`, a compressible content type); a cacheable 200 gets a strong ETag and a matching `If-None-Match` gets a 304. The static-file ETag format is unified to `W/"<size>-<mtime>"` on all four frameworks, so a cache revalidates once instead of on every deploy (`5f8e85e`)

• A `403` now negotiates HTML versus JSON the same way `404` and `500` already do, and the `404` now carries a `request_id` too (`ac26239`)

• The OpenAPI spec converges on one shape across languages: a secured operation documents a `401`, and `summary`/`tags` always populate. The Swagger UI CDN default moves to jsdelivr, off unpkg (`e28a08b`)

• Python, Ruby, and Node's route-group prefix join is normalized to match PHP's, so a route no longer mis-registers on a bare concatenation or an uncollapsed double slash (`7b8dc44`)

Dev tooling

• `tina4php serve` no longer crashes the process when the AI port (`base + 1000`) is busy; it warns and skips, matching the other three frameworks. `new Server()`'s default port changes from 7146 to 7145, which affects only a direct no-argument construct, since every real caller passes an explicit port (`fdf27d3`)

• The CLI's `--version` and `commands --json` now report the real framework version instead of `0.0.0` in a git checkout; outbound HTTP requests from the framework client now carry `User-Agent: Tina4/<version>` unless the caller already supplies one (`6a6ddc2`)

• The session cookie now emits under a testing `Response`, closing a gap the built-in server also hit (see "Bug fixes" below) (`1881b0a`)

• The inline `@tests` descriptor builders are renamed: `Testing::assertEqual`/`assertRaises`/`assertTrue`/`assertFalse` become `Testing::expectEqual`, `expectRaises`, `expectTrue`, `expectFalse`. `Testing::discover()` now scans only an explicit tests directory and parses `@tests` arguments as literals, never `eval`, so a docblock outside that directory is no longer discovered or executed. `bin/tina4php test` now discovers and runs the inline surface with a real exit code (`e797e9b`)

Proven in all four

Two conformance grids close out fully proven, tested against real services in every framework, no mocks:

• **Logging.** A real per-language runner now exercises the shared logger contract end to end. Closing it surfaced and fixed real bugs, including PHP's context circular-detection, which is rebuilt to be reference-aware (`8b14f11`)

• **Database adapters** (ADR-0044). A real runner against SQLite, PostgreSQL, MySQL, MSSQL, and Firebird proved every adapter implements the same interface. PHP's `DatabaseAdapter` interface still required the removed `query`/`lastInsertId`/`error` methods and was missing `connect`/`getDatabaseType`/`autocommit` - both `CachedDatabase` and `Database` would have hard-fatalled at class load. `Database::executeMany()` now delegates once instead of looping through the facade (`1488143`)

Bug fixes

• A route path containing a literal parenthesis, like `/products/(sale)`, now matches correctly everywhere. Python and Ruby were already correct; PHP and Node compiled the literal characters as regex syntax (`d12e5d8`)

• The built-in server's first-time session cookie is emitted correctly now. `emitSessionCookie()` branched on `headers_sent()`, which is never true under `Tina4\Server`'s raw socket, so a first-time login through `tina4 serve` silently produced no `Set-Cookie` header and session auth quietly failed. A new `Response::$rawSocket` flag (kept separate from the `testing` flag, which would also have broken SSE streaming) fixes it: `emitSessionCookie()` now checks `headers_sent() || isTesting() || isRawSocket()` (`d12e5d8`)

Possible breaking

Read these before you upgrade. Every entry here is a security, parity, or correctness fix, none is a change made for its own sake.

• `$request->params` **is route-params-only.** Read query-string values from `$request->query` and body values from `$request->body`. This is the single largest behaviour change in the release.

• **Security headers, including CSP, emit by default.** An app depending on inline scripts or a third-party CDN needs `TINA4_CSP` to relax the policy.

• `TINA4_CSRF=true` **now actually attaches the CSRF middleware.** If you set it and never noticed CSRF enforcement, it enforces now.

• **A blank** `TINA4_SECRET` **fails closed.** Set a real secret; the framework no longer falls back to a guessable public default.

• **The dev server binds** `127.0.0.1` **by default.** Set `TINA4_HOST=0.0.0.0` to expose it on your network.

• **An unparseable Mongo WHERE now raises instead of matching everything.** Add an explicit WHERE, or call `truncate()`.

• `App::background()` **returns a handle, not** `$this`**.** Split a chained call into two separate calls.

• `datetime` **name-inference is anchored.** A column that only substring-matched a datetime-ish name needs an explicit `\DateTime` property or a `*_at`/`*_date`/`*_time` name.

• **Validation messages changed wording**, including the dropped colon after a field name.

• `create_table()` **adds** `is_deleted` **for a soft-delete model automatically.** A model that already declares the column is unaffected.

• **The imperative** `hasMany` **no longer caps at 100 rows.** A caller relying on the old truncation now gets every row.

• `?limit`**/**`?per_page` **caps at 100.** A client can no longer request the whole table in one page.

• **AutoCrud returns** `422`**, not** `500`**, on invalid input**, and never accepts `is_deleted` or a client-supplied primary key in the write body.

• **The static-file** `ETag` **format changed**, so every cache revalidates once on upgrade.

• `new Server()`**'s default port changes from 7146 to 7145.** Only affects a direct no-argument construct.

• **The inline testing descriptors are renamed** `assert*` to `expect*`, and `Testing::discover()` no longer uses `eval` or scans outside the tests directory.

Coming in 3.13.100

Frond's compiler, extensibility, auto-escaping, sandboxing, and caching work (features 48 through 60) is deferred to the 3.13.100 fast-follow, alongside a refreshed Carbonah benchmark harness and a configurable database column-name casing option.

v3.13.98 (2026-08-11) - Skills discipline

A maintenance release. No API changed, and no behaviour changed in your app.

The bundled AI coding skills - the ones that install with `curl -fsSL https://tina4.com/install-skills.sh | sh` - moved onto one shared plan discipline. A plan file has one format (Scope, Tests, Bugs, Commits), and a checkbox is ticked only when the work is verified green at HEAD, never on a claim. That keeps an agent's plan file honest, so it always reflects what actually shipped.

The framework package also shed internal dead weight: duplicate design-system source files that were never compiled or served at runtime. Nothing your code imports or renders changed.

Nothing to upgrade. If you use the Tina4 AI skills, re-run the installer.

v3.13.97 (2026-08-07) - Behaviour corrections

A small bug-fix release on the road to **3.14 stable**. No new methods, no renames. Four behaviours come back into line with the Python reference. `forceDelete()` removes the row instead of throwing. A destroyed session stays destroyed. DocStore's `distinct()` counts two equal dates as one. And the queue tests that guard all of this can no longer pass while the code is broken.

ORM

- `forceDelete()` removes the row. It referenced an undefined `$whereParams`, so the `:id` never bound and the call threw a `TypeError` instead of deleting anything. It binds the id and deletes now (`91c33b5a`)

Sessions

- `destroy()` no longer resurrects. A `set()` then `save()` after `destroy()` re-created the session under the id you just destroyed. `destroy()` clears the session id now, so a later `save()` writes nothing (`146b5088`)

DocStore

- `distinct()` dedups by value. It compared non-ObjectId values by object identity, so two equal `DateTime` values came back as separate rows. It compares by value now, and equal dates collapse to one (`198b18bb`)

Under the hood

- The broker queue already refused `clear()` and `purge()` by name. A real RabbitMQ test now locks that refusal so it cannot regress (`d2e66ebd`)
- The queue-invariant tests were hiding a ghost. A `$this->fail()` inside a `catch (\RuntimeException)` was swallowed, because PHPUnit's `AssertionFailedError` extends `\RuntimeException`, so a test stayed green even after the code stopped refusing. The asserts moved outside the catch, and a mutation run proves they fail when the guard is gone (`f1ff3331`)

Possible breaking

- `forceDelete()` **now removes the row.** It threw a `TypeError` and deleted nothing before. Code that caught that error and assumed the row survived will find it gone. That was the bug.
- **A destroyed session no longer accepts a write.** A `set()` plus `save()` after `destroy()` used to re-create the session under the old id; it writes nothing now. Save before you destroy, or destroy last.

v3.13.96 (2026-08-07) - One paginate envelope, one message shape

Another step toward **3.14 stable**, and the theme is still parity: the same call means the same thing in Python, PHP, Ruby and Node. This release settles two subsystems that had drifted into four shapes. Pagination returns one envelope of seven keys with a true total. The Messenger IMAP path returns one message shape, addresses mail by a real IMAP UID, and hands back attachments you can write straight to disk. Some of these are behaviour changes, so read "Possible breaking" at the end before you upgrade.

Pagination

- `toPaginate()` takes no arguments and reads every field from the query that ran; pass one and it throws (`80257ecf`)
- The envelope is exactly seven snake_case keys: `records`, `total`, `page`, `per_page`, `total_pages`, `limit`, `offset` (`80257ecf`)
- `total` is a true `COUNT(*)` for the filter, never the page's row count (`80257ecf`)
- The AutoCrud REST list endpoint returns the same seven keys (`51b54018`)

```
// Fetch the page you want, then describe it. No arguments.
$result = $db->fetch("SELECT * FROM orders", [], 20, 40);   // 20 per page, page 3
return $response->json($result->toPaginate());
// ["records" => [...], "total" => 250, "page" => 3, "per_page" => 20,
//  "total_pages" => 13, "limit" => 20, "offset" => 40]
```

Messenger

- `send()` returns `{success, message, id}` on both paths, with `id` populated from a real `Message-ID` header (`48a7e6db`)
- `inbox()` and `search()` items are exactly `{uid, subject, from, to, date, snippet, seen}`: `to` was added, `msgno`, `flagged` and `size` were dropped, `date` is ISO-8601, and `snippet` is decoded plain text (`48a7e6db`)
- `read()` returns exactly the ten canonical keys `{uid, subject, from, to, cc, date, body_text, body_html, attachments, headers}`. The PHP-only extras `msgno`, `message_id`, `seen` and `flagged` are gone; read the Message-ID from `headers` (#70)
- IMAP authenticated to the SMTP account before and could return the wrong mailbox. Credentials are separate now: `TINA4_MAIL_IMAP_USERNAME` / `TINA4_MAIL_IMAP_PASSWORD` (falling back to the SMTP pair), plus `imapUsername` / `imapPassword` and an `imapEncryption` constructor parameter (`48a7e6db`)
- New methods: `markUnread()`, `sendTemplate()` (renders a Frond template) and `delete()` (`48a7e6db`)
- `read()` folds each attachment's decoded bytes into `attachments[i]["content"]`, so an attachment downloads straight from `read()` (`1abb1b43`)

Swagger

- `servers[0].url` defaults to `/`, not `http://localhost:7145`, and `info.description` defaults to an empty string (`8adb0256`)

- `components.schemas` carries a `required` array derived from column nullability; `info.contact` emits `name` and `url`, not only `email` (`8adb0256`)
- Framework-internal routes are excluded by one shared list, so PHP no longer publishes ten of its own service routes (`8adb0256`)
- `operationId` keeps a path's leading underscores, so `/__health` and `/health` stay distinct (`8adb0256`)

Migrations

- `code` is the canonical kind for a code migration, the same word in all four frameworks. An unknown kind throws and names the valid ones (`f0e76dda`)

Server

- An oversized request body answers `413`, not `500` (`8a4c3793`)
- `TINA4_PORT` is the port variable and wins; bare `PORT` still works but is deprecated and goes in 3.14 (`5b73b40a`)

Build and skills

- The framework SCSS compilers are gone; the `tina4` Rust CLI owns SCSS now (`be86c157`)
- tina4-css is pinned to one artefact and one URL (`21858bcb`)
- The skills point at the ADRs and make `tina4 metrics`, Carbonah and mcp.tina4.com the lean, green, grounded workflow (`a2827203`)

Proven in all four

The Messenger, Swagger and pagination behaviours are locked by machine-checked contract suites that run against real services in every framework: a live GreenMail server for the mailbox, a real 250-row SQLite table for pagination. No mocks.

Possible breaking

Read these before you upgrade. Each affects an app that relied on the old shape:

- `toPaginate()` **takes no arguments.** The old `toPaginate(page, perPage)` form throws. Fetch the page, then call `toPaginate()`.
- `total` **is the true count.** It was the page's row count in some frameworks; it is now a `COUNT(*)` for the filter. Use `count($records)` for the page size.
- **The paginate envelope is seven keys.** A reader of the dropped alias keys must move to the seven canonical keys.
- `inbox()` **and** `read()` **changed keys.** `inbox()` dropped `msgno`, `flagged` and `size` and added `to`; `read()` returns exactly ten keys, dropping `msgno`, `message_id`, `seen` and `flagged`; dates are ISO-8601. Read the Message-ID from `headers`, and use `uid` (never the removed `msgno`) to address a message.
- **Swagger defaults moved.** `servers[0].url` is `/`, `info.description` is empty. `TINA4_SWAGGER_SERVERS` and `TINA4_SWAGGER_DESCRIPTION` still override.
- **An unknown migration kind ~~throws instead of being ignore~~d.

- **The framework no longer compiles SCSS.** Use the `tina4` Rust CLI.

v3.13.95 (2026-08-06) - Preparing for the 3.14 stable release

One of several releases still to come before **3.14 stable**. The theme is parity: the same call now means the same thing in Python, PHP, Ruby and Node. Method shapes and inputs are unchanged - this is bug fixes, optimizations, and a much harder test suite behind them. See "Possible breaking" at the end for the handful of behaviour corrections that could affect an app relying on the old, wrong result.

Queues

- Queue operations act on the backend you configured, not the local file store (`04da2b54`)
- The dev dashboard queue panel lists the store it counts (`ef8bfae8`)
- The AMQP vhost is read as the path segment (`769923e6`)

Logging

- One logger format, file layout and input contract across all four (`6ba736ac`)
- Text is the default format; `production` no longer picks it for you (`544bb606`)
- The logger no longer needs `ext-mbstring`, and no longer masks the real error (`10355d5f`)
- No unguarded `mb_*` calls anywhere; string handling extracted to `Tina4\Str` (`d1ade9ab`, `9780525e`)
- An explicit argument beats the environment, which beats the default (`f6aab60a`)

Databases

- Every connect is bounded by `TINA4_DATABASE_CONNECT_TIMEOUT`, default 10s; `0` waits forever (`5ee2d5f2`)
- `count()` returns the true total, not the last page's row count (`0514d131`)
- `toPaginate` reports the page it is on (`0d638532`)
- `update()` matches the primary key in data case-insensitively (`0e3ccb72`)
- Firebird: parameterised statements no longer hold the table for the process lifetime (`f956edcb`, `05da26c0`)
- Firebird: one abandoned adapter no longer kills every other connection (`cfd4a761`)
- Firebird: a column name is folded back only when Firebird folded it (`4fccb9aa`)
- Firebird: `tableExists` matches either spelling Firebird may have stored (`2c7fc50d`)

Sessions

- `TINA4_SESSION_REDIS_PREFIX` and `_VALKEY_PREFIX` are honoured (`b532ce91`)
- `TINA4_SESSION_MONGO_COLLECTION` is honoured (`605a1fe3`)

PHP production runtimes

- `tina4 deploy docker --runtime cli|fpm|swoole` writes a working image and its companion files for each
- The Swoole integration works: request-type detection matched a PSR-7 shape before the Swoole one (`77942157`)
- The development server serves each request in its own process on Linux and macOS (`b93f5f94`, `b55a8683`)

Local by default, production by environment variable

The document store runs on a local SQLite file with no configuration and no services, and becomes a real MongoDB collection when you set one environment variable. The call sites are identical either way. The same shape holds across the framework: SQLite is the default database, the queue is file-backed until you point it at a broker, and the cache is in-memory until you give it a URL.

Bulk insert

Pass a list of rows to `insert()` and the framework builds one prepared statement and runs it inside a single transaction on a single connection, instead of a round trip per row. Same call in all four languages.

```
$db->insert("orders", [
    ["customer_id" => 1, "total" => 9.99],
    ["customer_id" => 2, "total" => 14.50],
    ["customer_id" => 3, "total" => 22.00],
]);
```

Tooling

The `tina4` CLI reached 3.8.67 over this cycle:

- `tina4 init` now wires the project up rather than printing instructions for it
- `tina4 deploy docker` gained `--runtime cli|fpm|swoole` for PHP
- The skills installer worked on macOS but installed nothing on Debian/Ubuntu. Fixed
- Skills install for Codex as well as Claude
- `tina4 metrics` gained cross-file duplication detection, Rust support, and now refuses a file it cannot parse instead of scoring it
- `tina4 serve` no longer opens a duplicate browser tab

The documentation site you are reading runs on **tina4press**, our own zero-Vue static site generator built on tina4-js. 271 pages, no framework runtime on the page.

Possible breaking

Method shapes and inputs did not change. These are behaviour corrections, so they only affect an application that depended on the previous, incorrect result:

- `count()` **in Ruby and Node** returned the number of rows the last page

produced. It now returns the true total. Code that treated it as a page size will see a different number.

- **The AMQP vhost** was read as `"/"` plus the path segment, so

`amqp://host/production` looked for `//production`. If you worked around this by naming your vhost with a leading slash, remove it.

- **Configuration precedence.** An explicit argument now beats the environment.

If you both set `TINA4_LOG_DIR` and passed a directory to `configure()`, the code now wins. Setting only one is unaffected. Ruby deployments that read `tina4.log` from the project root should read `logs/tina4.log`.

- **The Mongo session database default in Node** is now `tina4`, matching the

other three. Set it explicitly if you relied on the old default.

- **An unknown connection scheme, or an unknown** `TINA4_SESSION_BACKEND`**,**

raises instead of silently falling back to SQLite or to file sessions.

- **A missing MongoDB driver** raises rather than falling through.

v3.13.94 (2026-07-29) - A write with no filter is an error

`update()` and `delete()` with no WHERE clause used to be accepted. Two of the four frameworks then wrote every row in the table. The other two changed nothing and said they had succeeded. Neither answer is a write, and neither told you which one you got.

An unfiltered write now refuses to run and names what is missing. `truncate()` is the explicit spelling for the case where you really did mean every row.

Breaking. Read the migration note below before upgrading.

The write path (BREAKING)

A write needs a target. `update(table, data)` with no filter now takes the primary key out of `data` and uses it as the WHERE clause. With neither a filter nor the complete primary key, it raises and names the columns it wanted. `delete(table)` with no filter raises too. `truncate(table)` is the whole-table spelling, and it always was.

A failed write raises rather than returning a falsy result the caller never inspected.

Primary keys are read as a LIST, because a key can span several columns and every one of them belongs in the WHERE. Match a composite key on its first column alone and you hit every row that shares that value: the same data-loss shape the fix exists to prevent. Two of the four SQLite drivers had a second bug underneath that one. `PRAGMA table_info` returns `pk` as the 1-BASED POSITION within the key, not a boolean, so a composite key reports `pk=1, pk=2`. Code that tested `pk == 1` saw a key one column wide and truncated the rest.

Migration. Two changes to make, both mechanical:

- An `update()` that relied on the primary key travelling inside `data` still

works, and now works on composite keys. An `update()` with neither a filter nor a full primary key was already broken. Add the filter.

- Replace a deliberate delete-everything with `truncate(table)`.

Nothing else in the write path moved. A filtered write behaves as it did before.

The Frond sandbox revokes capability, it does not skip a step

`sandbox()` filtered the tag and filter lists but let a denied name fall through to the default path, so a template could still reach an escape filter it was never granted. A denied filter cannot confer safety now, and the tag gate collapsed to one check at dispatch instead of several that could disagree.

The shared render corpus grew from 72 cases to 84, covering `|safe` and `|escape`. It is one committed fixture with one answer key, byte-identical across all four frameworks, so a divergence is a failing test rather than a discovery six months later.

Messenger: one name, one signature

The dev branch and the send branch had drifted into two shapes of the same method, and capture keyed off debug mode rather than whether a transport was available. One signature behind one name now, and capture follows availability.

Base images that boot

No CI had ever built or run a base image. Two of the four had therefore never served a request at all, and the two that did boot were each wrong in their own way. `docker run` is a supported entry point, so all four are now gated in CI on Docker Hub, on `/health`, and on the version they actually serve.

The `FROM` to `COPY` recipe for adding your own database driver is documented in all four image headers, and in one case that documented line did not work until this release. An install instruction nobody runs is a claim, not a feature.

Frond, measured against the engines it replaces

The benchmark page carried seven categories and none of them timed a template. That absence flattered us, because template rendering is the one axis where Frond competes head-on with the engine it replaced. It is a headline category now, and the numbers are not kind.

Every engine renders the same page: a 20-row product list with a loop, an index, an even/odd class, an uppercase filter, two-decimal money, and a conditional footer. Output is compared byte for byte and proven identical before any timing counts, so nothing here is a strawman.

Frond loses in all four languages, on its own compiled path rather than the interpreter fallback. The competition compiles a template to code the host runtime optimises. Frond walks a tree and calls back into engine primitives per hole. What Frond buys is the zero in the dependency column, and one template language that renders the same across four runtimes.

The per-language multiples are in each framework's own note below. Publishing them costs us the comparison and buys the reader a real number.

Also in this release

MQTT test infrastructure locks its password file to `0600` and the broker's own user, so a run cannot leave a world-readable credential behind.

PHP specifics

`update()` with no filter appended no WHERE and overwrote every row. `delete()` with no filter removed every row in silence. PHP was already ahead on two of the five points, and those are locked in by tests now rather than left implicit: a failed write raises `DatabaseException`, and `writeResult(withLastId: false)` already kept `lastId` null on update and delete. `update()` also accepts an array filter as well as a string, matching `delete()` and the other three.

`getColumns()` already read composite keys correctly, so PHP needed no driver fix.

PHP has no `ext-rdkafka` path, so the socket client is the only Kafka client PHP has, and the queue backend did not work against a Kafka 4.x broker at all.

The base image never booted, and the three defects sat one per layer, each reachable only after fixing the one above it. The vendored autoloader described a different project and replaced the one composer had just generated. The demo seeded its data before `App::start()` had discovered the models. Composer ran its optimised dump against an empty directory. A fourth defect hid all of it: `.dockerignore` was itself ignored, so an earlier fix shipped nowhere.

Running `index.php` directly now points at `tina4 serve` instead of failing without explanation. The test suite no longer leaks the `php -S` fixture servers that made a piped run hang long after PHPUnit had finished, and a failed PostgreSQL connect says what libpq said instead of dereferencing a null handle.

Frond: 3.70x slower than Twig and 2.92x slower than Blade on identical output, with `FrondCompiler` engaged. Production mode buys little.

v3.13.92 (2026-07-27) - HS512 no longer rejects every token you mint

Set `TINA4_JWT_ALGORITHM=HS512` and your application locked everyone out. Not some users. Every token, on every request, including the one `Auth::getToken()` had minted a millisecond earlier.

Three reported issues sit behind this release, one shared contract, all four frameworks. PHP carried the worst of it.

HS384 and HS512 rejected every token (Fixed)

`Auth::validToken()` branched on `HS256`, branched on `RS256`, and fell through to `null` for anything else. Two of the four documented values of `TINA4_JWT_ALGORITHM` produced a total authentication outage, and the only evidence was a 401 with nothing behind it.

Verification now looks the digest up instead of branching, so the whole HMAC family verifies. This is the most serious item in the release: a variable the framework told you to set could take your

login page down.

The header names the algorithm that signed (php#187)

`Auth::sign()` hardcoded `sha256`. A token advertising HS512 in its header carried a 32-byte HMAC-SHA256 signature where the header promised 64. Any verifier that reads the header and computes what it names got different bytes and rejected the token.

The digest now comes from a lookup keyed by the configured algorithm, so the `alg` in the header is always the one that produced the signature. HS256, HS384 and HS512 all sign and verify through `hash_hmac`, with nothing new installed.

The assertion that catches this measures the signature's byte length: 32, 48, 64. Comparing one algorithm's token against another's proves nothing, because the header is part of the signing input, so two tokens differ either way.

`RS256` is unchanged and still works here. PHP has ext-openssl and Node has `node:crypto`, so RSA signing costs those two nothing. Python and Ruby would need a third-party package, and the core takes no dependencies, so `RS256` is a documented PHP and Node extra rather than part of the cross-framework contract. The other two are not getting it.

nbf is enforced, with 60 seconds of leeway (php#187)

A token stamped not-valid-until-noon was accepted at nine. `validToken()` checked `exp` and walked straight past `nbf`.

It now refuses a post-dated token until its `nbf` arrives, and accepts one up to `Auth::JWT_LEEWAY_SECONDS` early. RFC 7519 allows a small leeway, and without one a token minted on a host a second ahead is rejected for nothing at all.

A token carrying no `nbf` is unconstrained, which is what keeps this non-breaking for every token already in circulation. `getToken()` does not stamp one, matching Python and Node. Ruby used to, and no longer does, so all four now issue the same claims.

The header cannot choose the verifier

`validToken()` now rejects a token whose header names any algorithm other than the configured one, `alg: "none"` included, before it spends a single HMAC. The header used to be ignored entirely during verification.

This closed no live hole. Every framework already verified with its own configured algorithm and never read the header to pick one, so a forged header changed the signing input and broke the signature anyway. What the check buys is parity with Ruby, which always had it, an exit before the wasted work, and a test that fails the day someone refactors `validToken()` into trusting `alg` from the header. That mistake has its own CVE list. Now it has a test here.

authenticateRequest forwards its algorithm

`Auth::authenticateRequest()` accepted a `$algorithm` argument and dropped it on the floor, so a caller asking for a particular algorithm silently got the environment's. It is forwarded now, and its default changed from `'HS256'` to `null` so environment resolution applies. Passing `'HS256'` explicitly behaves exactly as before.

A misconfigured algorithm throws (Breaking)

`TINA4_JWT_ALGORITHM=HS-256` used to be swallowed. The typo went into the header, `sign()` produced an HMAC-SHA256 signature anyway, and `validToken()` then failed to recognise the algorithm and rejected every token. A one-character mistake in a deployment variable read as a silent 401 forever.

`resolveAlgorithm()` now throws an `\InvalidArgumentException` naming the variable and the values it accepts: `HS256`, `HS384`, `HS512`, `RS256`. A deployment error announces itself instead of impersonating a bad password.

A malformed or forged token still returns `null`, exactly as before. Only the configuration error escapes.

Migration: set `TINA4_JWT_ALGORITHM` to one of the four supported values, or leave it unset for the HS256 default. A value that used to be swallowed now stops the first token operation and tells you why.

The tests that hold it

Thirty-five tests, a positive and a negative case per issue, real `Auth`, real `hash_hmac`, real OpenSSL key pairs, no doubles anywhere. Each was reverted against the old code and watched to fail: forcing every HMAC algorithm back to `sha256` fails six digest assertions. Full suite on macOS with PHP 8.5.7 and PHPUnit 11.5.55: 4,311 tests, 12,704 assertions, zero failures.

v3.13.91 (2026-07-27) - The offenders list finally points at code worth fixing

`tina4 metrics` ranked `public/js/frond.js` as the worst code in the framework, at cyclomatic complexity 191. Second place went to `register_dev_tools` at 139. Neither number was real. The first belongs to a file wrapped in one anonymous function. The second belongs to a registrar that declares twenty small handlers inside itself.

Both scores were the same arithmetic mistake, and it had been in every framework since the metrics module shipped.

A function is no longer charged for the functions inside it

Complexity was measured across a function's whole span. A branch inside a nested function landed on that function and on every function wrapping it. Count the same branch twice, three times, four, once per level of nesting.

The deeper the nesting, the worse the lie:

```
def outer(a):
    def inner1(x):
        if x: return 1
        if x > 2: return 2
        return 3
    def inner2(y):
        if y: return 1
        if y > 2: return 2
        return 3
```

```
    return inner1(a) + inner2(a)
```

`outer` branches on nothing. It scored 5. It now scores 1, and each inner keeps its own 3. The branches moved to where they belong. None went missing.

Python and the Rust engine walk a real syntax tree, so they stop descending at a nested function or class. PHP, Ruby and Node scan text, where that surgery would be three risky rewrites. They apply an identity instead:

```
own(F) = raw(F) - sum over direct children C of (raw(C) - 1)
```

A raw score is 1 plus every decision in the span, so `raw - 1` is the total decision count of a whole subtree. Subtract that for each direct child and what remains is the function's own work. It needs only the line and LOC each extractor already reports.

Closures, blocks, lambdas and arrow functions stay exactly where they are. None of them appear in the function list, so nothing subtracts them, and their decisions remain with the function that contains them.

Breaking: every complexity number drops, and so does every file total. A `tina4 metrics --fail-on` gate that failed on an inflated score may now pass. Run the gate once before you trust the old threshold, and lower it if the inflation was holding your ceiling up.

Ruby complexity was double what it should have been

In the tree-sitter Ruby grammar, `if`, `unless`, `while`, `when` and `rescue` name two things: the construct, and the keyword token inside it. The engine matched on the name alone, so it counted both.

`return 1 if y` scored 3. It has one branch.

Every Ruby number the engine produced came out at roughly double. The fix ignores anonymous nodes, which is what a bare keyword token is. The engine and `metrics.rb` now agree on the same file, method for method.

LOC means one thing again

Every framework counted file LOC as code lines, skipping blanks and comments. Every framework counted function LOC as a raw line span. One word, two units, sitting side by side in the same payload.

The dashboard sized its bubbles in one unit and printed its function table in the other. A function documented with care looked bigger than a function with no comments at all.

Each language now has one definition of a code line, shared by both levels, so they cannot drift apart again. `Swagger.generate` reports 167 lines in Python and 167 in the Rust engine, which is the first time two implementations have been asked the same question and given the same answer.

Fixing it in PHP surfaced a third bug. A bare `}` is a token with no line number, so the scan for a function's last line landed on the whitespace before it. Every PHP function was one line short. That line is back.

Nothing but the dashboard reads function LOC. Violations and the maintainability index both use file LOC, which was right all along, so no threshold moves.

The dev dashboard reads the coupling it is given

The bubble chart now uses the numbers the engine resolves. Files that everything depends on drift to the centre. A ring around each bubble runs blue for stable and amber for unstable. Hovering prints the real figures behind them.

The chart draws none of this unless the coupling actually resolved. The older metrics modules never mapped an import to a file, so they reported zero dependents for everything and an instability of exactly 1.0. Drawing that would paint every ring at maximum instability, which is a confident lie, so the chart leaves those channels dark instead.

Thirty-three regression tests hold all of it in place across the four frameworks, and seven more in the engine. Each was run against the old code first and watched to fail. A test that has never failed has never proved anything.

v3.13.90 (2026-07-27) - A scaffolded model and its migration finally agree

`tina4 generate crud Todo` wrote a model that declared a `name` column and a migration that never created it. The first write died:

```
table todo has no column named name
```

This sits on the path https://tina4.com/llms.txt hands to AI assistants, so the documented way to stand up a REST API returned a 500 on its first POST.

One default, defined once (php#186, ruby#33, nodejs#38)

The default field set lived inside the model template. Other generators each carried their own copy, and the migration builder had none, so it built the table from an empty field list: `id` and `created_at`, nothing else. The default now lives in a single constant that flows into the model, the migration, the form, the view and the co-emitted test alike.

Ruby hid a second trap. An empty array is truthy in Ruby, so `fields_override || parse_fields(...)` short-circuited to the empty array and the fallback never fired. That check now tests for content, not truthiness.

Python shipped this fix in 3.13.89 and missed one generator: the form kept its own inline copy of the default. The output matches while the default happens to be `name`, and diverges the moment it changes, at which point the form renders an input for a column the table does not have. Fixed, with a source invariant that fails if any generator restates the literal again.

A failed write no longer reports success

The generated create and update routes serialised their result without checking it. `create()` and `save()` report failure by return value; they do not raise. In Node a failed insert therefore returned HTTP 201 carrying unsaved data. In Ruby it surfaced as a NoMethodError on `false` and buried the real cause. Both now check the result and return 400 with a clear message. PHP already checked, and a test pins that behaviour in place.

Every generator test in all four frameworks passed `--fields` explicitly, which is why one bug survived in four languages. Each framework now exercises the no-fields path and asserts that every field the model declares reaches the migration, ending with a real write against a real database.

v3.13.89 (2026-07-27) - A typo'd tag no longer renders what it was hiding

Two Frond bugs, both present identically in all four frameworks since v3, both compatibility gaps against Twig and Jinja2. One of them was quietly showing people content that was supposed to be gated.

An unknown tag raises instead of leaking its body (Breaking, Security)

Look at this template and decide what a visitor sees:

```
{% iff user.is_admin %}
    <a href="/admin">Admin console</a>
{% endiff %}
```

Everyone. `iff` is a typo, and Frond did not recognise it, so the tag rendered nothing and its body rendered as ordinary content. The admin link went to every visitor. Worse than a broken page: the template reads as guarded, so a reviewer scrolling past sees a check that is not there.

The same shape hid behind any misspelling. `{% ifff %}`, `{% unles %}`, `{% i %}`, a tag copied from another engine. Each one silently removed its own condition.

An unrecognised tag now raises, naming it and listing what Frond does know:

```
Frond: unknown tag "iff" -- known tags are: autoescape, block, cache, extends,
for, from, if, import, include, live, macro, raw, set, spaceless
```

Frond has no plugin system for tags, so an unrecognised name is always a typo, never an extension. Twig and Jinja2 both raise on one. Frond now does too.

Two things deliberately still render nothing. A stray terminator, an `{% endif %}` with no `{% if %}`, and an empty `{% %}`. Neither has a body, so neither can expose anything, and both have always been silent.

What to do: run your templates once. A typo'd tag now fails on the page that contains it, which is where you want to find it.

set works as a block (Fixed)

```
{% set badge %}
    <span class="badge">{{ order.status|upper }}</span>
{% endset %}

<td>{{ badge }}</td>
<td class="mobile-only">{{ badge }}</td>
```

That is core syntax in both Twig and Jinja2, and until now it did the wrong thing in all four frameworks: the body printed inline where the block stood, and the variable was never bound. `{% set g %}Hello{% endset %}[{{ g }}]` rendered `Hello[]` instead of `[Hello]`.

It captures now. The captured value is marked safe, so markup you wrote in the body renders as markup rather than as escaped angle brackets, matching both reference engines. A value interpolated into the body is still escaped on the way in, which is the right place for it: the escaping happens once, where the untrusted value enters.

Blocks nest, and a loop inside the body renders into the capture rather than to the page:

```
{% set rows %}{% for i in items %}<li>{{ i }}</li>{% endfor %}{% endset %}
<ul>{{ rows }}</ul>
```

One rule decides which form you get: an `=` anywhere in the tag means assignment. So `{% set label = "a = b" %}` stays an assignment and is never read as a block. The inline form is untouched.

The expression corpus grew to 82

Three entries cover the block form: a plain capture, a capture that must not be double-escaped, and an inline `set` whose value contains an `=`. The unknown-tag rule is locked by named tests in each framework rather than the corpus, because the corpus compares rendered output and this one raises.

v3.13.88 (2026-07-27) - json_encode gives you JSON again

3.13.87 HTML-escaped the `json_encode` filter in all four frameworks. That was wrong, and this release reverts it.

Entity-encoded JSON is not JSON. `{"a":1}` inside a `<script>` block is a SyntaxError, and a script block is most of what the filter is for. One change broke all four languages at once.

The same pass fixed a bug that had been there far longer: a value JSON cannot represent used to escape as itself, or as nothing at all.

json_encode emits JSON, not entities (Breaking, reverts 3.13.87)

`{{ product | json_encode }}` renders `{"id":1,"name":"Widget"}` again. Put it straight into a script block:

```
<script>
    const product = {{ product | json_encode }};
</script>
```

The output is still safe on the page. Frond escapes the four characters that can break out of HTML, as JSON unicode escapes rather than entities. A `</script>` inside a string arrives as `\u003c/script\u003e` and cannot close the block early. A single quote becomes `\u0027`, so the value also drops into a single-quoted attribute untouched:

```
<div data-product='{{ product | json_encode }}'>
```

Use single quotes there. JSON carries its own double quotes, so a double-quoted attribute needs `| e` on top. This is the model Jinja2 uses for `tojson`, and it is why the result is marked safe.

`| raw` after `json_encode` is now a no-op. If you added one for 3.13.87 you can delete it, and nothing breaks if you leave it.

A value JSON cannot represent becomes null (Fixed)

Reported as tina4-php#184 by justin-k-bruce, who hit it in production: two infinite sort keys in a 657-row grid blanked the whole screen.

JSON has no Infinity and no NaN. All four frameworks handled that differently and all four were wrong. Python wrote a bare `Infinity`, which no parser reads. PHP's `json_encode` returned false, which arrived as an empty string, so the payload vanished with no error anywhere. Ruby fell back to Ruby inspect output. Only Node.js already did the right thing.

Every framework now writes `null`, which is what `JSON.stringify` has always produced and the only answer the grammar allows:

```
{{ readings | json_encode }}
```

```
{"low":1.5,"high":null}
```

The rule behind it is wider than one value type. This filter never returns an empty string and never returns a token that would fail to parse. A payload always arrives, in the worst case as `null`. An empty script assignment is at least a loud error; a silently wrong one is not.

Two smaller differences went with it. Forward slashes stay unescaped, so `a/b` no longer comes back as `a\/b` from PHP. Non-ASCII text stays raw, so `cafe` with an accent is itself rather than a `\u00e9` escape from Python.

to_json and tojson are the same filter

All three names now share one serializer. They cannot drift apart again.

The `to_json` indent argument is gone. PHP's pretty printer has a fixed four-space indent and cannot honour an arbitrary width, so supporting the argument in three frameworks and not the fourth broke byte parity on the one filter whose entire job is a wire format.

The expression corpus grew to 79

The cross-framework expression fixture added seven entries: the json_encode escape shape, a non-finite value, a NaN nested in an object, the `to_json` alias, a filter pipe feeding a `~` concatenation, that same expression inside a ternary, and `number_format` with locale separators.

The last three close tina4-php#170 and tina4-php#171. Both were reported against 3.13.68 and both were already fixed by the 3.13.87 expression work; they are locked in now so they cannot come back.

What to change in your templates

Grep for `json_encode`. Three cases:

- Inside a `<script>` block: nothing to do. It works again.
- Inside a single-quoted attribute: nothing to do.
- Inside a double-quoted attribute: add `| e`.

v3.13.87 (2026-07-27) - Frond expressions agree in all four frameworks

Frond has always been described as one template language with four implementations. That was an assumption. Nobody had ever measured it.

So we measured it. Seventy-two expressions covering filters, operators, ternaries, null coalescing, concatenation, comparisons, math precedence, missing values, dotted paths, arrays, hashes, escaping and filter chains, rendered through Python, PHP, Ruby and Node.js against one identical dataset. Eleven of the seventy-two disagreed.

All eleven are fixed. Every framework now renders all seventy-two identically.

The corpus is no longer a one-off script. It ships as a test fixture in all four repositories, byte for byte identical, with a single shared answer key. If one framework drifts, its own suite turns red while the other three stay green, and the failure names the expression. Changing the contract on purpose now means changing the answer key in four repositories in the same change, which is the point.

Booleans print as true or false (Breaking)

PHP used to print `1` for true and an **empty string** for false. It now prints `true` and `false`.

The empty string was the real problem. A template printing a false boolean showed nothing at all, which reads as a missing value rather than a false one. Nobody notices a blank where a `false` belongs.

The old behaviour was Twig faithful, and defensible in isolation. It stopped being defensible once the four frameworks were compared side by side: Python printed `True`, Ruby printed two different things depending on where the value came from, and Node.js already printed `true`. Lowercase won because it is the only form usable directly in HTML and JavaScript.

Two things to check in your templates. A test against the rendered `1` now sees `true`. And a false value that used to render as nothing now renders the word `false`, so any layout that leaned on the blank output needs a conditional.

{% import "file" as alias %} now works (New)

The alias import form rendered as nothing at all. The tag parsed, the macros were never registered, and `{{ forms.button("Save") }}` produced an empty string with no error and no warning. Only `{% from "file" import name %}` worked.

Both forms now work and behave identically. A dotted callee such as `forms.button` is recognised as a macro call, and the macro registry is checked before the dotted-object path so an aliased macro resolves.

The not operator in output

PHP has always handled `{{ not user.active }}` correctly.

The note records that Python, Ruby and Node.js dropped the standalone form entirely, rendering it as an empty string, and now match PHP.

json_encode is HTML escaped (Breaking)

`{{ data | json_encode }}` used to return raw JSON. It is now escaped like any other value, which is what Python, Ruby and Node.js have always done.

Escaping is the secure default. Raw JSON dropped into an HTML attribute closes the attribute early and hands an attacker somewhere to write markup.

The `<script>` case is served by asking for it:

```
<script>
    const product = {{ product | json_encode | raw }};
</script>
```

Putting `raw` at the call site keeps the decision visible. A reader can see which JSON is escaped and which is not without opening the filter.

Grep your templates for `json_encode }}` and append `| raw` wherever the output lands inside a `<script>` block. Attribute uses need no change: escaped is what they always should have been.

The gate that keeps this true

Every one of these bugs survived because each implementation looked correct on its own. Reading the code four times would not have found them. Rendering the same expression through all four and diffing the bytes found eleven in an afternoon.

That diff is now a test. `frond_expression_corpus.txt` and `frond_expression_expected.txt` live in every framework's test directory as identical bytes, and every suite renders the corpus and compares against the shared answer key. Alongside it sit named regression tests for each bug above, each carrying the negative case that was failing before the fix.

One more thing worth naming, because it is a pattern rather than an incident. Every boolean bug was a falsy guard: `|| ""`, `a[k] || a[k.to_sym]`, `true ? '1' : ''`. In a template engine, "absent" and "false" are different things, and code that conflates them prints nothing where it should print something.

v3.13.86 (2026-07-25) - Writes return a DatabaseResult

`$db->insert()`, `$db->update()`, and `$db->delete()` now return a `DatabaseResult` instead of a bare `bool`. The object is truthy on success, so `if ($db->insert(...))` still reads the same. What is new is the metadata it carries: `->affectedRows` on every write, and `->lastId` after an insert.

This lines PHP up with the Python master and with Ruby and Node, so a write returns a result object in every Tina4 language.

Breaking: code that tested the return against a strict `true` (`=== true`, `assertSame(true, ...)`, `assertIsBool(...)`) no longer matches, because the return is now an object. Read `->affectedRows` or `->lastId`, or keep a plain truthiness check. A failed write still raises a `DatabaseException` rather than returning `false`, so wrap writes in `try/catch` and never test

for a falsy return.

The ORM is unaffected: `save()` uses its own insert path and `getLastId()`.

v3.13.85 (2026-07-24) - The dev-admin bundle ships once

PHP already shipped a single `tina4-dev-admin.min.js`; this release adds the PHP half of a four-framework gate that keeps it that way.

Python, Ruby and Node.js each carried `tina4-dev-admin.js` AND `tina4-dev-admin.min.js` as byte-for-byte identical copies, referenced by nothing: 0.92MB of dead weight in every install. Those duplicates are gone.

Nothing about the dashboard changes in any framework.

A gate, so the duplicate cannot come back

Nothing compared the two files, which is how 0.92MB per package went unnoticed. All four frameworks now test the shipped assets directly: the `.min.js` is present, the unminified duplicate is absent, exactly one `tina4-dev-admin*.js` exists (so a differently-named copy cannot slip through), and the surviving file is the real bundle rather than a stub.

The Ruby half of this was quietly broken in a way worth naming. Two specs read the unminified file behind a `skip ... unless File.exist?` guard. Deleting the file would have turned both into silent skips: a green suite with two dead assertions, and no signal that the asset had vanished. They now read the shipped bundle with no skip guard, so a missing asset fails loudly. A skip that hides a missing file is not a passing test.

v3.13.84 (2026-07-24) - Every generated Dockerfile actually starts

`tina4 deploy docker` wrote Dockerfiles that could not run. Building each one for real found that of the eight Dockerfile generators in the stack (four templates in the `tina4` CLI, plus one inside each framework's own CLI), exactly one was correct.

The Python image is the clearest case. Its `CMD` was:

```
CMD ["python", "-m", "tina4_python.cli", "serve", "--production"]
```

`tina4_python/cli.py` used to be a module, and `python -m` is valid for a module. The v3 restructure replaced it with the package `tina4_python/cli/`, which has no `__main__.py`, so `-m` cannot execute it. Containers died on startup with "'tina4_python.cli' is a package and cannot be directly executed". The code moved; the hard-coded string in a different repo did not, and nothing tested the generated file.

Every generator now names an entry point that exists, and asks for production mode:

language	CMD
Python	`tina4python serve --production`

PHP	`php vendor/bin/tina4php serve --host 0.0.0.0 --port 7145 --production`
Ruby	`bundle exec tina4ruby serve --production`
Node.js	`npx tina4nodejs serve --production`

All four were verified by scaffolding a project, running `tina4 deploy docker`, building the image, and starting a container.

`serve` no longer kills PID 1 (all four frameworks)

Before starting, the CLI reclaims the port from a stale dev server by reading `lsof -ti` and signalling what it finds. It never validated that output. Where `lsof` exists but prints a different shape, a non-numeric field coerced to 0 or 1, and signalling PID 0 sends the signal to every process in the caller's own process group.

In a container the server is PID 1, so it killed itself. The Node image logged `Killed existing process on port 7148 (PID: 1 ...)` and exited 143. The PHP image logged the same attempt and survived by luck.

Port reclaiming is now skipped entirely inside a container, where there is no stale sibling to reclaim from. Outside one, only all-digit PIDs are accepted, and PID 0, PID 1 and the current process are never signalled. The message reports only what was really killed.

A gate, so this cannot rot again

Nothing tested a generated artifact. The CLI's deploy tests covered argument parsing and nothing else, and the docs truth-check inspects prose, not template payloads.

The CLI now carries five contract tests over the Dockerfile templates that fail on exactly the mistakes above: no `python -m` on a package, no dev-only runner such as tsx in a production CMD, every CMD names a published entry point, every CMD requests production, and every template sets `TINA4_OVERRIDE_CLIENT`.

The port-reclaim rule is pinned down too, in all four frameworks. Deciding which PIDs to signal now lives in one pure function per language (`selectable_pids`, `tina4SelectablePids`, `selectablePids`), so the safety rule is testable without touching a real process: 43 tests assert that a junk token, PID 0, PID 1, the current process and the current process group are each refused, and that a genuine stale sibling is still reclaimed. One of them feeds in the exact line from the container log that started this.

Also in the CLI (tina4 v3.8.58)

The four bundled Dockerfile templates carry the corrected commands, and the Ruby template gains its build toolchain. Each framework's own `docker` generator was corrected to match, so the two paths agree. The PHP generator now detects whether a project has `bin/tina4php` or `vendor/bin/tina4php` and writes whichever exists, because composer does not link a root package's own bin into `vendor/bin`.

v3.13.83 (2026-07-24) - Swagger UI stops serving itself in production

Security. With swagger disabled, `/swagger/openapi.json` returned 404 and `/swagger` returned 200. The gate was real. The static files walked around it.

The framework ships the Swagger UI as files inside its own public directory. Static serving runs before route matching, and it resolves a directory to its index, so `/swagger` became `swagger/index.html` and never reached the gated route. A production server kept serving the whole UI on four paths:

```
/swagger                       200
/swagger/                      200
/swagger/index.html            200
/swagger/oauth2-redirect.html  200
```

The tell was the mismatch. The spec route obeyed `TINA4_SWAGGER_ENABLED`; the UI ignored it. Static serving now checks the swagger gate before it resolves an index, so all four paths return 404 when swagger is off and serve as before when it is on. Fixed in all four frameworks, each with a lock-in test that fails against the old code. (python#97)

The banner advertises only what answers (python#99)

Every boot printed both dev links, whatever the environment:

```
Swagger:   http://localhost:7145/swagger
Dashboard: http://localhost:7145/__dev
```

In production both URLs 404. That misled two readers at once. An operator scanning a production log believed a dev surface was exposed. A developer clicking the link landed on a dead page.

The banner now prints a row only when that surface answers. Each framework builds those rows through one pure function of the port and two booleans, so the contract is unit tested instead of inferred from stdout: `banner_surface_lines` in Python, `App::bannerSurfaceLines` in PHP, `Tina4.banner_surface_lines` in Ruby, `bannerSurfaceLines` in Node.

PHP printed its banner from two places, the `App` class and the `bin/tina4php` CLI. Patching one left the other unchanged, which is why the first attempt at this fix changed a file without changing behaviour. Both now call the shared helper, so the two copies cannot drift apart.

`$app->run()` under php-fpm serves the request (php#180)

The shipped `index.php` calls `$app->run()`, which binds a socket and takes the event loop. Under a web SAPI a server already sits in front of you, so every request looked for a free port, started a second server, and returned nothing. The nginx and php-fpm deployment documented in `nginx.conf.example` could not work at all.

`run()` now reads the SAPI first. Under `cli` it starts the standalone server exactly as before. Under `php-fpm`, `apache2handler`, or `php -S` it handles the current request and returns. The boot sequence still runs once. Every existing project is fixed without touching `index.php`.

The Composer package sheds 3.4MB (php#181)

`composer require tina4stack/tina4php` also downloaded the test suite, the example app, the benchmark scripts, the Docker files, and a 937K `tina4-dev-admin.js` that was a byte for byte copy of the `.min.js` next to it, with no reference anywhere in the codebase. A `.gitattributes` with `export-ignore` now keeps development files out of the published archive, and the duplicate JS is gone. The archive drops from 8.8MB to 5.4MB. Nothing a running application loads was removed.

The CLI runs no watcher in production (tina4 v3.8.57)

`tina4 serve --production` started the file watcher and the SCSS watcher anyway. Both burn CPU, both contend with the single server process, and neither has anything to do: production compiles SCSS once at boot and never re-imports code. `--production` now starts neither. The CLI stays to supervise the server process and shut its tree down cleanly on Ctrl-C.

A stale CA no longer turns a missing certificate into six failures (python#98)

The MQTT TLS tests trusted whatever CA file happened to sit in the shared temp directory. Once that file went stale, the tests did not skip. They failed, six of them, in all four frameworks, pointing at TLS code that was correct. Each suite now verifies that the CA actually validates the broker certificate before it treats the TLS environment as present, and skips when it does not.

v3.13.82 (2026-07-23) - MQTT 3.1.1: talk to any broker, no dependency

Tina4 now speaks MQTT. The new `Tina4\Mqtt` client publishes and subscribes to any MQTT 3.1.1 broker - Mosquitto, EMQX, HiveMQ, AWS IoT - and adds nothing to your dependency tree. It is built on PHP streams and `ext-openssl` alone, and it is the same client in all four frameworks.

```
use Tina4\Mqtt;

$mqtt = new Mqtt();   // reads TINA4_MQTT_URL, default mqtt://127.0.0.1:1883
$mqtt->publish("fleet/meter-42/telemetry", '{"kwh":12.5}', qos: 1);

foreach ($mqtt->consume("fleet/+/telemetry", qos: 1) as $message) {
    if ($message->isDuplicate()) {
        continue;
    }
    store($message->topic, $message->payload);
}
```

- **QoS 0 and QoS 1, retained messages, and a Last Will.** `publish`, `subscribe`, and `consume` mirror the Queue you already know. `consume` acknowledges a message only after your handler has processed it, so a handler that fails leaves the message for redelivery - at-least-once, which is what QoS 1 is for.
- **QoS 2 is refused, loudly.** Asking for exactly-once raises an error that names the limit and the fix - a QoS 1 consumer keyed on device id and timestamp - rather than silently downgrading to at-least-once and double-processing forever.

• **TLS with a per-client trust store.** An `mqtts://` connection verifies the broker against the CA you supply and no other. A self-signed certificate is rejected unless you provide its CA, and a CA loaded for one client never leaks into the next.

• **Username and password auth**, over plain or TLS connections, from the URL or from explicit arguments.

• **No mocks.** Every test runs against a real Mosquitto broker - the anonymous, authenticated, and TLS listeners - so the wire protocol is verified for real, not simulated.

This release also:

• **Counts 98 built-in features.** MQTT is the newest. 97 features are identical across all four languages; Ruby adds ERB as a native second template engine for 98.

• **Breaking:** `SqlTranslation` **is renamed to** `SQLTranslator` so the class name matches across all four frameworks. Update `use Tina4\SqlTranslation` to `use Tina4\SQLTranslator`.

• **Fails the test run on a** `setUpBeforeClass` **service skip**, closing a hole where a class-wide "service unavailable" skip passed CI green.

• **Stops and deregisters a single background task.** `$app->stopBackground($callback)` ends just that task and removes it from the registry.

• **Locks in that the test client dispatches through the real front controller**, so a test sees the same routing, middleware, and 404 path a real request does.

• **Ships the compiled Tina4 CSS assets** and honours SCSS `!default` instead of leaking it into the output.

• **Reaches** `doctor`**,** `setup`**, and** `deploy` from `tina4php` by delegating to the shared Rust CLI.

v3.13.81 (2026-07-21) - `tina4php test` fails the build when tests fail

The same hole Python had. `tina4php test` ran the suite through `passthru()` and never captured the exit code, so it always returned 0. A CI gate passed on a red suite. This release captures the runner's exit code and exits with it, so a failing suite fails the gate. A missing runner counts as a failure and exits non-zero too.

• **The exit code now propagates.** `tina4php test` exits with the runner's own code. A red suite stops the pipeline; a green suite exits 0.

• **The** `--verbose` **flag is gone.** PHPUnit 10 and up removed `--verbose`, so on the pinned PHPUnit 11 the runner exited 2. Once the exit code propagated, that would have turned a passing suite into a false failure. The command now runs with `--colors=always`, and the documented command matches.

• **Real lock-in tests pin both branches.** One covers the smoke-script path, one covers the phpunit path, each against a real temp project. No mocks. Nothing changes beyond the exit code and the flag.

This release re-aligns all four frameworks on one version. Python, Ruby, and Node skip 3.13.80, a PHP-only patch that shipped the `\Tina4\Test` base class; PHP moves from 3.13.80 to 3.13.81. Everyone is back on 3.13.81.

v3.13.80 (2026-07-19) - The xUnit base class Tina4\Test finally ships

The `\Tina4\Test` base class has been documented since 3.13.0 for class-based test suites - chapter 18 shows `class MyTest extends \Tina4\Test`. It never actually shipped. This release ships it.

- `Tina4/Test.php` **was silently kept out of the package.** An unanchored `test.php` line in `.gitignore` matched `Tina4/Test.php` case-insensitively (macOS git), so the file was never committed and never reached Packagist - even though `\Tina4\TestClient` and the parity test that extends the base class were both present. Anyone who followed chapter 18 got "Class Tina4\Test not found". The file is now committed, guarded by an explicit un-ignore, and ships.
- **3.13.79 is what surfaced it.** The switch to test-directory discovery in 3.13.79 finally ran the test that extends `\Tina4\Test`, so CI failed loudly instead of the file staying invisible.
- **PHP only.** The Python, Ruby, and Node test base classes were already committed and shipping.

v3.13.79 (2026-07-19) - The test suite runs every test file, a renamed session cookie is read back, and a stuck migration clears

The 3.13.78 session-cookie fix was PHP's, and its regression test never ran in CI. This release closes that hole, reads a renamed cookie back, and clears a stuck migration.

- **CI gap (#178): every test file runs now.** `phpunit.xml` hand-listed its test files, so 78 of 186 test classes never ran in CI - including the #174 session-cookie regression added in 3.13.78. The config now discovers `tests/` by directory, so every `tests/*Test.php` runs, and a guard test asserts the config collects them all. This is the same class of hole as a file registered by hand in 3.13.78.
- `TINA4_SESSION_NAME` **is now read back.** The write side honoured `TINA4_SESSION_NAME`, but the router read the incoming cookie under a hardcoded `tina4_session`, so a renamed cookie never resumed. Both sides now resolve the name through one method, `Session::cookieName()`; the default is byte-identical.
- **Migration (#176): a stuck row clears.** A bookkeeping row left at `passed = 0` - a prior failure, or one carried over from a v2 table - is treated as pending and re-applies cleanly on the next migrate. This was merged earlier; this release publishes it.
- **Test hygiene.** Four PHPUnit "risky" notices surfaced once directory discovery ran the dormant files, and they are cleared. They came from error and exception-handler bookkeeping in the test harness (an `ErrorTracker` static counter and `App` handlers restored at garbage-collection time); the fix is test-infra only and touches no framework runtime code, so

production behaviour is identical.

The session-cookie `Secure` fix itself shipped for PHP in 3.13.78, the proven reference for the other three this release; 3.13.79 closes the CI hole that let its test go unrun and adds the cookie-name parity.

Reported by justin-k-bruce (php#178, php#179; #176).

v3.13.77 (2026-07-16) - The native session cookie is no longer readable by JavaScript

- **Security: `PHPSESSID` now carries `HttpOnly` and `SameSite`.** The framework starts PHP's native session so `$_SESSION` persists, but it let PHP emit the cookie with the stock ini defaults: no `HttpOnly`, no `SameSite`. Any app keeping a login in `$_SESSION` had a session cookie readable by any XSS and sent on cross-site requests. Tina4's own `tina4_session` cookie was correctly attributed twenty-five lines further down the same method; that asymmetry was the bug. The cookie is now configured before it is emitted, reusing `TINA4_SESSION_SAMESITE` (default `Lax`) and the same SameSite=None implies Secure rule. Scope is unchanged: lifetime, path and domain still come from your ini.

- `toDict()` **is not deprecated.** The `@deprecated` tag said the default key casing changed from camel to snake in 3.11.22, but it sat on the method, so editors struck through every call and pointed nowhere: `toAssoc()`, `toObject()` and the response auto-serialization all delegate to `toDict()`. The tag is gone and the casing note is now plain documentation.

Both reported by justin-k-bruce. The cookie fix is pinned by a test that reads the real `Set-Cookie` off a live server.

v3.13.76 (2026-07-16) - Migrations apply again on a database created before 3.13.55

If your database was created by Tina4 v3 3.13.54 or earlier, every new migration failed and none could ever be applied. This release fixes that. PHP already built its insert this way and was never affected; the behaviour is now pinned by tests here too.

- **The bookkeeping insert now writes the columns your table actually has.** The 3.13.55 rename added `migration_name` and left the old `migration_id` column in place, calling it harmless. It was harmless on reads and anything but harmless on writes: that column is `NOT NULL`, the insert never filled it, so recording a migration raised a not-null violation, the migration rolled back, and the database was stuck. The runner now builds its insert from the table's real columns and fills any legacy one it finds. No schema change, no `ALTER`, every engine.

- **Fresh databases are untouched.** A table with no legacy column still gets exactly the six canonical columns. That is the case CI and every new project exercised, which is why this only ever bit long-lived staging and production databases.

Thanks to justin-k-bruce, who reported it against 3.13.75 with a full compatibility matrix and a working patch. Real-database regression tests now cover a pending migration on a legacy table in all four frameworks.

v3.13.75 (2026-07-14) - Static assets revalidate, so a deploy reaches users without a hard refresh

The built-in static file handler (everything under public/) now lets a browser cache an asset but forces it to revalidate on every use. A redeployed CSS or JS file reaches the browser on the next page load, with no manual hard refresh - and an unchanged file costs a cheap 304 Not Modified, not a full re-download.

- **Cache-Control and validators on every static response.** Each asset carries `Cache-Control: no-cache, must-revalidate`, an `ETag`, and a `Last-Modified`. Before, a static asset carried only its Content-Type and Content-Length.
- **Conditional requests get a 304.** The handler answers `If-None-Match` and `If-Modified-Since` with a `304 Not Modified` and no body, so a revalidation is a small round trip rather than a re-download. Real-file, real-request tests lock the behaviour in.

This lands identically across Python, PHP, Ruby, and Node.js. It closes the class of "I already reported this" where a browser kept serving a fixed-but-cached front-end asset.

v3.13.74 (2026-07-13) - The dev dashboard connection tester works again

The dev dashboard "Test connection" panel now connects, lists the tables, and shows the server version. On PHP it was broken two ways: it built the database with `new DataBase(...)`, which resolved to a class that does not exist and threw before anything ran, and it counted tables with a method the adapters do not expose. This release builds the connection with `Database::create(url, username:, password:)` (the same call the rest of the dashboard uses) and counts tables with `getTables()`.

- **A real-SQLite test drives the endpoint end to end.** It opens a live database with two tables and asserts the panel returns success, the real table count, and the version. No mocks.
- **Firebird writes are guarded against a silent loss (#132).** A regression test confirms that an explicit-transaction DELETE or UPDATE - the path the ORM takes - is visible to a separate connection, run against a real Firebird server. The fix itself shipped earlier; this locks it in so a write can never again commit an empty transaction unnoticed.
- **A PHPDoc house style is written down (#128).** CONTRIBUTING.md now states the standard: every public method carries a one-line behaviour summary plus `@param`, `@return`, and `@throws`, describes the behaviour rather than the fix, keeps types in step with the signature, and leaves no orphaned docblocks. The generated AI-context files carry the same rule.

v3.13.73 (2026-07-13) - A failed migration re-applies cleanly

This release makes a previously-failed migration run again on the next migrate, at full parity across the four frameworks.

- **A leftover `passed = 0` row no longer wedges the next run.** When a migration succeeds, the runner deletes any existing bookkeeping row for that migration name before it writes the

fresh `passed = 1` row. A migration that failed earlier - whether you recorded it with `recordMigration($name, $batch, 0)` or carried it over from a v2 table - re-applies cleanly instead of colliding on the unique `migration_name`. The `tina4_migration` table holds at most one row per migration, and the latest run wins. The delete-then-insert path is identical on every engine, so there is no dialect-specific behaviour to reason about.

• **The v2 upgrade tells you what happens next.** When the v2 to v3 upgrade finds `passed = 0` rows, it logs that those migrations re-apply on the next migrate, instead of asking you to clear them by hand.

v3.13.72 (2026-07-12) - Frond number_format, filter precedence, and a sandbox fix

This release sharpens the Frond template engine, locks in a database error contract, and brings the dev dashboard to parity across the four frameworks.

• `number_format` **reads all three arguments.** The filter now honours the full Twig signature, `number_format(decimals, decimalPoint, thousandsSep)`:

``twig {{ 1234.5 | number_format(2, ',', '.') }} {# renders 1.234,50 #} ``

The one-argument form is unchanged, so every existing template behaves as before. (#170)

• **The filter pipe binds tighter than concat.** `|` now groups before `~`:

``twig {{ amount|number_format(2) ~ ' EUR' }} {# (amount|number_format(2)) ~ ' EUR' -> 1,234.50 EUR #} ``

The rule holds at any nesting depth, including both branches of a ternary. (#171)

• **The sandbox allow-list covers every filter path (Security).** A filter applied inside a `~` concatenation or a ternary condition now respects the `{% sandbox %}` filter allow-list. A filter you did not allow-list no longer runs its code in sandbox mode. **Breaking:** in `{% sandbox %}` mode a blocked filter now skips and passes the value through unchanged, where PHP used to return an empty string. Both behaviours were secure; this only aligns PHP's output with Python, Ruby, and Node.

• **A malformed request path was already safe here.** The Node worker gained a guard this release for a path like `//` (or `///`, `/\`); PHP never crashed on it and returns a normal 404.

• **Database errors still fail loud (python#57).** `execute()` and `fetch()` raise on failure and record the message on `getError()` rather than returning `false` or an empty result. This shipped in 3.13.38; this release adds a real-PostgreSQL regression test across all four frameworks so it can never slip back to a silent failure.

• **Dev dashboard parity (`TINA4_DEBUG`).** The dev-admin dependency installer (`deps/install`), the grounding-token proxy, and the Migrate, Test, and Seed run-chips now match across all four frameworks. This is development-only; nothing changes in production.

v3.13.71 (2026-07-11) - AI skills: sharper tina4_code guidance

A skills-and-docs release; no change to the PHP package. The bundled Tina4 AI skills now state WHY `tina4_code` is deprecated: in a boot-and-verify gate (scaffold the output, boot it, run it) `tina4_code` failed where a strong model grounded with `tina4_context` passed, so the tools point to grounding plus a strong model over the self-hosted coder. The recommendation is unchanged - ground with `tina4_context` and write the code yourself; only the rationale is sharper. Running `curl -fsSL https://tina4.com/install-skills.sh | sh` now installs these updated skills by default.

v3.13.70 (2026-07-11) - Column defaults on INSERT, a Firebird charset override, and stacked Swagger metadata

ORM honours column defaults on INSERT (#165)

An INSERT now omits any column you never set on the model, so the database applies that column's own DEFAULT. Previously `save()` serialised every declared column, sending an explicit NULL for the ones you left alone; a `NOT NULL DEFAULT <x>` column then failed the insert, because a DB default applies only when the column is omitted, not when NULL is passed. The rule is now precise: a column you never touch is omitted and the database fills it in; a column you explicitly set to null is written as NULL; a column with a non-null ORM default is still written. When every insertable column is unset, the row inserts with the engine's all-defaults form. UPDATE is unchanged. Shipped across all four frameworks, verified with real-SQLite positive and negative tests.

Firebird connection charset override (#160)

The Firebird adapter no longer hardcodes the connection charset to UTF8. You can override it, in precedence order, with a `?charset=` query on the connection URL (`firebird://host:port/path?charset=NONE`), an explicit `charset:` option on `connect()`, or the `TINA4_DATABASE_CHARSET` environment variable. The default stays UTF8, so nothing changes unless you ask for it. This fixes double-encoded UTF-8 bytes read from a legacy NONE database. Shipped across all four frameworks.

Stacked Swagger metadata all survives (#59)

Stacking summary, description, and tags on one route now keeps every value in the generated OpenAPI spec, whatever the order. This was a PHP-visible drift where all but the annotation nearest the route method were dropped; it is fixed here and in Node (Python and Ruby were already correct), and all four now carry a lock-in test so the behaviour cannot drift again.

v3.13.69 (2026-07-10) - The Api client grows up, and a cross-origin redirect leak is closed

The built-in HTTP `Api` client gained four zero-dependency capabilities, shipped across all four frameworks.

- **Multipart upload.** `Api::upload()` POSTs a `multipart/form-data` body from a file on disk (`filePath`) OR from in-memory bytes (`fileBytes` plus `filename`), with optional extra

text fields, so you never need a temp file. The part Content-Type is guessed from the filename. A missing file or no source returns a clean error array, never throws, and sends nothing over the wire.

• **Streaming download.** `Api::download()` writes a GET body straight to disk in 64KB chunks, so a multi-megabyte file never lands in memory whole. It returns the status, headers, and the on-disk `path` (there is no `body` key), and writes nothing on an error status.

• **Transport seam.** An injectable `transport` callable lets application developers unit-test code that calls an `Api` without a live server. Tina4's own suite never injects a canned fake (the no-mock rule stands); its transport-seam test drives real socket I/O.

• **Opt-in cookie jar.** Pass `cookies: true` for a per-client in-memory jar: the client parses `Set-Cookie` (leading `name=value`, last write wins) and replays the accumulated `Cookie` header on later requests.

Security

• **Cross-origin redirect leak closed.** PHP's http stream wrapper auto-follows redirects and, before this release, forwarded the `Authorization` and `Cookie` headers to a cross-origin target (a different scheme, host, or port). That leaked a bearer token or session cookie to a host you did not authenticate to, verified against a real two-origin localhost server. The client now follows redirects with a manual loop and strips both headers on a cross-origin hop; same-origin redirects keep them. No new dependency (stream wrapper only; ext-curl is not required).

Also shipping (previously held)

• **Firebird `pdo_firebird` fallback.** The Firebird adapter prefers `ext-interbase` and silently falls back to `pdo_firebird` when the native extension is absent or broken (for example the macOS clumplet case). Force a driver with `TINA4_FIREBIRD_DRIVER=pdo` app-wide, or a `?driver=pdo` query param on one connection.

• **REPL database env fix.** The interactive console now connects using the project database URL and credentials (it previously read a legacy `DATABASE_URL` and skipped the bind), matching the running application.

• **AI coder rule-path skill.** The AI coding-assistant scaffolder writes each tool's rule and context files to the correct path, across all four frameworks.

v3.13.68 (2026-07-10) - Firebird counts again

`count()` and `recordExists()` return the right number on Firebird. Firebird folds an unquoted column alias to upper case, so `SELECT COUNT(*) as cnt` comes back under the key `CNT`. The ORM read a lower-case `cnt` that never existed and always returned 0, so every count looked empty and `recordExists()` always said no. Both now read the alias without caring about case. `insert()` also routes Firebird through `INSERT ... RETURNING` so a generated identity key lands back on the model, the same path Postgres already uses. Verified against a real Firebird 5.0.2 server. Reported in #132 (#133).

v3.13.67 (2026-07-10) - The MCP table browser lists your tables again

The `database_tables` dev-tool works again. It called `getDatabase()`, a method the base `Database` does not carry, so every call fataled with "Call to undefined method" and returned an error instead of your table list. The tool now calls `getTables()`, the adapter contract that actually lists tables. Drive the dev MCP server from an AI client and "list the tables" answers correctly.

The bug hid in plain sight because the test only checked that the tool was registered, never that it ran. The new test invokes the real handler against a real SQLite database and asserts a table list comes back, so a silent fatal cannot ship again. All four frameworks gained the same behavioural test; Python, Ruby, and Node already called the right method. Reported by skorteva (#164).

v3.13.66 (2026-07-10) - A self-describing CLI, and generators that ship their tests

The command line grew up. The `tina4` client no longer keeps its own copy of what each framework can do. It asks the framework, then forwards the request. A command you add to the framework shows up in the client automatically; one you remove simply stops being offered. The whole class of client-versus-framework drift is gone.

The self-describing CLI

- `commands` / `commands --json`**.** Every framework CLI now prints its own command

table from a single source. The `tina4` client reads that manifest to render an accurate `tina4 --help`, caches it by a cheap fingerprint of the resolved CLI, and refreshes with `--refresh`. Dispatch never depends on the manifest, so a discovery miss shortens the help listing and never breaks a command.

- **Pass-through dispatch.** The client keeps its own conductor commands (serve, scss,

setup, init, deploy, agent, doctor, install, update, build) and forwards everything else to the framework verbatim. It carries no per-command flag knowledge, so it can never fall out of parity.

- `queue` **is now a top-level command** in all four frameworks: `queue work`,

`queue stats`, `queue retry`, `queue clear`, wired to the real queue. Run a worker straight from the CLI instead of only scaffolding one.

- `build` **builds the deployable Docker image** (`docker build`), replacing the old

library-packaging behaviour. `build` produces the image; `deploy` ships it.

- **Ruby gains** `migrate:create`, matching the other three.

Generators now ship a test with the code

Every code-producing `generate` subcommand writes a real, passing test next to the code it scaffolds. `generate model Product` also gives you a `Product` test that talks to a real database. The tests use real collaborators (real SQLite, a real test client, a real queue), never mocks, and pass the moment they are generated.

Writing those tests exposed real bugs in the scaffolds that no string-matching test would have caught: the generated `auth` current-user endpoint rejected valid tokens in Python and PHP, and the generated migration created then immediately dropped its table in Ruby and PHP. All four are fixed and locked in by the new tests.

Databases

- **PHP silent PDO fallback.** SQLite and PostgreSQL adapters prefer the native

extension and fall back to the matching PDO driver when it is missing. The developer gets a working database either way, with identical behaviour (native types, raw-byte BLOBs, last insert id, transactions, fail-loud errors).

- **ORM `where()` takes an order.** `Model.where(...)` now accepts `order_by` /

`orderBy` (and Ruby also gains `limit`/`offset`), matching `find()`, `all()`, and the query builder.

Fixes

- The Frond browser helper now applies a 30 second request timeout by default.
- SQLite datetime adapters no longer emit a deprecation warning on Python 3.12+.
- Node route handlers accept a `void` return in TypeScript without a type error.

Breaking

- **Frond `request()` now times out after 30 seconds by default.** A request that

used to hang forever now fails after 30 seconds and calls `onError`. Pass `timeout: 0` to restore the old unbounded behaviour, or a millisecond value to set your own.

- `build` **changed target.** It now builds a Docker image rather than packaging the

framework as a library. Projects that relied on the old behaviour should call their packaging tool directly.

- `generate` **writes an extra test file per scaffold.** If you script generation and

assert on the exact set of created files, expect one more file (the co-emitted test).

v3.13.56 (2026-07-08) - Skills that own up when they drift

Every AI skill now tells the assistant how to report itself when it is wrong. A skill is documentation, and documentation drifts. When a skill still describes a method, default, or column the framework no longer has, an assistant writes confident code against an API that is gone. This release closes that loop.

Every skill, and every project context file the AI installer writes (CLAUDE.md, .cursorules, .github/copilot-instructions.md, .windsurfrules, CONVENTIONS.md, .clinerules, AGENTS.md), now carries one instruction: if Tina4 behaves differently from what the skill describes, that is a bug in the skill. Tell the developer, then report it at https://tina4.com/report-a-skill. The report lands as an issue on the documentation repository, gets fixed at the source, and ships to everyone.

The skills themselves are corrected too. The ORM soft-delete guidance now names the real `is_deleted` column (it wrongly said `deleted_at`), the tina4-js signal-persistence reference ships alongside the skill, and the per-framework skill copies are back in sync with the canonical set.

The web framework runtime does not change in this release. Update your installed skills with `curl -fsSL https://tina4.com/install-skills.sh | sh` (re-run to refresh in place).

v3.13.55 (2026-07-07) - One migration tracking schema on every engine

The `tina4_migration` bookkeeping table now has the same shape on every framework and every engine. Before this release the four frameworks each named and typed the tracking table a little differently. A project that moved between them, or a tool that read the table directly, met a different schema each time.

The canonical table is six columns: an auto-increment `id`, a `migration_name` (unique, the migration file stem), a `description`, a `batch`, an `executed_at` timestamp, and a `passed` flag. The auto-increment and the column types follow the engine: `AUTOINCREMENT` on SQLite, `SERIAL` on PostgreSQL, `AUTO_INCREMENT` on MySQL, `IDENTITY(1,1)` on SQL Server, and a generator on Firebird.

Existing installs upgrade in place, and no applied migration re-runs. The runner detects the old name column (`migration_id` in Python, `migration` in PHP, `name` in Node; Ruby already used `migration_name`), adds `migration_name`, copies the values across, and backfills the new columns. The old column stays where it is, ignored. A migration already marked applied stays applied.

No new third-party dependencies. Shipped across all four frameworks.

v3.13.54 (2026-07-07) - Migrations honour the SET TERM directive

A Firebird trigger or stored procedure now survives the migration splitter. Those bodies end their inner statements with a semicolon, the same character the runner uses to separate one statement from the next. Run under the default terminator, a trigger body split apart on its own punctuation and the migration failed.

A `SET TERM` line fixes it. Wrap the block in the universal isql idiom and the runner switches its active terminator, so the whole body travels as one statement:

```
SET TERM ^ ;
CREATE OR ALTER TRIGGER t_bi FOR t ACTIVE BEFORE INSERT AS
BEGIN
  IF (NEW.id IS NULL) THEN NEW.id = GEN_ID(GEN_T, 1);
END^
SET TERM ; ^
```

The runner consumes each `SET TERM` line instead of sending it to the engine, restores the previous terminator when the block ends, and handles a multi-character terminator such as `!!`. A

migration with no `SET TERM` splits on the semicolon exactly as before. Shipped across all four frameworks.

PHP and Ruby also repair the Firebird v2 to v3 upgrade. Firebird returns column names in upper case, so the migration tracker read a null migration name and treated every applied migration as pending, re-running the lot. The tracking-table reads now normalise to one key shape. PHP additionally records a row on an upgraded table with its original 14-character id and detects the Firebird dialect through the Database facade.

Thanks to justin-k-bruce for the contribution.

v3.13.53 (2026-07-06) - JSONField: JSON document columns

Store a JSON document in a column. A model field can now hold a whole object or array. The framework encodes it to JSON when it writes and decodes it back to a native array when it reads, so the attribute is always live data, never a raw string.

```
class Event extends \Tina4\ORM {
    public ?array $payload = null;   // JSON document
```

The column type follows the engine. PostgreSQL gets native `JSONB`, MySQL native `JSON`, SQL Server `NVARCHAR(MAX)`, Firebird a text `BLOB`, and SQLite `TEXT` (queryable through JSON1). One field declaration, the right column on every database.

A value that cannot be encoded to JSON does not slip through. `save()` fails loud: it rolls back, returns `false`, and records the cause, so a half-written row never reaches the table. A mutable default is copied per instance, so two records never share and mutate the same object.

No new third-party dependencies.

v3.13.52 (2026-07-04) - Frond live blocks, pgsql:// URL scheme, SCSS colour functions

Frond live blocks. A page can now carry a region that keeps itself current. Wrap the region in `{% live %}` and Frond paints it on the server with the first request, then refreshes it over the transport you name.

```
{% live "prices" poll 5 %}
  <ul>{% for row in rows %}<li>{{ row.name }}: {{ row.price }}</li>{% endfor %}</ul>
{% endlive %}
```

The block renders server-side on first paint, so a crawler and a client with no JavaScript both see real content. After that it refreshes on its own. `poll N` re-fetches every N seconds. `sse` streams updates over Server-Sent Events. `ws "/path"` rides a WebSocket route you already own. A data provider feeds every refresh: `@live_source` in Python, `Frond::liveSource` in PHP, `Frond.live_source` in Ruby, `Frond.liveSource` in Node. The provider re-runs with the live request, so a block that reads the signed-in user reads it again on every refresh, and an authenticated block cannot serve one user another user's data. For poll and SSE, Tina4 mounts one always-on endpoint, `GET /__frond/live/{name}`. For a WebSocket block, `push_live(name, data)` re-renders the block and broadcasts the fresh HTML to every client on that path. Nested live blocks are rejected, and a block's optional `src` attribute is same-origin

only.

One client script drives it, `frond.js`, byte-identical across Python, PHP, Ruby, and Node. No build step. No framework on the page.

pgsql:// is a Postgres URL scheme again (#58). A connection string like `pgsql://user:pass@host/db` was rejected. v3 registered only `postgresql://` and `postgres://` and dropped the older spelling, but `pgsql` is the scheme PDO, Laravel, and Doctrine all use, so real config files carried it and Tina4 refused to start. `pgsql://` now resolves to the PostgreSQL driver in all four frameworks, next to the two existing spellings. Same driver, three accepted names.

SCSS colour functions evaluate at compile time. `rgba(#3498db, 0.5)` used to pass through to the stylesheet as literal text, and the browser dropped the whole rule because `rgba()` cannot take a hex string. The built-in SCSS compiler now evaluates the colour functions: `rgba(#hex, a)` and `rgb(#hex)` expand to real channel values, `mix(c1, c2, weight)` blends two colours, and `lighten()` and `darken()` shift a colour through HSL. The output is byte-identical across all four compilers, down to the same integer rounding, so a shared stylesheet renders the same colour whichever framework served it.

No new third-party dependencies.

v3.13.51 (2026-07-03) - MCP Streamable HTTP transport, Firebird fixes

The built-in dev MCP server now speaks the current MCP Streamable HTTP transport, the one Claude Code and today's MCP clients expect. It still answers the older 2024-11-05 HTTP+SSE transport, so nothing that already worked stops working.

One endpoint carries the whole session. A client POSTs JSON-RPC to `/__dev/mcp` and reads the response inline. `initialize` mints a session and returns it in an `Mcp-Session-Id` header; the client sends that header back on every later call. A request with an unknown session gets a `404`, its cue to initialize again. A notification returns `202`. `GET` on the endpoint returns `405` with `Allow: POST, DELETE`, and `DELETE` ends the session. The server negotiates the protocol version: it echoes the version a client asks for when it can speak it, otherwise it picks the newest one it knows.

Tina4 writes the connection details to `.claude/settings.json` for you, now with `"type": "http"` and the bare `/__dev/mcp` URL. Prefer the command line? Run `claude mcp add --transport http tina4-dev http://localhost:7145/__dev/mcp`. The change lands uniformly across Python, PHP, Ruby, and Node. Python and Node keep a full persistent legacy SSE stream; PHP and Ruby serve the current transport plus a one-shot legacy handshake, with no long-lived connection required.

Why the transport slipped past us. Our MCP tests spoke our own JSON-RPC shape over the endpoints we built, never the wire a real client speaks. They stayed green while a real Claude Code client could not connect. Every framework now ships a no-mock transport test that drives the real session lifecycle, and each was verified end to end against a live server booted through the tina4 CLI.

Firebird (PHP). Three reported issues are fixed. The migration runner and the ORM now call `execute()` rather than the old `exec()` (#120). Parameterized DML no longer throws a type error when it fetches the last insert id (#121). NULL parameters bind correctly again, rewritten to a literal `NULL` because the ibase driver cannot bind a PHP null (#123). The `exec()` method stays as a deprecated alias for `execute()`.

Migration recording on upgraded schemas (PHP). The fail-loud `execute()` surfaced a quiet bug. On a database whose `tina4_migration` table had been upgraded in place from the old v2 layout, a new migration never recorded itself, so it re-ran on every boot. The old `exec()` had swallowed the constraint error. The runner now supplies the legacy `migration_id` column when it is present, so a migration records once and stays recorded.

No new third-party dependencies.

v3.13.50 (2026-07-02) - Path route params match INTEGER primary keys on SQLite (Ruby fix)

A route path parameter like `{id}`, matched against a real HTTP request, must find an INTEGER primary-key row. On Ruby it did not. Rack delivers the request path as ASCII-8BIT, so an untyped `{id}` capture reached the SQL bind as a binary string, and the sqlite3 gem bound it as a BLOB. SQLite gives a BLOB no numeric affinity, so `WHERE id = ?` never matched an INTEGER column - `GET /api/users/{id}` returned 404 for a row that plainly existed (and `GET /api/users` listed it). The router now relabels path captures as UTF-8 so they bind as TEXT, which SQLite coerces to the column's integer affinity, and the row matches. Typed `{id:int}` params were never affected - they cast to an Integer. The SQLite driver is left alone on purpose: coercing every binary string there would corrupt genuine BLOB writes, so the encoding is fixed at the source (the router).

Python, PHP, and Node were confirmed unaffected - their string path params already bind as TEXT - and each gains a real regression test: real router extraction feeding a real SQLite integer-primary-key lookup, no mocks, so the contract cannot silently drift. No new third-party dependencies.

v3.13.47 (2026-06-25) - Open-issue batch: migration comment splitting, global middleware, SCSS interpolation

Three reported issues, fixed and locked in with tests against the real thing.

Migration statement splitter (#54). A migration whose SQL carried a `;` inside a `-- ...` line comment could fragment into broken pieces in the frameworks that split before stripping comments. PHP's splitter was already a single-pass, quote- and comment-aware scanner and never fragmented, so it changed in one way only: it now strips `--` and `/* */` comments from the emitted SQL instead of carrying them through, byte-identical with Python, Ruby, and Node. Named regression tests cover a `;` inside a line comment, a `;` inside a block comment, a `;` and a `--` inside string literals, and an end-to-end migrate against a real temp SQLite database, with no mocks.

Global middleware lock-in (#55). Middleware registered globally with `Router::use(...)` / `Middleware::use(...)` already ran on every route in PHP. A lock-in test now guards the contract - `Router::use` registers into the one global registry the dispatcher reads, the before and after hooks fire, and a class registered twice is deduped - so the regression fixed in the Python master this release cannot creep in.

SCSS `#{}` interpolation (#116). The SCSS compiler did not support interpolation, so `calc(100% - #{$gap})` left the `#{...}` in the output and corrupted the CSS around it. The compiler now resolves `#{ ... }` before variable substitution and nesting: a `$variable` inside the braces resolves to its value and anything else inlines verbatim, so `calc(100% - #{$gap})` becomes `calc(100% - 20px)` and `.icon-#{$name}` becomes `.icon-home`. Shipped across all four frameworks for parity.

No new third-party dependencies. Full suite: 2,727 passing.

v3.13.46 (2026-06-24) - MySQL/MSSQL batch atomicity tests

The MySQL and MSSQL batch-insert tests checked that all three rows landed, but not that a failed batch rolls back. They now cover the same atomic-rollback and single-row contract the SQLite and PostgreSQL tests already enforce: a batch with a bad row (a NULL into a NOT NULL column) raises and leaves the table unchanged, run against the real engines with no mocks. No framework code changed - Database::executeMany already wraps the batch in one transaction and the adapters re-raise on a bad row; this locks the behaviour in as a regression guard. No new third-party dependencies.

v3.13.45 (2026-06-24) - SQLite commit resilience + real-service test hardening

A standalone write under autocommit lands through SQLite's own autocommit, so a later explicit `commit()` finds no open transaction to close. SQLite reports that as a harmless warning, and on some platform builds of libsqlite3 the warning surfaced as an exception the adapter did not catch - so a redundant commit raised instead of doing nothing. `commit()` and `rollback()` now absorb the no-transaction case on every build while still surfacing any other error loudly. The data was already committed either way; this only stops the no-op commit from throwing. Real-service test hardening rounds out the release. No new third-party dependencies.

v3.13.44 (2026-06-24) - Real-service bug-fix sweep (no mocks)

Standing up live infrastructure - PostgreSQL, MongoDB, Redis, Valkey, Memcached, RabbitMQ, Kafka - and running the suites against the real services surfaced a batch of bugs that mock-based and skipped tests had hidden. This release fixes them across the family and makes the no-mock rule absolute: a test that touches a dependency exercises the real service, never a stand-in. **Migrations on PostgreSQL/MySQL/MSSQL:** the migration runner built its bookkeeping table (`tina4_migration`) with SQLite-only DDL (`id INTEGER PRIMARY KEY AUTOINCREMENT`) for every non-Firebird engine, so `migrate()` failed on PostgreSQL with a syntax error and never applied a single file. The tracking-table id is now engine-aware - `SERIAL` on PostgreSQL, `AUTO_INCREMENT` on MySQL, `IDENTITY` on MSSQL, `AUTOINCREMENT` on SQLite. The existing migration tests only covered SQLite, which is why this shipped; a gated live-PostgreSQL migration test now guards the engine-aware path. **RabbitMQ:** the AMQP handshake now completes against a standard broker - it negotiates `Connection.TuneOk` instead of sending `channel-max=0`, which RabbitMQ rejected. **ORM:** a UUID primary-key insert returns its id through `INSERT ... RETURNING` instead of null. **Tests:** the database-factory tests assert both the `Database` facade and its underlying adapter, since `create()` returns the facade in v3. No new third-party runtime dependencies. All four suites pass against live services.

MySQL and MSSQL join the provisioned test services (#262). Both engines now run live round-trip tests by default, gated on reachability the same way the other services are. The non-skippable real-service gate fails on a MySQL or MSSQL skip under `TINA4_REQUIRE_SERVICES`, so a missing engine in CI breaks the build instead of passing quiet. CI gained a MySQL 8 container and a SQL Server 2022 container. Running the suites against these two engines for the first time surfaced adapter bugs that no prior test could reach.

The MSSQL adapter gained a `pdo_dblib` (FreeTDS) backend, used when the Microsoft `ext-sqlsrv` extension is absent. This is the same FreeTDS stack that Python (pymssql) and Ruby (tiny_tds) already lean on, so PHP now reaches SQL Server on macOS and CI without the Microsoft driver installed. `ext-sqlsrv` stays the primary path when it is present. Two more adapter fixes landed alongside it. `getLastId()` on MySQL now captures the insert id at write time, so a later read returns the real auto-increment value. The MSSQL row-count probe stripped its trailing top-level `ORDER BY` - the same fix the Python master needed, because the master carried the bug too: a `COUNT(*)` wrapper put the `ORDER BY` in a derived-table subquery, which SQL Server rejects, and the probe reported zero. Boolean binding was already correct in PHP, since the adapter coerces a raw boolean to `1`/`0` at the parameter boundary.

The no-mock rule reached further this release (#250). The messenger SMTP and IMAP tests now talk to a real GreenMail mail server, the WebSocket backplane tests run against a real Redis backplane, and the HTTP-client tests hit a real loopback HTTP server. Every in-test double in those paths is gone. The dev mailbox gained a non-ASCII round-trip regression test for parity with the family: write a message with an accented name, read it back, confirm the bytes survive. PHP escapes non-ASCII to ASCII in its message JSON, so it never carried the decode bug that bit Ruby, but the test now guards the contract.

v3.13.43 (2026-06-22) - Queue: MongoDB fail/retry/dead-letter route to the active backend

Part of a cross-framework queue-lifecycle unification: the active backend (whatever `TINA4_QUEUE_BACKEND` selects) now owns the whole job lifecycle. PHP completed MongoDB jobs correctly, but `fail()`/`retry()` and the inspection verbs (`failed()`/`deadLetters()`/`retryFailed()`) routed to the local file backend - so a MongoDB job that failed was not requeued or dead-lettered through MongoDB, and the dead-letter inspection read the wrong store. Now the full lifecycle routes to the active backend: `fail()` requeues under `maxRetries` (resetting `available_at` so it retries promptly) or dead-letters at the limit, and `failed()`/`deadLetters()`/`retryFailed()`/`retry()` read and act on the active store. RabbitMQ and Kafka delegate redelivery to the broker, so the routing is a safe no-op for them. A lock-in test injects a stub backend and asserts complete acks, fail-under-max requeues, fail-at-max dead-letters, and the inspection verbs route to the active backend. Full suite: 3,477 passing.

v3.13.42 (2026-06-22) - Swagger configurability for external and public APIs

Closes four gaps that pushed teams to hand-roll their own OpenAPI spec instead of using the built-in generator. **Configurable security schemes:** the built-in `bearerAuth` scheme honours `TINA4_SWAGGER_BEARER_FORMAT` (default `JWT`; set `opaque` for `sk_live_`-style keys), and setting `TINA4_SWAGGER_API_KEY_NAME` emits an `apiKeyAuth` scheme (`TINA4_SWAGGER_API_KEY_IN` is `header`/`query`/`cookie`). Register any scheme - including an `oauth2` flow with scopes - programmatically with `Swagger::addSecurityScheme(name, definition)` (`Swagger::resetRegistry()` clears it). **Per-route security:** a route declares its own requirement through its Swagger metadata - `security` as a scheme name plus a `scopes` array, a `{name: [scopes]}` map, a list of maps for an OR requirement, or `'public'` to mark a write route open (emits `security: []`). A secured route with no explicit declaration falls back to `TINA4_SWAGGER_DEFAULT_SCHEME` (default `bearerAuth`). Scopes stay spec-valid: only `oauth2`/`openIdConnect` schemes carry them, every other type gets `[]`, so the output validates against 3.0 and 3.1. **Path filtering:** `TINA4_SWAGGER_INCLUDE` documents only routes whose path starts with one of its comma-separated prefixes; `TINA4_SWAGGER_EXCLUDE` drops matching prefixes; framework internals (`/swagger`, `/__dev`) are always excluded. **OpenAPI 3.1 opt-in:** `TINA4_SWAGGER_OPENAPI` (default `3.0.3`) emits `3.1.0` when set to `3.1`/`3.1.0`. **Reusable component schemas:** register a shared shape with `Swagger::addSchema(name, schema)` and reference it from a route's `requestSchema` / `responseSchemas` metadata, extending the ORM-model `$ref` mechanism to arbitrary schemas. Identical behaviour and tests across all four frameworks. Zero new third-party dependencies. Full suite: 3,465 passing.

v3.13.41 (2026-06-22) - Queue reservation/visibility timeout (at-least-once delivery)

A targeted fix for silent job loss in multi-replica / rolling-deploy setups. When a consumer reserved a queue message and then died before acknowledging - a crash, an OOM kill, a Kubernetes pod eviction - the message was stranded forever: never re-delivered, never retried, never dead-lettered. The file backend deleted the job on pop (lost outright); the MongoDB backend flipped the document to `processing` without advancing `available_at` and never re-evaluated it. Now a popped job is held as a reservation with `available_at = now + visibility_timeout` (plus a `reserved_at` stamp). If the consumer does not acknowledge in time, the next dequeue reclaims the abandoned reservation: it increments `attempts` and re-enqueues the job, or dead-letters it once it has hit `maxRetries`. A dead consumer can no longer strand a job - standard at-least-once delivery, the contract SQS and RabbitMQ already provide. The window is configurable via `TINA4_QUEUE_VISIBILITY_TIMEOUT` (default 300 seconds; `<= 0` disables the reclaim) or the per-queue `visibilityTimeout` option; `acknowledge()`/`failJob()`/`retryJob()` clear the reservation. RabbitMQ and Kafka are unchanged - the broker already owns redelivery there. Regression tests lock the behaviour in across all four frameworks (file backend: reclaim after the timeout, no reclaim before it, dead-letter past maxRetries, complete/fail clear the reservation, env override, disable-at-zero; MongoDB: dequeue advances `available_at` and the reclaim requeues or dead-letters). Zero new third-party dependencies. Full suite: 3,449 passing.

v3.13.40 (2026-06-22) - MCP security hardening + Swagger/OpenAPI overhaul

A coordinated cross-framework release with two themes: MCP transport security and a full Swagger/OpenAPI sweep. **MCP security:** the built-in dev MCP server now authorises every request on the raw socket peer rather than a configured host name, closing a remote-reach surface where a debug box bound to `0.0.0.0` exposed the file and database tools to unauthenticated callers. The gate is two layers - a host-independent capability check (`TINA4_MCP`, else `TINA4_DEBUG`) and a per-request authorisation: loopback always passes, while a remote caller needs `TINA4_MCP_REMOTE=true` AND a token matching `TINA4_MCP_TOKEN` (fallback `TINA4_API_KEY`), sent as `Authorization: Bearer`, `X-MCP-Token`, or `X-Api-Key`. Every MCP surface returns 404 to a disallowed caller, the SQL tool is read-only (SELECT/WITH, no stacked statements), and the file tools are sandboxed to the project root. **Swagger:** the production on/off switch is wired for real - set `TINA4_SWAGGER_ENABLED=false` to disable `/swagger` and `/swagger/openapi.json` in any environment, or `true` to expose them in production (it falls back to `TINA4_DEBUG` when unset). Secured routes now carry a `bearerAuth` security requirement, so the documentation no longer presents protected endpoints as public. ORM models become reusable `components.schemas` referenced by `$ref` across all four frameworks. The spec is valid where it was not: wildcard and splat routes emit proper `{name}` path parameters, `operationId` values are de-duplicated, and WebSocket routes no longer leak an invalid method. **Swagger configuration:** `TINA4_SWAGGER_SERVERS` (comma-separated) drives a multi-server block, `TINA4_SWAGGER_UI_CDN` points the UI assets at a self-hosted mirror for air-gapped use, and the generator adds typed-parameter formats, enums, top-level tags, and multipart request bodies. **SqliteDocStore (new):** a pymongo-style document store with a zero-config SQLite fallback. `getCollection(name)` returns a real Mongo collection when a Mongo URI is set (`TINA4_MONGO_URI`, then `TINA4_SESSION_MONGO_URI`, then the legacy `TINA4_SESSION_MONGO_URL`), otherwise a SQLite-backed collection over a local file (`TINA4_DOC_STORE_PATH`, default `data/tina4_docstore.db`) - the call sites are identical, only the backend differs. It pushes filters down to JSON1 `json_extract` (equality, `$in`, `$nin`, `$gt`/`$gte`/`$lt`/`$lte`, `$ne`, `$exists`, `$regex`, `$or`, `$and`, dotted nested keys), supports `$set`/`$unset`/`$inc`/replace/upsert with lazy `sort`/`limit`/`skip`/projection cursors, ships a zero-dependency 12-byte ObjectId, and round-trips datetimes and ObjectIds so values stay queryable. Develop against the local store and switch to MongoDB in production by setting one env var. **Developer experience:** the blank-`TINA4_SECRET` warning now explains why it fired - the run was not detected as development - and gives both fixes (set `TINA4_SECRET`, or set `TINA4_DEBUG=true` to auto-generate one into `.env.local`); the legacy-env strict check now hints the names may come from a `.env` baked into a Docker image and points at `tina4 env --migrate`. **Session Mongo env parity:** the session and DocStore Mongo URI is canonical as `TINA4_SESSION_MONGO_URI` across all four frameworks, with `TINA4_SESSION_MONGO_URL` kept as a back-compat legacy alias (Python and PHP historically read `_URL`). **Tests:** new DB contract tests (execute raises on a bad statement, read-after-write, generator monotonicity, transaction bracketing) and a queue isolation contract (a job in one topic never leaks into another, and a queue on a fresh storage path starts empty). **Breaking:** the Swagger and MCP production gates are now enforced - if you relied on `/swagger` being reachable in production, set `TINA4_SWAGGER_ENABLED=true`, and a remote MCP caller now needs

`TINA4_MCP_REMOTE=true` and a token. Zero new third-party dependencies. Full suite: 3,434 passing.

v3.13.39 (2026-06-21) - Auto-migration, unified critical log level, fail-loud ORM, per-route WebSocket auth

A cross-framework parity sweep that hardens the data layer and tightens a few safe-by-default behaviours. **Migrations:** pending migrations can now run on startup, gated by `TINA4_AUTO_MIGRATE` and off by default so existing apps are untouched. A footgun pass adds numeric-aware ordering (so `10_` sorts after `2_`, not after `1_`), `CREATE TABLE` idempotency, a URL-safe `//` delimiter, and smart/curly-quote normalization in migration SQL before it runs. **ORM:** `save()`, `create()`, `QueryBuilder`, and the Mongo path now fail loud instead of swallowing errors - `save()` validates first and returns `false` with the reason on `getError()`, `create()` returns `false` when the write fails, and the silent fallbacks are gone. **Logging:** `critical` is now a first-class top-level severity, a new `Log::isEnabled($level)` lets callers skip building an expensive log payload, and logs default to stdout-only in production and container environments to avoid file bloat. **WebSockets:** a route can require authentication on the upgrade itself, before the socket is established. **Security:** the `/__dev/mcp` endpoint enforces its localhost guard and honors `TINA4_MCP_REMOTE`, and the built-in `Api` client strips the `Authorization` header on a cross-origin redirect and gains opt-in retry with backoff. **Env:** defaults align to the canonical `TINA4_` manifest with a uniform `.env.example` - CORS credentials are opt-in and AI hosts default to localhost. **Tooling:** the `tina4 metrics` coverage detection now counts full-package-path imports and short (3-character) class names, the complexity counter no longer over-counts string literals, and Kafka TLS/SASL config reaches the producer and consumer. Breaking: `critical` is now a top-level severity and the previous toggle is removed - update any logging configuration that relied on it. Full suite: 3,355 tests, zero failures.

v3.13.38 (2026-06-19) - Coordinated security & robustness release

A large bundled release closing a cross-framework hardening sweep. **WebSockets:** the Redis backplane is now wired for real - local-first delivery, then a published envelope on the shared `tina4:ws` channel, relayed with an origin guard (no own-echo, no cluster loop) - and, critically, the backplane class is now PSR-4 autoloadable (it had never loaded). Plus an origin allow-list (`TINA4_WS_ALLOWED_ORIGINS`), an idle reaper (`TINA4_WS_IDLE_TIMEOUT`), resilient broadcast, OP_CONTINUATION frame reassembly, and SSE hardening (generator error mid-stream). **Sessions:** the `DatabaseSessionHandler` SQL injection is fixed - the client-controlled session id is no longer interpolated into SQL; every query is parameterized. Plus a log-loud-and-degrade backend-failure policy (`TINA4_SESSION_STRICT` to re-raise). **GraphQL/WSDL:** a SOAP `<!DOCTYPE>` is rejected before parsing (SOAP 1.1 forbids DTDs - closes the XML entity-expansion / external-entity surface), a recursion-depth guard (`TINA4_GRAPHQL_MAX_DEPTH`, default 50) catches deep queries **and** circular fragments, and resolver/SOAP faults are masked in production with the real cause logged (full detail only under `TINA4_DEBUG`). **Tooling:** a new `tina4 metrics` command reports the top-N code-health offenders with `--top/--json/--fail-on/--path`, and the coverage test-detection is now precise (a real `use`/FQCN import or defined-class reference, not a name-substring scan). Zero new third-party dependencies. Full suite: 3,219 passing.

v3.13.37 (2026-06-18) - Dev-admin editor: Ruby + more syntax highlighting

The dev-admin code editor now highlights `.rb`/`.rs`/`.go`/`.java`/`.scss` (the CodeMirror bundle was missing those grammars). Also aligned the file-read language map to the Python master (scss->css, add rust/go/java, txt/csv/log->text, xml->html, shell variants, gemspec/rake->ruby, svg) and added no-extension `Dockerfile` detection. Dev-mode tooling only. Full suite: 3,078 passing.

v3.13.36 (2026-06-18) - Instant WebSocket dev-reload (parity)

Dev-reload is now a WebSocket push, matching Python. `tina4 serve` POSTs `/__dev/api/reload`; `DevAdmin` invalidates OPcache for the changed file and re-discovers routes in-process (`RouteDiscovery::rescan`, no respawn, same PID), then broadcasts `{type, file, mtime}` over `/__dev_reload`. The injected client is WebSocket-primary - instant reload, CSS hot-swap, and it only falls back to the `/__dev/api/mtime` poll when the socket drops. `Router` registers `(method, path)` with replace-in-place semantics so the re-loaded handler wins. Debug-mode only - production is untouched. Full suite: 3,078 passing.

v3.13.35 (2026-06-17) - Live MCP endpoint for AI agents

The built-in MCP server is now actually reachable. Its 48 dev tools (live DB queries, sandboxed file I/O, route list, project overview, docs search) were fully built but never mounted. `DevAdmin::register()` now mounts `/__dev/mcp` (JSON-RPC) + `/__dev/mcp/sse` in debug mode, so an AI agent (Claude Desktop/Code) gets live access scoped to the running project. The SSE handshake was made SAPI-safe for the built-in server. 15 new tests; full suite 3,064 passing.

v3.13.34 (2026-06-17) - Store images + dual-port test

Fixed blank product images in the example store: the storefront templates read `product.imageUrl` (camelCase) but `toDict()` emits snake_case `image_url`. Aligned the templates to `image_url` (matching the API and the Python store). Corrected stale env-var names in `example/.env.example` (notably `SECRET` -> `TINA4_SECRET`, which PHP rejects at boot). Added `DualPortReloadTest` locking in the AI dual-port dev mode (main port hot-reloads; port+1000 is the stable AI port). Full suite: 3,049 passing.

v3.13.33 (2026-06-17) - Queues: priority pop + automatic dead-lettering (Warning: behavioural change)

Behavioural change. `$job->fail()` now re-enqueues (incrementing `attempts`) until `attempts >= maxRetries`, then dead-letters - a `consume` loop retries `maxRetries` times automatically. `pop`/`consume` are now priority-ordered (was FIFO); new additive `retryBackoff` config. Bug fix: `Job::$topic` is now **public** (was private -> fatal when read). Only the file backend changed. Queue chapter rewritten to match. Full suite: 3,046 passing.

v3.13.32 (2026-06-17) - Caching: per-query bypass + X-Cache headers + string-middleware (chapter rewritten)

Added a per-query bypass - `$db->fetch(..., noCache: true)` (also `fetchOne`/`fetchAll`) skips lookup + store. `ResponseCache` now sets `X-Cache: HIT|MISS` + `X-Cache-TTL`, and the `"ResponseCache:300"` string-middleware form now works (parity with Python/Ruby) - this also fixed a dispatch bug where the response cache's store step never ran on the route path. The KV helpers live in `\Tina4\Middleware\`. The caching chapter was rewritten to match code (real `cacheStats()` shapes, all seven backends + file fallback, the three cache layers, accurate env/defaults), dropping earlier aspirational claims. Full suite: 3,035 passing.

v3.13.31 (2026-06-17) - Version alignment (no functional change in PHP)

Cross-framework version alignment with the Ruby request/response parity release. PHP's request body, query, headers, cookies, file uploads (raw-bytes `content`), and response surface were already in parity - no behavioural change here. Full suite: 3,024 passing.

v3.13.30 (2026-06-16) - Typed route params coerce + /__dev auth-bypass fixed (Warning: behavioural change)

Behavioural change. Typed path params now arrive coerced: `{id:int}` -> `int`, `{price:float}` -> `float` (other types and untyped params stay strings; matching unchanged). `compilePath()` computed the param-type map but `addRoute()` dropped it, so the existing cast in `matchInTable()` was dead code and `{id:int}` arrived as the string `"42"` - now wired through, bringing PHP in line with Python/Ruby/Node. Separately, a bug fix: the dev-admin auth bypass tested `$request->url` (always the full `scheme://host/path`) so it never matched - a write to `/__dev/...` (and the gallery prefixes) returned 401 instead of bypassing; it now tests `$request->path`. Full suite: 3,024 passing.

v3.13.29 (2026-06-16) - Live API search finds magic methods + ranks qualified queries

Parity with the Python master fix for the `api_*` live-reflection tools (what AI assistants query for real signatures):

- **Magic methods are now indexed.** `Frond::addFilter` / `addGlobal` / `addTest` dispatch through `__call`/`__callStatic`, so the token parser never saw them. `Docs` now reads `@method` docblock tags (and `Frond` declares them, which also helps IDE autocomplete), so `api_method("Frond", "addTest")` returns `addTest(string $name, callable $fn)`.
- **Class-qualified ranking.** `api_search("Frond.addTest")` now ranks `Frond::addTest` first - the owning class, fqn segments, and an exact `Class.method` match are scored.
- **Natural-name lookups.** `api_class`/`api_method` resolve a bare class name (`Database`) and leading-backslash variants, not just the exact fqn.

The bundled AI skills now tell assistants to query `api_*` before guessing. Full suite: 3,014 passing.

v3.13.26 (2026-06-16) - pooling fixes: standalone writes auto-commit + independent pooled PostgreSQL connections

Behavioural default change. A standalone write made **outside** an explicit transaction now auto-commits on its own connection before returning - the default `autoCommit` is flipped to *on* across all adapters. Previously autocommit was off by default, which broke connection pooling: a standalone write stayed uncommitted on one pooled connection while the next read round-robined to another and saw nothing. Explicit transactions (`startTransaction`/`commit`/`rollback`) stay atomic - MySQL/MSSQL suspend driver autocommit for the duration of the transaction, and Firebird's commit branch is gated on `transaction === null`. Set `TINA4_AUTOCOMMIT=false` for strict manual-commit mode (per-connection override: `Database::create($url, autoCommit: false)`).

PostgreSQL pooling fix. `pg_connect()` was reusing a single libpq connection for every adapter that shared a DSN, so a `pool` of N adapters all shared **one** connection - and closing one broke the rest. Pooled adapters now each open an independent connection

(`PGSQL_CONNECT_FORCE_NEW`), matching Python, Node, and Ruby.

Verified live on PostgreSQL: standalone write visible from a separate connection, explicit rollback discards, explicit commit persists, and pooled standalone writes visible across every round-robin connection. Full suite: 3,011 passing.

v3.13.24 (2026-06-15) - unified cache backends across response, KV, and persistent DB cache

The response/KV cache now supports **seven backends**, selected by `TINA4_CACHE_BACKEND`: `memory` (default), `file`, `redis`, `valkey`, `memcached`, `mongodb`, and `database`. `TINA4_CACHE_URL` carries the connection string for `redis`/`valkey`/`memcached`/`mongodb`, or a SQL URL for the `database` backend (which falls back to `TINA4_DATABASE_URL`). Credentials can be embedded in the URL (`redis://user:pass@host`, `redis://:pass@host`, `mongodb://user:pass@host`) or supplied via `TINA4_CACHE_USERNAME` / `TINA4_CACHE_PASSWORD` (mirroring `TINA4_DATABASE_USERNAME`/`_PASSWORD`); memcached is unauthenticated. The usual `TINA4_CACHE_TTL` (60), `TINA4_CACHE_MAX_ENTRIES` (1000), and `TINA4_CACHE_DIR` (`data/cache`) still apply.

Graceful fallback: if a configured backend's driver is missing or the service/credentials are unreachable or wrong, the cache logs a warning and falls back to the **file** backend - a real persistent cache, never a silent no-op.

The **persistent DB query cache** (`TINA4_DB_CACHE=true`) now routes through the same backend set via `TINA4_DB_CACHE_BACKEND` + `TINA4_DB_CACHE_URL`, so multiple instances share one cache with global write-invalidation. `cacheStats()` now reports a `backend` field alongside `mode`.

Full suite: 3,010 tests passing.

v3.13.23 (2026-06-15) - request-scoped DB query cache, on by default

A new **request-scoped query cache** protects your database from rapid repeat reads. Within a single request, identical `SELECT`s and ORM reads are deduped automatically - the DB is hit once and subsequent identical reads are served from memory. The cache is **cleared at the start of every request** (so it never serves stale rows across requests) and **flushed on any write** (insert/update/delete/execute). For non-request contexts (scripts, workers) a short safety TTL applies.

It is **on by default** via `TINA4_AUTO_CACHING=true` (off-switch `TINA4_AUTO_CACHING=false`); the in-request TTL is `TINA4_AUTO_CACHING_TTL` (default 5 seconds). The existing `TINA4_DB_CACHE` (default `false`) remains the separate *persistent* cross-request cache (TTL `TINA4_DB_CACHE_TTL`, default 30s) and is not cleared per request. `cacheStats()` now reports a `mode` field: `"request"` (default), `"persistent"`, or `"off"`.

Also fixed: the `\Tina4\Middleware\cache_get/cache_set/cache_delete/cache_clear/cache_stats` helpers now autoload on a plain `require` - previously they fataled with "undefined function" until the `ResponseCache` class had been touched.

Full suite: 2,992 tests passing.

v3.13.21 (2026-06-15) - docs: `render()` corrections + version re-sync

Documentation consistency pass - no behavior change. References to a `$response->template()` *method* (which never existed) are corrected to `$response->render()` - the real method; `template` is only the route-level binding, not a response method. Fixed across the AI guide, `llms.txt`, and the gallery page. Version re-synced to 3.13.21 with the other frameworks (this release also carries a Python-side JWT-secret security hardening).

Full suite: 2,433 tests passing.

v3.13.19 (2026-06-15) - return domain objects, construct from JSON, and one database binder

Three ergonomic improvements surfaced by the live side-by-side review of the book's own examples across all four frameworks.

`$response(...)` serializes domain objects

Return an ORM model, an array of models, or a query result straight from a route - Tina4 serializes it to JSON. No more hand-rolled `toDict()` / `toJson()`:

```
Router::get('/api/users', function ($request, $response) {
    return $response((new User())->all());     // array of models -> JSON array
});
```

A single model becomes a JSON object; an array of models or a `DatabaseResult` becomes a JSON array. Plain arrays and strings behave exactly as before - purely additive. (`Response::json()` still pretty-prints.)

Construct a model from JSON, or data-first

```
new User('{"name": "Alice"}');     // JSON object string -> one record
new User(['name' => 'Alice']);     // array data-first (NEW - no need for the data: arg)
new User(data: ['name' => 'Alice']); // still works
new User($db, ['name' => 'Alice']);  // still works ($db first)
```

The first constructor argument is now type-detected (a `DatabaseAdapter`, an array, or a JSON string). Passing a **list** to a single-record constructor throws `InvalidArgumentException`. To build many records, map over the list.

Warning: Breaking - one database binder: `bindDatabase`

The ORM-to-database binder is now `bindDatabase` (was `ORM::setGlobalDb`). The default is unchanged - models still auto-bind to `TINA4_DATABASE_URL` (via `Database::fromEnv()`), so apps relying on the `.env` default need **no change**.

```
// Most apps: nothing to do - the .env default is auto-bound.

\Tina4\ORM::bindDatabase(Database::create('sqlite:///app.db'));   // override the default
```

```
// Register a NAMED connection and point a model at it:
\Tina4\ORM::bindDatabase(
    Database::create('postgres://.../analytics', username: 'u', password: 'p'),
    name: 'analytics'
);

class Visit extends \Tina4\ORM {
    public \Tina4\Database\DatabaseAdapter|string|null $_db = 'analytics';  // uses the analytic
}
```

`bindDatabase($db, name: '...')` registers a named connection; a model selects it with `$_db = '...'`. A missing named connection throws a clear error.

Migration: rename `\Tina4\ORM::setGlobalDb(...)` -> `\Tina4\ORM::bindDatabase(...)`. That is the only change.

Full suite: 2,433 tests passing. Shipped with parity across all four frameworks.

v3.13.18 (2026-06-15) - ORM relationship + QueryBuilder fixes + boolean param binding

Found by the live side-by-side validation against PostgreSQL.

- **QueryBuilder `first()` / `count()` returned null/0** even with matching rows - `get()` returns a `DatabaseResult` but `first()`/`count()` read `$result['data']` (raw-array shape). All three now consume the result via a shared `extractRecords()`: `first()` returns the row, `count()` the integer, and `groupBy().get()` returns every group.
- `belongsTo` **returned null for a snake_case FK column** under autoMap - the lookup read `$this->{column}` but autoMap stores the value under the camelCase property. It now reverse-maps column -> property, so the documented `$foreignKeys=['author_id'=>'Author']` form resolves the parent (lazy `$post->author`, explicit, and eager `include:['author']`).
- `fetch()` **corrupted SQL that already ended in** `LIMIT` - it appended `LIMIT 100 OFFSET 0` unconditionally (`... LIMIT 1 LIMIT 100`), PG errored, the adapter swallowed it -> empty result. It now skips the append when a trailing `LIMIT` is present.
- **Bound PHP booleans are normalised** to the literal the column accepts - `'t'`/`'f'` on PostgreSQL's native `BOOLEAN`, `1`/`0` on the integer/BIT-backed engines (SQLite, MySQL, MSSQL, Firebird). Previously `fetch('... WHERE active = ?', [false])` bound `''` and PG rejected it.
- Doc: `DatabaseUrl` docstring corrected to `postgres://` / `postgresql://` (not `pgsql://`).

Python/Ruby/Node were already correct on the boolean binding (verified live). Full suite: 2,416 passing.

v3.13.17 (2026-06-15) - PostgreSQL: native-type reads + `execute()` reports real failure

Two fixes found by the live side-by-side validation against PostgreSQL.

Reads return native PHP types (not strings)

`ext-pgsql` returns every column as a string - `id` as `"1"`, a boolean as `"t"`, floats as `"12.50"`. A Tina4 app written on SQLite (native-ish types) silently changed behaviour when moved to PostgreSQL, and diverged from Python/Node which return native types. `PostgresAdapter` now coerces each column from its PG type: `int2`/`int4`/`int8` -> `int`, `bool` -> `bool`, `float4`/`float8`/`numeric` -> `float` (nulls preserved; `bytea` unchanged). So `$result[0]` is now `{"id":1,"active":true,...}` instead of all-strings. (`timestamp`/`date`/`json`/`uuid` stay strings - a minor diff vs Python's datetime objects.)

`execute()` propagates failure

`Database::execute()` returned `true` unconditionally for a plain write/DDL, discarding the adapter's boolean result - so a failed INSERT/UPDATE/DELETE/DDL reported success. (Python/Ruby/Node were already correct: their adapters *raise* and the facade catches; PHP adapters *return false*, which the facade ignored.) It now returns `false` and populates `getError()` from the adapter when the statement fails. A write affecting 0 rows is still a success.

Full suite: 2,405 passing. Python/Node already had both behaviours (Ruby ships the native-reads half), so this is primarily a PHP release.

v3.13.16 (2026-06-15) - `createTable()` works on PostgreSQL + `DatabaseResult` index access

Found by the live documentation-verification pass - running the book's own samples against a real PostgreSQL database. The documented code-first schema path, `ORM::createTable()`, was silently broken on PostgreSQL: it ignored the model entirely, emitted a hardcoded SQLite `INTEGER PRIMARY KEY AUTOINCREMENT`, PG rejected it, and it returned `true` while creating **no table**.

`createTable()` is now engine-aware

It now derives the DDL from the model's typed properties:

- **datetime ->** `TIMESTAMP` on PostgreSQL/Firebird (no `DATETIME` there); `DATETIME` on SQLite/MySQL/MSSQL.
- **bool -> native** `BOOLEAN` (PostgreSQL/MySQL), `BIT` (MSSQL), `INTEGER` (SQLite/Firebird); boolean `DEFAULT`s engine-aware (`TRUE`/`FALSE` vs `1`/`0`).
- Auto-increment translated per engine (`SERIAL` on PostgreSQL) via `SqlTranslation`.
- **A failed** `CREATE` **now returns** `false` (with a post-create `tableExists` re-check + `Log::error`) instead of reporting success.

`DatabaseResult` already implements `ArrayAccess`, so `$result[0]` works (no change needed there).

Verified against PostgreSQL 16: a model with `id` (auto-increment) + string + bool + datetime creates, inserts, and round-trips natively (`SERIAL`, `boolean`, `timestamp`; `WHERE active = TRUE` matches). New `CreateTablePostgresTest` (PG-gated). Full suite: 2,400 passing. Shipped with parity across all four frameworks.

v3.13.14 (2026-06-13) - Logs reach stdout in containers + per-request logging + schema-qualified tables (#48)

Cross-framework release (all four). Deployed Docker containers were getting no application logs. In production PHP set `Log::$stdout = false` (logs went only to `logs/tina4.log` inside the container), never read `TINA4_LOG_LEVEL`, and didn't flush stdout. `docker logs` reads PID 1 stdout - so it was empty. A follow-on report - the dev server going silent after startup - surfaced a second gap: requests were never logged.

PHP also: #119 - legacy-env guard crash under the built-in server

`App::checkLegacyEnvVars()` wrote its migration message with `fwrite(STDERR, ...)`. `STDERR` is only auto-defined for the `cli` SAPI - under `cli-server` (the built-in dev server) a bare `STDERR` in `namespace Tina4` resolved to the undefined `Tina4\STDERR`, so a user with a stray legacy var (e.g. `SMTP_HOST`) in `.env` got `Uncaught Error: Undefined constant "Tina4\STDERR"` instead of the actionable "rename these vars" message. Now writes to the `php://stderr` stream (available in every SAPI). The same latent pattern in `MCP.php` was fixed alongside. New `cli-server` subprocess regression test reproduces it.

Per-request logging - on by default in dev

Every request now logs one line through `Tina4\Log` (-> stdout), on by default in dev and opt-in for production via `TINA4_LOG_REQUESTS`:

```
2026-06-12T10:15:03.221Z [INFO   ] GET /api/users -> 200 (12.3ms)
```

`Router::dispatch()` emits it at the end of every request. Format is identical across all four frameworks: `METHOD /path -> STATUS (Nms)`. Default: on under `TINA4_DEBUG`, off in production; `TINA4_LOG_REQUESTS=true`/`false` overrides. The `RequestLogger` middleware's line now includes the status code for parity.

What changed (stdout)

- **stdout is ON by default** (was: only when `TINA4_DEBUG=true`). The default-case in `Log::configure()` now sets `$stdout = true`. `TINA4_LOG_OUTPUT=file` still opts out.
- `Log::configure()` **now reads** `TINA4_LOG_LEVEL` from the environment (it previously ignored it). Default level is **INFO** (was effectively DEBUG).
- **stdout is flushed** - `fflush()` after each `fwrite()` so logs appear immediately under the long-running built-in server instead of sitting in the stream buffer.
- **Production stdout is clean JSON** - `writeStdout()` no longer prepends ANSI colour when not in human-readable mode, so aggregators can parse the line.

```
// In a container (TINA4_DEBUG unset), default config:
\Tina4\Log::info("worker started");
// pre-v3.13.14: only in logs/tina4.log inside the container -> docker logs empty
// v3.13.14:    {"timestamp":"...","level":"INFO","message":"worker started"} on stdout
```

Why it spanned all four

The bug was the same architectural decision in every framework - production logged to a file (or suppressed stdout) when a container's stdout *is* the log sink:

Framework	Pre-v3.13.14 cause	Fix
Python	`not _is_production` gate suppressed stdout; default ERROR	stdout always on (flushed); default INFO
PHP	`$stdout = $development` (file-only in prod); no `TINA4_LOG_LEVEL` read	stdout default on + `fflush`; reads `TINA4_LOG_LEVEL`; default INFO
Ruby	stdout written but never flushed (block-buffered on non-TTY); default ALL	`$stdout.sync = true`; default INFO; accepts plain + bracket names
Node	`!isProduction()` gate suppressed console; default DEBUG	console always on; production emits JSON; default INFO

The Rust `tina4` CLI was already correct (inherits child stdio).

Schema-qualified tables (#48) + a PostgreSQL `fetch()` regression

Issue #48 - *"Database Table Does Not Exist"* on PostgreSQL. A model whose table lives in a non-default schema (`gift_cards.gift_card`, MSSQL `dbo.widget`, MySQL `otherdb.table`, SQLite ATTACH `extra.widget`) was invisible to the framework's introspection. `tableExists`, `getTables`, and `getColumns` hardcoded the default namespace (`public`) and matched the whole dotted string as one flat name - so plain reads worked, but `createTable`, migrations, and auto-CRUD were blind to the table and reported it missing.

All introspection is now schema-aware on every affected engine:

- **PostgreSQL** - `tableExists` uses `to_regclass()` (honours schema + `search_path`); `getColumns` filters by `table_schema`; `getTables` lists every non-system schema and returns non-`public` tables schema-qualified.
- **MySQL** - schema = database; a qualified name checks that catalog, a bare name defaults to `DATABASE()`.
- **MSSQL** - honours `dbo.table`; a bare name matches in any schema.
- **SQLite** - honours an ATTACH alias (`extra.widget`) for both `tableExists` and `getColumns`.
- **Firebird** - N/A (no schemas).

Verified against a live PostgreSQL 16 container: `tableExists('gift_cards.gift_card') -> true`, `getTables -> ['gift_cards.gift_card', 'gift_cards.transaction']`, `getColumns -> 12 columns` - identical results across all four frameworks.

> ***A v3.13.12 regression surfaced while cross-checking #48.*** `PostgresAdapter` *referenced* `stripTrailingSemicolons()` *(added in v3.13.12) and the new* `splitSchema()` *but never mixed in* `SqlNormalizerTrait` *- so* ***every PostgreSQL*** `fetch()` / `fetchOne()` / `getColumns()` ***fatalled*** *with "Call to undefined method". It*

shipped silently because PHP's PostgreSQL test suite skips without a live server. Fixed with a one-line trait mix-in and pinned by server-free reflection guards that assert all five SQL adapters expose the normalizer helpers - so this can never regress unnoticed again.

Tests

- PHP: 2,394 passed (+63 new - stdout/level/file gating; request-log format + gate; #119 cli-server repro + the previously-unregistered LegacyEnvGuard suite now gated in CI; #48 schema-qualified introspection + PG `SqlNormalizerTrait` regression guards)
- Family: Python 2,829 | PHP 2,394 | Ruby 2,999 | Node 3,628 - **11,850 total, zero regressions.**

---

v3.13.13 (2026-06-11) - PHP only: large-response truncation fix

PHP-only release. Python, Ruby, and Node stay at v3.13.12 - the bug is specific to PHP's built-in socket server, which is the only one of the four frameworks that hand-rolls a non-blocking socket write loop. Python (asyncio `StreamWriter.drain()`), Ruby (WEBrick), and Node (`node:http`) all delegate body writes to a server that handles a full send buffer correctly, so none of them can hit this.

Responses larger than the OS send buffer are no longer truncated

`Tina4\Server` (the standalone HTTP server behind `tina4 serve`, including when run as an nginx upstream) wrote responses with a non-blocking `fwrite()` loop. On a non-blocking socket, `fwrite()` returns `0` when the OS send buffer is full - this is **EAGAIN ("try again")**, *not* a closed socket. The pre-v3.13.13 loop treated `0` as fatal and `break`ed mid-body:

```
while ($written < $total) {
    $n = @fwrite($client, substr($httpResponse, $written));
    if ($n === false || $n === 0) {
        break;   // BUG: 0 is "buffer full", not "socket closed"
    }
    $written += $n;
    // ...only waited for drain AFTER a successful write
}
```

Symptom: a ~4 MB attachment download returned `200` with the correct `Content-Length` but only part of the body (the cutoff varied run-to-run - we saw 2.31 MB and 1.30 MB), nginx logged `upstream prematurely closed connection while reading upstream`, and the browser showed a failed download. Anything larger than the send buffer (~200 KB-1 MB depending on platform) was affected - the dev-admin JS bundle had hit a related case.

The fix

Body writes now go through `Server::writeFully()`, which:

- On `fwrite() === 0`, **waits for the socket to become writable** (`stream_select` on the write set, 5 s no-progress timeout) and retries, instead of bailing.
- Only gives up on a real error (`false`) or a client that has genuinely stopped reading for 5 s.

• Writes in 512 KB chunks so it doesn't recopy the entire remaining tail on every iteration (O(n) instead of O(n^2) for large bodies).

```
$n = @fwrite($client, substr($data, $written, 524288)); // 512KB
if ($n === false) break;                 // real error
if ($n === 0) {                          // buffer full - wait, don't quit
    $sw = [$client]; $sr = []; $se = [];
    if (@stream_select($sr, $sw, $se, 5) === 0) break; // 5s no progress -> gone
    continue;
}
$written += $n;
```

The error-response path (`sendHttpError`) routes through the same helper.

Why PHP only - cross-framework verification

Framework	Built-in server	Body write	Truncation risk
PHP	raw `stream_socket_server`	non-blocking `fwrite` loop	[ ] **was buggy** -> fixed
Python	asyncio `start_server`	`write()` + `await drain()`	[x] safe
Ruby	WEBrick	blocking `IO#write`	[x] safe
Node	native `node:http`	atomic `res.end(buffer)`	[x] safe

Tests

• New `tests/ServerLargeResponseTest.php`: pushes 4 MB through a deliberately stalled reader (via `pcntl_fork`) and asserts every byte arrives. Verified to **fail on the old loop** (truncated at the 8 KB socketpair buffer) and **pass on the fix** (full 4 MB).

• PHP: 2,331 passing.

> *Housekeeping caught alongside this fix: the CI test invocation (`vendor/bin/phpunit`, xml-file-list mode) ran fewer tests than `composer test` (directory mode), so newly-added test files could silently skip CI. The v3.13.12 SQL-normalizer tests and this release's socket test are now registered in `phpunit.xml`; fully reconciling the two invocations is tracked separately.*

---

v3.13.12 (2026-06-11) - SQL safety + implicit ORM binding + `fetchAll` correctness

Three high-impact fixes that close out long-standing footguns. All three ship with full parity across all four frameworks.

`fetchAll` actually fetches ALL rows now (no silent 100-row truncation)

Pre-v3.13.12 the convenience method defaulted to `$limit = 100` and silently truncated. The name says `fetchAll` - it should fetch them all:

```
// 150 rows in the table
$db->fetchAll("SELECT * FROM rows");
// pre-v3.13.12: returns 100 rows, silently drops the other 50
// v3.13.12:    returns all 150 rows
```

The new default is `$limit = 0`, which all six adapters (SQLite, PostgreSQL, MySQL, MSSQL, Firebird, ODBC) now interpret as "no pagination injection" - your SQL runs verbatim. To opt back into a cap, pass an explicit `$limit`:

```
$db->fetchAll("SELECT * FROM events", [], 500);   // capped at 500
$db->fetchAll("SELECT * FROM users");             // all rows
```

`$db->fetch()` (the paginated sibling that returns a `DatabaseResult` with count metadata) keeps its 100-row default - pagination is its job. Only the `fetchAll` convenience changed.

Breaking change: callers who relied on the silent 100-row cap now get every row. For very large tables, switch to `fetch()` (which paginates with metadata) or pass an explicit limit.

Trailing `;` is now stripped from user SQL in `fetch()` / `fetchOne()`

The framework appends `LIMIT n OFFSET m` to the user-supplied query (and wraps it in `SELECT COUNT(*) FROM (...) AS subq` for the count probe). When the user's query already ended with a `;`, both rewrites broke:

```
$db->fetch("SELECT * FROM users;");
// pre-v3.13.12: syntax error near "LIMIT" - the appended LIMIT followed a ;
// v3.13.12:    works - trailing ; is stripped before LIMIT is appended
```

The strip is conservative: only trailing whitespace + semicolons are removed (any number of them, including `;;`), nothing inside the statement is touched. Parameters and quoting are unchanged - the existing parameter-binding defense against injection still does all the heavy lifting.

The shared logic lives in a new `\Tina4\Database\SqlNormalizerTrait` and is `use`d by all five adapters: PostgreSQL, MySQL, SQLite, MSSQL, Firebird.

Implicit ORM binding from `TINA4_DATABASE_URL`

PHP already auto-discovered `TINA4_DATABASE_URL` on adapter init - this release simply documents and pins it as parity behaviour. When the env var is present, the first ORM model call binds the default adapter; an explicit `\Tina4\Database\Adapter` instance still takes precedence and can be used to bind a second database.

Cross-framework parity

Fix	Python	PHP	Ruby	Node
`fetch_all`/`fetchAll` returns ALL rows by default	[x] `limit=0` default	[x] `$limit = 0` default	[x] `limit: nil` default	[x] already correct (`limit?` undefined)
Strip trailing `;` from fetch SQL	[x] shared helper on `DatabaseAdapter`	[x] `SqlNormalizerTrait` on 5 adapters	[x] `Tina4::Database.strip_trailing_semicolons`	[x] exported `stripTrailingSemicolons`
Implicit ORM binding from env	[x] already worked	[x] already worked	[x] **fixed** (wired `auto_discover_db`)	[x] already worked

Tests

- Python: 2,811 passed (+24 new)
- PHP: 2,316 passed (+13 new)
- Ruby: 2,980 passed (+18 new)
- Node: 3,612 passed across 95 files (+16 new)

11,719 tests across the family, +71 new for v3.13.12, zero regressions.

---

v3.13.11 (2026-06-11) - ORM correctness pass (parity bump)

No PHP source changes. This release is a parity-version bump alongside Python's ORM correctness pass. Each issue in the Python report was checked against PHP and found to be either already-correct or N/A for the PHP framework.

Per-issue audit

- **#50.1 - Callable field defaults** -> **N/A**. PHP property defaults must be constant expressions in declarations (`public string $foo = 'bar';` is allowed; `public DateTime $foo = new DateTime();` is not). The Python/Ruby callable-default pattern doesn't apply.
- **#50.2 -** `save()` **correctly handles natural-key INSERTs** -> **already correct**. `Tina4\ORM::save()` (line 363) already routes through `recordExists($pkValue)` for natural-key models. The Python bug was specifically about that decision branch; PHP's branch was already right.
- **#49 - PostgreSQL error visibility follow-on** -> **N/A**. The cascade behaviour is psycopg2-specific (DB-API 2.0 implicit transactions). PHP's `pg_query` uses libpq in autocommit mode; every statement is its own transaction, so the cascade never happens.

- **BooleanField engine-aware DDL** -> **N/A**. PHP's `ORM::createTable()` is a minimal stub that creates a PK-only table - full schema is migration-driven, so the user controls the bool column type explicitly in their migration SQL.

Tests

2,888 passed - unchanged from v3.13.9.

---

v3.13.9 (2026-06-10)

Non-destructive AI installer - `AI::installSelected()` / `AI::installAll()` no longer clobber the user's `CLAUDE.md`. They write (or refresh) a marker-bracketed Tina4 skill block and leave the rest of the file alone.

The bug

Pre-v3.13.9 the installer wrote a full developer guide to `CLAUDE.md` (and to `.cursorules` / `.github/copilot-instructions.md` / `.windsurfrules` / `CONVENTIONS.md` / `.clinerules` / `AGENTS.md` / `.antigravity/context.md`) on every run, clobbering whatever the user had put there. If a user kept project-specific notes in `CLAUDE.md`, re-running the installer wiped all of it.

The fix

A marker-bracketed skill block - HTML comments for `.md` files, `#`-prefixed line comments for rule files:

```
<!-- tina4-skills:start -->
## Tina4 Skills
- **tina4-maintainer** - Read `.claude/skills/tina4-maintainer/SKILL.md` for framework-level cha
- **tina4-developer** - Read `.claude/skills/tina4-developer/SKILL.md` before building features.
- **tina4-js** - Read `.claude/skills/tina4-js/SKILL.md` for frontend work.
<!-- tina4-skills:end -->
```

Four behaviours:

- **Fresh install** -> write the framework guide plus the skill block.
- **Marker refresh** (idempotent) -> file exists with our markers -> replace only the bracketed block.
- **One-time migration** -> file starts with the pre-v3.13.9 framework header -> replace the old dump with the new framework guide + skill block.
- **Preserve user content** -> file exists with the user's own content (no markers, no old header) -> append the skill block to the end, leave everything else verbatim.

The actual skill content under `.claude/skills/tina4-*/SKILL.md` still gets cleanly overwritten - those are framework-owned packages, not user notes.

Same algorithm in Python / Ruby / Node

Identical four-branch logic, identical marker syntax, identical canonical action verbs in the log output. Skill content stays consistent across the family.

Tests

11 new tests in `tests/AIInstallerTest.php` (verified via reflection so private helpers stay private). All four branches plus marker detection, block replacement, idempotency, old-header detection, and rule-file vs markdown-file behaviour.

2,888 passed - no regressions.

What you'll see when you re-install

```
[OK] Migrated (replaced old framework dump in) CLAUDE.md   <- first run after upgrade
[OK] Refreshed skill block in CLAUDE.md                     <- every subsequent run
[OK] Appended skill block to CLAUDE.md                      <- user-curated file
```

---

v3.13.7 (2026-06-10)

Two changes from the 24rent app-platform team (PLATFORM-2159) - one observability hook, one production-safety fix. Both ship across **all four frameworks** with identical event payload shape.

NEW: `tina4.request.error` event

When `Router::dispatch()` catches a `Throwable`, it now emits `tina4.request.error` **before** rendering the 500 page. Listeners receive an assoc array `['exception' => $e, 'request' => $request]` and can ship the failure to CloudWatch / Sentry / Slack - even though the framework caught it.

```
use Tina4\Events;
use Tina4\Log;

Events::on('tina4.request.error', function ($payload) {
    /** @var \Throwable $e */
    $e = $payload['exception'];
    /** @var \Tina4\Request $request */
    $request = $payload['request'];

    Log::error(sprintf(
        'Route error: %s: %s',
        $e::class,
        $e->getMessage()
    ), [
        'method' => $request->method ?? null,
        'path'   => $request->path ?? null,
    ]);
    // ...or POST to your centralised logging pipeline
});
```

- **Fires for caught route throwables.** Does NOT fire for 404s - those aren't server errors.
- **Listener errors are swallowed + warning-logged** so a broken listener can't break the 500 render.
- **Listeners fire in priority order** (higher priority first, matching `Events::on($event, $cb, priority: N)`).

- **Identical event name + payload across Python / Ruby / Node** - only the per-language syntax differs.

The Router also now calls `Log::error` itself with the exception class, message, method, and path. Previously route exceptions were swallowed without any framework-side log; tail-the-log workflows now see them.

FIX: Stack trace removed from production 500 body (CWE-209)

Before v3.13.7, an unhandled route exception in PHP would render `$e->getMessage() . "\n" . $e->getTraceAsString()` into the 500 response body - absolute file paths, full call chain - **regardless of** `TINA4_DEBUG`. That's CWE-209 / OWASP A05: information disclosure.

The framework's own `Tina4/templates/errors/500.twig` now guards the trace block with `{% if error_message %}`. When `TINA4_DEBUG=false`, the Router passes an empty `error_message` and the trace block doesn't render. The trace stays in `Log::error` (server-side) and reaches observability via the new event.

When `TINA4_DEBUG=true`, the rich `ErrorOverlay` page is unchanged.

Tests

Six new tests in `tests/RouterErrorEventTest.php`: event payload shape, behaviour with no listeners, listener priority order, no traceback markers in prod body, request_id still surfaces, listener-error safety.

- 2,877 tests passing, no regressions.

Background

Reported by DevProx on the 24rent platform - they centralise observability by scraping structured JSON lines from stderr -> CloudWatch -> a Slack notifier. Route-level exceptions weren't surfacing because the framework caught them silently. The event hook fixes that without forcing any team's logging convention; the trace-leak fix is independently a security concern.

---

v3.13.6 (2026-06-09)

Parity-version bump alongside Python's #46 / #47 fixes. **No PHP source changes** - both issues were verified against the PHP codebase and required no action here.

#46 - PostgreSQL transaction cascade (no fix needed)

The cascade behaviour that prompted Python's fix is psycopg2-specific (DB-API 2.0 mandates an implicit transaction on first statement). PHP's `pg_query` / `pg_query_params` use libpq in autocommit mode by default - each statement is its own transaction, so a failed query does not poison subsequent ones.

`PostgresAdapter::query()` already populates `$this->lastError` from `pg_last_error()` on every failure, accessible via `$db->getError()`:

```
$db = \Tina4\Database\Database::create('postgres://localhost:5432/mydb');
$db->fetch("SELECT * FROM does_not_exist");
```

```
$error = $db->getError();  // already populated - has been since 3.x
```

#47 - Driver install hints (no change needed)

PHP's driver-missing exceptions already include OS-level install guidance (`sudo apt-get install php-pgsql`, `brew install php`). PHP database drivers are extensions, not Composer packages, so the Python/Ruby/Node-style "extras" pattern doesn't apply.

Tests

2,871 passing - no regressions.

---

v3.13.5 (2026-06-05)

Frond static-facade parity across PHP, Ruby, Node.js. Closes the last documented v3 parity gap (tina4-python task #32). Python's `Frond.add_filter` / `add_global` / `add_test` have worked as classmethods since v3.13.0 - now PHP / Ruby / Node match.

What changes

Filters, globals, and tests registered at app-startup persist across `new Frond()` instances. Every framework now supports the same pattern:

```
// PHP
\Tina4\Frond::addFilter("money", fn($v) => number_format((float)$v, 2));
\Tina4\Frond::addGlobal("APP_NAME", "My App");
\Tina4\Frond::addTest("positive", fn($v) => $v > 0);
```

```
# Ruby
Tina4::Frond.add_filter("money") { |v| "%.2f" % v.to_f }
Tina4::Frond.add_global("APP_NAME", "My App")
Tina4::Frond.add_test("positive") { |v| v > 0 }
```

```
// Node.js
Frond.addFilter("money", (v) => Number(v).toFixed(2));
Frond.addGlobal("APP_NAME", "My App");
Frond.addTest("positive", (v) => Number(v) > 0);
```

```
# Python - already shipped in v3.13.0
Frond.add_filter("money", lambda v: f"{float(v):.2f}")
Frond.add_global("APP_NAME", "My App")
Frond.add_test("positive", lambda v: v > 0)
```

In every framework, registering at the class level updates a static registry. The next `new Frond()` drains that registry into its own filter/global/test maps automatically. No need to thread a single `Frond` instance through the application - register at startup, render everywhere.

Instance form still works

Existing per-instance registration continues to work, and now propagates to the class registry too - so the lifecycle is symmetric:

```
$frond = new \Tina4\Frond();
$frond->addFilter("currency", $fn);
// Future `new Frond()` instances also see "currency"
```

`clearRegistry()` for test fixtures

Every framework exposes a class-level method to wipe user-registered filters/globals/tests without touching the built-ins (upper, lower, length, defined, even, ...). Useful in test setup/teardown to prevent state leaks between specs.

```
\Tina4\Frond::clearRegistry();
```

```
Tina4::Frond.clear_registry
```

```
Frond.clearRegistry();
```

```
Frond.clear_registry()
```

Implementation notes per framework

Framework	Mechanism
Python	`_ClassOrInstanceMethod` descriptor - one method, dual-callable via `__get__`
PHP	`__call` + `__callStatic` magic-method pair - PHP can't have same-name static and instance methods
Ruby	Same-name class method and instance method - Ruby naturally allows this
Node.js	TypeScript class supports same-name `static foo()` and `foo()` instance methods - distinct lookup spaces

Test count

Framework	Before	After	New
Python	2,741	2,741	0 (already covered)
PHP	2,858	2,871	+13
Ruby	2,907	2,928	+21
Node.js	3,508	3,526	+18
Total	**12,014**	**12,066**	**+52**

Upgrade

Drop-in patch. No breaking changes. Existing instance-form code (`$frond->addFilter(...)` / `frond.add_filter` / `frond.addFilter`) keeps working unchanged. The new static form is purely additive.

v3.13.4 (2026-06-04)

Three middleware/header bug fixes across all four frameworks, plus Python chapter 10 + 18 docs rewrites. Reported in tina4-book#140 and tina4-book#141 by MichaelC8E.

PY-10-02 - `@middleware()` no longer silently disables auth (SECURITY)

Before: Applying `@middleware(...)` to a POST/PUT/PATCH/DELETE route silently flipped `auth_required = false`, removing the framework's built-in Bearer-token gate. A developer adding custom logging or rate-limiting middleware to an admin endpoint would, with no warning, open it to unauthenticated callers.

After: Middleware is purely additive. Write routes stay Bearer-token-gated by default. Use `@noauth()` to open a write route, `@secured()` to lock a read route. Same rule across all four frameworks.

This is a **behaviour change** - if your code relied on the old auto-disable to handle auth in custom middleware, add `@noauth()` (and have your middleware enforce auth on its own).

PY-10-03 - `request.headers` is now case-insensitive

Before: `request.headers["Content-Type"]` returned `None`/`undefined`/`nil`. The dict was lowercase-only; mixed-case lookups silently failed. Six chapter 10 examples (`Content-Type`, `X-API-Key`, `Authorization`, `User-Agent`) were broken.

After: HTTP headers are case-insensitive per RFC 7230 Section 3.2. Same is true in every framework:

Framework	Implementation
Python	`CaseInsensitiveDict` (dict subclass, normalises string keys to lowercase on read/write)
PHP	`Tina4\CaseInsensitiveArray` (ArrayAccess + IteratorAggregate + Countable)
Ruby	`Tina4::CaseInsensitiveHash < Hash` (overrides `[]`, `[]=`, `key?`, `delete`, etc.)
Node	Proxy wrapper around `http.IncomingHttpHeaders`

`request.headers.get("Content-Type")`, `request.headers.get("content-type")`, and `request.headers.get("CONTENT-TYPE")` all return the same value. Existing lowercase code keeps working unchanged.

PY-10-01 - Function-based middleware now runs

Before: Chapter 10 taught Express-style `async def mw(req, resp, next_handler)` in 8+ examples, but the Python framework's dispatcher only looked for class-based `before_*`/`after_*` methods. Function-style middleware was silently inert - body never executed. PHP and Ruby had similar gaps (closures ran but no `next` continuation).

After: Express-style continuation chain is implemented across the family. Python adds `_is_function_middleware()` + `_invoke_handler_with_middleware()`. PHP wraps closures with `array_reverse` continuation. Ruby uses lambdas + `reverse_each`. Node already had `next()` continuation - added a regression test to keep it green.

```
@middleware(my_mw)
@post("/api/orders")
async def create_order(req, resp):
    ...

async def my_mw(req, resp, next_handler):
    if not authorised(req):
        return resp.json({"error": "forbidden"}, 403)
    result = await next_handler(req, resp)   # continue the chain
    return result
```

First-declared middleware is the outermost layer; calling `next_handler` descends to the next layer (or the route handler if last). Omitting the `next_handler` call short-circuits the chain.

Python chapter rewrites - book + docs

- **Chapter 18 (Testing)** - Fixed PY-18-04 (test runner output now shows real pytest output, not the fictional `[PASS] test_addition` format), PY-18-07a (added missing `from src.orm.Product import Product` import), PY-18-08 (`resp.status_code` -> `resp.status` across 14+ call sites, positional body `self.post(path, dict)` -> keyword `self.post(path, json=dict)`).
- **Chapter 10 (Middleware)** - Added two callouts: headers are case-insensitive in v3.13.4+; `@middleware()` is purely additive (does not change auth_required). Existing mixed-case header examples now work against v3.13.4.

Test count

Framework	Before	After	New
Python	2,725	2,741	+16
PHP	2,844	2,858	+14
Ruby	2,887	2,906	+19
Node.js	3,477	3,508	+31
Total	**11,933**	**12,013**	**+80**

Upgrade

PY-10-02 is a behaviour change with a security implication. Audit routes that use `@middleware()` on POST/PUT/PATCH/DELETE: if you rely on custom middleware to handle auth, add `@noauth()` above `@middleware()` (and make sure your middleware enforces auth). Otherwise, no action - your write routes were always supposed to require Bearer tokens.

PY-10-01 and PY-10-03 are purely additive - no breaking changes.

v3.13.3 (2026-06-03)

Two reporter-driven ergonomic additions, shipped across all four frameworks with full parity per `feedback_parity`.

`Env` typed env-var helpers (tina4-python#43)

Reading env vars by hand gets old fast: every boolean flag becomes a `os.getenv("TINA4_DEBUG", "false").lower() in ("1", "true", "on", "yes")` incantation. Every numeric tuning knob needs a try/except around `int()`. `Env` centralises it:

```
from tina4_python import Env

debug   = Env.bool("TINA4_DEBUG", default=False)
workers = Env.int("WORKERS", default=4)
rate    = Env.float("RATE_LIMIT", default=10.0)
region  = Env.str("AWS_REGION", default="us-east-1")
```

Same API across all four frameworks:

- **Python** - `from tina4_python import Env`
- **PHP** - `Tina4\Env::bool / int / float / str`
- **Ruby** - `Tina4::Env.bool / int / float / str`
- **Node.js** - `import { Env } from "@tina4/core"`

Truthy tokens (case-insensitive after `strip`/`trim`): `1`, `true`, `on`, `yes`, `y`, `t`. Falsy: `0`, `false`, `off`, `no`, `n`, `f`, empty string. Anything else returns the `default` - never raises. `int`/`float` parse failures log a warning via `Log` and fall back to default.

Function-name in log lines (tina4-python#41)

Opt-in via `TINA4_LOG_FUNC=true`. When enabled, the calling function name is injected into every log line so a `tail -f` gives you free context:

```
2026-06-03T14:22:18.341Z [INFO   ] [super_trooper] Hello from inside the function
```

Or in JSON mode:

```
{"timestamp":"...","level":"INFO","function":"super_trooper","message":"Hello..."}
```

Default off - zero overhead unless opted in. When on, ~5% per-call cost from the stack walk.

Per-framework implementation:

- **Python** - `inspect.currentframe()` walk past Log's own frames

- **PHP** - `debug_backtrace(DEBUG_BACKTRACE_IGNORE_ARGS)` + `{closure}` filter
- **Ruby** - `caller_locations(2, 16)` + block-noise regex
- **Node.js** - `new Error().stack` regex parse + anonymous filter

Anonymous frames (`<lambda>`, `<module>`, `{closure}`, anonymous IIFEs) are filtered as noise - showing `[<lambda>]` would be uglier than nothing.

Test count

Framework	Before	After	New
Python	2,675	2,725	+50
PHP	2,780	2,844	+64
Ruby	2,839	2,887	+49 (+1 pre-existing rack_app fail unchanged)
Node.js	3,420	3,477	+57
Total	**11,714**	**11,933**	**+220**

Upgrade

Drop-in patch. No breaking changes. Two new exports (`Env`, plus one new env var `TINA4_LOG_FUNC`). Existing logs and existing code keep working unchanged.

v3.13.2 (2026-06-03)

Bug-fix patch - three field reports, fixed with full cross-framework parity audit per `feedback_crosscheck_bugs`.

SCSS calc() with mixed units (tina4-python#42, tina4-php#116, tina4-nodejs#1)

The SCSS math evaluator silently folded mixed-unit arithmetic by keeping operand 1's unit and dropping operand 2's, producing wrong CSS:

- `max-height: calc(100vh - 170px)` -> `calc(-70vh)` (negative, layout-breaking)
- `width: 100% - 20px` -> `80%` (pixel term silently lost)
- `padding: 1rem + 4px` -> `5rem`

Fixed in Python, PHP, and Node - the evaluator now captures both operand units, only folds when units match (or one side is unitless for `*`/`/`), and masks `calc(...)` ranges so the browser computes them as intended. Ruby unaffected (delegates to libsass).

Router.group docs taught a crashing pattern (tina4-python#44)

The Python book and docs site showed `Router.group("/api/v1", lambda: [...])` with a zero-arg lambda. Source intentionally passes a `RouteGroup` instance to the callback, so users hit `TypeError: <lambda>() takes 0 positional arguments but 1 was given`. Docs rewritten to `lambda group: [group.get(...), group.post(...)]` matching the real contract (Node has always taught this correctly; PHP and Ruby use ambient state, no group arg needed).

DATABASE*URL -> TINA4*DATABASE_URL drift (tina4-python#45)

Three real bugs:

- **Python ORM error message** told users to "set DATABASE_URL in .env" - but the v3.12 boot guard rejects that bare name. Users following the error message hit a hard stop.
- **Python dev-admin `.env` writer** stripped the `TINA4_` prefix when updating existing rows, actively corrupting the config every time the user saved a new connection through the dashboard.
- **Node** `Database.fromEnv()` defaulted to `"DATABASE_URL"` as the env-var key, missing the project's actual connection. The ORM error message had the same drift.

All fixed. PHP and Ruby audited - already correct in both.

Test count

Framework	Before	After	New
Python	2,665	2,675	+10
PHP	2,774	2,780	+6
Ruby	2,839	2,839	0 (parity bump only)
Node.js	3,406	3,420	+14
Total	**11,684**	**11,714**	**+30**

Upgrade

Drop-in patch. No breaking changes. No new public API.

v3.13.1 (2026-06-02)

Cross-framework parity patch. Closes the remaining audit-flagged docs-vs-code gaps that didn't make 3.13.0 - the documentation claimed APIs across PHP / Ruby / Node that only Python had. This release ships those APIs everywhere and rewrites the PHP chapters that referenced fictional symbols.

Convenience parity additions (Groups A / B)

Three highest-impact cross-framework methods every documentation set already claimed existed. PHP / Ruby / Node now match Python:

- `db.fetchAll(sql, params)` / `db.fetch_all` / `$db->fetchAll` - returns the records list directly. Symmetric with `fetch_one`. For the 80% case where you don't need the `DatabaseResult` metadata.
- `Database.getConnection(url)` / `.get_connection` / `::getConnection` - classmethod factory matching SQLAlchemy's `engine.connect()`. Falls back to in-memory SQLite when no URL or env resolves.
- `Api(bearerToken=, username=, password=, headers=, verifySsl=)` **ergonomic kwargs** - three setter calls collapse to one constructor. Bearer wins over basic-auth when both are passed. `verifySsl=False` is the positive form of `ignoreSsl=true`.

Decorator-style GraphQL resolvers across the family

Python `@GraphQL.resolve` shipped in 3.13.0. This release adds:

- **PHP** - `GraphQL::resolve("Type", "field", $callable)` static method + class-level resolver registry that `new GraphQL()` drains into its schema.
- **Ruby** - `Tina4::GraphQL.resolve("Type", "field") { |root, args, ctx| ... }` with block-based registration.
- **Node.js** - `GraphQL.resolve(typeName, fieldName, resolver)` matching the cross-framework shape.

All four frameworks now support the FastAPI / Strawberry / Ariadne pattern where resolvers register at module-import time before any `GraphQL` instance is constructed, and where post-startup registrations land in the active default singleton via `setDefault(gql)` / `Tina4::GraphQL.default_instance = gql`.

Class-based service pattern across the family

`class FooWorker extends Service { run() { ... } }` - chapter 27 / equivalent docs have long taught this pattern. Until 3.13.1, only the runner was real:

- **PHP** - new `Tina4\Service` abstract base class + `ServiceRunner::registerService($name, $service)` static helper.
- **Ruby** - new `Tina4::Service` class + `Tina4::ServiceRunner.register_service(name, service)`.
- **Node.js** - new `Tina4Service` abstract class + `ServiceRunner.registerService(name, service)`.
- **Python** - new `tina4_python.service.Service` base + `ServiceRunner.register_service(name, service)` (this release closes the gap; Python had only the function-style runner before).

All four ship `run()` (abstract), `stop()`, and `should_stop()` / `shouldStop()` helpers backed by an internal flag. Function-style services using bare callables continue to work alongside the new class-based pattern.

PHP chapter rewrites (`docs/php/` and `book-2-php/`)

The 3.13.0 audit found that the PHP testing-chapter disaster was the tip of a larger pattern - multiple PHP chapters taught APIs that didn't exist. 3.13.1 rewrites all seven of them:

- **Chapter 15 - Logging** - primary surface now `Tina4\Log::info()/warning()/error()` instead of the legacy `Tina4\Debug::message()` shim (still works).
- **Chapter 18 - Testing** - `$response->statusCode` -> `$response->status` across 23 occurrences; CLI section updated (`tina4 test` runs the suite; `vendor/bin/phpunit` for targeted runs).
- **Chapter 19 - Scaffolding** - v2 `Tina4\Get::add()` / `Post::add()` / `Put::add()` / `Delete::add()` syntax replaced with `Tina4\Router::get/post/put/delete`; fictional `->description()` chain replaced with real `->swagger([...])`.
- **Chapter 22 - GraphQL** - chapter's decorator pattern (`GraphQL::resolve("Type", "field", $fn)`) now matches real source (built this release).
- **Chapter 25 - WSDL** - `@wsdl_operation` docblock replaced with `#[WSDLOperation([...])]` PHP attribute; methods now return associative arrays matching the response-shape spec; `Router::soap()` -> `Router::any()` + manual `(new Service($request))->handle()`.
- **Chapter 27 - ServiceRunner** - `new ServiceRunner()` + `->add()` instance API replaced with `ServiceRunner::registerService()` + `ServiceRunner::start()` static API. The `Tina4\Service` base class the chapter teaches now exists.
- **Chapter 34 - Deployment** - un-prefixed env vars (`SECRET`, `CORS_ORIGINS`, `SMTP_USER`, `JWT_SECRET`, `API_KEY`, `SWAGGER_TITLE`) replaced with `TINA4_`-prefixed forms. The v3.12 boot guard rejects the legacy names with `exit(2)`.

Test count

Net new across the family this release:

Framework	Before	After	New
Python	2,654	2,665	+11
PHP	2,749	2,774	+25
Ruby	2,827	2,839	+12
Node.js	3,384	3,406	+22
Total	**11,614**	**11,684**	**+70**

Upgrade

Drop-in patch - no breaking changes. Existing source-code patterns from 3.13.0 continue to work; the new methods are additive. Documentation rewrites in chapters 15 / 18 / 19 / 22 / 25 / 27 / 34 redirect copy-paste examples to the real APIs the framework actually ships.

---

v3.13.0 (2026-06-01)

The docs-vs-code parity release. A cross-framework audit of 381 markdown files surfaced 146 hallucinations, signature drifts, and stale references across Python, PHP, Ruby, Node, and tina4-js. 3.13.0 closes the chapter-18 disaster pattern - where documentation taught a class-based API that didn't exist - by shipping the missing pieces, renaming the misnamed pieces, and rewriting the aspirational chapters.

The headline fire: `Test` class with HTTP helpers

Every framework's chapter 18 has long shown integration tests like:

```
class UserApiTest(Test):           # tina4_python.test.Test
    def test_health(self):
        resp = self.get("/health")
        assert_equal(resp.status, 200)
```

Until 3.13.0, only `Test` (the bare assertion base) existed - calling `self.get(...)` crashed with `AttributeError`. The HTTP test client lived in a separate `TestClient` class that the docs never mentioned.

This release mixes `TestClient.get / post / put / patch / delete` into the `Test` base across every framework:

- **Python** - `tina4_python.test.Test` (extends `unittest.TestCase`, pytest auto-discovers)
- **PHP** - `Tina4\Test` (extends `PHPUnit\Framework\TestCase`)
- **Ruby** - `Tina4::Test` (zero-dep; built-in `run_all` runner)
- **Node.js** - `Tina4Test` (zero-dep; built-in `Tina4Test.runAll()` runner)

Plus positional assertions on every framework - `assertEqual(actual, expected, message)`, `assertNotEqual`, `assertTrue`, `assertFalse`, `assertNull`/`assertNullValue`, `assertNotNull`/`assertNotNullValue`, `assertRaises` - matching the documented `(actual, expected, message)` shape.

`Auth.valid_token` now returns the payload, not a bare bool - BREAKING

The most common silent-fail pattern caught by the audit. Every framework's docs claimed `valid_token` returned the decoded JWT payload; every framework's source returned `bool` and forced a second `get_payload` call.

Framework	Before	After	
Python	`Auth.valid_token(token) -> bool`	`Auth.valid_token(token) -> dict \	None`
PHP	`Auth::validToken(token) -> bool`	`Auth::validToken(token) -> array \	null`
Ruby	`Auth.valid_token(token) -> Boolean`	`Auth.valid_token(token) -> Hash \	nil`

Node	`validToken(token) -> boolean`	`validToken(token) -> Record<string, unknown> \`	null`

Matches PyJWT / firebase-jwt-ruby / firebase/php-jwt / jsonwebtoken conventions. Truthy/falsy contract preserved - existing `if (validToken(t))` callers keep working because a non-null object is truthy and null is falsy.

Python-specific groups (mirrored to PHP/Ruby/Node in follow-up patches)

The Python framework is the reference per `feedback_python_master`. Six groups landed in tina4-python:

- **Group A - ergonomic additions**: `Database.get_connection()`, `db.fetch_all()`, `db.pool`, `DatabaseResult.columns`, `Job.error`, `Queue.produce(delay_until=datetime)`, module-level `migrate(db)`/`rollback(db)`/`status(db)`, module-level `i18n.t()`, dict-style `session[key]`, `WebSocketConnection.connection_count`. All zero-risk additions - no signatures changed.

- **Group B - signature expansions**: `Api(bearer_token=, username=, password=, headers=, verify_ssl=)` kwargs, `Model.find(pk)` int overload (Active Record convention), `@description(summary, detail=, params=, query=)`, `@tags(str | list)`, `@example_response(status_code, data)`, `response.render(template, data, status_code)`, `response.cookie(name, value, options_dict)`, `response(data, headers={})`, `@get(path, description=, middleware=["ResponseCache:300"])` with string-form middleware parser.

- **Group C - mixins + decorators**: the Test HTTP mixin (covered above), `Frond.add_filter / add_global / add_test` callable as classmethod OR instance method via a `_ClassOrInstanceMethod` descriptor, `@GraphQL.resolve("Type", "field")` decorator with class-level registry - chapter 22's pattern now works as documented.

- **Group D - return-type changes (BREAKING)**: `Container.reset()` now clears singleton cache only (factories survive); new `Container.reset_all()` for the old wipe-everything behaviour. `queue.dead_letters()` returns `list[Job]` with `.error` populated, not `list[dict]`. `Model.where(..., with_count=True)` returns `(list, int)` tuple for pagination UIs.

- **Group E - renames (BREAKING)**: `ai.install_all()` -> `ai.install_context()`; new `ai.detect_ai()`, `ai.detect_ai_names()`, `ai.status_report()`. `queue.consume(id=)` -> `queue.consume(job_id=)`. `Api.send_request()` -> `Api.send()`. `I18n(locale=, path=)` preferred over `I18n(locale_dir=, default_locale=)` (legacy kept). `TINA4_TOKEN_EXPIRES_IN` preferred over `TINA4_TOKEN_LIMIT` for JWT expiry (both honoured; new wins; constructor arg overrides both).

- **Group F - top-level re-exports + scaffolder**: `from tina4_python import Api, WSDL, wsdl_operation, GraphQL, AutoCrud, Messenger, on, emit, once, off, tests` now resolve. `Model.select()` with no args defaults to `SELECT * FROM <table>` so the CRUD-list scaffolder template's emitted code actually runs.

PHP-specific: `Tina4\Debug` shim

Chapter 15 of the PHP logging docs taught `Tina4\Debug::message($msg, TINA4_LOG_INFO, [...])`. Neither the class nor the constants existed. Real logger is `Tina4\Log`.

This release ships a `Tina4\Debug` compatibility shim that forwards to `Tina4\Log`, plus defines the `TINA4_LOG_*` level constants - so the chapter's code samples run as-written. For new code, prefer `\Tina4\Log::info()` etc.

Documentation sweep

Aside from the source-side changes, the audit caught hundreds of stale references in docs site + book + AI skills + CLAUDE.md files. All fixed in this release:

- ~80 occurrences of `from tina4 import` -> `from tina4_python import` (the Python package is `tina4_python`, not `tina4`)
- `from tina4_python.router` -> `from tina4_python.core.router`
- `TINA4_SESSION_HANDLER` -> `TINA4_SESSION_BACKEND` (matches the env var the framework actually reads)
- `DATABASE_NAME=` -> `TINA4_DATABASE_URL=` (legacy un-prefixed names get rejected by the v3.12 boot guard)
- `@cached(True, max_age=N)` -> `@cached(max_age=N)` (bogus first arg)
- `Template.render()` -> `response.render()` (Template class doesn't exist; renamed to Frond)
- `Debug.error()` -> `Log.error()` in Python (Debug class doesn't exist)
- `Producer` / `Consumer` (removed in v3.2.0) -> `Queue.push / consume`
- `Email` -> `Messenger`, `event.fire / @listener` -> `emit / @on`, `gql` singleton -> `GraphQL()` + `@GraphQL.resolve`
- **Security fix**: `Auth.check_password(hash, password)` -> `(password, hash)` in skill ref - the bcrypt comparison was returning False every time due to reversed args (silent-failure auth)
- `request.files['content']` is **raw bytes** - drop `base64.b64decode()` from upload examples
- Deployment chapter env vars all `TINA4_`-prefixed (un-prefixed names brick boot under v3.12 guard)

Aspirational chapters rewritten

Two Python chapters were built on APIs that didn't exist:

- **Chapter 22 (GraphQL)**: rewritten around the new `@GraphQL.resolve("Type", "field")` decorator (the FastAPI/Strawberry pattern). The previous `gql.schema.add_query("name", {dict})` form still works but is no longer the primary documented path.

- **Chapter 25 (WSDL)**: rewritten around the real subclass pattern (`class Calculator(WSDL): @wsdl_operation({"Result": int}) def Add(self, ...): ...; Calculator(request).handle()`). The previous `WSDL(service_name=, namespace=, endpoint=)` constructor + `handle_wsdl` / `handle_request` API was entirely fictional.

tina4-js doc drift caught

The cross-framework audit's synthesis pass dropped tina4-js findings; the raw agent transcripts had 23 real findings:

- Every import in CLAUDE.md and `docs/js/09-graphql.md` used `"tina4-js"` (with hyphen). The npm package is named `tina4js`. Fixed.
- `pwa({...})` was treated as callable; real API is `pwa.register({...})`. `PWAConfig.icon` is a single string, not an `icons: [...]` array.
- `static props = { label: { type: String, default: "..." } }` - the `{ type, default }` wrapper is fictional. Real shape is `static props = { label: String }`.
- `router.navigate('/users/42')` - `navigate` is a top-level export, not a method on `router`.
- Chapter 14's `<slot>` inside a `static shadow = false` component - slots are a Shadow DOM feature. Chapter 14 contradicted chapter 4. Switched to `shadow = true`.

Test counts

Net new across the family:

Framework	Before	After	New
Python	2,537	2,654	+117
PHP	2,742	2,749	+20
Ruby	2,800	2,816	+16 (1 pre-existing unrelated rack_app failure)
Node.js	3,366	3,384	+18
Total	**11,445**	**11,603**	**+171**

Upgrade

`Auth.validToken` is the breakage to know about - your `if Auth::validToken($t)` style code keeps working unchanged because non-null arrays are truthy and null is falsy. If you do `=== true` / `=== false` strict comparisons, switch to `!== null` / `=== null`.

Python: `ai.install_all()` -> `ai.install_context()`, `queue.consume(id=)` -> `consume(job_id=)`, `Api.send_request()` -> `Api.send()`, `Container.reset()` semantic change (use `reset_all()` for old behaviour).

Everything else is additive - new properties, new kwargs, new convenience methods that match what the docs have promised for years.

v3.12.14 (2026-06-01)

Two independent fixes ship together as 3.12.14. **Python** - the `tina4_python.test` class-based xUnit testing surface that the chapter 18 documentation has always promised but never actually existed. Reports came in of developers copy-pasting `from tina4_python.test import Test, assert_equal, assert_true` straight out of the book and getting `ModuleNotFoundError`. The fix was to build the module to match the documentation, not the other way around. **PHP** - `:named` placeholder translation for the four non-PDO adapters where `ORM::save()` was silently failing.

Python - `tina4_python.test` xUnit testing surface

The testing chapter taught a `Test` base class with positional assertions:

```
from tina4_python.test import Test, assert_equal, assert_true

class BasicTest(Test):
    def test_addition(self):
        assert_equal(2 + 2, 4, "Basic addition should work")

    def test_string_contains(self):
        greeting = "Hello, World!"
        assert_true("World" in greeting, "Greeting should contain 'World'")
```

The module did not exist. Every developer who followed the chapter hit an immediate import error. The other surface, `tina4_python.Testing` with the inline `@tests` decorator, has always existed - but the two are for different purposes and the docs only documented one of them.

The fix ships the missing module - `tina4_python/test/__init__.py` - with the `Test` base class (inherits `unittest.TestCase`, so pytest discovers any subclass regardless of class-name convention) and 13 positional assertions. The signatures are uniform: `(actual, expected, message)`. The 2-arg legacy `(value, message)` form keeps working - a type-based dispatch detects which shape the caller used. `assert_raises` accepts three forms: docs form (`callable, exception, message`), context-manager form (`with assert_raises(X):`), and unittest order (`exception, callable`). Lifecycle hooks come in both flavours - snake_case `set_up`/`tear_down` (the Tina4 idiom) and camelCase `setUp`/`tearDown` (for users coming from unittest) - without double-calling when a subclass uses either one.

```
# 13 assertions, all uniform (actual, expected, message)
assert_equal(actual, expected, message="")
assert_not_equal(actual, expected, message="")
assert_true(actual, expected=True, message="")
assert_false(actual, expected=False, message="")
assert_none(actual, expected=None, message="")
assert_not_none(actual, expected="not None", message="")
assert_in(item, container, message="")
assert_not_in(item, container, message="")
assert_is_instance(value, expected_type, message="")
assert_greater(actual, expected, message="")
assert_less(actual, expected, message="")
assert_almost_equal(actual, expected, places=7, message="")
```

```
assert_raises(callable, exception_class, message="")
```

51 new tests in `tests/test_test_module.py` pin the contract: BasicTest from the chapter runs verbatim, every assertion fails when it should and passes when it should, both 2-arg and 3-arg shapes work, snake_case and camelCase lifecycle hooks fire once each (never both). Full Python suite: 2,537 passing.

`tina4 test` continues to run pytest (`subprocess.run([sys.executable, "-m", "pytest", "tests/"] + args)`).

Cross-framework parity check (testing)

PHP / Ruby / Node testing chapters already teach native conventions correctly - PHPUnit, RSpec, and `node:test` respectively. No fake API to fix. The Python-specific gap was that Tina4 had two testing surfaces (`tina4_python.Testing` for inline `@tests` decorator, `tina4_python.test` for class-based suites) and only one of the two existed. The other three frameworks defer to a single native runner each, so the same trap doesn't apply.

PHP - :named placeholder translation across non-PDO adapters

The ORM's `save()` emits `:named` placeholders because PDO would accept them. Four of the five PHP database adapters do not use PDO. `MySQLAdapter` (mysqli), `MSSQLAdapter` (sqlsrv), `FirebirdAdapter` (ibase/fbird), and `PostgresAdapter` (pgsql) all bind positionally. Every INSERT/UPDATE through `save()` against those four engines failed silently. Reads worked because read paths typically use `?` or no params.

A single helper, `SqlTranslation::namedToPositional($sql, $params)`, translates `:name` to `?` and reorders `$params` to match the SQL order. Wired into the four affected adapters at the top of their prepare/execute paths. The helper skips string literals and SQL comments, so a literal `:colon` inside a value stays as a value. Duplicate names bind once per occurrence, so `WHERE id = :id AND parent_id = :id` works as expected.

`SQLite3Adapter` is untouched. ext-sqlite3 natively accepts `:name` via `SQLite3Stmt::bindValue`. The other four did not, and now do.

15 unit tests pin the helper in `tests/SqlTranslationNamedToPositionalTest.php`: order preservation, duplicate names, quoted strings, line and block comments, unknown placeholders, null values, and the `0`-as-value case. Full PHP suite: 2,290 passing.

Cross-framework parity check (`:named` placeholders)

Python (`mysql-connector-python` uses `%s`), Ruby (`mysql2` uses `?`), and Node (`mysql2` uses `?`) build their INSERT/UPDATE SQL with positional placeholders from the ORM down. No `:named` ever emitted. Audited the MySQL adapter and `save()` path in each before shipping; confirmed clean. PHP-only fix.

Upgrade

Drop in for both Python and PHP. No `.env` changes, no API changes.

Python users who followed chapter 18 and hit `ModuleNotFoundError` - bump to `3.12.14`, the `from tina4_python.test import Test, assert_equal, ...` import now resolves.

Existing tests written against `tina4_python.Testing` (the inline `@tests` decorator) continue to work - that surface was not touched.

PHP users - `:named` and `?` both work, and the framework picks the right form for whichever driver is underneath. Existing ORM `save()` calls start succeeding on MariaDB/MySQL, PostgreSQL, MSSQL, and Firebird.

Ruby and Node users - no framework change shipped in 3.12.14. Stay on `3.12.13` or bump to `3.12.14` for version alignment. Both are functionally identical.

v3.12.13 (2026-05-29)

Consolidated parity release. PHP ran ahead through two independent patch releases (3.12.11-3.12.12) while Python / Ruby / Node stayed at 3.12.10. This release realigns all four frameworks on **3.12.13** and ships the cross-framework dev-admin parity sweep - five tiers of work that bring PHP, Ruby and Node up to Python's AI-assisted development surface.

Cross-framework dev-admin parity sweep (Tier 1-5)

The Python framework had pulled ahead on a series of dev-admin features driven by real frustration with the AI coder loop ("Applying a small patch went and messed up my whole file", "Says it is creating files but then doesn't", repeated import-error spirals). This release ports the full set to PHP, Ruby, and Node - same intent, language-idiomatic implementations.

Tier 1 - MCP defensive write layer. `file_write` and `file_patch` now refuse prose-as-filenames (the LLM occasionally emits `## FILE: I'll implement Step 1 by creating the database migration` and the parser used to write a zero-byte file with that sentence as its filename), normalise bare top-level `routes/` / `orm/` / `templates/` / `seeds/` / `controllers/` / `middleware/` paths to their canonical `src/<dir>/` form (auto-discovery only scans `src/`, so a file at `templates/foo.twig` was dead weight), back up existing files to `.tina4/backups/<flat-path>.<ISO-ts>.bak` before overwrite, and refuse suspicious truncations (>200B file -> <30% size = almost always a truncated LLM response). Every attempt logs to `.tina4/agent.log` with a structured category (`write.ok` / `write.refused` / `write.path_normalized` / `write.import_failed`) - the supervisor reads that file on every turn so it sees what broke last time and can self-correct without asking the developer "what's the error?".

Tier 2 - Post-write syntax verification. PHP shells out to `php -l`, Ruby to `ruby -c`, Node to `node --check` (and single-file `tsc --noEmit --allowJs --skipLibCheck` for `.ts`). On parse error the tool result gets an `import_error` field AND a `write.import_failed` log entry surfaces in the next supervisor turn's failure context. Catches hallucinated framework APIs (`CharField` doesn't exist in `tina4_python.orm.fields` - should be `StrField`; `auto_now_add` keyword on `Field.__init__()`) at write time instead of letting them propagate to a runtime 500 the user only discovers by hitting the URL.

Tier 3 - `/__dev/api/threads` + `/__dev/api/chat` proxy. The SPA now talks to the Rust supervisor agent the same way regardless of framework. `_supervisor_base_url()` matches Python's 4-step ladder (`TINA4_SUPERVISOR_URL` -> `TINA4_AGENT_PORT` -> `PORT+2000` -> `9145`). `active_file` rides through `/chat` POST verbatim so deictic phrases ("fix this", "explain

this") bind to the editor's open file without the supervisor asking. The Node port forwards SSE chunks as they arrive; PHP and Ruby buffer (functional - EventSource parses fine - but feels less snappy until a future round of Rack/PHP-FPM streaming work).

Tier 4 - Customer feedback widget. A floating bubble for end-users of a shipped Tina4 app, gated by `TINA4_ENABLE_FEEDBACK=true` AND a non-empty `TINA4_FEEDBACK_WHITELIST`. The framework's response middleware injects `<script src="/__feedback/widget.js" data-tina4-feedback></script>` immediately before the LAST `</body>` tag on text/html responses, ONLY for whitelisted users, NEVER on `/__dev` or `/__feedback` paths (no double-bubble UX on the developer dashboard). One conversational turn at a time POSTs to `/__feedback/api/turn` -> server-side identity stamp from the verified JWT (clients cannot fake `sender`) -> forward to the Rust agent's intake-only agent (zero tools, JSON-only output). Finalised tickets land in the dev admin sidebar with `kind:"feedback"`. Rate-limited at 5 turns/hour per user.

Tier 5 - Stale-source overlay badge + `list_plans()` merge. The error overlay now stamps `captured_at` on render and tags each stack frame whose source file has been modified since: "FILE MODIFIED @ HH:MM:SS UTC - source may not match what failed". Stops the user from chasing ghosts when the AI coder rewrote the file between the error and the page reload. `list_plans()` reads from BOTH `plan/` (user-curated canonical) AND `.tina4/plans/` (AI-planner output), dedupes by filename with `plan/` winning on collision, sorts newest-first, and returns a `path` field so the SPA can open the right file regardless of source dir.

Test counts. Per-framework deltas across the sweep:

Framework	Before -> After (full suite)
Python	2453 -> 2453 (canonical - no new tests, just released)
PHP	2235 -> 2714 (+479)
Ruby	2747 -> 2800 (+53)
Node	3263 -> 3368 (+105)

PHP's larger delta reflects new tests + the 3.12.11 + 3.12.12 lineage rolling forward.

Why all four frameworks at once. Per the cross-framework parity rule: a feature that exists in only one framework is technical debt. The Python-only Tier 1-5 surface had been accumulating for two weeks while the UX was settling. With it settled, this release closes the gap in one coordinated sweep.

Folded-in from PHP 3.12.11 - file upload regression (`tina4-book#139`)

`WebSocket::parseHttpHeaders()` previously split the entire raw HTTP request on `\r\n` and iterated every line for a `:` to fill the headers map. Multipart body parts have their own `Content-Type`, `Content-Disposition`, and `Content-Transfer-Encoding` headers - those lines matched the parser and overwrote the real request `Content-Type: multipart/form-data; boundary=...` with whatever the last body part's content type was (typically `application/pdf`, `image/png`). Downstream `str_contains($contentType, 'multipart/form-data')` then failed, the multipart branch was skipped, `$parsedFiles` was never set, and `$request->files` came out empty. Every file upload through the stream-socket server was silently lost - the body landed in `$request->body` as a raw multipart string with no way to parse it.

Fix. Stop the parser at the first `\r\n\r\n` (RFC 9112 Section 2.2 boundary between headers and body) before splitting into lines. One logical change in `Tina4/WebSocket.php`. 9 regression tests in `tests/BookIssue139Test.php` cover single-part, multi-part, and mixed-header cases.

Cross-framework parity check. Python (`http.server`), Ruby (`webrick`/`puma`), and Node (built-in `http` module) all delegate header parsing to upstream stdlib HTTP parsers that already split headers from body correctly. PHP was the only framework with a hand-rolled HTTP parser in this code path. No port needed.

Folded-in from PHP 3.12.12 + Python 3.12.13 - v2 `tina4_migration` auto-upgrade (#115)

Projects upgrading from tina4 ^2.x to ^3.x carried a v2-shaped `tina4_migration` table that v3's `ensureMigrationsTable()` left untouched (the `CREATE TABLE IF NOT EXISTS` short-circuited). The v3 reader then selected columns that didn't exist, fell into the "never seen this migration, run it" branch, and re-applied already-applied migrations - typically failing on duplicate-column / table-already-exists errors when the SQL was non-idempotent. The AirOffices ~190-migration codebase tripped on this in March 2026 and needed a manual SQL backfill at the time.

Framework	v2 schema	v3 schema
PHP	`migration_id VARCHAR(14)`, `description`, `content BLOB`, `passed`	`id INT PK`, `migration`, `batch`, `applied_at`
Python	`description` as identifier, `content`, `passed`	`migration_id`, `migration_name`, `executed_at`

Fix. `ensureMigrationsTable()` (PHP) and `_ensure_tracking_table()` (Python) now detect a v2-shaped table (v2 columns present, v3 columns absent) and call an in-place upgrade that ALTERs in the v3 columns alongside the v2 ones, then backfills v3 fields from the v2 data. v2 columns are kept in place so a manual rollback path stays open - they're simply ignored by v3 readers. The match is by file stem: a v2 row's identifier is matched against `migrations/` files by

basename (Python uses `000001_create_users.sql` -> stem `000001_create_users` -> v2 description `create_users`).

Cross-framework parity check. Ruby and Node never shipped a v2 migration table with the trapping shape - their v2 lineages used a different column layout that v3's tracker tolerated. Nothing to port.

Folded-in from PHP 3.12.11 - request URL parity

`$request->url` now returns the full absolute URL (`https://host:port/path?query`) instead of just the path. `$request->queryString` (raw query bytes) added for parity with `request.query_string` on the other frameworks. Drop-in - old code that read `$request->path` (untouched) keeps working.

Upgrade

Drop in. No `.env` changes, no API changes.

For projects upgrading from v2.x: the v2 `tina4_migration` auto-upgrade runs once on first boot against v3 - back up your migrations table beforehand if you're paranoid. The upgrade is non-destructive (v2 columns are kept alongside the new v3 ones).

For projects using the dev admin AI coder loop: the new MCP defensive layer will silently rewrite `## FILE: routes/foo.py` to `src/routes/foo.py` and log a `write.path_normalized` entry. If you were relying on the old behaviour (writes landing wherever the LLM emitted them), this will move some files. Run `tail -n 50 .tina4/agent.log | grep path_normalized` after upgrading to see what got rewritten.

For shipping apps that want the customer feedback widget: set `TINA4_ENABLE_FEEDBACK=true` AND `TINA4_FEEDBACK_WHITELIST=alice@example.com,bob@example.com` in `.env`. The widget appears only for those users on non-`/__dev` pages.

v3.12.10 (2026-05-14)

Version-alignment release. PHP ran ahead through three independent patch releases (3.12.7-3.12.9) while Python / Ruby / Node stayed at 3.12.6. This release realigns all four frameworks on **3.12.10** and ships the ORM `save()` fix.

PHP - `ORM->save()` no longer swallows write failures (#114)

`ORM->save()` called `update()`/`insert()` but ignored their `bool` return - it only caught exceptions. The PHP adapter's `exec()` returns `false` on a bad statement instead of throwing, so a failed `UPDATE` (commonly: one referencing a public model property with no matching DB column, since `getDbData()` includes every public property) slipped through. The empty transaction got committed and `save()` returned `$this` - the documented success signal. Callers relying on the `save(): static|false` contract believed the row persisted when nothing changed. **Silent data loss** - no exception, no log.

Fix. `save()` now captures the `bool` return of `update()`/`insert()`, rolls back, and returns `false` on a falsy result.

```
$ok = $this->_exists || ... ? $this->update() : $this->insert();
if ($ok === false) { $this->_db->rollback(); return false; }
$this->_db->commit();
```

Cross-framework parity check. Python, Ruby and Node don't have this exact failure mode - they build the write payload from declared fields only (not all public properties), and their DB adapters raise on bad SQL, which the existing `try/except` already catches. PHP was the outlier on both counts. 3 regression tests in `tests/Issue114Test.php`; PHP suite 2235 -> 2238 passing.

Also in the PHP 3.12.7-3.12.9 patch line

These shipped to PHP between 3.12.6 and this release; folded into the consolidated 3.12.10 line:

- **3.12.7** - `Request` now normalises caller-provided header keys to lowercase. Some upstream entry points (Apache+PHP-FPM custom mappings, certain proxies, hand-written test fixtures) hand headers in with original case. The constructor only looks them up by lowercase key, so without normalisation `multipart/form-data` content-type detection silently missed and the body fell through as raw bytes - a follow-up to the #135 fix.
- **3.12.8 / 3.12.9** - Router gained RFC 9110 HTTP method conformance: proper `HEAD` and `OPTIONS` handling, `405 Method Not Allowed` with an `Allow` header listing the methods a route does support.

Python / Ruby / Node

Version-only bump 3.12.6 -> 3.12.10 to realign with PHP. No behavioural changes in these three since 3.12.6.

Upgrade

Drop in. No `.env` changes, no API changes. PHP users on 3.12.9 get the `save()` fix; everyone else gets a version-number realignment.

v3.12.6 (2026-05-06)

Python-only fix release. PHP / Ruby / Node ship the same version stamp for parity but carry no behavioural changes.

Python - psycopg2 `%` substitution no longer trips PL/pgSQL function bodies (#40)

A migration containing a PL/pgSQL function with literal `%` characters in a `RAISE EXCEPTION` (or `format()`) call used to fail with the misleading:

> *RuntimeError: Migration failed: list index out of range*

The error message gave no hint that the `%` chars were the problem. The user-facing failure looked like a tina4 internal bug - actually psycopg2's argument-substitution system tripping on the literal percent signs.

Root cause. `PostgreSQLAdapter.execute(sql, params)` always called `cursor.execute(sql, params or [])`. psycopg2 interprets `%` as parameter placeholders WHENEVER the `params` arg is ~~supplied - even an empty list~~ `[]`. So a function body containing

`RAISE EXCEPTION 'thing % conflicts with %', a, b` (perfectly valid PL/pgSQL) blew up because psycopg2 thought `%` was a placeholder and there were no values to substitute.

Fix. New `PostgreSQLAdapter._safe_execute(cursor, sql, params)` helper routes empty/None params through `cursor.execute(sql)` (no second arg), which makes psycopg2 skip the substitution pass entirely. Literal `%` chars flow through untouched. Applied at every `cursor.execute(...)` call site in the adapter (5 spots across `execute`, `fetch`, `fetch_one`).

Tests. 5 new unit tests in `tests/test_postgres_percent_substitution.py` pin the helper's branching. 3 live-Postgres regression tests in `tests/test_postgres_plpgsql_percent.py` exercise a real CREATE FUNCTION + trigger flow with literal `%` in the body - skipped automatically when no Postgres is reachable. Full suite: 2453 passing (was 2448).

Cross-framework parity check. PHP (`pg_query` vs `pg_query_params`) and Ruby (`exec` vs `exec_params`) already branch on params presence so they don't have this bug. Node uses `$1` placeholders not `%`, so the same class of bug doesn't apply.

Long-standing tina4-js #37 confirmed fixed

`frond.form.submit` not following 3xx redirects - fixed in frond v2.1.2 back on April 11, 2026 (`xhr.responseURL` comparison + `window.location.href` navigation). All four framework `public/js/frond.min.js` copies carry the fix. The original issue stayed open because the reporter never confirmed against the patched build.

Upgrade

Drop in. No `.env` changes, no API changes.

v3.12.5 (2026-05-06)

PHP-only bug fix release. Python / Ruby / Node ship the same version stamp for parity but carry no behavioural changes.

PHP - multipart bodies with file uploads now parse correctly (#135)

Two stacked bugs in `Tina4\Request::__construct` made `$request->body` come through as the raw multipart bytes (~11 KB blobs starting with `------WebKitFormBoundary...`) whenever the request included a file upload:

- The constructor called `$this->parseBody()` BEFORE initialising `$this->files`. Inside parseBody's multipart branch, the line `$this->files = array_merge($this->files, $parsed['files'])` read an uninitialised typed property - fatal `Error`.
- After fixing the init order, that same line tried to mutate the `readonly` `$files` property - another fatal `Error`.

Both errors got swallowed by the upstream error handler and the route handler received the raw multipart payload instead of the parsed associative array. Routes that worked fine for ordinary form posts broke the moment a file field came along.

Fix. Move `$this->files` initialisation AFTER `parseBody()` runs. parseBody stashes extracted multipart files on a new private mutable `$multipartFiles`; the constructor merges

them into the readonly `$files` in a single assignment that respects the readonly contract.

4 new regression tests in `tests/Issue135Test.php` pin the constructor's contract. Full PHP suite: 2235 passing (was 2231).

Upgrade

Drop in. No `.env` changes, no API changes, no other framework changes.

v3.12.4 (2026-05-06)

Documentation-truth release. The `audit-truth.py` CI gate (introduced post-3.12.3) flagged 39 env vars referenced in docs that no framework actually read. This release closes that gap: 25 of them now exist in code, the other 14 are deleted from docs (11 hallucinations + 6 clustering vars deferred to tina4#2). Both audit gates (CLI drift + env-var drift) are now strict in CI.

25 new env vars across all 4 frameworks

Server: `TINA4_HOST`, `TINA4_SUPPRESS`, `TINA4_ENV_FILE`. Health: `TINA4_HEALTH_PATH` (default `/__health`, with `/health` kept as a legacy alias), `TINA4_TRAILING_SLASH_REDIRECT`. Sessions: `TINA4_SESSION_HTTPONLY`, `TINA4_SESSION_NAME`, `TINA4_SESSION_SECURE`. Templates: `TINA4_TEMPLATE_CACHE_TTL` (`0` = permanent). GraphQL: `TINA4_GRAPHQL_AUTO_SCHEMA`, `TINA4_GRAPHQL_ENDPOINT`. Mail: `TINA4_MAIL_IMAP_ENCRYPTION` (`tls`/`starttls`/`none`). MCP: `TINA4_MCP`, `TINA4_MCP_PORT`. Swagger: `TINA4_SWAGGER_ENABLED`, `TINA4_SWAGGER_CONTACT_EMAIL`, `TINA4_SWAGGER_LICENSE`. Database: `TINA4_DB_POOL` (env override on the existing `Database(url, pool=N)` constructor argument).

Logging - env-driven file output + rotation

Six new vars give you full control over logging without touching code:

Var	Default	What it does
`TINA4_LOG_FILE`	*(empty - stdout only)*	Path to a log file. Empty leaves you on stdout.
`TINA4_LOG_DIR`	`logs`	Directory for log files (joined with `_LOG_FILE` if relative).
`TINA4_LOG_FORMAT`	`text`	`text` or `json`. JSON mode emits one structured record per line.
`TINA4_LOG_OUTPUT`	`stdout`	`stdout`, `file`, or `both`. Strict - `stdout` means stdout only.
`TINA4_LOG_CRITICAL`	`false`	Enables a `Log.critical()` level above `error`. Off = no-op.

`TINA4_LOG_ROTATE_SIZE`	`10485760` (10 MB)	Rotate when the file exceeds this many bytes. `0` disables rotation.
`TINA4_LOG_ROTATE_KEEP`	`5`	Number of rotated files to retain (`app.log.1` ... `app.log.N`). Older ones are deleted.

Implementation uses each language's stdlib - Python's `logging.handlers.RotatingFileHandler`, Ruby's `Logger.new(path, shift_age, shift_size)`, and a roll-your-own atomic-rename pattern in PHP and Node. Zero new dependencies in any framework.

Documentation-truth CI gate now strict on both axes

The `audit-truth.py` script now blocks merges to `main` of `tina4-documentation` whenever a doc references a `tina4 <command>` or `TINA4_*` env var that doesn't exist in source. Previously CLI drift was strict; env drift was warn-only. Today both are strict.

Tests added

- Python: +53 tests in `tests/test_env_vars.py` (2395 -> 2448)
- PHP: +59 tests in `tests/EnvVarTest.php` (2172 -> 2231)
- Ruby: +51 examples in `spec/env_vars_spec.rb` (2696 -> 2747)
- Node: +59 tests in `test/envVars.test.ts` (3204 -> 3263)

Cross-framework total: 10,689 tests passing, +222 from 3.12.3.

Upgrade path

Drop in. No breaking changes - every new env var is opt-in with a sensible default. If you were setting any of the 17 deleted vars in your `.env`, the boot guard will warn (then ignore) - clean them out at your leisure.

v3.12.3 (2026-05-05)

Cross-framework parity sweep. Two minor breaking changes in the Ruby and PHP public API that bring all four frameworks onto the same shape.

Breaking changes (Ruby + PHP only)

Ruby Container - predicate now uses `?` suffix.

```
# before (3.12.2 and earlier)
Tina4::Container.has(:mailer)        # outdated

# after (3.12.3)
Tina4::Container.has?(:mailer)       # idiomatic Ruby predicate
```

This brings Ruby in line with Python (`has()`), PHP (`has()`), and Node (`has()`) while still respecting Ruby's `?`-suffix idiom for predicates returning bool. The pre-existing `resolve` -> `get` rename happened earlier; only the predicate was lagging.

ResponseCache public surface - middleware-only across all four frameworks.

The cache has always been middleware. Two of the four frameworks (PHP, Ruby) historically exposed lookup/store as public methods, which let users couple to internals. The public API is now consistent across all four: use the middleware on a route, and read stats with module-level helpers.

```
# Ruby - module-level helpers (parity with Python)
Tina4.cache_stats   # -> { hits:, misses:, size:, backend:, keys: }
Tina4.clear_cache   # flush all entries

# PHP - static methods on the class
\Tina4\Middleware\ResponseCache::cacheStats();
\Tina4\Middleware\ResponseCache::clearCache();
```

Internal methods that used to be public (`get`, `lookup`, `store`, `cache_response`) are now private. Tests that needed them retain access via `_internal*` test seams marked `@internal`.

Doc parity - CLAUDE.md and book chapter 33

- **CLAUDE.md**: every framework's "Key Method Stubs" section now covers the same surface area Python documents - Queue, QueryBuilder, Frond, Api, Background Tasks, ResponseCache, etc. PHP added 4 sections; Ruby added 5; Node added 13.
- **Book chapter 33**: env var tables are now grounded in source. Each framework's chapter 33 lists every `TINA4_*` var its source actually reads. Found and fixed several gaps - Ruby was missing `TINA4_CACHE_*`, `TINA4_QUEUE_*`, `TINA4_KAFKA_*`, `TINA4_RABBITMQ_*`, `TINA4_MONGO_*`, `TINA4_WS_BACKPLANE`, and the entire `TINA4_SESSION_VALKEY_*` block.

Other fixes

- **Ruby** `lib/tina4/ai.rb` - subprocess output is now force-encoded to UTF-8 before `String#strip`, fixing `Encoding::CompatibilityError` that crashed 4 ai specs on systems with non-ASCII pip output.
- **Node** `test/serverParity.test.ts` - sets `TINA4_OVERRIDE_CLIENT=true` so `start()` actually runs, plus emits the `N passed, M failed` summary line the runner expects. The test was effectively a no-op before; now it's recorded properly.

Genuine gaps surfaced by the parity audit (follow-up, not blocking 3.12.3)

The chapter 33 audit flagged env vars Python documents that no other framework actually reads - Ruby/PHP/Node lack `TINA4_OPEN_BROWSER`, `TINA4_DEV_POLL_INTERVAL`, `TINA4_PUBLIC_DIR`, `TINA4_TOKEN_EXPIRES_IN` alias, plus a few framework-specific gaps (Ruby has no Mongo session backend; Node `TINA4_CSRF` defaults to `false` vs Python's `true`). Tracked for a future patch.

Upgrade path

Symptom	Fix
Ruby: `NoMethodError: undefined method 'has' for Tina4::Container`	Replace `has(:key)` with `has?(:key)`
PHP: `BadMethodCallException` calling `$cache->lookup(...)`	Use the middleware: `[ResponseCache::class, 'beforeCache']` / `[..., 'afterCache']`. Or call `_internalLookup` if you really need direct access (test code only - `@internal`).
Ruby: `NoMethodError: undefined method 'get' for ResponseCache instance`	Use `Tina4.cache_stats` / `Tina4.clear_cache` for stats. Lookup goes through the middleware.

No `.env` changes from 3.12.2.

v3.12.2 (2026-05-05)

Quality-of-life patch. Two related portability fixes - no breaking changes from 3.12.1.

Firebird URL auto-detect

Firebird is the awkward one in the stack. Every other engine has a server-side database name (`postgres://host:port/dbname`), but Firebird wants either an absolute file path on the server, a Windows drive-letter path, or an alias. The classic URI form needs a double slash to keep the leading `/` of an absolute path through the URL parser - unintuitive to anyone used to the way postgres / mysql / mssql encode the database name.

The framework now accepts five equivalent forms and normalises all of them transparently:

URL path you write	Resolved Firebird identifier
`//abs/path/db.fdb` (classic double-slash)	`/abs/path/db.fdb`
`/abs/path/db.fdb` (single-slash, intuitive)	`/abs/path/db.fdb`
`/C:/Data/db.fdb` (Windows drive letter)	`C:/Data/db.fdb`
`/C%3A/Data/db.fdb` (URL-encoded colon)	`C:/Data/db.fdb`
`/employee` (Firebird alias)	`employee`

For ops setups that keep server URL and DB location in separate config layers - or for Windows backslash paths that fight URL encoding - set `TINA4_DATABASE_FIREBIRD_PATH`. The env override wins over whatever path is in the URL.

```
TINA4_DATABASE_FIREBIRD_PATH=C:\firebird\data\app.fdb
TINA4_DATABASE_URL=firebird://SYSDBA:masterkey@localhost:3050/ignored
```

Shipped to all 4 frameworks. 11 regression tests per framework (8 unit + 3 live).

Bug fix specific to PHP - `mysqli` localhost+port quirk

PHP's `mysqli` has a long-standing quirk where `host == "localhost"` triggers a Unix socket lookup and IGNORES the port argument entirely. Connecting to `mysql://...:53306` against a Docker container fails with "No such file or directory" - `mysqli` is hunting for `/tmp/mysql.sock` instead of opening a TCP connection. `MySQLAdapter::rewriteHostForTcp()` now rewrites `localhost` to `127.0.0.1` when a non-zero port is specified, forcing the TCP code path. Bare `mysql:///db` (no port) is preserved so existing socket-based setups keep working.

Other fixes

- **chore(python):** `pyproject.toml` had drifted to `3.10.41` while `__init__.py` read `3.12.1`. Synced both to 3.12.2 so `uv build` and runtime introspection now agree.
- **chore(claude.md, all 4):** stale framework version banners in `CLAUDE.md` headers updated.

No `.env` changes from 3.12.1, no migration needed. Existing 3.12.1 installs upgrade by changing one version number.

v3.12.1 (2026-05-04)

CI-only patch - no framework code changes from 3.12.0.

- **fix(ci, all 4):** every `publish.yml` workflow now declares `permissions: contents: write` on the publish job. Without this, `softprops/action-gh-release` 403'd against the default `GITHUB_TOKEN` on repos whose default Workflow permissions setting was read-only (Ruby and Node hit this every release; PHP and Python worked by luck of repo settings). The explicit declaration makes the workflow self-sufficient.
- **chore(ci):** bumped `softprops/action-gh-release` from `@v1` (unmaintained) to `@v2`.

No `.env` changes, no API changes, no migration needed. Existing 3.12.0 installs can upgrade without touching anything else.

The version-bump itself is the test: a successful 3.12.1 release proves the workflow fix works on Ruby and Node where 3.12.0 needed manual `gh release create`.

v3.12.0 (2026-05-04)

> ***Warning: Breaking change - read before upgrading.*** *Every framework env var now uses the `TINA4_` prefix. Existing `.env` files set with `DATABASE_URL`, `SECRET`, `SMTP_HOST`, `HOST_NAME`, etc. will cause the framework to refuse to boot. Run `tina4 env --migrate` to rewrite, or follow the rename table below.*

Why this release

Tina4's env vars had grown inconsistent. Some had the `TINA4_` prefix (`TINA4_DEBUG`, `TINA4_LOCALE`, `TINA4_CACHE_BACKEND`), others didn't (`DATABASE_URL`, `SECRET`, `SMTP_HOST`). Newcomers had to guess which convention applied to which feature. Existing tools and PaaS dashboards collided with un-prefixed names like `SECRET` and `API_KEY` that other libraries also read. Documentation drifted - 91 env-var names appeared in the docs that didn't exist in any framework, and 22 framework-specific env vars in the code didn't match the names users were told to set.

This release closes all three gaps with a single hard rename. No deprecation period, no fallback chain. The framework refuses to boot if it detects a legacy name in the environment, prints a list of every var to rename, and tells you which command to run.

What changed

• **22 env vars renamed** to `TINA4_*` form. See the migration table below.

• `tina4 env --migrate` **CLI** added to all four frameworks. Reads your `.env`, rewrites it in place, leaves a `.env.bak` backup, prints a diff. Idempotent.

• **Boot-time guard** scans `os.environ` (or the language equivalent) for the 22 legacy names. If any are present, prints the rename map and exits with code 2. Bypass with `TINA4_ALLOW_LEGACY_ENV=true` for migration scripts that need both names set during transition.

• **All 4 framework books rewritten.** Chapter 33 (Environment Variables) is now a clean canonical list - every var prefixed, descriptions current, legacy names removed.

• **Doc-vs-code drift closed.** Of the 91 stale env vars previously documented, 61 were renames (corrected), 32 were never implemented (removed). The `audit-links.py` CI gate stays at 0 broken links / 0 broken anchors.

• **Frond bundle** rebuilt at v2.1.3 - `frond.min.js` footer now shows the version explicitly so users can verify what they have.

Bug fixes shipped alongside the rename

• **#38 PostgreSQL UUID-PK transaction abort** - the post-INSERT `lastval()` probe is now wrapped in a SAVEPOINT, so UUID-PK INSERTs no longer poison the outer transaction with `InFailedSqlTransaction`. Live regression test against PostgreSQL 16. (Affects all 4 frameworks where the PG adapter does this probe.)

• **#39 Landing page + template auto-routing** -

• Auto-routing now scans `src/templates/pages/` only. Partials, layouts, base.twig, errors/, components/, and `_*` files never auto-serve from a URL.

• `TINA4_TEMPLATE_ROUTING=off` kills the feature entirely.

• `src/public/index.html` auto-serves at `/` (and `/foo/` serves `src/public/foo/index.html`) - SPA hosting Just Works.

• The framework landing page only renders when `TINA4_DEBUG=true`. Production never shows it; framework version, dev-admin link, and gallery don't leak to real users.

• The malformed `HTTP/1.1 404 OK` status line is fixed - every status code now uses its canonical RFC 7231/9110 reason phrase.

• **#37 frond.form.submit redirect handling** - verified shipped at v2.1.x; `xhr.responseURL` change triggers `window.location` navigation correctly.

• **#36 Session file handler** - re-verified safeguards (lazy save, WebSocket skip, probabilistic GC, new-and-empty skip) all still in place.

Migration - every renamed var

Legacy name	New name
`DATABASE_URL`	`TINA4_DATABASE_URL`
`DATABASE_USERNAME`	`TINA4_DATABASE_USERNAME`
`DATABASE_PASSWORD`	`TINA4_DATABASE_PASSWORD`
`DB_URL`	`TINA4_DATABASE_URL` (alias dropped)
`SECRET`	`TINA4_SECRET`
`API_KEY`	`TINA4_API_KEY`
`JWT_ALGORITHM`	`TINA4_JWT_ALGORITHM`
`SMTP_HOST`	`TINA4_MAIL_HOST`
`SMTP_PORT`	`TINA4_MAIL_PORT`
`SMTP_USERNAME`	`TINA4_MAIL_USERNAME`
`SMTP_PASSWORD`	`TINA4_MAIL_PASSWORD`
`SMTP_FROM`	`TINA4_MAIL_FROM`
`SMTP_FROM_NAME`	`TINA4_MAIL_FROM_NAME`
`IMAP_HOST`	`TINA4_MAIL_IMAP_HOST`
`IMAP_PORT`	`TINA4_MAIL_IMAP_PORT`
`IMAP_USER`	`TINA4_MAIL_IMAP_USERNAME`
`IMAP_PASS`	`TINA4_MAIL_IMAP_PASSWORD`
`HOST_NAME`	`TINA4_HOST_NAME`
`SWAGGER_TITLE`	`TINA4_SWAGGER_TITLE`
`SWAGGER_DESCRIPTION`	`TINA4_SWAGGER_DESCRIPTION`
`SWAGGER_VERSION`	`TINA4_SWAGGER_VERSION`
`ORM_PLURAL_TABLE_NAMES`	`TINA4_ORM_PLURAL_TABLE_NAMES`

Names that stay un-prefixed (not framework config)

`PORT`, `HOST`, `NODE_ENV`, `RACK_ENV`, `RUBY_ENV`, `ENVIRONMENT` - these are runtime / PaaS conventions, not framework config. Heroku, Railway, Vercel, and friends set them; we keep reading them.

How to upgrade

- **Backup your `.env`:** `cp .env .env.bak.pre-v3.12`
- **Run the migration:** `tina4 env --migrate` - rewrites your `.env` in place.
- **Update PaaS dashboards:** Heroku, Railway, Vercel, Render, Fly.io etc - rename the same vars in your provider's env-var UI.
- **Restart your app.** The boot guard verifies nothing legacy remains.

If your app uses `SECRET`, `DATABASE_URL`, or any other listed name in places besides `.env` (e.g. your CI pipeline's `env:` blocks), update those too - the boot guard checks `os.environ`, not just `.env`.

Parity

All 4 frameworks aligned at **3.12.0**:

- tina4-python 3.11.32 -> 3.12.0
- tina4-php 3.11.32 -> 3.12.0
- tina4-ruby 3.11.32 -> 3.12.0
- tina4-nodejs 3.11.32 -> 3.12.0

Coordinated release across PyPI, Packagist, RubyGems, npm.

v3.11.32 (2026-04-25)

Critical fix - pool + transactions are now actually atomic. Plus a coordinated parity release that aligns all four frameworks at the same version after months of drift.

Before this release, creating a `Database` with `pool > 0` silently broke transactions. The pool's round-robin checkout rotated to a different adapter on every call - so `start_transaction()` pinned its flag on adapter A, the executes autocommitted on adapters B and C, and the final `commit()` / `rollback()` landed on adapter D, which had nothing to commit. Result: `rollback()` was a no-op, writes leaked through, and no error or log surfaced the problem.

The fix pins one adapter to the calling context for the lifetime of a transaction. Each language uses its own primitive:

- **Python** - `threading.local()` on the `Database` instance
- **Ruby** - `Thread.current[:tina4_pinned_adapter_<obj_id>]`
- **Node.js** - `AsyncLocalStorage` from `node:async_hooks` (async-safe across overlapping awaits)
- **PHP** - per-instance property (PHP-FPM is one process per request; threading.local is unnecessary)

While pinned, every database call routes to the same adapter. `commit()` and `rollback()` release the pin so subsequent calls round-robin again.

- **fix (database / all 4):** adapter pinning across transaction scope in `Database._get_adapter()` (and language equivalents). Every backend is affected - SQLite, PostgreSQL, MySQL, MSSQL, Firebird. Firebird exposed it loudest because of its honest "commit-empty-txn is a real no-op" semantics; the others mostly hid the bug behind eager autocommits but still lost rollback atomicity.

- **tests (all 4):** new regression suite - three INSERTs followed by `rollback()` under `pool=4` now leaves zero rows (was leaking three). Three INSERTs followed by `commit()` persists exactly three. Pin-release after commit/rollback verified. `pool=0` regression test added so single-connection mode stays unaffected.

- **parity / version alignment:** all 4 frameworks bumped to 3.11.32 - closes the cross-framework version drift that had built up (PHP at 3.11.31, Python at 3.11.24, Ruby and Node at 3.11.19). A single coordinated release across all four registries: PyPI, Packagist, RubyGems, npm.

No migration needed. Code using `pool=0` (the default for every adapter except where explicitly raised) is unaffected. Code using `pool>0` will now actually honour transactions instead of silently dropping them.

If you've been seeing intermittent "writes vanished" or "rollback didn't help" reports on a pooled `Database`, this release is the cause and the cure.

v3.11.13 (2026-04-16)

Issue-driven release. Everything reported in the open tina4-book issues either was fixed in this version or is already fixed in 3.11.12; this release consolidates the remaining bits and corrects documentation drift.

- **feat (router / all 4):** Explicit typed-parameter system shared across Python, PHP, Ruby, Node. Adds `alpha`, `alnum`, `slug`, `uuid`, and explicit `string` types in addition to the existing `int`/`integer`, `float`/`number`, `path`/`.*`. **Unknown type names now throw at registration** - `{name:str}`, `{id:inetger}`, etc. raise with a clear message listing the valid types instead of silently falling through to the default matcher. Fixes tina4-book#125. +45 new tests across the four suites.

- **fix (gallery / python+php+ruby):** Gallery Try-It / View buttons now open the deployed example in a new tab (`window.open(url, '_blank')`) instead of navigating away from the gallery home. Fixes tina4-book#115.

- **fix (ruby gemspec):** `sqlite3` promoted from `add_development_dependency` to `add_dependency`. Matches the "zero-config SQLite on first run" promise. Fixes tina4-book#100.

- **docs (tina4-book):** PHP Chapter 2 updated - correct port (7145), `->noAuth()` on write-method examples, and an explicit callout explaining the secure-by-default policy for POST/PUT/PATCH/DELETE. Addresses tina4-book#87, #94, #123.

- **docs (tina4-book):** Python `@template` decorator ordering corrected (must sit BELOW the route decorator) in book chapters 04 and 10; Python `request->query` vs

`request->params` distinction in PHP chapter 1.

• **tests (python):** Session-handler tests updated to reflect the real default TTL of 3600s (were stale at 1800s).

• **verified already fixed in earlier 3.11.x releases** - closed comments posted on all of these:

• #79 dotted numeric index (`{{ items.0.name }}`)

• #80 `truncate` filter

• #82 `{{ parent() }}` / `{{ super() }}` across all 4 frameworks

• #83 Ruby dashboard - WEBrick is runtime dep

• #89 `load_dotenv` rename, `DatabaseResult` methods, SQLite WAL locking

• #91 Ruby `request.params` symbol + string keys via `IndifferentHash`

• #93 Ruby `/docs/*` and bare `/*` wildcard routes

• #97 Frond ternary operator

• **parity:** All 4 frameworks bumped to 3.11.13.

v3.11.12 (2026-04-16)

Breaking: `sqlite:///X` URLs are now relative to the project root (cwd), matching the documented convention. For absolute paths use four slashes (`sqlite:////abs/path.db`) or a Windows drive letter (`sqlite:///C:/Users/app.db`).

Before this release, `TINA4_DATABASE_URL=sqlite:///data/app.db` was interpreted differently by every framework. Python/Node/Ruby tried to open `/data/app.db` (absolute) which crashed on macOS with `OSError: [Errno 30] Read-only file system: '/data'`. PHP did the same under the hood. All four frameworks now agree: three slashes = relative, four slashes = absolute.

• **fix (all 4):** `sqlite:///X` resolves under cwd; parent directory auto-created only when inside cwd. Absolute paths are trusted and never mkdir'd at root.

• **fix (python):** `_ensure_folders` no longer creates a bogus `src/migrations/` directory. The migration runner always looks at `migrations/` at the project root - there is only one correct location.

• **parity (php, ruby, node):** Same `sqlite:///X` parsing as Python. Dedicated `resolve_path` / `resolveSqlitePath` helpers in each framework so adapters consistently handle `:memory:`, `./` forms, Windows drive letters.

• **tests:** 9 new Python tests in `TestSQLiteConnectionPath` + `TestProjectFolders`. 4 new PHP tests in `DatabaseUrlTest` covering relative/absolute/Windows/bruce-regression. 6 new Ruby specs in `database_drivers_spec.rb :: SqliteDriver.resolve_path`. Node URL tests expanded in `database.test.ts` with the full relative/absolute/Windows/:memory: matrix.

• **parity:** All 4 frameworks bumped to 3.11.12.

Migration note: If your `.env` has `TINA4_DATABASE_URL=sqlite:///data/app.db`, it will now create `./data/app.db` in the project root (which is what most users actually want). If you

genuinely want an absolute path, change to `sqlite:////data/app.db` (four slashes).

v3.11.11 (2026-04-16)

• **fix (python ORM):** `Field.validate` no longer re-coerces values that are already the correct type. Previously, any PostgreSQL/MSSQL read of a row containing a `DateTimeField` crashed because `datetime(datetime_instance)` raises `TypeError`. The fix accepts native driver types (`datetime`, `bytes`, `int`, `bool`, `float`, `str`) without re-wrapping, and parses ISO-8601 strings into `datetime` for SQLite. See `tina4-python/plan/orm-field-validate-native-types.md`.

• **fix (python ORM):** `BooleanField` vs `IntegerField` ordering handled explicitly. `BooleanField(1)` still coerces to `True`, `IntegerField(True)` still coerces to `1`; no regression for either direction (bool is a subclass of int in Python).

• **tests (python):** 10 new `TestFieldsNativeTypes` cases covering datetime/int/bool/float/bytes/string/ForeignKey round-trips.

• **tests (parity):** Regression-guard "datetime round-trip on read path" tests added to PHP (`ORMV3Test`), Ruby (`orm_spec`) and Node.js (`orm.test.ts`) so an equivalent bug can't creep in there later.

• **parity:** All 4 frameworks bumped to 3.11.11.

v3.11.10 (2026-04-15)

• **fix (php):** Hot-reload loop - DevAdmin's polling fallback used `mt=0` as the baseline, so the first poll after every page load triggered `location.reload()`, which reset `mt=0` again. Loop now initialises the baseline on the first poll.

• **fix (php):** Reload sentinel removed - PHP was the only framework recursively walking `src/` and touching `src/.reload_sentinel` on every reload POST. The sentinel lived inside the Rust CLI's watched tree and fed back into the watcher, triggering a second loop. Replaced with the same in-memory counter used by Python/Ruby/Node.

• **fix (php):** Polling no longer starts more than once when the WebSocket reconnect retry budget is exhausted (added a `pollStarted` guard).

• **feat (parity):** `GET /__dev/api/queue/topics` and `GET /__dev/api/queue/dead-letters` added to PHP, Ruby and Node (previously only in Python). PHP queue endpoints now read from the real `Tina4\Queue` backend instead of returning stubs.

• **feat (devadmin):** Refreshed `tina4-dev-admin.js` bundle (87.8 KB) across all 4 frameworks - adds the topic selector dropdown, inline payload expand/copy, and corrected version display.

• **tests:** 4-way parity tests for hot-reload: mtime starts at 0, POST /__dev/api/reload bumps the counter, no sentinel file is written to disk, mtime is monotonic across successive reloads. Mirrored in `tina4-php/tests/DevAdminTest.php`, `tina4-python/tests/test_dev_admin.py`, `tina4-ruby/spec/dev_admin_spec.rb`, `tina4-nodejs/test/devAdmin.test.ts`.

• **parity:** All 4 frameworks bumped to 3.11.10.

v3.11.9 (2026-04-15)

Catch-up release covering v3.11.0 -> v3.11.9 across all 4 frameworks.

• **feat (websocket):** Full WebSocket parity across Python/PHP/Node/Ruby - `get_client_rooms()` / `getClientRooms()`, `route()` usable as decorator or direct handler registration, matching room/broadcast semantics, plus new parity tests on all 4.

• **feat (graphql):** Input validation and field-level `@auth` directives with context threading.

• **feat (graphql):** Auto-discovery of schemas; removed legacy DevAdmin HTML/JS in favour of the new UI.

• **feat (devadmin - Python):** Queue tab with topic selector, dead-letter listing and replay endpoints, inline payload expand/copy, version display.

• **feat (cli):** Rust CLI now owns file watching - frameworks receive `POST /__dev/api/reload` and internal watchers are disabled when launched by the Rust CLI (`--managed`).

• **fix (cli):** `parseFlags` / `parse_flags` / `parseCliArgs` no longer swallow `host:port` or positional args after boolean flags.

• **fix (scss):** SCSS recompilation loop fixed; output path corrected to `src/public/css/` to match CLI and static serving.

• **fix (frond - Python):** Numeric dotted index for lists (`items.0.name`) now resolves correctly.

• **fix (router - Ruby):** Bare `/*` wildcard capture exposed under `"*"` key for parity.

• **fix (orm - PHP):** Three data-sync bugs fixed: `load()` double-fill, `getPrimaryKeyValue`, `save()` ID sync.

• **fix (graphql):** `from_orm` / `fromOrm` list resolver used `select(skip=)` instead of `all(offset=)`.

• **fix (metrics):** Windows backslash paths normalised to forward slashes.

• **fix (app - PHP):** No longer crashes on notices/deprecations in loaded files; `run()` now prints the banner when starting the server directly.

• **chore:** Example demo store ships with the repo; Windows-friendly setup; `.env.example` and setup scripts added.

• **parity:** All 4 frameworks bumped to 3.11.9. PHP aligned to the 3.x tag scheme on `v3`.

v3.10.99 (2026-04-12)

• **breaking:** `autoMap` now defaults to `true` - ORM models automatically map between camelCase properties and snake_case DB columns. Set `public bool $autoMap = false;` on your model to restore the old behaviour.

• **breaking:** `all()` now returns a flat array of model instances instead of `['data' => [...], 'total' => N, ...]`. Use `count()` separately if you need the total.

• **feat:** `toDict(include, case)` parameter - pass `case: 'snake'` to get snake_case keys matching DB columns, or `case: 'camel'` (default) for camelCase.

• **feat:** Frond `replace` filter now accepts dict args - `{{ v|replace({"T": " ", "-": "/"}) }}` for multiple substitutions in one call.

• **feat:** `$app->background(callback, interval)` - register periodic tasks that run cooperatively in the `stream_select` event loop. No threads, no separate processes.

• **feat:** Background timing guard - warns when callbacks exceed their interval, helping developers identify blocking operations.

• **feat:** WebSocket room management moved to `Server` class - `joinRoom()`, `leaveRoom()`, `broadcastToRoom()` now work reliably via `WebSocketConnection->server`.

• **feat:** Docker image now bundles the example store demo - `docker run tina4stack/tina4-php:v3` starts a working app out of the box.

• **fix:** AutoCrud updated for new `all()` return format.

• **fix:** Cart nav badge now updates reactively on quantity change and item removal.

• **fix:** Non-blocking queue consumer - `processOrders()` uses `$queue->pop()` instead of blocking `$queue->consume()`.

• **tests:** 6 new parity tests covering `toDict(case:)`, `autoMap` default, `replace` filter (dict + positional), and `background()` registration. 2,345 tests passing.

• **parity:** All features shipped identically across Python, PHP, Ruby, Node.js.

v3.10.97 (2026-04-11)

• **fix:** frond.form.submit redirect handling - XHR follows 3xx redirects transparently; fixed by detecting `xhr.responseURL` mismatch and navigating instead.

• **dep:** Updated frond.min.js to v2.1.2.

• **parity:** All 4 frameworks bumped to 3.10.97.

v3.10.93 (2026-04-11)

• **fix:** Frond bracket depth tracking in `findOutsideQuotes()` and `splitOutsideQuotes()` - expressions like `$arr[$i % 2]` no longer treated as top-level arithmetic.

• **fix:** Frond subscript expression evaluation - bracket content uses `evaluateExpression()` instead of `resolveVariable()`, enabling `arr[loop.index0 % 2]`.

• **fix:** Frond slice with variable bounds - `items[start:end]` evaluates bounds through `evaluateExpression()`.

• **docs:** Developer skills updated - Metrics Dashboard guidance, Frond Template Parity rules, `@noauth` security warnings.

• **parity:** All Frond fixes applied identically across Python, PHP, Ruby, Node.js. 2,339 tests passing (263 Frond).

v3.10.92 (2026-04-10)

• **feat:** Add `RateLimiterMiddleware` class with `beforeRateLimit()`, `check()`, `reset()` static methods.

• **breaking:** Rename `ErrorOverlay` methods - `render()` -> `renderErrorOverlay()`, `renderProduction()` -> `renderProductionError()`.

• **feat:** Add `Server::handle(Request $request): Response` for cross-framework parity.

• **feat:** Add `DatabaseResult::size()` method.

• **breaking:** Rename `WebSocketBackplane::create()` -> `WebSocketBackplane::createBackplane()`.

• **feat:** Add `DevAdmin::health()` method.

• **feat:** Add `ScssCompiler::compileScss()` method.

• **fix:** Add `DatabaseSessionHandler::delete()` delegating to `destroy()`.

• **fix:** `SmokeTest` - pass secret explicitly to `Auth::getToken()` to fix test ordering issue.

• **parity:** 44/44 cross-framework features green. 2,305 tests passing.

v3.10.91 (2026-04-10)

• **feat:** Add parity methods - `GraphQLType::parse()`, `Response::send()` params, `MCP::registerRoutes()` optional router.

• **breaking:** Rename `from()` -> `fromTable()`, `template()` -> `render()` - align with Python canonical names.

v3.10.90 (2026-04-09)

• **docs:** Chapter 4 (Templates) - new "Dumping Values for Debugging" section covering both `{{ $x|dump }}` and `{{ dump($x) }}` forms, their shared `<pre>var_dump()</pre>` output, and the `TINA4_DEBUG=true` production gate. Filter table entry updated to reference the new section.

• **docs:** `plan/parity/parity-template.md` updated with a cross-framework dump helper comparison table and marks dump parity as confirmed across all 4 frameworks at v3.10.89.

• **chore:** Version sync release - brings all 4 frameworks to the same patch version (3.10.90) so downstream users can upgrade PHP/Python/Ruby/Node.js in lockstep without hunting version mismatches.

v3.10.89 (2026-04-09)

• **feat:** `{{ dump(value) }}` global function form added to Frond alongside the existing `{{ value|dump }}` filter. Both call a single `Frond::renderDump()` helper and produce identical output (`<pre>var_dump()</pre>`).

• **security:** Dump is now **gated on** `TINA4_DEBUG=true`. In production (env var unset or `false`) both the filter and function silently return an empty string. This prevents accidental

leaks of internal state, object shapes, and sensitive values into rendered HTML when a developer leaves a `{{ dump($x) }}` call in a template.

• **test:** 4 new tests in `FrondTest.php` covering debug-mode output, production silencing, function/filter parity, and function-form production silencing.

v3.10.87 (2026-04-09)

• **fix:** Dev toolbar no longer vanishes after a hot-reload. `Server::onFilesChanged()` used to call `Router::clear()` and then loop `include_once` over every `.php` file in `src/routes/`. Because `include_once` is a no-op for already-included files, routes were never re-registered after a template/CSS/JS edit - subsequent requests fell through to the 404 handler and the dev toolbar injection was lost. The router is now left intact on template/asset edits (Frond re-reads templates in dev mode, static files are served from disk per request, so nothing else needs to move). PHP file edits log a warning that a full server restart is required (classes cannot be redeclared in-process).

• **fix:** This also resolves a related issue where rapid browser refreshes during hot reload would return 500s - the router wipe left a brief window with zero routes registered.

v3.10.86 (2026-04-09)

• **feat:** `$foreignKeys` property on `ORM` auto-wires both sides of a foreign key relationship. Declaring `public array $foreignKeys = ['user_id' => 'User']` injects a `belongsTo` accessor (`$post->user`) on the declaring model and a `hasMany` accessor (`$user->posts`) on the referenced model via a cross-model FK registry. Extended form supports a custom has-many key: `['user_id' => ['model' => 'User', 'related_name' => 'blog_posts']]`.

• **feat:** Cross-framework parity - same FK auto-wiring semantics now available in Python (`ForeignKeyField`), Ruby (`foreign_key_field`), and Node.js (`type: "foreignKey"`)

• **docs:** Chapter 6 (ORM) updated with a new "$foreignKeys - Auto-Wired Relationships" section

v3.10.85 (2026-04-09)

• **fix:** Removed duplicate `Job` class from `Queue.php` - canonical definition is `Job.php` only

• **fix:** `Job.php::fail()` now delegates to `writeFailed()` instead of calling private `getBasePath()` directly

v3.10.84 (2026-04-09)

• **fix:** Router/middleware was setting `request.user` / `request.auth` / auth payload to `true` (boolean) instead of the actual JWT payload after `validToken()` was changed to return bool - any code reading `request.user["sub"]` etc. would have failed silently or crashed

• **fix:** CSRF middleware was not correctly rejecting invalid tokens (nil check on bool result always passed)

• **fix:** `toObject()` declared wrong return type (`array` vs actual `object`)

- **fix:** Router `request.user` and gallery auth verify endpoint updated for bool `validToken`
- **add:** Headless routing auth payload integration tests to prevent regression

v3.10.83 (2026-04-08)

- **fix:** CORS headers now set before auth short-circuit (#106)
- **fix:** ORM find/all/where no longer crash with DatabaseResult object (#108)
- **fix:** toObject() returns stdClass, not array (#107)
- **fix:** Firebird absolute path no longer strips leading slash (#101)
- **feat:** WebSocket rooms - joinRoom, leaveRoom, broadcastToRoom, getRoomConnections, roomCount
- **feat:** queue signature parity - instance-scoped, no topic params on public methods
- **feat:** auth alias cleanup - removed createToken/validateToken aliases

v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framework handles chunked transfer encoding, keep-alive, and `text/event-stream` content type
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

Version History

Tina4 PHP follows semantic versioning. The major number changes when something breaks. The minor number changes when something new arrives. The patch number changes when something gets fixed. Each release is available on Packagist.

This chapter covers the full v3 line -- from the first release candidate through the current stable release. If you are upgrading from v2, read Chapter 37 first. It covers every breaking change and gives you a migration checklist.

---

v3.10.68 (2026-04-03) - Full Parity Release

- **100% API parity** across Python, PHP, Ruby, Node.js - 30+ issues fixed
- **ORM:** save() returns self/false, arrays not tuples, toDict/toAssoc, scope registers method, where()/all() on Node, count() on PHP
- **Auth:** expires*in minutes, PBKDF2 260k, env TINA4*SECRET fallback, API key fallback
- **Session:** dual-mode flash(), get_flash, cookieHeader, getSessionId
- **Database:** execute() bool/DatabaseResult, get*last*id/get_error, getColumns, cacheStats

- **Request/Response:** files dict, query, cookies, contentType, xml(), callable
- **Queue:** consume() poll_interval
- **WebSocket:** event naming, connection properties
- **GraphQL:** schema_sdl() + introspect() on all 4
- **Events:** emitAsync() on all 4
- **i18n:** zero-dep YAML support

v3.10.67 (2026-04-03)

- **load() returns bool** - `$model->load($sql, $params)` calls selectOne internally, populates the instance, returns `true`/`false`. Use `findById()` for PK lookups
- **api.upload()** added to tina4-js - sends FormData with Bearer token auth for multipart file uploads
- **ORM CLAUDE.md rewrite** - all method stubs now match actual API signatures
- **File upload docs** - `$request->files` format documented in CLAUDE.md

v3.10.66 (2026-04-03)

- **Metrics file detail fix** - clicking bubbles in framework scanning mode now resolves paths correctly via scan root tracking

v3.10.65 (2026-04-03)

- **Metrics 3-stage test detection** - filename, path, and content matching
- **Metrics framework mode** - scans framework source with correct relative paths
- **tina4 console** - interactive REPL with framework loaded
- **tina4 env** - interactive environment configuration
- **Brand** - "TINA4 - The Intelligent Native Application 4ramework"
- **Quick references** - 36 sections, DotEnv API documented
- **37 chapters** - 7 new (Events, Localization, Logging, API Client, WSDL/SOAP, DI Container, Service Runner)
- **MongoDB + ODBC adapters** across all 4 frameworks
- **Pagination standardized** - limit/offset primary, merged dual-key response
- **Port kill-and-take-over** on startup

---

v3.10.60 (2026-04-03)

- **tina4 console** - interactive PHP REPL with framework loaded ($db, $app, Router, Auth)
- **tina4 env** - interactive environment configuration

- **Brand update** - "TINA4 - The Intelligent Native Application 4ramework"
- **Dynamic version** - reads from composer metadata at runtime, no hardcoded constant
- **Packagist v2 API** - version checker uses repo.packagist.org
- **@noauth docblock** - annotations now affect dispatch (#114)
- **Port kill-and-take-over** - default port always reclaimed
- **MongoDB adapter** (ext-mongodb), **ODBC adapter** (pdo_odbc)
- **Pagination standardized** - limit/offset primary, merged dual-key response
- **#101** Firebird paths, **#102** autoMap uppercase, **#104** TINA4*DATABASE*URL, **#105** CORS fix

---

v3.10.57 (2026-04-02)

- **MongoDB adapter** - `Database("mongodb://host:port/db")`, requires ext-mongodb
- **ODBC adapter** - `Database("odbc:///DSN=MyDSN")` via pdo_odbc
- **Pagination standardized** - limit/offset primary, merged dual-key toPaginate() response
- **Test port at +1000** - user testing port (e.g. 8146) stable, no hot-reload
- **Dynamic version** - read from composer metadata, no hardcoded constant
- **Packagist v2 API** - version checker uses repo.packagist.org/p2/
- **#101** FirebirdAdapter path parsing preserves absolute paths
- **#102** ORM snakeToCamel handles uppercase columns
- **#104** ORM ensureDb() auto-discovers TINA4*DATABASE*URL
- **#105** CorsMiddleware matches request origin correctly
- **#114** @noauth docblock annotations now affect dispatch
- **108 features at 100% parity**, 2,220 tests

---

v3.10.54 (2026-04-02)

- **Auto AI dev port** - second socket on port+1 with no-reload when TINA4_DEBUG=true
- **TINA4*NO*RELOAD** env var + --no-reload CLI flag
- **#101** FirebirdAdapter path parsing preserves absolute paths
- **#102** ORM snakeToCamel handles uppercase columns (Firebird/Oracle)
- **#104** ORM ensureDb() auto-discovers TINA4*DATABASE*URL
- **#105** CorsMiddleware matches request origin correctly
- **SQLite commit()** no-op without transaction
- **Gallery fixes** - SQLite paths, auth bypass

- **QueryBuilder docs** - added to ORM chapter

---

v3.10.48 - April 2, 2026

Bug Fixes

FrankenPHP requires `--production` flag - FrankenPHP no longer auto-detected when debug is off. Use `tina4php serve --production` to enable it. Gallery tests (19) and live reload tests (36) added. Fixed `DotEnv::load()` -> `DotEnv::loadEnv()` in Server.php.

---

v3.10.46 - April 1, 2026

Test Coverage

Massive test expansion - 605 new tests added across session handlers, queue backends, database drivers, Frond template engine, dev admin, ORM, auth, seeder, log, service runner, container, CORS, form token, HTML element, migration, i18n, events, SCSS, CRUD, rate limiter, and CSRF middleware. PHP now at 1,937 tests with full parity across all 49 core areas.

Bug Fixes

CSRF query param check - Fixed `$request->params` shadowing `$request->query` in the CSRF middleware, so query string token detection now works correctly.

---

v3.10.45 - April 1, 2026

Bug Fixes

CLI serve hijack - When `index.php` calls `App::run()`, the CLI `serve` command now sets a `TINA4_CLI_SERVE` constant so `run()` returns early, letting the CLI manage the server lifecycle (port, debug mode, browser open).

---

v3.10.44 - April 1, 2026

New Features

Database tab redesign - The dev admin Database panel now uses a split-screen layout. Tables are listed on the left as a navigation sidebar with click-to-select highlighting. The query editor, toolbar, and results occupy the right panel.

Copy CSV / Copy JSON - Two new buttons in the database toolbar copy query results to the clipboard in CSV or JSON format.

Paste data - A Paste button opens a modal for pasting JSON arrays or CSV/tab-separated data. Auto-detects the format and generates INSERT statements. Prompts for a table name if none is

selected, and generates CREATE TABLE for new tables. SQL input passes through unchanged.

Multi-statement execution - The query runner handles multiple SQL statements separated by semicolons, running them in a single transaction with automatic rollback on error.

Database badge on load - The Database tab count badge shows the table count immediately on page load.

Star wiggle animation - The GitHub star button on the landing page uses an empty star (*) with a wiggle animation: 3-second delay, then wiggles at random 3-18 second intervals.

Bug Fixes

Default port - PHP default port confirmed as 7145 (PHP=7145, Python=7146, Ruby=7147, Node=7148).

SQLite LIMIT fix - Prevents double-LIMIT errors when browsing tables in the dev admin.

browseTable quote escaping - Fixed broken onclick handlers for table names using addEventListener.

Frond template engine - Fixed string concatenation (`~` operator) and inline if/else expressions (`{{ 'yes' if active else 'no' }}`). A greedy quoted-string fallback in `evaluateLiteral()` was treating compound expressions as single string literals.

Test Coverage

Major test expansion - 200 new tests added (FakeData 42, Cache 30, DevMailbox 33, Static files 31, Metrics 20, CLI scaffolding 31, plus v3.10.44 feature tests). 1,532 tests passing, 0 failures.

---

v3.10.40 - April 1, 2026

Bug Fixes

Dev overlay version check - Fixed misleading "You are up to date" message when running a version ahead of what's published on Packagist. The overlay now shows a purple "ahead of Packagist" message. Also added a breaking changes warning (red banner with changelog link) when a major or minor version update is available.

---

v3.10.39 - April 1, 2026

Breaking Changes

`Auth::hashPassword()` **- separator changed from** `:` **to** `$`

The password hash format now uses `$` as a separator (matching Python, Ruby, and Node.js):

```
# BEFORE: pbkdf2_sha256:100000:salt:hash

## v3.10.70 (2026-04-06)
```

```
- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

# AFTER:  pbkdf2_sha256$100000$salt$hash

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js
```

Existing hashed passwords stored in your database **will not verify** after upgrade. Rehash passwords on next user login:

```
if (!Auth::checkPassword($password, $storedHash)) {
    // try old format first, then rehash
}
```

`Database::update()` **and** `Database::delete()` **filter signature changed**

```
// BEFORE (v3.10.38 and earlier) - associative array filter
$db->update('users', ['name' => 'Alice'], ['id' => 1]);
$db->delete('users', ['id' => 1]);

// AFTER (v3.10.39+) - SQL string + params (matches Python, Ruby, Node.js)
$db->update('users', ['name' => 'Alice'], 'id = ?', [1]);
$db->delete('users', 'id = ?', [1]);
```

`Router::list()` **removed - use** `Router::getRoutes()` **or** `Router::listRoutes()`

```
// BEFORE
$routes = Router::list();

// AFTER
$routes = Router::getRoutes();   // or Router::listRoutes()
```

New Features

`ORM::findById(int|string $id)` - explicit primary method (with `find()` and `load()` as aliases).

`Session`**:** `TINA4_SESSION_HANDLER` **env var** - replaces `TINA4_SESSION_BACKEND` (old name still accepted for backward compatibility).

`Session\RedisSessionHandler` - new zero-dependency Redis session handler using raw RESP protocol over TCP sockets. Configure with `TINA4_SESSION_REDIS_HOST`, `TINA4_SESSION_REDIS_PORT`, `TINA4_SESSION_REDIS_PASSWORD`, `TINA4_SESSION_REDIS_DB`.

`Database::cacheStats()` **and** `Database::cacheClear()` - query cache wired to `TINA4_DB_CACHE=true` env var.

---

v3.10.38 -- April 1, 2026

Code Metrics & Bubble Chart

The dev dashboard (`/__dev`) now includes a **Code Metrics** tab with a PHPMetrics-style bubble chart visualization. Files appear as animated bubbles sized by LOC and colored by maintainability index. Click any bubble to drill down into per-function cyclomatic complexity.

The metrics engine uses `token_get_all()` for zero-dependency static analysis covering cyclomatic complexity, Halstead volume, maintainability index, coupling, and violation detection. File analysis is sorted worst-first. Results are cached for 60 seconds.

AI Context Installer

`bin/tina4php ai` now presents a simple numbered menu instead of auto-detection. Select tools by number, comma-separated or `all`. Already-installed tools show green. Generated context includes the full skills table.

Dashboard Improvements

Full-width layout, sticky header/tabs, full-screen overlay. Removed junk migrations (`mooo`, `hkhkhk`), sample routes, and test templates.

Cleanup

Removed old `plan/` spec documents, replaced with `PARITY.md` and `TESTS.md`. Central parity matrix added to tina4-book.

---

v3.10.x -- Previous Releases

Released: 28-30 March 2026

The v3.10 line introduced the Frond template engine as a singleton, automatic ORM field mapping, transaction safety, the `tina4 ai` CLI command, and a round of Frond parser fixes.

v3.10.29 -- `tina4 ai` Command (30 March 2026)

The `tina4 ai` CLI command stopped returning a stub message and started doing real work. It scans your project for AI coding tools -- Claude Code, Cursor, GitHub Copilot, Windsurf, Aider, Cline, and OpenAI Codex -- then installs context files for each one it finds.

```
# Detect tools and install context files

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

tina4 ai

# Install for all known AI tools
```

```
## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

tina4 ai --all

# Overwrite existing context files

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

tina4 ai --force
```

v3.10.28 -- DevReload Performance Fix (30 March 2026)

The development server's live-reload system polled too often and reloaded too fast. This patch fixed both problems.

- **Poll interval** increased from 2 seconds to 3 seconds (configurable via `TINA4_DEV_POLL_INTERVAL`)
- **Debounce** added a 500ms window before triggering a reload, preventing rapid successive refreshes
- **CSS hot-reload** now busts stylesheet cache links instead of forcing a full page reload

```
// .env -- set poll interval in milliseconds
TINA4_DEV_POLL_INTERVAL=1000  // 1 second
TINA4_DEV_POLL_INTERVAL=5000  // 5 seconds
```

v3.10.27 -- Frond Macro HTML Escaping Fix (30 March 2026)

Macro output was getting HTML-escaped when used inside expressions. A `{% macro %}` that returned HTML would render as visible `<div>` tags instead of actual markup. This patch marks macro output as safe, matching standard Twig behaviour.

Before (broken):

```
{% macro button(label) %}
  <button class="btn">{{ label }}</button>
{% endmacro %}

{# Rendered as: &lt;button class=&quot;btn&quot;&gt;Click&lt;/button&gt; #}
{{ button("Click") }}
```

After (fixed):

```
{# Renders as: <button class="btn">Click</button> #}
{{ button("Click") }}
```

v3.10.26 -- Rebrand and ORM Transaction Safety (30 March 2026)

Two changes landed together. The framework rebranded to "This Is Now A 4Framework" across all default pages, CLI banners, and READMEs. More important: ORM `save()`, `delete()`, and `restore()` now wrap their operations in explicit transactions.

Before (manual workaround for SQLite):

```
$db->startTransaction();
try {
    $user = new User();
    $user->name = "Alice";
    $user->save(); // Without this wrapper, SQLite could silently drop the write
    $db->commit();
} catch (\Exception $e) {
    $db->rollback();
}
```

After (automatic transaction wrapping):

```
$user = new User();
$user->name = "Alice";
$user->save(); // save() now calls startTransaction() / commit() / rollback() internally
```

The fix also removed the hardcoded `version` field from `composer.json`, which was blocking Packagist sync.

v3.10.22-24 -- Form Tokens and Template Fixes (29-30 March 2026)

- **v3.10.24** -- Fixed stale templates in dev mode. The Frond engine now checks file modification times during development
- **v3.10.23** -- Added `formTokenValue` and `form_token_value` as template globals for CSRF protection
- **v3.10.22** -- Form tokens now include a nonce in the JWT payload, making each token unique per form render

```
<form method="POST" action="/users">
  <input type="hidden" name="formToken" value="{{ formTokenValue }}">
  <input type="text" name="name">
  <button type="submit">Create</button>
</form>
```

v3.10.18-20 -- Frond Parser Fixes (28-29 March 2026)

- **v3.10.20** -- Race-safe `getNextId()` for pre-generating database IDs. Frond engine optimization for template compilation
- **v3.10.18** -- Fixed ternary and inline-if parsing when the expression contained quoted strings

Before (broken):

```
{# This crashed the parser because the colon inside the string confused the ternary operator #}
{{ isActive ? "status: active" : "status: inactive" }}
```

After (fixed):

```
{# Parser now tracks quote boundaries before splitting on operators #}
{{ isActive ? "status: active" : "status: inactive" }}
```

v3.10.14-16 -- Filters, Auto-Commit, and ID Generation (28 March 2026)

- **v3.10.16** -- Added `to_json`, `tojson`, and `js_escape` template filters
- **v3.10.15** -- Fixed the `|replace` filter when the replacement string contained backslashes
- **v3.10.14** -- Added `get_next_id()` for engine-aware ID pre-generation
- **v3.10.13** -- ORM now auto-commits on write operations

```
{# to_json filter -- pass PHP data to JavaScript #}
<script>
  const config = {{ settings|to_json }};
</script>

{# js_escape filter -- safe string embedding in JavaScript #}
<script>
  const message = "{{ userInput|js_escape }}";
</script>
```

v3.10.10-12 -- Firebird, Sessions, and Dictionary Access (28 March 2026)

- **v3.10.12** -- Session garbage collection and NATS backplane parity
- **v3.10.11** -- Fixed dictionary access with variable keys in Frond templates
- **v3.10.10** -- Fixed Firebird migration runner for idempotent schema changes

Before (broken in v3.10.9):

```
{% set key = "name" %}
{# This returned null instead of the value #}
{{ user[key] }}
```

After (fixed in v3.10.11):

```
{% set key = "name" %}
{# Now resolves the variable and uses it as the dictionary key #}
{{ user[key] }}
```

v3.10.5 -- Frond Quote-Aware Operator Matching (28 March 2026)

Template operators (`~`, `??`, comparisons) no longer match inside quoted strings. The expression evaluator gained `findOutsideQuotes()` and `splitOutsideQuotes()` helpers.

v3.10.1 -- autoMap for ORM Field Mapping (28 March 2026)

The ORM gained an `$autoMap` flag. Set it to `true` and the ORM converts between `snake_case` database columns and `camelCase` PHP properties without manual field mapping.

Before (manual mapping):

```
class UserProfile extends \Tina4\ORM {
    public $tableName = "user_profiles";
    public $fieldMapping = [
        "first_name" => "firstName",
        "last_name"  => "lastName",
        "created_at" => "createdAt",
    ];
}
```

After (automatic mapping):

```
class UserProfile extends \Tina4\ORM {
    public $tableName = "user_profiles";
    public $autoMap = true;

    // DB columns: first_name, last_name, created_at
    // Auto-mapped to: $firstName, $lastName, $createdAt
}
```

Explicit entries in `$fieldMapping` take precedence. The auto-mapper never overwrites manual mappings.

v3.10.0 -- Singleton Frond Engine (28 March 2026)

The Frond template engine became a singleton. One instance serves all template renders during a request, eliminating redundant initialization and token parsing. This is an internal change -- no API differences -- but template-heavy pages render faster.

---

v3.9.x -- QueryBuilder, Sessions, Security

Released: 26-27 March 2026

The v3.9 line brought three headline features: a fluent query builder, automatic sessions, and secure-by-default routing. It also fixed a critical `setcookie()` error in the built-in server.

v3.9.0 -- QueryBuilder, Sessions, Path Injection (26 March 2026)

QueryBuilder. A fluent SQL builder that works standalone or through the ORM.

```
// Through the ORM
$admins = User::query()
    ->where("role = ?", ["admin"])
    ->orderBy("name")
    ->limit(10)
    ->get();

// Standalone
$results = QueryBuilder::fromTable("orders")
    ->where("total > ?", [100])
    ->join("customers", "customers.id = orders.customer_id")
    ->orderBy("total DESC")
    ->get();
```

The builder supports `select`, `where`, `orWhere`, `join`, `leftJoin`, `groupBy`, `having`, `orderBy`, `limit`, `first`, `count`, `exists`, and `toSql`.

Path parameter injection. Route handlers receive path parameters as named function arguments.

```
// The framework injects $id directly into the handler
Router::get("/users/{id:int}", function (int $id) {
    $user = (new User())->load("id = ?", [$id]);
    return $user->toArray();
});
```

Auto-start sessions. Every route handler now has `$request->session` available with zero configuration.

```
Router::get("/dashboard", function ($request) {
    $request->session->set("lastVisit", date("Y-m-d H:i:s"));
    $username = $request->session->get("username");
    return ["user" => $username];
});
```

The session API includes: `get`, `set`, `delete`, `has`, `clear`, `destroy`, `save`, `regenerate`, `flash`, `getFlash`, and `all`.

v3.9.1 -- Secure by Default (27 March 2026)

POST, PUT, PATCH, and DELETE routes now require authentication by default. CSRF middleware ships built in with session-bound form tokens.

Breaking change. If your application has unprotected POST routes, they will return 401 after upgrading.

Before (v3.8.x):

```
// This worked without any auth
Router::post("/comments", function ($request) {
    // Anyone could post
});
```

After (v3.9.1):

```
// Option 1: Add authentication
Router::post("/comments", function ($request) {
    // Requires valid auth token
})->secure();

// Option 2: Explicitly allow public access
Router::post("/comments", function ($request) {
    // Public endpoint
})->allowAnonymous();
```

This release also standardized environment variables: `TINA4_CORS_CREDENTIALS`, `TINA4_RATE_LIMIT`, `TINA4_CSRF`, `TINA4_QUEUE_BACKEND`, and `TINA4_TOKEN_LIMIT`.

v3.9.2-3 -- NoSQL QueryBuilder and Cookie Fixes (27 March 2026)

- **v3.9.3** -- Fixed `SameSite` cookie handling and session `save()` visibility. Set `SameSite=Lax` as the default
- **v3.9.2** -- Extended QueryBuilder to work with MongoDB. Added WebSocket backplane support

v3.9.4 -- setcookie Fatal Error Fix (27 March 2026)

The built-in HTTP server called `setcookie()` after headers had already been sent. This caused a fatal error on any route that set a cookie. The fix buffers cookie headers before flushing the response.

---

v3.8.x -- Hot Reload, Security Headers, Connection Pooling

Released: 25-26 March 2026

v3.8.0 -- Hot Reload and Frond Filters (25 March 2026)

The PHP development server gained hot reload. Save a file and the browser refreshes. The Frond template engine added `base64encode` and `base64decode` filter aliases.

```
// .env
TINA4_DEBUG=true  // Enables hot reload in dev mode
```

v3.8.1-3 -- Security and Validation (25 March 2026)

Three patches shipped on the same day:

- **SecurityHeadersMiddleware** -- Adds `X-Frame-Options`, `Strict-Transport-Security`, `Content-Security-Policy`, and `X-Content-Type-Options` to every response
- **Validator class** -- Input validation with error response envelopes
- **Upload size limit** -- Configurable via `TINA4_MAX_UPLOAD_SIZE` (default 10MB)

```
// .env
TINA4_MAX_UPLOAD_SIZE=20  // 20MB limit
```

v3.8.7 -- Connection Pooling (26 March 2026)

Database connections now support round-robin pooling. Pass a `pool` parameter to the connection string.

```
// Pool of 5 connections
$db = new Database("sqlite:///data/app.db?pool=5");
```

---

v3.7.x -- Template Auto-Serve, Typed Route Params

Released: 25 March 2026

v3.7.0 -- Template Auto-Serve (25 March 2026)

Place an `index.html` or `index.twig` in `src/templates/` and the framework serves it at `/`. No route registration needed. If you register a `GET /` route, your route takes priority. If neither exists, the framework falls back to the default landing page.

This release also made Firebird migrations idempotent. `ALTER TABLE ADD` statements check `RDB$RELATION_FIELDS` before executing. Columns that already exist are skipped and logged.

v3.7.1-2 -- Typed Route Parameters (25 March 2026)

Route parameters gained type constraints: `{id:int}`, `{price:float}`, `{slug:path}`. The framework validates the type before the handler runs. Mismatched types return a 404.

```
Router::get("/products/{id:int}", function (int $id) {
    // $id is guaranteed to be an integer
});
```

```
Router::get("/files/{path:path}", function (string $path) {
    // $path matches the full remaining URL segment
});
```

Template lookup caching arrived for production mode. In development, the framework checks the filesystem on every request to pick up changes.

---

v3.6.x -- Reference Cleanup

Released: 24 March 2026

v3.6.0 (24 March 2026)

Fixed outdated API references across the codebase. Corrected `getToken`/`validToken` method names and the `TINA4_LOCALE` environment variable. No new features -- housekeeping only.

---

v3.5.x -- Bundled Frontend, Swagger, Middleware

Released: 24 March 2026

v3.5.0 (24 March 2026)

The `tina4js.min.js` reactive frontend library (13.6KB) now ships bundled with the framework. No CDN link, no npm install -- the file is included in the package.

- **AutoCrud** routes now include Swagger metadata
- **Swagger generator** parses docblock annotations from route handlers
- **Middleware** standardized around `before*` and `after*` naming with three built-in middleware classes

```
/**
 * @description Get user by ID
 * @tags Users
 * @param int $id User ID
 * @return User
 */
Router::get("/api/users/{id:int}", function (int $id) {
    return (new User())->load("id = ?", [$id]);
});
```

The Swagger generator picks up the `@description`, `@tags`, `@param`, and `@return` annotations and builds the OpenAPI spec.

---

v3.4.x -- DatabaseResult, File Uploads, WebSocket Broadcast

Released: 24 March 2026

v3.4.0 (24 March 2026)

The database layer gained a `DatabaseResult` class as a standardized return type for `fetch()`. Every query now returns an object with `data`, `error`, `count`, and a lazy `columnInfo()` method for schema metadata.

Breaking change. If your code accessed raw arrays from database queries, you need to use `->data` on the result.

Before (v3.3.x):

```
$rows = $db->fetch("SELECT * FROM users");
foreach ($rows as $row) {
    echo $row["name"];
}
```

After (v3.4.0):

```
$result = $db->fetch("SELECT * FROM users");
foreach ($result->data as $row) {
    echo $row["name"];
}
// Also available: $result->error, $result->count
```

Other changes in this release:

- **File uploads** normalized to a consistent format: `filename`, `type`, `content`, `size`. Added `data_uri` template filter for inline images
- **WebSocket broadcast** scoped to specific paths
- `Database::getConnection()` added as an alias for accessing the underlying connection
- **Queue job files** changed extension from `.json` to `.queue-data`
- **Auth method rename** -- `getToken` became the primary method, `createToken` became an alias

---

v3.3.x -- Queue API, Route Chaining, Dev Toolbar

Released: 24 March 2026

v3.3.0 (24 March 2026)

The queue system arrived with a full lifecycle API. Produce messages, consume them with a generator, and manage job state through `complete()`, `fail()`, and `reject()`.

```
use Tina4\Queue;

// Produce a job
Queue::produce("emails", ["to" => "alice@example.com", "subject" => "Welcome"]);

// Consume jobs
foreach (Queue::consume("emails") as $job) {
    try {
        sendEmail($job->data);
        $job->complete();
```

```
    } catch (\Exception $e) {
        $job->fail($e->getMessage());
    }
}
```

Route handlers became flexible. They accept zero, one, or two parameters with type-hint detection. Chaining modifiers arrived: `.secure()` and `.cache()`.

```
// Zero params
Router::get("/health", function () {
    return ["status" => "ok"];
});

// One param -- type-hint determines if it's request or response
Router::get("/users", function (\Tina4\Request $request) {
    return $request->query;
});

// Chain modifiers
Router::get("/admin/users", function ($request) {
    return User::query()->get();
})->secure()->cache(300);
```

The `render()` method arrived as an alias for `template()`. The dev admin dashboard made routes clickable -- each one opens in a new tab.

---

v3.2.x -- Flexible Route Handlers, Strict Mode, Live Reload

Released: 23 March 2026

v3.2.0 (23 March 2026)

Route handlers gained support for zero, one, or two parameters. The framework detects whether your single parameter is a request or response by checking the type hint.

```
// Single param with Request type-hint
Router::get("/search", function (\Tina4\Request $request) {
    return ["q" => $request->query["q"]];
});
```

The authentication API consolidated. `getToken` and `validToken` are the only token methods - there are no `createToken`/`validateToken` aliases.

```
$token = \Tina4\Auth::getToken(["userId" => 42, "role" => "admin"]);
$valid = \Tina4\Auth::validToken($token);
```

Other changes: inline `Router::` calls now work in route discovery. The error overlay passes full request details. Windows path separators are normalized after gallery deploy.

v3.2.1-2 -- Strict Mode and Benchmarks (23 March 2026)

- **Strict mode** -- PHP warnings and notices now throw `ErrorException`. Silent failures become visible failures
- **Stream-select server** gained a 35% throughput improvement

- **Live-reload polling** added for PHP dev mode

---

v3.1.x -- Drop-In Server Support, Migration Guide

Released: 22 March 2026

v3.1.0 (22 March 2026)

The framework gained drop-in support for production-grade PHP servers. The application exposes an `__invoke()` method compatible with Swoole, RoadRunner, FrankenPHP, and ReactPHP.

```
// RoadRunner worker
$app = new \Tina4\App();
// $app is callable -- pass it to your server's handler
```

The `toDict()` method was renamed to `toArray()` as the primary name. `toDict()` remains as an alias.

v3.1.1-2 -- README and CI Fixes (22 March 2026)

- Fixed `composer.json` to remove the `version` field (Packagist recommendation)
- Updated README code examples to match the v3 API
- Added the v2 to v3 migration guide (`MIGRATE.md`)

---

v3.0.0 -- The Rewrite

Released: 22 March 2026

The v3.0.0 release is a ground-up rewrite. Zero Composer dependencies. The HTTP server, template engine, ORM, and database drivers are all native PHP. No Guzzle, no Twig, no Doctrine.

What Changed

- **Custom HTTP server** built on `stream_select` with WebSocket support -- replaces `php -S`
- **Frond template engine** with pre-compilation and token caching -- replaces Twig
- **Native database drivers** for SQLite, PostgreSQL, MySQL, MSSQL, and Firebird
- **Standardized connection strings** -- `driver://host:port/database`
- **DevAdmin dashboard** with panels for routes, database, queue, mailbox, requests, errors, and connection info
- `tina4 generate` **command** scaffolds models, routes, migrations, and middleware
- **DB query caching** via `TINA4_DB_CACHE=true` for transparent read caching
- **Unified queue** switches between SQLite, RabbitMQ, and Kafka via `.env` with no code change
- **Unified messenger** for email sending, driven by `.env` configuration

- **Interactive gallery** with seven example apps, each with deploy and live-try buttons
- **Dead letter queue** support with `deadLetters()`, `purge()`, and `retryFailed()`

Breaking Changes from v2

These are the changes that will break existing code. See Chapter 37 for a full migration checklist.

PHP version. Requires PHP 8.0 or later.

Package. Still `tina4stack/tina4php`, now at `^3.0`, and it is the only package: the v2 split packages (`tina4php-core`, `tina4php-database`, `tina4php-orm`) are gone.

```
# v2

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

composer require tina4stack/tina4php

# v3

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

composer require tina4stack/tina4php:^3.0
```

Directory structure. `Tina4/` moved from `src/Tina4/` to the project root.

Database connection strings. Scheme aliases removed.

```
// v2 -- these aliases no longer work
$db = new \Tina4\DataSQLite3("database.db");
$db = new \Tina4\DataFirebird("localhost:database.fdb");

// v3 -- use standardized connection strings
$db = new \Tina4\Database("sqlite:///database.db");
$db = new \Tina4\Database("firebird://localhost/database.fdb");
```

Environment variable rename. `TINA4_AUTOCOMMIT` became `TINA4_AUTOCOMMIT` (no underscore between AUTO and COMMIT).

Method naming. All methods follow camelCase.

```
// v2
$db->start_transaction();

// v3
$db->startTransaction();
```

Release Candidates (21 March 2026)

Five release candidates shipped before the final v3.0.0 release. They covered the initial rewrite (`rc.1`), Frond pre-compilation with a 2.8x render speedup (`rc.4`), and final test fixes (`rc.5`). The RC period lasted one day. The test suite passed before the final tag.

Upgrade Path Summary

From	To	Action Required
v2.x	v3.0.0	Full migration -- see Chapter 37
v3.0-3.3	v3.4.0	Update `fetch()` calls to use `->data` on DatabaseResult
v3.0-3.8	v3.9.1	Add auth to POST/PUT/PATCH/DELETE routes or mark them `->allowAnonymous()`
v3.9.x	v3.10.x	No breaking changes -- update and test

The version numbers move fast. The API stays stable. Pick the latest v3.10.x patch and your application gets every fix listed here without changing a line of code.

Upgrading from v2 to v3

1. Overview

Tina4 v3 is a ground-up rewrite. Zero Composer dependencies. The built-in HTTP server, template engine, ORM, and database drivers are all native PHP. No Guzzle, no Twig, no Doctrine -- nothing external.

The patterns you know still work. Routes, ORM models, templates, migrations -- same concepts, cleaner implementation. The framework is faster, more consistent, and easier to deploy.

Three things to know before you start:

- **Methods are camelCase.** `$db->startTransaction()`, not `$db->start_transaction()`.
- **Classes are PascalCase.** `Router`, `Database`, `ORM`, `Response`.
- **Zero dependencies.** `composer install` pulls exactly one package: `tina4stack/tina4php`.

This chapter walks through every breaking change and gives you a step-by-step migration checklist at the end.

Step 0: Run the Automated Upgrade Command

Before doing anything manually, run the automated upgrade tool:

```
cd your-v2-project
tina4 i-want-to-stop-using-v2-and-switch-to-v3
```

This command automatically:

- Moves `routes/`, `orm/`, `templates/`, `scss/`, `public/`, `app/`, `locales/`, `seeds/` into `src/`
- Updates your `composer.json` dependency from v2 to v3
- Removes split packages (`tina4php-core`, `tina4php-database`, `tina4php-orm`)

After running the command, continue with the steps below for the changes that require manual attention.

---

2. Package and Installation

v2

```
composer require tina4stack/tina4php
```

This pulled in a tree of Composer dependencies -- HTTP clients, template engines, database abstractions.

v3

```
composer require tina4stack/tina4php
```

The package name is unchanged from v2: `tina4stack/tina4php`, no hyphen. This is the only package. It has zero Composer dependencies. Everything is built in.

Update your `composer.json`:

```
{
    "require": {
        "tina4stack/tina4php": "^3.0"
    }
}
```

Then run:

```
composer update
```

Remove any v2-era Tina4 packages from `composer.json` (`tina4stack/tina4php`, `tina4stack/tina4-database`, etc.). They are all consolidated into the single `tina4-php` package.

---

3. Project Structure Changes

v2 Structure

```
project/
  routes/
  orm/
  templates/
  migrations/
  .env
  index.php
```

v3 Structure

```
project/
  src/
    routes/
    orm/
    templates/
    app/
  migrations/
  .env
  index.php
```

The key change: source files live under `src/`. Routes go in `src/routes/`, ORM models in `src/orm/`, templates in `src/templates/`, and application logic in `src/app/`.

Auto-discovery still works the same way. Every `.php` file in `src/` and its subdirectories is auto-included at startup. No manual registration. Drop a file in, it loads.

Migration path: Move your existing `routes/`, `orm/`, and `templates/` directories into `src/`. Create `src/app/` for any utility classes or service files.

---

4. Routing Changes

Method-Based Registration

The core pattern is the same. The class name changed from `Route` to `Router`:

v2:

```
<?php
\Tina4\Get::add("/hello", function ($response, $request) {
    return $response("Hello, World!");
});
```

v3:

```
<?php
use Tina4\Router;

Router::get("/hello", function ($request, $response) {
    return $response->json(["message" => "Hello, World!"]);
});
```

Three changes:

- `\Tina4\Get::add()` becomes `Router::get()`. Same for `Post`, `Put`, `Patch`, `Delete`.
- The callback signature flips: `$request` comes first, then `$response`.
- Response methods are explicit: `$response->json()`, `$response->render()`, `$response->text()`.

Class-Based Routes with Attributes

v3 supports PHP 8 attributes for class-based routes:

```
<?php
use Tina4\Router;

class ProductController
{
    #[Router("/products", "GET")]
    public function list($request, $response)
    {
        return $response->json(["products" => []]);
    }

    #[Router("/products", "POST")]
    public function create($request, $response)
    {
        return $response->json(["created" => true], 201);
    }
}
```

Auth Defaults

This is a breaking change. v3 has opinionated auth defaults:

Method	v2 Default	v3 Default
GET	Public	Public
POST	Public	Requires auth
PUT	Public	Requires auth
PATCH	Public	Requires auth
DELETE	Public	Requires auth

Write operations require authentication by default. Two attributes control this:

- `#[NoAuth]` -- Makes a write route public (no auth required).
- `#[Secured]` -- Requires auth on a GET route.

```
<?php
use Tina4\Router;
use Tina4\NoAuth;
use Tina4\Secured;

// This POST route is public -- no auth needed
#[NoAuth]
Router::post("/register", function ($request, $response) {
    // Public registration endpoint
    return $response->json(["registered" => true], 201);
});

// This GET route requires auth
#[Secured]
Router::get("/admin/dashboard", function ($request, $response) {
    return $response->render("admin/dashboard.html");
});
```

If your v2 app had unprotected POST/PUT/PATCH/DELETE routes, add `#[NoAuth]` to each one or they will return 401. Review every write route during migration.

---

5. Database Changes

Connection Strings

The format is the same URL-based approach, but the syntax is cleaner:

v2:

```
$db = new \Tina4\DataSQLite3("data/app.db");
```

v3:

```
TINA4_DATABASE_URL=sqlite:///data/app.db
```

Or in code:

```
<?php
use Tina4\Database;
```

```
$db = new Database("sqlite:///data/app.db");
```

All drivers use the same `Database` class. The URL scheme selects the driver:

```
# SQLite
TINA4_DATABASE_URL=sqlite:///data/app.db

# PostgreSQL
TINA4_DATABASE_URL=postgres://localhost:5432/myapp

# MySQL
TINA4_DATABASE_URL=mysql://localhost:3306/myapp

# Firebird
TINA4_DATABASE_URL=firebird://localhost:3050/path/to/database.fdb

# Microsoft SQL Server
TINA4_DATABASE_URL=mssql://localhost:1433/myapp
```

Firebird Notes

v3 supports both the legacy `interbase` and modern `firebird-driver` PHP extensions. Column names are returned in lowercase by default. If your v2 code relied on uppercase column names from Firebird, update your references.

Transactions

Transaction methods are renamed to camelCase:

v2:

```
$db->beginTransaction();
// ... queries ...
$db->commit();
$db->rollBack();
```

v3:

```
$db->startTransaction();
// ... queries ...
$db->commit();
$db->rollback();
```

Note: `beginTransaction()` becomes `startTransaction()`, and `rollBack()` (capital B) becomes `rollback()` (lowercase b).

---

6. ORM Changes

Auto-Mapping

The biggest ORM improvement in v3 is `$autoMap`. Set it to `true` and Tina4 automatically generates `$fieldMapping` entries from your camelCase PHP properties to snake_case database columns.

v2:

```php
<?php
class Product extends \Tina4\ORM
{
    public $tableName = "products";
    public $primaryKey = "id";

    public $id;
    public $productName;
    public $unitPrice;
    public $inStock;

    public $fieldMapping = [
        "productName" => "product_name",
        "unitPrice" => "unit_price",
        "inStock" => "in_stock"
    ];
}
```

v3:

```php
<?php
use Tina4\ORM;

class Product extends ORM
{
    public string $tableName = "products";
    public string $primaryKey = "id";
    public bool $autoMap = true;

    public int $id;
    public string $productName;
    public float $unitPrice;
    public bool $inStock = true;
}
```

With `$autoMap = true`, Tina4 sees `$productName` and auto-generates the mapping to `product_name`. Same for `$unitPrice` to `unit_price` and `$inStock` to `in_stock`. No manual `$fieldMapping` needed.

Explicit Mappings Take Precedence

If you have a column that does not follow the convention, add it to `$fieldMapping` manually. Explicit entries override auto-generated ones:

```php
<?php
use Tina4\ORM;

class Legacy extends ORM
{
    public string $tableName = "legacy_table";
    public bool $autoMap = true;

    public int $id;
    public string $firstName;     // auto-maps to first_name
    public string $legacyField;   // override below

    public array $fieldMapping = [
        "legacyField" => "LEGACY_FLD"  // takes precedence over auto-map
    ];
```

```
}
```

Utility Methods

Two helper methods are available for manual conversions:

```
$snake = ORM::camelToSnake("firstName");  // "first_name"
$camel = ORM::snakeToCamel("first_name"); // "firstName"
```

Typed Properties

v3 models use PHP 8 typed properties. Add types to all your ORM properties:

```
// v2
public $price;

// v3
public float $price = 0.00;
```

---

7. Template Engine Changes

Tina4 v3 uses Frond as its template engine. If you were using Twig in v2, most syntax carries over -- Frond is Twig-compatible.

Frond Singleton

Access the Frond instance through the `Response` class:

```
<?php
use Tina4\Response;

// Get the Frond instance
$frond = Response::getFrond();

// Add a custom filter
$frond->addFilter("shout", function ($value) {
    return strtoupper($value) . "!!!";
});

// Add a global variable
$frond->addGlobal("appName", "My App");

// Set the instance back (if needed after modification)
Response::setFrond($frond);
```

Custom Filters and Globals

Custom filters and globals persist across requests within the same process. Register them once at startup (in `src/app/` or early in `index.php`) and they are available in every template render.

Method Calls in Templates

v3 adds the ability to call methods on array or object values inside templates:

```
<!-- Call a method on an object -->
<p>{{ user.getName() }}</p>

<!-- Call with arguments -->
```

```
<p>{{ translator.t("welcome_message") }}</p>

<!-- Chain with filters -->
<p>{{ user.getName()|upper }}</p>
```

This is new in v3. v2 only supported property access (`{{ user.name }}`), not method calls.

---

8. Migration Tracking Table

v3 uses an enhanced migration tracking table with additional columns for better tracking.

When you run migrations on a database that was previously managed by v2, Tina4 v3 auto-detects the old tracking table format and upgrades it. The upgrade adds three columns:

- `migration_id` -- Unique identifier for each migration record.
- `batch` -- Groups migrations that ran together.
- `executed_at` -- Timestamp of when the migration was executed.

Existing migration records are preserved. The upgrade happens automatically on the first migration run. No manual intervention needed.

---

9. New Features in v3

Features that did not exist in v2. Each is covered in its own chapter:

- **Built-in HTTP server** -- No Apache or Nginx needed for development (Chapter 1).
- **Connection pooling** -- `pool` parameter on database connections (Chapter 5).
- **Query builder** -- Fluent SQL without raw strings (Chapter 7).
- **Job queues** -- Background task processing (Chapter 12).
- **WebSocket support** -- Real-time communication built in (Chapter 23).
- **Email sending** -- Native SMTP, no SwiftMailer (Chapter 16).
- **Caching layer** -- File, Redis, Valkey, Mongo backends (Chapter 11).
- **GraphQL** -- Built-in GraphQL endpoint (Chapter 22).
- **CLI tooling** -- `tina4` command for scaffolding, migrations, serving (Chapter 31).
- **Auto-mapping in ORM** -- `$autoMap = true` eliminates manual field mappings (Chapter 6).
- **Rate limiting** -- Per-IP rate limiting via env vars (Chapter 10).
- **CSRF protection** -- Built-in, enabled by default (Chapter 10).

Common Pitfalls

1. POST/PUT/DELETE routes now require authentication

This is the most common upgrade issue. In v2, all routes were public by default. In v3, only GET routes are public -- POST, PUT, PATCH, and DELETE require a Bearer JWT token.

Symptom: Working v2 endpoints return `401 Unauthorized` after upgrading.

Fix: Add `#[NoAuth]` or `->noAuth()` to any write route that should remain public.

Find affected routes:

```
grep -rn "Router::post\|Router::put\|Router::patch\|Router::delete" src/routes/
```

Review each match -- if the endpoint should be public (webhooks, public forms, etc.), add the `#[NoAuth]` attribute.

2. Database connection strings changed

v2 used driver-specific classes. v3 uses URL format:

```
// v2
$db = new \Tina4\DataSQLite3("data/app.db");
// v3
$db = Database::create("sqlite:///data/app.db");
```

3. Template engine renamed

v2: `Template.render()` -- v3: `Frond.render()` (or `$response->render()`)

The Twig syntax is the same -- your `.twig` files work unchanged. Only the PHP API call changes.

---

10. Step-by-Step Migration Checklist

Follow these steps in order. Check each one off as you go.

- **Back up your v2 project.** Copy the entire project directory. You want a rollback point.
- **Update `composer.json`.** Change the `tina4stack/tina4php` constraint to `^3.0`. Remove any other `tina4stack/*` packages. Run `composer update`.
- **Restructure directories.** Move `routes/` to `src/routes/`, `orm/` to `src/orm/`, `templates/` to `src/templates/`. Create `src/app/` for utility classes.
- **Update namespace imports.** Replace `\Tina4\Get::add()` with `Router::get()`, `\Tina4\Post::add()` with `Router::post()`, and so on. Add `use Tina4\Router;` at the top of each route file.
- **Fix callback signatures.** Swap the parameter order from `($response, $request)` to `($request, $response)` in every route callback.
- **Update response calls.** Replace `$response("data")` with `$response->json()`, `$response->render()`, or `$response->text()` as appropriate.

- **Review auth on write routes.** Every POST, PUT, PATCH, and DELETE route now requires auth by default. Add `#[NoAuth]` to any write route that should be public (registration, public form submissions, webhooks).
- **Add `#[Secured]` to protected GET routes.** Any GET route that should require authentication needs the `#[Secured]` attribute.
- **Update database connection code.** Replace driver-specific classes (`DataSQLite3`, `DataMySQL`, etc.) with `new Database("url")`. Move connection strings to `TINA4_DATABASE_URL` in `.env`.
- **Fix transaction calls.** `beginTransaction()` becomes `startTransaction()`. `rollBack()` becomes `rollback()`.
- **Add typed properties to ORM models.** Replace untyped `public $name;` with typed `public string $name;`. Add `$autoMap = true` and remove manual `$fieldMapping` entries where auto-mapping covers the conversion.
- **Update template code.** If you were using Twig directly, switch to Frond. The syntax is compatible. Replace any Twig-specific PHP calls with `Response::getFrond()`.
- **Run migrations.** Start the server and trigger a migration run. Tina4 auto-upgrades the tracking table. Verify your existing migrations still apply cleanly.
- **Test every route.** Hit every endpoint. Check auth behaviour. Verify response formats. The test chapter (Chapter 18) covers how to write automated tests.
- **Remove leftover v2 files.** Delete the old `routes/`, `orm/`, and `templates/` directories at the project root (now that everything is under `src/`). Clean up any v2-specific config files.

The migration is mechanical. Most of it is find-and-replace. The auth defaults are the one change that can silently break things -- step 7 is the most important step on this list.

Feature Catalog

Tina4 3.14 has **140 numbered catalog entries**. The number describes the framework family's implementation and audit inventory. It does not mean that 140 features have reached four-language parity, and it does not mean that every entry lives inside each backend package.

This chapter is the map. The earlier chapters explain the public APIs available to PHP applications. The numbered audit packets define the parity work and the clean-room formula for another language.

Read the status correctly

A feature moves through four distinct states:

- **Catalogued** means the feature has a stable number, name, owner, and audit packet.
- **Shipped** means code exists in one or more released components.
- **Audited** means the team has measured all relevant implementations and recorded contradictions and decisions.
- **Contract-proven** means a shared fixture passes in every applicable implementation with real dependencies.

Do not collapse those states into one green tick. At version 3.13.101, the contract ledger reports **55 fixtures and 282 proven invariants, with 0 owed and 0 broken inside those fixture-covered contracts**. That result proves the named contracts. It does not certify the remaining catalog entries.

Where the features live

Range	Owner	Meaning
1-109, 126-132, 135-140	Backend frameworks	Runtime, developer, testing, and application-facing capabilities. Availability and maturity still require per-feature evidence.
110-125 and 134	Shared Rust CLI	One language-neutral client used with all four backends. These are not four separate backend implementations.
133	Verification contract	Carbonah benchmark and report shape, not an application runtime API.
Provider entries	Selected integration	Each selectable provider has its own number because another language must implement and test it separately. Providers can require an external service, driver, language extension, or extra package.

Feature 63 includes browser helpers and the separate tina4-js frontend package. The backend books explain their integration, but tina4-js is not embedded four times.

Current parity boundary

Evidence	Python	PHP	Ruby	Node.js
Catalog membership	Inventory source	Audit in progress	Audit in progress	Audit in progress
Fixture-covered contracts	Proven	Proven	Proven	Proven
Entire 140-entry catalog	Not yet proven	Not yet proven	Not yet proven	Not yet proven

The initial catalog followed the Python module inventory, but Python is not a permanent master. Tina4 promotes the best implementation after audit and captures that decision in an ADR and shared fixture.

Known missing feature surfaces remain tracked in PHP issue 185 and Node.js issue 37. An empty issue list in another repository is not parity evidence. The fixture ledger is the evidence.

Dependency boundary

The PHP core declares no required third-party packages. PHP 8.2 or newer and language extensions do not count as dependencies under this catalog rule. The MongoDB provider can require the optional `mongodb/mongodb` package; other selected capabilities may require their own extra package.

"Core dependency" and "provider dependency" are different facts. A local SQLite application can stay small while PostgreSQL, RabbitMQ, Kafka, Redis, MongoDB, S3, or hosted AI still requires the driver or service that speaks that protocol.

Numbered catalog

Foundation

#	Catalog entry
1	DotEnv and typed environment
2	Structured logger

Database and providers

#	Catalog entry
3	Database adapter interface
4	Database URL parser
5	Database facade and safe writes
6	Query builder
7	SQL translator
8	SQLite provider
9	PostgreSQL provider
10	MySQL provider
11	MSSQL provider
12	Firebird provider
13	ODBC provider
14	MongoDB SQL-translation provider
15	Migrations
16	Race-safe database sequences

ORM and data layer

#	Catalog entry
17	ORM base class
18	ORM fields and column mapping
19	Input and request validation
20	Soft delete
21	Declarative ORM relationships
22	Imperative ORM relationships
23	ORM scopes
24	Paginated database and ORM results
25	ORM result caching
26	ORM instance loading
27	Automatic CRUD from models
28	Seeder and fake data

HTTP core

#	Catalog entry
29	HTTP request model
30	HTTP response model and representation types
31	Router and dispatch
32	Route groups
33	Middleware pipeline

HTTP policies

#	Catalog entry
34	CORS middleware
35	Rate limiting
36	Security headers middleware
37	CSRF protection

HTTP runtime

#	Catalog entry
38	Health and readiness endpoints
39	Graceful shutdown
40	HTTP compression and ETag
41	Static assets and cache revalidation
42	Configurable error pages
43	Request ID tracking
44	File upload contract
45	Swagger and OpenAPI
46	Default landing page
47	In-process background tasks

Frond template engine

#	Catalog entry
48	Frond lexer
49	Frond parser
50	Frond compiler
51	Frond runtime
52	Frond filters
53	Frond tags
54	Frond expression tests
55	Frond functions
56	Frond extensibility API
57	Frond auto-escaping
58	Frond sandboxing
59	Frond template caching
60	Frond fragment caching

Frontend assets

#	Catalog entry
61	SCSS compiler
62	Tina4 CSS
63	Frond and Tina4 browser helpers

Authentication

#	Catalog entry
64	JWT and request authentication

Sessions and providers

#	Catalog entry
65	Session lifecycle
66	File session provider
67	Redis session provider
68	Valkey session provider
69	MongoDB session provider
70	Database session provider
71	Memcached session provider

Cache and providers

#	Catalog entry
72	Cache interface and provider selection
73	Memory cache provider
74	File cache provider
75	Redis cache provider
76	Valkey cache provider
77	Memcached cache provider
78	MongoDB cache provider
79	Database cache provider
80	HTTP response cache

Integrations and storage

#	Catalog entry
81	Standard-library HTTP API client
82	GraphQL
83	WebSocket protocol and server
84	Redis WebSocket backplane
85	NATS WebSocket backplane
86	WSDL and SOAP
87	Localization and i18n
88	Email and messenger
89	Queue lifecycle
90	Lite queue provider
91	RabbitMQ queue provider
92	Kafka queue provider
93	MongoDB queue provider
94	MQTT client
95	Document store interface
96	SQLite document store provider
97	MongoDB document store provider
98	Realtime collaboration
99	Local realtime attachment storage
100	S3 realtime attachment storage
101	Model Context Protocol server
102	Local source and documentation context index
103	Live framework and application API index
135	App-facing LLM client
137	GIS spatial points and queries
138	RBAC (role/permission guards)

139	Graph databases (Ultipa, Neo4j, Memgraph, ArangoDB)
140	Provider-neutral Web Push notifications

Enterprise authentication

#	Catalog entry
136	Configuration-first OpenID Connect SSO

Application runtime

#	Catalog entry
104	Event and listener system
105	Dependency injection container
106	Service runner
107	HTML element builder
108	AI coding-tool integration
109	Lazy feature loading and preload manifest

CLI

#	Catalog entry
110	CLI project initialization
111	CLI development server
112	CLI migrations
113	CLI seeding
114	CLI test runner
115	CLI route inspection
116	CLI interactive console
117	CLI environment management
118	CLI queue management
119	CLI container build
120	CLI code generation
121	CLI code metrics
122	CLI command discovery and help
123	CLI doctor
124	CLI guided setup
125	CLI deployment
134	CLI AI-skills installation and refresh

Developer runtime

#	Catalog entry
126	Development error overlay
127	Development admin dashboard
128	Dual development and test ports
129	Development port takeover
130	Dynamic framework version

Testing and verification tools

#	Catalog entry
131	In-process HTTP test client
132	Inline testing API
133	Carbonah benchmark contract

What parity means

Parity applies to observable contracts: response shapes, status codes, environment keys, error messages, file formats, and security rules. Host-language code remains idiomatic. A Python method can use snake_case while PHP and Node.js use camelCase, but all three must return the same public data.

The catalog gives another language a checklist. The audit packet supplies the decisions. The fixture supplies the proof. A name alone proves nothing; a green contract earns the claim.

Real-time Collaboration (WebRTC)

1. The Media Server You Do Not Have to Run

You want a call button. Two people click it and they are talking: audio, video, a shared screen. Next to the call sits a chat panel and a place to drop a file. This is the Slack shape, the Teams shape, and the usual advice is heavy. Stand up a media server. Wire in TURN. Operate all of it.

You do not need any of that to start. Modern browsers already speak WebRTC to each other directly, peer to peer. What they cannot do is *find* each other. One browser has to hand its connection offer to the other and get an answer back. That hand-off is called **signalling**, and it is a tiny amount of text relayed through a server. The media itself, the actual audio and video, never touches your server. It flows browser to browser.

Tina4's realtime module is that relay, plus the two things every collaboration tool needs around a call: persistent **chat** and permissioned **file** transfer. It shipped in 3.13.57, carries zero extra dependencies, and mounts with one line. Media is peer to peer, a **mesh**, by default. Tina4 carries no media and never parses your SDP. It forwards the handshake and nothing more. Outgrow mesh later and an SFU backend drops in with no route changes.

```
<?php
use Tina4\Realtime\Realtime;

Realtime::mount(); // one line: WebRTC signalling is live
```

---

2. What You Get, and How It Differs from Raw WebSocket

The WebSocket chapter gave you the raw `Router::websocket()` primitive: a path, a handler, and a secure flag. That is the tool you reach for when you build your *own* protocol. The realtime module is the opposite end. It is a *pre-built* protocol for calls, chat, and files, assembled from that same primitive underneath.

The module gives you three surfaces, and you enable only the ones you want:

- **calls** - a WebRTC signalling relay (mesh, peer to peer) plus a self-describing ICE-config endpoint. The relay forwards offer, answer, and ICE frames between peers in a room. It is public: no login to join a room.
- **chat** - persistent channels and messages backed by framework-owned ORM models, a secured chat WebSocket with live presence, typing, and read receipts, and a history endpoint for catch-up on reconnect.
- **files** - permissioned upload and download through a pluggable storage backend (local filesystem by default, S3-compatible optional).

The distinction in one line: with `Router::websocket()` you write the handler and invent the message protocol; with `Realtime::mount()` the protocol is written for you and the browser discovers every path from one config endpoint. Under the hood the realtime WebSockets *are* Tina4 WebSockets. Same room manager, same connection API. You are handed the handlers

pre-written.

One thing changes when you drop below the module and read the handlers, and it is the PHP handler convention. Tina4 PHP fires every WebSocket handler as `($connection, $data, $event)`. Position two is always the **payload**; position three is always the **event string**. Python and Node fire `(connection, event, data)`. Same feature, same JSON on the wire, different argument order. Name the parameters in the PHP order or your payload lands in `$event` and nothing works.

This backend pairs with the frontend tina4-js `rtc` module, whose `rtcConfig()` helper fetches the config endpoint so the client and server never drift on paths. Do the backend here. Do the browser side in tina4-js, or with the plain browser APIs shown in section 10.

---

3. Mounting the Module: `Realtime::mount()`

Call `mount()` once in your app bootstrap, **before the server starts**: in `index.php` after `new \Tina4\App()`, or in a `src/` bootstrap file. It registers the routes and returns the resolved path map.

```
<?php
use Tina4\Realtime\Realtime;

Realtime::mount();                                                    // calls only (default
Realtime::mount('', ['features' => ['calls', 'chat']]);               // add persistent chat
Realtime::mount('/api/collab', ['features' => ['calls','chat','files']]); // relocate the whole
```

The full signature:

```
\Tina4\Realtime\Realtime::mount(string $prefix = '', array $options = []): array
```

Key	Meaning
`$prefix`	Mounts the whole surface under `/<prefix>` (default: root). Normalised with `trim($prefix, '/')`, so `'/api/collab'`, `'api/collab'`, and `'api/collab/'` resolve identically.
`features`	`string[]`; any of `"calls"`, `"chat"`, `"files"`. Default `["calls"]`.
`authorize`	Membership guard `callable(string $identity, int $channelId): bool` used by `chat` and `files`. Defaults to a `ChannelMember` lookup. `$identity` is the string user id from the JWT.
`storage`	A `StorageBackend` for the `files` feature. Defaults to the env-selected store (`local`).

`media`	A media-plane backend. Defaults to the env-selected backend (mesh in Phase 1).

The call returns the resolved path map, the same map the config endpoint serves, so you can log it or assert against it:

```
Realtime::mount();
// ['backend' => 'mesh', 'config' => '/api/rtc/config', 'signalling' => '/ws/rtc']

Realtime::mount('', ['features' => ['calls', 'chat']]);
// ['backend' => 'mesh', 'config' => '/api/rtc/config', 'signalling' => '/ws/rtc',
//  'chat' => '/ws/chat', 'messages' => '/api/channels']

Realtime::mount('', ['features' => ['files']]);
// ['backend' => 'mesh', 'config' => '/api/rtc/config', 'files' => '/api/files']
```

`config` is added by any enabled feature, so even a chat-only or files-only mount still exposes `/api/rtc/config`. The config endpoint appends the template tokens (`/{room}`, `/{channel}`, `/{id}/messages`) to these base paths.

What each feature wires

Feature	Route registered	Auth
any	`GET {p}/api/rtc/config`	public - `->noAuth()`
`calls`	`WS {p}/ws/rtc/{room}`	public (unauthenticated)
`chat`	`WS {p}/ws/chat/{channel}`	secured - `Router::websocket(..., secure: true)`; valid JWT on upgrade
`chat`	`GET {p}/api/channels/{id}/messages`	secured - `->secure()`
`files`	`POST {p}/api/files`	auth-required (write route, no `->noAuth()`)
`files`	`GET {p}/api/files/{key}`	secured - `->secure()`

If `chat` or `files` is enabled, `ensureChatTables()` runs at mount time. A missing database logs an error but does not crash boot (section 11).

---

4. The Config Bootstrap: `GET /api/rtc/config`

The client never hardcodes a URL. It fetches this one public endpoint, and everything else comes back in the response: the signalling path, the ICE servers, the chat, messages, and files paths. Move `$prefix` on the server and the client follows automatically. It is registered with `->noAuth()`, public on purpose, because the client needs it before it can authenticate a call.

The body is feature-gated. Only keys for enabled features appear.

```
{
  "backend": "mesh",
  "iceServers": [ /* iceServers() output */ ],  // calls
  "signalling": "/ws/rtc/{room}",               // calls
  "chat": "/ws/chat/{channel}",                 // chat
  "messages": "/api/channels/{id}/messages",    // chat
  "files": "/api/files"                          // files
}
```

The `{room}`, `{channel}`, and `{id}` are literal template tokens. The client substitutes the real room name, channel id, or message id before connecting.

```
const cfg = await fetch("/api/rtc/config").then(r => r.json());

// Join a call room "standup":
const signalUrl = cfg.signalling.replace("{room}", "standup"); // /ws/rtc/standup

// Open chat channel 42:
const chatUrl = cfg.chat.replace("{channel}", "42");           // /ws/chat/42
```

This endpoint is public, and it returns your ICE and TURN configuration including freshly-minted ephemeral TURN credentials (section 5). That is intentional. The browser needs those before it can authenticate anywhere, which is exactly why the credentials are short-lived by design. Do not put secrets in it.

---

5. ICE and TURN Servers

Before two browsers connect, each gathers candidate addresses using **ICE**. A **STUN** server tells a browser its public address. A **TURN** server relays media when a direct path is impossible: strict NAT, symmetric firewalls. The realtime module builds this list from environment variables via `Realtime::iceServers()`.

```
\Tina4\Realtime\Realtime::iceServers(): array
```

A public static. It **always** includes a STUN entry. It adds a TURN entry only when both `TINA4_RTC_TURN_URL` and `TINA4_RTC_TURN_SECRET` are set, using coturn's `use-auth-secret` scheme with time-limited credentials:

```
$username   = (string)(time() + $ttl);
$credential = base64_encode(hash_hmac('sha1', $username, $secret, true));

// No TURN env - STUN only:
[['urls' => ['stun:stun.l.google.com:19302']]]

// TINA4_RTC_TURN_URL + TINA4_RTC_TURN_SECRET set:
```

```
[
  ['urls' => ['stun:stun.l.google.com:19302']],
  ['urls' => ['turn:turn.example.com:3478'], 'username' => '1783546725', 'credential' => 'ie7Mm.
]
```

STUN gets most peers connected; TURN is the relay-of-last-resort for peers behind symmetric NATs that STUN cannot punch through. Because TURN credentials are minted fresh on every `/api/rtc/config` call and expire after `TINA4_RTC_TURN_TTL` seconds, you never ship a long-lived TURN secret to the browser.

Environment variables

```
# .env
TINA4_RTC_BACKEND=mesh                          # only 'mesh' ships in Phase 1
TINA4_RTC_STUN_URLS=stun:stun.l.google.com:19302
TINA4_RTC_TURN_URL=turn:turn.example.com:3478   # enables TURN when paired with the secret
TINA4_RTC_TURN_SECRET=your-coturn-shared-secret
TINA4_RTC_TURN_TTL=3600                          # ephemeral credential lifetime (seconds)
```

Var	Default	Effect
`TINA4_RTC_BACKEND`	`mesh`	Media backend name. Only `mesh` ships in Phase 1. The reported `backend` is hardcoded to `mesh` regardless of this value.
`TINA4_RTC_STUN_URLS`	`stun:stun.l.google.com:19302`	Comma-separated STUN URLs.
`TINA4_RTC_TURN_URL`	-	Comma-separated TURN URLs; enables TURN when set together with the secret.
`TINA4_RTC_TURN_SECRET`	-	coturn `use-auth-secret` shared secret (drives the ephemeral credentials).
`TINA4_RTC_TURN_TTL`	`3600`	Ephemeral TURN credential lifetime, in seconds.

The media backend is a strategy object. Mesh is the default, zero-dependency backend: browsers connect peer to peer in a mesh. An explicit `media` option wins; otherwise the module reads `TINA4_RTC_BACKEND`. An SFU or LiveKit backend that returns a real join token is the documented Phase-2 drop-in, and because signalling paths are unchanged, the client keeps working.

---

6. Signalling: The Mesh Relay

The signalling handler is registered at `WS {p}/ws/rtc/{room}` and is public. There is no `secure:` flag, so anyone can join any room. It follows the PHP WebSocket handler convention:

```
Router::websocket($paths['signalling'] . '/{room}', function ($connection, $data, $event) {
    // $connection : the WebSocketConnection
    // $data       : the payload - a string on "message", null on "open"/"close"
    // $event      : "open" | "message" | "close"
});
```

Its behaviour is deliberately minimal, a raw relay:

- Reads `$room = $connection->params['room'] ?? '';`. An empty room is a no-op.
- On `"open"` it calls `$connection->joinRoom("rtc:{$room}")`.
- On `"message"` it calls `$connection->broadcastToRoom("rtc:{$room}", (string)$data, true)`, forwarding the raw payload to the other peers in the room, sender excluded, untouched.

Tina4 never parses the SDP. Peers put a `to` field in their own frames and filter for themselves. Rooms are namespaced `rtc:<room>` so signalling rooms never collide with chat channels, which use `chat:<channel>`, on the shared WebSocket manager.

The connection surface

Every realtime handler works through these `WebSocketConnection` members, all **camelCase** in PHP:

Member	Purpose
`$connection->params`	route params, e.g. `['room' => '...']` / `['channel' => '...']`
`$connection->auth`	the verified JWT payload on a secured socket (`null` on a public one)
`$connection->joinRoom($name)`	add this connection to a broadcast room
`$connection->broadcastToRoom($name, $message, $excludeSelf = true)`	send a string to a room
`$connection->sendJson($data)`	JSON-encode and send to this one connection
`$connection->getRoomConnections($name)`	the live connections in a room
`$connection->close()`	close this connection

A minimal browser peer, driven entirely by the config response:

```
const cfg  = await fetch("/api/rtc/config").then(r => r.json());
const room = "standup";
const ws   = new WebSocket((location.origin.replace(/^http/, "ws")) +
                           cfg.signalling.replace("{room}", room));

const pc = new RTCPeerConnection({ iceServers: cfg.iceServers });

// Relay ICE candidates to the other peers.
pc.onicecandidate = (e) => {
```

```
  if (e.candidate) ws.send(JSON.stringify({ type: "ice", candidate: e.candidate }));
};

ws.onmessage = async (evt) => {
  const msg = JSON.parse(evt.data);        // raw frame relayed by the server
  if (msg.type === "offer") {
    await pc.setRemoteDescription(msg.sdp);
    const answer = await pc.createAnswer();
    await pc.setLocalDescription(answer);
    ws.send(JSON.stringify({ type: "answer", sdp: answer }));
  } else if (msg.type === "answer") {
    await pc.setRemoteDescription(msg.sdp);
  } else if (msg.type === "ice") {
    await pc.addIceCandidate(msg.candidate);
  }
};
```

Because signalling is public, the room name is your only barrier. If a call must be private, gate access at your app layer: issue unguessable room ids, or check membership before you hand the client a room name.

For a private call, do not mount the built-in `calls` feature and assume its `authorize` option protects signalling: that option applies only to chat and files. Register an application-owned signalling route with `Router::websocket($path, $handler, secure: true)`, then verify the authenticated identity in `$connection->auth` belongs to the requested room on open and on every relayed frame. Browser clients carry the JWT with `new WebSocket(url, ["bearer", token])`.

The current `tina4-js` `rtc.call()` helper opens the built-in public signalling socket and has no signalling-token option. Use a small application WebSocket client for the protected route (or add token support to that client API in a separately versioned change). Unguessable room names reduce discovery; they are not authorization.

---

7. Chat: Secured Channels, Presence, History

Chat is the opposite of signalling: secured. The handler `Realtime::chatHandler($connection, $data, $event)` is registered with `Router::websocket(..., secure: true)`, so a valid JWT is required on the WebSocket upgrade. The router rejects an unauthenticated upgrade with a `401` before your handler runs.

A browser cannot set request headers on `new WebSocket()`, so it passes the token as the `bearer` **subprotocol** (or a `?token=` query param):

```
const token = localStorage.getItem("jwt");
const url   = (location.origin.replace(/^http/, "ws")) + cfg.chat.replace("{channel}", "42");

const ws = new WebSocket(url, ["bearer", token]);   // browser: bearer subprotocol
// or: new WebSocket(`${url}?token=${token}`)        // query-param fallback
```

Two rules shape the handler:

- **Channels are addressed by integer id.** If `{channel}` is not all digits, `chatHandler` returns silently (`ctype_digit()`) - the socket opens and does nothing, with no error frame.

- **Identity comes only from the verified token** (`$connection->auth`), never from the message payload. The room key is `chat:<channelId>`.

Every frame is JSON; every broadcast is a `json_encode(...)` string. The event flow:

Event / message	Server behaviour
`open`	Authorize. Fail: `sendJson(['type'=>'error','error'=>'not a member of this channel'])` then `close()`. OK: `joinRoom`, send the caller the roster `{"type":"presence","event":"roster","users":[...]}`, then broadcast `{"type":"presence","event":"join","user_id":<id>}` (exclude self).
`close`	Broadcast `{"type":"presence","event":"leave","user_id":<id>}` (exclude self).
`typing`	Broadcast `{"type":"typing","user_id":<id>}` (exclude self).
`read`	Advance the member's read cursor (`last_read_at = now`), broadcast `{"type":"read","user_id":<id>,"at":<iso>}` (exclude self).
`message`	Trim `body`; empty is ignored. Persist a `Message`, then broadcast `{"type":"message","message":<saved>}` to everyone including the sender, so an optimistic client reconciles with the server `id` and `created_at`.

`type` defaults to `"message"` when absent. Unknown types are ignored. Authorization is re-checked on every inbound frame, not just on join. Membership can be revoked mid-session, and the server never trusts an identity carried in the payload.

The `users` roster is the sorted set of distinct identities currently in the room. It is deliberately built as a list, not array keys: PHP would coerce numeric-string keys to ints and send `[1,2]` instead of `["1","2"]`, breaking the client's string comparison.

The saved-message JSON shape, also what history returns:

```
{ "id": 128, "channel_id": 42, "user_id": "17", "body": "ship it",
  "thread_id": null, "created_at": "2026-07-08T09:14:22Z" }
```

`thread_id` is `null` for a top-level message, or the parent message id for a threaded reply.

Chat history: `GET {p}/api/channels/{id}/messages`

The catch-up-on-reconnect endpoint, secured with `->secure()`. Load history over HTTP, then keep up over the socket. Identity comes from `$request->user`, the router-attached, already-verified JWT payload. Not a member returns `403`. Query params: `before` (return messages with `id < before`) and `limit` (default 50, clamped to 1..200). Messages come back newest-first (`ORDER BY id DESC`), the standard infinite-scroll-backwards shape.

```
curl -H "Authorization: Bearer $JWT" \
  "http://localhost:7145/api/channels/42/messages?limit=50"

# Older page, walking backwards from the oldest id you already have:
curl -H "Authorization: Bearer $JWT" \
  "http://localhost:7145/api/channels/42/messages?before=79&limit=50"
```

---

8. Files: Upload and Download

Enable with `features=['files']`. Both routes go through a pluggable `StorageBackend`: the `storage` option, or the env-selected store (default `LocalStorage`). The backend resolves once at mount via `Storage::select()`.

`POST {p}/api/files` **- upload (auth-required).** Multipart: a file field named `file` (`$request->files['file']`) plus a form field `channel_id` (required integer, read from body, query, or params). Missing or invalid `channel_id` returns `400`; not a channel member returns `403`; no file returns `400`. It stores the blob under an opaque, collision-free `storage_key` (`Storage::key()`: 16 random bytes as hex plus a sanitized extension, never a user-controlled path), inserts an `Attachment` row (metadata only), and responds `201`:

```
{ "id": 9, "key": "3f9c...b1.png", "filename": "diagram.png",
  "mime": "image/png", "size": 20481, "url": "<direct url OR {files}/{key}>" }
```

`url` is `$store->url($key)` when the backend exposes a direct URL (for example an S3 presigned GET), otherwise the app download route `{files}/{key}`. This route relies on the framework's default Bearer protection: it has no `->noAuth()`, so it is auth-required like any write route.

```
curl -H "Authorization: Bearer $JWT" \
  -F "channel_id=42" \
  -F "file=@diagram.png" \
  http://localhost:7145/api/files
```

`GET {p}/api/files/{key}` **- download (secured with `->secure()`).** Looks up the `Attachment` by `storage_key`; missing returns `404`. Authorizes against the attachment's `channelId`; a non-member returns `403`. If the backend has a direct URL it returns a `302` redirect; otherwise it streams the bytes (`200`) with `Content-Disposition: inline; filename="..."` and `Content-Type = $attachment->mime` (default `application/octet-stream`).

Storage backends

`Storage::select(?StorageBackend $storage = null)` resolves from the `storage` option or `TINA4_STORAGE_BACKEND` (`local` default, or `s3`). `S3Storage` requires the AWS SDK (`aws/aws-sdk-php`); if it cannot be built (SDK missing, or `TINA4_STORAGE_BUCKET` unset) it falls back to `LocalStorage` with a logged warning: a real store, never a silent no-op.

The `StorageBackend` interface:

```
interface StorageBackend
{
    public function put(string $key, string $data, string $mime): void;
    public function get(string $key): ?string;
    public function url(string $key, int $ttl = 3600): ?string;
    public function delete(string $key): void;
    public function exists(string $key): bool;
}
```

```
# .env - local (default)
TINA4_STORAGE_BACKEND=local
TINA4_STORAGE_DIR=data/rt_storage

# .env - S3-compatible (MinIO, AWS, ...)
TINA4_STORAGE_BACKEND=s3
TINA4_STORAGE_URL=https://s3.example.com
TINA4_STORAGE_KEY=...
TINA4_STORAGE_SECRET=...
TINA4_STORAGE_BUCKET=my-bucket
TINA4_STORAGE_REGION=us-east-1
```

`LocalStorage` resolves every key inside its root and rejects path traversal (keys containing `/`, `\`, `..`, or NUL); its `url()` returns `null`, so files serve through the permissioned download route. `S3Storage` returns a presigned GET URL from `url()`, so clients fetch large blobs straight from object storage and downloads become a `302` redirect.

---

9. Auth, Identity, and the Data Model

Chat and files are membership-gated. Two pieces make that work: how an identity is extracted, and how membership is checked.

Extracting identity. A stable string user id is pulled from a verified JWT payload, trying the claims `user_id`, then `sub`, then `id`, and returning `null` if none are present. Identities round-trip as strings, so an integer id, a UUID, or an email all work. WebSocket identity comes from `$connection->auth`, the verified payload the router attached on the secured upgrade. HTTP identity comes from `$request->user`, the router-validated payload on the secured route. That is the single source of truth in PHP; do not re-parse the `Authorization` header yourself.

Checking membership. The secure default: the caller must be a member of the channel.

```
(new \Tina4\Realtime\ChannelMember())
    ->count('channel_id = ? AND user_id = ?', [$channelId, $identity]) > 0;
```

Any exception logs and denies. Override it with the `authorize` option, a `callable(string $identity, int $channelId): bool` guard. The internal wrapper denies first whenever

`$identity` is null, so an unauthenticated caller is always rejected before your guard runs, and a custom guard never has to handle a null identity. Because it runs on every inbound chat frame, keep a custom guard cheap.

```
Realtime::mount('', [
    'features'  => ['calls', 'chat'],
    'authorize' => function (string $identity, int $channelId): bool {
        // Any authenticated user may read/write public channels;
        // fall back to the default membership check otherwise.
        $ch = (new \Tina4\Realtime\Channel())->where('id = ?', [$channelId], 1);
        if (!empty($ch) && $ch[0]->kind === 'public') {
            return true;
        }
        return (new \Tina4\Realtime\ChannelMember())
            ->count('channel_id = ? AND user_id = ?', [$channelId, $identity]) > 0;
    },
]);
```

Data model

The chat surface persists to framework-owned ORM models, all carrying the `tina4_rt_` table prefix so they never collide with your app's tables. Properties are camelCase (Tina4 PHP ORM convention); the ORM maps them to snake*case columns and to snake*case JSON keys, so the schema and wire shape stay byte-identical across every language. `ensureChatTables()` creates them in dependency order.

Model	Table	Key fields (camelCase to wire snake_case)
`Workspace`	`tina4_rt_workspaces`	`id`, `name`, `createdAt`
`Channel`	`tina4_rt_channels`	`id`, `workspaceId`, `name`, `kind` (`public`/`private`/`dm`, default `public`), `createdAt`
`ChannelMember`	`tina4_rt_channel_members`	`id`, `channelId`, `userId` (string), `role` (default `member`), `lastReadAt` (read cursor)
`Message`	`tina4_rt_messages`	`id`, `channelId`, `userId` (string), `body`, `threadId` (nullable parent id), `createdAt`, `editedAt` (nullable)
`Attachment`	`tina4_rt_attachments`	`id`, `channelId`, `messageId` (nullable), `storageKey`, `filename`, `mime`, `size`, `thumbKey` (nullable)

`userId` is a string field everywhere so any JWT identity shape fits (an int id, a UUID, or an email). Because these are ordinary `\Tina4\ORM` models, you seed a channel and its members exactly as in the ORM chapter:

```
use Tina4\Realtime\Channel;
use Tina4\Realtime\ChannelMember;

$ch = new Channel();
$ch->name = "general";
$ch->kind = "public";
$ch->save();

$member = new ChannelMember();
$member->channelId = $ch->id;
$member->userId    = "17";     // string identity from the JWT
$member->role      = "member";
$member->save();
```

Now user `17` can open `WS /ws/chat/{$ch->id}` and pass the history and file checks.

---

10. A Complete Example

A minimal collaboration backend: calls, chat, and files on one surface, with a public channel policy, plus a browser client that hardcodes nothing.

Backend - `index.php`

Mount the surface after `new \Tina4\App()` and before `$app->run()`. A bound database must exist before mounting chat or files (section 11).

```
<?php
require_once "vendor/autoload.php";

use Tina4\Realtime\Realtime;
use Tina4\Realtime\Channel;
use Tina4\Realtime\ChannelMember;

// A bound database MUST exist before mounting chat/files - see the footguns.
// Tina4 reads TINA4_DATABASE_URL (or your own DB init) here.
$app = new \Tina4\App();

$paths = Realtime::mount('', [
    'features'  => ['calls', 'chat', 'files'],
    'authorize' => function (string $identity, int $channelId): bool {
        $ch = (new Channel())->where('id = ?', [$channelId], 1);
        if (!empty($ch) && $ch[0]->kind === 'public') {
            return true; // any authenticated user may join a public channel
        }
        return (new ChannelMember())
            ->count('channel_id = ? AND user_id = ?', [$channelId, $identity]) > 0;
    },
]);
error_log("realtime mounted: " . json_encode($paths));

// One-time seed so a channel + member exist.
if (empty((new Channel())->where('name = ?', ['general'], 1))) {
```

```
    $ch = new Channel();
    $ch->name = "general";
    $ch->kind = "public";
    $ch->save();

    $member = new ChannelMember();
    $member->channelId = $ch->id;
    $member->userId    = "17";
    $member->role      = "owner";
    $member->save();
}

$app->run();
```

Boot it with a STUN/TURN and storage config:

```
export TINA4_DATABASE_URL="sqlite:./data/app.db"

# Calls
export TINA4_RTC_STUN_URLS="stun:stun.l.google.com:19302"
export TINA4_RTC_TURN_URL="turn:turn.example.com:3478"
export TINA4_RTC_TURN_SECRET="a-long-random-coturn-secret"
export TINA4_RTC_TURN_TTL="3600"

# Files (local by default; switch to s3 with the TINA4_STORAGE_* vars)
export TINA4_STORAGE_BACKEND="local"
export TINA4_STORAGE_DIR="./data/rt_storage"

tina4 serve
```

```
  Tina4 PHP v3.0.0
  HTTP server running at http://0.0.0.0:7145
  WebSocket server running at ws://0.0.0.0:7145
```

Browser - bootstrap from the config endpoint

The client fetches `/api/rtc/config` first and never hardcodes a path.

```
const wsBase = location.origin.replace(/^http/, "ws");
const cfg    = await fetch("/api/rtc/config").then(r => r.json());
const token  = localStorage.getItem("jwt");

// -- Call signalling (public, no token) --
const signal = new WebSocket(wsBase + cfg.signalling.replace("{room}", "standup"));
const pc     = new RTCPeerConnection({ iceServers: cfg.iceServers });

pc.onicecandidate = (e) => {
  if (e.candidate) signal.send(JSON.stringify({ type: "ice", candidate: e.candidate }));
};
pc.ontrack = (e) => { document.getElementById("remote").srcObject = e.streams[0]; };

signal.onmessage = async (evt) => {
  const msg = JSON.parse(evt.data);        // raw frame relayed by the server
  if (msg.type === "offer") {
    await pc.setRemoteDescription(msg.sdp);
    const answer = await pc.createAnswer();
    await pc.setLocalDescription(answer);
    signal.send(JSON.stringify({ type: "answer", sdp: answer }));
  } else if (msg.type === "answer") {
    await pc.setRemoteDescription(msg.sdp);
```

```
  } else if (msg.type === "ice") {
    await pc.addIceCandidate(msg.candidate);
  }
};

// The caller adds their camera/mic and sends an offer.
async function startCall() {
  const media = await navigator.mediaDevices.getUserMedia({ video: true, audio: true });
  document.getElementById("local").srcObject = media;
  media.getTracks().forEach((t) => pc.addTrack(t, media));
  const offer = await pc.createOffer();
  await pc.setLocalDescription(offer);
  signal.send(JSON.stringify({ type: "offer", sdp: offer }));
}

// -- Chat (secured: bearer subprotocol) --
const chat = new WebSocket(wsBase + cfg.chat.replace("{channel}", "42"), ["bearer", token]);

chat.onmessage = (evt) => {
  const m = JSON.parse(evt.data);
  if (m.type === "presence" && m.event === "roster") console.log("in room:", m.users);
  if (m.type === "message")                          console.log("msg:", m.message.body);
  if (m.type === "error")                            console.warn(m.error); // e.g. not a member
};
chat.onopen = () => chat.send(JSON.stringify({ type: "message", body: "hello team" }));

// -- History (secured HTTP) --
const history = await fetch(cfg.messages.replace("{id}", "42") + "?limit=50",
  { headers: { Authorization: `Bearer ${token}` } }).then(r => r.json());

// -- File upload (auth-required) --
async function upload(file) {
  const fd = new FormData();
  fd.append("channel_id", "42");
  fd.append("file", file);
  return fetch(cfg.files, { method: "POST", headers: { Authorization: `Bearer ${token}` }, body:
    .then(r => r.json()); // { id, key, filename, mime, size, url }
}
```

The signalling socket is public: anyone with the room name joins. The chat socket and the history and file endpoints require a valid JWT and channel membership. Two browser tabs, both members of channel `42`, joined to room `standup`, will see each other's video peer to peer and each other's messages through the relay.

---

11. Footguns and Hard Rules

The WebSocket handler signature is `($connection, $data, $event)`. `$event` is `"open"`, `"message"`, or `"close"`; `$data` is a string on `"message"` and `null` otherwise. This is the PHP order: position two is the payload, position three is the event. Python and Node use `(connection, event, data)`. Get it wrong and your payload lands in `$event`.

Chat needs a bound database, but a missing one does not crash boot. With `features=['chat']` or `['files']`, `ensureChatTables()` runs at mount. If no database is bound it logs an ERROR and continues: `mount()` still returns the full path map and registers

every route, and the failure only resurfaces at query time. Bind a database before mounting realtime with chat or files (`TINA4_DATABASE_URL` or your own DB init), or chat, history, and files will error per-request while the app looks healthy.

The signalling socket (`/ws/rtc/{room}`) is public. It is not `secure:`, so anyone can join any room and receive the relayed frames. Only the chat socket is JWT-secured. If a call must be private, gate the room name at your app layer.

The config endpoint (`/api/rtc/config`) is public. It returns your ICE and TURN configuration, including freshly-minted ephemeral TURN credentials. This is by design, which is exactly why the TURN credentials are time-limited (`TINA4_RTC_TURN_TTL`). Do not put secrets in it.

Channels are addressed by integer id. A non-integer `{channel}` makes `chatHandler` return silently (`ctype_digit()`), no error frame. The client sees a socket that opens and does nothing. Pass the numeric channel id, never its name.

Authorization is re-checked on every chat frame. Membership can be revoked mid-session, and identity is always taken from the verified token (`$connection->auth` / `$request->user`), never from the payload. A custom `authorize` must be cheap.

Empty messages are dropped. A `message` with an empty or whitespace-only `body` is not persisted and not broadcast. `read`, `typing`, and unknown types never persist anything.

`backend` **is hardcoded to** `'mesh'`**.** In the path map and config body regardless of `TINA4_RTC_BACKEND`, a Phase-1 shortcut. Only mesh ships in Phase 1 (browsers connect peer to peer). An SFU/LiveKit backend is the documented Phase-2 drop-in with no route changes.

File upload (`POST /api/files`) has no `->noAuth()`. It relies on the framework's default Bearer protection, so it is auth-required like any write route. Do not add `->noAuth()` to it.

Where to Go Next

- **The WebSocket chapter** - the raw `Router::websocket()` primitive these handlers are built on, and the connection API (`joinRoom`, `broadcastToRoom`, `sendJson`).
- **The ORM and Authentication chapters** - you seed workspaces, channels, and members with the ORM, and the JWT that secures chat is the same one from the auth chapter.
- **The tina4-js `rtc` module** - the browser side, whose `rtcConfig()` helper wraps `GET /api/rtc/config` so the client and server never drift.

You started this chapter wanting a call button. You end it with calls, chat, presence, read receipts, history, and file transfer, mounted in one line, carrying no media, running no server you did not already have. The hard part of real-time was never the video. It was finding the other browser, and Tina4 relays that in a few lines of text.

AI Client

One class. Three methods. No provider SDK. The Tina4 AI client sends chat, completion, embedding, and streaming requests through one provider-neutral API.

This chapter covers the application-facing `Tina4\AI` class. The `tina4 ai` command serves a different purpose: it installs Tina4 skills and context for coding assistants.

Configure a provider

The client supports `local`, `openai`, and `anthropic`. Local mode targets an OpenAI-compatible endpoint and needs no API key.

```
TINA4_AI_PROVIDER=local
TINA4_AI_URL=http://localhost:11437
TINA4_AI_MODEL=llama3.2
TINA4_AI_TIMEOUT=60
TINA4_AI_CONNECT_TIMEOUT=10
TINA4_AI_MAX_RETRIES=2
```

Use the hosted OpenAI service by changing three values:

```
TINA4_AI_PROVIDER=openai
TINA4_AI_URL=https://api.openai.com/v1
TINA4_AI_MODEL=gpt-4o-mini
TINA4_AI_KEY=your-api-key
```

Anthropic uses the same keys:

```
TINA4_AI_PROVIDER=anthropic
TINA4_AI_URL=https://api.anthropic.com/v1
TINA4_AI_MODEL=claude-3-5-haiku-latest
TINA4_AI_KEY=your-api-key
```

An explicit method argument wins over the environment. The environment wins over the built-in default. This lets one request use another model without changing the rest of the process.

Complete one prompt

`AI::complete` sends one user message and returns the response text.

```
<?php

use Tina4\AI;

$answer = AI::complete("Explain why this query needs an index");
echo $answer;
```

Pass named arguments when one call needs a different model or provider:

```
$answer = AI::complete(
    "Summarize this incident report",
    provider: "openai",
    model: "gpt-4o-mini",
    temperature: 0.2,
    maxTokens: 300,
```

```
    timeout: 20,
);
```

Hold a chat

`AI::chat` accepts a non-empty list of messages. Each message needs a `system`, `user`, or `assistant` role and string content.

```
use Tina4\AI;

$response = AI::chat([
    ["role" => "system", "content" => "Answer as a concise database engineer."],
    ["role" => "user", "content" => "When should I use a composite index?"],
]);

echo $response->text;
echo $response->model;
echo $response->usage["total_tokens"];
echo $response->finish_reason;
```

The `ChatResponse` object carries five fields:

Field	Meaning
`text`	Normalized response text
`model`	Model reported by the provider
`usage`	`prompt_tokens`, `completion_tokens`, and `total_tokens`
`finish_reason`	Provider finish reason, or `null`
`raw`	Original provider response

Use `raw` only when you need provider-specific metadata. Keep application logic on the normalized fields so a provider change does not spread through your code.

Stream text

Set `stream: true` to receive ordered text deltas. The generator yields text only and ignores provider metadata events.

```
use Tina4\AI;

$chunks = AI::chat(
    [["role" => "user", "content" => "Write a short release announcement."]],
    stream: true,
);

foreach ($chunks as $chunk) {
    echo $chunk;
    flush();
}
```

Tina4 may retry before the first delta arrives. It never retries after yielding text because that could duplicate content the caller has already displayed.

Create embeddings

`AI::embed` preserves the input shape. One string returns one vector. A list returns one vector per input, in the same order.

```
use Tina4\AI;

$vector = AI::embed("Tina4 keeps application code small");

$vectors = AI::embed([
    "The router finds a matching handler",
    "The ORM maps a row to a model",
]);
```

Set `TINA4_EMBED_URL` when the embedding service uses a different base URL. Anthropic does not expose embeddings through this contract, so `provider: "anthropic"` raises `AIConfigError`.

Handle failures

All client failures inherit from `AIError`:

Error	Meaning
`AIConfigError`	Missing key, invalid provider, bad option, or unsupported capability
`AIHTTPError`	Provider returned a failing HTTP status or the transport failed
`AITimeoutError`	Connection or total request deadline expired
`AIParseError`	A successful response did not match the provider contract

```
use Tina4\AI;
use Tina4\AIError;

try {
    echo AI::complete("Create a migration plan");
} catch (AIError $error) {
    echo "AI request failed: " . $error->getMessage();
}
```

Hosted providers fail before sending when `TINA4_AI_KEY` is missing. Error text never includes the key, prompt, or provider response body. A failure stays a failure; Tina4 never turns it into an empty answer.

Timeouts and retries

`TINA4_AI_CONNECT_TIMEOUT` bounds connection setup. `TINA4_AI_TIMEOUT` bounds the whole request, including retries and response reads. `TINA4_AI_MAX_RETRIES` applies only to connection failures, HTTP 429, and HTTP 5xx responses.

Other HTTP 4xx responses and malformed successful responses run once and fail. The client carries transient trouble for a bounded distance, then hands the error back to your code.

OpenID Connect SSO

Tina4 completes one provider-neutral OpenID Connect flow. Configure an issuer and Tina4 discovers the provider, runs Authorization Code with PKCE, regenerates the Session, and sends the normalized identity through the existing secured route gate. The browser keeps the normal Tina4 Session cookie.

Configure SSO

```
TINA4_SSO_ISSUER=https://identity.example.com/realms/my-app
TINA4_SSO_CLIENT_ID=my-app
TINA4_SSO_CLIENT_SECRET=replace-me
TINA4_SSO_REDIRECT_URI=https://app.example.com/auth/callback
TINA4_SSO_SCOPES=["openid", "profile", "email"]
TINA4_SSO_VERIFY=introspection
TINA4_SSO_POST_LOGOUT_REDIRECT_URI=https://app.example.com/
```

SSO stays off until the issuer, client id, and redirect URI exist. A confidential client also needs its secret. Production issuer and callback URLs require HTTPS. Loopback HTTP remains available for local development.

When configured, Tina4 mounts:

Method	Route	Purpose
`GET`	`/auth/login`	Start login and create one-use state, nonce, and PKCE values.
`GET`	`/auth/callback`	Validate state, exchange the code, and create the identity.
`POST`	`/auth/logout`	Destroy the local Session and request provider logout.

A route collision stops startup with an error.

Use the identity

Secured routes use the same authentication gate as local Tina4 authentication. The normalized identity is available as `$request->user`. Provider tokens never enter that value, logs, error pages, Swagger, or `Session::all()`.

```
use Tina4\Sso;

if (Sso::configured()) {
    $sso = Sso::fromIssuer();
    $metadata = $sso->discover();
    $identity = $sso->identity($request);
}
```

Method	Result

`Sso::configured()`	Reports whether required environment values exist.
`Sso::fromIssuer()`	Creates the client and discovers the issuer.
`discover($force = false)`	Returns validated metadata. Force refreshes it.
`login($request, $returnTo = '/')`	Stores one-use state and returns the authorization URL.
`callback($request, $query = null)`	Validates the callback and regenerates Session.
`identity($request)`	Returns the normalized identity or `null`.
`refresh($request)`	Replaces credentials and identity atomically.
`logout($request, $returnTo = '/')`	Destroys Session and returns the safe logout target.

`returnTo` accepts a local absolute path. Tina4 rejects schemes, hosts, protocol-relative URLs, backslashes, and control characters.

Map claims and providers

```
TINA4_SSO_CLAIM_MAP={"username":"preferred_username","roles":"realm_access.roles","groups":"grou
```

The runtime does not name providers. For Keycloak, use a realm issuer such as `https://sso.example.com/realms/my-realm`. For Microsoft Entra ID, use the tenant-specific v2 issuer. Register the exact Tina4 callback URI in either provider. Use a claim map only when roles or groups use different claim names.

Version 3.13.104 supports introspection without another PHP package and needs a client secret. Selecting `jwks` fails during configuration until the application installs a supported cryptography capability. State, nonce, code, and verifier are one-use. Logout always destroys the local Session first.

Swagger

When configured, Swagger publishes the issuer discovery URL as an `openIdConnect` scheme. Secured routes accept the normal Tina4 Session cookie or the configured bearer scheme.

GIS and PostGIS

Tina4 stores geographic points in PostGIS, measures distance in metres, and returns map-ready GeoJSON. Public coordinates always use longitude first: `[longitude, latitude]`. GPS may supply coordinates, but GIS is the feature.

Define a point field

Enable PostGIS first with `CREATE EXTENSION IF NOT EXISTS postgis;`.

```
use Tina4\ORM;
use Tina4\Point;

class ChargePoint extends ORM
{
    public ?Point $location = null;
    public array $pointFields = [
        'location' => ['srid' => 4326, 'spatialIndex' => true],
    ];
}

$site->location = new Point(18.4241, -33.9249);
$site->save();
```

Tina4 creates `geography(Point,4326)` and an idempotent GiST index. Unsupported database engines and PostgreSQL without PostGIS fail clearly.

`Point::parse()` accepts a Point, a two-number pair, WKT or EWKT, GeoJSON Point or Feature, and WKB or EWKB. It rejects invalid ranges, non-finite numbers, non-point geometry, and conflicting SRIDs before SQL runs. SQL `NULL` remains `null`; `(0, 0)` remains a real point.

Query spatial data

```
$nearby = ChargePoint::query()
    ->withinDistance('location', [18.42, -33.92], 5000)
    ->selectDistance('location', [18.42, -33.92], 'metres')
    ->orderByDistance('location', [18.42, -33.92])
    ->get();
```

Method	Meaning
`withinDistance($column, $point, $metres)`	Keep rows inside a non-negative radius.
`selectDistance($column, $point, $alias)`	Add distance in metres to each row.
`orderByDistance($column, $point, $descending)`	Sort with a stable primary-key tie-break.
`intersects($column, $geometry)`	Match bound WKT, EWKT, or GeoJSON geometry.

`bbox($column, $minLon, $minLat, $maxLon, $maxLat)`	Match a validated bounding box.

Spatial values use bound parameters. Column and alias names are validated as identifiers.

Return GeoJSON

```
$feature = $site->toFeature();
$collection = ORM::featureCollection($nearby);
return $response->json($collection);
```

`toFeature()` moves the selected Point into `geometry` and keeps other selected fields in `properties`. A null point creates `"geometry": null`. `featureCollection()` preserves query order. Supply a geometry field when a model has more than one Point field and an include list to limit properties.

Version 3.13.104 persists Point fields only. It does not persist lines, polygons, rasters, tiles, geocoding results, or routes. `intersects()` may still accept a polygon as bound query geometry. PostGIS is the only storage provider.

IoT and MQTT

Tina4 includes a zero-dependency MQTT 3.1.1 client for telemetry, device state, asset tracking, and EV charging. It connects to an external broker such as Mosquitto, EMQX, HiveMQ, or AWS IoT. Tina4 is not a broker.

Configure and publish

```
TINA4_MQTT_URL=mqtt://127.0.0.1:1883
TINA4_MQTT_CLIENT_ID=warehouse-api
TINA4_MQTT_KEEPALIVE=60
TINA4_MQTT_TLS_VERIFY=true
TINA4_MQTT_CA_FILE=/run/secrets/mqtt-ca.pem
```

Use `mqtt://` or `tcp://` for plain TCP and `mqtts://` for TLS. Put credentials in URL user information or explicit constructor arguments. Broker credentials do not have dedicated environment variables.

```
use Tina4\Mqtt;

$mqtt = new Mqtt();
$packetId = $mqtt->publish(
    'fleet/meter-42/telemetry',
    '{"kwh":12.5}',
    qos: 1
);
$mqtt->disconnect();
```

Construction connects by default. QoS 0 waits for no acknowledgement. QoS 1 returns the packet id after PUBACK. Tina4 refuses QoS 2 instead of silently downgrading it.

Consume safely

```
$mqtt = new Mqtt(cleanSession: false);

foreach ($mqtt->consume('fleet/+/telemetry', qos: 1) as $message) {
    process($message->topic, $message->text());
}
```

`consume()` acknowledges after successful loop handling. Use `receive(ack: false)` followed by `$message->acknowledge()` for manual control. Messages expose topic, byte payload, QoS, packet id, retained and duplicate flags, `acknowledge()`, `text()`, and `toArray()`.

Retained publishes store current device state; an empty retained payload clears it. Configure `willTopic`, `willPayload`, `willQos`, and `willRetain` to report an unclean disconnect. `disconnect()` suppresses the will; `kill()` lets the broker publish it.

Each client builds its own TLS trust store. Verification defaults to true. `tls()`, `cipher()`, and `tlsVersion()` report the negotiated connection. One client has one socket reader, so independent consumers need separate clients.